Flux v9

A Decentralized Cloud for Autonomous Compute

Tadeas Kmenta

Version 1.0.1 · October 25, 2026

A technical description of the Flux network as deployed, and a specification of the architecture it is being rebuilt into. Both halves are written to the same standard: every claim carries a provenance citation, every claim about deployed behaviour carries a source-code citation, and every element of the target architecture that is undecided is marked as such rather than filled with a plausible default.

Claims are tagged for *maturity* — shipped, in progress, planned, vision — and for *provenance* — read from code, from roadmap material, from design intake, or inferred. The two axes are independent and both are always given.

One label marks what this paper could not establish:

Not established by this survey.

Abstract

Flux is a decentralized cloud of 6,489 collateralised nodes running 1,195 deployed applications, on a chain that replaced proof-of-work with collateral-gated sortition at height 2,020,000. This paper does two things. It describes the deployed system — consensus, node tiering and benchmarking, monetary policy, orchestration and placement, addressing, attested execution, and the Zcash-derived shielded layer — at a depth from which the behaviour could be reimplemented or audited, citing source by file and line — with the caveat, stated in full in §29, that a minority of the components cited are private repositories a reader cannot open, and that one class of finding in those is deliberately stated without coordinates. It then specifies the target architecture at the same depth: revenue-funded operator compensation replacing issuance, a redesigned bridge, a content-reporting quorum, application specification v9, a bonded attestation network, post-quantum migration, and the machine-facing interfaces an autonomous tenant needs to provision, pay for and release capacity without a human. It also argues one positioning claim from the substrate rather than from intent: the interconnect requirement that centralises frontier-model inference does not apply to models that fit on a single device, so a dispersed fleet of collateralised commodity machines is a structurally appropriate home for small task-specialised models — once accelerators can be subdivided and offered on it: the collateralised fleet has no accelerator allocation today, and the only one Flux runs, FluxEdge's whole-device rental on separate, centrally managed machines, cannot subdivide a device.

The discipline that keeps the second half honest is that every undecided element is named rather than filled: each mechanism is given as state, transitions, invariants and failure handling, with open parameters marked as decisions and open constructions marked as open. Two axes of tagging let a reader tell, at any sentence, whether they are reading about code that runs or a design that does not.

Several findings are not what the roadmap or the design intent would predict, and the paper reports them as measured. A fee pool with a per-block drip is necessary because deterministic payment rotation otherwise admits a timing attack recapturing most of an operator's own fee; the same pool design pays every node regardless of what it hosts, so it supplies no gradient rewarding good hosting, and what compels performance is the eligibility gate — which the deployed benchmark and attestation path does not bind to a machine. Hardening that gate is urgent independently of revenue-sharing rather than because of it: at activation the gate would already be admitting 95.7 % of operator income as issuance, and the new mechanism would add the other 4.3 %. The distribution of that revenue would be a consensus rule; *which* revenue is subject to it, and at what price, would remain policy, and the paper marks that boundary wherever the decentralization claim rests on it. Collateral-weighted moderation is Sybil-resistant by construction and, at the measured ownership distribution, not censorship-resistant: four owner identities reach the proposed removal threshold, and no weighting or remedy in the design space raises that number. The shielded pools have been closed to new entrants by consensus since 2021 and hold 0.0225 % of supply; they are to be retired rather than reopened, leaving Flux settling transparently and treating payment privacy as an unsolved problem rather than a delivered feature. A header format chosen for consensus reasons forecloses light-client bridging. Of twenty-two mechanisms assessed, six are extensions of what exists, twelve are engineering, and four are open research problems — which the paper names rather than absorbs.

Revision history

1.0 2026-09-07. First publication.

1.0.1 October 25, 2026. Corrections to the 1.0 text: factual errors, contradictions between sections, figures that did not match their sources, statements of planned work that read as shipped, and the description of who operates which service and holds which

keys. No section was added or removed. Every change keeps the 1.0 wording beside the correction in the source, and a marked-up edition shows both.

Contents

1 Introduction 2
1.1 The problem, not the product 2
1.2 Flux today, measured 3
1.3 The three changes 3
1.4 What is new in this paper 5
1.5 What this paper does not claim 5
1.6 How to read this paper 6
1.7 Method 6
2 System model, adversary model, and notation 7
2.1 Participants 7
2.2 Network model 9
2.3 Trust boundaries 10
2.4 The adversary 12
2.5 Assumptions 15
2.6 Out of scope 17
2.7 Notation 17
2.8 How to read the tags 18
3 Flux as deployed today 19
3.1 Methodology 19
3.2 The fleet 20
3.3 Node tenure 21
3.4 Deployed applications 21
3.5 Specification version distribution 22
3.6 Pricing as deployed 22
3.7 The multi-chain footprint 23
3.8 The operational layer 23
3.9 The measured baseline 24
4 Consensus: from PoW to PoN, node tiering, benchmarking, and eligibility 26
4.1 The network-upgrade mechanism 26
4.2 What changes at PoN activation 27
4.3 PoN sortition, formally 28
4.4 Fork choice 29
4.5 The FluxNode lifecycle 30
4.6 Tiers and collateral 31
4.7 Benchmarking and eligibility 31
4.8 The emergency-block path 32
4.9 What activation actually cost 33
4.10 Measured rotation 33
5 Monetary policy as deployed 34
5.1 The subsidy function 34
5.2 The frozen halving 35
5.3 Tier partition and the dev-fund remainder 35
5.4 One-off and recurring fund payouts 36
5.5 The supply function 36
5.6 Plots 37
5.7 A discrepancy in the published model 38
5.8 Baseline for what follows 38
6 Parallel assets: the current multi-chain footprint and the case for consolidation 38
6.1 The inventory 39
6.2 The trend is the wrong way 39
6.3 How a parallel asset is acquired today 40
6.4 The supply-accounting problem 41

6.5 The case for consolidation . . . 43
6.6 The Base question . . . 44
6.7 What consolidation requires . . . 44
6.8 Sequencing: one coordinated transition . . . 45
7 PNR: revenue-funded operator compensation . . . 47
7.1 Assumptions . . . 48
7.2 The payment, and where the split is applied . . . 49
7.3 Settlement shapes not taken . . . 53
7.4 The pool as consensus state . . . 54
7.5 Fairness over a rotation . . . 57
7.6 Residual unfairness, quantified . . . 58
7.7 The timing attack, and why the drip defeats it . . . 60
7.8 What PNR does and does not incentivise . . . 62
7.9 Invariants . . . 64
7.10 Activation schedule . . . 68
7.11 The crossover, in both denominations . . . 68
7.12 Open parameters . . . 72
7.13 Recommended values for the remaining open parameters . . . 74
7.14 Comparison with prior art . . . 76
8 Chain operations and evolution: main-chain pruning, collateral maturity, and post-quantum migration . . . 76
8.1 Main-chain pruning: an operational decision, not a new capability . . . 77
8.2 Collateral maturity: a proposal, not yet a rule . . . 78
8.3 Post-quantum migration: a scope decision and an open problem . . . 79
8.4 The frontier: what Bitcoin is building, and the one move Flux can make that it cannot . . . 82
8.5 What the three share . . . 84
9 FluxOS: orchestration, placement, and the application lifecycle . . . 87
9.1 Process architecture . . . 87
9.2 The application lifecycle . . . 91
9.3 Placement: opportunistic self-selection with broadcast collision-avoidance . . . 92
9.4 The reconciler: a single, level-based actuator . . . 95
9.5 Soft versus hard redeploy . . . 96
9.6 The P2P layer . . . 96
9.7 Resource accounting . . . 98
9.8 Pricing . . . 99
9.9 Moderation, as it exists today . . . 99
9.10 Trust and failure model . . . 99
10 FluxOS application specification v9 . . . 100
10.1 Version gating as deployed . . . 100
10.2 The migration baseline, measured . . . 101
10.3 What v9 changes . . . 102
10.4 The finding this section exists to deliver . . . 103
10.5 The scale of the work, quantified . . . 103
10.6 The routing layer, precisely . . . 103
10.7 Two artifacts, not one . . . 104
10.8 The encryption-envelope regression . . . 105
10.9 The migration mechanism: forced, not opt-in . . . 105
10.10 The v9 schema, for reference . . . 106
10.11 Open parameters . . . 106
11 One platform: FluxCloud/FluxEdge convergence, one provider runtime, and the datacenter tier . . . 107
11.1 What the two things actually are today . . . 108
11.2 “One provider runtime”, concretely . . . 108
11.3 Capability flags as the architectural argument . . . 109
11.4 The datacenter tier and the field that names it . . . 110
11.5 What convergence costs . . . 111

11.6 Vertical integration, argued both ways . . . 111
11.7 What would make convergence checkable . . . 112
12 Networking and addressing: FDM, PDNS, mesh resolution, and private deployments. . . . 112
12.1 The reachability path, end to end . . . 113
12.2 FDM's run loop . . . 114
12.3 Certificates . . . 115
12.4 `flux-dnsd` mesh resolution: a local file-tail resolver, not a gossip protocol . . . 116
12.5 flux-dns-gateway: an mTLS proxy in front of the PDNS key . . . 116
12.6 flux-geo-cert: a citation trap . . . 116
12.7 fluxos-network-policy: the governance-relevant lever . . . 117
12.8 The centralization inventory . . . 118
12.9 Node network configuration . . . 120
12.10 Private deployments (VPON) . . . 120
13 Managed services: databases, RPC, archive, and indexing . . . 121
13.1 `Flux-Shared-DB`: ad-hoc leader election over the FluxOS location API . . . 122
13.2 `flux-pg-cluster`: Patroni/etcd-backed PostgreSQL HA . . . 123
13.3 The comparison . . . 125
13.4 How a tenant uses these . . . 126
13.5 Backup and restore . . . 126
13.6 RPC, archive, and indexing (PLANNED) . . . 127
13.7 Implication for lifecycle: eviction is where the guarantee is tested . . . 128
14 Attested execution: ArcaneOS, the SAS, and the attestation network . . . 128
14.1 The deployed attestation stack . . . 129
14.2 Disposition of the findings in this section . . . 133
14.3 The bonded attestation network . . . 134
14.4 Figures . . . 137
15 Accelerated compute: GPUs and the AI workload path . . . 139
15.1 GPU allocation as implemented . . . 139
15.2 The contention defect: a resource request that disappears . . . 139
15.3 "Pluggable" or hot-attachable GPU: no code, no exception . . . 140
15.4 Two product surfaces, easy to conflate . . . 140
15.5 A module name that is not a mechanism . . . 141
15.6 The machine-facing surface: scoped tokens, forward to §18 . . . 141
15.7 Metering and the overdraft window, forward to §16 . . . 141
15.8 A gap for agent autonomy: key issuance . . . 142
15.9 Natural-language deployment: `DeploymentYAML`, forward to §19 . . . 142
15.10 Resource accounting on the FluxOS side . . . 142
15.11 The workload class the substrate fits . . . 142
15.12 What can be honestly sold today . . . 144
15.13 Inference without a GPU: what the fleet can serve today . . . 144
15.14 Scientific simulation: the workload class already here . . . 146
16 Paying for capacity: direct settlement, and a service built over it . . . 148
16.1 Part A — Direct settlement . . . 149
16.2 Part B — A service built on Flux, and the credits layer that lives outside it . . . 154
17 Agent identity and delegated authority . . . 167
17.1 What exists today . . . 168
17.2 The two authorities . . . 169
17.3 The delegation certificate . . . 169
17.4 Deployment authority . . . 170
17.5 Revocation . . . 172
17.6 Spending authority, and the honest result . . . 173
17.7 Invariants . . . 174
17.8 Attack analysis . . . 175
17.9 Open parameters . . . 177

17.10 Comparison with prior art . . . 178
18 The agent-native cloud . . . 179
18.1 What such an agent would run . . . 183
19 Natural-language deployment: who holds the signing key . . . 183
19.1 What ships today . . . 183
19.2 Two signing paths, belonging to two systems . . . 184
19.3 Reviewability: the question that remains . . . 186
19.4 Self-hosting . . . 188
19.5 Toward bounded automation . . . 188
19.6 What would make the boundary real . . . 188
20 Shielded value on Flux . . . 188
20.1 Lineage: Bitcoin, Zcash, Zel, Flux . . . 189
20.2 The shielded machinery . . . 190
20.3 The trusted setup, stated exactly . . . 191
20.4 What is switched off . . . 192
20.5 Supply accounting . . . 195
20.6 The RPC surface . . . 195
20.7 Post-quantum scope: what is bounded, and why . . . 196
20.8 What a migration to Orchard/Halo2 would, and would not, buy . . . 197
20.9 Retiring the pools: the decision, and the measurements behind it . . . 197
21 Private payment with public obligation . . . 200
21.1 The verifier, and the requirement pair . . . 201
21.2 Amount hiding is unsatisfiable under PNR . . . 203
21.3 Three shapes, kept apart . . . 203
21.4 The scope result . . . 204
21.5 The blocker: the shielded pool cannot be entered . . . 205
21.6 Why the naive construction fails: CumulusVPN, worked through . . . 207
21.7 The deployed rule, and construction A . . . 208
21.8 Shape B: what a real construction would need . . . 211
21.9 The anonymity set, measured . . . 213
21.10 Cost, and the binding limit on shielded throughput . . . 216
21.11 Invariants . . . 217
21.12 Attack analysis . . . 218
21.13 Open problems, and open parameters . . . 219
21.14 Comparison with prior art . . . 222
22 Bounded, publicly verifiable bridging — replacing Fusion, chain consolidation, and why not atomic swaps . . . 223
22.1 The incumbent, stated factually . . . 225
22.2 Requirements . . . 227
22.3 The construction . . . 229
22.4 The supply invariant . . . 237
22.5 The attestation link . . . 244
22.6 Why not the alternatives . . . 247
22.7 The launch trust assumption: one quorum, bounded by a rate limit . . . 249
22.8 Migration and consolidation . . . 251
22.9 Recommended values for the open parameters . . . 257
22.10 What cannot be achieved . . . 259
22.11 Comparison with real incidents and with designs that held . . . 260
23 Governance: decentralization, operator protection, and the content-reporting quorum . . . 262
23.1 Governance as it operates today . . . 263
23.2 The roadmap principle, and the tension the quorum creates . . . 265
23.3 The content-reporting quorum, specified . . . 267
23.4 Safety and liveness, separated . . . 279
23.5 The measurement . . . 280

23.6 The negative result . . . 283
23.7 Dilution: why the honesty incentive does not bind . . . 284
23.8 Remedies, evaluated . . . 286
23.9 Attacks and failure surface . . . 289
23.10 The decentralization endgame . . . 291
23.11 Recommended values for the open parameters . . . 292
23.12 Comparison with prior art . . . 294
23.13 Open parameters . . . 295

24 End-to-end: how Flux works, today and at target . . . 298
24.1 Pass one: the deployed system . . . 298
24.2 Pass two: the target architecture . . . 303
24.3 What changes, and what does not . . . 310

25 Feasibility and distance to target . . . 316
25.1 Method . . . 316
25.2 The table . . . 316
25.3 The four Research items, which are not equally hard . . . 318
25.4 Where the claim holds, and where it does not . . . 319
25.5 The dependency that reorders the schedule . . . 320

26 Evaluation . . . 320
26.1 Methodology . . . 320
26.2 The fleet . . . 321
26.3 The payment rotation, measured . . . 321
26.4 Operator concentration . . . 322
26.5 Deployed applications . . . 323
26.6 Shielded value pools . . . 324
26.7 Supply, modelled . . . 324
26.8 The PNR crossover, modelled, in both denominations . . . 326
26.9 The drip trade-off . . . 328
26.10 Threats to validity . . . 328

27 Migration and rollout: sequencing, dependencies, and risk . . . 329
27.1 The dependency graph . . . 329
27.2 The edge that shapes the schedule: attestation alongside revenue . . . 331
27.3 The second critical path: SPEC v9 . . . 332
27.4 The privacy prerequisite . . . 333
27.5 The activation schedule . . . 333
27.6 An empirical base rate for consensus change . . . 334
27.7 Risk register . . . 334
27.8 What would have to be true . . . 336

28 Related work . . . 337
28.1 Decentralized compute markets . . . 337
28.2 Hyperscalers . . . 338
28.3 Storage and content networks . . . 338
28.4 Consensus . . . 339
28.5 Bridging . . . 339
28.6 Attestation and bonded assertions . . . 340
28.7 Privacy . . . 340
28.8 Delegated authority . . . 341
28.9 Agent interfaces . . . 342
28.10 What this paper adds . . . 342

29 Limitations, non-goals, and open problems . . . 342
29.1 Stated non-goals . . . 342
29.2 Limitations of this paper . . . 343
29.3 Properties that do not hold . . . 345
29.4 Open problems . . . 346
29.5 Open decisions . . . 347

29.6 What the next revision must do . . . 348
30 Conclusion. . . . 349
Acknowledgements . . . 350
A Complete Constant Tables . . . 355
A.1 Chain identity and network . . . 355
A.2 Network upgrades . . . 355
A.3 Emission . . . 357
A.4 Consensus-level addresses and funds . . . 358
A.5 Node tiering and lifecycle . . . 360
A.6 Block and consensus limits . . . 361
A.7 Difficulty . . . 362
A.8 Benchmarking . . . 363
A.9 FluxOS . . . 366
A.10 Application pricing and deployment . . . 374
A.11 Networking . . . 376
A.12 Attestation and boot . . . 378
A.13 Measured network state . . . 380
B Interface specifications . . . 383
B.1 The application specification . . . 383
B.2 The Provider Runtime Contract v0 . . . 389
B.3 The agent-facing provisioning API . . . 391
B.4 The FluxOS HTTP surface . . . 393
B.5 The `fluxbench` RPC surface and the `fluxsas` protocol . . . 394
C Emission model: derivation, discrepancy, and reproduction . . . 396
C.1 Provenance . . . 396
C.2 Arithmetic conventions, which are load-bearing . . . 396
C.3 The two modes, and the discrepancy . . . 396
C.4 The functions . . . 396
C.5 Reproduction . . . 397
C.6 What the model does not do . . . 397
D Notation index . . . 398
E Roadmap timeline and dependency graph . . . 402
E.1 Committed items . . . 402
E.2 Assessment notes . . . 412
E.3 Dependency graph . . . 413
E.4 Timeline-only items . . . 416
E.5 Terminology . . . 417

PART I

Foundations

1 Introduction

1.1 The problem, not the product

Where this comes from. Flux is not a proposal. The chain has produced blocks continuously since **2018**: mainnet genesis carries UNIX timestamp 1,516,980,000 — 2018-01-26T15:20:00Z — and its coinbase still reads `Zelcash06f40b01...`, the name the project launched under and the rebrand the consensus code has never shed, as `determ_zelnodes`, `zelid` and `ZelBack` attest throughout this paper SHIPPED [fluxd/src/chainparams.cpp:65,185]. The tiered node network was announced in October 2018 [48]; its orchestration layer — FluxOS, then ZelFlux — debuted in the autumn of **2019** [49]. Eight years of continuous operation is the reason this paper can describe half its subject in the past tense at all, and the reason its claims about deployed behaviour are checkable against running code rather than argued from a design.

That history has been written about before, and this paper does not restate it. The chain was specified in the **ZelCash whitepaper v2** of July 2018 [50]. Two documents are this paper's direct predecessors, and it supersedes both rather than amending them. The **Proof-of-Useful-Work v2 whitepaper** [24] — still linked from `fluxd`'s own README — specified the consensus mechanism now called Proof of Node, and is the one this document builds on most directly rather than departs from: §4 carries its construction forward as deployed code and measures it, and §6 reconciles its supply model to within one block of accrual (Table 14). Where the two differ, it is because something has since been built, measured or decided, and this document says so at the point of difference. **Flux Whitepaper 2.1** [23] is the ecosystem description it replaces. The author's own contemporaneous descriptions of node tiering [36] and of the FluxOS architecture [35] are prior art for parts of §4 and Part III, and the per-tier benchmark thresholds §4 reads out of `fluxbench` were published as an operator guide [47].

Not established by this survey. the publication dates of the historical Medium articles cited above, with one exception. Medium returns HTTP 403 to scripted retrieval, so only Kmenta [35] carries an independently confirmed timestamp (2020-03-18T03:17:28Z, read from the `datePublished` metadata of an Internet Archive capture of 2020-11-06); the remaining dates are those of the index compiled for this survey. Both articles under the author's own byline do carry confirmed archival fixity — the Wayback availability API reported status-200 captures for each on 2026-09-06 — so the *content* is anchored even where the publication date is not. None of the arguments in this paper depends on those dates in any case: the 2018 and 2019 anchors above rest on the genesis block and on `fluxd`'s source, not on any article.

A cloud provider today sells capacity to people. A human opens an account, attaches a card or signs a contract, and later closes the account when the capacity is no longer needed. Every part of that sentence is an assumption about who the customer is, and the assumption is load-bearing: the account presumes an identity a provider can bill, the card presumes a payment network built around recurring consent from a person, and the contract presumes a party who can be sued, renegotiated with, or simply asked. This paper's premise is that if the dominant consumer of cloud capacity becomes autonomous software rather than people, each of those three assumptions has to be replaced with something a program can satisfy entirely on its own — an identity a piece of code can hold, a payment a piece of code can make without a human tapping a button, and an authority a piece of code can prove it has without a lawyer's signature. That is a bet about demand, stated here as the premise the rest of the paper works from, and not a prediction this paper defends. Flux's own public roadmap states the destination directly: "a cloud that serves software agents as first-class customers... no account, no card, no human in the loop" VISION [roadmap: flux-roadmap-public.md:7]. Whether that demand materializes is outside this paper's scope; whether the architecture that follows from betting on it is coherent, and how much of it is actually built, is the whole of it.

Declining to forecast is not the same as declining to say why the bet is reasonable, and a reader entitled to skip Part IV on the grounds that its premise is unargued deserves the

argument. Three things make it worth taking seriously. *First, the direction is already observable rather than speculative*: software that calls APIs, provisions resources and completes multi-step tasks without a human approving each step is a shipping product category, not a projection, and every such call is one a card-and-account model was not designed to price. *Second, the bet does not require agents to dominate.* It requires only that the *marginal* new customer is increasingly machine-initiated, because the three assumptions being replaced — an account someone opens, a card someone consents to, a contract someone signs — fail at the first such customer, not at the majority. A provider that cannot serve one autonomous buyer cannot serve any. *Third, the failure mode is bounded*: every mechanism Part IV specifies — delegated authority with a budget, payment without a custodian, deployment without an account — is independently useful to a human tenant who wants to bound what a script may spend or deploy without a hosting relationship. If the demand does not arrive, the architecture is over-built for its market rather than wrong about its mechanism.

That is the case, and it is deliberately not stronger than the evidence. Section 26 names what would falsify it, and Section 30 states plainly that if agents are not the customer, much of the target architecture solves a problem the network does not have.

There is a second, narrower observation about *what* such software would run, and it turns out to have architectural consequences rather than only commercial ones. Much of the work an autonomous agent does for an ordinary person — drafting text, triaging mail, summarising a document, keeping a schedule — is served adequately by small models specialised to a task, not by frontier ones. That distinction matters here because the two have different physics. Frontier inference splits a model across many accelerators and exchanges state between them on every token, so it is bound to an interconnect one to two orders of magnitude faster than wide-area networking; that is why it concentrates in single datacentres, and it is a topology constraint rather than a preference. A model that fits on one device exercises no such interconnect, and concurrent requests to it are mutually independent. **For that class of work the structural advantage of centralisation is not reduced — it is absent**, and a dispersed fleet of collateralised commodity machines becomes an appropriate substrate rather than a compromised one. §15.11 argues this properly, including the part that cuts the other way: the only accelerator allocation Flux runs today is FluxEdge's, on separate, centrally managed machines that are not FluxNodes, and it allocates accelerators whole; serving many small models economically requires subdividing them, which is the one thing the current allocation model cannot do.

1.2 Flux today, measured

Before any forward-looking claim, the paper establishes what is running. Flux is not a proposal: on 2026-09-03 the network carried 6,489 collateralised nodes across three tiers — Cumulus, Nimbus and Stratus — backing 88,514,500 FLUX of locked collateral SHIPPED [measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03]; 1,195 applications were deployed across 7,486 requested instances SHIPPED [measured: api.runonflux.io/apps/globalappsspecifications, 2026-09-03]; and since block height 2,020,000 the chain has produced every block through a collateral-gated lottery over the registered node set rather than through Equihash proof-of-work SHIPPED [fluxd/src/chainparams.cpp:153] (§4). These numbers are the yardstick for everything that follows: a paper about restructuring a chain's incentives and execution layer around autonomous demand is a different exercise when the chain in question already has real operators, real collateral and real deployed workloads than when it does not. §3 gives the full measured baseline, with every figure in this section reproducible from the same public API on the same date.

1.3 The three changes

Flux describes itself as restructuring three things at once: how a node operator is paid, what a node operator's hardware can be trusted to have actually run, and how value moves between a

tenant and the network. Each is at a different point between specification and deployment, and this paper states that difference for each rather than presenting all three as equally real.

Emission replaced by demand-funded operator compensation. Every payment a tenant makes for an application today lands, in full, at a single transparent address, and every unit a node operator earns is newly issued rather than routed from that payment SHIPPED [measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]. Progressive Node Rewards (PNR) is the specified replacement: a consensus-enforced fraction of each application payment would be diverted into a protocol-held fee pool and paid out to operators through the existing block-production rotation, one small increment per block, so that operator income begins to track what tenants actually pay rather than a fixed issuance schedule alone. PNR is specified to implementation depth in §7 — the consensus rule, the pool's per-block update, and two fairness proofs — but nothing in it exists in any repository surveyed for this paper PLANNED [intent: design intake]. It is a design, stated precisely enough to build, not a description of running code.

The substrate hardened by attested execution. ArcaneOS pairs a measured Secure Boot chain with disk encryption and a signed, `dm-verity` root filesystem, and a local attestation service (the SAS) that other components can query for a signed claim about what a node has booted. That boot chain and that service run today SHIPPED (§14). What they do not yet provide is named precisely in that section rather than left implicit: the resulting signature is not bound to the specific machine that produced it, so a bond placed against it would secure a claim any operator's own software could reproduce, not a claim about a particular piece of hardware. The attestation network proposed to close that gap — committee sampling and a challenge protocol, with no slashing, since the paper records a decision that nothing in Flux is ever forfeited (§14) — is PLANNED and, as §25 argues at length, is the highest-priority security work on its own merits rather than a precondition for the first change above: the eligibility gate it would bind already guards all issuance today, and a compensation mechanism that pays every eligible node raises the payoff to passing that not-yet-machine-bound gate by exactly what it pays, so PNR's ramp should track it.

Value settled transparently, on rails whose privacy layer is being withdrawn. `fluxd` is a Zcash fork, and its shielded-transaction machinery — Sprout and Sapling notes, encrypted outputs, a note commitment tree, Groth16 proofs — is inherited, compiled into every node, and running today SHIPPED (§20). The shielded pools presently hold 96,479.94 FLUX, against a total issued supply of 429,466,823.97 FLUX at height 2,912,015 — 0.0225 % of supply SHIPPED [measured: api.runonflux.io/daemon/getblockchaininfo, 2026-09-03]. The pool is closed by consensus: transparent-to-shielded transfers have been rejected since height 835,554 SHIPPED [fluxd/src/main.cpp:1398–1408], so every shielded unit on the chain today predates the measured tip by over two million blocks. **Both pools are to be retired** [intent: design intake] (PLANNED). The measurement above is most of the argument: a mechanism holding 0.0225 % of supply, unable to grow, and requiring a Rust proving stack in the consensus path is a poor trade against a stated goal of less code and a more thorough audit (§20.9). The decisive argument is narrower and is worth stating here rather than only in §20: a shielded pool would not hide what a tenant most plausibly wants hidden, because a Flux application specification is a public message, gossiped to every node and committed on chain by hash in its payment transaction, and the control plane serves the global specification set over a public endpoint. The workload is public by construction; shielding the payment hides the funding source, not the workload graph.

So the honest statement of this pillar is the one the paper makes and defends rather than the one an earlier draft asserted: **Flux settles transparently, and treats payment privacy as an unsolved problem rather than a delivered feature**. §21 states the requirement formally and proves two results that outlast the decision — that amount hiding is unsatisfiable under

PNR, and that the residual problem is confined to one optional path — but its constructions are inapplicable to a chain with no pool, and it says so.

1.4 What is new in this paper

Nothing below claims novelty for what Flux inherited outright — the shielded transaction machinery is Zcash's (§20), the UTXO model and the limits on what its scripts can express are Bitcoin's (§17), and the sortition-based block production that replaced mining is deployed, described, and not invented here (§4). Four results in this paper are, to the corpus surveyed, new.

First, a proof that pooling and dripping fee revenue — rather than paying it out to next block's payee directly — defeats a specific timing attack that deterministic payee rotation otherwise admits. Under a next-block payout rule, an operator who is also a tenant can time its own payment to land just before a block it is scheduled to receive, and for a single node in the highest tier this recaptures up to roughly a third of the operator's own payment at the design's proposed parameters; under a fee-pool drip the same attack recaptures a fraction bounded by that tier's share of one drip step, several orders of magnitude smaller for the recommended drip constant PLANNED (§7).

Second, a measurement, not merely a construction, that collateral-weighted moderation is Sybil-resistant by construction and, at the distribution measured on 2026-09-03, not censorship-resistant. Voting weight cannot be increased without locking additional collateral, which is the resource-cost property the design is built on; measured against the live node-weight distribution, four owner identities already hold enough collateral-weighted voting power to reach the proposed 25 % removal threshold on their own SHIPPED [measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03] (§23). Both properties are true of the same mechanism, and the paper states both.

Third, an observation about the consensus header format, offered with an explicit hedge because it has not been checked against a bridge implementation [inferred]. Every block produced under the current consensus rule carries a fixed chainwork value of 2^{40}, a choice made so that fork choice is a block-count race rather than a work comparison SHIPPED [fluxd/src/pow.cpp:284–290] (§4). A header chain with identical work per block carries no signal a light client could use to prefer one candidate chain over another beyond counting blocks, which is not the guarantee header-only verification is built to provide; if that reading holds, it forecloses the kind of header-only light-client bridge design a work-bearing chain would otherwise support, a constraint this paper argues §22 must design around rather than treat as incidental.

Fourth, a separation of an agent's *deployment* authority from its *spending* authority that makes the first tractable with machinery already running and states precisely why the second is not tractable on the Flux L1 at all without either a new trusted party or new consensus state: script validation on a Bitcoin-derived chain is a pure function of the spending transaction and the outputs it consumes, so no script can read how much a key has already spent and therefore no script can enforce a rate limit or a cumulative cap PLANNED (§17).

1.5 What this paper does not claim

The paper makes no price claim, sets no target, and uses no investment framing anywhere; it confines itself to issuance, supply, operator income and demand [intent: design intake]. It does not assert that the agent-demand premise stated in §1.1 is correct — only that the architecture described in the rest of the paper follows from betting that it is. And it does not claim that the work remaining to reach that architecture is uniformly straightforward. §25 classifies every planned mechanism by what kind of work it needs and finds four that are Research rather than Engineering or Extension: post-quantum migration, for which nothing exists in the surveyed repositories and no scheme yet addresses outputs whose keys the protocol cannot move; bounded spending authority for an autonomous agent on the Flux L1, which the UTXO script model cannot express without a new trusted party or new consensus state; a shielded payment provably

funding one specific, unlinked deployment, which is hard because an application's address is itself a workload identifier; and the moderation quorum's concentration problem, where no reweighting, threshold change or eligibility gate the paper considered raises the number of colluding parties needed to censor an application above single digits at the measured distribution.

1.6 How to read this paper

Every substantive claim in this document carries two tags. A *maturity* tag — SHIPPED, IN-PROGRESS, PLANNED or VISION — states how real the claim is: SHIPPED means it runs today and is written in the present tense; PLANNED and VISION describe what does not yet exist and are written in the future or the subjunctive, never in the present. A *provenance* tag states where the claim comes from: a code citation ([repo/file:line]), a roadmap citation ([roadmap: item]), a design-intent citation ([intent: §n]), a measurement ([measured: endpoint, date]), or an explicit hedge ([inferred]) for a conclusion this paper draws rather than one the evidence states outright. The rule that makes the pairing meaningful is strict: *only* a code citation — to deployed source or to a named branch — can support a SHIPPED claim about behaviour, and *only* a measurement can support a SHIPPED claim about a fact of the live network, such as a node count, a balance or a price, which no line of code can establish. A roadmap item or a design-intent note can describe an aspiration precisely, and this paper cites them precisely, but neither can promote a claim to SHIPPED on its own. At any sentence, this pairing is meant to let a reader answer two questions without searching elsewhere in the document: does this exist, and how do we know. Where a *Settled*, *Not established by this survey* or *Proposed by this paper* callout appears, it marks respectively a decision that has been taken together with the value chosen, a fact no artefact read for this paper carries, or a construction the author has not ruled on — never a gap the authors failed to notice.

The document is organized in five parts and a bridge between them. Part I fixes the system and adversary model and the measured baseline this section previews. Parts II and III describe the deployed system — consensus, monetary policy, the orchestration layer, networking, attested execution — at a depth intended to support reimplementation, including the parts that do not work the way a casual description would suggest. Part IV specifies the target architecture for agents and value at the same depth, with every undecided parameter named rather than smoothed over. Part IV.5 composes the whole of it into one operational walkthrough, run once against the deployed system and once against the target. Part V tests the claim that the remaining distance is extension rather than invention and reports, in §25, that it is not uniformly so.

1.7 Method

Three different activities produced the claims in this paper, and keeping them visibly separate is the single most important discipline it follows. What was *read* is a corpus of Flux repositories, including private ones that the team has agreed may be cited freely by file and line in this public document [intent: design intake]; every SHIPPED claim about behaviour traces to a specific file and line range in that corpus. What was *measured* is a set of named public endpoints — chiefly `api.runonflux.io`'s node, application and blockchain-info routes — queried on 2026-09-03 at chain height 2,917,115, with every figure reproducible by running the accompanying scripts against the same live API. What was *designed* is different in kind from both: mechanisms specified in this paper's own design-intake documents ([intent: design intake]) that do not exist in any repository and are not measurements of anything running, but are this paper's own construction, stated to a level of precision that a reader could implement and that a reviewer could attack. A reader who cannot tell, at any given sentence, which of these three produced it has found a defect in this paper, not a subtlety to be inferred.

2 System model, adversary model, and notation

Every property this paper states later — about block production (§4), about revenue-funded compensation (§7), about delegated authority (§17), about private payment (§21), about bridging (§22) and about governance (§23) — is a statement about the model fixed here. This section names the participants and what each can do; states the two different consistency models the system runs (one for the chain, one for the orchestration layer) and says why the difference is the most important structural fact about Flux; enumerates the trust boundaries as they exist in the deployed code, including the ones a reader would not expect; gives the adversary an explicit capital budget denominated in FLUX and prices what each level of capital buys against the measured network; and records eleven numbered assumptions, **A1**–**A11**, that every later section cites by number. The assumptions are stated neutrally. Several of them foreclose properties a reader might otherwise assume hold; that is the point of writing them down.

Unless a claim is tagged otherwise, this section is SHIPPED and each claim carries either a source citation of the form [repo/file:line] or a measurement citation of the form [measured: endpoint, date].

2.1 Participants

Definition 2.1 (Node operator)**.** SHIPPED A *node operator* is a participant that controls, for some tier $t \in \mathcal{T} = \{\mathsf{C}, \mathsf{N}, \mathsf{S}\}$: (i) an unspent transparent output of exactly c_t, with $c_{\mathsf{C}} = 1{,}000\,\text{FLUX}$, $c_{\mathsf{N}} = 12{,}500\,\text{FLUX}$, $c_{\mathsf{S}} = 40{,}000\,\text{FLUX}$ [fluxd/src/fluxnode/fluxnode.h:57–63]; (ii) a node signing key, used to sign the registration and confirmation transactions and to sign PoN (Proof of Node) blocks; and (iii) a payment address, which need not equal either of the above.

Capabilities. Broadcast a `FLUXNODE_START_TX` spending a collateral output matured by at least 100 blocks [fluxd/src/coins.cpp:630–686]; broadcast a benchmark-signed confirmation transaction to enter the paid set [fluxd/src/fluxnode/activefluxnode.cpp:18–74]; re-confirm at least every 640 blocks or be removed [fluxd/src/fluxnode/fluxnode.h:31–34]; run one FluxOS instance that participates in the orchestration overlay and self-selects applications; and, in any slot τ for which its collateral outpoint is eligible, produce and sign one block.

Economic position. Every block pays exactly one node of each tier a reward of $\sigma_{PoN}(h) \cdot \beta_t / 14$, with tier bases $\beta_{\mathsf{C}} = 1.0$, $\beta_{\mathsf{N}} = 3.5$, $\beta_{\mathsf{S}} = 9.0$ against $\sigma_{PoN}(h_{PoN}) = 14\,\text{FLUX}$ [fluxd/src/main.cpp:2886–2939]. Because the per-tier payment queue is ordered by longest-time-since-last-paid [fluxd/src/fluxnode/fluxnode.h:313–328], the rotation length R_t equals the tier's node count n_t. Collateral is locked but is not destroyed by any lifecycle transition: the three removal conditions are collateral spent, failure to re-confirm, and collateral-amount mismatch during the migration windows [fluxd/src/fluxnode/fluxnode.cpp:231–290]. There is therefore no slashing, and the cost of misbehaviour to an operator is the opportunity cost of locked capital, not a forfeited bond.

Measured population. $n_{\text{tot}} = 6{,}489$ confirmed nodes, of which $n_{\mathsf{C}} = 3{,}247$, $n_{\mathsf{N}} = 1{,}615$, $n_{\mathsf{S}} = 1{,}627$; total collateral $88{,}514{,}500\,\text{FLUX}$ [measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03].

Definition 2.2 (Tenant)**.** SHIPPED A *tenant* is a participant that controls a FluxID (a base58 P2PKH-style address) and a spending key over transparent FLUX. It holds no collateral, has no consensus role, and is identified to the orchestration layer only by an ECDSA signature over a login phrase or over an application specification [flux/ZelBack/src/services/verificationHelper.js:17–48].

Capabilities. Register, update and remove a globally-registered application; pay the computed price to a fixed transparent sink address; start, stop and exec into its own containers via the `appownerabove` privilege [flux/ZelBack/src/routes.js:1196–1265].

Sub-types. A *human* tenant reviews and signs on a device it controls. An *autonomous* tenant — an agent $\mathcal{A}$ acting for a principal $\mathcal{U}$ — differs in exactly one respect that matters to this paper: it must be able to act without a human in the loop, and the deployed stack already permits that through a bearer token whose authority is bounded by action and unbounded by value (§17). No delegated-spending primitive with a budget $\mathcal{B}$ or a scope Σ exists in the deployed system.

Definition 2.3 (The Foundation as a distinguished participant)**.** SHIPPED "The Foundation" is used in this paper informally, for the community and the people behind the project; it is not a legal entity. Write $\mathcal{F}$ for the participant defined *extensionally*, as the holder of the following key sets and the operator of the following infrastructure. $\mathcal{F}$ is distinguished because several of its capabilities cannot be acquired with any amount of FLUX.

K1. The 4 hardcoded emergency public keys; any 2 of them suffice to author a block that bypasses PoN eligibility and per-node signature verification [fluxd/src/chainparams.cpp:242–253], [fluxd/src/emergencyblock.cpp:56–150].

K2. The 5-entry benchmark-signing allowlist `vecBenchmarkingPublicKeys`, each entry carrying a UNIX-timestamp validity start, against which every node's benchmark signature is verified by consensus [fluxd/src/chainparams.cpp:233–240].

K3. The dev-fund recipient address, to which a consensus-enforced minimum coinbase output of $\sigma_{PoN}(h) - \sum_{t\in\mathcal{T}} \sigma_{PoN}(h)\beta_t/14$ — i.e. $0.5/14 = 3.57\%$ of PoN emission at activation — must be paid [fluxd/src/main.cpp:2877–2884], enforced at [fluxd/src/main.cpp:1231–1249]. The address is a 2-of-4 multisignature held by the four named individuals below; every spend from it reveals the redeem script on chain. About once a year its accumulated outputs are consolidated into a separate 3-of-4 SSP Enterprise custody multisignature in which each signer is itself a 2-of-2 wallet-and-key pair, so six of the eight keys are required; the most recent consolidation was at block 2,998,902 on 2026-10-01.

K4. Two FluxIDs holding the `fluxteam` privilege on *every* node's FluxOS API, conferring `adminandfluxteam` rights including broadcast injection, daemon start/restart, and application updates on behalf of an owner [flux/ZelBack/src/services/verificationHelperUtils.js:96–156], [flux/ZelBack/src/routes.js:1386–1441].

K5. Merge access to the policy repositories whose documents every node fetches and enforces (`fluxos-network-policy`, and the `helpers/` mirror in `RunOnFlux/flux`) [fluxos-network-policy/.github/CODEOWNERS:1–15], [flux/ZelBack/src/services/appSecurity/imageManager.js:179–195].

K6. The UEFI Secure Boot Platform Key whose SHA-256 is pinned in the ArcaneOS enrolment binary; the node checks the firmware's enrolled PK against that pin [fluxsas/src/fes/fes.cpp:70–81].

K7. The `fluxsas` attestation signing material and the internal mTLS CA [private source: SAS API server], [fluxsas/tools/gen-certs.sh:38].

K8. The multisig that posts on-chain chain-parameter messages, which change the deployment price table with effect from their occurrence height [flux/ZelBack/src/services/utils/chainUtilities.js:9–52].

K9. The registrar, authoritative DNS and ACME accounts for the `runonflux.io` name space, the 16-host FDM fleet, the 5 PowerDNS servers, the single-host DNS gateway and the Arcane VIP, all inside one privately-operated `10.100.0.0/24` [flux-fdm-infrastructure/ansible/inventory/hosts.yaml:1–206].

Decided. custody is *personal*, not corporate. No legal entity holds these key sets. K9's servers are run by the team across several hosting providers, and its DNS is managed in a team Cloudflare account. The `runonflux.io` domain registrations and K1–K8 are held personally: four named individuals — Tadeas Kmenta, Valter Silva, Daniel Keller and Jeremy Anderson — each hold one, personally governed [intent: design intake], matching the four-key, 2-of-4 emergency configuration (§4) and the 3-of-4 bridge configuration of Section 23. The distinction bears on

every concentration argument in this paper: personal custody by four named people is a different accountability model from custody by a company, and both differ from the anonymous-operator model a reader of a decentralized-network paper might otherwise assume. No repository or endpoint read for this paper establishes it; it rests on the author's statement.

Definition 2.4 (Holder of a parallel asset)**.** SHIPPED A *parallel-asset holder* controls a token on one of ten non-Flux chains that represents FLUX. Such a holder has no consensus role, no collateral, and no orchestration capability. Its economic position is a claim against the operator of the incumbent Fusion bridge, not against Flux consensus; §22 treats the resulting custody structure. The inventory of ten chains is given in §6.

2.2 Network model

Definition 2.5 (Chain network)**.** SHIPPED The chain network is the set of `fluxd` instances exchanging blocks and transactions over a Bitcoin-style gossip protocol [fluxd/src/chainparams.cpp:175–180]. We assume *partial synchrony*: there is an unknown global stabilisation time after which any message between two correct nodes is delivered within an unknown finite bound. Before it, messages may be delayed arbitrarily.

PoN additionally requires bounded *clock* skew, not merely bounded delay: the slot index is $\tau(t) = \lfloor (t - t_{\text{genesis}})/\Delta_{PoN} \rfloor$ with $\Delta_{PoN} = 30\,\text{s}$ [fluxd/src/pon/pon.cpp:88–93], [fluxd/src/chainparams.cpp:105], and the only enforcement is a 300 s future-time tolerance on PoN headers plus a strict blocktime > prev blocktime ordering [fluxd/src/main.cpp:5330–5378,5564–5572].

Definition 2.6 (Orchestration overlay)**.** SHIPPED The orchestration overlay is the set of FluxOS instances exchanging JSON-over-WebSocket messages, each wrapped in an envelope $\{\mathsf{version}, \mathsf{timestamp}, \mathsf{pubKey}, \mathsf{signature}, \mathsf{data}\}$ signed by the node key [flux/ZelBack/src/services/utils/fluxBroadcastHelper.js:11–26]. There is no consensus: application state lives in per-node MongoDB collections — the `globalzelapps` database of messages, specifications and locations that every node replicates by gossip [flux/ZelBack/config/default.js:68–76].

Delivery is best-effort and threshold-gated. Duplicate suppression is a per-node message cache after a random 1 ms–75 ms jitter; a peer sending five malformed or badly-signed messages in ten minutes is disconnected [flux/ZelBack/src/services/fluxCommunication.js:549–618]. Above chain height 1,751,250 only the 40-hex message hash is gossiped and the body is pulled on demand [flux/ZelBack/src/services/fluxCommunication.js:494–498]. Convergence state is soft: a location record expires after 7,500 s, a temporary message after 3,600 s, an installing record after 900 s, and a graceful-exit notification extends an affected location by 420 s [flux/ZelBack/config/default.js:311–314], [flux/ZelBack/src/services/utils/appConstants.js:86].

Property 2.7 (Two consistency models — the structural fact)**.** SHIPPED Chain state is totally ordered and agreed by consensus. Application state — which applications exist, which nodes run them — is a set of per-node databases converged by signed but unordered gossip, with no total order and no termination guarantee. Consequently no predicate of the form "application a is running on node i at height h" is decidable from the chain.

Proof sketch. Placement is opportunistic self-selection: each node independently polls for applications missing instances and picks *uniformly at random* among the candidates, subject to soft deferrals and a broadcast collision-avoidance wait [flux/ZelBack/src/services/appLifecycle/appSpawner.js:254–336], [flux/ZelBack/config/default.js:321]. The resulting placement is published as a TTL'd location broadcast on the overlay, and nothing in the deployed code writes placement to the chain [flux/ZelBack/config/default.js:56–60]. The chain carries the *payment* and the registry of collateralised nodes; it does not carry the assignment of workloads to nodes. Hence any chain-verifiable statement about placement would have to be reconstructed from off-chain gossip, which by Definition 2.6 offers no agreement property. □

Remark 2.8 (Why this is the load-bearing fact). Four later sections turn on Property 2.7. §7 must place the fee pool Φ in *consensus* state rather than in overlay state, because a value that determines payment cannot be converged best-effort. §21 inherits the consequence that the payment is the only chain-visible artefact of a deployment, which is what makes payer/workload unlinkability hard. §22 cannot export any placement or capacity fact to another chain. §23 must site the moderation quorum on the chain, because a vote counted on the overlay has no agreed tally. A reader who carries a single mental model of "the Flux state" will draw wrong conclusions in all four.

What breaks when delivery assumptions fail. SHIPPED (i) *Chain partition*: competing tips are resolved by block count and first-seen rather than by accumulated work (**A3**); reorganisations are capped at 40 blocks [fluxd/src/main.h:74], a bound that was raised to 5,000 for the closed height window [2,020,000, 2,025,000] during PoN stabilisation [fluxd/src/main.cpp:8983–8988]. (ii) *Overlay degradation*: below 12 live peers the application spawner is paused and below 4 it is treated as degraded; a registration is refused outright below 8 outbound or 4 inbound peers [flux/ZelBack/config/default.js:262–269]. (iii) *Discovery outage*: with `api.runonflux.io` unreachable, FDM keeps serving its last configuration and propagates no new information — fail-static, with no independent source [flux-domain-manager/src/services/flux/dataFetcher.js:47–91]. (iv) *Policy outage*: a node that cannot fetch the policy documents keeps enforcing its last-read copy indefinitely and across restarts, and a node that never had one declines to answer — this is the publisher-side contract [doc: fluxos-network-policy/README.md:33–44]; the node-side fetch-and-cache path is verified in FluxOS [flux/ZelBack/src/services/appSecurity/imageManager.js:179–195]. (v) *Hash-list outage*: if both dm-verity allow-list endpoints are unreachable, the ArcaneOS watchdog treats the node as secure and takes no action — fail-open [private source: SAS watchdog].

2.3 Trust boundaries

Each row of Table 1 names something that is trusted, the participants who must trust it, and what a failure of that trust yields. The list is not a critique; it is the set of assumptions any later property must either use or discharge.

Table 1: Trust boundaries in the deployed system. All rows are SHIPPED.

	What is trusted	Trusted by	A failure yields
TB1	Consensus rules in `fluxd` [fluxd/src/main.cpp:5330–5572]	all participants	Divergent ledgers; reorganisation bounded at 40 blocks.
TB2	Collateral registry (`determ_zelnodes` cache, rebuilt at start) [fluxd/src/fluxnode/fluxnodecachedb.cpp:49–79]	consensus and placement (and, under §23, vote weight)	Crash inconsistency between the UTXO set and the node cache; a dedicated sync marker [fluxd/src/fluxnode/fluxnodecachedb.cpp:17,166–171] and an operator-invoked `rebuildfluxnodedb` repair path [fluxd/src/rpc/fluxnode.cpp:78] exist.
TB3	The *local* `fluxbench` process, invoked by `popen()` [fluxd/src/fluxnode/benchmarks.cpp:227–291]	the co-located `fluxd`	Benchmark-gating bypass: an operator's own host can produce the signature attesting that its hardware meets the tier thresholds. Not tier forgery — the registration must still match an on-chain collateral amount.
TB4	Benchmark maintainer allowlist [fluxd/src/chainparams.cpp:233–240]	consensus	Any holder of a currently-valid allowlist key can authorise benchmark signatures fleet-wide.

(Table 1 continued)

	What is trusted	Trusted by	A failure yields
TB5	`fluxsas` attestation service [private source: SAS API server]	`fluxbench` and `fdm-nginx` only; *no tenant path*	An attestation demonstrates possession of a value present in every copy of the software, not that a machine booted a given image.
TB6	dm-verity hash allow-list service (two Flux-operated hosts) [private source: SAS watchdog]	every ArcaneOS node	Whoever controls those names defines "known-good"; if both are unreachable the watchdog does nothing (fail-open).
TB7	UEFI Platform Key pinned by SHA-256 [fluxsas/src/fes/fes.cpp:70–81]; TPM seal against PCR 7 only [fluxsas/src/fes/fes.cpp:173–186]	the node's own boot path	Whoever controls that PK defines what may boot. PCR 7 measures Secure Boot policy, so the seal opens for any PK-signed kernel, not a specific measured image.
TB8	OS update channel: release metadata and checksum files on the Flux CDN [arcaneos-releases/publish_release.sh:37,65–73,98–101]	operators updating ArcaneOS	Whoever controls the CDN and metadata defines the candidate image set.
TB9	`fluxos-network-policy` documents, unsigned, fetched over plain HTTPS and enforced hourly by every node [flux/ZelBack/src/services/appSecurity/imageManager.js:179–195], [flux/ZelBack/src/services/serviceManager.js:530–531]; publication contract [doc: fluxos-network-policy/README.md:30–44,320–326]	every FluxOS node	Any org owner merging to `main` can block a container image, an owner address or an application hash network-wide, or revoke an enterprise-node grant, with effect on the next scheduled fetch: 6 h for blocked repositories and enterprise nodes, 12 h for the tampering list. No signature, no second reviewer, no staged rollout.
TB10	`api.runonflux.io` as sole application-discovery source [flux-domain-manager/src/services/flux/dataFetcher.js:47–91]	FDM, `flux-apps-dns-manager`	No independent answer to "which applications exist and where"; the routing plane fails static on the last-known answer.
TB11	FDM/DNS/certificate path: 16 FDM hosts, 5 PowerDNS servers, one DNS gateway host, VRRP VIP, one private `/24` [flux-fdm-infrastructure/ansible/inventory/hosts.yaml:1–206]	every tenant reachable by name, every TLS client	Name resolution, HAProxy election and certificate issuance for the application name space are controlled by one operator; a wildcard blacklist in the FDM config removes an application from that path unilaterally [flux-domain-manager/config/appsConfig.js:2–6].

(Table 1 continued)

	What is trusted	Trusted by	A failure yields
TB12	Application payment sink addresses [flux/ZelBack/config/default.js:241–244]	every paying tenant	Payment is a transfer to a fixed transparent address; entitlement follows from each node independently re-deriving the price. Underpayment is logged and the message is never promoted, which is indistinguishable to the payer from non-propagation [flux/ZelBack/src/services/appMessaging/messageVerifier.js:733–746,777].
TB13	FluxOS `fluxteam` privilege [flux/ZelBack/src/services/verificationHelperUtils.js:96–156]	every node	Two FluxIDs can broadcast on the overlay, restart the daemon and update applications on behalf of their owners, on every node.
TB14	Chain-parameter multisig posting price messages [flux/ZelBack/src/services/utils/chainUtilities.js:9–52]	every tenant and node	The deployment price schedule changes from the occurrence height of a posted message.
TB15	The 2-of-4 emergency key set [fluxd/src/emergencyblock.cpp:183–191]	all participants	Two key holders can insert a valid block at any height after PoN activation, bypassing sortition and per-node signature checks, with no cooldown, no stall detection and no on-chain justification.

Decided. the emergency-block path is observed *on-chain*: an emergency block is a block, so its production is visible to anyone following the chain and detection does not depend on an off-chain alerting system [intent: design intake]. That is the whole of the monitoring story — this paper does not claim a rate limiter, an alarm or a watch rota outside the chain, because none is established and none is asserted. Observability is not the same as response, and no control in consensus bounds the path: the threshold stays at 2-of-4 and time or stall gating is absent (Section 23).

Not established by this survey. whether ArcaneOS release artefacts are signature-verified beyond the published SHA-256 sums is not established in the repositories read (TB8).

Not established by this survey. the fetch interval for `vettedrepositories.json` is not stated in the policy repository, so the 6 h–12 h bound in TB9 covers four of the five documents.

Decided. the application payment sink is held by the same four individuals as the emergency and bridge keys above — Tadeas Kmenta, Valter Silva, Jeremy Anderson and Daniel Keller [intent: design intake]. The address itself is expected to change on migration to SSP Enterprise; the custody does not. This is a policy fact rather than a code fact: no artefact read for this paper establishes it, and it rests on the author's statement. It is stated because every application payment is sent to that one address today (Section 7), so who holds its key is load-bearing for the whole revenue path and not a detail a reader should have to infer.

2.4 The adversary

Definition 2.9 (Adversary $\mathcal{Z}$)**.** $\mathcal{Z}$ is parameterised by the triple $(\mathcal{C}_{\mathcal{Z}},\ \mathcal{K},\ \mathcal{N})$:

- $\mathcal{C}_{\mathcal{Z}} \in \mathbb{R}_{\geq 0}$, a *capital budget* denominated in FLUX and spendable on collateral or on deployment payments. By **A11** it is never converted to or from fiat.
- $\mathcal{K} \subseteq \{\mathrm{TB1}, \ldots, \mathrm{TB15}\}$, the set of trust boundaries $\mathcal{Z}$ controls. Membership in $\mathcal{K}$ is *not* priced in FLUX: no budget buys TB9 or TB15.

- $\mathcal{N} \in \{\mathsf{observe},\ \mathsf{delay},\ \mathsf{partition}\}$, network power over the two networks of §2.2.

$\mathcal{Z}$ is computationally bounded: it cannot forge ECDSA or Ed25519 signatures under keys it does not hold, and cannot invert SHA-256. It may deviate arbitrarily from any behaviour that is not consensus-enforced — in particular from the advisory rank delay of Property 2.10.

What $\mathcal{C}_{\mathcal{Z}} = 0$ buys. SHIPPED Several properties in this paper fail to an adversary with no capital at all.

Z1. *Produce attestations that any consumer of `fluxsas` accepts.* The signing seed for the default, `cdn`, `mesh` and `app` attestation purposes is derived from values compiled identically into every copy of the software; no input depends on the TPM, on Secure Boot state, or on any per-machine value [private source: SAS API server]. No node, no hardware and no collateral are required. This is why **A5** is stated as it is.

Z2. *Link payments to deployments.* All application payments land at fixed transparent addresses [measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03], and a payment names the application it funds; §21 measures the resulting anonymity set as a singleton within its block.

Z3. *Inject unsigned frames into the overlay.* The raw hash-gossip frames `messageHashPresent` and `requestMessageHash` bypass the signed envelope entirely [flux/ZelBack/src/services/fluxCommunication.js:511–536]. The capability is bounded — these frames carry a 40-hex hash and no application data — but it is free.

Z4. *Degrade.* Reducing a node's live peer count below the spawner thresholds, or causing confirmation loss, removes that node's applications; the enforcement is redundant across the fleet and therefore also amplifies a stale view [flux/ZelBack/src/services/appMonitoring/nodeStatusMonitor.js:62–149].

Z5. *Act as a tenant.* The price floor for a deployment is 0.01 FLUX per default term [measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03], so tenant-side capabilities are effectively free.

A zero-capital $\mathcal{Z}$ cannot produce a valid PoN block, cannot claim a tier it has not collateralised (**A4**), and cannot reach any collateral-weighted threshold.

What capital buys. Write $W = 88{,}514.5$ for the measured total collateral weight, in the units of 1,000 FLUX of Table 2 (that is, 88,514,500 FLUX of collateral), and $n_{\text{tot}} = 6{,}489$ for the measured node count [measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03].

Property 2.10 (Share of block production)**.** SHIPPED Under **A1** and **A3**, let $\mathcal{Z}$ control $m = f\, n_{\text{tot}}$ confirmed nodes. Then $\mathcal{Z}$'s expected share of produced blocks is at least f, and strictly exceeds f whenever the per-slot eligibility probability p is bounded away from 0.

Proof. Eligibility in slot τ is $H(\text{collateral} \| \text{prevhash} \| \tau) \le T$, one hash per confirmed node, *independent of tier* [fluxd/src/pon/pon.cpp:70–93,400–433]. Distinct collateral outpoints give distinct preimages, so per slot eligibility is Bernoulli(p) independently across nodes. A slot yields a block iff at least one node is eligible, with probability $g(n_{\text{tot}})$ where $g(x) = 1 - (1-p)^x$. $\mathcal{Z}$ takes the slot whenever at least one of its m nodes is eligible: the rank-based delay of 4,000 ms per rank is documented in the source as an orphan-reduction heuristic with no consensus enforcement [fluxd/src/pon/pon-minter.cpp:78–104,217–221,250], so an $\mathcal{Z}$ that ignores it publishes before any honest node of rank ≥ 1; the 5 s minimum-since-last-block guard in the same minter loop is likewise unenforced by consensus [fluxd/src/pon/pon-minter.cpp:200–207], and the only consensus bound is that a block's

time exceed its predecessor's [fluxd/src/main.cpp:5564–5572]. Hence $\mathcal{Z}$'s share is $g(m)/g(n_{\text{tot}})$. The map g is concave on $[0,\infty)$ with $g(0)=0$, so $g(fn_{\text{tot}}) \geq f\,g(n_{\text{tot}})+(1-f)g(0) = f\,g(n_{\text{tot}})$, giving share $\geq f$; the inequality is strict for $p>0$ by strict concavity. The per-slot model understates $\mathcal{Z}$: consensus does not pin a block's slot to wall-clock time, so $\mathcal{Z}$ may evaluate eligibility over every slot in the 300 s future-time window rather than the current slot alone (§4.3), which can only raise its share, so the bound stands. □

Property 2.11 (Cost of an eligibility share)**.** SHIPPED To reach a fraction f of eligibility draws by newly collateralising nodes, $\mathcal{Z}$ must add $m = \frac{f}{1-f} n_{\text{tot}}$ nodes, at minimum cost $\mathcal{C}_{\mathcal{Z}} = m \cdot c_{\mathsf{C}}$. At $f = 1/2$ this is $m = 6{,}489$ nodes and $\mathcal{C}_{\mathcal{Z}} = 6{,}489{,}000$ FLUX. Per eligibility draw, C is exactly $c_{\mathsf{S}}/c_{\mathsf{C}} = 40$ times cheaper than S.

Proof. $m/(n_{\text{tot}}+m) = f$ rearranges to the stated m. Cost is minimised by the cheapest tier because, by the proof of Property 2.10, sortition assigns one hash per confirmed node regardless of tier. The registration is visible on-chain in advance: a start transaction requires 100 blocks of collateral maturity [fluxd/src/fluxnode/fluxnode.h:20] and the confirm transaction is a separate on-chain event. □

Property 2.12 (Cost of the moderation threshold)**.** Let θ be a removal threshold expressed as a fraction of collateral weight W (§23, PLANNED). Then, in W's units of 1,000 FLUX, (i) acquiring existing collateral costs $\mathcal{C}_{\mathcal{Z}} = \theta\, W$, because the transfer leaves W unchanged; (ii) newly collateralising costs $\mathcal{C}_{\mathcal{Z}} = \frac{\theta}{1-\theta} W$. At $\theta = 0.25$ these are 22,128,625 FLUX and 29,504,833 FLUX respectively.

Proof. (i) is immediate. For (ii), solve $w/(W+w) = \theta$. The measured distribution corroborates (i): the three largest node keys hold 22,200,000 FLUX, which is 25.08% of W [measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03] — four addresses under the owner-identity model (§23; **A10**) — so an $\mathcal{Z}$ purchasing that same weight reaches the threshold, and does so by transacting with three or four counterparties. □

Remark 2.13 (The two mechanisms are not independently priced). Composing Properties 2.11 and 2.12: an $\mathcal{Z}$ that reaches $\theta = 0.25$ by newly collateralising 29,504 C nodes simultaneously holds $29{,}504/(6{,}489+29{,}504) = 82.0\%$ of eligibility draws. Conversely the 6,489,000 FLUX that buys half of all eligibility draws buys only $6{,}489/(88{,}514.5+6{,}489) = 6.8\%$ of moderation weight. The cheapest route to block production is the most expensive route to votes and vice versa, and the cheap route to the moderation threshold is also a route to block-production dominance. Both routes are visible on-chain in advance, which is the property §23 builds on.

Property 2.14 (Reorganisation cost)**.** SHIPPED Under **A1** and **A3**, the cost of an ℓ-block reorganisation is a function of collateral and of elapsed slots, not of computation, and the capital committed is recoverable.

Proof sketch. Every PoN block contributes a fixed 2^{40} to chainwork regardless of its target [fluxd/src/pow.cpp:284–290], so a private chain outranks the public chain exactly when its block count is larger. By the proof of Property 2.10, an $\mathcal{Z}$ holding fraction f of confirmed nodes extends its private chain at expected rate $g(fn_{\text{tot}})$ blocks per slot, with $g(x) = 1-(1-p)^x$; producing ℓ private blocks therefore costs $\mathcal{Z}$ elapsed slots, not hash trials, and the required capital is $\mathcal{C}_{\mathcal{Z}}$ from Property 2.11. That capital is not consumed by the attempt: the only three transitions that remove a node from the paid set are collateral spent, failure to re-confirm, and a migration-window amount mismatch (Definition 2.1), none of which destroys value. The binding constraint on ℓ is therefore the 40-block reorganisation cap [fluxd/src/main.h:74], not the cost of the attempt. □

Remark 2.15 (Fee-pool income). PLANNED Under PNR (§7) a node would additionally receive a share of the fee pool Φ once per rotation R_t of its tier — at the measured population, once every 1,627 blocks for S, which is 13.56 h at Δ_{PoN} [measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03]. The magnitude of that share is not fixed here.

Decided. $\mathcal{Z}$'s per-node fee-pool income under PNR is $\lambda_t D_h / n_t$ per block on average, with $D_h = \lfloor \Phi_{h-1}/K \rfloor$ (Proposition 7.18) — the per-block drip, times the tier's split, divided by the confirmed nodes in that tier; the operator share ρ enters through the pool's inflow rather than at payout. Every term is now fixed: the drip is Φ_{h-1}/K with $K = 86{,}400$, the operator share follows the ramp of §7 to its cap, and the tier split is the settled partition (Table 24). This section still assumes no *value* for it, because the pool balance is demand-determined and the adversary model must hold for any balance — but the *function* is no longer open.

2.5 Assumptions

Each assumption is cited elsewhere by its label. Each states what it enables and what it forecloses.

Assumption 2.16 (**A1** — chain synchrony)**.** SHIPPED The chain network is partially synchronous in the sense of Definition 2.5, and PoN additionally assumes clock skew bounded by the 300 s header tolerance [fluxd/src/main.cpp:5330–5378]. *Enables*: liveness arguments for block production, and settlement arguments at depth. *Forecloses*: any argument assuming synchrony, and any use of median-time-past ordering for PoN blocks, which use strict predecessor-time ordering instead [fluxd/src/main.cpp:5564–5572].

Assumption 2.17 (**A2** — overlay delivery)**.** SHIPPED The orchestration overlay delivers authenticated messages best-effort, with convergence conditional on the peer thresholds and TTLs of Definition 2.6 [flux/ZelBack/config/default.js:262–271,311–314]. *Enables*: eventual-convergence statements about placement, conditional on peer counts. *Forecloses*: any safety property over application state, any bound on time to convergence, and any chain-derived statement about placement (Property 2.7).

Assumption 2.18 (**A3** — fork choice is a block-count race)**.** SHIPPED Every PoN block contributes a fixed chainwork of 2^{40}, independent of its difficulty target [fluxd/src/pow.cpp:284–290]. Fork choice therefore compares block counts, resolved by first-seen, and the rank-based minting delay is advisory [fluxd/src/pon/pon-minter.cpp:78–104,217–221]. *Enables*: reorganisation cost derived from collateral and eligibility (Property 2.14); a security argument that does not depend on hashing. *Forecloses*: every cumulative-work argument, including light-client fork choice (**A9**) and any claim that reorganisation cost rises with hash rate.

Not established by this survey. `nMinimumChainWork` still carries its pre-PoN value [fluxd/src/chainparams.cpp:168–170]; whether it remains an effective initial-block-download guard under fixed-work PoN was not resolved.

Assumption 2.19 (**A4** — benchmark results are collateral-bound, not hardware-attested)**.** SHIPPED A node's tier is fixed by the collateral amount in its start transaction. Its confirmation transaction carries a `benchmarkTier` attesting that the host meets that tier's hardware minimum (consensus rejects a confirmation whose `benchmarkTier` is below the collateral tier [fluxd/src/main.cpp:1590–1596]) and is signed by `fluxbench` using keys shared across the fleet, validated by consensus against a maintainer allowlist [fluxd/src/chainparams.cpp:233–240], and obtained by `fluxd` from the local `fluxbench-cli` through `popen()`, with `status:"complete"` treated as sufficient [fluxd/src/fluxnode/benchmarks.cpp:227–291]. The registration must match an on-chain collateral amount for the claimed tier [fluxd/src/coins.cpp:630–686]. *Enables*: the statement that tier is collateral-bound — an operator cannot claim a tier it has not collateralised. *Forecloses*: any property conditioned

on the host actually meeting the tier's hardware thresholds, and any use of a benchmark result as evidence about a specific machine. This is a benchmark-gating bypass, not tier forgery, and §4 and §14 must both be written against the weaker statement.

Assumption 2.20 (**A5** — attestations are not machine-bound)**.** SHIPPED A `fluxsas` attestation demonstrates possession of a value present in every copy of the software; none of the derivation inputs depends on the TPM, on Secure Boot state, or on any per-machine value [private source: SAS API server]. Every `/v2/*` route requires an internal client certificate and is bound to loopback, so no tenant-facing verification path exists [fluxsas/src/api_server.h:1042–1053,220–252]. *Enables*: measured boot, dm-verity and LUKS as genuine host-local defences against a remote attacker. *Forecloses*: any protocol that consumes an attestation as a machine binding, any tenant-verifiable execution claim, and any argument that the eligibility gate is enforced by hardware. The bonded attestation network of §14 is the answer to exactly this.

Assumption 2.21 (**A6** — emergency blocks bypass PoN unconditionally)**.** SHIPPED A block whose collateral field equals the emergency sentinel and which carries 2 valid signatures from a hardcoded set of 4 public keys is accepted, bypassing `CheckProofOfNode` and per-node signature verification [fluxd/src/emergencyblock.cpp:56–150], [fluxd/src/pon/pon.cpp:205–252]. The only gate is that PoN is active: there is no time delay, no cooldown and no stall detection, despite an in-source comment stating that such restrictions were contemplated [fluxd/src/emergencyblock.cpp:183–191]. *Enables*: recovery from a chain halt without a coordinated software release. *Forecloses*: any safety property quantified over all valid blocks under PoN eligibility; any light-client rule derived from eligibility (**A9**); and, as a forward constraint, §7 must exclude emergency blocks from the fee-pool drip, or the emergency key set, facing no time gating, inherits the ability to accelerate the release of accumulated fees. It could not redirect them: an emergency block still fills the deterministic node payout [fluxd/src/miner.cpp:487–489], and connection rejects any block that omits it [fluxd/src/main.cpp:3878–3884].

Planned change. The team intends to move this set to 3-of-4 with newly generated keys, aligning it with the bridge quorum of §22 PLANNED [intent: design intake]. No activation is scheduled, and the assumption is unchanged in kind: at 3-of-4 the path still bypasses PoN eligibility and per-node signature verification with no time or stall-detection gating, so every property that references A6 continues to hold.

Assumption 2.22 (**A7** — discovery, naming and runtime policy are operated)**.** SHIPPED Application discovery for the routing layer (FDM and DNS) has a single source [flux-domain-manager/src/services/flux/dataFetcher.js:47–91]; DNS, HAProxy election and certificate issuance run on a 16-host fleet inside one private `/24` [flux-fdm-infrastructure/ansible/inventory/hosts.yaml:1–206]; and the runtime network policy is a set of unsigned documents fetched over plain HTTPS and enforced by every node [flux/ZelBack/src/services/appSecurity/imageManager.js:179–195], whose authenticity "rests on GitHub and on who can merge" and which takes fleet-wide effect on the next scheduled fetch of 6 h–12 h, with no staged rollout and no second-reviewer requirement [doc: fluxos-network-policy/README.md:30–44,279–296,320–326]. *Enables*: incident response measured in hours; the honest baseline against which §23 argues that a collateral-weighted quorum is an improvement. *Forecloses*: any end-to-end decentralization claim for reachability, naming or content policy, and any censorship-resistance claim about application availability as deployed.

Assumption 2.23 (**A8** — the shielded pool is closed to new entrants)**.** SHIPPED Since height 835,554, consensus rejects any transaction carrying transparent inputs together with a joinsplit or a shielded output, with reject reason `bad-txns-fluxrebrand-vprivacy-not-empty` [fluxd/src/main.cpp:1398–1408]. Shielded value may move from one z-addr to another, or exit from a z-addr to a t-addr; it cannot enter. *Enables*: an exact statement of the deployed privacy surface, and the measurement that 96,479.94 FLUX sits in the two pools [measured: api.runonflux.io/daemon/getblockchaininfo, 2026-09-03]. *Forecloses*: any construction in §21 that requires funding a shielded

note from transparent value, and any claim that the anonymity set grows. Re-opening the pool would be a consensus change; the team has instead decided to retire both pools (§20.9, [intent: design intake]), so this assumption describes a mechanism on its way out rather than a prerequisite for future work.

Assumption 2.24 (**A9** — no light client is constructible)**.** SHIPPED With the current header format no Flux light client can be constructed, for three compounding reasons: the PoN header carries no commitment to the node set, so eligibility cannot be checked without the full registry [fluxd/src/primitives/block.h:44–45,64–96]; chainwork is fixed per block (**A3**); and the emergency path can produce a valid block outside the eligibility rules (**A6**). *Enables*: an honest enumeration of the bridge design space in §22. *Forecloses*: light-client bridging and any trust-minimized verification of Flux state on another chain, until a header commitment to the node set exists (§8).

Assumption 2.25 (**A10** — identity is collateral and nothing else)**.** SHIPPED There is no Sybil-resistant identity in the system other than a collateralised node. The two public proxies for operator identity, `pubkey` and `payment_address`, are not proof of common ownership in either direction, so every concentration figure in this paper is a *lower bound* on concentration [measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03]. *Enables*: Sybil resistance by construction for any collateral-weighted mechanism. *Forecloses*: honest-majority-of-identities assumptions, one-node-one-vote designs, and any claim derived from counting distinct keys as if they were distinct parties.

Assumption 2.26 (**A11** — denomination and exogeneity of price)**.** All economic quantities in this paper are denominated in FLUX or in resource units. The market price of FLUX is exogenous and is never modelled, forecast or used in any derivation. *Enables*: statements about issuance, supply, operator income and demand that a reader can check against the chain. *Forecloses*: any fiat-denominated claim, and any crossover or parity result that would depend on price rather than on demand (§7, §26).

2.6 Out of scope

Three things are outside the model and are never assumed, argued or defended.

1. *The security of the container runtime.* Nothing in the attestation path measures or attests the Docker daemon, FluxOS itself, or the hypervisor [fluxsas/src/watchdog.h:547–559]. Isolation between co-tenant containers on one host is assumed to be whatever the runtime provides, and no property in this paper depends on it being more than that.

2. *The correctness of tenant workloads.* A tenant's image may do anything the runtime permits. Content policy is treated only where it is a network mechanism (§23); workload correctness is the tenant's.

3. *The market price of* FLUX (**A11**).

2.7 Notation

Table 2 lists the symbols used in this paper, grouped by domain, with the domain or unit of each. The full index, including every constant and its provenance, is Appendix D; the constant tables are Appendix A. Macros are defined once in `notation.tex` and are not redefined anywhere in the text.

Table 2: Notation by domain. Full index: Appendix D.

Symbol	Meaning	Domain / unit
Chain and consensus		
h	block height	$\mathbb{N}$
h_{PoN}, h_{PA}	PoN activation height; PA-depletion height	$\mathbb{N}$
τ	PoN slot index	$\mathbb{N}$
Δ_{PoN}, Δ_{PoW}	PoN / legacy target spacing	s
B_{yr}	blocks per year	blocks
I_{red}, N_{red}, γ	subsidy reduction interval, count, factor	blocks; $\mathbb{N}$; $\mathbb{Q} \cap (0,1)$
Tiers		
$\mathcal{T}$	$\{\mathsf{C}, \mathsf{N}, \mathsf{S}\}$	finite set
c_t	collateral required by tier	$\mathcal{T} \to \mathbb{Q}_{>0}$, FLUX
n_t, R_t	node count; rotation length	$\mathbb{N}$; blocks
β_t	tier base against a total of 14	$\mathcal{T} \to \mathbb{Q}_{>0}$, FLUX
Emission and supply		
σ, σ_{PoN}, σ_{PoW}	block subsidy	$\mathbb{N} \to \mathbb{Q}_{\geq 0}$, FLUX
δ	consensus dev-fund remainder	FLUX per block
S, Π	cumulative issued supply; premine	FLUX
MAX_MONEY	per-transaction sanity bound, *not* a cap	FLUX
sat	10^{-8} FLUX	unit
Applications and pricing		
a, k, d	an application; instance count; duration	— ; $\mathbb{N}$; blocks
$\mathbf{r}$, $\mathbf{p}$	resource vector; price vector at a height	$\mathbb{Q}^3_{\geq 0}$
$\mathcal{P}$	price of a deployment	FLUX
L	default lifetime	blocks
PNR (PLANNED, §7)		
Φ, K	fee-pool balance; drip constant	FLUX; blocks
ρ, $\rho_{\max}$, $\Delta\rho$	operator share, ceiling, per-step increase	$[0,1]$
$\mathcal{W}$, λ_t	nodes paid in a block; tier split of the drip	finite set; $[0,1]$
Moderation (PLANNED, §23)		
w_i, W	voting weight of identity i; total weight	units of 1,000 FLUX
θ, β_{rep}, $\mathcal{R}$	removal threshold; report bond; reporters	$[0,1]$; FLUX; finite set
Privacy and value		
t-addr, z-addr	transparent / shielded address	—
note, nf, cm, NCT	note, nullifier, commitment, commitment tree	—
π, ivk	zero-knowledge proof; incoming viewing key	—
Agents and delegation (PLANNED, §17)		
$\mathcal{U}$, $\mathcal{A}$	human principal; autonomous agent	—
$\mathcal{B}$, Σ	delegated spending budget; scope of authority	FLUX; finite set
Adversary		
$\mathcal{Z}$, $\mathcal{C}_{\mathcal{Z}}$	the adversary; its capital budget	— ; FLUX

2.8 How to read the tags

Every substantive claim in this paper carries one *maturity* tag and one *provenance* tag. This is a promise to the reader, and it is the mechanism by which a reader can tell, at any point, which half of the paper they are in.

Maturity. SHIPPED means the behaviour is in code that is deployed and reachable on mainnet. IN-PROGRESS means the code exists and is under active development but is not activated on any

network. PLANNED means a design exists and is specified in this paper but no implementation is deployed. VISION means a direction with requirements and interfaces but no construction.

Provenance. [repo/file:line] cites deployed source. [repo@branch:file:line] cites a branch, which a reader may not be able to fetch. [doc: name], [roadmap: item] and [intent: file] cite documents, and are hearsay about behaviour. [measured: endpoint, date] cites a public endpoint with its retrieval date. [inferred] marks a conclusion drawn by the author from cited material rather than stated in it; it never supports SHIPPED. Citations into `flux/` are against the corpus checkout, commit `933614b1c` on `feat/appspec-v9`; in `ZelBack/config/default.js` the line numbers above 127 differ from `master` by up to fifteen lines.

The rules.

1. SHIPPED requires a source citation or a measurement citation. A document alone is never sufficient to support SHIPPED; a roadmap or a README describes intent, not behaviour.

2. PLANNED and VISION material never appears in the present tense. Where this paper writes "the pool would pay" rather than "the pool pays", that is not hedging — it is the tag being honoured in the grammar.

3. A property the author cannot argue appears as a `Conjecture`, an `Open Problem`, or a *Not established by this survey* callout. It never appears as an unlabelled assertion.

4. An undecided parameter carries its domain, the trade-off, and — once decided — a *Settled* callout recording the value. A guess is never inserted in place of a decision.

3 Flux as deployed today

Most descriptions of Flux in circulation — including earlier drafts of this paper's own introduction — are anchored to whatever fleet size was true when someone last looked. The network does not stand still, and a reader who has not looked recently will underestimate both its scale and its heterogeneity: how many nodes, how much collateral is actually locked behind them, how many applications are running, and how old some of those applications' specifications are. This section replaces impression with a single dated measurement, taken directly from the public API surface that any third party can query, and states the numbers that the rest of this paper treats as the baseline against which every later claim — about consensus, about pricing, about the target architecture — is measured.

3.1 Methodology

Two conventions matter for reading every figure in this paper. First, the snapshot was taken over a short window on 2026-09-03 rather than at a single instant: the API captures correspond to chain height 2,917,115, while the supply figures computed by the emission model are stated at height 2,912,015, roughly 42 hours earlier at the 30 s target spacing. **Every figure therefore names the height it was taken at**, and heights, not wall-clock times, are the primary index. Second, the PoN transition quadrupled the block rate, so block-denominated durations carry a fork-aware 4× multiplier: a constant of 22,000 blocks corresponds to an effective 88,000 blocks after activation, and this paper gives the constant together with the multiplier rather than silently reporting one of the two.

Every number in this section is computed from a live query against four endpoints of the public `api.runonflux.io` API, retrieved on 2026-09-03: `/daemon/getzelnodecount` (fleet totals by tier and address family), `/daemon/listzelnodes` (the full per-node record set, from which tier collateral, node tenure, and the tier table below are recomputed directly from the records rather than trusted from the summary counts), `/apps/globalappsspecifications` (the full

set of currently registered application specifications), and `/apps/deploymentinformation` (the height-indexed pricing table and deployment limits currently enforced by FluxOS). No other data source is used in this section. The snapshot is reproducible: `scripts/05-network-analysis.py` issues the same four queries and recomputes every table and figure below from the raw response bodies, so a reader can re-run it against the live network and obtain a different, equally valid snapshot for a later date. Every figure, table, and number below carries its endpoint and the 2026-09-03 retrieval date explicitly, following the citation convention used throughout this paper (SHIPPED; [measured: api.runonflux.io, 2026-09-03]). This methodology is stated in full here because later evaluation (§26) reuses it without repeating it.

3.2 The fleet

At retrieval time the network comprised **6,489** FluxNodes, all reported as `stable` SHIPPED [measured: api.runonflux.io/daemon/getzelnodecount, 2026-09-03]. Of these, **2,322** report an IPv4 endpoint; **0** report IPv6, and **0** report an onion (Tor) endpoint SHIPPED [measured: api.runonflux.io/daemon/getzelnodecount, 2026-09-03]. Recomputing tier membership and collateral directly from the `/daemon/listzelnodes` record set (rather than trusting the summary counter) gives the following:

Tier	Nodes	Collateral per node (FLUX)	Tier collateral (FLUX)
Cumulus	3,247	1,000	3,247,000
Nimbus	1,615	12,500	20,187,500
Stratus	1,627	40,000	65,080,000
Total	**6,489**	—	**88,514,500**

Table 3: Fleet composition and locked collateral by tier, recomputed from per-node records SHIPPED [measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03].

What that fleet amounts to, and what it is. Two aggregates matter and they are not the same number. FluxOS declares each tier's allocation in `fluxSpecifics`, and records against each line, in the configuration's own comments, how much of it is available to applications after node overhead SHIPPED [flux/ZelBack/config/default.js:380–395]. Multiplying both by the census gives

	vCPU	**RAM**	**Storage**
Total allocation	51,940	166.4 TiB (170,426 GB)	2.86 PB (2,856,700 GB)
Usable by applications	45,451	153.8 TiB (157,448 GB)	2.60 PB (2,597,140 GB)

SHIPPED [doc: scripts/23-network-capacity.py]. The CPU figures count *threads*, not physical cores: `fluxSpecifics` states Cumulus as 40 tenths of a vCPU, and `fluxbench` states the same machine as 2 cores and 4 threads [fluxbench/src/fluxnode/benchmarks.h:58–84] — one machine counted two ways, and the smaller number is not the one FluxOS allocates against.

Both rows are floors rather than measurements. The tier line is what a node must provide to be confirmed, not a declaration of what it carries, and nothing in the protocol caps a node above its tier; the true capacity is larger by an amount this survey does not measure. Storage carries a third and stricter figure: the runtime admission check computes usable disk as $0.95 \times \text{total} - 60 - 20$ GB SHIPPED [flux/ZelBack/src/services/appRequirements/hwRequirements.js:385], which for a 220 GB Cumulus admits 129 GB rather than the 180 GB its tier line records, and across the fleet gives 2.19 PB — 77 % of the total allocation.

Decided. both figures are published, because they measure different things and neither substitutes for the other [intent: design intake]. The tier line states what the fleet was specified to provide; the runtime check states what a deployment can actually obtain. A reader sizing a workload wants the second; a reader assessing what the network was built to hold wants the first. The

23 % gap between them is a real property of the deployed configuration, reported rather than resolved away.

On that basis Flux is, to the author's knowledge, the **largest decentralized compute network by independently-operated nodes** [intent: design intake]. The paper states the measured basis rather than the bare superlative, and does not attempt a comparison table against other networks: no such measurement was made for this paper, and a claim of that shape is only worth as much as the census behind it. What is measured is the census above — 6,489 nodes, 88,514,500 FLUX of operator-posted collateral, every node independently operated and independently collateralised.

Not established by this survey. the exact all-time-high fleet size, though it is narrower than an open question. Secondary reports put the peak at over 14,000 nodes, with about 13,500 in September 2024 and over 10,000 in mid-2025 against the 6,489 measured here [intent: design intake]; these are marketing and review pages rather than primary sources and none is authoritative on its own. One independent check makes the 13,500 figure credible: the same reports state more than 107,000 CPU cores at that node count, and scaling this section's own per-tier method to 13,500 nodes gives 108,058 vCPU — a 1 % agreement, which corroborates both the reported size and the threads-not-cores basis used above. The exact number is recoverable without any statistics service, since FluxNode start and confirmation transactions are on-chain and the confirmed set at any past height can be reconstructed by replaying them; this paper states that method rather than running it.
The contraction is the more consequential fact. **The fleet stands at roughly half its peak**, which is why the scale claim above is made against the measured present and not against a historical high — and which bears directly on Section 7's open question about exit composition under a demand shortfall: the contraction that section reasons about in the abstract has already happened at scale, and whether the operators who left were disproportionately single-node holders is still unmeasured.

The total of **88,514,500** FLUX is the sum of collateral each operator has locked to run a node at that tier; it is not a measure of trading volume, market value, or any other quantity this paper's register excludes. Stratus nodes are the smallest population by count (1,627 of 6,489, or 25.1%) but hold the largest share of locked collateral (65,080,000 of 88,514,500 FLUX, or 73.5%) SHIPPED [measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03].

3.3 Node tenure

Measuring blocks elapsed since each node's `added_height` against the chain tip at retrieval time gives a median tenure of **193,620** blocks (approximately 67 days at the current 30-second target spacing), a 10th-percentile tenure of **12,909** blocks, a 90th-percentile tenure of **881,770** blocks, and a maximum of **1,648,700** blocks SHIPPED [measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03]. The distance between the 10th and 90th percentiles spans roughly two orders of magnitude, which is consistent with a fleet that is simultaneously accumulating a long tail of multi-year operators and continuing to admit new entrants rather than having converged to a stable, unchanging membership [inferred].

3.4 Deployed applications

The global application registry held **1,195** application specifications at retrieval time, comprising **1,371** components (compose entries) across **7,486** total requested instances SHIPPED [measured: api.runonflux.io/apps/globalappsspecifications, 2026-09-03]. Summed across all requested instances, the committed resource footprint specified by these applications is **8,972.0** vCPU-equivalents, **17,544,229** MB of RAM (17,133.0 GiB), and **160,107** GB of disk SHIPPED [measured: api.runonflux.io/apps/globalappsspecifications, 2026-09-03]. Instances requested per application have a median of **2**, a mean of **6.26**, a maximum of **100**, and a minimum of **0**; **439** applications (36.7%) sit at 3 instances, the floor for specifications below SPEC v8, while **599** SPEC v8 applications request one or two, which SPEC v8 has allowed since block 2,176,519. The minimum of 0 comes from

two SPEC v2 specifications that predate the `instances` field, which FluxOS treats as 3 and the census counts as zero SHIPPED [measured: api.runonflux.io/apps/globalappsspecifications, 2026-09-03].

These figures are *requested* resources as declared in each application's specification, not measured utilisation. No public endpoint reports realised CPU, memory, disk, or bandwidth utilisation on running containers across the fleet; this paper does not estimate one, and the absence is recorded as an open problem rather than papered over with an inferred figure (§29).

3.5 Specification version distribution

The 1,195 deployed applications are governed by seven distinct application specification versions, all live simultaneously at retrieval time:

Version	Applications	Share
SPEC v2	2	0.2%
SPEC v3	27	2.3%
SPEC v4	11	0.9%
SPEC v5	3	0.3%
SPEC v6	70	5.9%
SPEC v7	209	17.5%
SPEC v8	873	73.1%
Total	**1,195**	**100%**

Table 4: Distribution of deployed applications by application specification version SHIPPED [measured: api.runonflux.io/apps/globalappsspecifications, 2026-09-03].

Seven application specification versions are live simultaneously, spanning from SPEC v2 (registered years before SPEC v8 became the newest enforced version) to SPEC v8 itself. No version among them has ever been forced out of the running population by the introduction of a later one [inferred]— this reading follows from the observed coexistence rather than from a direct record of deprecation history, and it is stated hedged for that reason. §10 examines the version-gating mechanism that produces this coexistence, and the pending SPEC v9 redesign that would eventually extend it to eight.

3.6 Pricing as deployed

The rates FluxOS charges for deployment resources are set by a height-indexed table served from `/apps/deploymentinformation`. At retrieval time the table held six recorded price states: an initial schedule in force from height −1 (i.e., from genesis of the pricing mechanism) and five subsequent revisions.

Height	cpu	ram	hdd	minPrice	port	scope	staticip
−1	3.00	1.00	0.500	1.00	2.0	6.0	3.0
983,000	0.30	0.10	0.050	0.10	2.0	6.0	3.0
1,004,000	0.06	0.02	0.010	0.01	2.0	6.0	3.0
1,288,000	0.15	0.05	0.020	0.01	2.0	6.0	3.0
1,594,832	0.15	0.05	0.020	0.01	1.5	6.0	3.0
1,597,156	0.03	0.01	0.004	0.01	0.4	0.8	0.4

Table 5: Height-scheduled deployment pricing table, in FLUX per priced unit SHIPPED [measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03].

The rates in force at retrieval time are the row at height 1,597,156. Across the six recorded rows, the `cpu` rate has fallen from 3 to 0.03 FLUX per unit; this is stated as a fact about the schedule as recorded, with no comment on cost, value, or trend implied. Payments for

deployment and renewal are sent to `t3NryfAQLGeFs9jEoeqsxmBN2QLRaRKFLUX` SHIPPED [measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]. The same endpoint reports `minimumInstances` of **3**, `maximumInstances` of **100**, `blocksLasting` of **22,000** blocks, `maxBlocksAllowance` of **1,056,000** blocks, and `maxImageSize` of **5,000,000,000** bytes (5 GB) SHIPPED [measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03].

3.7 The multi-chain footprint

Beyond the main chain, Flux is represented today by a parallel asset on ten separate chains, each carrying its own contract address, mint, or module identifier:

Chain	Identifier	Provenance
Ethereum	`0x720cd16b011b987da3518fbf38c3071d4f0d1495` (ERC-20)	[holder-snapshot/index.js:22]
BNB Chain	`0xaff9084f2374585879e8b434c399e29e80cce635` (BEP-20)	[holder-snapshot/index.js:23]
Polygon	`0xA2bb7A68c46b53f6BbF6cC91C865Ae247A82E99B`	[holder-snapshot/index.js:26]
Avalanche C-Chain	`0xc4B06F17ECcB2215a5DBf042C672101Fc20daF55`	[holder-snapshot/index.js:24]
Base	`0xb008bdcf9cdff9da684a190941dc3dca8c2cdd44`	[flux-supply-tracker/index.html:346]
Solana	`FLUX1wa2GmbtSB6ZGi2pTNbVCw3zEeKnaPCkPtFXxqXe` (SPL mint)	[holder-snapshot/index.js:20]
Algorand	`1029804829` (ASA ID)	[holder-snapshot/index.js:27]
Ergo	`e8b20745ee9d18817305f32eb21015831a48f02d40980de6e849f886dca7f807` (token ID)	[holder-snapshot/index.js:25]
Tron	`TWr6yzukRwZ53HDe3bzcC8RCTbiKa4Zzb6` (TRC-20)	[holder-snapshot/index.js:21]
Kadena	`runonflux.flux` (Pact module)	[flux-supply-tracker/index.html:391]

Table 6: Chains carrying a Flux parallel asset, with identifiers as recorded in the team's own asset-inventory tooling SHIPPED.

These identifiers are as recorded in the codebases that track them (`holder-snapshot`, `flux-supply-tracker`); this survey did not independently re-verify each address against its chain's own explorer [inferred]. §6 takes this footprint as the starting inventory for an argument that consolidating it reduces audit and reserve-accounting surface.

3.8 The operational layer

Around FluxOS on every node sits a cluster of smaller, single-purpose daemons that this paper returns to in later sections: `flux-telemetryd` ships per-container metrics and logs to a backend the application owner — not Flux — chooses and credentials, never touching data FluxOS has not vouched for SHIPPED [flux-telemetryd/src/identity.rs:1–31,56–74] (§9); `flux-fstrimd` paces filesystem TRIM operations so that routine storage maintenance does not stall a node's I/O SHIPPED [flux-fstrimd/src/main.rs:1–6]; `flux-shutdownd` is building a coordinated, reason-aware graceful-shutdown and upgrade pipeline in place of an unceremonious container kill, though at present only its signal-and-cleanup stages run in production and its drain and pre-stop execution stages are implemented but not yet wired to real events IN-PROGRESS [flux-shutdownd/crates/shutdownd/src/pipeline/state_machine.rs:3–7]; and, operated centrally by InFlux rather than per-node, `appsmonitor` tracks on-chain application expiry and sends renewal notifications SHIPPED [appsmonitor/src/services/appsSubscriptionRenewalMonitor.js:112–368] (§16), `fluxstorage` holds oversized environment-variable and command payloads that do not fit an application specification SHIPPED [fluxstorage/src/routes.js:17–67] (§23), and `fluxos-release-api` serves the FluxOS/ArcaneOS upgrade bundles that nodes poll for SHIPPED [fluxos-release-api/src/flux_release_api/api.py:176–229] (§23). FluxOS's own placement and lifecycle behaviour that these services support is described in §9; the centrally-operated members of this list reappear in §12's inventory of components the decentralization roadmap must still address.

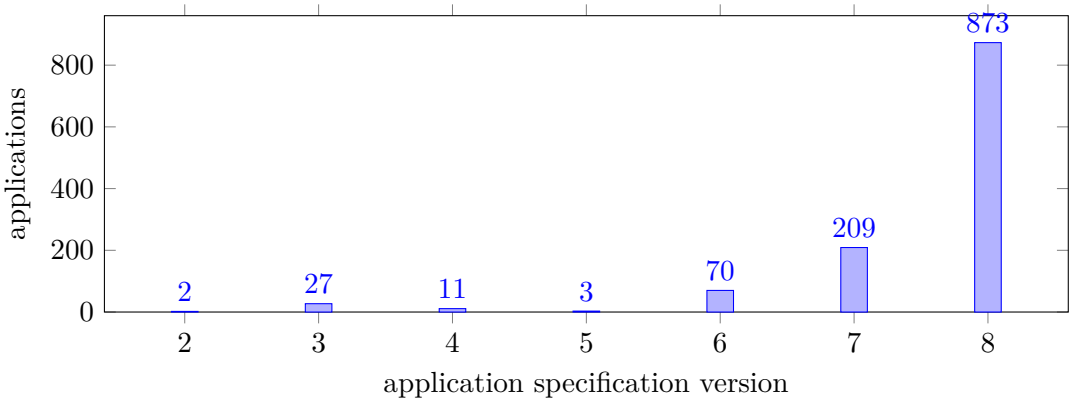


Figure 1: Number of deployed applications by application specification version, out of 1,195 total SHIPPED [measured: api.runonflux.io/apps/globalappsspecifications, 2026-09-03]. The distribution is heavily concentrated at SPEC v8 (873 of 1,195) while six older versions remain live.

3.9 The measured baseline

Three numbers from this section recur throughout the rest of this paper as the measured baseline against which every later architectural claim is checked: **6,489** nodes, **1,195** deployed applications, and **88,514,500** FLUX of locked collateral SHIPPED [measured: api.runonflux.io, 2026-09-03]. Neither the consensus mechanism of §4, nor the compensation design of §7, nor the target architecture of Part IV should be read as describing a network smaller or more homogeneous than this.

PART II

Chain and money

4 Consensus: from PoW to PoN, node tiering, benchmarking, and eligibility

For roughly the first two million blocks of its history, Flux's base-layer daemon, `fluxd`, produced blocks the way most Zcash-lineage chains do: miners raced to solve an Equihash puzzle, and the chain followed whichever fork carried the most accumulated hashing work. Since height 2,020,000 it does not. Block production is instead awarded, once every 30 seconds, to whichever registered FluxNode wins a per-slot lottery keyed by its collateral outpoint. Flux calls this mechanism Proof-of-Node (PoN). The name suggests that operating a node is what earns the right to produce a block. The mechanics below show something narrower: the right is earned by a node that has posted the correct collateral amount on-chain and that has, at some point, produced one signed message from its own locally-run benchmarking process. Whether that local, same-host message is a hard-to-fake attestation of real hardware or a token any operator's own machine can produce for itself is the question this section spends most of its length answering precisely, because the answer is not the one the name implies. The remainder of this section is formal.

4.1 The network-upgrade mechanism

Network upgrades are indexed by `Consensus::UpgradeIndex` SHIPPED [fluxd/src/consensus/params.h:24–40]. Upgrade state is not a persisted flag; it is a pure function of block height, recomputed on every call by `NetworkUpgradeState`: an upgrade is `UPGRADE_DISABLED` if its activation height equals the sentinel `NO_ACTIVATION_HEIGHT` (-1), `UPGRADE_PENDING` below its activation height, and `UPGRADE_ACTIVE` at or above it SHIPPED [fluxd/src/consensus/upgrades.cpp:72–97]. For PoN specifically this reduces to a single inequality: `IsPONActive(nHeight)` $\Leftrightarrow$ `nHeight` $\geq$ `GetPONActivationHeight()` SHIPPED [fluxd/src/pon/pon-fork.cpp:13–20,45–47]. There is no signalling, no miner- or node-voting window, and no grace period at any upgrade boundary, PoN included: activation is a hard height cutoff.

Table 7: Mainnet network-upgrade heights, `CMainParams::CMainParams()`. Hashes are truncated to 16 hex characters for display; full values are at the cited lines.

Upgrade	Height	Activation block hash (truncated)	Provenance
`UPGRADE_LWMA`	125,000	—	[fluxd/src/chainparams.cpp:116–117]
`UPGRADE_EQUI144_5`	125,100	—	[fluxd/src/chainparams.cpp:119–120]
`UPGRADE_ACADIA`	250,000	`0000001d65fa78f2...`	[fluxd/src/chainparams.cpp:122–125]
`UPGRADE_KAMIOOKA`	372,500	`00000052e2ac144c...`	[fluxd/src/chainparams.cpp:127–130]
`UPGRADE_KAMATA`	558,000	`000000a33d38f37f...`	[fluxd/src/chainparams.cpp:132–135]
`UPGRADE_FLUX`	835,554	`000000ce99aa6765...`	[fluxd/src/chainparams.cpp:137–140]
`UPGRADE_HALVING`	1,076,532	`000000111f8643ce...`	[fluxd/src/chainparams.cpp:142–145]
`UPGRADE_P2SHNODES`	1,549,500	`00000009f9178347...`	[fluxd/src/chainparams.cpp:147–150]
`UPGRADE_PON`	**2,020,000**	*unset*	[fluxd/src/chainparams.cpp:152–153]

`BASE_SPROUT` (always active from genesis) and `UPGRADE_TESTDUMMY` (never activated on mainnet) are omitted from Table 7 because neither carries a meaningful mainnet height SHIPPED [fluxd/src/chainparams.cpp:108–114].

Height 2,020,000 for `UPGRADE_PON` is a code fact SHIPPED [fluxd/src/chainparams.cpp:152–153]. The calendar date sometimes attached to this activation is not: `UPGRADE_PON` is the only upgrade in Table 7 with no `hashActivationBlock` set, and no timestamp literal tying height 2,020,000 to a wall-clock date appears anywhere in the source SHIPPED [fluxd/src/chainparams.cpp:152–153]. The nearest bracketing evidence is a checkpoint at height 2,022,000 carrying UNIX timestamp `1761701944`, which is 2025-10-29 SHIPPED [fluxd/src/chainparams.cpp:281,284]; two thousand blocks at 30 s spacing is approximately 16.7 hours, so this brackets but does not establish an activation

date. [measured: block 2,020,000 carries timestamp 1,761,415,235, i.e. 2025-10-25T18:00:35Z, retrieved from `api.runonflux.io/daemon/getblock`; the date is therefore a chain fact rather than an inference from a nearby height, 2026-09-04]

4.2 What changes at PoN activation

- **Subsidy.** `GetBlockSubsidy` branches first on `IsPONActive(nHeight)`, bypassing the legacy slow-start/halving formula entirely for PoN heights SHIPPED [fluxd/src/main.cpp:2796–2853]. For $h \geq h_{PoN}$,

$$\sigma_{PoN}(h) \;=\; 14\,\mathrm{FLUX} \cdot \gamma^{\,n(h)}, \qquad n(h) \;=\; \min\Big(\Big\lfloor \frac{h - h_{PoN}}{I_{\mathrm{red}}} \Big\rfloor,\; N_{\mathrm{red}} \Big),$$

  with $\gamma = 9/10$ applied once per elapsed reduction interval, $I_{\mathrm{red}} = 1{,}051{,}200$ blocks mainnet, and a cap of $N_{\mathrm{red}} = 20$ reductions SHIPPED [fluxd/src/main.cpp:2812], [fluxd/src/chainparams.cpp:156–158]. The full emission derivation, including the tail after N_{red} is reached, is given in §5.

- **Difficulty retarget.** `GetNextWorkRequiredByFork` dispatches to `GetNextPONWorkRequired` for PoN heights and to the legacy staged Digishield/LWMA/LWMA3 retarget otherwise SHIPPED [fluxd/src/pon/pon-fork.cpp:22–42], [fluxd/src/pow.cpp:21–97].

- **Block header format.** Block version $\geq$ `PON_VERSION` $= 100$ SHIPPED [fluxd/src/primitives/block.h:31] adds two consensus fields: `nodesCollateral` (the `COutPoint` of the winning FluxNode's collateral) and `vchBlockSig` (an ECDSA signature over the block hash) SHIPPED [fluxd/src/primitives/block.h:44–45,64–96]. `IsPON()`/`IsPOW()` are simply `nVersion` $\geq/<$ 100 SHIPPED [fluxd/src/primitives/block.h:95–96].

- **Block spacing.** Legacy target spacing $\Delta_{PoW} = 120\,\mathrm{s}$ SHIPPED [fluxd/src/chainparams.cpp:101] drops to $\Delta_{PoN} = 30\,\mathrm{s}$ SHIPPED [fluxd/src/chainparams.cpp:105].

- **Header validation.** `CheckBlockHeader` replaces the Equihash-solution check and `CheckProofOfWork` with `CheckProofOfNode` plus a signature-size cap SHIPPED [fluxd/src/main.cpp:5330–5378]. `ContextualCheckBlockHeader` additionally requires `block.IsPON()` to equal `IsPONActive(nHeight)` exactly, rejecting a mismatch as `"not-pon-block"` or `"premature-pon-block"`, and requires a PoN block's timestamp to be strictly greater than its parent's timestamp rather than median-time-past-based as the legacy path is SHIPPED [fluxd/src/main.cpp:5516–5545,5564–5572].

- **Future-time tolerance.** Accepted clock skew for a new block's timestamp drops from 2 hours (pre-LWMA) / 360 s (post-LWMA PoW) to 300 s for PoN blocks SHIPPED [fluxd/src/main.cpp:5330–5378].

- **Coinbase dev-fund rule.** Once PoN is active, the coinbase must pay a minimum amount to a hardcoded address or the transaction is rejected. That minimum is a remainder:

$$\delta(h) \;=\; \sigma_{PoN}(h) \;-\; \sum_{x \in \mathcal{T}} \beta_x \cdot \frac{\sigma_{PoN}(h)}{14\,\mathrm{FLUX}},$$

  with $\beta_{\mathsf{C}} = 1\,\mathrm{FLUX}$, $\beta_{\mathsf{N}} = 3.5\,\mathrm{FLUX}$, $\beta_{\mathsf{S}} = 9\,\mathrm{FLUX}$ SHIPPED [fluxd/src/main.cpp:2892–2895], which sum to 13.5 FLUX of the 14 FLUX initial subsidy, leaving $\delta(h_{PoN}) = 0.5\,\mathrm{FLUX}$ at the first post-activation block, decaying at the same rate as σ_{PoN} thereafter SHIPPED [fluxd/src/main.cpp:2877–2884]. This is enforced twice: loosely (a $\geq$ check) in `ContextualCheckTransaction`, which rejects a coinbase missing it as `"tx-coinbase-missing-dev-funding"` SHIPPED [fluxd/src/main.cpp:1231–1249], and again — also as a $\geq$ check, but against the remainder left after the exact tier payouts — in `FluxnodeCache::CheckFluxnodePayout` SHIPPED [fluxd/src/fluxnode/fluxnode.cpp:822–889]. The recipient is the hardcoded mainnet address `t3hPu1YDeGUCp8m7BQCnnNUmRMJBa5RadyA` SHIPPED [fluxd/src/chainparams.cpp:306].

4.3 PoN sortition, formally

Definition 4.1 (PoN slot and eligibility)**.** Let t_0 be network genesis time. The PoN slot at wall-clock time t is

$$\tau(t) \;:=\; \left\lfloor \frac{t - t_0}{\Delta_{PoN}} \right\rfloor, \qquad \Delta_{PoN} = 30\,\mathrm{s}$$

SHIPPED [fluxd/src/pon/pon.cpp:88–93]. For a confirmed FluxNode with collateral outpoint c, define

$$H(c, \tau) \;:=\; \mathrm{Hash}\big(c \,\|\, h_{\mathrm{prev}} \,\|\, \tau\big),$$

where h_{prev} is the hash of the current chain tip SHIPPED [fluxd/src/pon/pon.cpp:70–86]. Node c is *eligible* for slot τ iff $\mathbf{1}[H(c,\tau) \le T_\tau] = 1$, where T_τ is the current PoN difficulty target (below). Every confirmed FluxNode, of any tier, computes exactly one such hash per slot; eligibility does not depend on tier or on collateral size beyond confirmation SHIPPED [fluxd/src/pon/pon.cpp:400–433]. This is sortition over the confirmed-node set, not a lottery weighted by stake amount.

The target T_τ comes from `GetNextPONWorkRequired`. Before PoN activation it is unused; for the first `nPonDifficultyWindow` = 30 blocks (mainnet) after activation it is held fixed at `ponStartLimit`, which the source's own comment describes as giving each node roughly a 1-in-10,000 chance of eligibility per slot SHIPPED [fluxd/src/chainparams.cpp:104]. Thereafter every block retargets: the actual timespan over the trailing 30-block window is clamped to $\left[\frac{4}{5}, \frac{5}{4}\right] \times (30-1)\Delta_{PoN}$ (i.e. $-20\%/+25\%$) and used, Digishield-style, to scale the previous target SHIPPED [fluxd/src/pon/pon.cpp:149–157]. The protocol floor is `ponLimit`, described in the same source comments as giving roughly a 1-in-16 minimum per-slot eligibility chance, mainnet SHIPPED [fluxd/src/chainparams.cpp:103].

Algorithm 1: PoN minting loop, per node (one thread, wall-clock polled)

Input: collateral outpoint c, FluxNode private key sk, chain tip pointer tip
Output: a signed candidate block submitted to the network, or no action this slot

```
wait for network peers and end of initial block download   // [fluxd/src/pon/pon-minter.cpp:121–136]
while node is running do
    if PoN is active and this node's own confirmation status is confirmed then
        t ← current wall-clock time
        τ ← τ(t) (Definition 4.1)   // [fluxd/src/pon/pon-minter.cpp:167–177]
        if τ ≠ lastAttemptedSlot and t − tip.nTime ≥ 5 s then
            compute eligibility and rank of c over all confirmed nodes for slot τ   // [fluxd/src/pon/pon-minter.cpp:222–236]
            if c eligible for τ then
                sleep rank(c) × 4000 ms   // orphan-avoidance heuristic only -- no consensus enforcement, [fluxd/src/pon/pon-minter.cpp:78–104,217–221,250]
                if τ has not expired and tip has not moved then
                    build candidate block, coinbase paying one node per tier and the dev-fund remainder   // [fluxd/src/pon/pon-minter.cpp:288–314]
                    sign block hash with sk via SignPONBlock
                    submit via ProcessNewBlock   // [fluxd/src/pon/pon-minter.cpp:29–76]
            mark τ as lastAttemptedSlot
    sleep 1 s (poll interval)   // [fluxd/src/pon/pon-minter.cpp:175]
```

Invariant. A node attempts at most one block per slot: `lastAttemptedSlot` is checked before eligibility is computed and set on every exit from the slot, so a repeated poll of the same slot is a no-op SHIPPED [fluxd/src/pon/pon-minter.cpp:167–177].

Failure mode. Only `std::runtime_error` is caught at the top of the loop, and on catching it the thread logs once and returns, with no restart logic SHIPPED [fluxd/src/pon/pon-minter.cpp:343–346]; any other exception type escapes the bare `std::thread` body [fluxd/src/pon/pon-minter.cpp:356] and, under C++ semantics, calls `std::terminate`, aborting the whole daemon rather than only the minting thread.

The rank-based delay in the algorithm's inner branch is worth stating plainly, because "Proof of Node" invites a reading in which rank determines who is allowed to produce a block. It does not. The $\mathrm{rank}(c) \times 4000\,\mathrm{ms}$ wait is documented in the source itself as an orphan-reduction heuristic: a lower-ranked (slower) eligible node waits longer before broadcasting, so that a higher-ranked node's block has time to propagate and preempt it. No peer enforces this delay; any validly-signed block for an eligible node's slot is accepted on ordinary first-seen/most-work rules regardless of the sender's rank SHIPPED [fluxd/src/pon/pon-minter.cpp:78–104,217–221].

Consensus does not pin the slot a block claims to wall-clock time either: the slot is derived from the block's own `nTime` SHIPPED [fluxd/src/pon/pon.cpp:63–67], and the only rules on `nTime` are that it exceed the parent's [fluxd/src/main.cpp:5564–5567] and lie at most 300 s ahead of the validator's adjusted time [fluxd/src/main.cpp:5368–5371]. A producer may therefore evaluate $H(c, \tau)$ over every slot in that window — about eleven at 30 s spacing — and publish at once for any slot in which one of its nodes qualifies, whereas the minting loop of Algorithm 1 tries only the current slot.

4.4 Fork choice

Property 4.2 (PoN chain-work is a block count)**.** For every block with nVersion ≥ 100 (every PoN block), `GetBlockProof` returns the fixed value 2^{40}, independent of the block's actual difficulty target SHIPPED [fluxd/src/pow.cpp:284–290]. Consequently, for two competing PoN chains of length n_1 and n_2 blocks since their most recent common ancestor, cumulative chain work is exactly $n_1 \cdot 2^{40}$ versus $n_2 \cdot 2^{40}$: "most cumulative work" fork choice is equivalent to "most blocks," and among chains of equal length, the deciding factor is which block a node saw and connected first, not any comparison of cryptographic hardness SHIPPED [fluxd/src/pow.cpp:284–290].

Proof sketch. Chain work under the inherited Bitcoin/Zcash chain-selection code is the sum of `GetBlockProof` over every block on a candidate chain since the fork point. `GetBlockProof` is defined piecewise on `nVersion`; for nVersion ≥ 100 it returns the constant `arith_uint256(1)` $\ll 40$ regardless of `nBits` SHIPPED [fluxd/src/pow.cpp:284–290]. Summing a constant n times over a chain of n PoN blocks gives $n \cdot 2^{40}$ exactly, with no dependence on the per-block difficulty target computed in §4.3. Comparing $n_1 \cdot 2^{40}$ against $n_2 \cdot 2^{40}$ is therefore equivalent to comparing n_1 against n_2; the constant factor cancels. When $n_1 = n_2$, the comparison is a tie under this metric, and the surviving chain is whichever block the node's own chain-selection logic connected first, an ordering determined by network propagation and arrival time rather than by proof of work. □

The consequence is not a defect to be patched quietly; it is a different security argument than the one "proof of work" trains a reader to expect. PoN's resistance to a chain reorganization does not come from the cost of recomputing accumulated hashing work. It comes from the cost of controlling enough collateral to win sortition (Definition 4.1) across enough consecutive slots to outpace and outlast the honest chain, and from how quickly an alternate chain of blocks can be produced and propagated — a length race, not a work race. A reader who assumes Nakamoto-style work-accumulation will draw the wrong conclusion about what a reorg of a given depth costs an attacker on PoN.

Remark 4.3. `nMinimumChainWork`, the anti-eclipse floor consulted during initial block download, was last updated to match the chain work associated with `UPGRADE_KAMIOOKA` SHIPPED [fluxd/src/chainparams.cpp:168–170]; it has not been revised for the fixed-2^{40} -per-block regime that PoN introduced. Whether a floor computed under the old work-accumulation model still functions as

intended against a fake alternate chain once every block after height 2,020,000 contributes the same fixed increment is not established by the source [inferred].

4.5 The FluxNode lifecycle

A deterministic FluxNode moves through the states `FLUXNODE_TX_ERROR`(0) → `FLUXNODE_TX_STARTED`(1) → `FLUXNODE_TX_DOS_PROTECTION`(2) / `FLUXNODE_TX_CONFIRMED`(3) → `FLUXNODE_TX_MISS_CONFIRMED`(4) / `FLUXNODE_TX_EXPIRED`(5) SHIPPED [fluxd/src/fluxnode/fluxnode.h:109–116].

- **Entry (START).** The operator broadcasts a `FLUXNODE_START_TX_TYPE` transaction spending a collateral UTXO of a valid tier amount, signed by the collateral key SHIPPED [fluxd/src/fluxnode/activefluxnode.cpp:253–288]. Consensus requires the collateral to have ≥ 100 confirmations (`FLUXNODE_MIN_CONFIRMATION_DETERMINISTIC`) SHIPPED [fluxd/src/fluxnode/fluxnode.h:20] and to match a tier's valid amount SHIPPED [fluxd/src/coins.cpp:630–686]. The node enters `mapStartTxTracker` SHIPPED [fluxd/src/fluxnode/fluxnode.cpp:304–352].
- **STARTED → DOS_PROTECTION.** If no confirm transaction arrives within `FLUXNODE_START_TX_EXPIRATION_HEIGHT` = 60 blocks (240 post-PoN, `_V2`) of the start height, the node moves to `mapStartTxDOSTracker` SHIPPED [fluxd/src/fluxnode/fluxnode.cpp:479–508]; it is fully purged once `FLUXNODE_DOS_REMOVE_AMOUNT` = 180 blocks (720 post-PoN, `_V2`) have elapsed since the *start* height, i.e. 120 (480 post-PoN) blocks after entering the DoS list SHIPPED [fluxd/src/fluxnode/fluxnode.cpp:205–229], [fluxd/src/fluxnode/fluxnode.h:26–28].
- **STARTED → CONFIRMED.** The operator builds a `FLUXNODE_CONFIRM_TX_TYPE` (`INITIAL_CONFIRM`), which is benchmark-signed by `fluxbench` before wallet broadcast (see §4.7) SHIPPED [fluxd/src/fluxnode/activefluxnode.cpp:380–407,18–74]. On chain acceptance the node enters `mapConfirmedFluxnodeData` and is inserted at the *front* of its tier's payment list, which is then re-sorted SHIPPED [fluxd/src/fluxnode/fluxnode.cpp:1502–1511,1466–1474].
- **Re-confirmation (CONFIRMED, periodic).** `CheckIfNeedsNextConfirm` triggers a `UPDATE_CONFIRM` once $h - $ `nLastConfirmedBlockHeight` exceeds `FLUXNODE_CONFIRM_UPDATE_MIN_HEIGHT_V1` = 40 / `_V2` = 120 / `_V3` = 500 (PoN), by active upgrade SHIPPED [fluxd/src/fluxnode/fluxnode.cpp:669–688].
- **CONFIRMED → EXPIRED.** If no re-confirmation lands within `FLUXNODE_CONFIRM_UPDATE_EXPIRATION_HEIGHT_V1` = 60 / `_V2` = 80 / `_V3` = 160 / `_V4` = 640 (PoN) blocks of `nLastConfirmedBlockHeight`, the node is removed from the paid set SHIPPED [fluxd/src/fluxnode/fluxnode.cpp:1716–1732], [fluxd/src/fluxnode/fluxnode.h:31–34].
- **CONFIRMED → removed (collateral spent).** If the confirmation collateral UTXO is spent, the node is removed SHIPPED [fluxd/src/fluxnode/fluxnode.cpp:285–290].
- **CONFIRMED → removed (collateral migration mismatch).** During the V1→V2 collateral-amount transition window (§4.6), a confirmed node whose collateral no longer matches the *new* valid amount for its tier is removed once that tier's transition-end height is reached SHIPPED [fluxd/src/fluxnode/fluxnode.cpp:231–283].

Payment queue ordering. `FluxnodeListData::operator<` orders each tier's list by `nLastPaidHeight` (or `nConfirmedBlockHeight` if never paid) ascending, tie-broken by whether `nLastPaidHeight` is set and then by outpoint SHIPPED [fluxd/src/fluxnode/fluxnode.h:313–328] — a strict longest-time-since-last-paid FIFO per tier. `GetNextPayment` reads the front of the tier's list, skipping and popping any stale (unconfirmed) entries it finds there SHIPPED [fluxd/src/fluxnode/fluxnode.cpp:713–778].

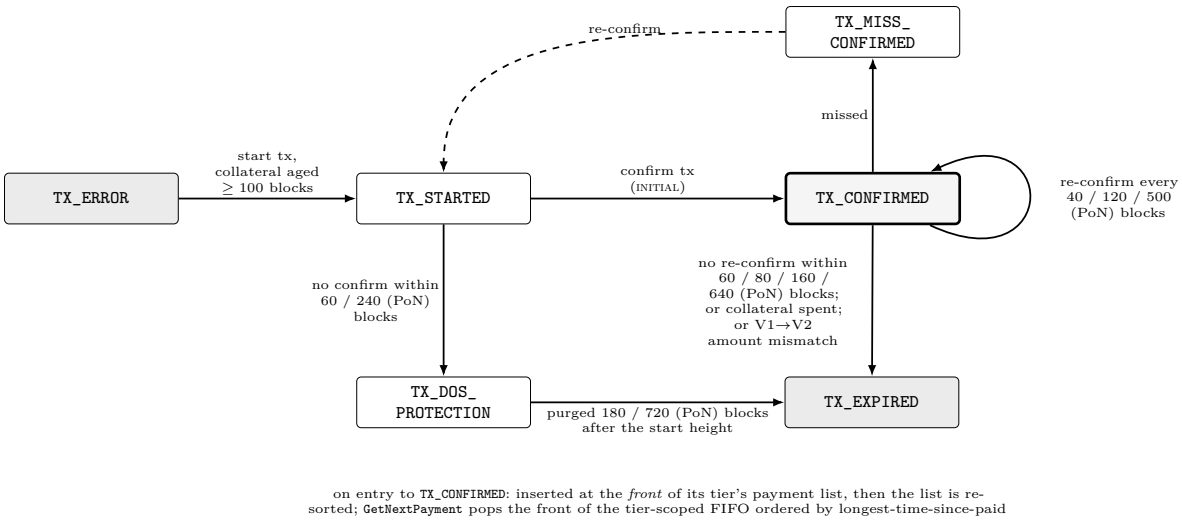


Figure 2: The FluxNode lifecycle state machine: states, transitions, and their governing height constants SHIPPED [fluxd/src/fluxnode/fluxnode.h:109–116]. Where two or more values are shown, they are the successive versions of the constant selected by the active network upgrade, the last being the PoN-era value [fluxd/src/fluxnode/fluxnode.h:23–44]. Only `TX_CONFIRMED` is in the paid set.

4.6 Tiers and collateral

Table 8: FluxNode collateral by tier and version, mainnet. Confirmation requirement is uniform across tiers and versions.

Tier	V1 collateral	V2 collateral	Transition window $[h_{\text{start}}, h_{\text{end}})$	Confirmation
Cumulus (C)	$c_{\mathsf{C}} = 10{,}000$ FLUX	1,000 FLUX	[1,076,532, 1,086,612)	100 blocks
Nimbus (N)	$c_{\mathsf{N}} = 25{,}000$ FLUX	12,500 FLUX	[1,081,572, 1,092,372)	100 blocks
Stratus (S)	$c_{\mathsf{S}} = 100{,}000$ FLUX	40,000 FLUX	[1,087,332, 1,097,412)	100 blocks

SHIPPED [fluxd/src/fluxnode/fluxnode.h:57–63] (V1/V2 amounts), [fluxd/src/chainparams.cpp:308–315] (transition windows), [fluxd/src/fluxnode/fluxnode.h:20] (100-block confirmation requirement, `FLUXNODE_MIN_CONFIRMATION_DETERMINISTIC`). Within a transition window, both the old and new collateral amounts for that tier are accepted SHIPPED [fluxd/src/coins.cpp:774–809]. The V2 amounts match the collateral recorded for every one of the 6,489 currently registered FluxNodes in the public registry [measured: api.runonflux.io/daemon/getzelnodecount, 2026-09-03], i.e. the migration is complete on mainnet as measured.

4.7 Benchmarking and eligibility

Confirming or re-confirming a FluxNode requires a benchmark-signed transaction produced by `fluxbench`, a companion daemon that runs on the same host as `fluxd`. `fluxbench` measures CPU throughput with `sysbench cpu --cpu-max-prime=60000` SHIPPED [fluxbench/src/fluxnode/benchmarks.cpp:2066–2364]; disk write speed with three `dd` runs per target, dropping the worst and averaging the remaining two, floored at 40 MB/s for reporting SHIPPED [fluxbench/src/fluxnode/benchmarks.h:602–615,1020–1104], [fluxbench/src/fluxnode/benchmarks.cpp:1936]; bandwidth with the Ookla `speedtest` CLI, combined with an `iftop` sample of bandwidth already in use SHIPPED [fluxbench/src/fluxnode/benchmarks.cpp:1358–1465,1980–2032]; and RAM headroom with a `stress-ng` allocation test sized 8G/4G/2G for Stratus/Nimbus/Cumulus SHIPPED [fluxbench/src/fluxnode/benchmarks.cpp:3649–3667].

Table 9: Per-tier hardware thresholds enforced by `ValidateBenchmarkRequirements`, mainnet.

Tier	Cores	Threads	RAM	Storage	EPS	Disk write	Up/Down
Cumulus	2	4	7 GB	220 GB	240	180 MB/s	25/25 Mbit/s
Nimbus	4	8	30 GB	440 GB	640	180 MB/s	50/50 Mbit/s
Stratus	8	16	61 GB	880 GB	1,520	400 MB/s	100/100 Mbit/s

SHIPPED [fluxbench/src/fluxnode/benchmarks.h:58–84]. Cumulus's RAM requirement is 7 GB on standard hardware and 3 GB on Jetson boards specifically SHIPPED [fluxbench/src/fluxnode/benchmarks.h:58–66]; measured bandwidth additionally receives an additive tolerance bonus before the threshold check of +2/+4/+9 Mbit/s for Cumulus/Nimbus/Stratus SHIPPED [fluxbench/src/fluxnode/benchmarks.h:86–88].

The result is signed by `BenchmarkSign`, which hashes the transaction's `sig`, `benchmarkTier`, `benchmarkSigTime`, and `ip` fields (`benchmarkTier` is the tier whose hardware minimum the benchmark checked; it does not set the node's tier, which is fixed by collateral, and consensus rejects a confirmation whose `benchmarkTier` is below the collateral tier [fluxd/src/main.cpp:1590–1596]) and signs with the fluxnode key selected for the current time window SHIPPED [fluxbench/src/fluxnode/benchmarks.cpp:3439–3480]. The same verification routine, `CheckBenchmarkSignature`, is linked into `fluxd` and run as part of consensus validation of the registration transaction SHIPPED [fluxbench/src/fluxnode/benchmarks.cpp:3511–3522]. Three further mechanisms can substitute a stored or historical value for a fresh measurement under specified conditions — an enterprise-app short-circuit, a FluxOS-reported fallback, and a single-thread-extrapolation fallback for the CPU test — and all three converge on the same `ValidateBenchmarkRequirements()` acceptance gate SHIPPED [fluxbench/src/fluxnode/benchmarks.cpp:2178–2234,771–783,2883–3222].

The property that does not hold. On the `fluxd` side, this same signature is verified against `vecBenchmarkingPublicKeys` — five keys, each with a UNIX-timestamp validity window, i.e. a rotating allowlist SHIPPED [fluxd/src/chainparams.cpp:233–240]. These keys are shared across the entire fleet: they identify "signed by something holding one of the five network-wide benchmarking keys," not "signed by this specific operator's own key." `fluxd` obtains the benchmark result itself by shelling out — `popen()`-ing the local `fluxbench-cli` process — and treats a returned `status:"complete"` field as sufficient to sign and broadcast the registration transaction SHIPPED [fluxd/src/fluxnode/benchmarks.cpp:74–102,227–291]. The registration transaction must still match a valid on-chain collateral amount for the claimed tier SHIPPED [fluxd/src/coins.cpp:630–686].

Put precisely: collateral binds the tier; the benchmark signature does not bind the hardware to the tier it claims. An operator cannot register a tier they have not collateralized, but nothing beyond a local, same-host process attests that the collateralized hardware in fact clears that tier's CPU/RAM /disk/bandwidth thresholds in Table 9. **The correct claim is benchmark-gating bypass, not tier forgery.** §14 takes up the structurally identical problem one layer up, for ArcaneOS/SAS attestation, and presents the planned bonded attestation network — where a bond is put at stake rather than a shared secret being checked — as the mechanism intended to fix exactly this property PLANNED [roadmap: timeline: new benchmark tool, signed node attestation].

4.8 The emergency-block path

A second, parallel block-authorization path exists outside PoN sortition entirely, intended for chain-halt recovery. A block whose `nodesCollateral` outpoint hash equals a fixed marker value and whose index is 0 is treated as an *emergency block* SHIPPED [fluxd/src/emergencyblock.cpp:24–30]. Such a block requires at least `nEmergencyMinSignatures` = 2 valid ECDSA signatures over `block.GetHash()`, drawn from a fixed set of 4 hardcoded public keys — a 2-of-4 multisignature, on mainnet and testnet alike SHIPPED [fluxd/src/chainparams.cpp:242–253]. Accepting an emergency block bypasses `CheckProofOfNode` entirely SHIPPED [fluxd/src/pon/pon.cpp:205–207] and bypasses

the per-node signature verification that an ordinary PoN block requires SHIPPED [fluxd/src/pon/pon.cpp:236–252].

The only gate on *when* an emergency block may be produced is `IsEmergencyBlockAllowed(nHeight, nTime)`, which checks nothing beyond `IsPONActive(nHeight)` — there is no time-since-last-block check and no stall-detection logic SHIPPED [fluxd/src/emergencyblock.cpp:183–191] — despite a doc-comment on the type stating "Can add additional restrictions like time delays" SHIPPED [fluxd/src/emergencyblock.h:41–44]. Concretely: any two holders of the four emergency keys can mint a fully valid block at any height once PoN is active, at any time, with no on-chain evidence that the network was in fact stalled. This is a consensus-level centralization surface, and it is stated here as one, not as a caveat.

The team does not intend to change this threshold: `nEmergencyMinSignatures` stays **2** against the current four-key set on mainnet and testnet SHIPPED [fluxd/src/chainparams.cpp:242–253]. A higher threshold would not change the shape of the disclosure just made in any case: a key set of any size on this path produces blocks outside PoN eligibility and outside the per-node signature check, with no time or stall-detection gating; only the number of colluding or compromised keys required would differ. The absence of any temporal or liveness constraint on *when* an emergency block may be produced — the more consequential fact of the two — is untouched by this parameter.

4.9 What activation actually cost

Activating PoN required two emergency interventions visible directly in the code. `MAX_REORG_LENGTH`, normally capped at 40 blocks SHIPPED [fluxd/src/main.h:74], was overridden to 5,000 for heights [2,020,000, 2,025,000], commented "Temporary deep reorg limit during PON stabilization period" SHIPPED [fluxd/src/main.cpp:8983–8988]. A checkpoint cluster at heights 2,020,500; 2,021,000; 2,021,500; 2,022,000; and 2,029,000 was added, labelled "PON activation and stabilization checkpoints - EMERGENCY FORK RECOVERY" SHIPPED [fluxd/src/chainparams.cpp:277–283]. §27 returns to this as evidence for what a consensus-level change actually requires of this network.

4.10 Measured rotation

Under the front-of-list, longest-time-since-paid ordering described in §4.5, and given that each block pays one node from every tier, the expected number of blocks between successive payments to a node in tier x is $R_x := n_x$, the tier's live node count. Table 10 compares this expectation against the maximum observed gap in the public registry, chain tip height 2,917,115 [measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03].

Table 10: Measured PoN payment rotation vs. the deterministic per-tier model.

Tier	Nodes (n.)	Expected rotation (R., blocks)	Max blocks since paid	Expected duration at Δ_{PoN}
Cumulus	3,247	3,247	3,202	27.06 h
Nimbus	1,615	1,615	1,597	13.46 h
Stratus	1,627	1,627	1,631	13.56 h

[measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03]. Stratus's observed maximum exceeds its own node count by 4 blocks; this is consistent with the stale-entry skip loop in `GetNextPayment` (§4.5) rather than a violation of the FIFO ordering rule. This measured rotation is the empirical confirmation on which §7's fairness proposition for PNR depends: that proposition rests on the determinism of the PoN *payment* rotation, not on placement determinism, and the two must not be conflated.

5 Monetary policy as deployed

Flux issues its native asset on a schedule fixed once, in `fluxd`'s consensus code, and not adjusted by discretion afterward. Two different mechanisms have produced that schedule over the chain's life: a Bitcoin-style halving curve during the proof-of-work era, and, since block height 2,020,000, a fixed initial block reward that decays by a constant fraction once a year under Proof of Node. Neither curve reaches zero on any realistic horizon, and at least one of them was adjusted by hand along the way rather than left to run the formula a reader might expect. This section states the exact, currently-enforced arithmetic for both eras in the notation of notation.tex, cross-checks the resulting figures against the network's own public reconciliation of issued supply, and states plainly what the schedule implies for the long run: issuance never stops, and only its rate settles to a small constant. Every subsequent economic claim in this paper is denominated against the numbers derived here.

All amounts below are computed as integers over satoshi, $\mathrm{COIN} = 100{,}000{,}000\,\mathrm{sat}$ per FLUX SHIPPED [fluxd/src/amount.h:17], matching `fluxd`'s `CAmount` (`int64`) representation. "$\lfloor\cdot\rfloor$" denotes truncating (floor) integer division throughout, exactly as C++ integer division behaves for non-negative operands [doc: flux-roadmap/supply/emission_model.py:196-200, citing fluxd]. This distinction is not cosmetic: the PoN reduction below is an r-fold sequence of *independent* truncations, not a single closed-form power, and the two are not interchangeable.

5.1 The subsidy function

Definition 5.1 (Legacy proof-of-work subsidy)**.** For $0 \le h < h_{PoN}$,

$$\sigma_{PoW}(h) \;:=\; \begin{cases} 0 & h = 0 \\ \Pi = 13{,}020{,}000\,\mathrm{FLUX} & h = 1 \\ \left\lfloor \frac{150\,\mathrm{COIN}}{5000} \right\rfloor \cdot h & 2 \le h < 2500 \\ \left\lfloor \frac{150\,\mathrm{COIN}}{5000} \right\rfloor \cdot (h+1) & 2500 \le h < 5000 \\ \left\lfloor \dfrac{150\,\mathrm{COIN}}{2^{\min\left(\left\lfloor \frac{h-2500}{655{,}350} \right\rfloor,\, 2\right)}} \right\rfloor & 5000 \le h < h_{PoN} \end{cases}$$

SHIPPED [fluxd/src/main.cpp:2818-2852], constants [fluxd/src/chainparams.cpp:90-91]. This is the literal transcription of `GetBlockSubsidy`'s legacy branch as currently compiled SHIPPED [flux-roadmap/supply/emission_model.py:207-218]. The chain's actual historical payout for heights 1–4,999 differs from this formula by one block index (an older binary paid the premine at height 2, not 1, and shifted the slow-start ramp accordingly) SHIPPED [flux-roadmap/supply/emission_model.py:220-229]; that discrepancy is the source of the persistent offset discussed in §5.7 below. The code's separate halvings $\ge 64 \Rightarrow 0$ branch is never reached before h_{PoN} and so never fires on the deployed chain SHIPPED [fluxd/src/main.cpp:2839-2840].

Definition 5.2 (Proof-of-Node subsidy)**.** For $h \ge h_{PoN}$, let

$$r(h) \;:=\; \min\left(\left\lfloor \frac{h - h_{PoN}}{I_{\mathrm{red}}} \right\rfloor,\; N_{\mathrm{red}} \right), \qquad s_0 := 14 \cdot \mathrm{COIN} = 1{,}400{,}000{,}000\,\mathrm{sat},$$

$$s_{k+1} := \lfloor s_k \cdot \gamma \rfloor, \quad \gamma = \frac{9}{10},$$

for $k = 0, \ldots, N_{\mathrm{red}} - 1$. Then

$$\sigma_{PoN}(h) \;:=\; s_{r(h)}.$$

SHIPPED [fluxd/src/main.cpp:2799-2816]; $h_{PoN} = 2{,}020{,}000$ SHIPPED [fluxd/src/chainparams.cpp:153]; $I_{\mathrm{red}} = 1{,}051{,}200$ blocks ($365 \times 24 \times 60 \times 2$, one year at the 30-second PoN spacing) SHIPPED [fluxd/

src/chainparams.cpp:157]; $N_{\text{red}} = 20$ SHIPPED [fluxd/src/chainparams.cpp:158]. The combined block subsidy is $\sigma(h) = \sigma_{PoW}(h)$ for $h < h_{PoN}$ and $\sigma_{PoN}(h)$ otherwise SHIPPED [fluxd/src/main.cpp:2796-2798].

Integer semantics. s_r is r *independently truncated* multiplications, not a closed form: $s_r \neq \lfloor s_0 \cdot (9/10)^r \rfloor$ in general, because each step's rounding error compounds on top of the previous step's rather than being computed once against the original s_0 SHIPPED [fluxd/src/main.cpp:2811-2813]. At $r = 20$ the two expressions already differ at the eight printed decimal places used below: the iterated value is 1.70207313 FLUX against 1.70207316 FLUX for the closed form, and the two first diverge at $r = 10$ SHIPPED [flux-roadmap/supply/emission_model.py:195-204]; the paper states the iterated definition as the one `fluxd` actually computes.

5.2 The frozen halving

The $\min(\cdot, 2)$ term in Definition 5.1 is not an idealization; it is a hand-frozen consensus rule. `GetBlockSubsidy`'s legacy branch reads `if (halvings >= 2) { nSubsidy >>= 2; return nSubsidy; }`, with an in-source comment describing it as a "temp solution while we work on the next blockchain improvement protocols" SHIPPED [fluxd/src/main.cpp:2842-2848]. Emission was therefore held at the second-halving level, 37.5 FLUX/block, rather than continuing to halve every 655,350 blocks SHIPPED [fluxd/src/chainparams.cpp:91]. This was not merely a theoretical stopping point: had halving continued, a third halving (to 18.75 FLUX/block) would have taken effect at height $1{,}968{,}550 = 2500 + 3 \times 655{,}350$, which is before $h_{PoN} = 2{,}020{,}000$; instead, the frozen floor held the reward at 37.5 FLUX/block for the roughly 51,450 blocks between that point and PoN activation, at which point the entire legacy path was replaced by Definition 5.2 SHIPPED [fluxd/src/main.cpp:2845-2848].

5.3 Tier partition and the dev-fund remainder

Definition 5.3 (PoN tier reward)**.** For tier $X \in \mathcal{T} = \{\mathsf{C}, \mathsf{N}, \mathsf{S}\}$ and $h \geq h_{PoN}$,

$$\text{reward}_X(h) \;:=\; \left\lfloor \sigma_{PoN}(h) \cdot \frac{\beta_X}{14\,\text{FLUX}} \right\rfloor,$$

with $\beta_{\mathsf{C}} = 1.0\,\text{FLUX}$, $\beta_{\mathsf{N}} = 3.5\,\text{FLUX}$, $\beta_{\mathsf{S}} = 9.0\,\text{FLUX}$ — ratios chosen so that two Nimbus earn less than one Stratus while three earn more, and three Cumulus less than one Nimbus while four earn more [25] — SHIPPED [fluxd/src/main.cpp:2892-2895], applied against the current, possibly-reduced $\sigma_{PoN}(h)$, not against a value fixed at PoN activation SHIPPED [fluxd/src/main.cpp:2886-2939].

The 14 FLUX denominator is coded as `PON_INITIAL_TOTAL`, a separate local constant in `GetFluxnodeSubsidy`, rather than read from the consensus parameter `nPONInitialSubsidy` that fixes s_0 in Definition 5.2 SHIPPED [fluxd/src/main.cpp:2892]. A future upgrade that changed `nPONInitialSubsidy` without also editing this literal would silently desynchronize the tier ratios from the actual base subsidy; this is a maintenance hazard in the deployed code, stated here because it bears on what a future constant change could do without touching the tier logic at all SHIPPED [fluxd/src/main.cpp:2892].

Definition 5.4 (Dev-fund remainder)**.**

$$\delta(h) \;:=\; \sigma_{PoN}(h) - \text{reward}_{\mathsf{C}}(h) - \text{reward}_{\mathsf{N}}(h) - \text{reward}_{\mathsf{S}}(h), \qquad h \geq h_{PoN}.$$

SHIPPED [fluxd/src/main.cpp:2877-2884] (`GetMinDevFundAmount`). At $r = 0$, the unreduced 14 FLUX subsidy, $\delta = 0.5\,\text{FLUX} = 50{,}000{,}000\,\text{sat}$, i.e. $1/28 \approx 3.57\%$ of PoN emission. Because δ is defined as a remainder of the *current* subsidy rather than a fixed floor, this share decays proportionally as σ_{PoN} reduces SHIPPED [flux-roadmap/supply/emission_model.py:358-368].

Payment is consensus-enforced twice, independently. Loosely: any post-PoN coinbase lacking an output of at least $\delta(h)$ to the hardcoded address is rejected with `"tx-coinbase-missing-dev-funding"` SHIPPED [fluxd/src/main.cpp:1231-1249]. Strictly: `FluxnodeCache::CheckFluxnode`

`Payout` independently requires the coinbase value not already claimed by a tier payout to be at least $\delta(h)$ SHIPPED [fluxd/src/fluxnode/fluxnode.cpp:837-875]. The mainnet address is `t3hPu1YDeGUCp8m7BQCnnNUmRMJBa5RadyA` SHIPPED [fluxd/src/chainparams.cpp:306].

No source comment names this remainder a "dev fund" at the point where the tier ratios themselves are defined; the routing is discoverable only by following `GetMinDevFundAmount`'s call graph out to the two enforcement sites above, not from any single comment at the site of the constants SHIPPED [fluxd/src/main.cpp:2877-2884]. That is a documentation gap worth stating plainly rather than leaving for a reader to reconstruct.

5.4 One-off and recurring fund payouts

Three further payments are enforced directly in coinbase validation, by exact scriptPubKey and amount match, independent of the tier/dev-fund split above SHIPPED [fluxd/src/main.cpp:1252-1304]: two one-off payments and one recurring schedule.

Table 11: One-off and recurring consensus-enforced fund payouts, mainnet. All entries SHIPPED, cited directly to `fluxd`.

Payment	Address	Height / schedule	Amount	Provenance
Exchange fund	`t3PMbbA5YBMrjSD3dD16SSdXKuKovwmj6tS`	$h = 836{,}274$, one-off	7,500,000 FLUX	[fluxd/src/chainparams.cpp:293-294]
Foundation fund	`t3XjYMBvwxnXVv9jqg4CgokZ3f7kAoXPQL8`	$h = 836{,}994$, one-off	2,500,000 FLUX	[fluxd/src/chainparams.cpp:297-298]
Swap pool	`t3ThbWogDoAjGuS6DEnmN1GWJBRbVjSUK4T`	from $h = 837{,}714$, every 21,600 blocks, $\times 10$	22,000,000 FLUX per tranche (220,000,000 FLUX total)	[fluxd/src/chainparams.cpp:301-304]

5.5 The supply function

Definition 5.5 (Cumulative supply)**.**

$$S(h) \;=\; \mathrm{premine}(h) + \mathrm{pow_slow_start}(h) + \mathrm{pow}_{150}(h) + \mathrm{pow}_{75}(h) + \mathrm{pow}_{37.5}(h) + \mathrm{exch}(h) + \mathrm{found}(h) + \mathrm{swap}(h) + \mathrm{pon}(h),$$

with

$$\begin{aligned}
\mathrm{premine}(h) &= \Pi \cdot \mathbf{1}[h \geq 2] \\
\mathrm{pow_slow_start}(h) &= \textstyle\sum_{k=2}^{\min(h,\,4999)} \sigma_{PoW}(k) \\
\mathrm{pow}_{150}(h) &= 150\,\mathrm{FLUX} \cdot \big|\{k : 5000 \leq k \leq \min(h, 657{,}849)\}\big| \\
\mathrm{pow}_{75}(h) &= 75\,\mathrm{FLUX} \cdot \big|\{k : 657{,}850 \leq k \leq \min(h, 1{,}313{,}199)\}\big| \\
\mathrm{pow}_{37.5}(h) &= 37.5\,\mathrm{FLUX} \cdot \big|\{k : 1{,}313{,}200 \leq k \leq \min(h, 2{,}019{,}999)\}\big| \\
\mathrm{exch}(h) &= 7{,}500{,}000\,\mathrm{FLUX} \cdot \mathbf{1}[h \geq 836{,}274] \\
\mathrm{found}(h) &= 2{,}500{,}000\,\mathrm{FLUX} \cdot \mathbf{1}[h \geq 836{,}994] \\
\mathrm{swap}(h) &= 22{,}000{,}000\,\mathrm{FLUX} \cdot \textstyle\sum_{i=0}^{9} \mathbf{1}[h \geq 837{,}714 + 21{,}600\,i] \\
\mathrm{pon}(h) &= \textstyle\sum_{k=h_{PoN}}^{h} \sigma_{PoN}(k) \cdot \mathbf{1}[h \geq h_{PoN}]
\end{aligned}$$

SHIPPED [flux-roadmap/supply/emission_model.py:321-367]. The premine trigger height above uses 2 (the observed mode used for every current- and historical-supply figure in this section); the literal code branch triggers at height 1 instead SHIPPED [fluxd/src/main.cpp:2820-2822] — the one-block difference re-enters as part of the discrepancy in §5.7. All nine terms are additive; the PoW and PoN subsidy terms are mutually exclusive by height, and the premine, exchange, foundation and swap-pool terms are new issuance layered on top of whichever subsidy term is active SHIPPED [flux-roadmap/supply/emission_model.py:469-477]. The tier rewards and dev-fund remainder of §5.3 are a display-only split of $\mathrm{pon}(h)$; they are never added on top of it SHIPPED [flux-roadmap/supply/emission_model.py:358-368].

Proposition 5.6 (No terminal supply)**.** *S does not converge at any height. For every $h \geq h_{20} := h_{PoN} + N_{\mathrm{red}} \cdot I_{\mathrm{red}} = 23{,}044{,}000$,*

$$\sigma_{PoN}(h) = s_{20} = 170{,}207{,}313\,\mathrm{sat} = 1.70207313\,\mathrm{FLUX}$$

is constant, because the reduction loop in Definition 5.2 terminates after $N_{\text{red}} = 20$ *iterations rather than continuing indefinitely* SHIPPED *[fluxd/src/main.cpp:2811-2816]. Consequently* S *grows without bound at the fixed asymptotic rate*

$$s_{20} \cdot I_{\text{red}} = 1.70207313\,\text{FLUX} \times 1{,}051{,}200 = 1{,}789{,}219.274256\ \text{FLUX/year}, \quad \textit{forever.}$$

SHIPPED [flux-roadmap/supply/emission_model.py:514-546], executed 2026-09-03. Height $h_{20} = 23{,}044{,}000$ is a direct consequence of the consensus constants above and is exact; the corresponding calendar date, ∼2045-10-20, is a 30-second-spacing extrapolation from the PoN-activation timestamp anchor, and the model itself validates that extrapolation only against block 2,816,820, about 800,000 blocks after activation, not over horizons this long [doc: flux-roadmap/supply/emission_model.py:389-391]. One PoN year later, at height 24,095,199 (∼2046-10-20, same caveat), cumulative supply is 548,043,625.860464 FLUX SHIPPED [flux-roadmap/supply/emission_model.py:321-367]. This is not a cap; it is only the point at which the annual emission *rate* has become constant, per Proposition 5.6. The team's own public statement is consistent with this derivation: "block rewards reduce 10% a year for twenty years and then continue at a small fixed amount indefinitely — about 1.79 million FLUX a year, forever" [roadmap: flux-roadmap-public.md:189].

Current supply. $S(2{,}912{,}015) = 429{,}466{,}823.9700\,\text{FLUX}$, observed mode, computed 2026-09-03 SHIPPED [flux-roadmap/supply/emission_model.py:321-367]. This is consistent with the team's own public reconciliation — "about 428.1 million FLUX, at block ∼2,816,845 . . . reconciles with our emission model to within 0.0005%" — [roadmap: flux-roadmap-public.md:185], an earlier height on the same monotone curve.

max_money is not a supply cap. MAX_MONEY = 440,000,000 FLUX is defined in `MoneyRange()` as a per-transaction/output sanity bound, not a chain-wide accumulator SHIPPED [fluxd/src/amount.h:22-31]. Cumulative issuance crosses this value at height 3,730,295 (∼2027-06-11) SHIPPED [flux-roadmap/supply/emission_model.py:550-561], and nothing in consensus checks or breaks when it does. The team states this publicly in the same terms: "Neither 440 million nor 560 million is a cap enforced by the protocol . . . issuance passes it in 2027 without anything breaking . . . there is no hard cap today. We would rather say that than repeat a number" [roadmap: flux-roadmap-public.md:189,191].

5.6 Plots

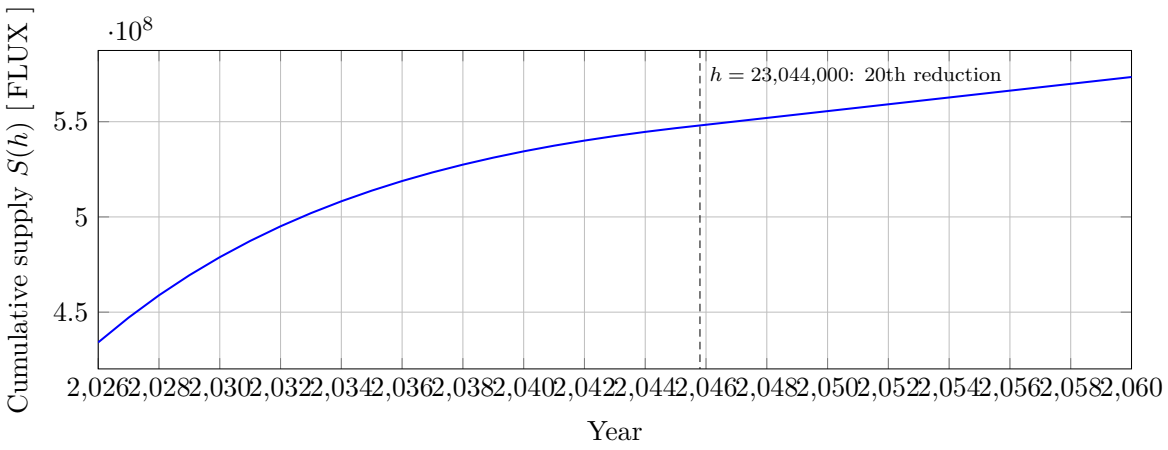


Figure 3: Cumulative issued Flux supply, 2026–2060, mode=observed SHIPPED [flux-roadmap/supply/emission_model.py:321-367]. Data: `paper/data/emission.dat` (columns `year`, `height`, `cumulative_supply`, `annual_emission`, `pow_emission`, `pon_emission`). The dashed line marks the onset of the flat tail rate, per Proposition 5.6, read from `paper/data/pon_reductions.dat` row $n = 20$.

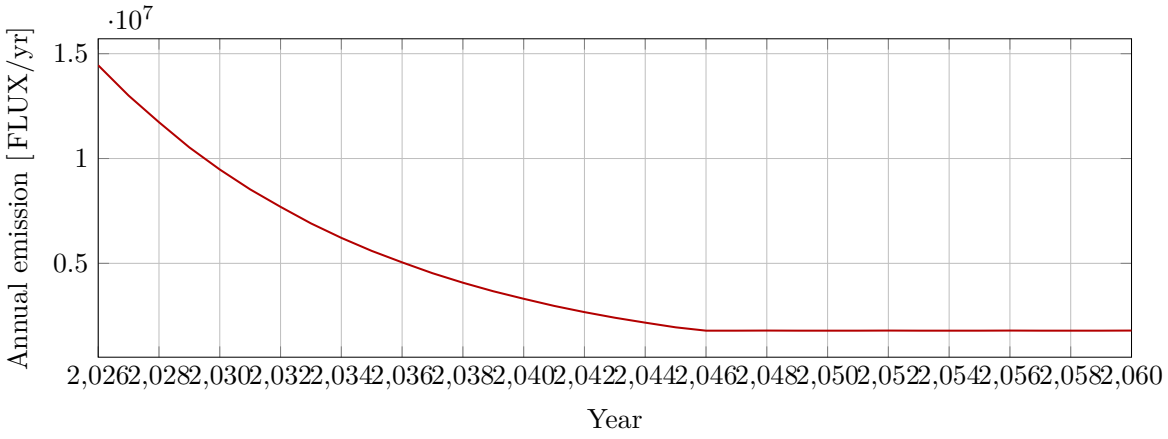


Figure 4: Annual Flux emission by calendar year. `pow_emission` is exactly zero for every plotted year (all postdate PoN activation) SHIPPED [flux-roadmap/supply/emission_model.py:299-318]. Data: `paper/data/emission.dat`, column `annual_emission`. From calendar year 2046 onward the series is 1,789,219.274256 FLUX per 365-day year, per Proposition 5.6; leap years (2048, 2052, ...) carry one more day of blocks and read 1,794,121.2449 FLUX.

5.7 A discrepancy in the published model

The emission model's own docstring claims that its two computation modes — `observed` (what mainnet actually paid) and `code` (a literal transcription of the current `GetBlockSubsidy`) — differ by 0.06 FLUX in total, and calls the choice between them "irrelevant at publication precision" [doc: flux-roadmap/supply/emission_model.py:105-106]. Executing the model shows an exact, height-independent difference of 150.00000000 FLUX at every tested height $\geq$ 5,000, arising from the one-block index shift itself, which lowers the payout of every one of the 4,996 ramp blocks by 0.03 FLUX and of blocks 2 and 2,500 by 0.06 FLUX each, where the docstring's figure counts block 2 alone SHIPPED [flux-roadmap/supply/emission_model.py:207-229]. This is a finding about the published emission model's own arithmetic, not about `fluxd`'s consensus code, which is unaffected. This paper uses `observed` mode throughout, as the model's own default for historical and current-supply claims; full derivation and reproduction steps are in Appendix C.

5.8 Baseline for what follows

The subsidy, tier, dev-fund and fund-payout functions given here are the quantities Section 7 measures Progressive Node Reward parity against, and every issuance, supply or operator-income figure elsewhere in this paper is denominated against the schedule fixed in this section.

6 Parallel assets: the current multi-chain footprint and the case for consolidation

A holder of Flux is not necessarily holding anything on the Flux main chain. Since early in the project's life, tokens representing FLUX have also been issued on other blockchains — a "parallel asset" is a token on a chain other than Flux's own that is meant to track a FLUX balance, historically backed by reserves held at addresses the team controls rather than by a supply-capped, on-chain-verifiable contract [roadmap: flux-roadmap-public.md:96]. Each such asset adds a chain whose contract, custody keys, exchange relationships and accounting have to be kept in sync with the main chain and with each other. This section inventories what actually exists today, shows that the inventory has grown rather than shrunk even as the team has stated an intent to consolidate it, describes the mechanism by which a parallel asset changes hands today and the trust it requires, and states the settled but only partially resolved plan for reducing the footprint. The bridging construction itself — mint-and-burn with attested proof-of-reserve —

is specified in §22; this section establishes why that construction is needed and what it has to replace.

6.1 The inventory

Table 12 lists every chain on which a FLUX-denominated token is identified by contract or account address in the surveyed code, together with the exact on-chain identifier and its source. Ten entries are shipped by this definition: the address or module name appears in code that a live, currently-operated service uses to read balances, construct transfers, or both.

Table 12: The parallel-asset footprint as read from the team's own indexing and swap tooling. Ethereum, BNB Chain, Polygon and Avalanche C-Chain addresses agree between `holder-snapshot` and `flux-supply-tracker`; Base appears only in the latter. SHIPPED for every row — each identifier is consumed by code that treats it as a live contract or account, not asserted from a roadmap or design note. None of these addresses was independently verified against the chain itself by the evidence gathering this section relies on.

Asset	Chain	Contract / account identifier	Provenance
FLUX (ERC-20)	Ethereum	`0x720cd16b011b987da3518fbf38c3071d4f0d1495`	[holder-snapshot/index.js:22][flux-supply-tracker/index.html:310]
FLUX (BEP-20)	BNB Chain	`0xaff9084f2374585879e8b434c399e29e80cce635`	[holder-snapshot/index.js:23][flux-supply-tracker/index.html:319]
FLUX (ERC-20-compatible)	Polygon	`0xA2bb7A68c46b53f6BbF6cC91C865Ae247A82E99B`	[holder-snapshot/index.js:26][flux-supply-tracker/index.html:328]
FLUX (ERC-20-compatible)	Avalanche C-Chain	`0xc4B06F17ECcB2215a5DBf042C672101Fc20daF55`	[holder-snapshot/index.js:24][flux-supply-tracker/index.html:337]
FLUX (ERC-20-compatible)	Base	`0xb008bdcf9cdff9da684a190941dc3dca8c2cdd44`	[flux-supply-tracker/index.html:346] (absent from `holder-snapshot`)
FLUX (SPL mint)	Solana	`FLUX1wa2GmbtSB6ZGi2pTNbVCw3zEeKnaPCkPtFXxqXe`	[holder-snapshot/index.js:20][flux-supply-tracker/index.html:355]
FLUX (ASA)	Algorand	`1029804829`	[holder-snapshot/index.js:27][flux-supply-tracker/index.html:364]
FLUX (Ergo token)	Ergo	`e8b20745ee9d18817305f32eb21015831a48f02d40980de6e849f886dca7f807`	[holder-snapshot/index.js:25][flux-supply-tracker/index.html:373]
FLUX (TRC-20)	Tron	`TWr6yzukRwZ53HDe3bzcC8RCTbiKa4Zzb6`	[holder-snapshot/index.js:21][flux-supply-tracker/index.html:382]
FLUX (Pact fungible-v2)	Kadena	`runonflux.flux`	[flux-kda/flux.pact:1-3][flux-supply-tracker/index.html:391]

The EVM entries (Ethereum, BNB Chain, Polygon, Avalanche C-Chain, Base) are read via each chain's ERC-20 `Transfer` log topic by the same crawler code SHIPPED [holder-snapshot/eth.js:63-88], so they are one contract shape deployed five times rather than five independent designs; the Ethereum/BNB-Chain pair is additionally attested by its own source repository as the same compiled bytecode on both chains [doc: flux-eth-bsc/README.md:1-14]. That repository's own Truffle build record, however, contains no deployed network address for any chain — the `networks` field of `flux-eth-bsc/bin/Flux.json` is absent or empty [doc: flux-eth-bsc/bin/Flux.json, `networks` field]. Every address in Table 12 is taken from the team's own indexing tools, not from the contract-source repository itself.

6.2 The trend is the wrong way

Three snapshots of the team's own tooling, taken at different times, give three different chain counts, and the count only goes up. `holder-snapshot`'s `README` documents support for six chains [doc: holder-snapshot/README.md:4-10]; its own `index.js` code handles eight, adding Polygon and Algorand beyond what the README claims SHIPPED [holder-snapshot/index.js:20-27]; and `flux-supply-tracker`'s live `CHAINS` configuration, last touched 2026-08-05, carries ten, adding Base and folding Kadena in as a tenth SHIPPED [flux-supply-tracker/index.html:308-397]. Read plainly: documentation said six, shipped code already did eight, and the newest measurement tool does ten. This is the team's own data, gathered to run its own reserve and rich-list computations, and it is the single strongest argument in this section for consolidation — not because any one parallel asset is individually a problem, but because the footprint has been growing under a team that has also, in the same period, stated a public non-goal of not adding new parallel-asset chains while the existing ones are consolidated [roadmap: flux-roadmap-public.md:209]. The growth

already happened before that statement was made, which is exactly why consolidation is a live problem and not a preventative one.

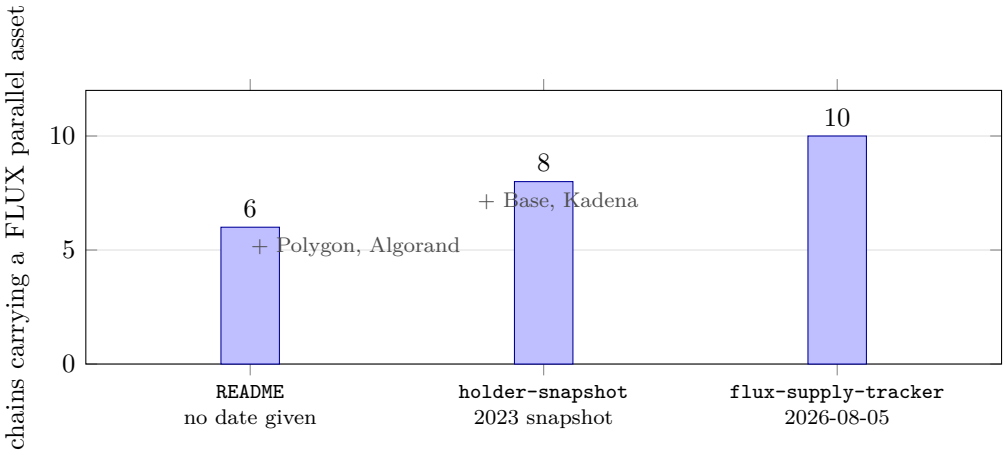


Figure 5: The parallel-asset footprint grew between every source that documents it, and no single source is current. The count is six in the repository README, eight in the `holder-snapshot` code path, and ten in `flux-supply-tracker`; the two chains added at each step are annotated. This is the measurement problem §6.4 addresses: a consolidation cannot be scoped from a source that does not know how many chains exist. SHIPPED [doc: holder-snapshot/README.md:4-10][holder-snapshot/index.js:20–27][flux-supply-tracker/index.html:308–397].

One further caveat belongs here rather than in a footnote: as of the most recent local state of `holder-snapshot`, only its Ergo crawl is actually invoked — every other chain's `start()` call is commented out SHIPPED [holder-snapshot/index.js:29-36] — so the tool that produced the eight-chain figure is not itself being run against eight chains today. That does not change the ten-chain count in the newer tracker, which is the operative figure.

6.3 How a parallel asset is acquired today

Two mechanisms exist for turning a claim on one chain into a balance on another, and both are mediated by the same custodial system, Fusion (`ZelCore-io/fusion`), an Express service the team operates at `fusion.runonflux.io` SHIPPED [fusion/config/default.js:6], [fluxswap/lib/api/services/swap_service.dart:10].

Cross-chain swap. A holder deposits to a Fusion-controlled address on the source chain (one of eleven `swapsecrets` addresses, one per chain — the main chain and the ten parallel chains — SHIPPED [fusion/config/default.js:434-482]); Fusion's automation loop detects the deposit SHIPPED [fusion/src/services/swapAutomationService.js:736-800], and once confirmation thresholds are met, its resolver signs and broadcasts the equivalent send on the destination chain from that chain's own single hot-wallet key SHIPPED [fusion/src/services/swapService.js:997-1006]. Nothing is signed by the holder in this path beyond the original deposit transaction.

Snapshot claim. For balances established by an earlier, offline snapshot (an operator runs `flux-cli printsnapshot` and the resulting balances are loaded into Fusion's database [doc: fusion/README.md:29]), the holder proves ownership by signing a server-issued, single-use message phrase with the private key of the recorded source address SHIPPED [fusion/src/services/claimSnapshotService.js:202-207]. Fusion verifies that signature against the recorded address using a chain-specific signature-validation routine SHIPPED [fusion/src/services/generalService.js:349-563], then pays out the claimed amount from the same per-chain custodial hot wallet used for swaps SHIPPED [fusion/src/services/claimSnapshotService.js:250-286].

In both paths, the trust assumption is the same at the point of payout: a single private key per chain per role, held by the Fusion operator, with no multisig, threshold signature, or on-chain

escrow gating the outbound send SHIPPED [fusion/src/coinLibs/eth.js:66-101]. What a holder signs proves ownership of a source-chain balance; it authorizes nothing about the destination-chain payment, which is entirely at the operator's discretion once the claim is validated.

Kadena is a distinct, additional case. Kadena-denominated FLUX increases only through the Pact module's `fund` function, which is gated by a 2-of-3 multisig keyset held by the team SHIPPED [flux-kda/flux.pact:12-14,84-89], [flux-kda/env:2–9]. Eligibility for a Kadena node-reward payout is computed off-chain: a node self-reports its status SHIPPED [fluxstats/src/services/kadenaService.js:119], a team-run crawler polls that status every two hours SHIPPED [fluxstats/src/services/kadenaService.js:171-173], and nothing in the Pact contract itself constrains what a `fund` call may credit or on what basis SHIPPED [flux-kda/flux.pact:84-89]. The claim path that converts an eligibility figure into a credited balance does exist, and it was located outside the repositories this section first examined. A holder proves ownership of a main-chain address by signing a one-time server-issued phrase, verified against the legacy Zelcash signature scheme SHIPPED [fusion/src/services/generalService.js:378-390]. Eligibility is then computed by Fusion's backend from its own private index of Flux daemon coinbase transactions, at a flat 10% per chain SHIPPED [fusion/src/services/coinbaseService.js:6-25,103-157]. Payout is not a mint: it is a plain Pact `transfer` from a team treasury account, signed by a single team-held hot key SHIPPED [fusion/src/coinLibs/kda.js:231-300], [fusion/config/default.js:393-395].

So the `fund` capability and the claim path are two separate mechanisms, and the claim path never invokes the mint. Every step is team-mediated: a team-run database decides eligibility, a single team-held key executes the payout, and the 2-of-3 team keyset alone can increase supply. The only step not under team control is the holder's initial signature proving address ownership. What remains undocumented in any surveyed repository is how and when the treasury account is replenished through that 2-of-3 mint — which is the one place where Kadena-denominated supply actually grows. The repository named for this mechanism, `kadena-distribution-public`, still contains only a licence and a two-line README [doc: kadena-distribution-public/README.md:1-2].

6.4 The supply-accounting problem

Any published figure for how much FLUX is in circulation, net of parallel-asset reserves the team itself holds, depends on a list of which addresses count as "team reserve." Three such lists exist in the surveyed corpus, and only one of them is authoritative: `balance-checker` (`RunOnFlux/balance-checker`), the repository the team itself runs to monitor its own issuer-controlled addresses SHIPPED [balance-checker/config/default.js:39-232] — not `fluxstats` and not `flux-supply-tracker`. `flux-supply-tracker`'s own source comment corroborates this: it states that its locked-reserve figures come "from the public Fusion balance checker (github.com/RunOnFlux/balance-checker), which monitors the same set" SHIPPED [flux-supply-tracker/index.html:301–303].

On Ethereum, `fluxstats`'s rich-list service excludes twelve addresses SHIPPED [fluxstats/src/services/richListService.js:146-153], while `flux-supply-tracker`'s reserve-wallet configuration for the same chain excludes seven SHIPPED [flux-supply-tracker/index.html:311-317]. An earlier reading of this gap in the paper treated it as a genuine disagreement over what counts as reserve. **That reading is superseded.** The root cause is known and it is benign: `fluxstats`'s Ethereum exclude list still names the ETH `LOCKED` address as it stood before an August 2026 key rotation, `0xa23702e9349fbf9939864da1245f5b358e7ef30b`, rather than the current address, `0x3d1759846bbbdeb7f71558d1fc6f00916006a795` SHIPPED [fluxstats/src/services/richListService.js:152] [balance-checker/config/default.js:98]. The same stale-rotation lag, not a deeper disagreement, also accounts for `fluxstats`'s BSC `LOCKED` entry SHIPPED [fluxstats/src/services/richListService.js:171] and its Flux main-chain `COLD` entry SHIPPED [fluxstats/src/services/richListService.js:74], both of which still name the address each replaced. Neither tool was conceptually wrong about what counts as a reserve address; one was simply not updated when the addresses it tracks rotated.

Of the two addresses an earlier draft of this section left as disputed: `0x3d1759846bbbde`

`b7f71558d1fc6f00916006a795` is confirmed issuer-held — it is `balance-checker`'s own current ETH `LOCKED` address SHIPPED [balance-checker/config/default.js:98] and holds 279,000,000 FLUX SHIPPED [measured: ethereum-rpc.publicnode.com, 2026-09-03]. `0x9b77c6af70878c5c748d0ab456dcac1dfc01f962` (labelled "fees" in `flux-supply-tracker` only) is absent from the authoritative registry on every chain and holds zero on every EVM chain queried; it is dropped from this accounting SHIPPED [measured: public RPC, per-chain endpoints, 2026-09-03].

`totalSupply` is not the outstanding liability. Every EVM parallel asset was minted at the full nominal 440,000,000 FLUX at contract construction: Ethereum, BNB Chain, Avalanche C-Chain and Base each report a `totalSupply` of exactly 440,000,000 FLUX, as do the Algorand ASA and the Ergo token; the Solana mint reports 59,999,990.375 FLUX SHIPPED [measured: public RPC, per-chain endpoints, 2026-09-03]. 440,000,000 is also MAX_MONEY in `fluxd`, a per-transaction sanity bound rather than a supply cap [roadmap: flux-roadmap-public.md:189,191] (Section 5). Eight of the ten chains (Table 13) each holding the entire nominal cap, against a main chain that has issued 429,466,823.97 FLUX in total (Section 5), makes the point on its own: `totalSupply` measures what was minted into a contract, not what remains outstanding. What becomes of the issuer-held remainder is decided: the team is to immobilise the issuer's own legacy holdings by transferring them to a published per-chain dead address (§22.8; PLANNED [intent: design intake]).

Measured outstanding liability, all ten chains resolved. Using `balance-checker`'s registry as the operand, the outstanding liability — `totalSupply` minus the sum of the six current, post-rotation issuer addresses per chain (`SNAPSHOT`, `MINING`, `SWAP`, `LOCKED`, `LOCKED SNAPSHOT`, `LOCKED MINING`) — is now measured for all ten legacy assets. Retrieved 2026-09-03T11:23:06Z (Ethereum, BNB Chain, Avalanche C-Chain, Base) and 2026-09-04 (Solana, Algorand, Polygon, Ergo, Tron, Kadena); reproducible with `scripts/13-issuer-balances.sh`, `scripts/15-nonevm-liability.sh` and `scripts/16-remaining-liability.sh`, `scripts/18-tron-final.sh` and `scripts/19-kadena-ledger.js`.

Table 13: Outstanding parallel-asset liability by chain, using `balance-checker`'s registry of issuer-controlled addresses as the operand. All ten legacy assets are measured from key-free public endpoints. Nominal supply differs by chain: the other eight deployments (five EVM, plus Algorand, Ergo and Tron) were each minted at 440,000,000, Solana at 59,999,990.375, and Kadena at 39,998,820.81. SHIPPED [measured: scripts/13-issuer-balances.sh, 2026-09-03]; [measured: scripts/15-nonevm-liability.sh, scripts/16-remaining-liability.sh, scripts/18-tron-final.sh, scripts/19-kadena-ledger.js, scripts/20-kadena-issuer.js, 2026-09-04].

Chain	`totalSupply` (FLUX)	Issuer-held (FLUX)	Outstanding (FLUX)
Surviving — snapshot-carried, no holder action			
Ethereum	440,000,000.00000000	283,782,657.21951687	156,217,342.78048313
BNB Chain	440,000,000.00000000	379,416,462.37025207	60,583,537.62974790
Retiring — holder-initiated redemption within the window			
Base	440,000,000.00000000	437,471,141.57008934	2,528,858.42991062
Polygon	440,000,000.00000000	439,352,071.26584000	647,928.73416000
Avalanche C-Chain	440,000,000.00000000	439,371,417.37793958	628,582.62206042
Ergo	440,000,000.00000000	439,513,253.38117500	486,746.61882500
Algorand	440,000,000.00000000	439,610,356.99374300	389,643.00625700
Solana	59,999,990.37500000	59,626,923.17797100	373,067.19702900
Kadena	39,998,820.80974744	39,200,613.15299041	798,207.65675703
Tron	440,000,000.00000000	439,626,133.46693521	373,866.53306479
Total (all 10)	3,619,998,811.18474744	3,396,971,029.97645248	223,027,781.20829496
of which retiring			6,226,900.79806390

Two chains required a different operand rather than resisting measurement. Tron is re-

trievable from Tronscan without an API key, which the team's own checker already uses [balance-checker/src/services/balanceService.js:104]. Kadena's Pact module genuinely exposes no total-supply function — `describe-module` lists `get-balance`, `details`, `transfer`, `credit`, `debit` and `snapshot`, and no supply accessor — but its `ledger` table is enumerable in local execution, so the supply is recoverable as the sum of all account balances across the twenty Chainweb chains rather than read from a single call. That sum is 39,998,820.80974744 FLUX over 14,702 accounts, of which all non-zero balances outside chain 0 total 405,968.76 FLUX SHIPPED [measured: scripts/19-kadena-ledger.js, scripts/20-kadena-issuer.js, 2026-09-04]. The outstanding liability across the full ten-chain footprint is therefore fully determined: every one of the ten chains, including the two largest legacy balances by nominal supply, resolves to an exact, measured figure.

The consequence is concrete: a stated supply figure that subtracts team reserves from a parallel asset's total supply is only as well-defined as the exclusion list it used, and this section's finding is that `balance-checker`'s registry is the list to use, not either of the other two tools' independently hand-rolled lists. §5 states the main chain's own supply function exactly and cross-checks it against the team's published reconciliation; the measurement above extends that same discipline to all ten parallel-asset chains. The roadmap's own Q3 2026 commitment to a live, public supply tracker [roadmap: flux-roadmap-public.md:36-37] is IN-PROGRESS: it exists as the `flux-supply-tracker` repository [flux-supply-tracker/README.md:1–3], whose source states that its reserve addresses come from `balance-checker`'s registry [flux-supply-tracker/index.html:301–303], but whether it is deployed publicly this survey did not verify.

6.5 The case for consolidation

Cost here scales with the number of chains, not with the number of holders. Each additional parallel asset in Table 12 means: another contract or module whose semantics must be understood and kept correct (ten deployments across five EVM variants, an SPL mint, an ASA, a Pact module, a TRC-20 contract and an Ergo token, per §6.1); another set of custody keys — eleven chains times up to three role-specific secrets each, i.e. up to thirty-three live private keys held by one operator SHIPPED [fusion/config/default.js:342-482]; another entry in the reserve-accounting problem of §6.4; and, plausibly, its own exchange-listing and liquidity relationships [inferred](not directly evidenced in this corpus, but implied by the roadmap's framing of consolidation as concentrating on "chains where FLUX holders and DePIN users actually are" [roadmap: flux-roadmap-public.md:130-131]). Per-chain failure modes are not hypothetical: Fusion maintains at least three separate, mutually inconsistent tables of how many confirmations count as "enough" for the same nominal purpose, one per code path SHIPPED [fusion/src/services/swapService.js:1085-1090,1181-1219][fusion/src/services/swapAutomationService.js:164-182] — a direct, citable instance of the maintenance burden that having eleven independently-configured chains imposes on the operating team.

The settled decision responds to that cost structure by shrinking the chain count and by treating every survivor as a fresh build rather than a migration of the existing contract. Ethereum and BNB Chain are mandatory at launch, with Solana following later [intent: design intake]. Every surviving token — Ethereum and BNB Chain included, not only Solana — is to receive a full contract relaunch rather than a continuation of the existing deployment [intent: design intake], with independent audit before deployment stated as a precondition, alongside multi-signature administrative control, per-chain issuance caps, pausability, and published reserve accounting [roadmap: flux-roadmap-public.md:96-97]. This is PLANNED in its entirety: no relaunched contract, audit, or migration has occurred as of the evidence surveyed here.

Ethereum and BNB Chain are both EVM chains, and the design intake states this explicitly as a point in the decision's favor: the mandatory pair at launch is one EVM codebase deployed twice, not two independent contract designs, with a separate Solana program to follow later [intent: design intake]. Against the ten-chain footprint of Table 12, that is a material reduction in the number of independent things an auditor, an exchange, or the operating team has to reason about, even before the remaining eight chains are formally retired.

6.6 The Base question

Two documents disagreed on which two chains ship alongside Solana. The public roadmap names Ethereum and Base as the first pair, and later names "Ethereum, Base and Solana" as the chains consolidation concentrates on [roadmap: flux-roadmap-public.md:97,131]. The design intake stated, twice, that the mandatory launch pair is Ethereum and BNB Chain, with Solana following later [intent: design intake]. Base (an Ethereum L2, OP Stack) and BNB Chain (an independent L1) are not the same chain by any reading.

This is now settled directly by the team: **Base is to be retired, and no later re-addition is contemplated** (PLANNED; [intent: design intake]). That closes the question in the direction the design intake already implied — Base is not a survivor — rather than leaving open which second chain accompanies Ethereum. The roadmap's naming of Base is recorded here because it is a real divergence between the team's own public and internal statements, not because the paper adjudicates it silently: the more recent, direct answer to this paper is the one the paper follows.

Base already carries a deployed FLUX contract today SHIPPED [flux-supply-tracker/index.html:346], per Table 12, and that contract does not get relaunched. Base's existing holders go through the same retirement and redemption path as every other non-surviving chain, specified in §6.7: a published schedule, a one-directional redemption window through the existing bridge at 1:1 — open from $h^{\text{mig}}_{\text{open}}$ (March 2027) to shutdown at $h^{\text{mig}}_{\text{close}}$ (August 2027) — and the same disposition for balances left unclaimed when that window closes. Base's weight in that path is worth stating, because it is not the weight its share of total liability suggests: its measured outstanding balance of 2,528,858.42991062 FLUX SHIPPED [measured: scripts/13-issuer-balances.sh, 2026-09-03] is 1.13 % of the ten-chain total but 40.6 % of the 6,226,900.80 FLUX actually exposed to the window [inferred]. Base is most of the holder-initiated migration problem, which makes the divergence between the roadmap's naming of Base as a survivor and its actual retirement a communication liability concentrated on precisely the holders who must act.

6.7 What consolidation requires

The construction that replaces today's custodial Fusion mechanism — canonical mint-and-burn with attested proof-of-reserve — is specified in §22. The migration's terms are now set, and staged: the new Ethereum and BNB Chain contracts go live first, on readiness and before the cut; from $h^{\text{mig}}_{\text{open}}$ (March 2027) the eight retiring chains become one-directional; the emission cut lands at PA Depletion (h_{PA}, height 3,787,502, §6.8); and the retiring chains shut down at $h^{\text{mig}}_{\text{close}}$ (August 2027). Nothing is switched off without notice (PLANNED; [intent: design intake]; [intent: 08-answers-round-2.md, round 34]).

Two migration paths, and they are different mechanisms. A holder's exposure to the migration depends entirely on which of two paths that holder's chain is on, and the paper separates them because conflating them misstates the exposure by two orders of magnitude (PLANNED; [intent: design intake]).

Snapshot-and-carry, for Ethereum and BNB Chain only. Ethereum and BNB Chain survive. Their contracts are nonetheless replaced (§6.5), so **their holders are carried across to the new contracts by a snapshot migration, coordinated with exchange partners and ecosystem integrators. They take no action.** The mechanism — a published snapshot height, a `Transfer`-log replay of the holder set, and an issuer-executed credit into the relaunched contract whose correctness any third party can check against the legacy contract at that height — is specified as Algorithm 9 in §22.8, together with the elements of it that are undecided. **This path covers exactly these two chains.** Solana is a survivor in the sense that a new deployment follows later, but its *legacy* holders are not carried: they redeem through the second path below and re-enter the new Solana deployment later if they choose (PLANNED; [intent: design intake]). "Surviving"

and "snapshot-carried" are therefore not the same set, and only the latter escapes the window.

Holder-initiated redemption, for the retiring chains. Avalanche C-Chain, Base, Polygon and the non-EVM assets are to be retired. From $h^{\text{mig}}_{\text{open}}$ (March 2027) until shutdown at $h^{\text{mig}}_{\text{close}}$ (August 2027) — about five months, spanning the emission cut — a **redemption window** is open, during which Fusion — the existing custodial swap and claim mechanism of §6.3 — is the stated route back to main-chain FLUX, at 1:1 (PLANNED; [intent: design intake]). The holder must act. **That route is one-directional**: from $h^{\text{mig}}_{\text{open}}$ the retiring chains permit migrating *to* the main chain and claiming mining entitlement — on the chain itself or on the main chain — and nothing inbound: no swap *into* a retiring chain [intent: 08-answers-round-2.md, round 35] (PLANNED; [intent: design intake]).

Unclaimed balances at window close are to pass to the team behind the project (informally, the Flux Foundation) to fund development (PLANNED; [intent: design intake]). That term reaches only the second path, so it is stated next to the right number rather than next to the whole measured liability. Of the 223,027,781.21 FLUX measured across all ten chains in Table 13, 216,800,880.41 FLUX sits on Ethereum and BNB Chain — surviving, snapshot-migrated, no holder action, not exposed — and 6,226,900.80 FLUX sits on the eight retiring chains, which are therefore subject to the window and the forfeiture term (SHIPPED; [measured: scripts/13-issuer-balances.sh, 2026-09-03]; [measured: scripts/15-nonevm-liability.sh, scripts/16-remaining-liability.sh, scripts/18-tron-final.sh, scripts/19-kadena-ledger.js, 2026-09-04]). **The amount exposed to the redemption window and the unclaimed-balance term is** 6,226,900.80 FLUX **across the eight retiring chains** (§6.4). This is no longer a lower bound: every legacy asset is now measured, and 97.21 % of all parallel-asset liability sits on the two chains that require no holder action at all. What fraction of that exposure fails to migrate within the window is unknown, and this paper does not estimate it. The policy is a legitimate one — unclaimed value must belong to someone, and a stated destination is better than an unstated one — but it is also a transfer from holders who do not act within the window to the issuer, and the paper names it in those terms rather than omitting it. What the snapshot path changes is not the term's character but its reach: it is the reason the great majority of FLUX holders on parallel chains are unaffected by it.

A measurement prerequisite precedes any of this, and it is now partly satisfied. The bridge design's conservation requirement — that minted supply on a destination chain never exceed what is actually backed — forbids any migration mint before the per-asset legacy supply on each retiring chain, and whether that chain's reserves actually cover it, has been confirmed [intent: mechanism specification]. Section 6.4 above supplies the first half of that confirmation — the measured outstanding liability itself — for all ten chains in Table 12: all ten legacy assets, totalling 223,027,781.21 FLUX outstanding SHIPPED [measured: scripts/13-issuer-balances.sh, 2026-09-03]; [measured: scripts/15-nonevm-liability.sh, scripts/16-remaining-liability.sh, scripts/18-tron-final.sh, scripts/19-kadena-ledger.js, scripts/20-kadena-issuer.js, 2026-09-04]. Whether reserves sufficient to cover that figure actually exist is a separate question, addressed in Section 22.8; it remains a prerequisite task for the migration, not a gap in this paper.

6.8 Sequencing: one coordinated transition

The public roadmap and the design intake originally stated this differently, and an earlier draft of this paper carried it as an open conflict (C1a). It is now settled directly by the team: PNR activation and the parallel-asset emission cut are **simultaneous**, both at PA Depletion, height $h_{\text{PA}} = 3{,}787{,}502$ (2027-07-01) (PLANNED; [intent: design intake]). Around it the parallel-asset schedule is staged [intent: 08-answers-round-2.md, round 34]: Ethereum and BNB Chain move to the new contracts earlier, on readiness; from $h^{\text{mig}}_{\text{open}}$ the eight retiring chains are one-directional — nothing can be swapped into them, while swapping back and claiming continue; the chains shut down at $h^{\text{mig}}_{\text{close}}$, after which unclaimed balances pass to the team behind the project (§6.7).

Decided. $h^{\text{mig}}_{\text{open}}$ = 3,450,000 (2027-03-06) and $h^{\text{mig}}_{\text{close}}$ = 3,880,000 (2027-08-02): proposed by this paper at the 30 s target for the author's months, March and August 2027, and confirmed [intent: 08-answers-round-2.md, round 36].

Remark 6.1 ("Depletion" is the computed exhaustion height). The event's name is the design intake's, and it is literally accurate: h_{PA} is the block at which the parallel-asset coinbase pools are exhausted, and that block is computable exactly from the emission schedule. Each chain was funded with 22,000,000 FLUX by the main-chain swap pool [flux-roadmap/supply/emission_model.py:165–172], split three ways: a user snapshot of 12,313,785.94991485 FLUX, a 1,000,000 FLUX development-and-exchange fund, and a coinbase pool of 8,686,214.05008515 FLUX SHIPPED [insight-api/lib/status.js:277–279]. From block 825,001 every coinbase receiver is entitled to a further 10 % of each output on every parallel chain [doc: fusiondocs/openapi/openapi.yaml:667], so the pool accrues at one tenth of the main-chain block subsidy per block — roughly 1.4 FLUX after PoN [fusion/src/services/coinbaseService.js:16–19,49–51]. Across the ten chains that is the whole block subsidy again in claimable tokens, so until the cut an operator who claims earns about twice the main-chain subsidy [inferred]. Summing that accrual over the emission schedule, the pool is exhausted at **block** 3,787,502, about 1–4 July 2027 [inferred] — and three independent models agree on it to within a block: this paper's emission model, the prior whitepaper's supply model, and that whitepaper's own stated remainder of 2,449,214.05 FLUX at block 2,000,000. The intake's earlier figure of 3,466,630 preceded this by 320,872 blocks and was superseded on that ground [intent: 08-answers-round-2.md, round 33]; no consensus constant yet encodes the height. Table 14 gives the sum epoch by epoch. Claiming, however, is voluntary, per chain and fee-bearing, with a start height and no coded end [fusion/config/default.js:239]. Nor does a claim require touching the parallel chain at all: the mining-claim endpoint accepts up to twelve *main-chain* addresses, computes each one's entitlement from its coinbase receipts, and pays the claim in main-chain FLUX from a Fusion-operated wallet SHIPPED [fusion/src/services/claimMiningService.js:1222–1235,1492]. The team confirms this is the intended route [intent: 08-answers-round-2.md, round 31]. Claiming on the retiring chains themselves continues until shutdown; what closes at $h^{\text{mig}}_{\text{open}}$ is the swap *into* them [intent: 08-answers-round-2.md, round 35]. Where a claim is paid on the main chain it is paid from the per-chain swap pool, so its counterpart in that chain's `MINING` and `LOCKED MINING` balances must be excluded at the snapshot or destroyed, or the backing invariant of §22 fails [inferred]. Two consequences follow. Unclaimed entitlement is not stranded on a retiring chain, so the retirement cut-off interacts with main-chain claiming and not only with the parallel-chain redemption window. And the main-chain side must hold a balance to honour those claims; the wallet that pays them, and what backs it, are not in the evidence corpus, so this paper records the payout path and not its reserve. Measured on 2026-09-07, between 3,000,322 FLUX (Ethereum) and 3,782,411 FLUX (Tron) sits per chain in the `MINING` and `LOCKED MINING` addresses [measured: scripts/13-issuer-balances.sh; public RPC, 2026-09-07]. That is more than the 1,112,549.95 FLUX not yet accrued at the measurement height, and the difference is entitlement that has accrued and never been claimed — on Ethereum about 1,887,772 FLUX, a quarter of everything accrued to date [inferred]. Tron's `MINING` address paid out 428 FLUX per day over 74 days, about 0.148 FLUX per block against 1.4 accruing [measured: apilist.tronscan.org token_trc20/transfers, 2026-09-07]; claims lag accrual and arrive in lumps. At h_{PA} nothing is left to accrue; what the height does not settle is that accrued-but-unclaimed quarter, and the round-7 provision that unclaimed balances pass to the team behind the project is what disposes of it. The prior whitepaper's "2027–2028" [24] is consistent with the computed July 2027 (conflicts C64, C65).

blocks	subsidy (FLUX)	10 %/block	blocks in epoch	epoch accrual (FLUX)	cumulative (FLUX)
825,001 – 1,313,199	75.00	7.500	488,199	3,661,492.50	3,661,492.50
1,313,200 – 2,019,999	37.50	3.750	706,800	2,650,500.00	6,311,992.50
2,020,000 – 3,071,199	14.00	1.400	1,051,200	1,471,680.00	7,783,672.50
3,071,200 – 3,787,501	12.60	1.260	716,302	902,540.52	8,686,213.02
block **3,787,502** completes the pool: 1.03008515 FLUX of 8,686,214.05008515 remained for it					

Table 14: Per-chain coinbase-pool accrual: 10 % of the block subsidy per block from block 825,001 [fusion/config/default.js:239], credited on every coinbase output, dev fund included [fusion/src/services/coinbaseService.js:113–118], over the subsidy schedule as coded [fluxd/src/main.cpp:2796–2850]: halvings at 657,850 and 1,313,200 with the third suppressed; PoN 14.0 at 2,020,000, ×9/10 at 3,071,200. The pool is 22,000,000 less the 12,313,785.94991485 snapshot and the 1,000,000 chain fund [insight-api/lib/status.js:277–279]. The second reduction (11.34 at 4,122,400) lies 334,898 blocks past depletion and does not enter. Cross-check: through block 2,000,000 this sum gives 6,236,996.25 against the prior whitepaper's 6,237,000 [24], one block of accrual apart. Block 3,787,502 falls on 2027-07-01 at the 30 s target and 2027-07-04 at the observed 30.13 s spacing [inferred].

This **supersedes** the roadmap's stated gate — that retirement of any parallel asset's mining rewards "comes only after PNR v2 is demonstrably paying operators," with retirement itself scheduled in H2 2027, "announced at least 90 days ahead" [roadmap: flux-roadmap-public.md:133,159] — a sequence that would have placed retirement a half-year after PNR v2's own H1 2027 rollout. The team's direct answer to this paper is the more recent and more specific statement, and the paper follows it.

The practical consequence is that this section and §27 describe *one* coordinated transition rather than two phases with a proving interval between them: at h_{PA}, PNR would begin paying operators and parallel-asset mining rewards would be cut, at the same height, inside the redemption window of §6.7, which opens earlier, at $h^{\mathrm{mig}}_{\mathrm{open}}$. The absence of the proving period the roadmap's original sequencing would have created — the interval in which PNR could have been observed actually paying operators before the parallel assets it was meant to justify retiring were cut — is a risk-relevant fact in its own right, and is addressed as such in §27.7.

7 PNR: revenue-funded operator compensation

Everything a tenant pays to run an application on Flux today lands at a single transparent address, and every satoshi a node operator earns is newly issued SHIPPED [measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03], [fluxd/src/fluxnode/fluxnode.cpp:892–936]. Progressive Node Rewards (PNR) is the planned change to the second half of that sentence, and it is designed so that the first half would look unchanged to whoever is paying: the tenant would still make *one* payment, of the full price, to *one* address, and consensus — not the payer, not the wallet, and not FluxOS— would divide what arrives there. A fixed fraction would accrue to a protocol-held fee pool, and that pool would be paid out to node operators through the block subsidy's existing payee rotation, one small increment per block PLANNED [intent: design intake]. The purpose of this section is to state that mechanism precisely enough to implement, to prove the fairness property the design rests on, to prove that the pooling defeats a timing attack that would otherwise invert the mechanism, and — with equal care — to state the two things the mechanism does *not* do: it does not make operator income proportional to collateral, and it makes operator income independent of which applications a node runs. That second point is a statement about the eligibility gate rather than about willingness to work, because placement is the network's choice and not the operator's (§7.8).

Terminology hazard, stated once and before anything else. PoN (*Proof of Node*) and PNR (*Progressive Node Rewards*) are different mechanisms, in different layers, at different maturities, and the names are trivially confused. PoN is the deployed consensus rule that replaced hashing: it selects one payee per tier per block from a collateral-gated queue and pays that payee out of the block subsidy SHIPPED [fluxd/src/fluxnode/fluxnode.cpp:713–778]. PNR is the planned revenue-sharing rule specified in this section: it takes a fraction of application payments and routes it to whichever nodes PoN has already selected PLANNED [intent: design intake]. PNR consumes PoN's payee set and adds no selection mechanism of its own. Wherever this section says "the rotation" it means PoN's rotation; wherever it says "the pool" or "the drip" it means PNR.

Symbols local to this section. Beyond `notation.tex` this section uses: F_h for the pool inflow of block h (in sat); G_h for the Foundation share of the same block's receipts, i.e. the share paid through the existing dev-fund coinbase output ("the Foundation" is used informally, as in Definition 2.3, for the holder of that output, not a legal entity); D_h for the drip paid out of the coinbase of block h and $D_{h,t}$ for its tier-t part; ϵ_h for the integer partition residue and U_h for tier parts that find no payee; A_F and A_Φ for the Foundation and pool parts into which consensus divides a single received payment; sink for the consensus application-payment address, $\mathcal{O}_h$ for the set of outputs paying it in block h and Rev_h for its balance at height h; m_a for the canonical serialisation of application a's specification and H for the hash carried in the payment memo; $q := 1 - 1/K$; $r(h) := 100\,\rho(h)$ for the integer form of the operator share; V for annual application revenue in FLUX per year; g for annual revenue growth; m for an exogenous price multiple used only in the clearly-labelled secondary analysis of §7.11.2; and $w_{\mathcal{Z}}$ for an adversary's share of the drip. All consensus arithmetic is over integers in sat, with 10^8 sat $= 1$ FLUX [fluxd/src/amount.h:17].

7.1 Assumptions

Assumption 7.1 (Model facts this section depends on)**.** All properties below are stated against the system and adversary model of §2. Five of its facts are load-bearing here and are restated so that no proof silently assumes more than the model gives.

A1 *Deterministic payee selection.* For each tier $t \in \mathcal{T}$ the payee of block h is the front of a queue sorted by last-paid height ascending, with never-paid nodes keyed by confirmation height; the front entry is returned and stale entries purged SHIPPED [fluxd/src/fluxnode/fluxnode.h:313–328], [fluxd/src/fluxnode/fluxnode.cpp:713–778]. The selection is a public, deterministic function of chain state — it is predictable to every observer, and §7.7 treats that predictability as the adversary's capability, not as an oversight.

A2 *Fixed chainwork.* Every PoN block contributes a fixed 2^{40} to chainwork, so fork choice is a block-count race rather than a cumulative-work comparison SHIPPED [fluxd/src/pow.cpp:284–290] (C14). No proof below may appeal to reorg cost scaling with hashing.

A3 *Emergency block production exists.* A hardcoded 2-of-4 key set can produce a block outside the normal PoN path, with no time or stall-detection gating, and the remainder output of its coinbase defaults to the dev-fund address while its tier outputs remain bound by the deterministic payout check SHIPPED [fluxd/src/emergencyblock.cpp:183–191], [fluxd/src/rpc/emergencyblock.cpp:64–77], [fluxd/src/miner.cpp:487–489] (C9). This is an adversary capability the mechanism must be designed against, not a hypothetical. It is also, already, a public fact: the key count, the threshold and the absent gating are all readable in an open-source repository and every emergency block ever produced is on the chain, so stating it here discloses nothing that was not disclosed by shipping it. What publication does change is that the capability becomes *legible*, and the paper's position is that the right response is to

close it rather than to leave it undescribed: time or stall gating in `fluxd` is cheap relative to the design work S1 below has to do to contain it.

A4 *The eligibility gate is not operator-bound and not machine-bound.* The benchmark signature that admits a node to the payee set is produced by a process on the operator's own host under network-wide shared keys (C13), and the ArcaneOS attestation signature is derived from values compiled identically into every binary (C22). Anything PNR pays for is gated by exactly this, and by nothing stronger.

A5 *Consensus can recognise only what it can parse — and a payment to a fixed address is parseable.* Consensus can identify value received at an address it carries as a constant, and can therefore divide that value without the payer's cooperation and without any off-chain party's. What consensus cannot decide is whether a given application *ought* to have been paid for: the admission rule that binds a payment to a deployment is a FluxOS validation rule, and the FLUX-denominated price table is policy, set by on-chain messages from a multisig held personally by the four co-founders (§2, K8). The precise scope of "consensus-enforced" is thus: *the distribution of application revenue is a consensus rule; the obligation to pay, and the price, are policy* [doc: reconciliation record].

Assumption 7.2 (Fleet and revenue baseline)**.** Fleet figures are $n_{\mathsf{C}} = 3{,}247$, $n_{\mathsf{N}} = 1{,}615$, $n_{\mathsf{S}} = 1{,}627$, total 6,489 [measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03]. The settlement model and its generated data files were computed at $n_{\mathsf{C}} = 3{,}246$ [doc: scripts/07-pnr-settlement.py, run 2026-09-07]; the one-node difference changes nothing at the precision reported. The activation-scale revenue figure $V_0 = 1{,}431{,}600$ FLUX per year is the team's assumption of 1,193 applications at 100 FLUX per month, *not* a measured receipt total [intent: design intake]; the census over the same window returned 1,195 application records and 7,486 requested instances [measured: api.runonflux.io/apps/globalappsspecifications, 2026-09-03]. Every absolute FLUX-denominated figure in §7.11 is linear in V_0; the growth-rate conclusions are not.

7.2 The payment, and where the split is applied

Today an application is charged for by a transparent payment to a single policy-configured address, `t3NryfAQLGeFs9jEoeqsxmBN2QLRaRKFLUX`; FluxOS checks that the payment exists and covers the specification, and `fluxd` sees an ordinary transfer SHIPPED [measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03], [flux/ZelBack/src/services/appMessaging/messageVerifier.js:734–763] (C9). The carrier that binds a payment to what it buys is also already in the fleet: CumulusVPN pays a fixed address with an `OP_RETURN` memo, and every gateway independently derives entitlement by scanning the chain for correctly-memoed payments, with no account and no server-issued credential SHIPPED [cumulusvpn/gateway/internal/entitle/entitle.go:99–108,218–287], [cumulusvpn/clients/core-ts/src/paymentCode.ts:17–23] (C19).

PNR would keep that shape at the payer and move the address into consensus. One output, one memo, no second recipient, no fee component and no new transaction type: the settled decision is that the tenant would pay the full price once, and the division would be performed by rules in which no wallet, no agent and no FluxOS release participates PLANNED [intent: design intake] [doc: reconciliation record].

Definition 7.3 (PNR application payment, PLANNED)**.** Fix the operator-share schedule

$$\mathrm{red}(h) := \min\left\{N_{\mathrm{red}},\ \left\lfloor \frac{h - h_{PoN}}{I_{\mathrm{red}}} \right\rfloor\right\}, \qquad \rho(h) := \begin{cases} 0, & h < h_{\mathrm{PA}}, \\ \min\{\rho_{\max},\ 0.20 + \Delta\rho\,(\mathrm{red}(h) - 1)\}, & h \geq h_{\mathrm{PA}}, \end{cases}$$

with $h_{PoN} = 2{,}020{,}000$ [fluxd/src/chainparams.cpp:153], $I_{\mathrm{red}} = 1{,}051{,}200$ blocks [fluxd/src/chainparams.cpp:157], $N_{\mathrm{red}} = 20$ [fluxd/src/chainparams.cpp:158], $h_{\mathrm{PA}} = 3{,}787{,}502$, $\Delta\rho = 0.05$ and $\rho_{\max} = 0.50$ [intent: design intake]. Write $r(h) := 100\,\rho(h) \in \mathbb{Z}$.

Let sink denote the *application-payment address*: one transparent address carried as a consensus constant from height h_{PA} onward, whose identity is O14. Let m_a be the canonical serialisation of the specification of application a and H the hash already used for application identity, so $H(m_a) \in \{0,1\}^{256}$.

A *PNR application payment* for a at height $h \geq h_{\mathrm{PA}}$ is an ordinary transparent transaction of the shape PS1–PS2, over an address governed by the consensus rules PS3–PS4. The division of the four is itself the content of the design and is not cosmetic: PS1–PS2 are what a payer constructs and what FluxOS checks, PS3–PS4 are what `fluxd` applies to *everything* that arrives at the address, memo or no memo.

PS1 (one recipient, full price). The transaction carries exactly one value-bearing output paying sink, of value $v \in \mathbb{Z}_{\geq 0}$ in sat, with $v \geq \mathcal{P}(a,h)$ (Definition 7.8). The payer constructs one recipient and its own change, and performs no other arithmetic.

PS2 (memo). The transaction carries one zero-value `OP_RETURN` output whose data field is the 32-byte value $H(m_a)$. Consensus carries the memo and does not interpret it; FluxOS reads it to attribute the payment to an application, and rejects a registration whose payment is missing, mis-memoed or short of $\mathcal{P}(a,h)$. A sink payment carrying no memo is still split by PS4 — it simply buys nothing.

PS3 (the sink is unspendable). No transaction may spend an output paying sink; a block containing such a spend is invalid. Value paid to sink leaves circulation at the moment of payment.

PS4 (consensus split). On connecting the block containing the payment at height h, consensus divides the received value: $A_\Phi := \left\lfloor \frac{r(h)\,v}{100} \right\rfloor$ is credited to the fee pool Φ, and $A_F := v - A_\Phi$ is credited to the Foundation and paid through the *existing* dev-fund coinbase output. The floor is taken once, over integers, per received output, and favours the Foundation by strictly less than 1 sat per payment. The magnitude belongs at the point of definition and not only in §7.14: at activation this rule would send the Foundation **four times what it sends operators** from every payment, and ρ does not reach $\rho_{\max} = 0.50$ until the reduction of October 2032. What PS4 changes is not, at first, how much — today 100 % of the same money reaches the same address by policy — but *who can alter it*, and by what procedure. That is the honest form of the claim, and a reader who wants the fraction to be different should read §7.14, where it is compared against Akash's, rather than infer that consensus enforcement is itself the concession.

Write F_h and G_h for the sums of A_Φ and of A_F over the outputs paying sink in block h; gross receipts in that block are $F_h + G_h$, and $\mathrm{Rev}_h := \sum_{j=h_{\mathrm{PA}}}^{h} (F_j + G_j)$ is the balance held at sink. The domain of PS3 and PS4 is $h \geq h_{\mathrm{PA}}$; below the activation height a payment to the deployed sink is an ordinary spendable output, so if O14 retains the address in use today, Rev is defined from h_{PA} and not from that address's first receipt.

The three decisions inside Definition 7.3 that were open in the first draft of this specification are now settled and are recorded here rather than left as parameters: the memo binds $H(m_a)$ and is carried in an `OP_RETURN`, which closes O3; the payment is not a new transaction type and needs no new serialisation, which closes half of O18; and the split is applied by consensus rather than by the payer or by FluxOS, which closes the other half [intent: design intake] [doc: reconciliation record].

Remark 7.4 (O7 — resolved by the retirement of the shielded pools). PS4 is computable only if the value received at sink is visible to consensus, and the question was which of three regimes applied to shielded payers. It is answered by a decision taken elsewhere: both shielded pools are to be retired (§20.9), so **all settlement is transparent** and PS4 is computable unconditionally.

The domain collapses to its first element, not by argument within this section but because the alternative ceases to exist.

Worth noting for the record: an intermediate answer had permitted shielded payers with a revealed-amount output [intent: design intake], which would have made PS4 computable while hiding the payer. That is a coherent design and it is now moot. PNR is consequently simpler than any draft of this section assumed — every receipt at sink is publicly attributable, and the fee pool's inflow is auditable by anyone.

The coinbase already carries four value-bearing outputs that consensus checks: a dev-fund minimum paid to a hardcoded address, and one output per tier paid to that tier's PoN payee SHIPPED [fluxd/src/main.cpp:2877–2884], [fluxd/src/main.cpp:1231–1249], [fluxd/src/fluxnode/fluxnode.cpp:837–875] (C9). Those four outputs would be the whole of the machinery PNR needs on the paying side.

Property 7.5 (The split is a consensus rule, and the single output is why, PLANNED)**.** Under Definition 7.3 the 80 %/20 % division would be applied by `fluxd` alone. Three consequences follow, and they are stated together because they follow from one decision.

E1 *The payer could not get it wrong.* There would be one recipient and one amount, so there would be no split for a wallet to compute, mis-round or omit, and no second recipient for a wallet that exposes only one to refuse. Any wallet that can pay an address and attach an `OP_RETURN`, and any agent holding a bare wallet API, could construct a valid payment — which is the property §18's agent-native argument rests on, and which the deployed CumulusVPN payment path cited above is the existence proof for.

E2 *FluxOS could not change it.* FluxOS would check only that a payment exists, carries $H(m_a)$ and covers $\mathcal{P}(a, h)$. The fraction $\rho(h)$, the tier partition λ_t and the destination of either share would be alterable only by a consensus change, never by a FluxOS release through the channel of §12 and §14.

E3 *The coinbase would gain no output.* The Foundation share would be added to the dev-fund output and the drip to the three tier outputs, and nothing else would be added. The team's constraint that the coinbase must not grow [intent: design intake] would be met strictly, and met better than by any shape that adds an output.

Justification. E1 is immediate from PS1: the transaction has one sink output and the payer's change, and PS4 is evaluated by the validator, not by the payer. E3 is immediate from PS4 and from Definition 7.9, which route G_h and $D_{h,t}$ into outputs that already exist. E2 is the substantive one and is a statement about *where a rule can be edited.* A FluxOS release can change any predicate FluxOS evaluates; it cannot change a predicate evaluated in `ConnectBlock`, because a node running the changed rule would be rejected by every other node. Under Definition 7.3 the division is evaluated in `ConnectBlock` and the admission rule is evaluated in FluxOS, so the two are separated by the strongest boundary the system has. Note precisely what E2 does not say: it does not say the *amount* is beyond policy. The price table is set by the price-message multisig (Definition 7.8, O9) and which channels settle through sink at all is a FluxOS rule (O8). Consensus would fix the distribution; policy would fix the base. □

Property 7.6 (Supply neutrality, and a public revenue counter, PLANNED)**.** Let $S(h)$ be the issuance function of §5, and let $S'(h)$ denote circulating supply under PNR— issuance, less value locked at sink by PS3, plus the coinbase re-issuance of PS4 and of the drip. Then for all $h \geq h_{\mathrm{PA}}$,

$$S'(h) \;=\; S(h) \;-\; \Phi_h.$$

PNR would therefore create no coin and destroy none; the only deviation from §5's schedule would be the pool balance in flight, which is bounded by $KF_{\max}$ (Property 7.32, I4) and equals

30 d of the operator share at steady state. Separately, Rev_h — the balance of a single address — would be the cumulative gross application revenue settled through this path since h_{PA}.

Proof. By PS3 the value at sink, namely $\mathrm{Rev}_h = \mathrm{In}_h + \sum_{j\le h} G_j$ with $\mathrm{In}_h = \sum_{j\le h} F_j$, is unspendable, so circulating supply is reduced by exactly that amount. It re-enters through the coinbase in two streams: the Foundation share $\sum_{j\le h} G_j$ through the dev-fund output, and the operator share as the drip actually placed in tier outputs, Paid_h (Property 7.32). Hence $S' - S = \sum_{j\le h} G_j + \mathrm{Paid}_h - \mathrm{Rev}_h = \mathrm{Paid}_h - \mathrm{In}_h = -\Phi_h$, the last step by I1. For the counter: by PS1 every settled payment pays sink and by PS3 nothing ever leaves it, so its balance is the running sum of received values. □

Remark 7.7 (What the counter is worth, and what it does not cover). §5 argues that supply should be verifiable by arithmetic rather than by trust. A consensus-unspendable sink turns the application-revenue term of that arithmetic into a single number anyone can query from a block explorer, with no party privileged to publish it and none able to withhold it. The consequence for §26 is concrete: that section presently reasons from an *assumed* 100 FLUX per application per month (Assumption 7.2) [intent: design intake], and after activation the same quantity would be measured rather than assumed, which is also what would make O17 closable by observation. One qualification travels with the counter and must not be dropped: it would measure what settles through this path, so every channel excluded by O8 — credits (§16), CumulusVPN, enterprise contracts — is revenue the counter would not see and the consensus split would not touch.

Definition 7.8 (Payment function, interface, PLANNED)**.** For an application a with resource vector $\mathbf{r} = (\mathrm{cpu}, \mathrm{ram}, \mathrm{hdd}) \in \mathbb{R}^3_{\ge 0}$, instance count k, and duration $d \in [1,\ 1{,}056{,}000]$ blocks, the price is a function $\mathcal{P}\colon \mathcal{A} \times \mathbb{Z}_{\ge 0} \to \mathbb{Z}_{\ge 0}$, where $\mathcal{A}$ denotes the set of well-formed application specifications, of the form

$$\mathcal{P}(a,h) \;=\; \max\Big\{\, p_{\min},\quad k \cdot \frac{d}{L^*(h)} \cdot \big(\mathbf{p}_h^{\top}\mathbf{r} + \mathrm{surcharges}(a)\big) \Big\},$$

where $\mathbf{p}_h$ is the height-scheduled price vector and $p_{\min} = 0.01$ FLUX. The duration denominator carries a fork branch: $L^*(h) = L = 22{,}000$ blocks before PoN activation and $4L = 88{,}000$ blocks from height 2,020,000, because PoN quadrupled the block rate and the multiplier preserves wall-clock duration SHIPPED [flux/ZelBack/src/services/appRequirements/appValidator.js:864-869], [fluxd/src/chainparams.cpp:101,105]. The instance count is bounded above at 100; its lower bound is 3 in general but 1 for SPEC v8 applications from height 2,176,519, which is the value consensus-side validation enforces even though the published `deploymentinformation` endpoint continues to report 3 SHIPPED [flux/ZelBack/src/services/appRequirements/appValidator.js:818-828], [doc: reconciliation record]. The duration floor is likewise 1 block from height 1,964,447 by the same validator, where the endpoint still reports 5,000 SHIPPED [flux/ZelBack/src/services/appRequirements/appValidator.js:852-863], [flux/ZelBack/src/services/appQuery/deploymentInfoService.js:41-46]. The deployed schedule has six rows, i.e. five revisions since genesis; the current row, effective from height 1,597,156, is $\mathbf{p} = (0.03,\ 0.01,\ 0.004)$ FLUX per unit with surcharges $(\mathrm{port},\ \mathrm{scope},\ \mathrm{staticip}) = (0.4,\ 0.8,\ 0.4)$ FLUX SHIPPED [measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]. Settlement consumes $\mathcal{P}$ and never recomputes it.

The aggregation, now read from the source rather than stated as an interface. An earlier draft carried the units, rounding and multipliers of $\mathcal{P}$ as unestablished. They are established here, because a FLUX-denominated consensus split over a quantity nobody has pinned down is not a specification. In FluxOS's `appPricePerMonth`, the applicable row is the last table entry with height $< h$, and the rates carry implicit units: `cpu` is priced *per* 0.1 *core* (the core count is multiplied by ten), `ram` *per* 100 MB (divided by one hundred) and `hdd` *per* 1 GB, summed across all components of a SPEC v4-or-later composition SHIPPED [flux/ZelBack/src/

services/utils/appUtilities.js:31–33,91–115]. The three surcharges are added once each where they apply: `scope` for an enterprise or node-targeted application, `staticip` for a static-IP request, and `port` *per enterprise port* SHIPPED [flux/ZelBack/src/services/utils/appUtilities.js:116–122].

Two details matter to §7.11 and are easy to get wrong. First, **the table prices three instances**: the summed total is divided by three to obtain the per-instance price, and instances beyond the first are added on top, with a 0.5 FLUX floor per extra instance where the base price is below that and more than two extras are requested SHIPPED [flux/ZelBack/src/services/utils/appUtilities.js:124–134]. Second, rounding is a *ceiling* to 0.01 FLUX, taken twice — once on the per-instance price and again after the duration multiplier $d/L^*(h)$ is applied SHIPPED [flux/ZelBack/src/services/utils/appSpecHelpers.js:55–63]. No geolocation multiplier exists in this path. A marketplace multiplier and duration discounts of 3/6/12 % do exist, but only in the USD-denominated *quote* (§7.12, O9), never in the on-chain FLUX price that FluxOS validates a payment against — which is the quantity PS4 would divide. The aggregation is therefore no longer open.

Two consequences of Definition 7.8 deserve to be visible rather than discovered. First, the pool would receive $\rho(h)\,v$ regardless of how $\mathcal{P}$ was computed: consensus would divide whatever arrives at sink, not check that what arrives is correct. Second, any change to the price table would change pool inflow one-for-one with no consensus involvement. The table is discretionary today and has been revised five times, the CPU rate falling 100× from 3 to 0.03 FLUX per unit SHIPPED [measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]. *Consensus would fix the distribution; policy would fix the base.* That sentence is the honest form of the decentralization claim this mechanism supports — it is now stronger on the first half than the first draft of this specification could claim (Property 7.5, E2) and unchanged on the second, and §7.12 records the two discretionary levers it leaves standing: the price table itself (O9) and the set of channels required to settle through sink (O8).

7.3 Settlement shapes not taken

Two other shapes were considered against the same constraints — the coinbase must not grow, not every wallet exposes multiple recipients, adoption should be fast — and both were rejected. They are recorded because a reader will assume one of them was the easy option [doc: reconciliation record].

Not a blockchain fee. Carrying the operator share as transaction fee is the shape that looks free, and it is not, for four independent reasons. *First*, fees accrue to the block producer, so routing them into a pool instead would be a consensus change anyway — the same implementation cost as a sink, with worse properties, since a producer that includes a payment would capture part of it in the same block, which is exactly what Theorem 7.23 exists to remove. *Second*, the paid amount would become $\sum \text{inputs} - \sum \text{outputs}$, a quantity the sending wallet computes, so a change-output calculation would silently determine what the tenant paid — and a payment amount that no party states explicitly is not a payment a tenant can audit. *Third*, wallets clamp, warn on, or refuse anomalously large fees, and no fee-estimation interface is designed for a fee that is 20 % of a deployment price, so the shape would fight the tooling on every payment. *Fourth*, an ordinary transaction carrying a large fee is indistinguishable from an application payment except by its memo, so the memo would become load-bearing for *consensus* rather than for attribution — a strictly stronger requirement than PS2 makes, and one that puts a FluxOS-defined field inside a money rule.

Not a new transaction type, at least not first. A typed transaction is the most rigorous shape, and Flux has the in-house precedent: transaction versions 5 and 6 and the FluxNode start/confirm types SHIPPED [fluxd/src/fluxnode/activefluxnode.cpp:409–427]. It was not chosen as the primary path for two reasons. It would force every payer to link a Flux-specific library in order to pay, which is precisely the friction §18 exists to remove and which no ordinary wallet or agent

runtime would carry; and it would have to be rolled out across every wallet, hosted tool and agent before any payment could settle at all, making adoption serial rather than immediate. It would remain available as a later addition carrying richer fields, and it would coexist with the address-and-memo path without invalidating it PLANNED [doc: reconciliation record].

7.4 The pool as consensus state

Definition 7.9 (Fee pool and per-block transition, PLANNED)**.** The *fee pool* is a single consensus-state integer $\Phi_h \in \mathbb{Z}_{\geq 0}$ in sat, with $\Phi_{h_{\mathrm{PA}}-1} := 0$. Its value would be *committed in the coinbase* of block h and recomputed by every validator from the parent value and the block body (rule PC below). Write $\Delta_h := \Phi_h - \Phi_{h-1}$ for the per-block delta, which an implementation may cache against the block index but which carries no authority: the pool is a pure function of the chain. Fix the *drip constant* $K \in \mathbb{Z}_{\geq 1}$, denominated in blocks and settled at $K = 86{,}400$ (§7.6), and write $\mathcal{O}_h$ for the set of outputs paying sink in block h. For every block $h \geq h_{\mathrm{PA}}$:

$$D_h := \left\lfloor \frac{\Phi_{h-1}}{K} \right\rfloor \qquad \text{(drip, taken from the } \textit{parent} \text{ state)}, \tag{1}$$

$$D_{h,t} := \left\lfloor \frac{2\beta_t\, D_h}{27} \right\rfloor, \quad t \in \mathcal{T} \qquad \text{(tier partition, over the integers } 2\beta_t \in \{2,7,18\}), \tag{2}$$

$$\epsilon_h := D_h - \textstyle\sum_{t\in\mathcal{T}} D_{h,t} \in \{0,1,2\}\,\mathrm{sat} \qquad \text{(partition residue, retained in the pool)}, \tag{3}$$

$$F_h := \textstyle\sum_{o\in\mathcal{O}_h} A_\Phi(o) \qquad \text{(inflow, over the sink-paying outputs of block } h), \tag{4}$$

$$G_h := \textstyle\sum_{o\in\mathcal{O}_h} A_F(o) \qquad \text{(Foundation share of the same receipts)}, \tag{5}$$

$$\Phi_h := \Phi_{h-1} - D_h + \epsilon_h + U_h + F_h \qquad \text{(update)}. \tag{6}$$

For each tier t with $\mathcal{W}_h^t \neq \varnothing$, the existing tier output to $\mathcal{W}_h^t$ carries value $\sigma_t(h) + D_{h,t}$; the existing dev-fund output carries $\delta(h) + G_h$. One further rule closes the state:

PC (pool commitment). The coinbase of every block $h \geq h_{\mathrm{PA}}$ carries Φ_h as an 8-byte little-endian integer in sat, in the coinbase's data-bearing field — its input signature script, which holds data rather than value, so the coinbase gains no output (Property 7.32, I4). A validator recomputes Φ_h by (1)–(6) and rejects the block if the committed value differs.

U_h is the sum of $D_{h,t}$ over tiers with $\mathcal{W}_h^t = \varnothing$, retained in the pool. The tier weights are $\lambda_t = \beta_t / \sum_u \beta_u$ with $(\beta_{\mathsf{C}}, \beta_{\mathsf{N}}, \beta_{\mathsf{S}}) = (1,\ 3.5,\ 9)$ FLUX [fluxd/src/main.cpp:2886–2939], giving $(\lambda_{\mathsf{C}}, \lambda_{\mathsf{N}}, \lambda_{\mathsf{S}}) = (2/27,\ 7/27,\ 18/27) = (7.41\,\%,\ 25.93\,\%,\ 66.67\,\%)$.

Decided. Stays in Φ (Table 24). O5 — destination of $D_{h,t}$ when tier t has no confirmed node. Domain: {stays in pool, dev fund, redistributed to the other tiers}. Definition 7.9 takes the conservation-trivial default (pool) so that the invariants of §7.9 hold as written; the dev-fund option would mirror the subsidy fallback reported for PoN, and the redistribution option rewards concentration.

Decided. Stays in Φ (Table 24). O6 — destination of the partition residue $\epsilon_h \leq 2$ sat per block. Domain: {pool, dev fund}. Worth at most 0.021 FLUX per year either way; it must be fixed explicitly because an implementation difference here is a chain split (§7.9, I5).

Decided. Drip from Φ_{h-1}, the default kept (Table 24). O10 — drip ordering. Domain: {drip from Φ_{h-1}, drip after applying F_h}. Definition 7.9 drips from the parent state, which buys two properties used later: the coinbase is checkable before the block body is executed, and a block producer cannot influence its own drip. Dripping after inflow would remove one block of lag and would reintroduce same-block recapture of λ_t/K.

Remark 7.10 (O21, settled: the division is applied on arrival). The settlement decision created a question this specification had not needed to ask — at which height the Foundation share G_h is

computed — and it is answered here: **on arrival**, at the ρ in force at the receipt height, as PS4 states, and paid in the same block [intent: design intake].

The alternative considered was to let the pool receive the entire receipt and apply the division to the drip at payout, adding $\lfloor (100 - r(h))\, D_h/100 \rfloor$ to the existing dev-fund output. It is attractive on its face: it needs *one* consensus integer rather than two, and every coinbase output would remain checkable from the parent state alone, because D_h depends only on Φ_{h-1}.

It is rejected for a reason that is easy to miss. ρ is not constant — it ramps by $\Delta\rho$ at each subsidy reduction — and Φ is a single integer that cannot carry the rate that applied when each receipt arrived. Applying the division at payout therefore re-prices the entire accumulated balance at every ramp step: of order $\Delta\rho\,\Phi$, once per reduction, moved from the Foundation to operators on revenue earned under the earlier share. The ramp is a schedule for dividing revenue *as it is earned*; deferring the division to payout silently converts it into a schedule over *distributions*. That is a different policy, and not the one the ramp was specified to express.

What is given up is less than it appears. Parent-state checkability of the coinbase serves a verifier that does not hold the block body, and Section 22 establishes that no such verifier is constructible for Flux. A dev-fund output that depends on the receipts in its own block is in any case the ordinary arrangement wherever transaction fees are paid.

Construction 7.11 (PNR block connect, PLANNED)**.** INPUTS: parent pool Φ_{h-1}, read from the parent block's coinbase commitment; the block at height h with its transaction list; the PoN payee tuple $\mathcal{W}_h = (\mathcal{W}_h^{\mathsf{C}}, \mathcal{W}_h^{\mathsf{N}}, \mathcal{W}_h^{\mathsf{S}})$ returned by the existing queue; the constants K, $\{\beta_t\}$, sink, $\rho(h)$.
INVARIANT (maintained across every call): $\Phi_h = \sum_{j \le h} F_j - \sum_{j \le h} (D_j - \epsilon_j - U_j) \ge 0$.
FAILURE MODE: any step that does not hold makes the block invalid; there is no partial application. A divergent implementation is detected at h by step 6, not one block later. On `DisconnectBlock` the cached Δ_h is subtracted, restoring Φ_{h-1} exactly; if the cache is absent or suspect, Φ_{h-1} is read from the parent coinbase or recomputed from h_{PA}, and there is no repair path to invoke because there is nothing that can diverge from the chain.

1. Read Φ_{h-1}. If $h < h_{\mathrm{PA}}$, set $\Delta_h := 0$ and stop.
2. Compute D_h by (1), then $D_{h,t}$ and ϵ_h by (2)–(3). All three depend only on the parent state and on constants.
3. Scan the block's outputs for payments to sink. For each such output of value v, compute A_Φ and A_F by PS4 of Definition 7.3 at this height; accumulate F_h and G_h. Reject the block if any transaction in it spends an output paying sink (PS3).
4. For each tier t: if $\mathcal{W}_h^t \neq \varnothing$, require the coinbase tier output to equal exactly $\sigma_t(h) + D_{h,t}$; otherwise add $D_{h,t}$ to U_h. Require the dev-fund output to equal $\delta(h) + G_h$.
5. Set $\Delta_h := F_h - D_h + \epsilon_h + U_h$ and $\Phi_h := \Phi_{h-1} + \Delta_h$; cache Δ_h against the block index.
6. Require the coinbase commitment PC to equal Φ_h; reject the block on mismatch.
7. If the block is an emergency block (Assumption 7.1 A3), step 2 is skipped and $D_h := 0$, so $D_{h,t} = \epsilon_h = 0$ and the tier clause of step 4 reduces to today's rule — see Property 7.33.

Where the pool lives, and the two places it must not. O2 asked whether Φ_h is held as a per-index delta, a coinbase field or a header field; the answer is the coinbase field, PC above, with every validator recomputing rather than trusting it [doc: reconciliation record]. Three properties decide it. *Reorg-exactness would come free*, because the pool is a pure function of the chain: a reorg recomputes it, with no rollback journal and no possibility of the pool diverging from the UTXO set. *No new database would be needed*, because validators already parse the coinbase for

the dev-fund minimum and the three tier payouts SHIPPED [fluxd/src/main.cpp:1231–1249], [fluxd/src/fluxnode/fluxnode.cpp:837–875], so the parsing and enforcement path exists and is exercised every block. And *anyone could check it from chain data alone*, which is what §5's supply-transparency argument and Property 7.6 both require.

A *separate LevelDB store* is rejected on this network's own precedent. The FluxNode registry lives in its own store and needed a dedicated crash-recovery marker, precisely because the coins database and the node database can diverge after an unclean shutdown, together with a repair RPC to rebuild it SHIPPED [fluxd/src/fluxnode/fluxnodecachedb.h:22–24], [fluxd/src/rpc/fluxnode.cpp:2676]. That pattern is tolerable for a registry that can be rebuilt from chain data. It is not tolerable for money, where a divergence between the store and the chain is a wrong payout rather than a stale index.

A *header field* is rejected for a different reason. It is the most rigorous option, but a header change is the widest-blast-radius fork available to this chain, and it would further complicate the light-client position of §22 — which is already foreclosed by the absence of a node-set commitment, by fixed 2^{40} chainwork (Assumption 7.1 A2) and by the emergency path (A3) (C26). One circumstance would reverse this: if a node-set commitment is ever added to the header to make light clients constructible (§8, §22), that is the moment to reconsider carrying the pool commitment alongside it, since the fork is being taken anyway.

Proposition 7.12 (Closed form, steady state and half-life, PLANNED)**.** *Relax the floors in* (1)–(3) *(they retain at most* 3 sat *per block in the pool and therefore only* understate *the pool). Write* $q := 1 - 1/K \in (0,1)$ *for* $K \geq 2$*. Then for all* $h \geq h_0 \geq h_{\mathrm{PA}}$*,*

$$\Phi_h \;=\; q^{\,h-h_0}\Phi_{h_0} \;+\; \sum_{j=h_0+1}^{h} q^{\,h-j} F_j .$$

Under constant inflow $F_j \equiv F$ *this gives* $\Phi_h = q^h\Phi_0 + FK\,(1-q^h)$*, hence*

$$\Phi^* \;:=\; \lim_{h\to\infty}\Phi_h \;=\; F\,K, \qquad D^* \;=\; \Phi^*/K \;=\; F.$$

The half-life of a perturbation — equivalently, of the pool under zero inflow — is

$$h_{1/2} \;=\; \frac{\ln 2}{-\ln(1-1/K)} \;=\; K\ln 2 + \tfrac{1}{2}\ln 2 + O(1/K).$$

A single contribution A_Φ *made in block* h_0 *is paid out as* $A_\Phi\, q^j/K$ *in block* h_0+1+j *for* $j \geq 0$*, and* $\sum_{j\geq 0} A_\Phi q^j/K = A_\Phi$*.*

Proof. With the floors relaxed, (6) reads $\Phi_h = q\,\Phi_{h-1} + F_h$; unrolling from h_0 gives the stated sum. For $F_j \equiv F$ the sum is geometric, $F\sum_{i=0}^{h-h_0-1} q^i = FK(1-q^{h-h_0})$ since $1-q = 1/K$. Letting $h \to \infty$ with $|q| < 1$ gives $\Phi^* = FK$, and $D^* = \Phi^*/K = F$. For the half-life, solve $q^n = 1/2$ for n, giving $n = \ln 2/(-\ln q)$; expanding $-\ln(1-1/K) = 1/K + 1/(2K^2) + O(K^{-3})$ gives $n = K\ln 2\,(1 + 1/(2K) + O(K^{-2}))$, i.e. the stated expansion. The impulse response is the F_j term of the closed form with a single non-zero $F_{h_0} = A_\Phi$, and its sum over j is $A_\Phi \cdot (1/K) \cdot (1-q)^{-1} = A_\Phi$. □

Remark 7.13 (The pool is a delay line, not a treasury). Proposition 7.12 says that at steady state the drip equals the inflow: everything paid in is paid out, with lag. Nothing accumulates permanently, nothing is burned, and there is no balance that anyone could later choose how to spend — the value held at sink is not a treasury either, because PS3 makes it unspendable by every party including the Foundation, and its only exit is the coinbase schedule. The pool is a delay line with time constant K blocks. This framing matters at scale: at parity-scale revenue (§7.11, 31.9M FLUX per year at $\rho = 0.5$) the accumulator would hold about 1.31M FLUX at the recommended K, and a reader who mistakes that number for a treasury will draw exactly the wrong conclusion about who controls it. Nobody controls it; it is 30 d of pipeline.

At $K = 86{,}400$ blocks and $\Delta_{PoN} = 30\,\text{s}$ [fluxd/src/chainparams.cpp:105], the fill from zero would be $D_h/F = 1 - q^{h-h_{\text{PA}}}$: 3.3 % after one day, 20.8 % after a week, 63.2 % after K blocks, 95.0 % after 90 d [doc: mechanism specification]. PNR would therefore ramp over roughly a quarter rather than switch on, and the paper should not describe activation as a switch.

Nor should the activation announcement. These four figures belong in it verbatim, before activation rather than after. An operator who has been told that PNR pays 20 % of application revenue, and who observes 3.3 % of it on day one, will reasonably conclude the mechanism is broken; the same operator told in advance that the pool fills on a thirty-day time constant will read the same observation as confirmation that it works. This is the one place in PNR's deployment where the difference between a successful launch and a credibility problem is a communications decision rather than a design one, and it is recorded here so that it is nobody's oversight.

Decided. No seed: $\Phi_{h_{\text{PA}}-1} = 0$ (Table 24). O11 — initial seed $\Phi_{h_{\text{PA}}-1}$. Domain: $\{0,$ a Foundation-funded seed$\}$. A seed removes the ninety-day ramp; it is also a one-off Foundation transfer to operators and would have to be labelled as one.

7.5 Fairness over a rotation

Proposition 7.14 (Rotation, SHIPPED)**.** *Under the queue rule of Assumption 7.1 A1, with a static confirmed set of n_t nodes in tier t, the payee sequence of tier t is periodic with period $R_t = n_t$ and every node in the tier is paid exactly once per period.*

Proof. Order the tier's nodes by their sort key (last-paid height, or confirmation height for a node never paid). The front node is paid at block h and its key becomes h, which strictly exceeds every other key, since every other node was either paid at some height $< h$ or confirmed at some height $< h$. It therefore moves to the back, and the relative order of the remaining $n_t - 1$ nodes is unchanged. Each block thus rotates the sequence by exactly one position; after n_t blocks the sequence is the original. Hence the period is n_t and each node is paid exactly once within it. □

Remark 7.15 (Churn, and what the chain is actually doing). A node joining is keyed by its confirmation height and enters at the back; a node leaving — by spending its collateral, or by failing to re-confirm within 640 blocks — shortens the cycle SHIPPED [fluxd/src/fluxnode/fluxnode.h:31–34]. In a window of n_t blocks a node is therefore paid at most $1 +$ (departures) times. Proposition 7.14 is not a modelling assumption: at chain tip 2,917,115 the maximum observed tip − last-paid height was 3,202 against 3,247 Cumulus nodes, 1,597 against 1,615 Nimbus, and 1,631 against 1,627 Stratus [measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03]. Cumulus and Nimbus sit inside one rotation; Stratus exceeds it by four blocks, consistent with four joins during that cycle at the measured median tenure of 67 d. Rotation lengths at 30 s are 27.06 h, 13.46 h and 13.56 h respectively.

Proposition 7.16 (Equal income within a tier, PLANNED)**.** *Assume Proposition 7.14 and zero churn. Over M full rotations every node in tier t would receive exactly M PNR payments, and for any two nodes i, j in the tier with cumulative PNR incomes I_i, I_j,*

$$\frac{|I_i - I_j|}{\min(I_i, I_j)} \;\le\; s_t \;:=\; \max_{\text{rotation}} \frac{\max_h D_h}{\min_h D_h} - 1,$$

where the maximum and minimum are taken over the blocks of one rotation. The mixing bound is one rotation, i.e. R_t blocks.

Proof. By Proposition 7.14 each rotation gives each node of tier t exactly one payment, of size $\lambda_t D_h$ for the block h at which it is the payee. Within one rotation any two such payments differ by at most a factor $1 + s_t$ by the definition of s_t. Summing M rotations, $I_i = \sum_{k=1}^{M} \lambda_t D_{h_k^i}$ and likewise for j, and a term-by-term comparison preserves the factor. Dividing by $\min(I_i, I_j)$ gives the claim. □

Remark 7.17 (The bound does not vanish with averaging). A node's phase within the rotation is fixed — it is paid at the same position every cycle — so an inflow pattern periodic with the rotation could favour one phase systematically. Averaging over many rotations therefore does *not* drive the difference to zero; it remains bounded by s_t and no better. This is why s_t must be small in absolute terms, which is the subject of §7.6, rather than small on average.

Proposition 7.18 (Cross-tier income is issuance-shaped, not collateral-proportional, PLANNED)**.** *Per-block-averaged PNR income per node in tier t is $\lambda_t \bar{D}/n_t$, where $\bar{D}$ is the mean drip. Hence for tiers t, t',*

$$\frac{I_t}{I_{t'}} = \frac{\beta_t}{\beta_{t'}} \cdot \frac{n_{t'}}{n_t}, \qquad \textit{and} \qquad \frac{\textit{PNR income}}{\textit{PoN issuance income}} = \frac{\bar{D}}{\frac{13.5}{14}\,\sigma_{PoN}}$$

for every *node, the same constant across all tiers.*

Proof. By Proposition 7.14 each node of tier t is paid once every n_t blocks, and its payment is λ_t of that block's drip; averaging over blocks gives $\lambda_t \bar{D}/n_t$ per node per block. Taking the ratio for two tiers, $\bar{D}$ cancels and $\lambda_t/\lambda_{t'} = \beta_t/\beta_{t'}$ by Definition 7.9. For the second identity, PoN issuance income per node in tier t is likewise $\lambda_t \cdot \frac{13.5}{14}\sigma_{PoN}/n_t$ per block, because the drip and the subsidy are paid to the same payees in the same tier proportions and the dev-fund remainder is 0.5/14 of the subsidy [fluxd/src/main.cpp:2877–2884]; the ratio of the two expressions is independent of t. □

Remark 7.19 (The correction, stated without softening). Substituting the measured fleet into Proposition 7.18: Nimbus-to-Cumulus is $3.5 \times 3{,}246/1{,}615 = 7.04$, and Stratus-to-Cumulus is $9 \times 3{,}246/1{,}627 = 17.96$, i.e. **a Stratus node would earn** 18.0× **a Cumulus node while posting** 40× **the collateral** ($c_{\mathsf{C}} = 1{,}000$ FLUX, $c_{\mathsf{N}} = 12{,}500$ FLUX, $c_{\mathsf{S}} = 40{,}000$ FLUX [fluxd/src/fluxnode/fluxnode.h:57–63]). Return on collateral relative to Cumulus is $7.04/12.5 = 0.563$ for Nimbus and $17.96/40 = 0.449$ for Stratus: **per** FLUX **of collateral, Cumulus is the better-compensated tier, by a factor of** 2.23 **over Stratus.** The design intake's phrase "proportional to collateral across tiers" is therefore incorrect and is not used anywhere in this paper [intent: design intake] (C24). What PNR income *is* exactly proportional to is PoN issuance income — it is issuance-shaped — so the tilt toward Cumulus on a return-on-collateral basis is a pre-existing property of PoN that PNR would inherit and reinforce.

7.6 Residual unfairness, quantified

Proposition 7.16 bounds within-tier inequality by s_t but does not make it zero, because the pool decays between the first and last payee of a rotation. Two effects set s_t.

Proposition 7.20 (Intra-rotation spread, PLANNED)**.** *Within a rotation of length R under zero inflow, the node paid at position $i \in \{0, \dots, R-1\}$ receives $\lambda_t \Phi_{h_0} q^i/K$, so first-versus-last payee gives*

$$s^{\downarrow} = q^{-(R-1)} - 1 \approx e^{R/K} - 1.$$

A single contribution A_Φ arriving mid-rotation raises the pool multiplicatively, so the node paid just after it receives $(1 + A_\Phi/\Phi)$ times the node paid just before; at steady state $\Phi^* = FK$ this upside jump is $\rho\mathcal{P}/(FK)$. Combining both sides, with all of a rotation's inflow arriving as a lump at its end,

$$s \le e^{R/K}(1 + R/K) - 1 \approx 2R/K.$$

Proof sketch. The decay term is Proposition 7.12 with $F \equiv 0$: Φ is multiplied by q each block, so payments at positions 0 and $R-1$ stand in ratio $q^{-(R-1)}$, and $q^{-n} = e^{-n\ln(1-1/K)} = e^{n/K + O(n/K^2)}$. The upside term is immediate from $D_h = \Phi_{h-1}/K$ being linear in Φ. The combined bound multiplies the worst decay factor by the worst single-lump jump $1 + RF/(FK)$ and is loose by construction: it assumes the entire rotation's inflow arrives in one block at the least favourable moment. A tight bound would require a distributional assumption on F_h that this specification does not make. □

Evaluated against the longest rotation, Cumulus at $R_C = 3{,}246$ blocks [doc: paper/data/pnr_drip.dat]:

K (blocks)	$h_{1/2}$ (days)	pool decay per Cumulus rotation	$s^{\downarrow}$ (first vs. last, zero inflow)	loose steady-state bound	upside jump of one 1,000 FLUX payment at V_0
3,246	0.78	63.22 %	171.79 %	443.7 %	22.6 %
10,000	2.41	27.72 %	38.34 %	83.3 %	7.3 %
32,460	7.81	9.52 %	10.51 %	21.6 %	2.3 %
64,920	15.62	4.88 %	5.13 %	10.4 %	1.1 %
86,400	**20.79**	3.69 %	3.83 %	7.7 %	0.85 %

Table 15: Residual within-tier unfairness as a function of the drip constant K, against the measured Cumulus rotation $R_C = 3{,}246$ blocks. Steady-state pool at V_0 and $\rho = 0.20$: $\Phi^* = F_0 K$ with $F_0 = 0.27237$ FLUX per block, giving 884/2,724/8,841/17,683/23,533 FLUX down the column. Computed by `scripts/07-pnr-settlement.py`, 2026-09-03.

At activation-scale revenue the *upside* term, not the decay term, would dominate the realised spread: with $\Phi^* \approx 23{,}533$ FLUX at $K = 86{,}400$, a 100 FLUX payment moves the drip 0.085 %, a 1,000 FLUX payment 0.85 %, a 10,000 FLUX payment 8.5 %, and a 100,000 FLUX payment 85 % — and deployments at the upper end are constructible, since $k \leq 100$ and $d \leq 1{,}056{,}000$ blocks [measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]. The decay term is what a larger K controls; the upside term shrinks only as revenue and therefore Φ^* grow. At parity-scale revenue the same 10,000 FLUX payment would move the pool 0.4 %. Both statements belong together, and neither should be quoted without the other.

Fleet growth at fixed $K = 86{,}400$ raises $s^{\downarrow}$: 3.83 % at 3,246 Cumulus nodes, 5.96 % at 5,000, 7.80 % at 2×, 11.93 % at 3×, 16.21 % at 4×, and 26.05 % at 20,000 [doc: scripts/07-pnr-settlement.py].

The last row is not a hypothetical, and the review trigger recommended below should be read in that light. The fleet has been substantially larger than it is now, and Cumulus is the tier that grows first when it grows, because it is the cheapest to enter. **A return to a fleet size this network has already sustained would fire the 2× trigger**, so the trigger should be treated as a scheduled review of a constant that is fixed by consensus — and therefore revisable only by network upgrade — rather than as a remote contingency. Recording that now costs nothing; discovering it at 2× costs a hard fork under time pressure.

The value, and what it costs. $K = 86{,}400$ blocks — exactly 30 d at 30 s spacing — giving a decay-side spread of 3.83 %, a 20.79 d half-life, and the recapture bound proved in §7.7, with headroom for fleet growth. The costs are stated rather than netted: demand changes would reach operators with a thirty-day time constant; PNR would ramp from zero over about ninety days at activation unless seeded; and the accumulator would hold roughly 1.3M FLUX at parity-scale revenue, which is a delay line and not a treasury (Remark 7.13). The value $K = R_C$ that the intake first suggested is rejected on the 171.79 % figure in Table 15. K is not to be adaptive to node count: node count is itself gameable by registration, and a fixed constant with margin dominates [intent: design intake].

Ratified. $K = 86{,}400$ blocks is settled and O1 is closed [intent: design intake]. The trade-off it resolves does not disappear with the decision and is restated once so that a later revision starts from it: spread $\approx R_C/K$ and targeted recapture $\approx \lambda_t/K$ both fall with K, while lag, activation ramp and steady-state pool size FK all rise with it, over the domain $[10^4,\ 2 \times 10^5]$ blocks in which the constant was chosen. What remains open is not the constant but the guard on it under fleet growth (O12), and the constant is fixed by consensus, so revisiting it would be a network upgrade rather than a configuration change.

Decided. A documented review trigger at 2×, not an automatic rule (Table 24). O12 — fleet-growth guard on K. Domain: {none, a documented hard-fork review trigger}. At 4× the current Cumulus count the decay-side spread reaches 16 %. An automatic adaptive rule is excluded (it is a manipulation surface); a documented trigger is not.

7.7 The timing attack, and why the drip defeats it

Definition 7.21 (Payout-timing attack, PLANNED)**.** Let $\mathcal{Z}$ be an operator controlling m_t confirmed nodes in tier t, with a capital budget $\mathcal{C}_{\mathcal{Z}}$ sufficient to have registered them, and let $\mathcal{Z}$ also be a tenant — it deploys its own applications and pays for them. By Assumption 7.1 A1 the payee sequence is a public deterministic function of chain state, so $\mathcal{Z}$ knows every future height h at which one of its nodes satisfies $\mathcal{W}_h^t \in \mathcal{Z}$. The *payout-timing attack* is: submit a payment of size $\mathcal{P}$ so that it is included in block $h-1$, where h is such an aligned height, and collect whatever share of that payment the settlement rule returns at h. The attack costs nothing — $\mathcal{Z}$ times a payment it would have made anyway — and requires waiting at most one rotation, i.e. at most 27.06 h, for an aligned height.

Proposition 7.22 (Recapture under next-block payout, PLANNED)**.** *Under the intake's first-cut settlement rule — fees collected in block $h-1$ distributed in block h to $\mathcal{W}_h$ by tier ratio — the attack of Definition 7.21 recaptures*

$$\mathrm{recap}_{\mathrm{next}} \;=\; \rho\,\mathcal{P}\sum_{t\in\mathcal{T}}\lambda_t\,\mathbf{1}\left[\mathcal{W}_h^t\in\mathcal{Z}\right].$$

Proof. Immediate from the rule: the operator share $\rho\mathcal{P}$ of the block-$(h-1)$ payments is split across the three tier payees of block h in proportions λ_t, and $\mathcal{Z}$ collects the terms whose payee it controls. □

$\mathcal{Z}$ controls	of the *operator share*	of the payment, $\rho = 0.20$	of the payment, $\rho = 0.50$
one Cumulus node	7.41 %	1.48 %	3.70 %
one Nimbus node	25.93 %	5.19 %	12.96 %
one Stratus node	66.67 %	13.33 %	33.33 %
one node of each tier, aligned	100 %	20 %	50 %

Table 16: Targeted-block recapture under next-block payout. **The intake's headline figures of** 66.7 % **and** 100 % **are fractions of the *operator share*, not of the payment**; as fractions of what the tenant pays they are 13.3 % and 20 % at activation, rising to 33.3 % and 50 % at the cap [intent: design intake].

With corrected magnitudes the intake's conclusion stands: every operator holding a Stratus node would obtain a 13 % to 33 % discount on its own hosting, and an operator holding one node of each tier would host free at activation. The operator share would become a rebate to operator-tenants rather than income. Next-block payout must not ship.

Theorem 7.23 (Timing resistance of the drip, PLANNED)**.** *Under Definition 7.9 the attack of Definition 7.21 recaptures, in the targeted block h, exactly*

$$\frac{\mathrm{recap}_{\mathrm{drip}}(h)}{A_\Phi} \;=\; \frac{\lambda_t}{K}, \qquad A_\Phi = \rho\,\mathcal{P}.$$

For a single Stratus node at $K = 86{,}400$ this is $\lambda_S/K = (2/3)/86{,}400 = 7.72\times10^{-6}$, i.e. 0.00077 % ***of the operator share*** *and 1.54×10^{-6} of the payment at $\rho = 0.20$. Requirement H5 of the mechanism — targeted recapture below 10^{-4} of the operator share for a single node — is met with three orders of magnitude to spare. Moreover, the advantage of* perfect timing *over* random timing *is at most $1-q^{R_t}\approx R_t/K$ of the long-run dividend, which for one Stratus node at $K = 86{,}400$ is 1.9 % × 0.041 % ≈ 8×10^{-6} of the operator share.*

Proof. $\mathcal{Z}$'s payment is included in block $h-1$, so by (4)–(6) its contribution A_Φ is part of Φ_{h-1}. By (1) block h drips $D_h = \lfloor \Phi_{h-1}/K \rfloor$, of which the tier-$t$ output receives $\lambda_t D_h$ up to the integer residue. The drip is linear in Φ_{h-1}, so the part of D_h attributable to A_Φ is A_Φ/K and the part reaching $\mathcal{Z}$'s tier-t node is $\lambda_t A_\Phi/K$. Substituting $\lambda_{\mathsf{S}} = 18/27 = 2/3$ and $K = 86{,}400$ gives the numeric claim.

For the second statement: $\mathcal{Z}$'s node is paid again at heights $h + jR_t$, $j \geq 1$, and by the impulse response of Proposition 7.12 recovers $\lambda_t A_\Phi q^{jR_t}/K$ each time. Summing,

$$\sum_{j\geq 0} \frac{\lambda_t A_\Phi q^{jR_t}}{K} = \frac{\lambda_t A_\Phi}{K\,(1-q^{R_t})} \;\approx\; \frac{\lambda_t A_\Phi}{R_t} = \frac{\lambda_t}{n_t} A_\Phi,$$

using $1 - q^{R_t} \approx R_t/K$ for $R_t \ll K$ and $R_t = n_t$ from Proposition 7.14. This is exactly the un-timed dividend of Proposition 7.24 below. An operator who pays at a uniformly random phase $U \sim \mathrm{Unif}[0, R_t)$ receives the same series delayed, i.e. multiplied by $q^U \geq q^{R_t}$. The ratio of best to worst phase is therefore at most q^{-R_t}, and the excess of perfect over random timing is bounded by $1 - q^{R_t} \approx R_t/K = 3{,}246/86{,}400 = 3.76\,\%$ of the dividend in the worst case and $1.9\,\%$ at the Stratus rotation $R_{\mathsf{S}} = 1{,}627$. □

The drip does not merely shrink the attack; it removes the value of knowing the schedule. Knowing the schedule perfectly is worth less than one part in 10^4 of the operator share of a single payment.

Proposition 7.24 (Long-run dividend, PLANNED)**.** *Because the pool pays out everything it takes in (Property 7.32, I1) in proportion to the drip, a contributor recovers a fixed share of its own payment over the long run* regardless of timing. *Define the adversary's drip weight*

$$w_{\mathcal{Z}} \;:=\; \sum_{t\in\mathcal{T}} \frac{m_t\,\lambda_t}{n_t} \;\in\; [0,1].$$

Then the long-run recapture of a payment $\mathcal{P}$ is $w_{\mathcal{Z}}\,\rho\,\mathcal{P}$.

Proof. By Proposition 7.12 every satoshi of $A_\Phi = \rho\mathcal{P}$ is eventually dripped. By Proposition 7.14 node i in tier t receives a fraction λ_t/n_t of all drip in the long run, since it takes λ_t of the drip once every n_t blocks. Summing over $\mathcal{Z}$'s nodes gives $w_{\mathcal{Z}}$, and multiplying by A_Φ gives the claim. Since the weight is independent of the height at which the contribution is made, timing cannot change it. □

$\mathcal{Z}$	$w_{\mathcal{Z}}$, of the operator share	of payment, $\rho = 0.20$	of payment, $\rho = 0.50$
one Cumulus node	0.0023 %	0.0005 %	0.0011 %
one Nimbus node	0.0161 %	0.0032 %	0.0080 %
one Stratus node	0.0410 %	0.0082 %	0.0205 %
one node of each tier	0.0593 %	0.0119 %	0.0297 %
largest measured identity (237 Stratus)	9.71 %	1.94 %	4.86 %
worst case for the 479-node identity, if all Stratus	19.63 %	3.93 %	9.81 %

Table 17: Long-run dividend $w_{\mathcal{Z}}$ by holding. The largest single identity in the measured census holds 237 nodes and 10.71 % of collateral-weighted voting power; the most nodes held by any one identity is 479 [measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03].

The honest reading of Table 17: this is not an attack. It is the same fraction that identity would receive of *everyone else's* payments, it cannot be increased by timing (Theorem 7.23), and it does not distort placement, because placement is not price-based (§9). But it is a statement the paper should make plainly rather than leave for a reader to derive: **PNR would give the largest operators a** 1.9 % **to** 4.9 % **discount on their own hosting**, exactly as PoN

issuance already gives that same identity 9.71 % of block rewards SHIPPED [measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03]. The figure is small because $\rho \le \rho_{\max} = 1/2$ and because the largest identity holds 14.57 % of one tier; it would not stay small under further concentration, and the concentration measurements of §3 — a Gini coefficient of 0.8375 over payment addresses, four identities reaching 25 % — belong next to it.

Proposition 7.25 (Wash deployment is a strict loss at every weight, PLANNED)**.** *An adversary paying $\mathcal{P}$ to deploy to itself has net payoff*

$$\Pi_{\text{wash}} \;=\; -\mathcal{P}\,(1 - w_{\mathcal{Z}}\,\rho) \;\le\; -\tfrac{1}{2}\,\mathcal{P} \qquad \textit{for every } w_{\mathcal{Z}} \le 1,\ \rho \le \rho_{\max} = \tfrac{1}{2}.$$

Proof. By PS1 the whole of $\mathcal{P}$ leaves the adversary at sink, and by PS4 the Foundation share $A_F = (1-\rho)\mathcal{P}$ is credited to the Foundation and never returns to it. By Proposition 7.24 it recovers $w_{\mathcal{Z}}\rho\mathcal{P}$ of the pool share and nothing else. Netting gives the expression; maximising the recovered term over $w_{\mathcal{Z}} \le 1$ and $\rho \le 1/2$ gives $\Pi_{\text{wash}} \le -\mathcal{P}(1 - 1/2)$. □

Remark 7.26. Even an adversary owning *every node on the network* would lose at least half of every wash payment at the cap. The largest measured identity would lose 95.14 % at the cap and 98.06 % at activation; a single Stratus operator loses 99.98 %. The defence is dilution, and the cap on ρ is what makes it a strict loss at every weight — note that the roadmap's original defence, "rewards are funded only by that workload's own burn," presupposed a host-pays model and a burn, and neither exists in the settled design (C1). One asymmetry survives and should be stated: a *Foundation* workload paying $\mathcal{P}$ returns $(1-\rho)\mathcal{P}$ to the Foundation, so its effective cost would be $\rho\mathcal{P}$ — 20 % of list price at activation, 50 % at the cap. Operators are indifferent, since they receive $\rho\mathcal{P}$ either way, so this is a distortion rather than a leak: the party that sets the price table would be exposed to one fifth of it.

7.8 What PNR does and does not incentivise

This is the sharpest question about the design and it is answered here rather than buried. It is also the one most easily stated wrongly, so the distinction comes first.

PNR would be a strong incentive to *operate a node*. It would be the mechanism by which application revenue reaches operators at all — today none does — and on the team's own projection it would become the largest component of operator income [intent: design intake] PLANNED. Nothing below qualifies that.

What the property proves is narrower: income does not vary with *which* applications a particular node happens to run. That is not a free-riding opportunity, because **an operator does not choose what to host**. FluxOS's spawner selects a candidate at random from the network-wide application list SHIPPED [flux/ZelBack/src/services/appLifecycle/appSpawner.js:311], and no opt-out, allowlist or per-application refusal exists anywhere in the deployed orchestration code. Placement is the network's decision, not the operator's. An operator who wanted to collect PNR while hosting nothing could not simply decline work; they would have to appear eligible while providing no real capacity — which is a statement about the *eligibility gate*, not about the reward.

What that does and does not imply for sequencing, at the right magnitude. It is tempting, and this paper was tempted, to conclude that hardening attestation is a *precondition* for PNR because PNR raises the payoff to passing the gate without doing the work by exactly what it pays. The first half is true; the inference is not, and the arithmetic is in this section already. At activation PNR would pay 286,320 FLUX per year against 6,386,040 FLUX per year of issuance already paid through *the same unbound gate* (§7.11). PNR would raise the prize for faking eligibility by 4.5 %. **The gate problem is a PoN problem today, and 95.7 % of it exists whether or not PNR ever ships.**

Two conclusions follow, and the second is the one this paper now carries.

1. Hardening the eligibility gate is the network's most urgent security work *on its own merits*, independent of PNR. Framing it as a PNR precondition understates it by more than an order of magnitude and invites the reply that a 4.3 % supplement is not worth blocking.

2. The defensible sequencing rule is therefore not "PNR waits" but **PNR's ramp tracks attestation's completion**: activation at $\rho = 0.20$ adds little to an exposure that already exists, while each step of the ramp toward $\rho_{\max} = 0.50$ moves a larger share of operator income behind a gate whose strength should have risen to meet it. This is a schedule coupling, not a blocking dependency, and it does not hand an opponent of PNR a principled veto.

Section 25 and Section 27 state the same edge in the same terms. What makes the coupling achievable rather than aspirational is the finding in §14.1.1: the image-bound PCR policy the hardening needs is already generated on every capable machine, and what is missing is a key bound to it and a verifier that reads it.

Property 7.27 (Zero marginal hosting incentive, PLANNED)**.** Under Definition 7.9 a node's PNR income is a function of its tier and its position in the PoN queue and of nothing else. In particular

$$\frac{\partial\,(\text{node } i\text{'s PNR income})}{\partial\,(\text{applications hosted by node } i)} \;=\; 0,$$

identically, at every parameter value. The marginal *cost* of hosting — CPU, memory, bandwidth, disk, and under §23 the operational burden of evictions — is strictly positive. Therefore the private incentive of every node is to remain in the paid set at minimum real cost — which, given that placement is not the operator's to refuse, is a claim about how cheaply a node can satisfy the eligibility gate, not about whether it will accept the work it is given.

Proof. By requirement H4 the PNR payees of block h are exactly $\mathcal{W}_h$, and by Assumption 7.1 A1 $\mathcal{W}_h$ is determined by the queue keys, which are functions of last-paid and confirmation heights only. Neither (1) nor (2) references any placement, metering or workload variable. Hence income is invariant to hosting, and the derivative is zero. □

Remark 7.28 (The collective incentive is real and individually negligible)*.* Pool inflow depends on aggregate demand, and aggregate demand depends on the network hosting well, so hosting is a public good among 6,489 nodes. Quantify the private stake. Suppose one node's refusal to host reduced network revenue by its 1/6,489 share, i.e. $\Delta V = 220.7$ FLUX per year at V_0. That node's own income would fall by $w\,\rho\,\Delta V$, which at $\rho = 0.20$ is 0.001 FLUX per year for a Cumulus node, 0.007 for Nimbus and 0.018 for Stratus; at parity-scale revenue (22×) and at the cap, 0.06–1.0 FLUX per year. **The collective interest in the network hosting well is real; the private share of it is negligible at every parameter value in range.** This is not a tuning problem; it is structural to the settled decision to pay all nodes.

What that does *not* say — and an earlier draft of this remark did say — is that free-riding dominates. Free-riding would require the ability to decline the work, and no such ability exists (§7.8). The correct reading is narrower and points somewhere useful: PNR supplies no incentive gradient that rewards hosting *well*, so what protects hosting quality is entirely the eligibility gate and the 640-block re-confirmation that removes nodes failing it SHIPPED [fluxd/src/fluxnode/fluxnode.cpp:790–820] — which is the same conclusion the sequencing argument above reaches, from the other side.

Remark 7.29 (The correct framing, and the cross-section consequence)*.* PNR is an incentive to *be a node*, not an incentive to *host*. That is a defensible design — the network wants capacity available, and availability is what collateral and eligibility secure — but it must be said, because a reader who assumes that revenue-funded rewards align operators with tenants will be wrong.

The decoupling is deliberate, and the reason is anti-gaming rather than expediency. Asked directly whether some fraction of PNR should be conditioned on hosting, the team's position is that it should not: the network decides placement, so paying only the nodes an application landed on "would be a luck game and can be also misused by node ops that will try to play it trying to force installation" [intent: design intake] (PLANNED). That reasoning is sound and follows from a mechanism this paper documents elsewhere. Placement is opportunistic self-selection with a 90 s broadcast collision wait resolved earliest-broadcast-wins (§9.3); attaching income to placement would convert that race into a financially-motivated one, rewarding whoever is fastest to claim work rather than whoever serves it best, and giving every operator a standing reason to manipulate what lands on them. Under opportunistic placement, hosting-conditioned pay would select for latency and manipulation, not for service quality.

So Definition 7.27 should be read as a *choice with a stated cost*, not as an oversight. What the choice does not do is remove the cost: the whole hosting guarantee is transferred onto the eligibility gate. A node is a PoN payee, and therefore a PNR payee, only if it is confirmed with a `fluxbench`-signed transaction and, in the target architecture, attested (§14). PNR's hosting guarantee is precisely the strength of that gate and nothing more.

By Assumption 7.1 A4 that gate is weaker than it looks in exactly the relevant way: the benchmark is self-reported by a process on the operator's own host under network-wide shared keys (C13), and the attestation signature is not machine-bound (C22). The two findings compose badly. **PNR would raise the payoff to passing the gate without doing the work, by exactly what it pays** — 4.29 % of operator income at activation (§7.11), more as the ramp advances (C24).

The complementary figure is what turns that observation into a schedule rather than a veto, and it is stated here so the two never travel apart: the remaining 95.71 % of operator income at activation is *issuance already paid through the same gate*. The gate problem is not created by PNR and would not be removed by cancelling it. §4 and §14 establish the gate's properties; §25 classes the remaining attestation work as engineering once it is scoped to the TPM population; and §27 couples PNR's *ramp* to that work rather than blocking activation on a bonded network that is neither necessary for the fix nor deliverable by Table 18's first row. Attested execution is not adjacent to PNR; it is what would make PNR pay for something, and it is what already makes issuance pay for something.

Open Problem 7.30 (Hosting-conditioned eligibility, dependency D1). Specify a gate admitting a node to $\mathcal{W}$ that is conditioned on hosting — FluxOS running, resources real, placed applications actually executing — and that is bound to a machine rather than to a shared secret. Interfaces required: a per-node attestation whose signing key is derived from a hardware root; a verifier reachable by a tenant, not only by loopback mTLS; and a liveness predicate consumable by the confirmation transaction. The construction is open and belongs to §14 and §25; without it PNR pays for membership, and membership is only as real as the gate.

Conjecture 7.31 (No hosting incentive is reachable inside the settled constraints)**.** *Within the settled constraints — beneficiaries are all confirmed nodes (H4), and there is no metering or attestation dependency between payment and placement (P7) — no modification of the settlement rule creates a positive marginal hosting incentive. Payment-conditioned hosting is excluded by P7; placement is opportunistic self-selection and carries no reputation signal; and no collateral slashing for non-hosting exists anywhere in the evidence corpus. The conjecture is stated rather than proved because a proof would require a formal characterisation of the admissible settlement-rule space, which this specification does not give.*

7.9 Invariants

Property 7.32 (Settlement invariants, PLANNED)**.** Let $\mathrm{In}_h := \sum_{j \le h} F_j$ and $\mathrm{Paid}_h := \sum_{j \le h}(D_j - \epsilon_j - U_j)$, the drip actually placed in coinbase outputs. Under Definition 7.9 and Construction 7.11:

I1 *Conservation.* $\Phi_h = \mathrm{In}_h - \mathrm{Paid}_h$ exactly, for all $h \geq h_{\mathrm{PA}}$. Settlement would neither create nor destroy FLUX; no term of the issuance function $S(h)$ of §5 changes, circulating supply differs from it by exactly Φ_h (Property 7.6), and there is no burn.

I2 *Non-negativity.* $\Phi_h \geq 0$ for all h.

I3 *The drip never empties the pool.* A positive pool remains positive forever, decaying under zero inflow toward a floor below K sat ($= 0.000864$ FLUX at $K = 86{,}400$). There is no epoch boundary and no "last block of the pool".

I4 *Bounded coinbase.* The coinbase gains no outputs, and the pool commitment PC occupies a data field rather than one. Its *operator-facing* value is bounded by $\sigma(h) + \Phi_{h-1}/K$, and with $\Phi_{h_{\mathrm{PA}}-1} = 0$, $F_j \leq F_{\max}$ and $U_j = 0$ for all j, by $\sigma(h) + F_{\max}$. Its total value additionally carries G_h, which passes through in one block and is therefore as spiky as demand.

I5 *Determinism.* Φ_h is a pure function of the block sequence through h; a disagreement between two implementations surfaces as rejection of the block at which it first arises, never as a silent fork.

I6 *Reorg safety.* Disconnecting block h applies $-\Delta_h$ and restores Φ_{h-1} exactly, and Φ_{h-1} is independently readable from the parent's coinbase commitment, so recovery needs no journal, no crash marker and no repair RPC. On any single chain no payee receives a drip twice for the same block.

I7 *Liveness independent of demand.* No validity rule references $\Phi > 0$. $D_h = 0$ is a valid coinbase, identical to today's.

I8 *No double claim.* A node receives PNR value only as the increment of its PoN tier output, only in a block in which it is that tier's payee, and at most once per block.

I9 *Split invariant.* For every output of value v paying sink in a block at height h, $A_\Phi \leq \rho(h)\, v < A_\Phi + 1$ sat.

Proof. **I1** by induction on h. Base: $\Phi_{h_{\mathrm{PA}}-1} = 0 = \mathrm{In} - \mathrm{Paid}$. Step: (6) gives $\Phi_h = \Phi_{h-1} + F_h - (D_h - \epsilon_h - U_h)$, which is exactly the increment of $\mathrm{In}_h - \mathrm{Paid}_h$. Every satoshi paid to sink leaves circulation at payment (PS3) and re-enters through exactly one coinbase output — A_F through the dev-fund output, A_Φ through a tier output as drip over the blocks that follow — so a tenant's $v = A_F + A_\Phi$ is fully accounted (PS4). The sum of coinbase values over the chain is $\sum \sigma + \sum_j G_j + \mathrm{Paid}_h$, and $\sum_j G_j + \mathrm{Paid}_h \leq \mathrm{Rev}_h$ is value that already existed, so no term of S changes; Property 7.6 gives the exact circulating-supply identity.

I2: $D_h = \lfloor \Phi_{h-1}/K \rfloor \leq \Phi_{h-1}$ for $K \geq 1$, and $\epsilon_h, U_h, F_h \geq 0$, so $\Phi_h \geq \Phi_{h-1} - D_h \geq \Phi_{h-1}(1 - 1/K) \geq 0$.

I3: if $\Phi_{h-1} \geq K$ sat then $\Phi_{h-1} - D_h \geq \Phi_{h-1}(1 - 1/K) > 0$; if $\Phi_{h-1} < K$ sat then $D_h = \lfloor \Phi_{h-1}/K \rfloor = 0$ and the pool is unchanged. Hence positivity is preserved in both regimes and the floor is K sat.

I4: output count is unchanged because G_h is added to the dev-fund output and $D_{h,t}$ to the three tier outputs, all four of which already exist, and because PC writes into a data field rather than creating an output. For the value bound, $D_h \leq \Phi_{h-1}/K$ is immediate. For the second form, induct: if $\Phi_{h-1} \leq K F_{\max}$ then $\Phi_h \leq \Phi_{h-1}(1 - 1/K) + K F_{\max}/K \leq K F_{\max}$, so $D_h \leq F_{\max}$. Thus the drip never exceeds the largest single-block inflow ever observed, and $F_{\max}$ is itself bounded by `MAX_BLOCK_SIZE` $= 2{,}000{,}000$ bytes and by `MoneyRange`. The G_h term is not smoothed and is not claimed to be: it is a pass-through to the dev-fund output in the block that receives it, and it reaches no operator, so it affects the coinbase's size but nothing in §7.5 or

§7.7. *Contrast*: under next-block payout the coinbase would carry the entire *operator* share of a block's payments and would spike with demand, which is the property that would break fairness.

I5: Φ_h is defined by integer arithmetic on the block sequence, so any node with the chain recomputes it. Two implementations that disagree at h disagree with the commitment PC carried by that block (step 6 of Construction 7.11), so the block at which the divergence arises is the block that is rejected — a detected fault, localised, and one block earlier than a per-index delta would have surfaced it.

I6: Δ_h is subtracted on disconnect, which is exact because (6) is additive; and because Φ_{h-1} is committed in the parent's coinbase, correctness does not depend on the cache surviving. A PNR application payment in an orphaned block returns to the mempool and re-enters the pool when re-mined; the orphaned coinbase and its drip vanish with the block. *Proof sketch only for the implementation obligation*: the pool must not be a global mutable singleton. The documented reorg fragility of `g_fluxnodeCache` is the anti-pattern to avoid [fluxd/src/fluxnode/fluxnode.cpp:790–820], and the in-tree precedent to follow is the per-index shielded value-pool accumulator (§20).

I7: no clause of Construction 7.11 tests $\Phi > 0$; step 2 yields $D_h = 0$ when $\Phi_{h-1} < K$, and step 4 then requires the tier outputs to equal $\sigma_t(h)$, which is today's rule.

I8: the payee set is $\mathcal{W}_h$, one node per tier (Assumption 7.1 A1), and the drip is added to that tier's single output. A node whose collateral moves re-enters the queue at the back with last-paid reset and waits one full rotation; it carries no accumulated claim, so there is nothing to double-claim.

I9: PS4 sets $A_\Phi = \lfloor r(h)v/100 \rfloor$ with $r(h) = 100\rho(h)$ an integer, and $\lfloor x \rfloor \le x < \lfloor x \rfloor + 1$. □

Property 7.33 (Two security constraints the design exports, PLANNED)**.** The following two rules are not optimisations. Each is a defect if violated.

S1 *Emergency blocks must not drip.* For any block produced through the 2-of-4 emergency key path (Assumption 7.1 A3), $D_h := 0$ and $\Phi_h := \Phi_{h-1} + F_h$.

S2 *Moderation forfeitures must never enter* Φ. If §23 forfeits a removed tenant's prepaid balance, that balance must go anywhere other than the fee pool.

Justification. **S1**. An emergency block is produced outside the PoN eligibility check, with no time or stall gating [fluxd/src/emergencyblock.cpp:183–191], and the remainder output of its coinbase defaults to the dev-fund address, as in every block [fluxd/src/rpc/emergencyblock.cpp:64–77], [fluxd/src/pon/pon-minter.cpp:288–306] (C9). Its coinbase is nonetheless bound by the deterministic payout check, so its tier outputs go to the queue's payees exactly as in any other block [fluxd/src/miner.cpp:487–489], [fluxd/src/main.cpp:3878–3884]. The capability the drip would add to the emergency path is therefore not redirection but acceleration: the holders of two of four hardcoded keys could produce blocks faster than the 30 s target and run the drip forward, Φ_{h-1}/K per block — at the recommended K and parity-scale revenue, 15.2 FLUX per block from a pool holding 1.31M FLUX — paid to whichever nodes stand at the front of the queue, including any they operate, with no gating on how many such blocks are produced. Setting $D_h = 0$ removes that lever entirely while preserving I1 (nothing is paid, so Paid does not advance) and I7 (a zero drip is a valid coinbase). The Foundation term G_h needs no equivalent rule and is left to flow: an emergency block that contains sink payments would pay their 80 % to the dev-fund address, which is the destination that rule already has, so the emergency path gains no capability it did not have. The drip is the only term over which it would gain one, and S1 removes exactly that.

S2. Routing forfeitures into Φ would give every node a positive expected payoff from every eviction, since by Proposition 7.24 each node receives λ_t/n_t of everything the pool takes in. Report brigading would then be paid for by the settlement mechanism, and the moderation quorum of §23 — which is Sybil-resistant by construction but, at the measured distribution, not censorship-resistant (C6) — would acquire a direct financial motive. The constraint is a

requirement exported to §23, not a preference; note also that "no burn" is a PNR decision and does not bind §23's choice of destination. □

Decided. $D_h = 0$ on emergency blocks (Table 24). O15 — emergency blocks and the drip. Domain: $\{D_h = 0$ (required by S1), normal drip$\}$. Originally recorded as open because it is a consensus rule nobody has written; S1 states what it must be.

Remark 7.34 (O16, destination of forfeited balances — burn, and why the question matters). This is the one open parameter in this section whose *wrong* answer creates an attack rather than an inefficiency, so it is worth restating why it was asked before recording the recommendation.

Two forfeiture events exist and they are not the same. **Ordinary expiry** forfeits nothing: an application's prepaid period is consumed as it runs, the application expires when it is exhausted, its data is deleted, and there is no refund and no residue [intent: design intake]. **Moderation eviction** is different, because it interrupts a period that has been paid for and not yet consumed: an application evicted at height h_b with paid-through height $e_a > h_b$ leaves an unearned remainder, and the author has settled that this remainder is forfeited rather than refunded [intent: design intake]. The question is where that value goes.

It must not go to Φ. If forfeited balances flowed to the fee pool, the operators who vote to evict an application would be paid, pro rata, out of the balance they voted to forfeit. Under the 25 % collateral-weighted threshold of §23, and given the concentration measured there, that is a direct financial incentive to evict paying applications — strongest precisely against the largest prepaid balances, which are the most committed tenants. This is why S2 forbids it, and it is the substance of the question rather than an accounting detail.

Recommendation: burn. Of the three admissible destinations, burning is the only one that leaves no party with a financial interest in the outcome of a vote. A refund to the tenant is excluded by the settled forfeiture decision. Routing to the Foundation is defensible — the Foundation does not vote, so the incentive is not created at the voter — but it makes the Foundation a beneficiary of network moderation, which is an association worth paying a small price to avoid, and the amounts are small enough that the price is small: the enforcement floor is 0.01 FLUX per application (§23). Burning also composes correctly with the supply argument of §5, since a burn is exactly representable in the supply function whereas a Foundation transfer is not.

Decided. O16: **burn** the unearned remainder on moderation eviction, adopted [intent: design intake]. Φ is excluded by S2, and refund by the settled forfeiture position. The implementing choice — an explicit output to a provably unspendable script, or an omission from the coinbase — belongs to §23, which owns the enforcement path; both realise the same supply effect.

Failure modes not covered by the invariants. Three deserve naming. *A scheduled payee that is offline at payout* resolves under the existing rule: either it is still confirmed, in which case it is paid — drip included, exactly as its subsidy is — or it has failed to re-confirm within 640 blocks and is purged before payout SHIPPED [fluxd/src/fluxnode/fluxnode.cpp:790–820]. The consequence a reader should see is that the drip, like the subsidy, would be paid for being in the set, not for being up at the block. *Sustained low demand* decays the pool geometrically to dust (I3) and the coinbase to today's, with no liveness effect (I7); since PNR would be 4.29 % of operator income at activation, low demand is the baseline rather than a failure mode. *A deep reorg* — normally bounded at 40 blocks, but temporarily 5,000 during the PoN stabilization window [fluxd/src/main.cpp:8983–8988] (C16) — undoes drips and inflows together and is tolerated by I6. The single-output shape removes an obligation that the two-output shape carried: because the payer commits to no split, a payment returned to the mempool cannot become invalid at a new height. It would simply be divided at the height at which it is re-mined, so a payment straddling a reduction boundary changes the destination of $\Delta\rho = 0.05$ of its value and nothing else, and FluxOS's coverage check is unaffected.

7.10 Activation schedule

event	height	date	σ (FLUX/block)	ρ
current tip (baseline, SHIPPED)	∼2,912,015	2026-09-03	14.0	—
reduction 1 (×9/10)	3,071,200	2026-10-25	12.6	—
PA Depletion (×1/2)	**3,787,502**	**2027-07-01**	**6.3**	**0.20** — PNR begins
reduction 2	4,122,400	2027-10-25	5.67	0.25
reduction 3	5,173,600	2028-10-24	5.103	0.30
reduction 4	6,224,800	2029-10-24	4.5927	0.35
reduction 5	7,276,000	2030-10-24	4.1334	0.40
reduction 6	8,327,200	2031-10-24	3.7201	0.45
reduction 7 (cap reached)	9,378,400	2032-10-23	3.3481	**0.50** = $\rho_{\max}$
annually thereafter	$+I_{\text{red}}$		×9/10, 20 times	0.50

Table 18: PNR activation and ramp PLANNED. Heights lie exactly on the PoN reduction grid $h_{PoN} + k\,I_{\text{red}}$ with $h_{PoN} = 2{,}020{,}000$ and $I_{\text{red}} = 1{,}051{,}200$ [fluxd/src/chainparams.cpp:153,157]; the subsidy column is the as-coded per-step-truncating path $14 \cdot (9/10)^{\text{red}(h)}$ multiplied by 1/2 from h_{PA} [intent: design intake]. Dates are 30 s extrapolations from the PoN activation timestamp anchor and carry the drift documented in §26 (about 0.8 d over 796,820 blocks); the intake gives one of them a day later. The intake says the cap is reached "∼2033"; the grid gives October 2032.

Decided. Uniform ×1/2 (Table 24). O20 — whether the PA-Depletion halving applies uniformly to the three tier bases and to the dev-fund remainder. Domain: {uniform ×1/2, other}. Table 18 and every parity figure in §7.11 assume uniform; a non-uniform cut changes operator issuance and therefore every crossover number.

Decided. Fixed schedule plus a stated review (Table 24); see also Definition 7.36 on the contraction response. O13 — ramp policy if demand undershoots. Domain: {fixed schedule (settled as far as it goes), a revision process}. With $g = 0$ there is no crossover ever (§7.11); the schedule's behaviour in that case is unstated.

Remark 7.35 (Sequencing conflict C1a — settled as simultaneous). The design intake activated PNR at PA Depletion simultaneously with the parallel-asset emission cut, while the public roadmap stated that parallel-asset mining retirement comes *only after* PNR is demonstrably paying operators, ninety days announced, with the two in different halves of 2027. The team has settled it: activation is **simultaneous**, both at $h_{\text{PA}} = 3{,}787{,}502$ [intent: design intake] (PLANNED), superseding the roadmap's strictly-after ordering (§6.8).

The consequence is recorded rather than smoothed over, because simultaneity removes a check the roadmap's ordering provided: **no external observer can confirm that PNR pays operators before parallel-asset mining rewards are retired**, since both happen at one height. The roadmap's sequence would have produced that evidence; the settled sequence does not. §24 carries this as an accepted risk that should stay visible in any activation announcement rather than as an oversight.

7.11 The crossover, in both denominations

Denomination discipline. FLUX terms are primary, because that is what consensus governs. Real terms are a clearly-labelled secondary analysis with price treated as an exogenous parameter. No price prediction, target, or investment framing appears anywhere in this section, and a claim made in one denomination is never defended with reasoning from the other [intent: design intake]. **Parity depends on demand growth, not on price.**

Bases. Operator issuance excludes the dev-fund remainder: it is $\frac{13.5}{14}\,\sigma\,B_{\text{yr}}$, not $\sigma\,B_{\text{yr}}$, because 0.5/14 of every PoN subsidy is a consensus-enforced dev-fund payment SHIPPED [fluxd/src/

main.cpp:2877–2884] (C9). **The intake's own parity table is** 3.7 % **high** for exactly this reason: it reports 33,112,800 FLUX per year of parity revenue at activation where the operator-only basis gives 31,930,200, and 6,622,560 FLUX per year of issuance where the operator-only basis gives 6,386,040 [intent: design intake]. The generated data carries both bases [doc: paper/data/pnr_crossover.dat].

At activation. With $\rho = 0.20$ and $\sigma = 6.3$ FLUX per block, operator issuance would be 6,386,040 FLUX per year and PNR income at V_0 would be $0.20 \times 1{,}431{,}600 = 286{,}320$ FLUX per year. Defining the PNR share of operator income as PNR/(issuance + PNR),

$$\frac{286{,}320}{6{,}386{,}040 + 286{,}320} \;=\; 4.29\,\%.$$

Parity revenue — the annual application revenue at which PNR income equals operator issuance — would be $6{,}386{,}040/0.20 = 31{,}930{,}200$ FLUX per year, or $22.3 \times V_0$.

7.11.1 FLUX terms (primary)

stage start (height)	date	σ (FLUX)	ρ	operator issuance (FLUX/yr)	parity revenue (FLUX/yr)	growth needed from V_0
3,787,502	2027-07-01	6.3000	0.20	6,386,040	31,930,200	—
4,122,400	2027-10-25	5.6700	0.25	5,747,436	22,989,744	16.1×
5,173,600	2028-10-24	5.1030	0.30	5,172,692	17,242,308	12.0×
6,224,800	2029-10-24	4.5927	0.35	4,655,423	13,301,209	9.3×
7,276,000	2030-10-24	4.1334	0.40	4,189,881	10,474,702	7.3×
8,327,200	2031-10-24	3.7201	0.45	3,770,893	8,379,762	5.9×
9,378,400	2032-10-23	3.3481	**0.50**	3,393,803	6,787,607	4.7×
12,532,000	2035-10-23	2.4407	0.50	2,474,083	4,948,165	3.5×
17,788,000	2040-10-21	1.4412	0.50	1,460,921	2,921,842	2.0×
23,044,000	2045-10-20	0.8510	0.50	862,659	1,725,319	1.2× (perpetual tail)

Table 19: Parity requirement in FLUX terms PLANNED. The two levers compound as intended: the requirement falls from 31.9M to 6.8M FLUX per year in 5.3 years and to 1.7M on the perpetual tail. The final row is the flat tail of §5: after twenty reductions the subsidy freezes, so operator issuance never reaches zero [fluxd/src/chainparams.cpp:158]. Source: `paper/data/pnr_crossover.dat`.

annual revenue growth g	crossover	revenue at crossover (FLUX/yr)	ρ at crossover
0 %	**never**	—	—
10 %	2038-10	4.21M	0.50
25 %	2034-10	7.33M	0.50
50 %	2032-10	12.4M	0.50
100 %	2030-10	14.3M	0.40
200 %	2029-10	18.3M	0.35

Table 20: Crossover year under constant annual revenue growth from V_0, FLUX terms PLANNED. Required sustained growth to reach parity by a given year: 2029, 162 %; 2030, 82 %; 2031, 51 %; 2032, 34 %; 2033, 26 %; 2035, 16 %. Source: `paper/data/pnr_crossover.dat`, columns `pnr_g000` . . . `pnr_g200`.

With flat demand there is no crossover, ever. The perpetual tail issuance of 862,659 FLUX per year to operators exceeds $\rho_{\max} \cdot V_0 = 0.5 \times 1{,}431{,}600 = 715{,}800$ FLUX per year, and the tail

is perpetual because the subsidy freezes after twenty reductions rather than going to zero (§5, C7). So the correct statement, in these words: *as scheduled, PNR would begin as a supplement; whether it becomes the main component of operator income is a statement about demand, and this paper makes no such statement.*

Remark 7.36 (The stated response to a demand shortfall is contraction, and it works). Asked what happens if demand does not arrive, the team's answer is that the network shrinks: hosting that many FluxNodes stops being economic, some operators leave, "which would mean more flux allocation for other node ops" [intent: design intake] (PLANNED). The team also reports the observed direction is the opposite — demand rising, with faster growth anticipated from AI-driven deployments — and this paper records that as stated intent, not as a measurement it can confirm.

The equilibrium argument is sound and worth making precise, because it is the mechanism's genuine answer to its own worst case. PoN pays a fixed per-block subsidy divided among confirmed nodes, so per-node issuance income is inversely proportional to the confirmed node count. If total operator income falls below the cost of operating a node, exit raises the income of every node that remains, and exit continues until income again covers cost. The node count is therefore self-correcting against demand: the network converges on the size its revenue plus issuance can support, with no governance intervention and no parameter change. That is a stronger position than most demand-funded designs hold, and it does not depend on demand materialising.

The cost is a node count that falls exactly when the paper's other mechanisms need it to rise, and the two answers are in tension in a way neither alone reveals. Section 23.5 measures four owner identities reaching the 25 % moderation threshold, and the stated remedy for that concentration is growth: "as network grows it will be more decentralized" [intent: design intake]. But the stated response to weak demand is contraction — and contraction concentrates. Under the demand shortfall this subsection describes, the operators who exit first are the marginal ones, which are disproportionately single-node operators rather than the large holders whose collateral drives the concentration measurement. The concentration ratio would therefore *worsen* in precisely the scenario where the mechanism relies on it improving.

Both answers are individually reasonable and the paper endorses neither as a prediction. What it records is the dependency: **the moderation quorum's censorship-resistance is a function of demand for Flux Cloud**, through node count, and no part of the moderation design references demand at all. Section 23.8 evaluates remedies on the measured distribution; none of them is robust to that distribution getting worse.

Not established by this survey. the composition of exit under a demand shortfall — whether marginal operators are disproportionately single-node holders — is not measured in this corpus. The directional claim that contraction concentrates follows from the tier and collateral structure, but its magnitude is not established.

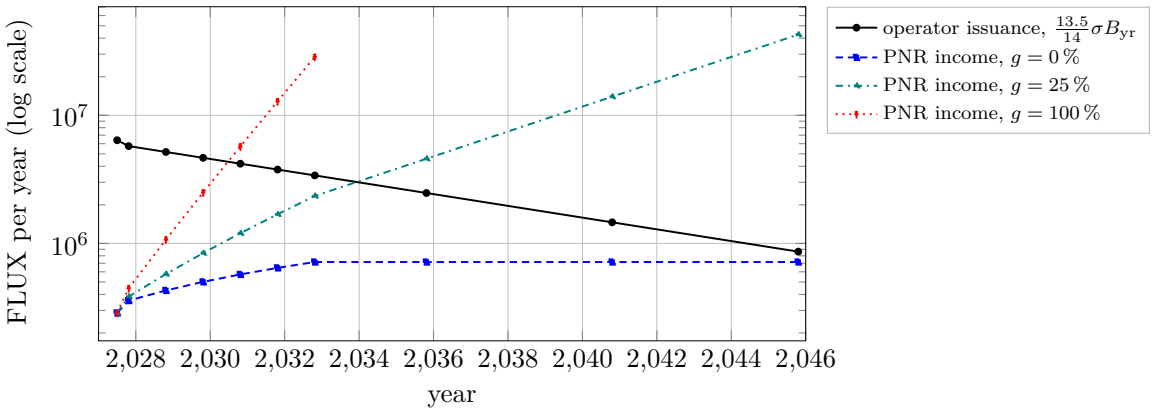


Figure 6: Crossover in FLUX terms PLANNED. Operator issuance falls on the reduction grid while the operator share ρ ramps from 0.20 to the cap $\rho_{\max} = 0.50$; PNR income is $\rho(h)\, V_0(1+g)^t$. The $g = 0\,\%$ series flattens at 715,800 FLUX per year and never meets the issuance curve, whose perpetual tail is 862,659 FLUX per year. Price does not appear on either axis and no price claim is made.

Remark 7.37 (Figure provenance). The coordinates plotted in Figures 6 and 7 are the tabulated output of `scripts/07-pnr-settlement.py` (run 2026-09-07), which also wrote `paper/data/pnr_crossover.dat` and `paper/data/pnr_crossover_real.dat`. Every plotted point in Figure 6 has been checked against the corresponding row of that data file and agrees exactly; the data files carry their own `#` provenance headers and the figures are reproducible from them.

7.11.2 Real terms (secondary, price exogenous)

The FLUX-denominated price table is revised by policy, and the revisions have historically tracked real cost: the CPU rate has fallen 100× across five revisions SHIPPED [measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]. Under a continuation of that rebasing policy, FLUX inflow to the pool would be real demand divided by an exogenous price multiple, $V_t^{\mathrm{FLUX}} = V_0(1+g)^t/m_t$, while issuance is fixed in FLUX. Treating m as an axis and not as a forecast gives the following years-to-parity after activation.

	exogenous price multiple m				
real growth g	0.5	1	2	5	10
0 %	14.3	never	never	never	never
10 %	7.3	11.3	14.3	19.3	26.1
25 %	5.3	7.3	9.3	12.3	14.3
50 %	3.3	5.3	6.3	7.3	9.3
100 %	2.3	3.3	4.3	5.3	6.3

Table 21: Years from activation to FLUX-terms parity, as a function of real demand growth g and an exogenous price multiple m PLANNED. m is a sensitivity axis, not a projection. Source: `paper/data/pnr_crossover_real.dat`.

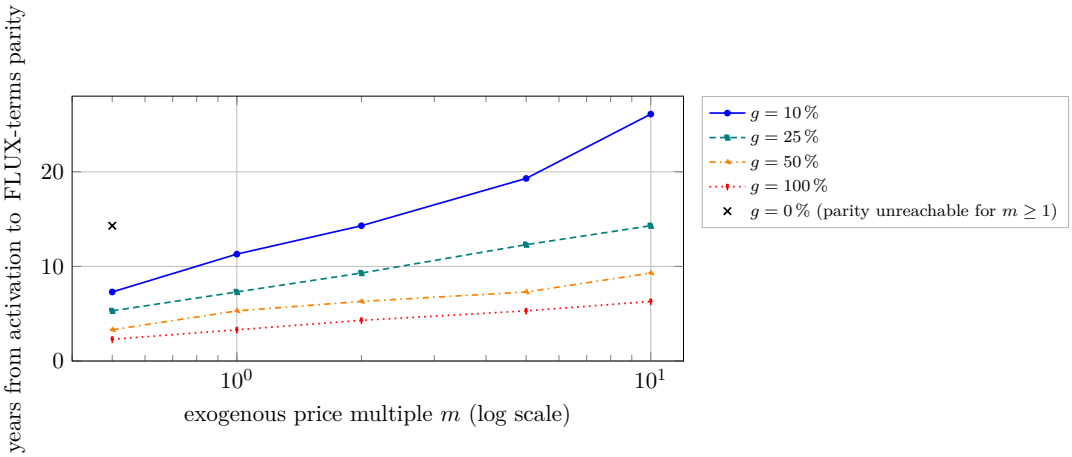


Figure 7: Real-terms sensitivity PLANNED, secondary analysis. Reading, and it is the point of the figure: under a rebasing price policy, *price appreciation delays* FLUX-*terms parity* — every curve rises to the right. Appreciation can raise operator income in real terms while the FLUX-denominated ratio barely moves. The two statements live in different denominations and must never be used to defend one another.

Decided. Trailing receipts at sink (Table 24). O17 — the measured revenue baseline that should replace "100 FLUX per application per month". Domain: {trailing receipts at the payment sink, list-price value of the measured resource totals}; the census gives 8,972 vCPU, 17,544,229 MB of RAM and 160,107 GB of disk across 7,486 instances [measured: api.runonflux.io/apps/globalappsspecifications, 2026-09-03]. Every absolute figure in this subsection scales linearly with it; the growth-rate conclusions do not.

Decided. Rule-based rebasing (Table 24). The decision is that rebasing follows a published rule rather than discretion; the rule's functional form is specified in §7.13 as rebasing against a real-terms anchor, with the price feed and its failure behaviour stated as part of the rule, and its coefficients are a policy calibration rather than a mechanism choice. O9 — price-table rebasing policy. Domain: {discretionary (today), rule-based, USD oracle}. A rule removes a discretionary lever, held by the multisig that posts price messages, over pool inflow; an oracle would formalise the off-consensus price feed the quoting path already runs (§7.13), whose failure modes would need their own treatment. The deployed `getappspecsusdprice` endpoint already returns USD-denominated quotes SHIPPED.

7.12 Open parameters

Of the twenty decisions this specification opened, four are now settled and are recorded in Table 22; the settlement decision created one more, O21, which Definition 7.10 settles. Eleven more are settled by the author's confirmation of Table 24, and O16 by Definition 7.34; four therefore remain open (O4, O8, O14 and O19), and the load-bearing one among them is **O8** — which payment channels must settle through sink — because every channel outside it is revenue the consensus split never touches and the counter of Property 7.6 never sees. None of the open values is guessed anywhere in this section, and none of the closed ones is stated without its provenance.

id	decision	settled value	provenance
O1	drip constant K	86,400 blocks, i.e. exactly 30 d at 30 s	[intent: design intake]
O2	commitment of Φ_h	committed in the coinbase (PC) and recomputed by every validator; no separate store, no header field	[intent: design intake], [doc: reconciliation record]
O3	memo semantics	`OP_RETURN` binding $H(m_a)$, carried and not interpreted by consensus (PS2)	[intent: design intake]
O18	serialisation and enforcement boundary	one output of the full price to sink (PS1); consensus applies the split (PS4); FluxOS checks only existence, memo and coverage	[intent: design intake], [doc: reconciliation record]

Table 22: Decisions closed since the first draft of this specification PLANNED. All four are team ratifications: O2 originated as this specification's recommendation and was subsequently adopted. Each would be a consensus rule once activated, so changing any of them afterwards would be a network upgrade.

id	decision	domain	trade-off
O4	fee floor and spam resistance	$p_{\min} \geq 0.01$ FLUX; per-instance or per-deployment	dust-priced deployments contribute nothing yet occupy capacity; a floor prices out the micro-workloads §18 wants
O5	$D_{h,t}$ with no payee in tier t	stays in pool; dev fund; other tiers	pool: conservation-trivial; dev fund: one rule for subsidy and drip; other tiers: rewards concentration
O6	partition residue ϵ_h	pool; dev fund	≤ 2 sat per block either way; must be fixed to avoid divergence
O7	shielded payers	transparent value required; Sapling with revealed amount; excluded	consensus needs the received value for PS4; the hide-payer/reveal-amount construction is §21's
O8	scope of payments required to settle at sink	new deployments; renewals; credit top-ups (§16); CumulusVPN; enterprise	every excluded channel is revenue outside the consensus split and invisible to the revenue counter
O9	price-table rebasing	discretionary; rule-based against a real-terms anchor; receipts-referencing (withdrawn as circular)	revisions are already on-chain and dated; a real-terms rule needs the price feed the quoting path already runs
O10	drip ordering	from Φ_{h-1}; after applying F_h	parent-state: coinbase checkable early, producer cannot influence own drip; post-inflow: one block less lag, same-block recapture returns
O11	initial seed $\Phi_{h_{\mathrm{PA}}-1}$	0; Foundation-funded seed	seed removes the ∼90-day ramp but is a one-off transfer and must be labelled one
O12	fleet-growth guard on K	none; documented hard-fork review trigger	at 4× Cumulus the spread is 16 %; an automatic rule is a manipulation surface
O13	ramp policy if demand undershoots	fixed schedule; a stated revision process	with $g = 0$ there is no crossover; the schedule's behaviour is unstated
O14	identity of sink	keep `t3Nryf...`; a fresh address; rotation/PQ path (§8)	keeping it preserves tooling and one continuous history but mixes pre-h_{PA} spendable receipts into the counter; a consensus-level address is a consensus-level commitment and rotation is a hard fork absent a mechanism
O15	emergency blocks and the drip	$D_h = 0$ (required, S1); normal drip	otherwise 2-of-4 keys, facing no time gating, can accelerate the release of accumulated fees
O16	forfeited tenant balances (§23)	anywhere but Φ (required, S2)	routing to the pool pays for report brigading
O17	measured revenue baseline for V_0	explorer receipts; list-price of measured resources	§7.11 scales linearly with it; closable by observation once Rev exists
O19	tier partition basis	PoN bases 1 : 3.5 : 9	confirm only; must not be described as collateral-proportional (Remark 7.19)
O20	PA cut applied uniformly	uniform ×1/2; other	changes operator issuance and every parity figure

Table 23: Parameters of the PNR settlement mechanism, after the closures of Table 22; the values since settled are in Table 24, and O4, O8, O14 and O19 remain open. The bold row is the one that gates the rest, and it is mirrored in the open-decisions record.

Three requirements are exported to other mechanisms and are restated here so that they are not lost at a section boundary: to §16, *every application-hosting charge, whatever the payment channel, settles as exactly one L1 payment to* sink*, or the consensus split is void and the revenue counter under-reports*; to §23, *forfeited balances never enter* Φ (Property 7.33, S2); and to §14, *the eligibility gate admitting a node to* $\mathcal{W}$ *must be hosting-conditioned* (Open Problem 7.30).

7.13 Recommended values for the remaining open parameters

Each parameter still open above is given a recommended value here, with the reasoning that produced it. Each was proposed by this paper and **confirmed by the author** [intent: design intake], and several are simply a recommendation to keep the default that Definition 7.9 already takes — which is worth stating explicitly, because a default that nobody has ratified is not the same object as a default that has been examined and kept.

Table 24: Values for the open PNR parameters. Proposed by this paper and **confirmed by the author** [intent: design intake]. PLANNED, [inferred] for the derivations.

	Recommended	Reason
O5 tier with no confirmed node	**stays in** Φ	conservation-trivial; self-correcting when that tier next confirms
O6 partition residue	**stays in** Φ	0.021 FLUX/year does not justify dev-fund plumbing
O7 shielded payers	**moot**	pools retired; all settlement transparent (Definition 7.4)
O9 price rebasing	**rule-based, real-terms anchor**	a receipts-referencing rule is circular; the price feed it would need is already running off-consensus, with the on-chain table as a floor
O10 drip ordering	**from** Φ_{h-1}	keep the default: coinbase checkable before the block's own receipts
O11 initial seed	**0**	a seed is a one-off Foundation transfer to operators
O12 fleet-growth guard	**review trigger at** $2\times$	documented, not automatic
O13 undershoot policy	**fixed schedule + stated review**	no silent re-cutting
O15 emergency blocks	$D_h = 0$	required by S1
O17 revenue baseline	**trailing sink receipts**	measured, not assumed
O20 halving uniformity	**uniform** $\times 1/2$	every parity figure in §7.11 assumes it

The three that are not housekeeping. O11, no seed. Seeding $\Phi_{h_{\mathrm{PA}}-1}$ above zero would remove the ninety-day ramp and give operators income from the first post-activation block. It would also be a one-off Foundation transfer to node operators, and it would have to be described as one — in a section whose entire argument is that PNR would add demand-side revenue to issuance. Starting at zero makes the ramp visible and makes the first ninety days an honest measurement of whether demand exists. If the ramp is judged too harsh, the correct instrument is the activation schedule, not a subsidy that muddies what the pool balance means.

O9, price-table rebasing — and a circularity the first draft of this recommendation did not see. The price table is a lever over pool inflow, which sits awkwardly beside a mechanism whose legitimacy rests on operators being paid from receipts rather than from discretion. Two corrections have to be made before the recommendation can stand.

First, the lever is narrower than "policy" suggests. A price revision is not a configuration edit: it is an on-chain message, sent from one of the application-payment multisig addresses to itself (held personally by the four co-founders, §2, K8), carrying the seven-field table `p_cpu_ram_hdd_minPrice_port_scope_staticip`, which every node parses from its own copy of the chain and applies from the message's height onward SHIPPED [flux/ZelBack/src/services/explorerService.js:279–303,387], [flux/ZelBack/src/services/utils/chainUtilities.js:9–52]. The authority is one multisig; the *exercise* of it is public, dated, ordered and non-repudiable, and every node computes prices from the same on-chain history. That is a materially better starting point than a table published over an API, and the paper should credit it.

Second, "a published function of trailing measured receipts" is circular. Receipts are price times volume. A rule that sets next period's price from last period's receipts has no exogenous anchor: it is a feedback loop that can drift arbitrarily far from the cost of the resources being sold, in either direction, and calling it a rule rather than a lever changes who turns the crank, not whether the number means anything. What the deployed table has actually tracked across its five revisions — a 100× fall in the CPU rate — is real cost and the FLUX price, not receipts.

So the choice is a real one and it should be stated as such. *Either* the rule targets a real-terms

anchor, in which case it needs a price feed, *or* it stays discretionary and the paper says so. What it cannot be is a rule that appears to remove discretion while quietly reintroducing it.

On the first branch, one fact changes the cost of the option: **a price feed already exists in the deployed system**, and this paper's earlier framing of an oracle as a new dependency was wrong. The quoting path fetches a live FLUX/USD rate from `viprates.runonflux.io`, falls back to CoinGecko, and falls back again to a compiled-in constant; it prices a USD-denominated table, applies duration discounts and a FLUX-payment discount, and returns the *larger* of that figure and the on-chain FLUX price SHIPPED [flux/ZelBack/src/services/utils/appSpecHelpers.js:440–522]. The on-chain table is thus already a floor beneath a USD-anchored quote. A real-terms rebasing rule would not import a trust dependency the network lacks; it would move one it already runs into a place where its failure modes have to be specified — which is a smaller step than this section previously claimed, and a more honest description of the status quo.

Recommendation, revised. Adopt a rule that rebases against a real-terms anchor, and state the feed and its failure behaviour as part of the rule rather than as an implementation detail; the consensus-side floor already in place bounds what a bad feed can do to receipts. If that is judged too much trust to formalise, keep the table discretionary and say so plainly. The one option this paper withdraws is the receipts-referencing rule, which is circular.

O17, trailing receipts rather than list-price value. The placeholder baseline of "100 FLUX per application per month" should be replaced by the trailing receipts actually observed at sink, not by the list-price value of the measured resource totals. The two differ by exactly the utilisation gap, and §29 records that realised utilisation is not measurable from public data. A baseline computed from list prices would therefore embed an unmeasured assumption in the one number the crossover analysis is most sensitive to, and would do so invisibly. Trailing receipts are smaller, real, and reproducible by anyone watching the sink address.

7.14 Comparison with prior art

system	who is paid	from what	what Flux would do differently
Ethereum, post-EIP-1559	block proposer (priority fee); base fee burned	per-transaction fee, same block	no burn; the operator share is pooled and dripped to *all* nodes through the rotation, and a producer gains nothing by including a payment. The same-block capture EIP-1559 grants the proposer by design is precisely what Theorem 7.23 removes
Filecoin [41]	the specific storage provider	per-deal escrow released on proof of storage, with slashing	payment strictly conditional on proven service there; unconditional and network-wide here (H4, P7). Flux would trade the proof-of-service dependency for fairness and simplicity, and inherit the free-rider problem of §7.8 in exchange
Akash [4]	the winning provider	tenant escrow drawn down per block at the bid price, with a take rate to a community pool	no per-lease escrow to the host and no auction; the network take would be 80 % to 50 % to the dev-fund address rather than a governance-controlled pool. Stated without softening
Render [42]	node operators in proportion to work	burn-and-mint: payments burned, emissions minted to workers	burn-and-mint ties reward to metered work. The settled design rejects both the burn and the metering, which is exactly the roadmap model superseded by C1
Dash masternodes	one masternode per block, longest-since-paid queue	block subsidy plus a treasury share	the nearest structural precedent for Proposition 7.14; Dash never routed application fees through the rotation. Flux would be that rotation plus a fee pool [doc: mechanism specification]
Cosmos SDK `x/distribution`	all validators in proportion to stake, every block, lazily accumulated	transaction fees, with a community tax	exact and immediate, but requires per-validator accumulators and withdraw transactions. Flux would pay through three existing coinbase outputs with no per-node state and no claim transaction (P4, P5), accepting a bounded 3.83 % spread and a twenty-day lag instead. This is the road not taken [doc: mechanism specification]

Table 25: PNR against fee-handling and work-payment prior art.

The combination specific to Flux is: coinbase-only payment with no per-node accounting; a deterministic rotation that makes network-wide distribution provably fair over one rotation (Propositions 7.14 and 7.16); and a proportional drip with no epoch boundary (Property 7.32, I3) that converts a schedule everyone can read into a schedule nobody can profit from reading (Theorem 7.23). The cost is stated once more because it is the design's real weakness, and because §25 and §27 both depend on it: **PNR would pay for membership, and membership is only as real as the gate**.

8 Chain operations and evolution: main-chain pruning, collateral maturity, and post-quantum migration

The roadmap groups three chain-level changes under a single H1 2027 heading, and a reader could be forgiven for assuming they are equally ready because they share a quarter [roadmap: H1 2027 chain-changes section]. They are not. One of them, main-chain pruning, is already-shipped software wired into `fluxd`; the operational decision is what is new. The second, collateral maturity, is a named consensus-rule proposal with an explicit open item that decides whether it works at all. The third, post-quantum migration, has a deliverable for H1 2027 that is a published plan, not a change to any key. This section keeps the three separate, states a maturity tag and a feasibility class for each, and closes with the constraint all three share: none of them can rotate a key the protocol does not itself control. Collateral maturity delays when an existing key may spend; it does not replace that key. Post-quantum migration is precisely the case in which the protocol would like to replace a key and structurally cannot without the key-holder's cooperation.

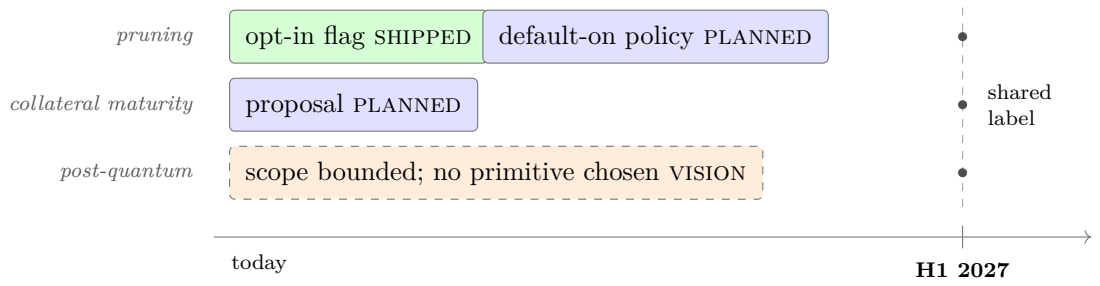


Figure 8: Three chain-evolution tracks that the roadmap places under one "H1 2027" label, shaded by *maturity* rather than by calendar position. The label is shared; the readiness is not. Pruning exists as an opt-in flag today and needs a policy decision to become a default; collateral maturity is a proposal with no code path in `fluxd` yet; post-quantum migration has a bounded scope and no chosen primitive, and is the only one of the three whose hardest sub-problem (Definition 8.7) is unsolved industry-wide. A reader who takes the shared date as shared readiness would mis-plan all three.

8.1 Main-chain pruning: an operational decision, not a new capability

`-prune` is standard, inherited Bitcoin/Zcash functionality and is fully wired in `fluxd` today SHIPPED [fluxd/src/init.cpp:986–1064, 1919] [fluxd/src/main.cpp:4312–4313, 5930]. `nPruneAfterHeight` is set to 100,000 on mainnet [fluxd/src/chainparams.cpp:181]. No PoN-specific pruning constraint was found in the code. So "main-chain pruning" as a roadmap item is not the implementation of a new capability; it is the decision to turn on a capability that already exists, together with the archival tier that has to sit beside it once history is no longer kept by every node [roadmap: Main-chain pruning, H1 2027].

The roadmap's own justification is that pruned nodes remain able to produce blocks:

> "30-second blocks mean the chain grows quickly. Pruning cuts what a node must store and speeds up syncing. Done properly, pruned nodes stay fully able to produce blocks, with designated archive nodes preserving full history for explorers and the data services above." [doc: flux-roadmap-public.md:121-122]

Under PoN this is a claim worth proving rather than restating, because block production requires two things that a naive reading of "pruning" might endanger: evaluating slot eligibility $H(\text{collateral} \| \text{prevHash} \| \tau) \leq \text{target}$ and signing [fluxd/src/pon/pon.cpp:70–86], and validating the FluxNode registry against which that collateral outpoint is checked. The reason the claim holds is that the registry does not live inside the block store it would otherwise be pruned along with: FluxNode state, which every node derives deterministically from the START and CONFIRM transactions on the chain, is held per node in a local cache, `g_fluxnodeCache`, persisted in its own LevelDB database keyed by datadir path `determ_zelnodes` [fluxd/src/fluxnode/fluxnodecachedb.cpp:26], and is rebuilt from that store at every startup independently of how much of the block chain is retained [fluxd/src/fluxnode/fluxnodecachedb.cpp:49–79].

Property 8.1. A pruned PoN node retains block-production capability if and only if the FluxNode registry and the UTXO set for collateral outpoints are retained, regardless of pruning depth.

This is stateable precisely because each node's locally derived copy of the registry is persisted outside the block store rather than being re-derived from block history on demand; a chain whose node registry had to be replayed from full block history would not have this property, and `fluxd`'s separation of the two stores is what makes pruning and PoN compatible without further engineering SHIPPED [fluxd/src/fluxnode/fluxnodecachedb.cpp:26,49–79].

Remark 8.2 (Archival is a tenant product, not a node obligation — settled). The pruning-depth policy and the archival-redundancy target were carried as open. They are settled by a decision that resolves both at once and relocates the problem: **archive nodes are to be applications running on FluxCloud**. The team would deploy them as tenant workloads and sell access to RPC and other node APIs as a product [intent: design intake] (PLANNED).

Two consequences, and the second is the one worth stating carefully.

Pruning is unconstrained by any retention obligation. If no node is obliged to keep history, the pruning-depth policy is a storage-efficiency choice for ordinary operators rather than a network-safety parameter, and it can be set as aggressively as operators want. The awkward coupling that made this an open question — how deep can we prune without endangering history — dissolves, because history is not being kept by the pruning nodes in the first place.

History availability becomes a function of demand for a product. This is the honest form of the trade and the paper states it plainly: if archival capacity is a paid service, then history survives exactly as long as someone finds it worth paying for. That is a different guarantee from "every node keeps everything", and it is weaker in a specific way — there is no protocol rule setting a minimum number of archive nodes, so the redundancy target is a commercial decision, not a consensus one. A chain whose full history rests on the commercial viability of an archive product should say so, and this one does.

Not established by this survey. no redundancy target for archive deployments is stated in any artefact read for this paper, and none is derivable from the decision that archival is a product. The number of independent archive deployments required before history is durable is therefore not stated here.

The roadmap pairs pruning with a paid product built from the archive nodes pruning requires, describing RPC, archive and indexing services as the network's "first capacity-selling product" and stating that this "pairs directly with chain pruning: the archive nodes pruning requires become the product" [doc: flux-roadmap-public.md:33-34]. This is the clearest instance in the plan of the demand-funded thesis applied to the chain layer itself: an operational cost the network would otherwise carry for free (full history) would become a service the network sells, and the archive tier pruning makes necessary would be the same tier that generates the revenue. [roadmap: RPC, archive and indexing services, Q3 2026]

See Definition 8.2: archival capacity is *to be* a paid product, deployed as FluxCloud applications (PLANNED; not yet deployed), and the consequence for history availability is stated there rather than left conditional. [inferred]

Class: Extension. The mechanism exists; what remains is the operational decision to enable it and the archival-capacity policy above.

8.2 Collateral maturity: a proposal, not yet a rule

Collateral maturity is not deployed and has no code path in `fluxd` today; the roadmap presents it explicitly as a proposal that has not yet gone to operators [roadmap: Collateral maturity, H1 2027] PLANNED. The stated rule is a rolling maturity on the collateral outpoint, modelled on coinbase maturity:

> "For attestation to carry real weight, a node's stake in the network has to outlast the window in which its claims can be challenged. We will propose a consensus rule: a node-collateral UTXO becomes spendable only 30 days after the node's last confirmed activity — a rolling maturity, like coinbase maturity, applied prospectively and never to standing collateral. Moving collateral directly into a new or upgraded node (including into multisig custody) stays instant; only leaving waits. There is no confiscation and never will be." [doc: flux-roadmap-public.md:143-144]

Formally: for a collateral outpoint o with last confirmed activity height $h_{\text{act}}(o)$, spending o would be valid at height h if and only if

$$h \;\geq\; h_{\text{act}}(o) + M \qquad \text{or} \qquad \text{dest}(o) \in \mathcal{D}_{\text{node}},$$

with $M = 86{,}400$ blocks — 30 days at the PoN target spacing $\Delta_{PoN} = 30\,\text{s}$ — and $\mathcal{D}_{\text{node}}$ the set of destinations that constitute continued participation.

Not established by this survey. the definition of $\mathcal{D}_{\text{node}}$ — which destinations count as continued participation, so that a move into a new or upgraded node or into multisig custody stays instant while an exit disguised as a move does not — is stated in no artefact read for this paper. It is the parameter on which the rule's acceptability rests: too permissive and the rule does nothing, too restrictive and funds are held by a mechanism promised not to hold them.

Decided. $M = 86{,}400$ blocks is this paper's recommendation, and the constraint that made it non-free-standing is satisfiable: it must satisfy $M \geq \Delta_{\text{chal}}$, and §14 recommends $\Delta_{\text{chal}} = 2{,}880$ blocks. With $86{,}400 \geq 2{,}880$ the constraint holds with a factor of 30 in hand, so M can be treated as a free choice after all — and it reuses the PNR constant K, so one number serves three mechanisms. [inferred].

Non-retroactivity is stated as part of the proposal and is what makes it non-confiscatory:

Property 8.3. For every collateral outpoint existing before the activation height $h^{\star}_{\text{act}}$, the maturity rule does not apply: no standing collateral is retroactively locked.

The roadmap also states a second-order effect openly rather than leaving it to be found later — a maturity rule smooths collateral unlock waves — and frames this as a proposal that would ride an already-planned network upgrade rather than force its own:

> "We state the second-order effect ourselves rather than leaving it to be discovered: a maturity rule also smooths collateral unlock waves. Both effects are the point. This is a proposal — it goes to operators for discussion before any activation is scheduled, and it would ride an already-planned network upgrade rather than forcing its own." [doc: flux-roadmap-public.md:146]

Stated honestly, the rule buys a bounded window rather than permanence: an operator willing to stop operating and wait M blocks can still exit with the collateral intact, exactly as before the rule existed. What the maturity rule changes is the cost of lying about continued participation while attempting to exit early; what makes lying about an attestation claim actually costly is the bond described in Section 14, not the maturity window itself. Section 8 and Section 14 divide the labour this way: the maturity window guarantees stake outlasts a challenge; the bond is what is at risk if the challenge succeeds.

Class: Engineering. There is no unsolved technical problem in the maturity rule as stated; the open item is the definition of $\mathcal{D}_{\text{node}}$, and the real cost is the political work of operator consent the roadmap itself names as a prerequisite [roadmap: Collateral maturity, H1 2027].

8.3 Post-quantum migration: a scope decision and an open problem

The H1 2027 deliverable is a published migration path, not a migration:

> "The realistic threat is store-now-decrypt-later, not live breakage. We publish a migration path on the NIST-standardised signature schemes. Migration itself is a multi-year effort — this is where Bitcoin and Ethereum are too." [doc: flux-roadmap-public.md:148-149]

VISION [roadmap: Post-quantum roadmap, H1 2027]. Consistent with that, a repository-wide grep for the obvious primitive names returns nothing: [fluxd/src:repo-wide grep for `quantum|dilithium|kyber|falcon|sphincs|schnorr`: zero hits outside a vendored `secp256k1/.gitignore`]. No post-quantum code, comment, or dependency exists in `fluxd` today.

What is at risk. The signature primitives at risk are all classical-ECDSA over secp256k1: transparent-address spending; FluxNode registration and confirmation signing, via `SignDeterministicStartTx` and `BuildDeterministicConfirmTx` [fluxd/src/fluxnode/activefluxnode.cpp:253–288,380–407]; the PoN block header's own `vchBlockSig` field, an ECDSA signature over the block hash carried in every post-activation header alongside the winning collateral outpoint [fluxd/src/primitives/block.h:44–45,64–96]; the 2-of-4 emergency-block key set [fluxd/src/chainparams.cpp:242–253]; and the benchmark-signing key allowlist [fluxd/src/chainparams.cpp:233–240], alongside the attestation-signing keys discussed in Section 14. Hashing is a separate matter: SHA-256 and the historical chain's Equihash instantiation degrade only quadratically under a quantum adversary, which is a general property of quantum search against hash-based primitives rather than a Flux-specific finding, and is not the part of the system this scope decision needs to address [inferred].

The scope decision. The team has settled, as design intent, that the post-quantum scope is deliberately bounded to consensus, transparent transactions, and the keys securing them, with the shielded layer explicitly excluded and deferred as the field matures, on the grounds that privacy is optional on Flux and the transparent layer is the one every user and every consensus rule depends on [intent: design intake] VISION. This is presented as a considered position, not a gap, and it carries a consequence the paper states plainly because a reviewer would otherwise find it: Groth16, the proving system underlying the shielded pool described in Section 20, rests on a pairing assumption and is not post-quantum secure. An unqualified "quantum-ready Flux" claim made alongside a live shielded pool built on Groth16 would therefore be an error; the bounded scope stated here is exactly what avoids making it [intent: design intake].

The retirement decision simplifies this considerably. With both shielded pools retired (§20.9), Groth16 leaves the consensus path altogether, and with it the one component whose presence made a "quantum-ready Flux" claim require careful qualification. After retirement the post-quantum scope is not *bounded* to transparent transactions — it is *coextensive* with the chain, because transparent transactions and the keys securing them are all that remains. That is a genuine simplification of the migration problem stated in this section, and it was not among the reasons given for the decision, which makes it worth recording as a consequence rather than a justification.

A related and separate point, kept separate deliberately: adopting Halo2/Orchard-class proving would remove the shielded layer's inherited trusted setup, which is a real, independently motivated improvement discussed in Section 20 — but it would not make the shielded pool post-quantum secure, since Halo2 rests on discrete-log hardness rather than a quantum-hard assumption. Eliminating a trusted setup and achieving post-quantum security are different properties, and this section, Section 14, and Section 20 must not conflate them [intent: design intake].

The migration problem. Signature migration on a UTXO chain is not an ordinary protocol upgrade, because the protocol cannot move funds it does not hold the key to.

Open Problem 8.4. Let U_{cls} be the set of outputs spendable only by a classical signature. No consensus rule can move an output in U_{cls} without the corresponding private key. A store-now-decrypt-later adversary harvests public keys revealed at spend time and, once a capable quantum adversary exists, spends any output in U_{cls} whose public key it has recorded. A migration must therefore (i) offer a post-quantum output type, (ii) incentivize or compel movement of value into it, and (iii) decide what happens to outputs in U_{cls} that never move.

Item (iii) has no good answer anywhere in the industry today, and the honest statement is that Flux is in the same position as Bitcoin and Ethereum, not ahead of them [doc: flux-roadmap-public.md:149]. The design space for the disposition of unmoved classical outputs has three known shapes, each with a distinct trade-off:

1. *A flag-day freeze.* After a fixed height, any output in U_{cls} whose owner has not moved it becomes permanently unspendable. This forces the migration but is confiscatory by construction for anyone who cannot act in time — lost keys, custodial delay, simple inattention — which is exactly the property the collateral-maturity rule above was designed to avoid.

2. *A permanent classical tail.* Nothing is ever frozen; the network accepts that any output in U_{cls} whose public key becomes known remains at risk indefinitely once a capable adversary exists. Nothing is confiscated, but the exposure never closes.

3. *A hash-based commitment allowing a post-quantum proof of prior ownership.* An owner could later prove, using a post-quantum-secure proof, that they controlled a given classical output before some cutover, without needing a pre-committed post-quantum key at the time of the original output. This is the least destructive of the three on its face, but so far as this section's sources establish, no production-grade construction of this kind is deployed anywhere at the scale a base-layer chain would require [inferred].

Remark 8.5 (FluxNode collateral public keys are already public, by protocol design). The three dispositions above are the general framing, and it is the framing Bitcoin and Ethereum share. Flux is not in the same position, and the difference is material enough that the disposition question has to be answered per class rather than once.

On a Bitcoin-derived chain the structural mitigation for unmoved funds is that a P2PKH output commits to HASH160(pk) rather than to pk: the public key is not on the chain until the output is spent, so an adversary running Shor's algorithm has nothing to attack. That protection does not exist for FluxNode collateral. The `collateralPubkey` field is serialised directly into the FluxNode START transaction and written to the chain at registration SHIPPED [fluxd/src/primitives/transaction.h:1038–1039]; [fluxd/src/fluxnode/activefluxnode.cpp:368]. Every operating node has therefore *published* its collateral public key as a condition of operating, permanently and in the clear.

Table 26: Collateral whose public keys are on-chain by protocol design. Node counts and tier split SHIPPED [measured: `listzelnodes`, 2026-09-03]; supply from the emission model at height 2,912,015.

Tier	Nodes	Collateral each (FLUX)	Tier total (FLUX)
Cumulus	3,247	1,000	3,247,000
Nimbus	1,615	12,500	20,187,500
Stratus	1,627	40,000	65,080,000
Total	6,489		88,514,500
Share of issued supply (429,466,824 FLUX)			20.61 %

So roughly one fifth of issued supply sits in outputs that a cryptographically relevant quantum computer could spend immediately, without waiting for the owner to reveal anything, and that fraction is not the result of user carelessness — it is what the registration protocol requires. This is a genuine architectural disadvantage relative to Bitcoin on this specific axis, and the paper states it rather than inheriting Bitcoin's more comfortable framing.

A correction this paper owes its own earlier reasoning. An earlier draft concluded that option (iii) — a post-quantum proof of prior ownership — was *unavailable* for FluxNode collateral, on the argument that such a proof rests on the owner knowing a secret the adversary cannot compute, which fails once pk is published and Shor yields the private key. That argument is wrong, and Bitcoin's own work shows why: the secret need not be the private key. BIP-361 rests its rescue path on the knowledge asymmetry created by **BIP-32 hardened derivation** [37]. A hardened step derives the child key through a one-way function of the parent seed, so recovering the child private key does not yield the seed. An owner can therefore prove *"I know a*

seed and a path that derive this key" while a quantum adversary holding the private key cannot. Option (iii) survives public-key exposure, provided the key was seed-derived through a hardened step — which is the normal case for wallet-generated keys and not the case for raw imported ones.

So the design space for FluxNode collateral does *not* collapse to freeze-or-expose. It collapses to something better, and §8.4 builds on it.

8.4 The frontier: what Bitcoin is building, and the one move Flux can make that it cannot

Bitcoin is the right comparison for this problem, because it faces the same one at larger scale and its work is public, specified and adversarially reviewed. Two proposals define the current state of that art, and Flux should be read against them rather than against a generic notion of "quantum readiness".

BIP-360 defines *Pay-to-Merkle-Root* (P2MR): an output type offering nearly the functionality of P2TR with the key-path spend removed, so that no internal public key is exposed and a long-exposure attack by a cryptographically relevant quantum computer has nothing to work on. It is Draft status, and it deliberately introduces *no* post-quantum signature scheme — the algorithm choice is left to later work [6]. **BIP-361** supplies the migration policy: Phase A, at 160,000 blocks after activation, disallows sending funds *to* quantum-vulnerable addresses; Phase B, two years later, tightens verification of ECDSA and Schnorr signatures, freezing what has not moved. Its authors reject allowing theft outright, and its rescue path rests on the hardened-derivation asymmetry described above — with zk-STARK and commit-reveal named as candidate proof systems and the honest admission that how much supply such a protocol could cover "remains to be seen" [37]. As of 2026-03-01, over 34 % of all bitcoin had revealed a public key on chain [inferred].

Where Flux sits against that. Two differences matter, and they point in opposite directions.

Worse: Bitcoin's exposed fraction is the product of user *behaviour* — spending, address reuse — and it grows monotonically because each spend reveals a key. Flux's exposed fraction for node collateral is the product of *protocol mandate*: 20.61 % of supply (Table 26) is exposed because the START transaction requires it, and an operator cannot decline. No amount of careful user behaviour reduces it.

And a consequence Bitcoin's approach would not fix: BIP-360's remedy is an output type that hides the key in the *scriptPubKey*. That does not help Flux, because Flux's collateral exposure is not in the output script at all — it is in the `collateralPubkey` field of the FluxNode transaction [fluxd/src/primitives/transaction.h:1038–1039]. Adopting a P2MR-equivalent output type would leave the exposure exactly where it is. **The registration transaction format is what would have to change**, and that is a Flux-specific problem with no upstream solution to inherit.

Better, and decisively: **Flux has a mandatory, recurring, authenticated channel to precisely the exposed set, and Bitcoin has none.** Bitcoin cannot reach a dormant holder; its only instruments are to stop new exposure, to freeze, and to hope a rescue proof covers enough. Every Flux node, by contrast, must broadcast a START transaction to register and CONFIRM transactions to keep being paid. The set with exposed keys *is* the set transacting regularly and drawing income — and that turns an unreachable population into a reachable one.

Remark 8.6 (Pre-staged commitment: the move the mandatory registration makes available). The recommendation of Definition 8.7 is eligibility-forced migration, and the team has adopted it [intent: design intake] (PLANNED). The frontier version of it exploits a timing asymmetry that Bitcoin cannot use, and costs almost nothing to start now.

Extend the FluxNode START and CONFIRM transactions with a **post-quantum key commitment**: a pair $(\mathsf{alg_id}, H(pk_{\mathrm{pq}}))$, the algorithm identifier and a hash of a post-quantum

public key the operator controls. Three properties make this the right shape.

1. **It is nearly free to add, because the transaction is already mandatory.** These transactions already carry a public key and are already broadcast by every node on a schedule. A commitment field is marginal cost on a message that must be sent anyway — which is exactly the cost structure Bitcoin does not have, because it has no mandatory message.

2. **It commits to a hash, not a key, and names its algorithm.** Publishing pk_{pq} itself would recreate the long-exposure problem one algorithm later; publishing $H(pk_{\mathrm{pq}})$ does not. Carrying alg_id keeps the scheme agile, so the NIST primitive can be chosen later without invalidating commitments made earlier — and §20.7 is explicit that no primitive is chosen yet.

3. **It inverts the migration.** If commitments are collected from today, then by the time a CRQC is credible every economically active node already has a post-quantum key committed on chain, and the cutover is a *flag*, not a migration: consensus begins requiring a pk_{pq}-authorised spend path, and the keys it needs are already there. Bitcoin's Phase A/Phase B schedule spends roughly five years doing what a mandatory registration channel can pre-stage silently.

And no collateral is ever frozen. This is the substantive divergence from BIP-361, which chooses freezing and defends the choice against the alternative of tolerated theft [37]. Flux does not face that dilemma for its largest exposed class, because eligibility does the work freezing does: a node that has not committed and migrated stops being confirmed and stops earning, while its collateral stays spendable by its owner. The instrument is an income incentive rather than a confiscation, and it applies continuously rather than at a flag day.

What this does not solve. Two residues, stated rather than absorbed. Collateral held behind *raw imported* keys has no seed to prove derivation from, so the hardened-derivation rescue does not reach it; those operators must migrate or lose eligibility, with no fallback. And ordinary transparent outputs outside the collateral set — class (c) of Definition 8.7 — get no benefit from any of this, because their holders have no registration channel. For that class Flux is in exactly Bitcoin's position and should adopt whatever BIP-361's rescue work establishes rather than invent a parallel scheme.

Decided. The pre-staged post-quantum key commitment of Definition 8.6 is this paper's construction. What is settled is the disposition it extends — eligibility-forced migration for FluxNode collateral [intent: design intake]. The commitment is separable from it: eligibility-forced migration works without pre-staging, and pre-staging simply makes the eventual cutover a flag rather than a migration. Its value of alg_id is gated on NIST primitive selection and is out of scope here; the height at which the commitment becomes mandatory for confirmation is a scheduling decision that costs nothing to defer, because commitments collected voluntarily before that height are equally useful.

Remark 8.7 (The adopted disposition: per class, using eligibility rather than confiscation). **The team has adopted the per-class disposition below** [intent: design intake] (PLANNED). It assigns a disposition per output class rather than choosing one of the three globally, because the classes differ in exactly the property the dispositions turn on — whether the public key is already known.

(a) FluxNode collateral (20.61 % of supply): forced migration through *eligibility*, not through freezing. This is where Flux has an instrument Bitcoin does not, and it is the recommendation this section rests on. Collateral is a *registered* set: every outpoint is enumerable from `g_fluxnodeCache` [fluxd/src/fluxnode/fluxnodecachedb.cpp:26,49–79], and continued

operation already requires periodic confirmation transactions. After a stated height, require that a node's collateral be held in a post-quantum output type as a condition of *confirmation*. A node that does not migrate stops being confirmed, leaves $\mathcal{W}$, and stops earning — but **its funds are never frozen and nothing is confiscated**. The operator faces a continuous economic incentive to migrate, applied through a mechanism that already exists, with no new consensus concept and no flag day. For the class that carries the largest exposure and the fewest options, this converts an unsolved industry problem into an incentive-compatible one.

(b) Ordinary transparent outputs whose public key has never appeared on chain: no flag-day action. These retain the HASH160 protection. The honest characterisation is that Shor breaks ECDSA outright, in polynomial time, whereas Grover only quadratically accelerates preimage search — so a 160-bit hash retains roughly 80 bits of security against a quantum adversary. That is far stronger than the exposed-key case and weak enough that it should not be described as permanent safety; it buys time to specify a post-quantum spend path, which is the correct use of it.

(c) Ordinary transparent outputs with a revealed public key (address reuse, and any output already spent from): permanent classical tail, stated as exposed. Freezing this class is confiscatory and its size is small relative to (a). The recommendation is to accept the exposure and say so, rather than to confiscate.

Not established by this survey. the size of class (c) — transparent outputs with revealed public keys outside FluxNode collateral — is not measured in this corpus. It is computable from the chain by scanning spent scriptSigs and is a prerequisite for stating the total exposed fraction rather than the collateral component alone.

Decided. Eligibility-forced migration for FluxNode collateral, per class as above [intent: design intake]. The finding it rests on — Definition 8.5, that collateral public keys are on-chain by design — is verified in source. Open: the height at which the confirmation requirement applies, the post-quantum output type, and the pre-staged commitment of Definition 8.6. The primitive itself is gated on NIST selection and is out of scope for this paper.

Class: Research for item (iii) above; **Class: Engineering** for items (i) and (ii) — defining a post-quantum output type and a movement incentive are ordinary, if substantial, engineering tasks once NIST-standardised primitives are chosen, whereas the disposition of unmoved funds is unsolved in the field.

One advantage Flux has that a general UTXO migration does not. FluxNode collateral is a *registered* set: every collateral outpoint and the keys and payment address registered with it are enumerable from the same `g_fluxnodeCache/determ_zelnodes` registry already cited in Section 8's pruning discussion [fluxd/src/fluxnode/fluxnodecachedb.cpp:26,49–79]. A migration that targets node collateral first is therefore tractable in a way a general transparent-UTXO migration is not — the set of holders is known in advance, and node operators, who depend on continued participation for income, are plausibly a more reachable and motivated constituency than an anonymous holder of an old address [inferred]. This narrows where migration effort would be spent first; it does not by itself resolve the disposition question above, which Definition 8.7 answers per class for collateral that is never moved.

8.5 What the three share

All three changes discussed in this section are consensus changes proposed for or riding on a chain that completed its last major consensus transition roughly ten months before this writing and needed emergency intervention to do it [inferred] [fluxd/src/chainparams.cpp:281,284]. `MAX_REORG_LENGTH`, normally 40 blocks, was overridden to 5,000 for the height range $[2{,}020{,}000, 2{,}025{,}000]$ [fluxd/src/main.cpp:8983–8988], alongside a checkpoint cluster at heights 2,020,500 through 2,029,000

labelled “PoN activation and stabilization checkpoints – EMERGENCY FORK RECOVERY” in the source [fluxd/src/chainparams.cpp:277–283]. This is the empirical base rate for what a consensus change has cost this network so far, and it is the frame Section 27 uses to weigh future risk.

Read against that history, the roadmap’s decision to have collateral maturity go to operators for discussion and ride an already-planned network upgrade, rather than force its own, is the correct lesson drawn from it [roadmap: Collateral maturity, H1 2027]. Neither pruning nor post-quantum migration is described as forcing a dedicated upgrade either: pruning needs no consensus change at all, and post-quantum migration is explicitly a multi-year effort with no activation height proposed yet.

One further observation belongs here because it connects three otherwise separate sections rather than staying inside this one. The PoN block header commits to a single winning collateral outpoint, not to the FluxNode registry as a whole [fluxd/src/primitives/block.h:44–45,64–96], and every PoN block is assigned the same fixed chainwork contribution of 2^{40} regardless of its actual difficulty target [fluxd/src/pow.cpp:284–290]. Together with the 2-of-4 emergency-block path described above, this means a light client cannot today verify PoN block validity or compare competing chains without the full registry — which forecloses the most trust-minimizing bridge architecture discussed in Section 22. A future header change that commits to the node set as a whole would reopen that possibility, and it is a natural candidate for the same network upgrade this section already expects collateral maturity to ride [inferred].

PART III

The cloud

9 FluxOS: orchestration, placement, and the application lifecycle

FluxOS is the Node.js daemon that every FluxNode operator runs. It exposes the node's HTTP/HTTPS API, keeps a gossip mesh open to other FluxOS instances, and decides — locally, with no message sent to any other component asking permission — whether to pull an image, start a container, leave it running, redeploy it, or tear it down. There is no scheduler in the sense a reader from conventional cluster orchestration would expect: no process anywhere holds a global view of node load and assigns work to relieve it. Instead, every node runs the same loop over the same gossiped state and reaches its own conclusion about which of the network's under-replicated applications to attempt next. What keeps two thousand independent conclusions from producing three thousand copies of the same container is not agreement — it is a fixed wait, a broadcast, and a rule for breaking ties by whoever broadcast first. That mechanism, its guarantees, and its failure mode are the center of this section. Everything below is SHIPPED and cited to the deployed FluxOS source unless a claim is explicitly marked otherwise; where the evidence gathered for this paper itself records a code path as not fully traced, this section says so rather than smoothing the gap into confident prose. Line numbers in `flux/...` citations are read against the organisation repository's working checkout at commit `933614b1c`, one unpublished commit ahead of `master` — the flagged SPEC v9 schema gate described in §10, which also appends a price-table row at height 2,950,000 with unchanged rates; where a cited construct exists only on that branch, the citation says so.

9.1 Process architecture

9.1.1 Processes and ports

FluxOS starts as two entry points: `apiServer.js` (the HTTP/HTTPS API) and `homeServer.js` (a static UI server) [flux/apiServer.js:285]. `apiServer.js` calls `serviceManager.startFluxFunctions()` only after both listeners are up [flux/apiServer.js:285], so the boot sequence described below begins after the ports are already bound.

Table 27: FluxOS process ports and adjacent service ports.

Listener	Value	Source
API port (default)	16127	[flux/ZelBack/config/default.js:28]
Allowed API ports (boot aborts otherwise)	{16127,16137,16147,16157,16167,16177,16187,16197}	[flux/ZelBack/config/default.js:26]
API HTTPS port	apiPort + 1	[flux/apiServer.js:38]
Home UI port	apiPort − 1	[flux/homeServer.js:19]
Syncthing (by convention)	apiPort + 2, absolute default 8384	[doc: flux/ZelBack/config/default.js:405]
MongoDB	127.0.0.1:27017	[flux/ZelBack/config/default.js:31–32]
`fluxbench` RPC / P2P port	16224 / 16225 (testnet 26224 / 26225)	[flux/ZelBack/config/default.js:102–106]
`fluxd` P2P / RPC / ZMQ port	16125 / 16124 / 16123 (testnet 26125 / 26124)	[flux/ZelBack/config/default.js:109–115]

The HTTPS listener uses a self-signed certificate generated on first boot by `helpers/createSSLcert.sh` if `certs/v1.key` is absent [flux/apiServer.js:248–258]. Boot hard-blocks on `dbHelper.waitForMongo()` and `dockerService.waitForDocker()` before any other service starts; the code's own comment reads "hard dependencies — nothing starts until these are confirmed" [doc: flux/ZelBack/src/services/serviceManager.js:132] [flux/ZelBack/src/services/serviceManager.js:133–134].

9.1.2 Boot sequence

`serviceManager.startFluxFunctions` runs a long, mostly-`await`ed sequence [flux/ZelBack/src/services/serviceManager.js:126–572]. In order: (1) reject an unsupported API port and exit [flux/ZelBack/src/services/serviceManager.js:126–129]; (2) start the enterprise node/owner map sync from `helpers/enterprisenodes.json`, then resync from GitHub every 6 h, with fetch failure swallowed by a `.catch` handler so boot proceeds on stale data [flux/ZelBack/src/services/

serviceManager.js:130–136]; (3) hard-block on Mongo and Docker availability (above); (4) start UPnP firewall maintenance if a router IP is configured [flux/ZelBack/src/services/serviceManager.js:138–153]; (5) start `fluxNodeService`, repair Docker log corruption, and start `systemService.monitorSystem()` [flux/ZelBack/src/services/serviceManager.js:158–163]; (6) drop the `loggedusers`/`activeloginphrases`/`activesignatures` collections — every restart invalidates all sessions — and (re)create TTL indexes: `loggedusers` 14 d, `activeloginphrases`/`activesignatures` 900 s, `activepaymentrequests` 3,600 s, `completedpayments` 7 d, `apptamperingevents` 30 d [flux/ZelBack/src/services/serviceManager.js:165–215]; (7) run tamper-reboot detection and merge pre-unique-index duplicate runtime-state documents [flux/ZelBack/src/services/serviceManager.js:216–220]; (8) prepare global-apps indexes, including TTL-by-`expireAt` on the location, event, and installing-broadcast collections [flux/ZelBack/src/services/serviceManager.js:227–260]; (9) run a one-off repair of a legacy `valueSat: NaN` bug [flux/ZelBack/src/services/serviceManager.js:265–267]; (10) schedule volume-mount validation at 45 s and hardware-requirement validation (which removes non-compliant apps) at 50 s, explicitly "before boot reconciliation" [flux/ZelBack/src/services/serviceManager.js:285–297]; (11) migrate any container still on Docker's native `unless-stopped`/`always` restart policy to `no`, "because FluxOS manages container startup after dbReady" [doc: flux/ZelBack/src/services/serviceManager.js:301] [flux/ZelBack/src/services/serviceManager.js:299–302]; (12) start the container reconciler and its Docker die-event bridge, which begin queuing triggers but do not yet drain them [flux/ZelBack/src/services/serviceManager.js:304–310]; (13) read boot context and, fire-and-forget, reconcile apps on boot [flux/ZelBack/src/services/serviceManager.js:312–319]; (14) build the `fluxd` client and block on `waitForDaemonRpc()` — the code comment notes this daemon must be running before `daemonReady` is set, and that boot reconciliation separately enforces an internal 5-minute timeout on the same event [flux/ZelBack/src/services/serviceManager.js:321–328]; (15)–(19) wire the app-sync orchestrator, adjust the firewall, remove any stray watchtower container, and start P2P discovery unless `config.fluxapps.discoveryAutostart === false` [flux/ZelBack/src/services/serviceManager.js:319–414]; (20) register the long tail of periodic monitors listed in Table 28. On any thrown exception anywhere in this function, the entire function is retried after a flat 15,000 ms — not an exponential backoff — with no retry cap [flux/ZelBack/src/services/serviceManager.js:568–572]; boot is therefore a self-healing infinite retry loop rather than a circuit breaker, a property revisited in §9.10.

Table 28: Representative periodic monitors registered at the end of boot.

Monitor	Interval
Firewall/UFW re-check + mount test	30 s
Stop-non-Flux-container sweep, app-availability monitor, port restore	1 min (port restore repeats every 600,000 ms)
Enterprise identity resolution	every 5 min until resolved
Node-status monitor	1.5 min
CPU-usage check (`cpuCheckIntervalMs`)	900,000 ms (15 min)
App-availability check	3 min
Image compliance sweep	3,600,000 ms (1 h)
Forced-removal sweep	30 min, then every 2 h
Daemon health monitor	5 min, then every 15 min
Storage-space check	20 min
Local-backup cleanup	25 min

[flux/ZelBack/src/services/serviceManager.js:433–563]

9.1.3 Databases and collections

FluxOS keeps five Mongo databases, all on the local instance bound above [flux/ZelBack/config/default.js:33–87]:

- `zelfluxlocal` — session and local-node bookkeeping: `loggedusers`, `activeloginphrases`, `activesignatures`, `activepaymentrequests`, `completedpayments`, `geolocation`, `benchmark`, `apptamperingevents`, `nodestartuptracker`.

- `zelcashdata` — a mirror of `fluxd` chain-scan state: `scannedheight`, `utxoindex`, `addresstransactionindex`, `zelnodetransactions`, `zelappshashes`, `coinbasefusionindex`.
- `localzelapps` — this node's own truth about its containers: `zelappsinformation` and `zelappsruntimestate` (the per-component controller state consumed by the reconciler in §9.4: desired state, restart history/crash-backoff position, last exit).
- `globalzelapps` — network-global truth, replicated by P2P gossip and not by consensus (§9.6): `zelappsmessages`, `zelappsinformation`, `zelappstemporarymessages`, `zelappslocation`, `appsinstallinglocations`, `appsInstallingErrorsLocations`, `appstateevents` (an event log of `apprunning`/`sigterm`/`appremoved`/`evicted` transitions), `fluxappinstallingbroadcasts`, `fluxappinstallingerrorsbroadcasts`.
- `chainparams` — `chainmessages`: soft-fork price/parameter messages posted on-chain by the multisig whose keys the four co-founders hold personally (K8 in Definition 2.3), active from their occurrence height (§9.8).

9.1.4 HTTP API surface and the privilege model

The route table (`ZelBack/src/routes.js`) is organized into groups, each guarded by `verificationHelper.verifyPrivilege(privilege, req, appName)`. Every check is keyed off a `zelidauth` header `{zelid, signature, loginPhrase}` verified with `bitcoinMessage.verify` against the ZelID, a base58 P2PKH-style address [flux/ZelBack/src/services/verificationHelper.js:17–48,90–106].

Table 29: Privilege tiers enforced by `verifyPrivilege`.

Privilege	Definition	Freshness window	Representative endpoints
`admin`	`zelid === userconfig.initial.zelid` — the node's own operator [flux/ZelBack/src/services/verificationHelperUtils.js:19–46]	re-signed each call	`/daemon/stop`, `/daemon/reindex`, `/daemon/createfluxnodekey`, `/daemon/signrawtransaction`
`fluxteam`	`zelid` ∈ {`fluxTeamFluxID`, `fluxSupportTeamFluxID`} [flux/ZelBack/src/services/verificationHelperUtils.js:96–122]	re-signed each call	cross-node views spanning more than this node's own data
`adminandfluxteam`	union of the two above [flux/ZelBack/src/services/verificationHelperUtils.js:130–156]	re-signed each call	`/daemon/start`, `/daemon/restart`, `/daemon/ping`, `/daemon/startbenchmark`, `/flux/broadcastmessage*`
`appowner`/`appownerabove`	caller matches the app's current owner as resolved by `registryManager.getApplicationOwner(appName)`; the `-above` variant also admits admin/fluxteam [flux/ZelBack/src/services/verificationHelperUtils.js:164–248]	2 h	`/apps/appstart`, `/apps/appstop`, `/apps/apprestart`, `/apps/appremove`, `/apps/redeploy`, `/apps/redeploycomponent`
`user`	any ZelID with a valid signature over a `loginPhrase` carrying a 13-digit millisecond timestamp no older than 16 h, unless already in `loggedusers` [flux/ZelBack/src/services/verificationHelperUtils.js:54–88]	16 h	`/apps/appregister`, `/apps/appupdate` (self), `/daemon/prioritisetransaction`

All six checks re-verify the ECDSA signature over the login phrase even when a `loggedusers` session document exists; the session document only relaxes the timestamp-freshness check, never the signature check [flux/ZelBack/src/services/verificationHelperUtils.js:54–248]. The route groups themselves are: `/daemon/*` (a thin proxy to `fluxd`'s JSON-RPC surface, split across public/cached, `user`, `admin`, and `adminandfluxteam` endpoints) [flux/ZelBack/src/routes.js:78–1408]; `/apps/*` (public/cached reads, `appownerabove` lifecycle actions, a `user`-level `/apps/appregister`, and an `/apps/appupdate` reachable by the owner or by `adminandfluxteam` on the owner's behalf) [flux/ZelBack/src/routes.js:372–1388]; `/id/*` (ZelID session management) [flux/ZelBack/src/routes.js:508–518]; `/benchmark/*` (a proxy to `fluxbench` RPC) [flux/ZelBack/src/services/benchmarkService.js:126–427]; and `/flux/*` (network-control endpoints, including the broadcast primitives used by §9.6) [flux/ZelBack/src/routes.js:1022].

9.2 The application lifecycle

An application moves through eight named stages between a user's signature and a running container. Table 30 collects the timing constants; the paragraphs below name each stage and its transition.

Construct and submit. The client signs `{appSpecification, timestamp, signature, type, version}`, where `type` must be `zelappregister` or `fluxappregister` and the envelope `version` (distinct from `appSpecification.version`) must be 1 [flux/ZelBack/src/services/appDatabase/registryManager.js:1755–1764], and `POST`s it to `/apps/appregister` at `user` privilege [flux/ZelBack/src/services/appDatabase/registryManager.js:1714–1737].

Verify. The node rejects the request if it has fewer than 8 outbound or 4 inbound peers ("not enough peers for safe application registration") [flux/ZelBack/src/services/appDatabase/registryManager.js:1744–1750]; rejects a SPEC v9 spec unless `config.fluxapps.acceptV9` is set (default `false`; §10 covers the version-gating scheme in full) — a check that exists only on an unpublished working branch, not in the deployed source (IN-PROGRESS) [RunOnFlux/flux@feat/appspec-v9:ZelBack/src/services/appDatabase/registryManager.js:1766-1769]; rejects a timestamp more than 1 h old or more than 5 min in the future [flux/ZelBack/src/services/appDatabase/registryManager.js:1777–1781]; and, once the daemon is confirmed synced, runs the full schema/version gate (`appValidator.verifyAppSpecifications`; §10 details the version-gating rules) [flux/ZelBack/src/services/appDatabase/registryManager.js:1783–1798]. For enterprise apps the signature is verified against the *unformatted* spec — the signature covers the pre-encryption content — after which `contacts/compose` are zeroed out of the object that is actually hashed and stored [flux/ZelBack/src/services/appDatabase/registryManager.js:1809–1824]. The node then computes `messageHASH` = SHA-256(type ‖ version ‖ JSON(spec) ‖ timestamp ‖ signature) [flux/ZelBack/src/services/appDatabase/registryManager.js:1827–1829].

Broadcast (temporary). The message is stored with a 1 h TTL and broadcast to all peers [flux/ZelBack/src/services/appDatabase/registryManager.js:1831–1841]. The API call then blocks through a fixed confirmation round-trip — `delay(1200)`, request the message back from peers, `delay(1200)`, then poll for its own existence up to 20 times at 500 ms — and throws *"Unable to register application on the network. Try again later."* if it is still absent [flux/ZelBack/src/services/appDatabase/registryManager.js:1843–1858]. On success the API returns only the message hash: this endpoint never touches the chain, and the caller must still pay (§9.8) for the registration to ever become permanent [flux/ZelBack/src/services/appDatabase/registryManager.js:1737].

On-chain confirmation. `fluxd`'s chain scan (`explorerService.processInsight`, run per block) recognizes a payment by its output address — `fluxapps.address` at any height, or one of two height-gated multisig addresses — sums `valueSat` across matching outputs, and extracts the temp-message hash from the transaction's `OP_RETURN` [flux/ZelBack/src/services/explorerService.js:309–345], queuing `messageVerifier.checkAndRequestApp` [flux/ZelBack/src/services/explorerService.js:444].

Promote to permanent global state. `checkAndRequestApp` re-verifies update signatures against the current permanent owner immediately before promotion — an explicit anti-race measure against two updates racing the same base state [flux/ZelBack/src/services/appMessaging/messageVerifier.js:636–682] — computes the PoN-fork-aware expiration height, and for a fresh registration requires `valueSat` $\geq$ `appPrice` $\times 10^8$. **If underpaid, the registration is silently dropped**: the code logs a warning and returns, with no error surfaced to the payer, no retry, and no refund path visible in this repository [flux/ZelBack/src/services/appMessaging/messageVerifier.js:735–763]; updates have the same silent-drop behavior on underpayment [flux/ZelBack/src/services/appMessaging/messageVerifier.js:797–828]. This is a correctness gap, not a documented design choice, and §16 takes up what a payer-facing signal should look like. On success, `insertAppSpecifications` makes the spec part of `globalzelapps`, and any updates queued while the registration was outstanding are flushed [flux/ZelBack/src/services/appMessaging/messageVerifier.js:735–763].

Placement. Every node independently decides whether to install (§9.3).

Docker deployment and running. `appInstaller.registerAppLocally` performs port setup, image pull/verification, and Docker network creation before calling `installApplicationHard` or `installApplicationSoft` [flux/ZelBack/src/services/appLifecycle/appInstaller.js:415–746]; the evidence available does not trace the exact Docker API calls (container-create options, network-attach order, volume mount construction) inside those two functions beyond their call sites [inferred]. Container start/stop itself is centralized in the reconciler (§9.4), not in the installer.

Monitoring and expiry. While running, the app is subject to the node-health, benchmark, and image-compliance gates of §9.7 and §9.9, and — where the operator has opted the app into it — per-container telemetry shipped by `flux-telemetryd` to a customer-chosen backend, sampled every 15 s [flux-telemetryd/src/identity.rs:1–31,56–74] [flux-telemetryd/src/collector.rs:26] [doc: flux-telemetryd/README.md:1-20] (SHIPPED, a companion daemon evidenced among the node-level services surrounding FluxOS rather than in FluxOS itself). A departing node's `fluxnodesigterm` broadcast extends its apps' location TTL by `SIGTERM_EXPIRY_MS` = 420,000 ms (7 min) rather than expiring them immediately (§9.6); a companion node-wide/per-app graceful-drain daemon, `flux-shutdownd`, exists to broadcast a machine-readable termination reason to the container before this happens, but only its signal-and-cleanup pipeline stages are wired to production events today — drain-broadcast and `preStop` execution are modelled and unit-tested but not yet reachable [doc: flux-shutdownd/crates/shutdownd/src/pipeline/state_machine.rs:3-7] (IN-PROGRESS). An app's actual expiration height, once reached, is enforced by a periodic expiry pass every `expireFluxAppsPeriod` = 100 blocks [flux/ZelBack/config/default.js:298].

Table 30: Lifecycle timing constants.

Stage / constant	Value
Registration timestamp window	> 1 h old or > 5 min future rejected [flux/ZelBack/src/services/appDatabase/registryManager.js:1777–1781]
Temp-message TTL	3,600 s [flux/ZelBack/config/default.js:314]
Registration confirmation round-trip	2 × 1,200 ms delay + up to 20 × 500 ms poll (≈8–10 s) [flux/ZelBack/src/services/appDatabase/registryManager.js:1843–1858]
`minOutgoing` / `minIncoming` peers to register	8 / 4 [flux/ZelBack/config/default.js:262–265]
Location TTL	7,500 s [flux/ZelBack/config/default.js:311]
Installing-broadcast TTL	900 s [flux/ZelBack/config/default.js:312]
Install-error TTL	3,600 s [flux/ZelBack/config/default.js:313]
SIGTERM grace window	420,000 ms (7 min) [flux/ZelBack/src/services/utils/appConstants.js:86]
`blocksLasting` (default lifetime)	22,000 blocks (≈1 month pre-PoN, ≈1 week post-PoN) [flux/ZelBack/config/default.js:285]
Expiry sweep period	100 blocks [flux/ZelBack/config/default.js:298]
Update sweep / removal sweep period	9 / 11 blocks [flux/ZelBack/config/default.js:299–300]

9.3 Placement: opportunistic self-selection with broadcast collision-avoidance

There is no hash-seeded, deterministic node-selection algorithm, and no component that assigns an application to a node. Every node runs the same self-looping procedure (`appSpawner.trySpawningGlobalApplication`, invoked as `spawnLoop` with the delay returned by its previous iteration) [flux/ZelBack/src/services/appLifecycle/appSpawner.js:54–65,77–787], against its own local read of the gossiped `globalzelapps` state. This is the mechanism the outline's single claim is about, and Algorithm 2 states it in full.

Definition 9.1 (Opportunistic self-selection)**.** A node that observes an application with fewer running instances than its specification requests may attempt to become one of those instances. It decides to attempt *unilaterally*: no other node, and no on-chain transaction, grants or denies it that attempt in advance. Conflicts among simultaneous attempts are resolved after the fact, by broadcast and a wait, not before the fact by assignment.

Algorithm 2: One iteration of the per-node placement loop [flux/ZelBack/src/services/appLifecycle/appSpawner.js:77–787]

Data: this node's local state; its local replica of `globalzelapps`

if ***not*** *(enterprise identity resolved* ***and*** *daemon synced* ***and*** *local DB ready* ***and not*** *DoS-flagged* ***and*** *node confirmed* ***and*** *`fluxbench` not erroring* ***and*** *own external IP known* ***and*** *locally installed apps <* *`maxAppsPerNode` = 200)* **then**
  return (retry after the loop's error-path delay) `// gate,` [flux/ZelBack/src/services/appLifecycle/appSpawner.js:78–150]

$M \leftarrow$ apps whose actual (PoN-fork-aware) expiration height has not passed and whose running instance count is below `instances` $\vee 3$ `// Mongo aggregation,` [flux/ZelBack/src/services/appLifecycle/appSpawner.js:152–247]

if *a due entry exists in the deferred-recheck queue* ***`appsToBeCheckedLater`*** **then**
  $a \leftarrow$ pop that entry `// deterministic, not random`
else
  $C \leftarrow \{a \in M : a.$`nodes` names this node's own IP$\}$
  if $C \neq \emptyset$ **then**
    $a \leftarrow$ uniform-random draw from C
  else
    $a \leftarrow$ uniform-random draw from M
  `//` [flux/ZelBack/src/services/appLifecycle/appSpawner.js:254–336]`: own-IP bias, then uniform`

if *a fails a node-targeting or geolocation filter* **then**
  discard a this cycle, continue the loop `// enterprise nodes is strict at any version; non-enterprise nodes binds only versions < 8; geolocation allow/deny by ac<geo>/a!c<geo> prefix match (exact encoding not fully traced)` [flux/ZelBack/src/services/appLifecycle/appSpawner.js:288–300] [inferred]

if *a trips a soft-deferral condition (non-enterprise app on an ArcaneOS node; static-IP mismatch; datacenter mismatch; targeted-`nodes` app not naming this node; a capacity-gap tier where this node is too large for a small app)* **then**
  push a onto `appsToBeCheckedLater` with a condition- and enterprise-specific delay (Table 31); **return** `//` [flux/ZelBack/src/services/appLifecycle/appSpawner.js:520–630]

verify the image against the Docker Hub whitelist and pull it; verify port availability, including a public reachability probe relayed through peers `// portManager.checkInstallingAppPortAvailable`

broadcast `fluxappinstalling` {type, version: 1, name, own IP, `broadcastedAt`} to all peers and store it locally

wait `installCollisionWaitMs` = 90,000 ms `//` [flux/ZelBack/config/default.js:321]`, comment: "give it 1.5m so messages are propagated"`

re-read the global installing + running location lists for a

if *combined count > required instances* **then**
  rank all installing nodes by `broadcastedAt` ascending
  if *this node's rank does not fit within the remaining slots* **then**
    abandon a; **return**
  `//` [flux/ZelBack/src/services/appLifecycle/appSpawner.js:686–703]`: earliest-broadcast wins - the entire conflict-resolution mechanism`

$ok \leftarrow$ `appInstaller.registerAppLocally`(a) `// Docker deployment`

if ***not*** *ok* **then**
  cache a's hash in `spawnErrorsLongerAppCache` (no automatic retry until it expires); **return** `//` [flux/ZelBack/src/services/appLifecycle/appSpawner.js:730–743]

wait 60 s; re-read running locations for a; rank by `runningSince` (nodes with none yet sort first)

if *this node's rank >* ***`minInstances`*** **then**
  uninstall a locally, immediately `// self-correction,` [flux/ZelBack/src/services/appLifecycle/appSpawner.js:745–775]

Table 31: Soft-deferral delays (push back into `appsToBeCheckedLater`, do not abandon).

Condition	Enterprise app	Standard app
Non-enterprise app on an ArcaneOS node	—	120,000 ms [flux/ZelBack/src/services/appLifecycle/appSpawner.js:34]
Targeted `nodes` not naming this node	1,800,000 ms	3,420,000 ms [flux/ZelBack/config/default.js:341]
Static-IP mismatch	1,620,000 ms	3,420,000 ms [flux/ZelBack/config/default.js:342]
Datacenter mismatch	1,620,000 ms	3,420,000 ms [flux/ZelBack/config/default.js:343]
Capacity gap, large/medium/small	1,800,000 / 1,260,000 / 720,000 ms	7,020,000 / 5,220,000 / 3,420,000 ms [flux/ZelBack/config/default.js:345–347]

The datacenter-mismatch row exists in the deferral table but the branch that would trigger it is flagged by the code's own comment as currently unreachable: apps with `datacenter=true` are routed through an ownership filter to enterprise nodes before this deferral chain runs, so `datacenter` is always false by the time this check executes [doc: flux/ZelBack/src/services/appLifecycle/appSpawner.js:568]. A syncthing-specific tie-break additionally picks the earliest broadcaster among nodes sharing a `/24`-like IP-prefix range when several attempt installation concurrently, so replicated storage does not converge two masters on the same subnet [flux/ZelBack/src/services/appLifecycle/appSpawner.js:706–727].

Property 9.2 (Earliest-broadcast-wins is the entire conflict-resolution mechanism)**.** When more nodes attempt installation than an application's specification requests, the network resolves the conflict by ranking the competing `fluxappinstalling` broadcasts by their self-reported `broadcastedAt` timestamp and letting only the earliest ones proceed. There is no leader election, no quorum, and no on-chain arbitration anywhere in this path [flux/ZelBack/src/services/appLifecycle/appSpawner.js:686–703].

Invariant and failure mode. The mechanism's target property is that, given enough peer connectivity for gossip to be reliable (the spawner is only active above an `appSyncPeerThreshold` of 12 live peers and is explicitly paused below an `appSyncDegradedThreshold` of 4, "gossip unreliable" [flux/ZelBack/config/default.js:268–269], §9.6), the number of nodes that both install and keep an application converges to its requested instance count within roughly one 90-second wait plus one 60-second post-install check. The mechanism's honesty assumption is exactly as narrow as that sentence: it depends on every competing node broadcasting an honest `broadcastedAt` and actually backing off when its rank does not fit. Nothing in the code paths read re-verifies `broadcastedAt` against the signed message envelope's own timestamp for this specific message type [inferred]; a node that broadcasts a forged early `broadcastedAt`, or that never re-checks its rank, can over-claim install slots, and a network partition that delays propagation past the fixed 90,000 ms window can produce the same effect without any dishonesty at all. There is no mechanism above this one — no on-chain record, no elected coordinator — that would notice or correct such an outcome directly; the only correction is the pair of independent loops described next.

Remark 9.3 (Convergence by opposing correction, not by agreement). The system does not converge on the target instance count because nodes agree on who should run an application. It converges because two independent, opposing loops keep nudging the count back toward the target from either side. First, immediately after its own install, a node re-checks its rank by `runningSince` (nodes with no `runningSince` yet — i.e. still starting — sort first) and uninstalls itself at once if it now ranks below `minInstances` [flux/ZelBack/src/services/appLifecycle/appSpawner.js:745–775]. Second, independently and periodically, `advancedWorkflows.checkAndRemoveApplicationInstance` scans every locally installed app whose live location count exceeds `instances`, ranks all running instances by `runningSince` descending (newest first) with IP descending as a tiebreak, and removes locally if this node ranks 0 — the newest — waiting `removal.delay` = 300 s before evaluating the next over-provisioned app "so we don't have more removals at the same time" [flux/ZelBack/src/services/appLifecycle/advancedWorkflows.js:2899–2966,2928–2963]. Both loops correct over-installation; neither one, nor anything else in this design, corrects under-installation faster than the next node's own random draw happens to land

on the under-replicated app. Under a network partition that produces two disjoint views of `runningSince` ordering, both loops can independently conclude "I am over-ranked," causing more removals than intended, or conclude the opposite and remove none, until gossip re-converges.

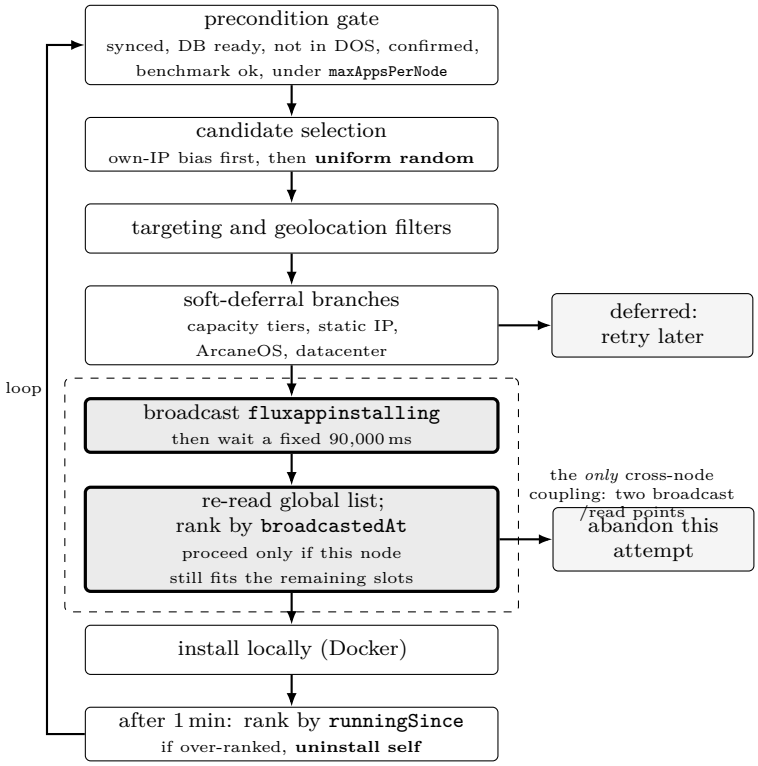


Figure 9: Placement flow for one node's attempt at one application. The loop is private to a single node: every step is a local decision, and the only coupling to other nodes is the broadcast/read pair shown dashed. Conflicts are resolved by earliest-broadcast-wins, not by election or arbitration — there is no scheduler in this picture, and that is the point SHIPPED [flux/ZelBack/src/services/appLifecycle/appSpawner.js:77–787].

9.4 The reconciler: a single, level-based actuator

Once a container exists, everything that starts or stops it goes through one component, documented in its own source as "the single, level-based actuator for app containers" [doc: flux/ZelBack/src/services/appMonitoring/appReconciler.js:20-27]. Every trigger — a Docker `die` event, a stream reconnect, an hourly tick, boot, a post-install callback, or a master/slave (syncthing) decision — only *enqueues* a component identifier; `reconcile()` is the only function that calls `appDockerStart`/`appDockerStop` [flux/ZelBack/src/services/appMonitoring/appReconciler.js:20–27].

Desired state is resolved from four inputs, in strict priority order, highest wins [doc: flux/ZelBack/src/services/appMonitoring/appReconciler.js:29-45]:

1. `operatorStopped` — durable, persisted in `appsRuntimeState`. The source calls this "user lock, wins over all."

2. `controllerDesired` — in-memory only, re-derived every cycle by the master/slave and syncthing deciders. The source states why it is *not* persisted: "a stale election after a reboot must not act."

3. `dataDesired` — an in-memory "clear" flag requesting an app-data wipe before the next run, also not persisted, for the same class of reason: "a stale wipe intent surviving a restart could delete the only good copy."

4. Restart policy and the last exit code, as the residual default.

The syncthing/master-slave decider that feeds `controllerDesired` is a second, large state machine (`syncthingFolderStateMachine.js`, `syncthingMonitor.js`) that this evidence pass did not read in depth; only its output channel into the reconciler is characterized here [inferred].

Concurrency is single-flight per component identifier: an `inFlight` set prevents two reconciles of the same component from running at once, and a `dirty` set forces one more pass if the identifier was re-enqueued while a reconcile was already running [flux/ZelBack/src/services/appMonitoring/appReconciler.js:51–52]. A boot drain gate (an `AsyncGate`) holds all boot-time triggers in `bootPending` until every boot-held component has completed *one* reconcile pass — not until all are running — capped at `BOOT_DRAIN_SETTLE_CAP_MS` = 2 min, so a single wedged reconcile can never indefinitely suppress the node's own network-presence broadcast [flux/ZelBack/src/services/appMonitoring/appReconciler.js:55–64]. A container found detached from its own Docker network — a stale libnetwork endpoint that a plain start cannot repair — is force-removed and recreated rather than merely restarted [doc: flux/ZelBack/src/services/appMonitoring/appReconciler.js:24-26]. Crash recovery follows a fixed backoff ladder, `crashBackoffDelaysMs` = [0, 30,000, 300,000, 900,000, 1,800,000] ms (0 s, 30 s, 5 min, 15 min, 30 min), reset to the first rung after `crashBackoffStableRunMs` = 600,000 ms (10 min) of stable running [flux/ZelBack/config/default.js:130–131].

9.5 Soft versus hard redeploy

Redeploy, removal, and installation are mutually exclusive with each other through four boolean globals — `globalState.removalInProgress`, `installationInProgress`, `softRedeployInProgress`, `hardRedeployInProgress` — checked at entry. If any is already set, the call returns a warning and does nothing; it does not queue itself and does not retry [flux/ZelBack/src/services/appLifecycle/advancedWorkflows.js:1245–1281,1355–1383].

Table 32: Soft versus hard redeploy.

	Soft redeploy	Hard redeploy
Teardown	`softRemoveAppLocally`: stops/removes the container, **keeps** the app volume/data	`appUninstaller.removeAppLocally(force=false)`: full teardown, **drops** volumes
Wait	`config.fluxapps.redeploy.delay` = 30 s	30 s (same constant)
Rebuild	`checkAppRequirements` then `softRegisterAppLocally` (recreate container against existing data)	`checkAppRequirements` then `appInstaller.registerAppLocally(..., sendRemovalMessage=true)` (full fresh install)

[flux/ZelBack/src/services/appLifecycle/advancedWorkflows.js:1313–1330,1394–1420]

Automatic escalation. For SPEC v8-and-later apps, if the installed component structure differs from the target specification — component count or composition changed — `softRedeploy` escalates itself to `hardRedeploy` instead of proceeding, "for component structure safety" [flux/ZelBack/src/services/appLifecycle/advancedWorkflows.js:1281–1305]. If either path fails, the app is force-removed (`removeAppLocally(..., force=true, sendMessage=true)`) as cleanup [flux/ZelBack/src/services/appLifecycle/advancedWorkflows.js:1332–1341,1422–1427]. Both paths stream progress back over the original HTTP response (`res.write`) when one is available: this is a synchronous, long-running HTTP call, not a job queue a client polls.

9.6 The P2P layer

9.6.1 Wire format, dispatch, dedup

Every FluxOS-to-FluxOS message is JSON over WebSocket, wrapped in a signed envelope `{version: 1, timestamp, pubKey, signature (over version+message+timestamp), data}` [flux/ZelBack/src/services/utils/fluxBroadcastHelper.js:11–26]. Observed `data.type` values: `zelappregister/fluxappregister`, `zelappupdate/fluxappupdate`, `fluxapprequest` (a v1 single-hash form and a v2 form batching up to 500 hashes), `fluxapprunning`, `fluxipchanged`, `fluxappremoved`, `fluxappinstalling`, `fluxappinstallingerror`, `fluxnodesigterm`, and four sync-round types (`fluxapptempsync`, `fluxapprunningsync`, `fluxappinstallingsync`, `fluxappinstallingerrorssync`) [flux/ZelBack/src/services/fluxCommunication.js:585–602,644–660] [flux/ZelBack/src/services/fluxCommunicationMessagesSender.js:53–74]. A separate raw, non-JSON channel —

`messageHashPresent` (a 40-char hex hash) and `requestMessageHash` — bypasses the signed envelope entirely, because it only ever carries a hash and never application data [flux/ZelBack/src/services/fluxCommunication.js:511–536].

Inbound dispatch (`dispatchFluxMessage`) runs, per message: raw hash-gossip frames first, handled and returned immediately, no signature check [flux/ZelBack/src/services/fluxCommunication.js:511–536]; a malformed envelope closes the connection [flux/ZelBack/src/services/fluxCommunication.js:538–546]; deduplication after a random 1–75 ms jitter delay against an in-memory `messageCache` keyed on `hash(data)` — a seen hash is silently dropped [flux/ZelBack/src/services/fluxCommunication.js:549–554] (the exact TTL/eviction policy of this cache was not traced to its definition in the evidence available [inferred]); a blocklist check against `wsPeerCache` [flux/ZelBack/src/services/fluxCommunication.js:556–563]; then signature/timestamp verification, yielding `OK`, `NODE_NOT_FOUND` (originator not in the deterministic node list — dropped without penalizing the relaying peer, "the relay peer is not at fault"), or a bad-signature/malformed result, which is tracked per-peer in a rolling 10-minute window and closes the connection at 5 or more bad messages in that window (the timestamp array is capped at 20 entries, truncated to the last 10, bounding memory from an adversarial peer) [flux/ZelBack/src/services/fluxCommunication.js:592–618]. A valid, fresh message is dispatched by type via `setImmediate` — deferred one tick, not otherwise queued or rate-limited per type [flux/ZelBack/src/services/fluxCommunication.js:604–618]; a stale but otherwise valid message gets a NAK rather than a silent drop [flux/ZelBack/src/services/fluxCommunication.js:601]. Four message types (`fluxapprunning`, `fluxipchanged`, `fluxappremoved`, `fluxnodesigterm`) use a 240,000 ms (4 min) freshness window [flux/ZelBack/src/services/fluxCommunication.js:317,405,437,465].

9.6.2 Rebroadcast and convergence

Rebroadcast follows one of two mutually exclusive strategies, selected by comparing daemon height to `messagesBroadcastRefactorStart` = 1,751,250 ("expected block at 13th October 2024" per the code's own comment [doc: flux/ZelBack/config/default.js:124]): below it, `fluxCommunicationMessagesSender.relay(...)` flood-fills the full signed message to every other peer; at or above it, only the 40-character hash is gossiped (`peerManager.broadcastHash`), and a receiver that lacks the body pulls it on demand via `requestMessageHash` [flux/ZelBack/src/services/fluxCommunication.js:407–412,431–436,486–491]. This is a live protocol transition keyed on chain height, not a node-local configuration flag. A `fluxnodesigterm` broadcast extends the affected apps' `expireAt` in `zelappslocation` to `broadcastedAt` + `SIGTERM_EXPIRY_MS` (420,000 ms, 7 min) rather than letting the location expire immediately [flux/ZelBack/src/services/fluxCommunication.js:472–478] [flux/ZelBack/src/services/utils/appConstants.js:86]. Beyond continuous gossip, a periodic hash-sync protocol runs with its own bounded parameters — up to `hashSyncMaxRounds` = 4 rounds, `hashSyncPeersPerRound` = 3, `hashSyncEphemeralPeers` = 5, `hashSyncMaxRetries` = 3 at `hashSyncRetryMs` = 300,000 ms, with a `hashSyncSettleMs` = 4,000 ms settle window [flux/ZelBack/config/default.js:355–363]; the exact message sequence of one full round (who is asked, in what order, what happens on a partial response) was not traced beyond these constants [inferred].

This is gossip convergence, not consensus. Global application state — which node is running, installing, or has removed which application — lives in `globalzelapps`, is propagated exclusively by the dedup/rebroadcast/hash-sync mechanism above, TTL-expires by the constants in Table 30, and is not committed to any block, checked against any quorum, or given any finality. Two nodes with a partitioned or lagging gossip view can disagree about the current state of an application for as long as the partition lasts; nothing above the mesh corrects that disagreement faster than the mesh itself re-converges.

9.6.3 Peer topology and liveness

`FluxPeerManager` is a single in-memory registry of inbound and outbound sockets, ephemeral sync peers, a reconnect queue tracking per-peer attempts, an unstable-node tracker, IP-group/unique-IP counters, and a bounded circular history buffer of peer lifecycle events [flux/ZelBack/src/services/utils/FluxPeerManager.js:31–77]. Inbound admission is capped at the larger of `maxPeers` = 4×`minIncoming` and `maxNumberOfConnections`, which is normally 9×`minIncoming` but scales down when `numberOfFluxNodes`/160 falls below that figure, so small networks get a proportionally smaller inbound cap rather than a flat one, floored at `maxPeers` [flux/ZelBack/src/services/utils/FluxPeerManager.js:839–847]. Reconnect backoff follows [120,000, 300,000, 600,000, 900,000] ms (2/5/10/15 min) [flux/ZelBack/src/services/utils/FluxPeerManager.js:32]. Liveness is a WebSocket ping every `wsPingIntervalMs` = 15,000 ms; a peer that misses 3 consecutive pongs (`wsMaxMissedPongs`) is force-terminated [flux/ZelBack/config/default.js:18–19] [flux/ZelBack/src/services/utils/FluxPeerSocket.js:86–145,209–212,298]. Placement itself is gated on this liveness signal: the spawner runs only above an `appSyncPeerThreshold` of 12 live peers and pauses below an `appSyncDegradedThreshold` of 4 [flux/ZelBack/config/default.js:268–269] [flux/ZelBack/src/services/appLifecycle/appSpawner.js:38–52].

9.7 Resource accounting

Table 33: Per-tier resource allowances (total / available to applications).

Tier	CPU (units of 0.1 core)	RAM (MB)	HDD (GB)	Collateral, current / legacy (FLUX)
Cumulus (C)	40 / 30	7,000 / 5,000	220 / 180	1,000 / 10,000
Nimbus (N)	80 / 70	30,000 / 28,000	440 / 400	12,500 / 25,000
Stratus (S)	160 / 150	61,000 / 59,000	880 / 840	40,000 / 100,000

[flux/ZelBack/config/default.js:380–403]

Resources locked by the host system, subtracted before any application allocation: `cpu`: 10 (one core, in units of 0.1 core), `ram`: 2,000 MB, `hdd`: 60 GB, `extrahdd`: 20 GB [flux/ZelBack/config/default.js:375–378], with a further 0.95 usable-disk multiplier [flux/ZelBack/src/services/appRequirements/hwRequirements.js:385]. `hwRequirements.checkAppHWRequirements` enforces, per component:

$$\text{usable HDD} = 0.95 \cdot \text{total HDD} - 60 - 20,$$
$$\text{reject if app HDD} > \text{usable HDD} - \text{HDD locked by other apps};$$

$$\text{usable CPU } (\times 10) = 10 \cdot \text{cores} - 10,$$
$$\text{reject if } 10 \cdot \text{app CPU} > \text{usable CPU} - 10 \cdot \text{CPU locked by other apps};$$

$$\text{usable RAM} = \text{total RAM} - 2{,}000,$$
$$\text{reject if app RAM} > \text{usable RAM} - \text{RAM locked by other apps}$$

[flux/ZelBack/src/services/appRequirements/hwRequirements.js:370–408]. A separate, enterprise-only check additionally rejects an install that would leave 4 or fewer free vCores after it (`freeCoresAfterInstall` = cores − 1 system-reserved core − locked cores − app cores), reserved so the CFS CPU-burst feature always has spare capacity to draw on [flux/ZelBack/src/services/appRequirements/hwRequirements.js:411–437]. That burst feature is enabled by default (`cpuBurst.enabled: true`, CFS period `periodUs` = 100,000 (100 ms), 1 reserved core) [flux/ZelBack/config/default.js:440–453], with the per-host caveat, stated in the source itself, that "multiple burstable apps on the same host can collectively oversubscribe during simultaneous spikes" [doc: flux/ZelBack/config/default.js:441-449].

9.8 Pricing

Price is computed from a per-component resource vector $\mathbf{r}$, the requested instance count k, and the requested duration d (in blocks, against L), evaluated at the transaction's block height h against a height-scheduled price table (`config.fluxapps.price[]`, overridable by `chainmessages` of type `p` posted on-chain by the multisig whose keys the four co-founders hold personally, K8 in Definition 2.3) [flux/ZelBack/src/services/utils/chainUtilities.js:9–52]. Concretely, for a SPEC v4-or-later (composed) app, `cpuPrice` = $\sum$(CPU units) $\times$ table.cpu $\times$ 10; `ramPrice` = $\sum$(RAM units) $\times$ table.ram/100; `hddPrice` = $\sum$(HDD units) $\times$ table.hdd; plus `table.scope` if the app declares `nodes` or is `enterprise`, plus `table.staticip` if it declares `staticip`, plus `table.port` per enterprise port; that sum is quoted per three instances — divided by three, ceiled to two decimals, and, from height 1,890,000, scaled by the requested instance count k, with a floor of 0.50 per additional instance for the cheapest specifications [flux/ZelBack/src/services/utils/appUtilities.js:89–134]. The registration price is then scaled by d/defaultExpire, where `defaultExpire` is L or $4 \times L$ post-PoN fork, re-ceiled, and only then floored at `minPrice` [flux/ZelBack/src/services/appMessaging/messageVerifier.js:733–743]; an update is priced at 90% of the fresh price minus a further discount proportional to the unused life of the prior spec, floored again at `minPrice` [flux/ZelBack/src/services/appMessaging/messageVerifier.js:801–816]. §16 gives the full pricing mechanics and treats the credits/renewal layer built on top of them; here it is enough to record the one finding that changes what a payer can rely on: **an underpaid registration or update is silently dropped, with only a server-side log warning and no signal of any kind returned to the payer** (§9.2; [flux/ZelBack/src/services/appMessaging/messageVerifier.js:735–763,797–828]). §16 takes up what a correct failure signal would require.

9.9 Moderation, as it exists today

Docker image compliance is enforced against a GitHub-hosted list, not an on-chain or network-quorum one: `getBlockedRepositores()` fetches `helpers/blockedrepositories.json` from `raw.githubusercontent.com/RunOnFlux/flux`, cached in `fluxCaching.blockedRepositoriesCache` [flux/ZelBack/src/services/appSecurity/imageManager.js:179–195]; a parallel `vettedrepositories.json` lets listed apps bypass user-defined blocked ports and repositories [flux/ZelBack/src/services/appSecurity/imageManager.js:202–236,395–410]. `checkApplicationsCompliance` runs every `imageComplianceIntervalMs` = 3,600,000 ms (1 h), removing any locally installed app that matches the blocklist and waiting 3 min between removals "so we don't have more removals at the same time" [flux/ZelBack/src/services/serviceManager.js:529–531] [flux/ZelBack/config/default.js:319] [flux/ZelBack/src/services/appSecurity/imageManager.js:644–676]. The equivalent GitHub-sourced enterprise node/owner map resyncs every 6 h with fetch failure explicitly swallowed by a `.catch` handler, proceeding on the last-cached value [flux/ZelBack/src/services/serviceManager.js:130–136]; the evidence gathered for this paper did not separately trace a distinct failure-handling code path for the blocklist fetch itself beyond its cache, but the two fetches share the same GitHub-hosted, silently-cached design. §23 analyzes what this means for censorship-resistance and for the roadmap's stated network-quorum vision; this section states the mechanism and defers the analysis.

9.10 Trust and failure model

1. **Placement convergence assumes peer honesty.** Instance-count convergence depends entirely on nodes honestly reporting and honoring `broadcastedAt` rank; nothing in the paths read re-verifies it against the signed envelope's own timestamp for this message type [inferred](§9.3).

2. **Underpayment is swallowed, not rejected.** A registration or update that pays too little produces only a log warning; the payer receives no on-chain refund and no distinguishable failure signal [flux/ZelBack/src/services/appMessaging/messageVerifier.js:735–763,797–828].

3. **GitHub is the moderation trust root.** The image blocklist, vetted-repository list, and enterprise owner map are all fetched from one GitHub repository on a schedule, with fetch failure silently swallowed and the last-good value kept [flux/ZelBack/src/services/serviceManager.js:130–136]; a GitHub outage or a compromise of that specific file is a single point of moderation trust for the whole network, not a network-quorum mechanism (§9.9).

4. **Boot is a flat-backoff infinite retry, not a circuit breaker.** Any uncaught exception in the boot function restarts it after a fixed 15,000 ms with no retry cap and no escalation, so a persistently failing dependency retries forever rather than surfacing a fatal state [flux/ZelBack/src/services/serviceManager.js:568–572].

5. **Distributed enforcement trusts a synchronized local view.** The node-status monitor's network-wide eviction pass — which every confirmed node runs independently against every other node's claimed locations — trusts its own local copy of the deterministic node list; if that local view lags, it can evict still-valid locations or fail to evict dead ones [inferred].

6. **Self-correction can amplify churn under partition.** Because every node independently re-derives whether an application is over-replicated and removes itself on that basis, a partition that produces disjoint `runningSince` orderings can cause more nodes to remove an application than intended, or none, until gossip re-converges; there is no distributed lock (§9.3, remark above).

7. **`admin` is a local file, not a bound credential.** `admin` privilege is granted to whoever controls `userconfig.initial.zelid` on that specific node's disk; there is no additional hardware or HSM binding checked at this privilege level [flux/ZelBack/src/services/verificationHelperUtils.js:19–46].

10 FluxOS application specification v9

Every application that runs on Flux is described by a small, owner-signed JSON document: a container image, how much CPU, memory and disk it needs, how many copies to run, and for how long. Because FluxOS has no central scheduler and instead relies on every node independently deciding what to run (§9), that document is closer to a consensus artifact than to an ordinary application-programming detail — every node on the network must parse and validate it the same way, which is why changing its shape is slow and height-gated, exactly like a price table. This section describes the specification version currently enforced on the network, and a substantial, actively-developed redesign that is not enforced anywhere yet. This paper calls the specification version *v9* throughout, written in full here and as SPEC v9 from this point on; it is unrelated to `fluxd` client version 9.1.0, the version number of the consensus daemon `fluxd`, which is maintained independently and is never the referent of "v9" in this document [intent: design intake]. The reason this section exists is structural, not archival: three fields SPEC v9 adds — `paymentMemo`, `referralCode` and `machineType` — are the substrate that PNR (§7), the referral program, and the VPS/datacenter tier (§11) are each stated to depend on, and as of this writing none of the three has any consumer code anywhere in FluxOS.

10.1 Version gating as deployed

FluxOS accepts an incoming specification s at chain height $h$only if its declared version does not exceed the maximum version enforced at that height, its top-level and per-component fields are drawn from that version's whitelist, and it otherwise satisfies that version's own validity rules:

$$\mathrm{Accept}(s,h) \;\equiv\; v(s) \le V(h) \;\wedge\; \mathrm{Fields}(s) \subseteq \mathcal{F}_{v(s)} \;\wedge\; \mathrm{Valid}_{v(s)}(s),$$

with $V(h)$ the highest specification version whose enforcement height has been reached. This is the same height-scheduling mechanism the network uses for price-table changes, applied to the specification schema itself: `verifyAppSpecifications` runs type correctness, then per-version restriction checks, then the enforcement-height gate, then object-key correctness against a per-version field whitelist, in that fixed order, before anything is broadcast SHIPPED [flux/ZelBack/src/services/appRequirements/appValidator.js:1243-1400].

The enforcement-height table, as configured today, is:

Version	Enforcement height	Notes
1	0	no longer accepted for registration or update via the API
2	0	
3	983,000	
4	1,004,000	composition (`compose` array)
5	1,142,000	`contacts`, `geolocation`
6	1,300,000	`expire`, price change
7	1,420,000 (mainnet) / 1,390,000 (dev)	`nodes` targeting, `secrets`, `staticip`
8	**1,932,380** (mainnet) / **1,921,500** (dev)	enterprise / ArcaneOS apps

Table 34: Specification-version enforcement heights as shipped in FluxOS. There is no row for version 9: no enforcement height for it exists in the shipped configuration or the live pricing endpoint (below).

SHIPPED [flux/ZelBack/config/default.js:231-240], cross-checked against the live public pricing endpoint, which returns the identical mainnet heights through version 8 and carries no key for version 9 at all [measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]. SPEC v8's enforcement height is **1,932,380** on mainnet and **1,921,500** in the development network SHIPPED [flux/ZelBack/config/default.js:231-240]. Above that height, a specification whose top-level keys are anything other than `version, name, description, owner, compose, instances, contacts, geolocation, expire, nodes, staticip, datacenter, enterprise` is rejected outright SHIPPED [flux/ZelBack/src/services/appRequirements/appValidator.js:1063-1067].

IN-PROGRESS The organisation's own working checkout additionally carries a single, unpublished commit that begins to vendor a copy of the RFC-0 schema (§10.7) behind a feature flag rather than a height: registering or updating a `version: 9` specification is rejected before schema validation ever runs unless `config.fluxapps.acceptV9` is `true`, and that flag defaults to `false` with no caller anywhere in the codebase that sets it [RunOnFlux/flux@feat/appspec-v9:ZelBack/src/services/appDatabase/registryManager.js:1766-1769].[1] No node on the network today enforces any rule for `version: 9` beyond "reject it."

10.2 The migration baseline, measured

The concrete cost of this gating scheme is visible in what is actually deployed. Of 1,195 applications registered on the network on 2026-09-03 [measured: api.runonflux.io/apps/globalappsspecifications, 2026-09-03]:

[1] This sits on a working checkout of the organisation repository that had not been pushed to `RunOnFlux/flux`'s public remote at the time of writing, and it is a much smaller and separate effort from the contributor fork discussed in §10.5. It is cited here as an unpublished working branch, not as a fetchable public artifact.

Version	Applications	Share
v8	873	73.1%
v7	209	17.5%
v6	70	5.9%
v3	27	2.3%
v4	11	0.9%
v5	3	0.3%
v2	2	0.2%
Total	1,195	100%

Table 35: Deployed-application count by specification version, ordered by share.

SHIPPED **322 applications (26.9%) are on a version older than spec v8.** Seven specification versions — SPEC v2 through SPEC v8 — are live on the network simultaneously [measured: api.runonflux.io/apps/globalappsspecifications, 2026-09-03]. The acceptance predicate above has a ceiling ($v(s) \leq V(h)$) but no floor: nothing in it forces an already-running application to move forward when a newer version becomes enforceable, and the measured coexistence of seven versions is the direct, observable consequence. The network has not deprecated any specification version to date; every version ever shipped is still validated, by every node, for as long as an application on that version continues to exist. That is the real cost of height-scheduled versioning with no deprecation mechanism, and it is the baseline against which SPEC v9's eventual migration has to be measured (§10.9).

10.3 What v9 changes

IN-PROGRESS SPEC v9 is defined, at present, only as a draft JSON Schema (RFC-0) in a standalone repository [flux-spec/schema/v9.json:1-350] — see §10.7 for why that draft and what FluxOS actually consumes are not the same object. Table 36 summarises the delta against SPEC v8: four field renames, one removal, and five additions, each against what depends on it and how far its implementation actually reaches.

Change	SPEC v9 name (SPEC v8 name)	Depends on	Implementation status
Rename	`components` (`compose`)	—	schema-level only; breaks every existing client, hence bundled with the other renames rather than dripped
Rename	`image` (`repotag`)	—	schema-level only
Rename	`memory` (`ram`)	—	schema-level only
Rename	`rootFsGb` (`hdd`)	—	schema-level only
Removal	— (`enterprise`; its envelope replacement named but not specified)	private/encrypted specifications	regression; see §10.8
Addition	`loadBalancing`	routing, health checks, backend TLS, connection draining	partially implemented: draining and per-app-CA TLS wired; config generation stays in FDM (§12)
Addition	`duration`	subscriptions and auto-renewal (§16)	first-class in the schema; the fork's pricing engine reads the same concept off `spec.ttl`, and whether `duration` maps to it is unresolved (§B)
Addition	`paymentMemo`	PNR (§7)	**zero consumers in FluxOS**
Addition	`referralCode`	referrals, preview environments	**zero consumers**
Addition	`machineType`	VPS/datacenter tier (§11)	**zero consumers**

Table 36: SPEC v8 →SPEC v9 field changes, their dependents, and implementation status.

Field shapes and required-field membership per [flux-spec/schema/v9.json:40-46] (`components`),

[flux-spec/schema/v9.json:47-61] (`loadBalancing`), [flux-spec/schema/v9.json:62-82] (`duration`, `paymentMemo`, `referralCode`, `machineType`), and [flux-spec/schema/v9.json:103-122] (`image`, `memory`, `rootFsGb`); the removal of `enterprise`, with its envelope replacement deferred to a later revision, is stated in the draft's own README [doc: flux-spec/README.md:58-67,95-96]; the `paymentMemo`/`referralCode` dependency mapping is drawn from the fork's own code comments ("basis for paying operators under PNR v2", "rewards paid as hosting credits") [RunOnFlux/flux@feat/appspec-v9:ZelBack/src/services/appRequirements/appSpecV9.js].

10.4 The finding this section exists to deliver

IN-PROGRESS Stated plainly and without spin: `paymentMemo`, `referralCode` and `machineType` have **zero consumer code anywhere in FluxOS** — no billing path, no referral-reward path, no capacity-tier scheduling path reads any of the three, confirmed by grepping every JavaScript file under `ZelBack/` and `tests/` for all three field names [MorningLightMountain713/flux@feat/fluxos-v9:(grep -rn "paymentMemo|referralCode|machineType" –include=*.js ZelBack tests)]. Those are exactly the fields PNR (§7), the referral program, and the VPS tier (§11) are stated to depend on. The orchestration substrate of SPEC v9 — placement, mesh identity, routing plumbing, persistent storage, pricing — is under active development with an extensive unit-test suite, on a contributor fork that has not been merged or shipped (§10.5); the economic fields the roadmap's dependency chain actually rests on are declared in a schema and nothing more. The dependency chain the roadmap draws between SPEC v9 and PNR, referrals and the VPS tier is real at the level of the schema; it currently terminates in fields no code reads. §25 returns to this as one instance of a broader pattern in how far the remaining work actually reaches.

10.5 The scale of the work, quantified

The overwhelming majority of SPEC v9 engineering effort does not live in the organisation repository. It lives on an individual contributor's fork[2] that diffs 932 files against `upstream/master`, with +180,317/ − 49,561 lines [MorningLightMountain713/flux@feat/fluxos-v9:(git diff –stat upstream/master origin/feat/fluxos-v9)], decomposing the old monolithic application-service layout into 23 focused service packages, ten of them absent from `upstream/master` (`appMesh`, `appPlacement`, `pricing`, `quorumGrant`, and others) [MorningLightMountain713/flux@feat/fluxos-v9:(find ZelBack/src -maxdepth 3 -type d)]. This is IN-PROGRESS work, and it is substantial: 289 unit-test files containing 7,763 test cases run on every push [MorningLightMountain713/flux@feat/fluxos-v9:(find tests/unit -name *.test.js | wc -l = 289; grep count of it/test call sites in tests/unit = 7763)] [MorningLightMountain713/flux@feat/fluxos-v9:.github/workflows/nodejs.yml:1-3,90-93], alongside a from-scratch, 121-scenario integration harness that provisions up to 16 simulated nodes with per-daemon stubs [MorningLightMountain713/flux@feat/fluxos-v9:(ls test-infra/runner/tests); (find test-infra/config -maxdepth 1 -type d)] — but that harness runs only on manual dispatch, on a self-hosted runner, and is not part of the automatic merge gate [MorningLightMountain713/flux@feat/fluxos-v9:.github/workflows/integration-harness.yml:1-38].

This is an ordinary open-source pattern, not a criticism, but it is a fact about where development authority currently sits for the largest in-flight change to how applications are described on Flux: on a contributor's fork rather than in the organisation's own repository. §23 returns to this as a governance observation.

10.6 The routing layer, precisely

[roadmap: flux-roadmap-public.md:52-53] The public roadmap states that SPEC v9's routing layer has been "rewritten." Read precisely against the fork, that claim is partially, not wholly, supported. IN-PROGRESS Two mechanisms declared through the `loadBalancing` block are genuinely wired: connection draining, served over a UNIX-socket JSON-RPC endpoint that `flux-shutdownd` calls

[2] `MorningLightMountain713/flux`, branch `feat/fluxos-v9`, last pushed 2026-09-02 — an external fork of `RunOnFlux/flux`, not a branch of the organisation repository itself. Citations against it use [repo@branch:file:line] notation for that reason, and none of them can be fetched from the organisation's own remote.

with `drain_app/stop_app` to mark an application draining ahead of node shutdown [MorningLightMountain713/flux@feat/fluxos-v9:ZelBack/src/services/appMessaging/drainServer.js:12-24,85-136,176-201]; and per-application-CA backend TLS, under which a node generates a local Ed25519 keypair, obtains a 30-day host certificate signed by the application's own certificate authority through `fluxbench`, and renews it automatically before expiry [MorningLightMountain713/flux@feat/fluxos-v9:ZelBack/src/services/appLifecycle/backendTlsService.js:1-18,73-116]. What is *not* true is that FluxOS now owns HAProxy configuration generation: grepping every service file for the routing-layer vocabulary (`loadBalancing`, `healthCheck`, `stickySession`, `connectionDrain`, `customDomain`) turns up three files, none of them a route-table or config-generation engine [MorningLightMountain713/flux@feat/fluxos-v9:(grep -rln "loadBalancing|healthCheck|stickySession|connectionDrain|customDomain" ZelBack/src/services)]. That responsibility stays where it already sits, in FDM (§12). And the actual signal a routing layer would consume when an application starts draining is not a new push protocol; marking a component draining triggers an immediate presence rebroadcast, so the mechanism by which FDM would pull a backend out of rotation is the same gossip/presence-announcement path FluxOS already used before SPEC v9 [MorningLightMountain713/flux@feat/fluxos-v9:ZelBack/src/services/appMessaging/drainServer.js:79-91]. So: the spec-side declaration and two real backend mechanisms are new and wired; the config-generation engine the roadmap's phrase might suggest is not, and never was scoped to live here.

10.7 Two artifacts, not one

"SPEC v9" currently names two different objects, and this section describes them separately rather than presenting one as a stand-in for the other.

The first is `RunOnFlux/flux-spec`, a standalone repository holding the RFC-0 draft schema: two commits, both dated 2026-09-01, self-described as "Status: RFC-0 — draft for comment (2026-09-01). Not live on the network" [doc: flux-spec/README.md:3], unpublished (`"private": true`, no `bin`/`main`/`exports`), with no CI and no tags. The second is the vendored copy the contributor fork actually consumes at runtime, which is pulled not from this repository but from a different, private, monorepo-structured `flux-spec` pinned at a specific commit hash [MorningLightMountain713/flux@feat/fluxos-v9:test-infra/flux-spec/pin:1]; the two are structurally different (the private monorepo has `packages/spec`, `spec-backend`, `spec-policy` subpackages that do not exist in the public draft) and, on the evidence gathered, disagree on at least one field name — the draft's `duration` [flux-spec/schema/v9.json:15,62] versus the fork's internal `spec.ttl` [MorningLightMountain713/flux@feat/fluxos-v9:ZelBack/src/services/pricing/v9PricingRegime.js:95,108,122,128,183,228,279,308] — which this paper flags rather than resolves, since the vendored validator classes themselves are not available to inspect.

IN-PROGRESS The public draft carries two specific defects worth naming, because a specification whose entire purpose is that every node agrees on it should not have them. First, it has **no signature field at all**: nothing in `schema/v9.json` names a field a submitter signs, and the only "signature"/"signed" references anywhere in the repository describe an unrelated `fluxbench` attestation workstream [flux-spec/schema/v9.json:1-350]. A "recommended" canonicalize-then- double-SHA256 fingerprint recipe exists for comparison purposes, but the draft offers it only as a fingerprint and nowhere defines what gets signed for submission authenticity [doc: flux-spec/validation/canonicalization.md:53-63]. Second, the draft's own canonicalization rules — array-sort ordering for fields such as `customDomains` and `certificateAuthorities`, and explicit materialization of defaulted fields — are stated as `MUST` requirements in prose, with **no schema or code enforcement** anywhere in the repository [doc: flux-spec/validation/canonicalization.md:31-45]. JSON Schema has no array-sort keyword, so nothing here can express these rules as validation; a document can be schema-valid and canonically invalid at the same time, and nothing in the draft detects that. For a document meant to be compared byte-for-byte across roughly 6,500 independent validators, that is the defect to name.

10.8 The encryption-envelope regression

PLANNED The public roadmap states that SPEC v9 will deliver "a cleaner encryption envelope for private specs" [roadmap: flux-roadmap-public.md:61]. IN-PROGRESS The RFC-0 draft does the opposite of deliver it: it removes SPEC v8's `enterprise` field outright, names the encryption envelope as its replacement **without specifying it**, and lists that envelope for private deployments as an explicitly open question deferred to a future schema revision [doc: flux-spec/README.md:58-67,95-96]. Concretely, SPEC v9 as drafted has no way to express a private or enterprise deployment at all. This is not a marginal population: 223 of the 1,195 deployed applications measured in §10.2 (18.7%) use `enterprise` today [measured: api.runonflux.io/apps/globalappsspecifications, 2026-09-03]. If the published draft is what ships, the roadmap's promise is currently unmet in it, and this gap has to be closed before `enterprise` can be removed on any node (§10.9).

10.9 The migration mechanism: forced, not opt-in

IN-PROGRESS No mechanism for migrating deployed applications from SPEC v8 (or older) to SPEC v9 exists in code today: the fork's own `migrations/` service registers exactly two node-runtime-state migrations (node identity, container labels), neither of which touches application specifications [MorningLightMountain713/flux@feat/fluxos-v9:ZelBack/src/services/migrations/index.js:39-51].

PLANNED What is decided supersedes an earlier draft of this subsection, which proposed a non-destructive version transition of this paper's own design: a version floor $v_{\min}$ with a deprecation schedule, below which existing deployments would be refused renewal rather than evicted. The team's design is a different one: migration to SPEC v9 will be **forced, not opt-in**. Every application will be migrated by an application-update message carrying a special signature that compels the SPEC v9 version; updates and upgrades on older specification versions will then be disabled, and once the entire fleet is on SPEC v9 the legacy code paths will be removed [intent: design intake]. This paper states the change rather than silently substituting it: the forced design is operationally cleaner than the deprecation window it replaces, because it would remove the 322-application, 26.9% legacy-version tail (§10.2) in a single step, at the moment the forcing message is broadcast, rather than letting it trail off across a schedule at each owner's own pace.

The invariant this section stated for the superseded design was that no deployed application is evicted by a version change — it lapses under the rules it was created under. Forced migration is intended to preserve non-eviction by a different route, and the difference is worth stating precisely, as a design target rather than a shipped guarantee:

Property 10.1. No application is evicted by the migration to SPEC v9: instead of lapsing under the rules it was created under, every application is rewritten forward onto the new version.

PLANNED What this would trade against the superseded invariant is consent: nothing in the design gives an owner a way to remain on their signed specification, or to decline the rewrite [intent: design intake]. Non-eviction would be preserved; opt-out would not exist.

A key able to rewrite any tenant's application specification without that tenant's own signature would be a privileged capability, and [inferred] this paper places it, on that basis, in the same class it already inventories elsewhere: the unsigned network policy (§12), the OS release channel (§14), and the 2-of-4 emergency block keys (§4). Naming it is the point; a capability of this shape left unnamed is a finding, not a footnote. PLANNED Per the team's decision, three properties are intended to bound it: it would be **single-use**, scoped to the migration rather than a standing signing service; **migration-scoped**, existing only to perform this one version transition and nothing else; and **hardcoded**, expected to be baked-in values rather than a live signer that could be invoked again later [intent: design intake]. Those three properties are what would make the capability acceptable rather than alarming.

PLANNED The scope and the enforcement class are now decided rather than open. The scope is the **version field and the required-field additions only** — never the container image and never the application's functionality — and the stated purpose is strictly to bring the

network into alignment on one schema version, nothing more [intent: design intake]. Enforcement is by **FluxOS**, not by `fluxd`: every node validates the forced-update message against that scope, which is the same enforcement class as specification validation generally — the `verify AppSpecifications` check described above — checked by every node, but as a FluxOS policy rule rather than a consensus rule. That distinction bounds the capability's blast radius precisely: a future FluxOS release could in principle change what the forcing message is permitted to touch, in exactly the sense that §23's deployed policy channel is FluxOS-enforced and not consensus state, and the two capabilities belong to the same class for that reason. It also means the scope is a stated design constraint rather than a proof: nothing in `fluxd` itself rejects a forced-update message that exceeded it, so the guarantee rests on the migration tooling matching its own specification rather than on a rule `fluxd` would enforce independently. Explicit retirement of the key once the migration completes remains a step the team has not stated as a commitment, and leaving hardcoded values live in a binary indefinitely after that point would be a departure from the single-use property above.

PLANNED Under this design the four field renames (`compose`→`components`, `repotag`→`image`, `ram`→`memory`, `hdd`→`rootFsGb`) would need no separate compatibility shim: the same forced-update message that compels the version bump would rewrite the renamed fields as part of the same rewrite, so the translation question this section previously posed as open is resolved by the mechanism itself rather than by added code.

10.10 The v9 schema, for reference

IN-PROGRESS Listing 1 reconstructs the root object of the draft schema from the field table gathered during evidence collection; it is not a verbatim excerpt of the source file; the citations that follow it, not the listing itself, are the evidence that these fields exist.

```
{
  "type": "object",
  "additionalProperties": false,
  "required": ["version","name","description","owner",
               "components","loadBalancing","duration",
               "paymentMemo","referralCode","machineType"],
  "properties": {
    "version": { "const": 9 },
    "duration": { "type": "integer", "minimum": 1 },
    "paymentMemo": { "type": "string", "minLength": 1, "maxLength": 512 },
    "referralCode": { "type": "string", "minLength": 1, "maxLength": 64 },
    "machineType": { "enum": ["standard", "gpu", "vps"] }
  }
}
```

Listing 1: Root object of the SPEC v9 RFC-0 draft schema (reconstructed). Look at `additionalProperties:false` together with the five new required fields: this combination is what makes an unrecognised key — including v8's removed `enterprise` — a hard validation failure rather than a silent drop, and what makes `paymentMemo`/`referralCode`/`machineType` required inputs to a document that §10.4 shows no FluxOS code yet reads.

Root object, `additionalProperties:false`, and the ten required top-level fields: [flux-spec/schema/v9.json:7-19]. `version`: [flux-spec/schema/v9.json:21-24]. `duration`: [flux-spec/schema/v9.json:62-66]. `paymentMemo`: [flux-spec/schema/v9.json:67-72]. `referralCode`: [flux-spec/schema/v9.json:73-78]. `machineType`: [flux-spec/schema/v9.json:79-82].

10.11 Open parameters

Remark 10.2 (SPEC v9 timing: a planned quarter and a proposed height, neither yet reached). SPEC v9 is in development and is to be **activated in Q4 2026, with the migration of existing applications in the same quarter** [intent: design intake] (PLANNED). That places $h^\star$

before $h_{\mathrm{PA}} = 3{,}787{,}502$ (2027-07-01), so the specification change lands ahead of PNR activation and the parallel-asset cut rather than with them — which is the right order, because §25 places PNR, credits, referrals, the VPS tier and the provisioning API all downstream of the SPEC v9 economic fields.

Round 11 fixed no height, and the constraint on it is unchanged: $h^{\star}$ must *follow* the implementation of the `paymentMemo`/`referralCode`/`machineType` trio (§10.4), never precede it, or the height would activate a schema whose economic fields are declared but inert. At the 30 s target a Q4 2026 activation corresponds to roughly 3,000,000–3,260,000, and the paper states that range as an implication of the stated quarter rather than as a number the team has given. A target height of $h^{\star} = 3{,}050{,}000$ (approximately 2026-10-20 at the 30 s target) was stated in design intake [intent: design intake], but it was never configured in FluxOS: SPEC v9 is still in development, has not shipped and has not activated, and no enforcement height for it exists in the shipped configuration. The constraint of Definition 10.2 still binds: the `paymentMemo`/`referralCode`/`machineType` implementation must be complete before any enforcement height is set, or that height would activate a schema whose economic fields are inert.

Decided. `quorumGrantActivationHeight` activates *after* SPEC v9, once SPEC v9 is stable, at a height not fixed now [intent: design intake]. This is a deliberate decoupling rather than an unfinished decision: the mastership/active-standby quorum plane and the SPEC v9 schema change have unrelated failure modes, so binding them to one height would mean a fault in either forces a rollback of both. The cost is a second upgrade event and a second announcement. The height is left unset because the trigger is an observed condition — SPEC v9 running stably — and a height chosen today could only be a guess at when that condition will hold. Until it activates, the field remains what §27.3 records it as: read by production code, set only in integration-test fixtures, and inert on mainnet.

11 One platform: FluxCloud/FluxEdge convergence, one provider runtime, and the datacenter tier

The roadmap names Q4 2026 the quarter in which "the platform becomes one product" [roadmap: flux-roadmap-public.md:41]. Read literally, that is a claim about two products merging their sign-up flows. Read structurally, it is a claim about two runtimes — one that places Docker containers by gossip and self-selection, one that rents Kubernetes-scheduled compute by the hour (per-second billing is planned, §18) — being asked, eventually, to speak one protocol to one fleet. This section takes the second reading and argues it on its own terms. Today there are two separate products, two orchestration models and two separate sets of machines: FluxCloud applications run on collateralised FluxNode s orchestrated peer-to-peer by FluxOS, while FluxEdge is a centralized service that rents machines which lock no FLUX collateral, are not FluxNode s and are not part of the FluxNode network. The two share no machines, runtime or orchestration. None of the convergence has shipped (PLANNED): the near-term plan is one FluxCloud platform with one interface offering both decentralized FluxCloud compute and FluxEdge GPU compute, while the two systems behind it stay separate, with GPU rentals still served by FluxEdge's centralized backend, and that interface is in development; merging the two systems themselves comes after that, and how is not yet worked out; the long-term plan is one decentralized network running all kinds of applications. The interesting question is not whether the two products get one web front end; it is whether a single runtime can advertise what a node can do and let placement decide what runs where. The paper's claim is that convergence, properly stated, is an architectural proposition about a capability-typed provider runtime, not a UI decision — and that the concrete evidence for it is thin enough that the honest thing to do is describe exactly what exists, what is drafted, and what has not been designed at all.

11.1 What the two things actually are today

FluxOS is a per-node orchestration daemon: every Flux node runs an independent Node.js process that exposes an HTTP/HTTPS API, gossips application state over a signed peer-to-peer mesh, and decides — with no central scheduler — whether to pull and run a given application's Docker containers (SHIPPED, [flux/ZelBack/src/services/appLifecycle/appInstaller.js:415-746]). Placement is opportunistic self-selection: a node scans for applications short of their required instance count, picks among the eligible candidates largely at random, broadcasts an installing-intent message, waits 90 seconds for that message to propagate, and only proceeds if its own broadcast rank still fits the remaining slots — earliest-broadcast-wins, with no leader election and no on-chain arbitration (SHIPPED, [flux/ZelBack/src/services/appLifecycle/appSpawner.js:254-336, 666-684]). This runs today across a fleet measured at 6,489 nodes ([measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03]; Section 9 gives the full mechanism).

FluxEdge, in code, is not a variant of this. It is `pouwbackend`, a Go monolith that orchestrates rented compute through Rancher/Kubernetes: a documented OpenAPI 3.1.1 REST surface (`GET /gpus`, `POST /rental/start`, `POST /deployment/start`, and siblings), bearer-token authentication with enumerated, per-key privileges (`start_rental`, `stop_rental`, `start_deployment`, `stop_deployment`, `get_balance`) enforced against each request's path (SHIPPED, [pouwbackend/rest_api/v1/openapi.yaml:2571-2581]; [pouwbackend/rest_api/ratelimit.go:375-390]), and GPU assignment via Kubernetes device-plugin resources — a pod requests `nvidia.com/gpu: N`, and the backend reconciles that count against the rented machine's physical inventory by writing `NVIDIA_VISIBLE_DEVICES` directly (SHIPPED, [pouwbackend/rancher/deployment.go:914-982]). Isolation is whole-device: there is no MIG, vGPU or time-slicing code anywhere in the repository, and on contention the GPU resource request is silently stripped from the pod spec rather than queued or refused — a workload can be scheduled without the accelerator it asked for, with no error surfaced to the caller (Section 15 gives this its own treatment).

These are two different orchestration models — gossip-and-self-select over Docker versus API-driven Kubernetes rental — not two skins on one placement engine. That difference is exactly why convergence is an architectural claim rather than a rebrand: unifying the interface a tenant sees does not by itself unify how a workload gets a machine.

The two stacks share no machines, runtime or orchestration. FluxAI is a separate product and an ordinary tenant (customer) of FluxEdge: FluxEdge's own contractor-authored GitOps repository packages FluxAI's inference models (Llama 3.1, DeepSeek, FluxOne/Flux.1-dev) as Kubernetes manifests deployed through an Argo workflow whose node-selection parameters are documented as injected "dynamically" by FluxEdge at submission time (SHIPPED, [fluxai-fluxedge/custom-workflow-fluxone.yaml:7-9,17-18]). FluxAI sources its compute from FluxEdge machines, including home-hosted ones, and from Vast.ai and other AI compute providers; it does not run on FluxCloud or the FluxNode network. That is a customer relationship with one of the two runtimes, not a coupling between them, and no part of the convergence the roadmap describes has shipped. Today no tenant-facing workload can move between the two runtimes, and neither runtime knows the other's resource model.

11.2 "One provider runtime", concretely

The only concrete artifact for the convergence claim is a small, brand-new specification: the *Provider Runtime Contract v0*, in the `flux-spec` repository, alongside the unrelated v9 application-specification schema described in Section 10. The repository's own README states its status without qualification: "Status: RFC-0 — draft for comment (2026-09-01). Not live on the network" (IN-PROGRESS, [doc: flux-spec/README.md:3]). At the time of this survey the repository is two days old — its entire git history is two commits, both on 2026-09-01 ([flux-spec (git log):706db1d, dbaf9cb, 2026-09-01]) — and the package manifest declares `"private": true` with no `bin`, `main` or `exports` field, meaning no other Node.js project can currently `require` it (IN-PROGRESS, as a fact about the code today, [flux-spec/package.json:1-12]). Nothing in the corpus

reads it: it is not the schema the more mature FluxOS `feat/fluxos-v9` fork actually vendors for its own application-spec validation, which is a separate, private, commit-pinned monorepo ([MorningLightMountain713/flux@feat/fluxos-v9:package.json:87-90]), and no repository in this survey imports or calls into the provider-runtime schema specifically. IN-PROGRESS is the honest tag: it is a complete, independently-executable artifact, not vision language, but it has no consumer.

The provider-registration schema names eleven fields, three of them nested inside `capabilities` and two inside each entry of the optional `attestations` array (IN-PROGRESS, [flux-spec/provider-runtime/provider-registration.schema.json:7-80]):

Field	Constraint
`contractVersion`	`const: 0`, not upwards-compatible
`providerId`	string, opaque to the platform, $\leq$128 chars
`capabilities.bonded`	boolean
`capabilities.gpu`	boolean
`capabilities.staticIP`	boolean
`heartbeatIntervalSeconds`	integer, $[10, 300]$
`attestations[]`	optional; absence = standard trust tier
`.evidenceType`	open string token, not a closed enum
`.reference`	string, $\leq$512 chars
`payoutDestination`	string, $\leq$128 chars (a ZelID today)

A provider that has registered is tracked by a heartbeat state machine, defined entirely in prose in the companion contract document, with no implementation anywhere in the repository (IN-PROGRESS, [flux-spec/provider-runtime/contract.md:63-85]):

State	Trigger in	Trigger out
on-time	authenticated message within interval	interval expires with no heartbeat → LATE
LATE	recorded, no action taken	3 consecutive expired intervals → SUSPENDED
SUSPENDED	no new work assigned; in-flight work runs to its booking boundary	10 consecutive expired intervals (total) → LAPSED
LAPSED	registration removed from scheduling	only path back is a fresh, full registration

A heartbeat is defined payload-agnostically as "any authenticated message from the provider within the interval" [doc: flux-spec/provider-runtime/contract.md:67-69], and "clock authority is the platform's" [doc: flux-spec/provider-runtime/contract.md:83] — a provider that declares too short an interval only hurts itself. Attestation absence never blocks entry; it caps trust tier, a permissive-by-absence rather than deny-by-default posture [doc: flux-spec/provider-runtime/contract.md:39,121-123]. The document also records a five-business-day sign-off window between the FluxCore team and the operator submitting a contract, with the explicit fallback that an elapsed window without sign-off is recorded as "contract unilaterally published, sign-off pending" rather than blocking work [doc: flux-spec/provider-runtime/contract.md:144-155] — a governance process, not a technical guarantee, and one with no enforcement code anywhere.

11.3 Capability flags as the architectural argument

The claim worth making precisely is this: if one runtime advertises what a node can do — GPU presence, machine class, network-grade, storage durability — placement stops being "which of two systems handles this request" and becomes a constraint-satisfaction problem over one heterogeneous fleet. That requires three things to exist together: a capability vocabulary a provider can declare against, an advertisement protocol that gets the declaration (and its changes) to whoever places work, and a placement algorithm that actually reads the vocabulary when it chooses a node. Only the first two are drafted, and even the vocabulary is thinner than the full list above.

Vocabulary: drafted, and narrow. The provider-runtime contract's `capabilities` object carries exactly three boolean flags — `bonded`, `gpu`, `staticIP` — and the v9 application schema separately carries a three-value `machineType` enum, `standard` | `gpu` | `vps` (IN-PROGRESS, [flux-spec/schema/v9.json:79-82]). Nothing named "datacenter-grade networking" or "persistent storage" exists as a provider capability flag in either schema: persistent storage is expressed today as a per-component `volumes` request on the tenant side, not as something a provider advertises, and "datacenter-grade" is roadmap prose, not a schema token. A working capability vocabulary for the fleet described in this section would need to grow considerably before it covered what the roadmap's own language implies.

Advertisement protocol: drafted. Registration is the advertisement act, and the heartbeat keeps it alive — but because a heartbeat is defined as payload-agnostic (any authenticated message within the interval), the contract does not specify whether a provider updates its declared capabilities by heartbeat or must re-register to change them. That is an open question the draft leaves open, not a gap this paper is inventing.

Placement algorithm: the one in Section 9, and it consumes none of this. FluxOS's spawn loop selects candidate applications and candidate nodes with no reference to any capability schema. Concretely: `paymentMemo`, `referralCode` and `machineType` — three of v9's five required new fields — have zero occurrences anywhere in the `feat/fluxos-v9` fork's `ZelBack/src/services` tree (IN-PROGRESS, as a fact about the current codebase, [MorningLightMountain713/flux@feat/fluxos-v9:(grep across ZelBack: paymentMemo, referralCode, machineType: 0 hits)]). The only tier concept the placement code reads today is the pre-existing v8 Cumulus/Nimbus/Stratus `nodeTier()`, which is a collateral tier, not a capability declaration, and is unrelated to the v9 `machineType` axis ([MorningLightMountain713/flux@feat/fluxos-v9:ZelBack/src/services/generalService.js:55]). So of the three requirements the architectural claim depends on, one is drafted-but-thin, one is drafted-but-loose, and the third does not exist at all in the placement code that would have to consume it.

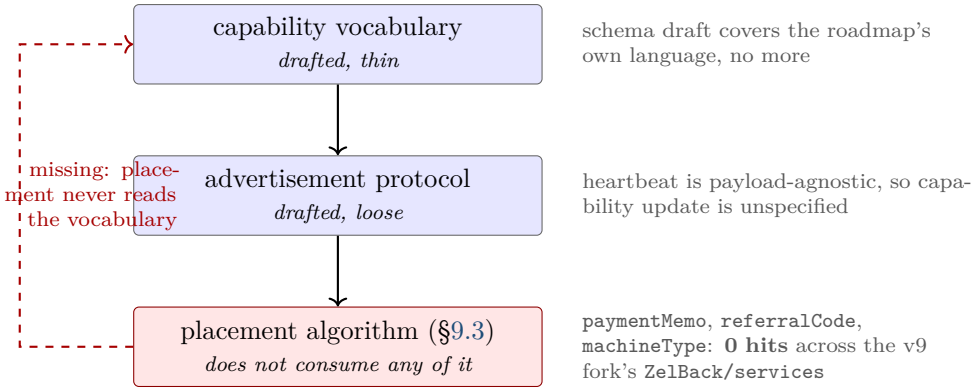


Figure 10: The three components a capability-typed provider runtime requires, and which of them exist today.

11.4 The datacenter tier and the field that names it

The roadmap states its position on uniform capacity plainly, under the heading "What we are not doing": " 'VPS on every node' in this cycle" is rejected because "home connections cannot serve it reliably. The datacenter tier is what we can do properly now" [roadmap: flux-roadmap-public.md:208]. That sentence has an architectural consequence the roadmap does not spell out: a fleet that includes both home-connection Cumulus nodes and purpose-built datacenter capacity is explicitly heterogeneous, and a specification that wants to route a workload to one class and not the other has to say so. That is what `machineType` is for — a field on the tenant side of the v9 schema that names `standard`, `gpu`, or `vps` as the class of machine a deployment requires (IN-PROGRESS, [flux-spec/schema/v9.json:79-82]).

The field is declared and unconsumed. As established above and in Section 10, `machineType` has zero consumer code anywhere in FluxOS, in either the organisation repository or its more mature external fork. The VPS tier the roadmap schedules for H1 2027 is stated to depend on this exact flag [roadmap: flux-roadmap-public.md:124-125], and is itself gated behind the Q4 2026 operator-protection work [roadmap: flux-roadmap-public.md:125]. So the tier has a name in the schema and nothing behind it: no scheduling logic reads `machineType`, no node advertises which value it satisfies, and no enforcement height is configured in FluxOS for the specification version that carries the field, and SPEC v9 has not activated: it is still in development (Section 10). Section 25 should carry this forward as one of the concrete, enumerable gaps between the roadmap's dependency chain and what the code implements, rather than as a vague "not yet finished."

11.5 What convergence costs

A single provider runtime is not free to build and would not be free to operate. The case for it is real: one registration schema, one heartbeat mechanism, and one placement algorithm mean one code path to secure and one place to add a new capability, instead of maintaining parity between FluxOS's gossip-based spawn loop and FluxEdge's Kubernetes reconciler independently. A tenant who can express `machineType: gpu` once and have it satisfied by whichever runtime currently holds the right hardware gains something no product-level merge can deliver on its own — portability across a heterogeneous fleet instead of a choice between two separately-billed products. The roadmap's "payment is the account, and per-second GPU without an account" framing depends on exactly this: it is stated to work "with FluxEdge folded into the platform" [roadmap: flux-roadmap-public.md:79], i.e. it assumes the runtime-level unification this section has just shown is not yet designed.

The cost is that a single interface is shaped by its weakest supported node class. If the capability vocabulary and placement algorithm are built to serve both a home-connection Cumulus node and a datacenter-tier machine through one schema, either the interface degrades toward what the weaker class can honestly promise, or the schema grows enough branching (as `machineType` already begins to do) that "one runtime" becomes, in practice, several code paths gated by a flag — which is close to where FluxOS and FluxEdge already are, just inside one repository instead of two. A home-connection node and a datacenter node do not share a failure model: the roadmap's own reasoning for rejecting "VPS on every node" is precisely that home connectivity cannot be relied on for a workload class that assumes always-on reachability [roadmap: flux-roadmap-public.md:208]. Any placement algorithm that consumes capability flags has to encode that asymmetry correctly, or a tenant who requested `vps` could in principle land on hardware whose network characteristics were never meant to carry it — precisely the failure the tier distinction exists to prevent, and precisely the risk of declaring a capability vocabulary before the placement algorithm that must honour it has been designed.

11.6 Vertical integration, argued both ways

Flux controls the chain (`fluxd`), the orchestration daemon (FluxOS), and the attested host (ArcaneOS). One party controlling all three makes a guarantee reachable that no single-layer actor could build alone: call it attested placement — a tenant's payment names a specific workload, FluxOS records which node placed it, and ArcaneOS attests that the node booted a known-good image, with all three artifacts meeting inside interfaces one party authors end to end.

No shipped mechanism assembles that chain today, and each leg is weaker than the phrase suggests. The first leg does not yet exist: today a payment settles to a fixed transparent sink address and does not reach the machine running the workload at all — the roadmap states this directly, "today when someone pays to run an application, that payment does not reach the machine running it" [roadmap: flux-roadmap-public.md:111-112] — and binding a payment to a specific workload is exactly what the PNR v2 `paymentMemo` field is for (PLANNED; Section 7). The second

leg is real but off-chain: FluxOS's placement decision is recorded in gossiped, signed messages, not on the chain itself. The third leg exists but is weaker than "attests the host" implies: ArcaneOS's attestation service already defines a purpose-scoped signing key for exactly this use — a `purpose=app` key sits alongside the existing, unconsumed `purpose=mesh` key (SHIPPED, [private source: SAS API server]) — but the signing material for every purpose is derived by HKDF from salt and domain values compiled identically into every binary, not from anything TPM- or machine-specific, so the attestation demonstrates that the signer holds a value shipped in every copy of the software, not that a particular machine booted a particular image; there is also no tenant-facing verification path today, since every attestation route requires an internal mTLS client certificate bound to loopback. So "attests the host" is, at present, a Flux-operated assurance a tenant cannot independently check, not a hardware-rooted one. The three names — the payment-memo field, the placement gossip message, the attestation purpose token — are each Flux's own schema decision; only a single controlling party is positioned to make them interoperate at all, which is the sense in which the guarantee is uniquely reachable through vertical integration. Whether it is actually reachable depends on work none of which has shipped: VISION, [inferred].

The same integration that makes this reachable has already foreclosed something else. The PoN block header carries a fixed chainwork contribution per block regardless of difficulty ([fluxd/src/pow.cpp:284-290]), and a 2-of-4 emergency key set can produce a valid block outside normal consensus rules. A header format chosen to make PoN's collateral-and-eligibility sortition work is the same header format a light client on another chain would need to verify Flux independently — and it cannot: without a commitment to the node set, without cumulative work to compare forks by, and with an emergency path that can override normal validity, no light-client rule derived from this header is sound. The most trust-minimising bridge architecture available to other chains is closed to Flux until the header changes [intent: mechanism specification]. That is the same asymmetry as the attested-placement argument, run in the other direction: a design decision made because one party controlled the whole stack, for one purpose, closed an option for a different purpose that a less integrated system might have kept open. Integration is not, on this evidence, a virtue or a defect on net; it is a set of coupled decisions with gains on one axis and closed doors on another, and this section states both rather than choosing one.

11.7 What would make convergence checkable

Three things would turn this from an architectural claim into a testable one. First, a published capability vocabulary — not the three-flag `capabilities` object and the three-value `machineType` enum as they stand, but a vocabulary that actually covers GPU class, network grade and storage durability, published somewhere a second implementation could target without vendoring a private repository. Second, a placement algorithm — whether a revision of Section 9's spawn loop or a successor — that reads that vocabulary when choosing a node, rather than the current mechanism, which ignores every field convergence would depend on. Third, an enforcement height in the shipped configuration: v9 has none today and has not activated (Section 10), and neither the provider-runtime contract nor the application schema that carries `machineType` becomes binding on the network until one is set. Until all three exist, "one platform" describes an intent the roadmap has stated clearly and a runtime the code has not yet built.

12 Networking and addressing: FDM, PDNS, mesh resolution, and private deployments

An operator can point a browser or an API client at an application running somewhere on the several-thousand-node Flux fleet and get an answer. That works today, and the path by which it works is worth naming precisely, because every hop on it is presently operated by the Flux Foundation (the informal name this paper uses for the people behind the project, Definition 2.3;

it is not a legal entity). This is not a claim about node operators: the compute itself is dispersed across independent Cumulus, Nimbus and Stratus operators exactly as advertised. It is a claim about *reachability* — DNS resolution, TLS termination, and the reverse proxy that decides which container answers a given hostname all run on Foundation-controlled infrastructure, discovered through one configured discovery endpoint, by default the Foundation-operated `api.runonflux.io`, with no automatic fallback to another node. Naming this path precisely, hop by hop, is what makes the decentralization roadmap in §23 and elsewhere legible: a reader cannot evaluate a plan to decentralize a system without first knowing exactly what is centralized in it today.

12.1 The reachability path, end to end

Application placement itself is decided elsewhere, by FluxOS (§9); nothing in this section governs *where* an application runs. What this section governs is how a client finds the IP:port that FluxOS placed it on. SHIPPED

1. **Discovery.** FDM (`flux-domain-manager`) does not talk to `fluxd` or to any peer-to-peer layer to learn what applications exist or where they run. It polls a single configured REST API base URL, by default the Foundation-operated `https://api.runonflux.io` (the routes it reads are served by every FluxOS node, see below), for `apps/globalappsspecifications` and `apps/locations` [flux-domain-manager/src/services/flux/dataFetcher.js:47-71]. This is FDM's only source for "what apps exist and where"; no fallback source exists in the codebase [flux-domain-manager/src/services/flux/dataFetcher.js:47-91].

2. **Polling cadence.** Three independent, self-rescheduling timers run per FDM process: `globalAppSpecs` at a default 30,000 ms (conditional on HTTP `ETag`), `appsLocations` hardcoded to 10,000 ms regardless of change, and `permMessages` at a default 120,000 ms [flux-domain-manager/src/services/flux/dataFetcher.js:47-91,409-417,689-721].

3. **Naming.** For a native application a with port P, FDM computes a_P`.app2.runonflux.io` (per port) and a`.app2.runonflux.io` (main alias) by string concatenation — `app.` rather than `app2.` on the production shard [flux-domain-manager/src/services/domain/index.js:9-32]. An operator may also set a custom domain in the application spec, which FDM verifies against its own IP via `dns.resolve4` before requesting a certificate for it [flux-domain-manager/src/services/domain/cert.js:128-157].

4. **DNS.** A hostname is served either by Cloudflare, used strictly as authoritative "DNS-only" (grey-cloud) DNS — FDM actively deletes any record it finds with `proxied === true` — or by the self-hosted PDNS zone service, selected per FDM host by `config.cloudflare.enabled/config.pDNS.enabled` [flux-domain-manager/src/services/domain/cert.js:170-181] [flux-domain-manager/config/default.js:32-47]. The evidence surveyed establishes how each backend behaves but not, from code alone, which one is authoritative for `runonflux.io` at any given moment: the checked-in default config is itself a certificate-only variant with `cloudflare.enabled: true`, and per-host Ansible templating can override this per fleet member [flux-domain-manager/config/default.js:38,46].

5. **TLS termination and routing.** The FDM host's own `HAProxy` instance terminates TLS, binding `*:443` against every `.pem` file under `/etc/ssl/fluxapps/` [flux-domain-manager/src/services/haproxyTemplate.js:81-134], then routes on the `Host` header via an ACL to the matching backend, forwarding to one of the application's container IPs at its declared port [flux-domain-manager/src/services/haproxyTemplate.js:260-319]. Per-server health checks (§12.2) decide which of an application's IPs are actually in that backend at any moment.

6. **Game-application shortcut.** For roughly two dozen "G-mode" game-server application types (Minecraft, Rust, Valheim, and similar), a separate, single-instance service, `flux-`

`apps-dns-manager`, reads the master IP FDM already elected into its own `/appips` output and publishes it directly as an `A` record via the DNS Gateway (§12.5) — client traffic, often UDP, goes node-to-node and never passes through HAProxy [flux-apps-dns-manager/src/services/appsDnsManager.js:202-255] [doc: flux-apps-dns-manager/README.md:3-15].

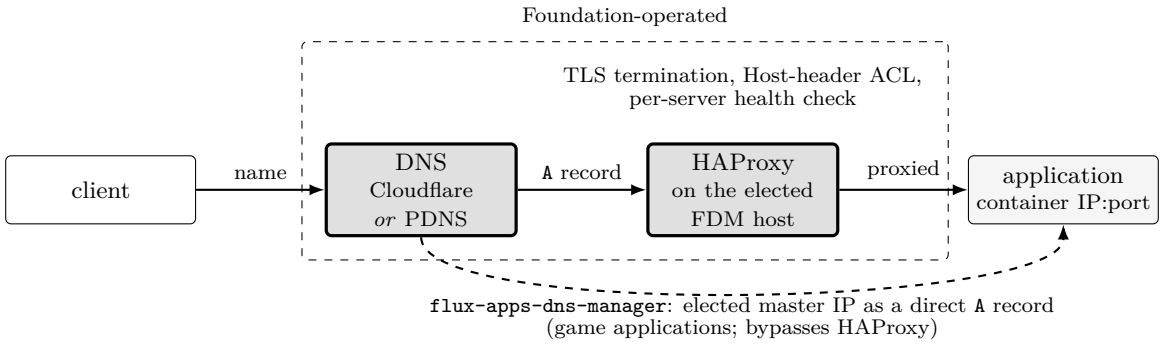


Figure 11: The reachability path from client to a running application, and the game-application shortcut that bypasses FDM's HAProxy entirely. Both intermediate hops are Foundation-operated SHIPPED; §12.8 inventories what each depends on and what happens when it is unavailable.

The `*.app.runonflux.io`/`*.app2.runonflux.io` wildcard certificate that step 5 depends on is treated by FDM's own code as always present — the presence check short-circuits to `true` for any domain ending in these suffixes, so FDM never calls `certbot` for them [flux-domain-manager/src/services/domain/cert.js:18-22].

Decided. the wildcard certificate is managed by InFlux Technologies USA LLC and renewed periodically against the `runonflux.io` domain [intent: design intake]. This is less of a concentration than it first reads. Behind any certificate there must be some company managing the domain — that is a property of the web PKI rather than of Flux — and what matters is whether the arrangement can be replicated. It can: `flux-domain-manager` is public under AGPL-3.0, the application specification already admits tenant-set custom domains [flux/ZelBack/src/services/appsService.js:passim], and any operator may run their own FDM, their own PDNS and serve applications from a domain they control.

Discovery is replicable too, and this paper's earlier reading of `api.runonflux.io` conflated two different claims. The endpoint is not a Foundation-only service: `/apps/globalappsspecifications` is an ordinary FluxOS route served by *every* node SHIPPED [flux/ZelBack/src/routes.js:402], and FDM takes its host as a constructor parameter, `fluxApiBaseUrl`, whose shipped value merely happens to be that name SHIPPED [flux-domain-manager/src/services/flux/dataFetcher.js:886]. Pointing an independent FDM at any of the 6,489 nodes, or at a bare IP, is a configuration change rather than a fork — and the code is AGPL-3.0 if a fork is wanted anyway.

What survives is narrower and is a resilience property, not a centralisation one: as shipped there is *one* base URL and *no automatic failover*, so an FDM whose configured endpoint becomes unreachable stalls on its last-known data rather than trying another node. That is worth fixing and is recorded in Table 38 on those terms. **What is centralised here is the default, not the capability.**

12.2 FDM's run loop

FDM's inputs are the three polled feeds above (application specs, locations, permanent messages); its output is a generated `HAProxy` configuration file, validated with `haproxy -f ... -c` before it is ever installed — a failed validation leaves the previously-loaded configuration serving, not a hard outage [flux-domain-manager/src/services/domainService.js:196-200,355-359] [flux-domain-manager/src/

services/haproxyTemplate.js:626-646]. Reconfiguration of the main HAProxy pool (the `home/api/cloud` UI/API backends) is driven purely by a 30,000 ms self-rescheduling timer, independent of any external event [flux-domain-manager/src/services/domainService.js:96-210]; that regeneration throws and aborts the cycle if fewer than 10 Stratus candidates pass its 8-stage health chain, keeping the old configuration running rather than publishing an under-populated pool [flux-domain-manager/src/services/domainService.js:110-183]. Reconfiguration of per-application backends is driven by the `appsLocationsUpdated` event fired on every 10-second location poll, which re-checks every location IP with a per-application-type coded health check and writes into the backend only the IPs that pass [flux-domain-manager/src/services/domainService.js:730-767]. SHIPPED

Table 37: FDM/PDNS reachability-path health-check and reconfiguration constants.

Mechanism	Constants	Source
Generic app backend, HAProxy-level	`inter 3s fall 2 rise 2 fastinter 500`	[flux-domain-manager/src/services/haproxyTemplate.js:311]
detection floor	≈6 s down (2×3 s)	[flux-domain-manager/src/services/haproxyTemplate.js:311]
Main UI/API backend, HAProxy-level	`inter 10s fall 2 rise 3 maxconn 100`	[flux-domain-manager/src/services/haproxyTemplate.js:384,408,427]
detection floor	≈20 s down, 30 s up	[flux-domain-manager/src/services/haproxyTemplate.js:384,408,427]
G-mode sticky-master retries	3 attempts, 3,000 ms apart	[flux-domain-manager/src/services/domainService.js:29-30]
G-mode sticky-unhealthy grace	90,000 ms since last healthy	[flux-domain-manager/src/services/domainService.js:31]
`appsLocations` poll (control plane)	10,000 ms, hardcoded	[flux-domain-manager/src/services/flux/dataFetcher.js:66,707]
Main HAProxy pool regeneration	30,000 ms, self-rescheduling	[flux-domain-manager/src/services/domainService.js:201-208]
Main pool minimum candidates	10 (throws below this)	[flux-domain-manager/src/services/domainService.js:181-183]
CDM certificate obtain/renew loop	15 min (900,000 ms)	[flux-domain-manager/src/services/domainService.js:1046-1088]

Because the G-mode control-plane grace window (90 s) is a much slower mechanism than HAProxy's own data-plane health check (≈6 s detection floor), an already-open connection to a dead sticky IP typically stops receiving traffic within HAProxy's own ≈6–9 s window; the 90 s constant instead bounds how long FDM keeps re-publishing the same, possibly-dead IP into the configuration before attempting election of a new one [flux-domain-manager/src/services/domainService.js:253-321]. SHIPPED

12.3 Certificates

Every certificate in this path is issued by Let's Encrypt, exclusively, via `certbot certonly --standalone --http-01-port=8787` [flux-domain-manager/src/services/domain/cert.js:38-46]; no wildcard issuance is coded — only exact-domain HTTP-01 challenges. The ACME account state (keys, order history) lives wherever `certbot` itself keeps it, under `/etc/letsencrypt/`, outside FDM's own code. The artifact `HAProxy` actually serves is a single, world-readable-by-haproxy file per domain, `/etc/ssl/{certFolder}/{domain}.pem`, formed by concatenating `fullchain.pem` and `privkey.pem` — public certificate and private key in one file [flux-domain-manager/src/services/domain/cert.js:38-46] [flux-domain-manager/src/services/haproxyTemplate.js:141-150]. SHIPPED

Issuance is technically permissionless: any host that can bind port 8787 for the HTTP-01 challenge and run `sudo certbot` can obtain a certificate for a domain it is authoritative for. Operationally, in the deployed fleet, only the 16 Flux-Foundation-run FDM/CDM hosts ever do so; an ordinary Cumulus, Nimbus or Stratus node operator never runs FDM or its certificate-only sibling [flux-fdm-infrastructure/ansible/inventory/hosts.yaml:1-100]. Two PM2 processes run from the same repository on the per-shard app hosts: `FDM` itself (`manageCertificateOnly: false`, handles HAProxy) and, on the EU-private-network hosts, a second clone named `CDM` ("cert-domain-manager", `manageCertificateOnly: true`), dedicated purely to issuance and renewal [flux-domain-manager/deployment/fdm_setup.yml:1-9,140-259]. CDM's loop runs every 15 min, classifying each custom domain into skip/obtain/renew — renewing when fewer than 30 days of validity remain — and deleting any certificate that expired more than 30 days ago [flux-domain-manager/src/services/domain/cert.js:18-36,91-126,214-249,308-340]. SHIPPED

12.4 `flux-dnsd` mesh resolution: a local file-tail resolver, not a gossip protocol

The name `flux-dnsd` invites a reading in which nodes discover each other's application members by talking to one another — a gossip or peer-to-peer resolution protocol. That reading is wrong, and the paper states this plainly because the mistake is an easy one to make from the name alone. `flux-dnsd` is a purely local, single-node process. It holds an in-memory view built from exactly one JSON file, `/var/lib/flux-mesh/resolver/membership.json`, and answers `*.mesh.flux` queries from that view, scoped to the querying container's source IP [flux-dnsd/src/snapshot.rs:319-363,79-84]. It never opens a connection to another `flux-dnsd` instance, never queries a central server at request time, and implements no notion of node-to-node consensus [flux-dnsd/src/snapshot.rs:319-363]. The file is written by FluxOS, which computes the per-node membership view and installs it with a temporary-file-then- rename, an atomic operation `flux-dnsd`'s file watcher depends on: only an inotify `IN_MOVED_TO` event whose destination is `membership.json` triggers a reload — a plain write-in-place is inert by construction [flux-dnsd/src/watcher.rs:20-69]. As the component's own documentation states, "FluxOS's database is the authority and rewrites it on boot-sync" [doc: flux-dnsd/README.md:75-85]. SHIPPED

The cross-node propagation that makes membership on one node visible to a container on another — the actual mechanism a reader might expect "mesh" to name — happens entirely inside FluxOS, outside `flux-dnsd`. From `flux-dnsd`'s own source this propagation mechanism is opaque: the daemon "only accepts or rejects [the file] whole," never repairing, merging, or arbitrating between disagreeing sources [doc: flux-dnsd/README.md:83-85] [flux-dnsd/src/snapshot.rs:72-77]. A snapshot whose generation number does not strictly exceed the currently loaded view's is rejected outright, so a stale or replayed file cannot regress the served view [flux-dnsd/src/snapshot.rs:397-416]. Answers carry a fixed 5-second TTL, justified in the source as "membership changes must propagate and nothing in the path caches anyway" [flux-dnsd/src/main.rs:57-60]. If no valid snapshot has ever arrived, or the source IP is unscoped, the daemon answers `REFUSED` rather than `SERVFAIL/NXDOMAIN`, so a client's stub resolver fails over to its secondary nameserver [flux-dnsd/src/server.rs:82-90]. SHIPPED

12.5 flux-dns-gateway: an mTLS proxy in front of the PDNS key

`flux-dns-gateway` is an mTLS-fronted proxy that lets the 16 FDM instances write DNS records into PDNS without any of them ever holding the PDNS API key themselves [doc: flux-dns-gateway/README.md:1-29]. Nginx terminates and verifies the client certificate and sets `X-SSL-Client-Verify/X-SSL-Client-CN` headers that the FastAPI backend trusts unconditionally, with no re-verification of the certificate itself in the Python layer [flux-dns-gateway/src/dns_gateway/api/middleware/auth.py:18-69]. Each client's ACL is enforced by `client_cn`, `allowed_zones`, `allowed_record_types`, and `allowed_prefixes`, deny-by-default for any unrecognised `client_cn` [flux-dns-gateway/src/dns_gateway/config/acl.py:13-97,99-151]. SHIPPED

The README's example ACL configuration also documents two further safety fields, `max_records_per_zone` and `rate_limit_per_minute` [doc: flux-dns-gateway/README.md:259-274]. Neither has any corresponding reader anywhere in the source: the `ACLConfig/FDMPermissions` parser reads only the four fields listed above [flux-dns-gateway/src/dns_gateway/config/acl.py:21-30]. A documented per-zone record cap and a documented per-client rate limit are, as deployed, no-ops. SHIPPED

12.6 flux-geo-cert: a citation trap

`flux-geo-cert`'s name and its corpus label ("Certificate Orchestrator for Flux Geo CDN") suggest node-location attestation. It is not that. It is a Python service that automates Let's Encrypt DNS-01 certificate renewal for the geo-routed CDN domain and pushes the resulting certificates over SSH to a small set of CDN nodes (Germany, US, Hong Kong) [doc: flux-geo-cert/README.md:1-13] [flux-fdm-infrastructure/ansible/inventory/hosts.yaml:155-177]. It has no inbound network interface at all — "pure outbound service" [doc: flux-geo-cert/ARCHITECTURE.md:54] — and

performs no measurement, claim, or signature about where any Flux node physically is. No code anywhere in the repositories surveyed for this section implements cryptographic attestation of node geography; the geolocation machinery that actually feeds placement decisions (fault-domain counting, residential-versus-hosting classification) lives in `fluxos-network-policy`'s `iplocation.bin.gz` artifact (§12.7), a build-time table sourced from RIR delegated-extended files and commercial geolocation data, not from any on-node measurement. This paper does not cite `flux-geo-cert` for, and makes no claim anywhere that, node geography is cryptographically attested.

12.7 fluxos-network-policy: the governance-relevant lever

`fluxos-network-policy` is a small set of JSON/binary documents — blocked repositories, vetted repositories, a tampering blocklist, an enterprise-node owner allow-list, and a ∼2.0-million-row IP→(organisation, country, region) geolocation table — that every FluxOS node fetches on a fixed schedule over plain HTTPS and enforces immediately, fleet-wide, with no staged rollout [doc: fluxos-network-policy/README.md:1-9,11-17,30-44]. Fetch intervals are 6 h for the blocked-repositories and enterprise-node documents and 12 h for the tampering blocklist, with the geolocation table refreshed on a daily conditional check [doc: fluxos-network-policy/README.md:34-35] [flux/ZelBack/src/services/utils/cacheManager.js:126-129] [flux/ZelBack/src/services/utils/enterpriseConfig.js:12] [flux/ZelBack/src/services/appTamperingBlocklistService.js:11]. A node that cannot reach the repository keeps enforcing whatever it last successfully fetched, indefinitely and across restarts — failure here is neither fail-open nor fail-closed-to-empty but fail-frozen, and a node that has never successfully fetched anything declines to answer rather than guess [doc: fluxos-network-policy/README.md:33-44]. SHIPPED

The documents are unsigned and served over plain HTTPS; by the component's own account, "their authenticity rests on GitHub and on who can merge to this repo" [doc: fluxos-network-policy/README.md:320-326]. Write access is restricted to GitHub organisation owners, and there is deliberately no second-reviewer requirement: CI runs a shape validator and a dual-write mirror check only, reaching green in roughly 15 seconds, after which the author of the pull request may merge their own change [doc: fluxos-network-policy/README.md:297-303] [fluxos-network-policy/.github/CODEOWNERS:1-7] [fluxos-network-policy/.github/workflows/validate.yml:1-27]. The stated reasoning is explicit: "the moment you most need to change policy is during an incident, and a rule requiring a second person is one that gets bypassed at 3am" [doc: fluxos-network-policy/README.md:300-302]. Once merged, every FluxOS node enforces the new document on its own next scheduled fetch — within 7–13 h depending on the document, the fetch interval plus the hourly sweep (§23), with no canary and no staged rollout [doc: fluxos-network-policy/README.md:3-5,34-35]. SHIPPED

Concretely, a single merged pull request can: block or unblock any container image, namespace, owner address, or 64-hex application hash network-wide (`blockedrepositories.json`, `vettedrepositories.json`); grant or revoke which owner addresses may install applications on which enterprise-designated nodes (`enterprisenodes.json`); mark a node's collateral transaction hash as blocked for tampering (`tamperingblockednodes.json`), a mechanism that holds the node registered with that collateral out of service (a DOS state) for that offense, without slashing or forfeiting the collateral itself; and, via the monthly geolocation-baseline pull request, reclassify which organisations or regions count as residential versus hosting for placement fault-domain counting and enforcement purposes [doc: fluxos-network-policy/README.md:320-326,99-172]. This is, by the component's own documentation, "the sharpest single point of centralized, unsigned control" identified across the network- and addressing-plane components covered in this section [doc: fluxos-network-policy/README.md:320-326]; §23 argues directly from this lever when it assesses censorship- and Sybil-resistance at the measured node distribution. SHIPPED

12.8 The centralization inventory

Table 38 states, without further comment, which surfaces described in this section are Foundation-operated, what depends on each, and what happens if each is unavailable.

Table 38: Centralization surfaces in the reachability and naming path.

Surface	Operator	Depends on it	If unavailable
`api.runonflux.io` (application discovery) — the *configured default*, not a unique source	Flux Foundation operates this endpoint; the route it serves is on every node	Every FDM/CDM control-plane loop reads app specs, locations and permanent messages from one configured base URL. The route is an ordinary FluxOS one served by all 6,489 nodes [flux/ZelBack/src/routes.js:402] and the host is a constructor parameter [flux-domain-manager/src/services/flux/dataFetcher.js:886], so an independent operator repoints it by configuration	**Resilience, not centralisation:** no automatic failover to another node is coded, so a loop whose configured endpoint is unreachable keeps polling, logs errors and serves the last-known HAProxy configuration (fail-static) [flux-domain-manager/src/services/flux/dataFetcher.js:47-91]
Foundation-private `10.100.0.0/24` (WireGuard hub, PDNS masters, DNS Gateway, Arcane load balancers, SAS decrypt endpoint)	Flux Foundation	All non-EU FDM/CDM and `flux-apps-dns-manager` instances, reachable outside the EU segment only through a single WireGuard hub, `wireguard-eu-01` (`10.100.0.167`, public `5.39.57.49:51000`) [flux-fdm-infrastructure/ansible/inventory/hosts.yaml:180-185]	Custom-domain certificate issuance, enterprise application-spec decryption, and game-application DNS publication all stop for every host that reaches this segment only through that hub [flux-domain-manager/src/services/domainService.js:1099-1105]
16-host FDM/CDM fleet (6 EU direct-private, 6 US and 4 Singapore over WireGuard) [flux-fdm-infrastructure/ansible/inventory/hosts.yaml:1-100]	Flux Foundation	HAProxy routing and TLS termination for every `app(2).runonflux.io` and custom-domain hostname network-wide	Stale-but-serving: a failed configuration validation leaves the previous configuration running, but no new application is routed or certified until a host recovers [flux-domain-manager/src/services/domainService.js:196-200]
`flux-apps-dns-manager` (single, unreplicated instance on `flux-cert-1` = `10.100.0.172`)	Flux Foundation	Direct node-to-node DNS for ∼26 G-mode game-server application types [flux-apps-dns-manager/config/gamesConfig.js:30-57]	No coded high availability, leader election, or standby process; game-application `A` records stop updating until the single host/co-located DNS Gateway returns [flux-apps-dns-manager/ansible/inventory.yaml:6-7]
Let's Encrypt (ACME certificate authority)	External (ISRG)	Every TLS-terminated `app(2).runonflux.io` and custom-domain certificate; exclusively used, no alternate CA coded [flux-domain-manager/src/services/domain/cert.js:38-46]	No fallback CA; the CDM renewal loop retries every 15 min indefinitely, and the 30-day renewal floor provides slack, but new domains cannot be certified at all during an outage
Cloudflare (where `config.cloudflare.enabled`)	External (Cloudflare)	DNS resolution for domains on the Cloudflare-selected path, used strictly DNS-only (grey-cloud), never as a proxy or CDN [flux-domain-manager/src/services/domain/cert.js:170-181]	No fallback registrar or DNS backend is coded per domain; whichever hostnames are Cloudflare-hosted stop resolving

12.9 Node network configuration

At install and reconfiguration time, network interfaces are discovered automatically: a `probert`-backed (netlink and udev) scan produces interface objects carrying type, name, state, index, MAC, vendor, model, address, VLAN and bridge identifiers with no operator input [fluxos-iso-networking-shared/src/flux_networking_shared/models/network.py:326-388]. What the operator configures, per interface, is the allocation mode — DHCP or static — through a TUI screen; choosing static reveals a form for subnet, address, gateway and DNS that is blur-validated before the change may be saved [fluxos-iso-networking/src/flux_iso_networking/screens/interface_modal_screen.py:54,79-84,200-284]. Per-interface traffic shaping is likewise operator-set, from a closed set of rates ($\{35, 75, 135, 250\}$ Mbps), written to a policy file and applied by a separate, privileged D-Bus helper because the configuring process itself runs without the capability to apply it directly [fluxos-iso-networking-shared/src/flux_networking_shared/models/network.py:77,83,139-161,224]. SHIPPED

The stack is IPv4-only by construction, not merely by current deployment. Every network dataclass — `Route`, `SourceAddress`, `NetworkInterface.address` — is typed `IPv4Network`/`IPv4Address`/`IPv4Interface` and nothing else, and the shared connectivity helper pins `TCPConnector(family=AF_INET)` explicitly [fluxos-iso-networking-shared/src/flux_networking_shared/models/network.py:17-29,326-337] [fluxos-iso-networking-shared/src/flux_networking_shared/helpers.py:147,242]. The measured fleet crawl underlying the network-wide baseline in §3 reports 2,322 IPv4 addresses and zero IPv6 or onion (Tor hidden-service) addresses [measured: api.runonflux.io/daemon/getzelnodecount, 2026-09-03]. SHIPPED

A reachability design for NAT'd and non-public nodes exists only as a docstring. The helper `is_directly_public()` states that a node with a globally-routable public IP bound to one of its own interfaces would act as a "Tor SOCKS-only hub," while a NAT'd node would instead run a "Tor hidden service" to become reachable [fluxos-iso-networking-shared/src/flux_networking_shared/models/network.py:461-479]. No Tor-related code exists anywhere else in either the installer or the shared networking library; the design is fully specified in prose and entirely unimplemented. VISION

12.10 Private deployments (VPON)

The roadmap lists private deployments, referred to as VPON s (private overlay networks), as a Q4 2026 item, described as "the complement to private payments and chain privacy" [roadmap: Private deployments (VPONs), Q4 2026]; the roadmap's own timeline artifact labels the same item "Private overlay networks," sub-labelled "VPONs · private deployments" [doc: flux-roadmap/public/timeline.html:172-173]. No implementation of a VPON was found in any repository surveyed for this section or flagged elsewhere in the corpus; the roadmap evidence itself notes no repository name observed anywhere in the codebase corresponds to a VPON [doc: flux-roadmap/public/flux-roadmap-public.md:83]. PLANNED

Because no such system exists to describe, this section states what a VPON would need to provide rather than how one works. At minimum, consistent with the notation this paper uses for the term and with the roadmap's framing of VPON s as private overlays rather than a naming variant on the public network:

- **Membership by key, not by discoverability.** The reachability path in §12.1 and the mesh namespace in §12.4 both resolve any application's members to any container scoped correctly on the querying node; a VPON would need membership gated by possession of a private key rather than resolvable by any party who can reach the mesh namespace or the public API [inferred].
- **Reachability-plane confidentiality.** If a VPON exists to complement private payment (§21) and chain privacy, then the infrastructure this section describes — the discovery API, FDM, and PDNS, all Foundation-operated — should not be positioned to learn which

nodes host a private deployment or on whose behalf, since today's reachability path is Foundation-legible end to end by the construction in §12.1 [inferred].

The roadmap sequences VPON s alongside private payments and the chain-privacy work without stating an explicit blocking dependency between them [doc: flux-roadmap/public/flux-roadmap-public.md:81,84]. Beyond this framing, no wire protocol, key-management scheme, or placement-isolation mechanism for a VPON is specified anywhere in the roadmap or in the repositories surveyed. The construction is open.

What does ship, today, are WireGuard-based transports inside CumulusVPN — a single VPN product, not a general per-application private-overlay primitive — that a future VPON design could draw on as building blocks:

- **AmneziaWG obfuscation.** A fork of `wireguard-go` that reshapes WireGuard's handshake framing without altering its cryptography, wired up as an additive listener sharing the vanilla gateway's server key. Gated off by default via an environment flag, but the fleet deployment manifest sets it on by default [cumulusvpn/gateway/internal/wg/device.go:19,67-124] [cumulusvpn/deploy/countries.yaml:23]. SHIPPED, disabled by default in code, enabled by default in the deployed fleet manifest.
- **WireGuard-over-TLS ("stealth" transport, port 443).** A TLS session terminated on a TCP port and bridged, byte-for-byte, to the local WireGuard UDP listener; the client does not verify the certificate chain, and trust is anchored entirely in the inner WireGuard handshake [cumulusvpn/gateway/internal/tlsrelay/relay.go:1-16,229-236]. Also gated off by default in code, also on by default in the fleet manifest [cumulusvpn/deploy/countries.yaml:24]. SHIPPED, same disabled-in-code/enabled-in-fleet pattern as above.
- **Multi-hop.** Implemented client-side only, with no corresponding change to the gateway protocol: the client runs two stacked WireGuard interfaces under the same key, and the entry hop decrypts only the outer layer before forwarding opaque ciphertext to the exit hop [doc: cumulusvpn/docs/11-multihop.md:8-33]. Because both hops see the same tunnel key in this version, an adversary controlling both a user's chosen hops could correlate traffic by key — a gap the design documentation states and leaves open, not a hypothetical one [doc: cumulusvpn/docs/11-multihop.md:47-58]. SHIPPED, client-side, with a stated unmitigated correlation risk.

Nothing in the evidence surveyed shows any of these three transports wired into a general, per-application VPON primitive; they are cited here only as the concrete pieces that exist and that a future VPON specification would not need to invent from nothing.

13 Managed services: databases, RPC, archive, and indexing

A stateless application on Flux tolerates a bad host: if the container dies, FluxOS restarts it, and if its node leaves the network another node self-selects to run a replacement instance (§9.3); either way the workload resumes with no state to lose. A managed database cannot make that trade. Its correctness is defined by what happens to data that exists only on machines the operator does not control, that are geographically dispersed, that may leave the network without notice, and that share none of a datacenter's usual assumptions — no shared storage fabric, no operator-trusted network, no guaranteed clock, no assurance that the node reporting a given IP is the node that was there a minute ago. Every other node-level daemon this paper describes (telemetry shipping, coordinated shutdown, expiry notification) is stateless with respect to the application: it observes or forwards, and if it fails the application's own state is untouched. A managed database is the one place on Flux where the platform's usual escape hatch — reschedule and move on — is exactly the operation that can destroy data. This section describes the two implementations that exist for this problem today, states precisely what each one guarantees, and shows that the two guarantees are not the same guarantee.

13.1 `Flux-Shared-DB`: ad-hoc leader election over the FluxOS location API

`Flux-Shared-DB` (SHIPPED) is a Node.js "Operator" sidecar that clusters independent MySQL/MariaDB engine instances running on separate FluxNode s into one logical database. It intercepts the MySQL wire protocol, sequences writes on an elected master, and replays them on followers via its own replication log, called the "backlog" [doc: Flux-Shared-DB/README.md:5]. It is deployed as an ordinary tenant-deployable FluxOS application component [doc: Flux-Shared-DB/README.md:20-49].

Leader election. Membership and master selection are derived entirely from the FluxOS application-location API, not from a consensus protocol: there are no term numbers, no quorum proof, and no Raft or Paxos implementation anywhere in the repository. `findMaster()` fetches the app's instance IPs from `https://api.runonflux.io/apps/location/{appName}`, polls every reachable peer's status, and ranks candidates by (1) highest replication sequence number, descending; (2) a static IP, true first; (3) OS uptime, descending [Flux-Shared-DB/ClusterOperator/Operator.js:1464-1489]. If the local node ranks first, it does not declare itself master outright: it asks the second-ranked candidate to confirm before doing so. If the local node does not rank first, it asks the top-ranked candidate who the master is, and if that answer disagrees with the candidate list, it asks the *reported* master to confirm itself [Flux-Shared-DB/ClusterOperator/Operator.js:1494-1543]. Any ambiguous or null response causes the routine to recurse with no fixed retry cap or backoff [Flux-Shared-DB/ClusterOperator/Operator.js:1508,1519,1538,1552]. This is a self-built, ad-hoc leader-election protocol layered directly on an HTTP location service, and it is the default, unconditional path — there is no feature flag selecting a different mechanism.

Failover detection. A health-check timer runs every 120,000 ms (2 minutes) and re-derives the master if any of: more than one node reports a conflicting master IP; the master fails three consecutive `getMaster` probes; the master's IP disappears from the app's location list; or, on the master itself, the Socket.IO server has zero connected clients [Flux-Shared-DB/ClusterOperator/Operator.js:1265-1338] [Flux-Shared-DB/ClusterOperator/server.js:1543-1544]. Independently of the poll, a follower's WebSocket `disconnect/connect_error` handler triggers immediate re-election on that follower, so the effective failover-detection latency is bounded above by the 2-minute poll but can be much shorter when the transport itself notices the drop [Flux-Shared-DB/ClusterOperator/Operator.js:178-196].

Write durability. This is the load-bearing finding for the section.

Property 13.1 (`Flux-Shared-DB` acknowledges before it replicates)**.** A write reaches the connecting MySQL client as soon as the elected master finishes executing that query against the tenant's own database on the master's local engine [Flux-Shared-DB/ClusterOperator/Backlog.js:145-149]. Only *after* the client has been told the write succeeded does the master broadcast it to connected follower sockets, via `serverSocket.emit('query', ...)`; no follower acknowledgment is awaited or required before that call returns [Flux-Shared-DB/ClusterOperator/Operator.js:379-389]. No code path awaiting a follower ack was found anywhere in `Operator.js`, `Backlog.js`, or `server.js`: this is fire-and-forget asynchronous replication, and there is no synchronous mode in this codebase. If the master fails between acknowledging the client and completing that broadcast, the write is gone: the next master is whichever surviving node holds the highest *already-replicated* sequence number [Flux-Shared-DB/ClusterOperator/Operator.js:1483-1489], and a write that never left the old master leaves no trace anywhere else in the code reviewed. The size of this data-loss window is not bounded by anything in the codebase examined: in the best case it is the single most recent write; under sustained load or follower lag it can be an unbounded number of recent writes, limited only by how far behind the surviving followers happened to be.

A second, compounding detail: even the master's own record of the write in its local backlog table is not confirmed before the client is told the write succeeded. The insert into `flux_backlog.backlog` is issued without an `await`, and the subsequent error check on its result is short-circuited by a literal `if (false && ...)` guard that can never fire [Flux-Shared-DB/ClusterOperator/Backlog.js:126-144]. Reads compound the problem in the other direction: ordinary (non-write) queries execute directly against whichever node the client happens to be connected to, regardless of replication lag, so a client on a lagging follower can read stale data indefinitely and two clients on two different followers can observe different states at the same instant [Flux-Shared-DB/ClusterOperator/Operator.js:720-733]. An `AUTH_MASTER_ONLY` setting restricts DB connections to the master only, but it is opt-in and defaults to `false` [Flux-Shared-DB/ClusterOperator/config.js:24].

Partition behavior. There is no quorum requirement to elect a master. A minority partition containing the node with the highest sequence number will elect that node as master regardless of how much of the mesh it can actually reach, so a classic split-brain — two nodes simultaneously believing themselves master and accepting writes — is possible whenever the mesh disagrees about who the master is for up to the 2-minute health-check interval; the code's only defense is detecting that disagreement on the next cycle, not preventing it in the first place [Flux-Shared-DB/ClusterOperator/Operator.js:1265-1271]. No explicit split-brain test or fencing mechanism was found in the repository, so this specific scenario is stated here as a structural consequence of the mechanisms above, not as an observed incident [inferred].

Maturity, honestly assessed. `Flux-Shared-DB` is shipped: it is the default, unconditional replication path, and a CI pipeline builds and pushes three Docker Hub image variants (`latest`, `latest-mysql`, `latest-mariadb`) on every merge to `master` [Flux-Shared-DB/.github/workflows/docker-image.yml:1-33], which is evidence of active use, not merely of a working build. But none of the properties above — leader election correctness, split-brain avoidance, or the failover data-loss window in Property 13.1 — has an automated test. The only test file in the repository exercises Bitcoin-message-signature verification, unrelated to replication, failover, or backup/restore [Flux-Shared-DB/test/bitcoinMessage.test.js:1-18]. The hardest properties of this system are, in this precise sense, unverified.

Two further findings, stated as facts rather than as speculation about intent: (1) an AES-256-CBC "comm" encryption layer for the internal cluster protocol exists in code (`Security.encryptComm`, a `shareKeys` socket event), but the key-exchange handshake that would populate the AES key and IV on the connecting side is commented out, so this layer is not exercised on the path reviewed and internal cluster traffic — queries, keys, session tokens — runs as plaintext Socket.IO by default [Flux-Shared-DB/ClusterOperator/Operator.js:151-168]. (2) The MySQL wire-protocol proxy parses the client's username and password scramble at connection time but never verifies the scramble against a password [Flux-Shared-DB/lib/mysqlServer.js:291-320]; the operator's actual authorization gate is an IP allowlist (`WHITELIST`, the first `172.x` address to connect, or membership in the app's own instance-IP list) [Flux-Shared-DB/ClusterOperator/Operator.js:324-359]. Authentication to this managed database is, in the code reviewed, IP-allowlist-only.

13.2 `flux-pg-cluster`: Patroni/etcd-backed PostgreSQL HA

`flux-pg-cluster` (SHIPPED) is a Go agent (`flux-agent`) that turns three or more FluxOS application instances into a standard Patroni-managed PostgreSQL high-availability cluster, adding Flux-specific glue: membership discovery via the FluxOS location API, a primary-routing TCP proxy, and conservative bootstrap/recovery logic intended to prevent split-brain across an unreliable, geographically dispersed, possibly NAT'd node set [doc: flux-pg-cluster/README.md:7] [flux-pg-cluster/cmd/flux-agent/daemon.go:25].

Leader election: standard, not custom. Unlike `Flux-Shared-DB`, this system does not reimplement consensus. `etcd` holds the distributed configuration store, and Patroni elects the primary via `etcd` leader keys with `PATRONI_TTL`= 30 s and `PATRONI_LOOP_WAIT`= 10 s, both defaults [flux-pg-cluster/patroni.yml.tpl:30-33] [flux-pg-cluster/internal/config/config.go:150-151]. The Flux-specific code renders Patroni's configuration template from discovered peer IPs and manages the `etcd` process lifecycle and cluster-membership reconciliation; it does not reimplement the election itself [flux-pg-cluster/cmd/flux-agent/init.go:422].

Primary-routing proxy. A Go TCP proxy listening on port 5433 polls every candidate node's Patroni `/primary` and `/replica` REST endpoints every `PROXY_HEALTH_INTERVAL_SECONDS`= 3 s (default) and forwards new TCP connections to whichever node most recently answered 200 on `/primary` [flux-pg-cluster/cmd/flux-agent/proxy.go:64,114-154] [flux-pg-cluster/internal/patroni/client.go:49-57]. In-flight connections are not killed on failover — the documentation states this explicitly, and the code performs no active disconnect [flux-pg-cluster/cmd/flux-agent/proxy.go:26-27] — so an application holding an open connection across a failover must detect the stale connection itself.

Bootstrap and recovery gating. On a from-scratch or dead-DCS boot, the agent probes every expected peer's HMAC-authenticated identity endpoint and requires either (a) exactly one node reporting non-empty PostgreSQL data, or (b) unanimous total-DCS-loss with matching PostgreSQL system IDs across every reachable peer, before selecting a recovery authority; ties are broken by the most advanced replica's durable WAL position, with deterministic node-name tie-breaking as a last resort [flux-pg-cluster/cmd/flux-agent/bootstrap_identity.go:263-378]. Any unreachable peer, membership mismatch, or multiple-primary condition blocks automatic recovery outright [doc: flux-pg-cluster/README.md:206] [flux-pg-cluster/cmd/flux-agent/bootstrap_identity.go:271-368]. Separately, a quorum-loss recovery path writes to a quorum-probe `etcd` key every reconcile cycle; if that write fails for `DesiredStateStabilityCycles`= 3 (default) consecutive cycles while the local node still holds PostgreSQL data, the agent rebuilds the `etcd` cluster with `--force-new-cluster`, gated by a cooldown [flux-pg-cluster/cmd/flux-agent/daemon.go:121-153,388-427]. This path is exercised end-to-end by `test_ec3_majority_replacement.py`, which kills two of three nodes and asserts recovery within a 300-second test budget [flux-pg-cluster/tests/scenarios/test_ec3_majority_replacement.py:1-60]. `ALLOW_PG_DATA_WIPE` defaults to `false`; on a system-ID mismatch, data is wiped only if this flag is explicitly `true` *and* the live primary confirms authority, otherwise the agent logs an error and leaves the data directory intact [flux-pg-cluster/internal/config/config.go:137-141].

Remark 13.2 (Default replication favors availability over guaranteed durability). Default replication in `flux-pg-cluster` is asynchronous: "the primary commits writes without waiting for replicas" [doc: flux-pg-cluster/README.md:210], confirmed by `PatroniSynchronousMode` defaulting to `false` [flux-pg-cluster/internal/config/config.go:158] and templated directly into Patroni's `bootstrap.dcs` configuration [flux-pg-cluster/patroni.yml.tpl:37-39]. This is a documented data-loss-on-failover trade-off, not an oversight, and the durability semantics themselves are delegated entirely to upstream Patroni/PostgreSQL streaming replication rather than reimplemented in this codebase. Setting `PATRONI_SYNCHRONOUS_MODE=true` is an opt-in toggle that makes every commit wait for `PATRONI_SYNCHRONOUS_NODE_COUNT`= 1 (default) replica acknowledgment before returning success, eliminating the failover-data-loss window at the cost of write latency; with `PATRONI_SYNCHRONOUS_MODE_STRICT=true` it additionally blocks all writes when no synchronous replica is reachable [doc: flux-pg-cluster/README.md:164-232] (the config plumbing for this is confirmed in code [flux-pg-cluster/internal/config/config.go:158-161]; the runtime commit-blocking behavior itself lives inside Patroni/PostgreSQL, outside this repository's Go code, and is asserted here only as configuration wiring, not as directly observed runtime behavior). Even in the default asynchronous mode, `PATRONI_MAX_LAG`= 33,554,432 bytes (32 MiB, default) caps how far behind a replica may fall and still be eligible for promotion during failover [flux-pg-cluster/internal/config/config.go:153] [flux-pg-cluster/patroni.yml.tpl:34]: the tolerance is generous, but it is bounded.

Test coverage. In sharp contrast to `Flux-Shared-DB`, this repository has a pytest integration-test suite (`tests/scenarios/`) that spins up three-node Docker Compose clusters and a mock FluxOS location API and asserts on real failure injection: leader kill followed by election within 120 s (`test_normal_failover.py`), unreachable-node and late-API-registration handling, majority-node replacement recovering via `force-new-cluster` within a 300-second budget (`test_ec3_majority_replacement.py`), and system-ID mismatch handling [flux-pg-cluster/tests/scenarios/:test_normal_failover.py, test_ec3_majority_replacement.py, et al.]. It also has Go unit tests colocated with the agent code (`bootstrap_identity_test.go`, `recovery_test.go`, `proxy_test.go`, `init_test.go`, `config_test.go`, `etcdmgr_test.go`, `fluxapi/client_test.go`). The repository's only CI workflow builds and pushes Docker images and does not invoke the pytest suite, so the suite is not a merge gate [flux-pg-cluster/.github/workflows/docker-image.yml:1-55]. `DEAD_CLUSTER_RECOVERY` and `ALLOW_PG_DATA_WIPE` are both live, tested safety toggles gating destructive recovery paths (the former on by default, the latter off by default) [flux-pg-cluster/internal/config/config.go:136-141] — concrete evidence that split-brain and data-loss edge cases were found and hardened against, rather than left theoretical.

13.3 The comparison

Both systems are SHIPPED, as operators a tenant deploys with its own application (§13.4); neither is the managed-database *product* the public roadmap lists as planned — "Postgres and MongoDB as one step in the deployment flow, with backup and restore attached" [doc: flux-roadmap/public/flux-roadmap-public.md:71-72]. Both solve the same problem: keeping a database consistent across nodes that are untrusted, dispersed, and may vanish. The engineering postures are not comparable.

	`Flux-Shared-DB`	`flux-pg-cluster`
Election protocol	Ad-hoc, hand-rolled over the FluxOS location API; no term numbers, no quorum proof [Flux-Shared-DB/ClusterOperator/Operator.js:1464-1489]	Patroni/`etcd`, standard leader-lease election, TTL 30 s, `loop_wait` 10 s [flux-pg-cluster/internal/config/config.go:150-151]
Failover detection	2-min health-check poll + immediate reconnect-triggered re-election [Flux-Shared-DB/ClusterOperator/server.js:1543-1544]	Patroni lease expiry (TTL 30 s) + proxy health poll every 3 s [flux-pg-cluster/internal/config/config.go:165]
Default write durability	Ack after master's local write only; broadcast to followers is unawaited (Property 13.1)	Ack after primary's local commit; async streaming replication (Remark 13.2)
Synchronous option	None found in the codebase	`PATRONI_SYNCHRONOUS_MODE=true`, opt-in [flux-pg-cluster/internal/config/config.go:158]
Split-brain defenses	Detection only, on next 2-min cycle; no fencing found [Flux-Shared-DB/ClusterOperator/Operator.js:1265-1271]	Quorum-loss gating, unanimous-identity bootstrap check, `force-new-cluster` cooldown [flux-pg-cluster/cmd/flux-agent/daemon.go:121-153,388-427]
Automated failure-injection tests	None (only a signature-verification unit test exists) [Flux-Shared-DB/test/bitcoinMessage.test.js:1-18]	pytest suite killing nodes and asserting recovery within stated time budgets [flux-pg-cluster/tests/scenarios/test_ec3_majority_replacement.py:1-60]

The lesson is not that one team is better than the other; it is that on a decentralized substrate, the durability guarantee a managed service can offer is bounded by the replication protocol beneath it, and the two protocols here are not equivalent. A tenant choosing between `Flux-Shared-DB` and `flux-pg-cluster` is choosing between different, precisely different, durability guarantees — not between two skins on the same underlying service — and the platform should say which is which rather than presenting both under a single "managed database" label.

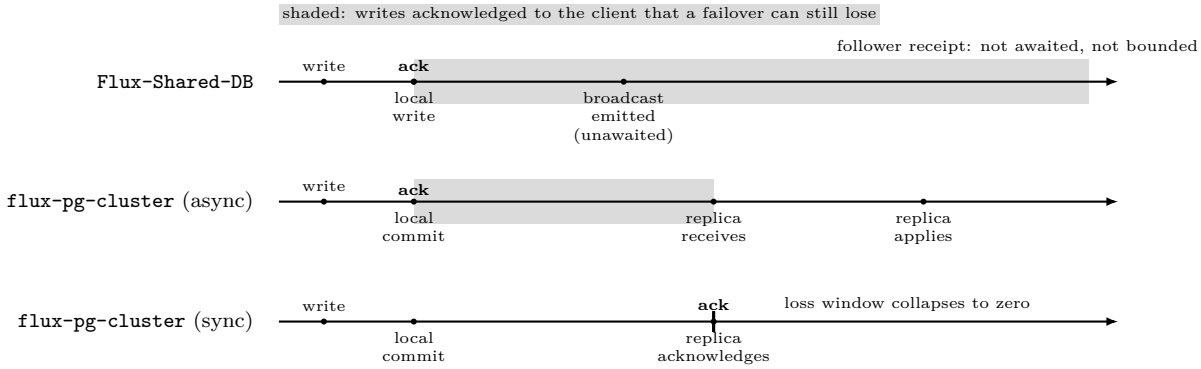


Figure 12: Write-acknowledgment timing relative to replication, compared across `Flux-Shared-DB` and the two `flux-pg-cluster` replication modes.

13.4 How a tenant uses these

`Flux-Shared-DB`. A tenant registers two FluxOS application components — a stock `mysql:latest/mariadb` engine and the `runonflux/shared-db` Operator image — or a single bundled image with the engine baked in [doc: Flux-Shared-DB/README.md:20-49,73-101]. Credentials are set at first boot via `DB_INIT_PASS`/`DB_USER` environment variables and are not re-keyable afterward: "changing it later will not re-key an already-initialized `/app/db` datadir" [doc: Flux-Shared-DB/README.md:88]. The tenant application connects with an ordinary MySQL connection string to the Operator's proxy port (default 3307) [doc: Flux-Shared-DB/README.md:46-49]. As established in §13.1, authorization at that proxy is IP-allowlist-based, not credential-based, and internal cluster traffic (queries, keys, session tokens) is plaintext Socket.IO by default, since the AES "comm" layer's key exchange is commented out. The debug UI can optionally run over TLS with a self-signed certificate if `SSL=true`, default `false` [Flux-Shared-DB/ClusterOperator/Security.js:163-185] [Flux-Shared-DB/ClusterOperator/config.js:25]. No at-rest encryption of the MySQL/MariaDB data directory was found in the repository.

`flux-pg-cluster`. A tenant registers a single FluxOS application component with a fixed port set (5432, 5433, 8008, 2379, 2380) [doc: flux-pg-cluster/README.md:78]. Superuser and replication credentials are supplied as `POSTGRES_SUPERUSER_PASSWORD`/`POSTGRES_REPLICATION_PASSWORD` environment variables and persisted, shell-quoted, to `/etc/cluster_env` for reuse by later agent subcommands [flux-pg-cluster/internal/config/config.go:331-332]. Tenants connect through the primary-routing proxy on port 5433, which always routes writes to the current primary across failovers, or bypass it to a specific node on 5432 for read-only access to a replica [doc: flux-pg-cluster/README.md:284-338]. Encryption is opt-in: `SSL_ENABLED` defaults to `false` [flux-pg-cluster/internal/config/config.go:117]; when set, a deterministic self-signed CA and per-service TLS certificates are generated for PostgreSQL, `etcd` peer/client traffic, and the Patroni REST API, and `pg_hba` accepts `hostssl/cert clientcert=verify-full` for replication connections alongside plain `host` md5 entries that the template and the reconciler retain in either mode [flux-pg-cluster/generate-certs.sh:45-306] [flux-pg-cluster/patroni.yml.tpl:69-73] [flux-pg-cluster/cmd/flux-agent/daemon.go:452-457]. With `SSL_ENABLED=false`, the default, inter-node PostgreSQL replication and `etcd` traffic run over the public internet in plaintext — the project's own architecture documentation labels this path "Replication via Public Internet" [doc: flux-pg-cluster/README.md:55]. As with `Flux-Shared-DB`, no at-rest encryption of the PostgreSQL data directory was found in the repository.

13.5 Backup and restore

`Flux-Shared-DB`. Backups are periodic full `mysqldump` snapshots, taken as part of the log-compaction cycle described in the evidence, written to `./dumps/*.sql` inside the container and retaining only the two most recent files [Flux-Shared-DB/ClusterOperator/Operator.js:504-548]. If

the application's declared container data includes `s:/app/dumps`, FluxOS's own directory-sync mechanism replicates that path across the app's instances — this sync is a FluxOS-level feature, not code in the operator repository itself [doc: Flux-Shared-DB/README.md:76,81-83]. Anyone who can reach the debug UI and authenticate via a Bitcoin-message-signature login tied to the application owner's ZelID can list, download, or delete backups, or execute an uploaded or URL-fetched `.sql` file directly into the live cluster via `/executebackup`, which first rolls the database back to sequence zero — i.e., wipes it — before importing [Flux-Shared-DB/ClusterOperator/server.js:653-715]. No evidence of restore being exercised by an automated test was found; the only test file in the repository covers signature verification, unrelated to backup or restore [Flux-Shared-DB/test/bitcoinMessage.test.js:1-18].

`flux-pg-cluster`. An opt-in (`BACKUP_ENABLED=false` by default [doc: flux-pg-cluster/README.md:170]) backup agent runs `pg_dumpall`, gzip-compresses the result, and overwrites a previous backup only after an integrity check (valid gzip, and presence of the "PostgreSQL database dump complete" marker) via a temp-file-then-atomic-rename pattern [flux-pg-cluster/cmd/flux-agent/backup.go:90-143,204-223]. Backups run only on whichever node Patroni currently reports as the healthy running primary, avoiding duplicate copies [flux-pg-cluster/cmd/flux-agent/backup.go:93-99,145-164]. Retention keeps the newest `BACKUP_RETENTION_COUNT`= 1 (default) files, then prunes further by a total-size cap that is unlimited by default [flux-pg-cluster/cmd/flux-agent/backup.go:275-337]. Backup storage is `/var/lib/postgresql/backups`, explicitly documented as *not* covered by persisting the main data-directory volume, and lost on reschedule unless separately persisted [doc: flux-pg-cluster/README.md:77,276]. Restore is a manual, documented `gunzip | psql` command [doc: flux-pg-cluster/README.md:280-282]; the README itself instructs the operator to "verify restores regularly" as their own responsibility [doc: flux-pg-cluster/README.md:186], which is a stronger honesty signal than a silent absence of guidance but does not amount to an automated restore guarantee. A file named `test_restore_reconciliation.py` exists in the same test suite [flux-pg-cluster/tests/scenarios/:test_restore_reconciliation.py], but its contents were not reviewed for this section, so it is not counted here as verified automated restore coverage.

For neither system, then, has restore been exercised in a way this section can verify end to end: `Flux-Shared-DB` has no restore test at all, and `flux-pg-cluster`'s restore path is documented as a manual operation with an unreviewed test file of unknown scope.

13.6 RPC, archive, and indexing (planned)

The roadmap names RPC, archive, and indexing services the network's "first capacity-selling product," scheduled for Q3 2026 [roadmap: flux-roadmap-public.md:33-34], and states that this product "pairs directly with chain pruning: the archive nodes pruning requires become the product" [roadmap: flux-roadmap-public.md:34]. Main-chain pruning itself is treated at reimplementation depth in §8; the roadmap's own framing of pruning is that "pruned nodes stay fully able to produce blocks, with designated archive nodes preserving full history for explorers and the data services above" [roadmap: flux-roadmap-public.md:121-122]. The direction of dependency between the two items is stated loosely ("pairs directly with"), not as a strict blocking gate [inferred].

No RPC, archive, or indexing implementation was surveyed for this section: the evidence available covers only the two managed-database operators above, and no repository matching an RPC gateway, an archive-node service, or an indexing service was read. This item is therefore reported here purely as a roadmap commitment, and its construction is left explicitly open rather than described as a system that does not yet exist.

Remark 13.3 (The shape of the RPC product — settled at the architectural level). The request-routing model was carried as undecided. Its architecture is now settled, and in a way that answers the structural half of the question while leaving the commercial half open: **archive nodes run as FluxCloud applications**, deployed by the team, with access to RPC and other node APIs sold as a product [intent: design intake] (PLANNED).

That decides three things. Routing is to *operator-deployed shared endpoints* rather than to per-tenant archive deployments — a tenant buys API access, not a node. The pruning-exempt storage requirement does not interact with the tiering and collateral model of §4 at all, because an archive node is a tenant workload subject to the application resource model, not a FluxNode tier: it needs no collateral and confers no eligibility. And history availability is consequently a property of this product rather than of the node set (Definition 8.2).

What remains commercial rather than architectural: the indexing schema and the pricing unit — per request, per method, per byte returned, or a flat subscription. Those do not constrain any mechanism in this paper and are not stated here.

Whatever this product becomes, it will inherit the same question this section has just answered for the two managed-database implementations: what does the platform promise about durability and consistency, and under what replication or storage protocol. Archived chain history is, if anything, a harder case than a tenant's own database, because it is by definition unrecoverable once lost from every archive node simultaneously; the paper takes no position here on which of the two postures compared above — hand-rolled and untested, or consensus-backed and failure-tested — an archive service would follow, because no such service has been surveyed.

13.7 Implication for lifecycle: eviction is where the guarantee is tested

The containers that implement both systems above are, from the node's point of view, ordinary Docker containers, and they are stopped through the same generic pipeline as any other application when a node evicts, redeploys, or is asked to stop an application. In production today, that pipeline executes only its Signal and Cleanup stages; the drain and pre-stop hook stages exist in the state machine and are unit-tested, but are "not yet reachable from real events" [flux-shutdownd/crates/shutdownd/src/pipeline/state_machine.rs:3-7], even though the underlying mechanism already reads per-container `shutdown.{drain,prestop,graceful}-s` labels intended to drive exactly this behavior [flux-shutdownd/crates/shutdownd/src/config.rs:41-65]. For a stateless workload, skipping a drain step is a latency blip. For a managed database, it is the difference between an orderly checkpoint and whatever durability guarantee — or absence of one — §13.3 describes: an application that is killed rather than quiesced is, from the perspective of Property 13.1, indistinguishable from a crash.

This is the requirement §16 must carry forward: a managed database is a stateful service whose eviction destroys data, so the grace, suspension, and eviction state machine that governs how and when a node stops an application matters more here than for any stateless workload this paper describes elsewhere.

14 Attested execution: ArcaneOS, the SAS, and the attestation network

A node that lies about its own hardware is one problem; a node that boots something other than the image it claims to run is a different, sharper one. ArcaneOS is Flux's answer to the second problem: a hardened Linux image with measured boot, disk encryption tied to the host, and a signed root filesystem, running a local service (`fluxsas`, the SAS) that other components query before they trust a node. The chain from firmware to a locked-down running system does real, verifiable work, and the update mechanism that ships new images is a carefully engineered, deliberately centralized ceremony. What the resulting *attestation* — the signed statement a node can produce about itself — does not do is bind that statement to the specific machine that produced it. This section describes the deployed chain first, states exactly where that binding fails, and then specifies the bonded attestation network the roadmap proposes as the fix: an approach that replaces "trust the key" with "trust the capital at risk," which is a primitive Flux already has and a hardware root of trust is not.

14.1 The deployed attestation stack

14.1.1 The boot chain

On a bare-metal or VPS install, ArcaneOS's Secure Boot Platform Key (PK), Key Exchange Key (KEK) and signature database (db) are written into the machine's live UEFI variables at install time from `.auth` artifacts staged on the EFI system partition, replacing whatever keys were present, after which the revocation database (dbx) is made immutable with `chattr +i` SHIPPED [fluxos-iso/flux_fs/fs_scaffold/curtin/curtin-hooks:26–43]. This is a Flux-controlled Platform Key, not a hardware-vendor or Microsoft key: its SHA-256 hash is a build-time constant (`FLUX_PK_SHA256SUM`) baked into the `fes` ("Flux Enrollment Service") binary SHIPPED [fluxos-iso/Makefile:65].

At boot, `fes` and `fluxsas` independently re-derive that trust. `fluxsas`'s `checkSecureBoot()` reads the UEFI `SecureBoot` variable and calls a fatal `terminateApp()` if Secure Boot is not enabled SHIPPED [fluxsas/src/sas.cpp:101–110]. `fes` performs the identical check during first-boot enrollment and, on the initrd side, `fes-wrapper` treats `fes`'s exit code as its entire contract: exit `50` means Secure Boot is disabled, `51` means the enrolled Platform Key's hash did not match the pinned value, `52` means no TPM2 device is present, and any other nonzero code is a generic enrollment failure — on any of these, the initrd prints a warning to the console, waits up to 30 seconds, and powers the machine off rather than continuing to boot SHIPPED [fluxos-iso-pkcs11/99flux-crypt/fes-wrapper:1–45]. The fuller code set (53–57, covering TPM2 enrollment failure, the VPS check, PKCS#11 enrollment, keystore failure and device-unlock failure) is defined alongside these in the same enrollment tool SHIPPED [fluxsas/secrets:FES_EXIT_* block]. This is a real fail-closed boot gate, correctly credited: a node that cannot prove Secure Boot is on, with the right key, and a TPM2 present, does not come up at all.

Where a TPM2 device exists, `fes` enrolls it against the system LVM volume with `systemd-cryptenroll --wipe-slot=all --tpm2-device=auto --tpm2-pcrs=7` SHIPPED [fluxsas/src/fes/fes.cpp:173–186]. Only PCR 7 is sealed against — the PCR that measures Secure Boot policy state (PK/KEK/db/dbx contents and the enabled flag), not the kernel, initrd, or root filesystem image. The direct consequence, read from the enrollment code rather than inferred, is that a TPM-sealed key unseals for *any* kernel and initrd signed under the pinned Platform Key, not specifically the build that was running when the key was sealed — image-version binding is not something the TPM seal itself provides.

That is the enrollment path, and it is not the whole of what the image measures — a distinction that matters for §14.3 and that an earlier reading of this section missed. ArcaneOS also ships `systemd-pcrlock` and runs it on every capable machine: `flux-pcrlock-init` registers the `systemd-boot` binary and the current Unified Kernel Image as pcrlock components, and the service drop-in then executes `systemd-pcrlock make-policy --pcr=4 --pcr=7 --pcr=11`, retrying with firmware overrides until all three registers are covered SHIPPED [fluxos-iso/flux_fs/bin/flux-pcrlock-init:18–62,76–152], [fluxos-iso/flux_fs/systemd/system/systemd-pcrlock-make-policy.service.d/flux-init.conf:1–9]. PCR 4 and PCR 11 are the registers that measure the boot loader and the UKI's own sections, so the policy this produces *does* bind to an image version, which the PCR 7 seal above does not. It is gated on the TPM supporting `TPM2_CC_PolicyAuthorizeNV` at revision 1.38 or later, tested by `flux-tpm-pcrlock-capable` before the service is allowed to run SHIPPED [fluxos-iso/flux_fs/bin/flux-tpm-pcrlock-capable:1–32]. This has been on `master` since March 2026.

What is missing is a consumer for the key this section is about. The disk-unlock enrollment cited above binds to PCR 7 rather than to this policy, and the root-volume attach passes `tpm2-device=auto,pkcs11-uri=auto` rather than `tpm2-pcrlock=` SHIPPED [fluxos-iso-pkcs11/99flux-crypt/cryptsetup-wrapper:5–7]. More to the point for what follows: no path in the read repositories binds the *attestation* key of Property 14.2 to the pcrlock policy, or to any other per-machine value. The accurate statement is therefore narrower, and more actionable, than "image-version binding does not exist": the machinery that produces an image-bound TPM

policy is deployed and running across the TPM population, and the attestation key is not bound to it.

Root disk unlock is offered both hardware and software paths simultaneously: `systemd-cryptsetup@root.service` attaches the root LUKS volume with `tpm2-device=auto,pkcs11-uri=auto` set together SHIPPED [fluxos-iso-pkcs11/99flux-crypt/cryptsetup-wrapper:5–7]. Where no TPM2 device is present, `fes` first checks whether the host is a hosting/VPS IP via a third-party lookup, then falls back to a software PKCS#11 token: an RSA-4096 key generated inside a SoftHSM2 store held in an encrypted loopback image SHIPPED [private source: SAS enrollment]. **This fallback is where the guarantee weakens**, and it weakens for a structural reason rather than a fixable one: a software token has no per-machine secret to bind to, so the material protecting it is necessarily something every copy of the software carries. Per-machine sealing degrades to per-machine-plus-anyone-who-can-read-the-software sealing. The fallback exists mainly for VPS hosts that do not expose custom Secure Boot key enrollment to the guest, and the ArcaneOS documentation itself notes this population is shrinking as providers close that off [doc: fluxos-iso/README.md:289–296].

Root filesystem integrity is enforced by dm-verity: `fluxsas` parses `veritysetup status` for the active root hash and its `verified` status SHIPPED [fluxsas/src/watchdog.h:449–506]. This confirms the running rootfs is mounted under dm-verity; the boot-time source of the *expected* root hash consumed by the initrd is not established by the evidence available for this section and is not asserted here. Finally, the initrd mitigates the physical-access window between an unattended install and first boot: once a disk-unlock token has been enrolled, `flux-init-cleanup` mounts the plaintext, `root`-labelled filesystem and deletes everything on it except the current squashfs image, its verity file, install logs, and small configuration files, specifically "to mitigate the primary drive being mounted by an attacker after installation, but before first boot" SHIPPED [fluxos-iso-pkcs11/99flux-crypt/flux-init-cleanup:20–60]. A real window remains between install completion and that cleanup pass, during which the plaintext root and the unencrypted EFI system partition are both mountable by anyone with physical access to the disk.

14.1.2 The update channel

The strongest link in the chain is how a new ArcaneOS image reaches every node. Release signing runs through a physically Flux-operated YubiHSM 2, whose objects include a dedicated Ed25519 release-signing key (`FluxOsReleaseKey`) alongside the Secure Boot signing keys SHIPPED [flux-iso-hsm/src/flux_hsm/hsm_manager.py:178–544]. A human custodian authenticates to the HSM with their own personal YubiKey via challenge-response key derivation, so no raw password ever leaves the YubiKey SHIPPED [flux-iso-hsm/src/flux_hsm/hsm_manager.py:686–764], and a build session's actual signing credentials are fresh, single-use authentication keys created for that session and expected to be deleted afterward SHIPPED [flux-iso-hsm/src/flux_hsm/hsm_manager.py:1064–1110]. The HSM's own master wrap key is backed up by Shamir's Secret Sharing across an M-of-N set of custodians, driven end to end over a dedicated ceremony protocol SHIPPED [flux-iso-hsm/src/flux_hsm/server/sss.py:12–37]. A finished build is bundled into a release tarball and signed with that Ed25519 key over a chunked SHA-256 digest of the whole artifact SHIPPED [fluxos-iso/flux-release-signer/src/flux_release_signer/main.py:69–152]; every node verifies incoming updates against a single Ed25519 public key baked into the image at build time SHIPPED [fluxos-iso/flux_fs/build_target.sh:81–84].

Stated plainly: **only Flux can publish an ArcaneOS update.** Every node verifies an incoming update against exactly one embedded Ed25519 public key and no other SHIPPED [fluxos-iso/flux_fs/build_target.sh:81–84], and the only private key that verifies against it lives inside the Flux-operated HSM behind the custodian ceremony described above SHIPPED [flux-iso-hsm/src/flux_hsm/hsm_manager.py:336,1064–1110]; whoever holds that HSM key can therefore produce an update every node's updater will accept. This is not an oversight; the ceremony design — personal-YubiKey custodian authentication, single-use per-build signing keys, and Shamir-split master-key recovery, all cited above — is a deliberate, carefully engineered centralization of

exactly one function. It should be read as such rather than apologized for. (A WireGuard-tunneled fast path for the signing session exists alongside the default SSH-tunneled one but is gated behind a flag documented as defaulting to off, and is tagged IN-PROGRESS here rather than folded into the ceremony's shipped description [doc: flux-iso-hsm/docs/SETUP_FLOW.md:333–353].) The one caveat worth stating alongside it: the same build scripts that produce production releases also carry explicit debug bypasses that ship an unsigned tarball, UKI or kernel module when set SHIPPED [private source: ArcaneOS image build]. If one is set on a production build the signature chain for that artifact is void, and no code in these repositories checks that a released artifact was actually signed before publication. The ceremony above is therefore strong at the point of signing and unenforced at the point of release — a gap in process rather than in cryptography, and one §14.2 places in the class being closed.

One structural property of this arrangement deserves stating, because it is the fleet's best-known single point of failure and the paper would otherwise leave it implicit. The verification key is a single Ed25519 public key installed into the image at build time SHIPPED [fluxos-iso/flux_fs/build_target.sh:81–84], so *rotation* exists in exactly one direction and only while the current key is uncompromised: an update signed by today's key can install tomorrow's. *Revocation* does not exist in any direction. There is no second key, no threshold over custodians at the artifact level — the Shamir split protects the HSM's master wrap key, not the release signature — and no published mechanism by which a node learns that a key it already trusts should no longer be trusted. An adversary who obtains the signing key can therefore sign an image that pins their own key, and the recovery procedure is a manual re-install of every node. Threshold-signing releases, or shipping an explicit revocation channel, is the change that removes this; neither is present in the read repositories.

14.1.3 The `fluxsas` protocol

`fluxsas` serves a single listener at `https://127.0.0.1:16100`, bound to loopback, with mutual TLS mandatory (`SSL_VERIFY_PEER | SSL_VERIFY_FAIL_IF_NO_PEER_CERT`) SHIPPED [fluxsas/src/api_server.h:1042–1053,1020]. Authorization beyond the TLS handshake is by client-certificate Common Name, checked once in a routing handler rather than pinned per route at the TLS layer, and the two authorized callers are not equally scoped: one is restricted to a three-route subset while the other is admitted to every route, with any other or missing CN rejected SHIPPED [private source: SAS API server]. Both authorized callers are Flux-operated, node-local processes, not tenant-facing ones — a fact with consequences below. **The consequence of the asymmetry is that the blanket-authorized caller is a single point of compromise for everything `fluxsas` can sign, encrypt or decrypt on that host**, and its client identity toward `fluxsas` is not per-node. That is an authorization-design finding rather than a property of attestation, and §14.2 states how it is handled.

The two checkouts read for this subsection are three months apart and do not interoperate as read, so the client behaviour described here is one component's and the listener behaviour the other's, each at its own checkout.

The route family of interest for attested execution is `/v2/attest` (Ed25519-signs a caller-supplied message under a domain-separated key selected by a `purpose` argument — absent, `cdn`, `mesh`, or `app`) and `/v2/attestationPublicKey` (returns the corresponding public key) SHIPPED [private source: SAS API server]. Adjacent routes cover a per-app transport key with its own domain-separated attestation SHIPPED [fluxsas/src/api_server.h:582–653] and a blob-upload signer SHIPPED [fluxsas/src/api_server.h:750–829]. Freshness is enforced unevenly: only the `upload` and `manifest` kinds of `/v2/signBlobUpload` check a ±300-second timestamp window at signing time SHIPPED [fluxsas/src/api_server.h:183,780–829]; `/v2/attest` and `/v2/transportPublicKey` carry no server-enforced freshness at all, so a verifier consuming either must supply its own staleness policy.

What a verifier learns from a successful `/v2/attest` call under `purpose=app` — the route

intended to bind a stored content hash to a validated envelope hash — is that some running `fluxsas` process produced a valid signature over that pair. Whether that constitutes evidence about *which* host produced it, and under what boot state, is exactly the question the next two subsections answer.

14.1.4 The trust root

Assumption 14.1 (The deployed attestation trust root)**.** The statement "this node booted something Flux considers legitimate" rests today on a compound root that is entirely Flux-operated:

1. a Flux-controlled UEFI Secure Boot Platform Key, pinned by hash inside the `fes` binary and checked against whatever key the firmware has enrolled SHIPPED [fluxsas/src/fes/fes.cpp:70–81];
2. a Flux-operated pair of web services (`stats.runonflux.io` and `arcanehashes.app.runonflux.io`) that publish the set of dm-verity root hashes a node's watchdog treats as known-good, polled roughly once every 24–48 hours, and which **fail open**: if both endpoints are unreachable, the watchdog treats the running node as secure and takes no action SHIPPED [private source: SAS watchdog]. That is the opposite of the fail-frozen behaviour the network-policy channel of §23 exhibits under the same conditions, and two policy channels in one system failing in opposite directions is the finding, not the outage handling of either; and
3. a Flux-operated internal mTLS certificate authority that issues the client and server certificates every `fluxsas` caller and listener presents.

No independent hardware vendor, third-party attestation authority, or externally verified TPM quote appears anywhere in this root.

None of this makes the boot chain worthless: measured boot, dm-verity, and LUKS unlock all impose real, verifiable cost on a remote attacker with no local access. It does mean the assurance a node offers is a Flux-operated one, not a hardware-rooted, independently verifiable one, and later claims in this paper build on Assumption 14.1 exactly as stated, not on a stronger reading of it.

14.1.5 The property that does not hold

Property 14.2 (Attestation is not machine-bound)**.** The Ed25519 signing key `/v2/attest` uses for the default, `cdn`, `mesh`, and `app` purposes is derived by a key-derivation function from static values compiled identically into every `fluxsas` binary; none of the derivation's inputs depends on the TPM, on Secure Boot state, or on any other per-machine value SHIPPED [private source: SAS API server]. The one exception, the node-identity key pair (`/v2/nodeIdentityPublicKey`, `/v2/nodeIdentitySign`), does mix in a value derived from local machine identifiers — the one place the deployed stack does per-machine work — but the material protecting that value's random component at rest is again carried identically in every copy of the software SHIPPED [private source: SAS disk encryption], so the exception does not survive the same argument. Consequently, a valid `fluxsas` attestation demonstrates possession of something every copy of the software contains — it does not demonstrate that a particular machine booted a particular image. Compile-time obfuscation of these constants changes what a casual binary scan reveals; it does not change this property.

This is stated as a property that fails, not as a procedure: no derivation input, salt, domain value, or extraction path appears above or anywhere in this section, and none is needed to state the property precisely. Property 14.2 is the reason the bonded network described in §14.3 exists, and it is the reason its Requirement 0 is written the way it is.

14.1.6 What a tenant can verify

Today, nothing. Every `/v2/*` route on `fluxsas` requires a client certificate issued by the internal Flux mTLS CA and is reachable only on loopback SHIPPED [fluxsas/src/api_server.h:1042–1053,220–252]; an external tenant deploying an application onto a FluxOS node has no network path to `fluxsas` at all, mTLS or otherwise. The closest thing to a tenant-visible signal is the `purpose=app` attestation described above, and no code path in the read repositories surfaces that signature to a tenant. Even if one did, per Property 14.2 it would prove "some `fluxsas` process signed this," not "this specific host booted a known-good image." There is no tenant-facing verification API, no TPM-quote endpoint, and no remote-attestation client anywhere in `fluxsas` or its provisioning tooling.

14.1.7 Residual trust surface

Beyond the trust root of Assumption 14.1, the deployed system asks a tenant to accept the following, none of which `fluxsas` measures or attests to:

- **The container runtime.** Nothing in `fluxsas` measures the Docker or Podman daemon; `areAllServicesInactive()` checks that `containerd`, `docker.service`, and related units are *inactive* before an unmount, which is a shutdown gate, not an integrity check SHIPPED [fluxsas/src/disk_encryption.h:547–559,642].
- **FluxOS itself.** Application orchestration code runs outside `fluxsas`'s scope entirely; no file in the read evidence measures or attests to it.
- **Physical access.** The account checks `fluxsas` applies do not cover a console session SHIPPED [private source: SAS watchdog], and the install-to-first-boot window described in §14.1.1 is a further, time-bounded physical exposure.
- **The update channel.** A single Flux-operated signing ceremony (§14.1.2) decides what code every node will accept as an upgrade; whoever holds that key can push code the entire fleet runs.
- **The hash allow-list service.** `stats.runonflux.io` and `arcanehashes.app.runonflux.io` decide what a running node considers a known-good image, fail open on outage, and are both Flux-operated.
- **The absence of a hypervisor boundary between tenants**. Application workloads run as containers directly on the ArcaneOS host — the filesystem ships a `docker-to-podman-migration.service` unit, consistent with containers rather than per-tenant virtual machines as the isolation primitive on production nodes — and no evidence in the read repositories shows a per-tenant hypervisor-enforced boundary[inferred]. This is the only item in the list that no amount of attestation work would remove.

14.2 Disposition of the findings in this section

The findings above are not one kind of thing, and a reader deciding how much weight to put on them should be told which is which — and, on one class, told why the paper stops short of coordinates.

Class A — ordinary defects, bounded in size, being closed. Credential material held where it should not be; an authorization table that admits one node-local caller to everything rather than to what it needs; a policy poller that fails open where its counterpart fails frozen; account checks that do not reach every session type; release bypasses with no publication-time signature gate; and a version-pairing question between two node-local components. None of these is a research problem, and none changes any property this paper asserts. Each is a change

of bounded size to code that already exists, and the disposition is to close them [intent: design intake].

Why this class is stated without file and line, uniquely in this paper. Everything else this paper cites, it cites precisely — that discipline is the point of the document. The components in this class are the exception because they are the ones that are *not public* (§29). Publishing exact coordinates into a closed component does not help a reader verify anything, since they cannot open the file; it helps only someone attacking a defence they otherwise cannot read. And these particular components are closed for that exact reason: they are what makes a node's eligibility hard to fake, and the whole of §7's sequencing argument is about how much that matters. So the paper states each finding as a property and omits the coordinates. The properties are what a reader evaluating Flux as infrastructure needs; the coordinates are what an attacker needs. Where the same class of defect appears in a *public* repository — the unsigned policy documents of §23, the emergency-block path of §4 — this paper cites file and line as it does everywhere else, because withholding those would hide from honest readers and from nobody else.

Class B — the structural one. Property 14.2 is not a defect and does not have a patch. An attestation key derived from values every copy of the software carries cannot be made machine-bound by fixing a bug; it requires binding a key to a hardware root and exposing a verifier, which is the work §14.3's Requirement 0 describes and which the pcrlock finding in §14.1.1 makes considerably nearer than a first reading of this section suggests. This is the item that stays open even once the Class A items close, and it is the one the rest of the paper's attestation dependencies actually rest on. It is stated in full, without redaction, because it is a property rather than a procedure — no derivation input, salt, domain value or extraction path appears anywhere in this section, and none is needed to state it precisely.

Saying all of this explicitly is a deliberate choice. A section that reads as an undifferentiated list of weaknesses invites both of the wrong conclusions — that the system is unsound because there are many findings, or that the findings are unimportant because most are small. The correct reading is that most are small *and* being closed, that the one which is neither is stated as a property, and that the paper has drawn the line at the point where precision stops informing a reader and starts arming an attacker.

14.3 The bonded attestation network

The roadmap's proposed fix would not be a better key; it would be a design that stops depending on a key at all and depends on collateral instead. This subsection specifies a mechanism that does not exist yet, and every claim in it is framed accordingly.

Assumption 14.3 (Requirement 0)**.** A bonded attestation network would be meaningful only once two conditions hold that do not hold today: the attestation signing key would need to be derived from, or sealed to, a per-machine hardware root — TPM-sealed or Secure-Boot-bound, in a way Property 14.2 does not currently satisfy — and a tenant-reachable verification path would need to exist where today there is none (§14.1.6). Absent the first condition, a bond would secure a signature that anyone holding the `fluxsas` software, not merely the bonded node, can produce today (Property 14.2), which would make the bond forfeitable for statements the bonded party never made. [intent: mechanism specification]

Construction 14.4 (The bonded network)**.** A node i would opt into the attested tier by posting a bond β_i *separate from* its FluxNode collateral. Base collateral would remain exactly as it is today — self-custodied, refundable, untouched by any slashing path in this mechanism.

A *claim* would be a tuple

$$\mathsf{cl} = (\mathsf{subject}, \mathsf{predicate}, \mathsf{value}, h, \mathsf{nonce}), \qquad \sigma_{\mathsf{cl}} = \mathrm{Sign}_{sk_i}(H(\mathsf{cl})),$$

where subject would name what is attested (an image digest, a reserve balance on a foreign chain, a computation result, an observed state) and predicate would name the assertion made about it.

For each claim, a committee of n bonded nodes would be sampled deterministically from chain state,

$$\mathcal{C}(\mathsf{cl}) = \mathrm{Top}_n\big(i \mapsto H(\mathsf{subject} \,\|\, \mathsf{prevHash}(h) \,\|\, i)\big),$$

reusing the sortition already implemented for PoN slot eligibility in `fluxd` — deterministic, recomputable by any observer from chain state alone, and requiring no separate coordinator. The index i here must range over *owner identities* in the sense of Definition 23.6, not over nodes. Sampling over nodes would give the largest operator — 480 of 6,489 nodes at the measured distribution (§23.5) — about 7.4 % of every committee's seats in expectation, and a far from negligible probability of holding the $n - k + 1$ seats that capture one. Sampling over identities, or capping seats at one per identity, costs nothing and removes that. A claim would be *accepted* once k of the sampled n nodes sign an identical value, *disputed* if the committee splits, and *challengeable* by any party for a window Δ_{chal} after acceptance. A challenger would post a bond β_{chal} alongside an assertion of the correct value; on a resolved challenge against the original signers, each would be slashed by a fraction s of β_i, and the challenger would be paid from the slashed amount. A signer's bond would not be released while any claim it signed remains inside its challenge window. [intent: mechanism specification]

Every slashing step above presumes something this network has decided against. A bond held as a plain UTXO cannot be taken from its owner by consensus, and Flux's inherited Bitcoin script cannot express a forfeiture condition. That much is a limitation. The decision on top of it is not: Definition 22.16 records that **there is no slashing anywhere in Flux — no mechanism destroys or forfeits a holder's** FLUX [intent: design intake]. Definition 14.4 is therefore written above in the form the mechanism-design literature would recognise, and it is *not* the form this network would deploy. Both halves have to be said, in that order, because a reader who sees only the construction will assume a forfeiture path exists.

What would have to change, stated once so that the cost of reversing the decision is visible rather than vague: the bond would have to be a registered output of the FluxNode family rather than a plain UTXO, held under a consensus lock; forfeiture would be consensus *refusing the spend permanently*, which is a burn and is the only forfeiture primitive expressible without an adjudicating quorum; and the challenger's reward would have to be paid from the coinbase, which §7 proposes as the route for value that consensus decides to move. That is a new transaction type, new chain-derived state and a network upgrade — the same class of change as PNR itself. It is not adopted here.

Without it, the honest name for the tier is the one Definition 22.16 gives it: **registered, not bonded** — attributable but not slashable. The deterrent becomes exclusion from future participation and personal accountability by a named attester set, not loss of principal, and every quantitative statement below that multiplies a slash fraction by a bond should be read as conditional on a decision that has been taken the other way. The paper does not present accountability and forfeiture as equivalent.

Open Problem 14.5 (Which claims a bond can actually secure). The mechanism of Construction 14.4 would be sound exactly over claims that are *objectively re-checkable*: an image digest, or a foreign-chain reserve balance at a stated height, can be independently re-evaluated by any node, so a challenge would have an objective resolution and slashing would be safe. "A computation the node ran," stated as a general claim class, is not re-checkable in general — inputs may be gone, the computation may be non-deterministic, or re-execution may itself be disputed — and slashing on a dispute with no objective resolution would be a griefing vector against honest signers rather than a deterrent against dishonest ones. The honest scope for a launch would be to restrict admissible claims to the re-checkable class and leave general computation-integrity claims open until a resolution procedure exists; this paper does not have one and does not invent one. [intent: mechanism specification] Reserve attestation for the bridge (§22) — the roadmap's own

first customer for this mechanism — would fall inside the re-checkable class, which is one reason to launch there first.

Decided. Re-checkable claims only at launch. A claim that cannot be re-evaluated has no resolution procedure — the dispute branch of Figure 14 terminates in nothing — so admitting one would mean accepting a bond that can never be adjudicated. Non-re-checkable classes can be added once a resolution procedure exists for them; launching without that procedure would put the mechanism's credibility on claims it cannot settle. [inferred].

A further constraint would be quantitative rather than procedural. For a reserve-attestation claim, the gain from a successful false claim could be the entire balance being attested to, while the cost of lying would be bounded by $s \cdot \beta_i$ times the probability of detection. **A fixed bond cannot secure an unbounded reserve**: β_i would need to scale with the value being secured by the claims a node is sampled to sign, not with the node's FluxNode collateral tier, or the mechanism would be profitable to defeat exactly where it matters most — the largest reserves.

Decided. β_i is a function of value secured, not of tier: $\beta_i = \max(\beta_{\min}, \chi \cdot V_i)$ for value secured V_i, with χ the coverage ratio and $\beta_{\min}$ a floor that keeps small claims worth adjudicating. Tier-based sizing is rejected because tier measures collateral, not exposure — a Cumulus node attesting a high-value claim would post less than a Stratus node attesting a trivial one, which inverts the incentive. The coefficients χ and $\beta_{\min}$ are calibration and are set with (k, n) below. [inferred].

Decided. $n = 9$, $k = 6$, $\Delta_{\text{chal}} = 2{,}880$ blocks (24 h), and — conditional on the forfeiture decision above being revisited, which it has not been — slash fraction $s = 1$. Reasoning: $n = 9$ keeps per-claim bandwidth and latency modest while making a colluding majority expensive; $k/n = 2/3$ is the standard Byzantine threshold and the smallest that tolerates $\lfloor (n-1)/3 \rfloor = 2$ faults; Δ_{chal} of one day gives a challenger a full business cycle to notice and act, and is the term $M \geq \Delta_{\text{chal}}$ in §8 constrains. Full slashing rather than partial: a partial slash prices dishonesty rather than deterring it, and the bond is already sized to the value secured. [inferred].

One seed of this mechanism already exists in the deployed system, IN-PROGRESS. The `purpose=mesh` option on `/v2/attest` is described in `fluxsas`'s own source as "the mesh membership voucher," with FluxOS supplying `(meshCa, appName, outpoint, recent block hash)` as the payload for peers to verify against a mesh-purpose key [fluxsas/src/api_server.h:438–445] — a node-to-node vouching primitive that is wired into the API today with no caller anywhere in the read repositories that actually builds or checks these vouchers. (The FluxOS development fork's `wip/ingress-attestation` branch is not a second seed: despite the name, the service it introduces attests the observed network source of an application specification submission, sealed to a team key and gossiped between nodes, and references `fluxsas` nowhere [MorningLightMountain713/flux@feat/fluxos-v9:ZelBack/src/services/appMessaging/ingressAttestationService.js:1-40].) It is the concrete starting point a bonded network would extend, not evidence that one exists.

14.4 Figures

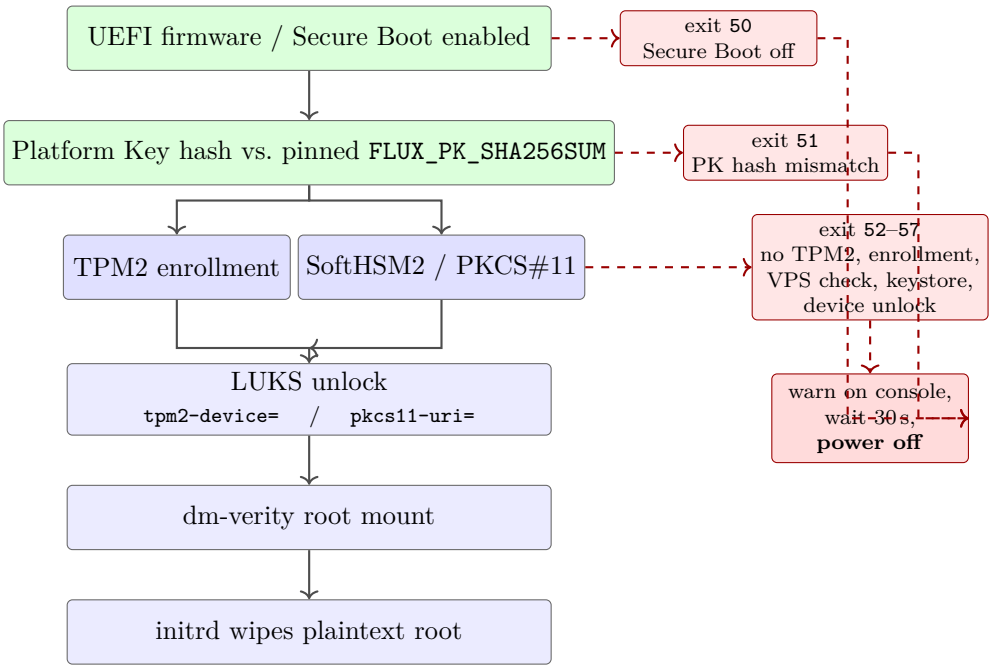


Figure 13: The ArcaneOS boot chain from firmware to a running, verity-protected root filesystem.

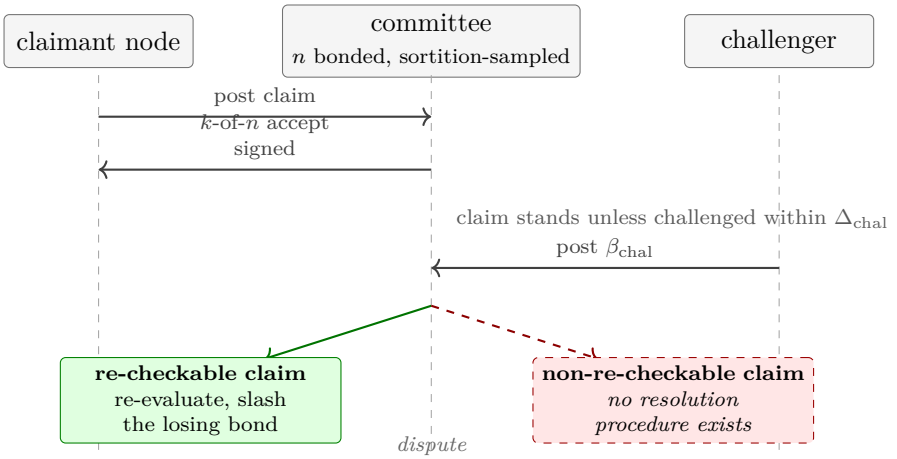


Figure 14: The bonded-network challenge protocol, showing where the mechanism's soundness depends on the claim class.

PART IV

Agents and value

15 Accelerated compute: GPUs and the AI workload path

An accelerator is not a fungible resource the way a CPU core or a gigabyte of RAM is: a GPU is either present on a physical host or it is not, and whatever isolates one tenant's use of it from another's is either enforced by the device driver and the orchestrator or it is not enforced at all. This section asks two narrow questions about what Flux actually ships today. First, what does "pluggable GPU" mean in code — is a GPU attached to a workload as a whole device, a slice of a device, or something attachable and detachable at runtime? Second, what is the isolation boundary between two tenants placed on the same machine? The answers are more restrictive than the roadmap's language suggests, and one of the mechanisms below has a correctness defect worth stating plainly rather than filing as a performance footnote. The evidence surveyed spans four repositories: `fluxai` and `fluxai-enterprise` (an inference-as-a-service product), `pouwbackend` (the FluxEdge compute-rental marketplace), and `fluxai-fluxedge` (the GitOps glue between the first two).

15.1 GPU allocation as implemented

On the FluxEdge marketplace, a rented workload's container requests a GPU the same way any Kubernetes workload does: a resource quantity under the vendor device-plugin key `nvidia.com/gpu` in the pod's `resources.limits/requests` SHIPPED [pouwbackend/rancher/deployment.go:2155-2163]. `setResourceLimit` reconciles the requested GPU *count* against the rented computer's actual GPU inventory, walking a bus-indexed enumeration of the machine's physical devices (`newGPUIndexer`, NVIDIA-relative indexing) and appending each chosen bus index to the set of GPUs already consumed by earlier containers of the same deployment SHIPPED [pouwbackend/rancher/deployment.go:914-982], [pouwbackend/rancher/deployment.go:1245-1270]. Isolation between deployments sharing a rented computer — which on FluxEdge belong to one renter, since a computer is rented exclusively [pouwbackend/models/market_place.sql.go:1182-1190] — is then a single mechanism: the chosen bus indices are written into the container's `NVIDIA_VISIBLE_DEVICES` environment variable SHIPPED [pouwbackend/rancher/deployment.go:967-971]. There is no further Flux-specific sandboxing layered on top of this; the isolation guarantee is whatever the Kubernetes NVIDIA device plugin and the container runtime provide for an environment-variable-scoped device list.

No time-slicing, no NVIDIA Multi-Instance GPU (MIG) partitioning, and no virtual-GPU (vGPU) mechanism exists anywhere in the four repositories surveyed SHIPPED [pouwbackend/rancher/deployment.go:914-982]. This is a coherent design choice, not an oversight: whole-device passthrough with environment-variable isolation is simple to reason about and gives strong isolation — a device cannot be double-assigned across pods on the same node, because the device plugin never advertises more `nvidia.com/gpu` resources than there are physical devices. The consequence is equally direct: the granularity of what can be sold is fixed at one physical GPU. FluxEdge cannot today offer a fractional share of a GPU to two tenants concurrently; the only way to approximate a smaller unit of accelerated compute is to dedicate an entire device to one renter and price accordingly.

15.2 The contention defect: a resource request that disappears

If none of a container's requested GPUs can be assigned — the computer reports no NVIDIA device, or every device is already taken by an earlier container of the same deployment — `setResourceLimit` does not reject the request and does not queue it for a computer with sufficient capacity. It deletes the `nvidia.com/gpu` key from both `Limits` and `Requests` on the container spec entirely SHIPPED [pouwbackend/rancher/deployment.go:972-975]. The workload is then scheduled and runs CPU-only, with no accelerator, and with no error surfaced to the caller from this code path; the only trace is a log line from a failed resource-consumption write, which is itself only logged rather than propagated SHIPPED [pouwbackend/rancher/deployment.go:978-980]. If only some of

the requested GPUs can be assigned, the container receives that subset in `NVIDIA_VISIBLE_DEVICES` while its declared limit is left unchanged, again with no error SHIPPED [pouwbackend/rancher/deployment.go:950-968].

This is a correctness defect, not a performance characteristic. A caller who pays for and requests a GPU-bearing deployment can receive a deployment silently missing the one resource that made the request meaningful, and nothing in the API response distinguishes that outcome from a fully provisioned rental. The correct behaviour, on contention, is to reject the rental request outright or to queue it until a matching computer becomes available — either choice preserves the invariant that a caller can trust an accepted request to mean what it says. Silently stripping the resource line instead breaks that invariant in a way a paper describing this system should not pass over.

15.3 "Pluggable" or hot-attachable GPU: no code, no exception

The roadmap's language of a "pluggable" or hot-attachable GPU has no counterpart in any of the four repositories surveyed [doc: reconciliation record]. An exhaustive search across all four repositories for hot-attach, hot-plug, hot-swap, and pluggable terminology returns exactly one hit outside vendor code, and it is unrelated: a `HotSwap` boolean field on OVH's public cloud catalog API describing disk RAID hot-swap on dedicated servers, not GPU attachment [pouwbackend/premium/ovh/ovh.go:320]. There is no code anywhere in `fluxai`, `fluxai-enterprise`, `fluxai-fluxedge`, or `pouwbackend` that attaches a GPU to an already-running container, or that reallocates a GPU between tenants without a pod restart or reschedule. This is tagged VISION: it names a capability the roadmap gestures toward that does not exist in code today, and the paper does not speculate about what a future implementation might look like beyond that.

15.4 Two product surfaces, easy to conflate

The corpus contains two separate GPU-bearing product surfaces, and describing either as if it were the other would misstate the system.

`fluxai` and `fluxai-enterprise` are an inference-as-a-service product — chat completions, image generation, document intelligence — backed by a fixed, admin-provisioned pool of GPU servers internally called "RGP" (Remote GPU). An administrator creates an RGP record, selects a model, and triggers an asynchronous Ansible playbook that connects over SSH and stands up the inference stack on the target rig SHIPPED [fluxai/fluxai_admin/pages/views_rgp.py:36,468]. There is no self-service provisioning path for an external caller in this product; capacity changes only through this admin workflow, and `fluxai` is itself an ordinary tenant (customer) of FluxEdge and of other GPU providers such as Vast.ai, not an application on FluxCloud or the FluxNode network.

`pouwbackend` is the backend of FluxEdge, a separate, Rancher/Kubernetes-orchestrated compute-rental marketplace with its own OpenAPI-documented REST interface and its own CLI SHIPPED [pouwbackend/rest_api/v1/openapi.yaml:1-4]. This is the mechanism described in §15.1–§15.2.

`fluxai-fluxedge` is the GitOps glue between the two: a repository of Kubernetes manifests and Argo `Workflow` templates that clone the repository, build a `kustomize` overlay for a given model, and `kubectl apply` it onto a node selected at submission time by FluxEdge SHIPPED [fluxai-fluxedge/custom-workflow-fluxone.yaml:7-9,21-52]. In other words, `fluxai`'s own inference pods are provisioned as ordinary tenant workloads on FluxEdge, as they are on Vast.ai and other providers — a customer–provider relationship, not a coupling of the two products, and one that involves neither FluxCloud nor the FluxNode network. A reader should not assume that describing one product describes the other.

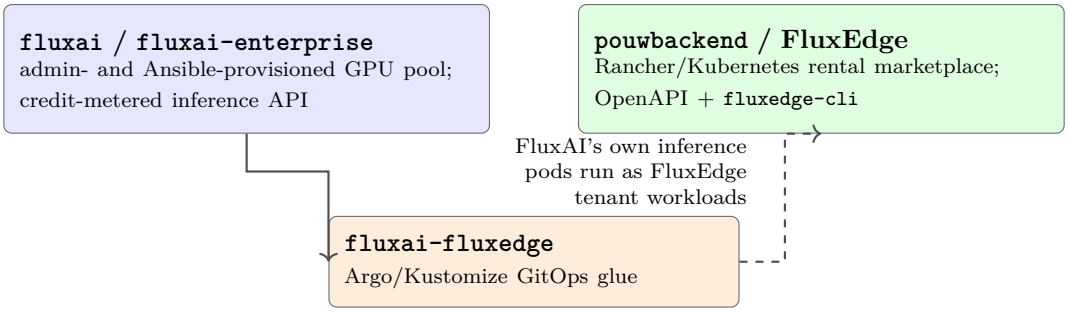


Figure 15: The two GPU-bearing product surfaces and the tenant relationship between them.

15.5 A module name that is not a mechanism

`pouwbackend` is a mature compute-rental marketplace, and the paper describes it as exactly that. Despite the name, "Proof of Useful Work" is not a designed consensus or verification mechanism anywhere in this code. The only in-code hits for the term are a GPU mining-overclock preset applied to a rig when its rental ends SHIPPED [pouwbackend/grpc_server/market_place/computer.go:502,517], and a Postgres schema documentation file that self-describes the database as the "flux Prove Of Useful Work Database (POUW)" — branding, not a mechanism SHIPPED [pouwbackend/sql/.tbls.yml:3]. There is no submit-work, verify-work, or reward-work RPC triple; no on-chain anchoring of compute results; and no linkage of any kind to `fluxd` consensus. "PoUW" survives in this codebase only as a Go module path and a database nickname. The paper does not describe Flux as having a proof-of-useful-work mechanism anywhere, and this section is where that is stated once, plainly, for the record.

15.6 The machine-facing surface: scoped tokens, forward to §18

The FluxEdge REST interface — OpenAPI 3.1.1, base path `/api/v1` — and its accompanying CLI, `fluxedge-cli`, are the machine-facing surface this paper's agent-provisioning discussion in §18 depends on, not the fixed-credit inference API described above SHIPPED [pouwbackend/rest_api/v1/openapi.yaml:1-4]. Authentication is by bearer token, and a key's privileges are enumerated rather than all-or-nothing: `start_rental`, `stop_rental`, `start_deployment`, `stop_deployment`, and `get_balance` SHIPPED [pouwbackend/rest_api/v1/openapi.yaml:2576-2581], enforced in code by mapping each rental and deployment endpoint path to its required privilege string — the balance privilege is documented but has no entry in that map SHIPPED [pouwbackend/rest_api/ratelimit.go:375-390]. Requests are rate-limited to two per second per key and two per second per source IP SHIPPED [pouwbackend/rest_api/ratelimit.go:41-42]; a key or IP that accumulates five rate-limit violations within a five-minute window is automatically revoked, its owner notified by email, and its cache entry evicted SHIPPED [pouwbackend/rest_api/ratelimit.go:43-45,203-222]. This is a bounded, revocable *action* authority — a human operator can mint a key that may start rentals but not stop them, for example — suitable for handing to an autonomous agent with a defined blast radius. No MCP server and no Terraform provider exist in any of these four repositories; the MCP tools that do exist in the Flux stack live elsewhere and are outside this section's scope, taken up in §18.

15.7 Metering and the overdraft window, forward to §16

`fluxai`'s credit metering is asynchronous and eventually consistent rather than synchronous. Each request is gated on a positive credit balance at call time, but usage is not deducted immediately: token consumption accumulates in a Redis cache keyed per user with a 300-second expiry SHIPPED [fluxai/api/cache.py:9-32], and a separate scheduled command later scans the accumulated keys, applies the deduction to the persisted balance inside a locked transaction, and clears the cache SHIPPED [fluxai/api/management/commands/process_token_usage.py:41-99]. The 300-second cache window is a real overdraft bound: a caller can spend up to that much usage

before the persisted balance reflects it, and the periodic flush job that closes the window is itself scheduled through a live database-configured dispatcher rather than a fixed interval fixed in source SHIPPED [fluxai/setup/crons/django_cron:2]. Any future product that prices consumption rather than a subscription tier inherits this same window unless it is metered differently; §16's discussion of billing accuracy should carry this bound forward rather than assume synchronous debiting.

15.8 A gap for agent autonomy: key issuance

Nothing in §15.6 solves the problem of an agent obtaining its own credential in the first place. API-key issuance is not exposed anywhere in the documented REST v1 surface — only consumption is. The API's own quick-start documentation points a caller to the web dashboard to obtain a key SHIPPED [pouwbackend/rest_api/v1/openapi.yaml:61]. A human must mint a scoped key through that dashboard before an agent can use the FluxEdge API autonomously; there is no REST or CLI self-service path for an agent to request or rotate its own credential. This gap is exactly the boundary §17 and §18 need to address for a fully autonomous provisioning workflow.

15.9 Natural-language deployment: `DeploymentYAML`, forward to §19

`pouwbackend` exposes a gRPC method, `DeploymentYAML`, that generates Kubernetes YAML from a natural-language prompt SHIPPED [pouwbackend/grpc_server/market_place/deployment.go:1784-1812]. The generation pipeline first performs a vector search over existing marketplace application YAMLs to build retrieval-augmented context SHIPPED [pouwbackend/ai/fluxai.go:188-191,241-244], then dispatches a chat-completion request against a pinned model ("Llama 3.1 8B") or, depending on configuration, a third-party provider SHIPPED [pouwbackend/ai/fluxai.go:212,266]. This is the strongest existing evidence in the surveyed corpus for agent-native provisioning: a natural-language deployment path is not a hypothetical for a future release, a form of it already ships. It is also, for that reason, the concrete artifact §19 should hold up when it takes up the question of whether a generated specification is reviewable before it is signed and run.

15.10 Resource accounting on the FluxOS side

On the orchestration side, FluxOS's per-tier resource allowances and its per-application hardware check both express only the resource vector $\mathbf{r} = (\text{cpu}, \text{ram}, \text{hdd})$: the Cumulus/Nimbus/Stratus tier table gives total and application-available CPU (in tenths of a core), RAM, and HDD per tier SHIPPED [flux/ZelBack/config/default.js:380-403], and the per-application admission check (`checkAppHWRequirements`) validates a candidate application against exactly those three dimensions SHIPPED [flux/ZelBack/src/services/appRequirements/hwRequirements.js:370-408]. No GPU resource class appears anywhere in this accounting. The v9 application specification (§10) adds a `machineType` enum with values `standard`, `gpu`, and `vps`, which is the field the specification intends to eventually carry a GPU-bearing workload declaration IN-PROGRESS [flux@feat/appspec-v9:ZelBack/src/services/appRequirements/schemas/v9.json]. As §10 establishes, this field currently has no consumer code anywhere in FluxOS [doc: reconciliation record]: it is declared in the schema and gated behind a feature flag that nothing in the codebase flips, not wired to any placement, pricing, or admission logic.

15.11 The workload class the substrate fits

The preceding subsections describe what the accelerated-compute path can and cannot do. This one makes an architectural claim about *which kind of inference workload the network is structurally suited to*, because the answer is not the one a reader primed by the frontier-model discourse would guess, and because it bears directly on the thesis in §1.

Why frontier inference centralises, and why that reason does not generalise. Serving a frontier model is not a single-device workload. Weights that exceed one accelerator's memory are split across many, by tensor or pipeline parallelism, and every token generated requires synchronous exchange between those devices. The interconnect carrying that exchange — NVLink within a chassis, InfiniBand across a rack — is one to two orders of magnitude faster than commodity wide-area networking, and the workload is latency-bound on it. This is why frontier inference concentrates in single datacentres: it is a topology constraint, not a commercial preference, and no amount of geographic distribution can be traded against it.

Property 15.1 (The interconnect requirement is workload-specific)**.** [inferred] A model whose weights fit on one accelerator, or in host memory, requires no inter-device communication during inference. Requests to such a model are mutually independent: there is no synchronisation between concurrent users, and no state shared across nodes between requests. Consequently the parallel efficiency of serving n concurrent users across n dispersed single-device nodes is, to first order, the same as serving them from n co-located devices.

The consequence is the section's substantive point. **For the workload class that fits on one device, the structural advantage of centralisation disappears**, because the property that creates it — the interconnect — is not exercised. A geographically dispersed, heterogeneous, commodity fleet is not a compromised version of a datacentre for this class of work; it is an appropriate substrate for it, and it brings two properties a datacentre does not: physical proximity to the user, and — on FluxEdge, where a computer is rented to one renter exclusively — per-tenant isolation that is a whole machine rather than a scheduler abstraction (§15.1).

What the network measurably has for it. Small quantised models are frequently served acceptably on CPU and host memory alone, with no accelerator involved. The fleet's committed capacity is 8,972 vCPU and 17,133.0 GiB of RAM across 6,489 nodes [measured: api.runonflux.io/apps/globalappsspecifications and /daemon/getzelnodecount, 2026-09-03], and that is capacity already committed to applications rather than the fleet's ceiling. This is the part of the argument that needs no new engineering at all: the CPU-served portion of this workload class is deployable on the substrate as it stands today, through the ordinary application path of §9.

The tension, which is the honest half. FluxEdge's GPU allocation is whole-device passthrough (FluxNodes have no GPU allocation at all, §15.10), with no MIG partitioning, no vGPU and no time-slicing anywhere in the surveyed code (§15.1). **That is precisely the wrong granularity for the workload this subsection argues the network suits.** A small quantised model may need a fraction of a modern accelerator's memory and a fraction of its throughput; under whole-device allocation it nonetheless occupies the entire device, and the economics of serving many small models — which depend on packing many of them per accelerator — do not survive that. The contention behaviour compounds it: a request that cannot be satisfied has its accelerator requirement silently stripped rather than queued or refused (§15.2).

So the correct reading is not that the substrate is ready for this workload and merely lacks demand. It is that **the substrate's shape fits the workload and its accelerator allocation model does not**, and the allocation model is the thing that must change. That reframes the finding in §15.1 from a defect report into a prerequisite: fractional allocation is not a refinement of GPU support, it is the feature on which this workload class depends. §25 classes it as Engineering, not research — MIG and time-slicing are vendor-supported and widely deployed elsewhere — but it is unbuilt here, and nothing in the surveyed code anticipates it.

How this composes with the rest of the paper. Three mechanisms specified elsewhere are, taken together, what would distinguish this from renting a virtual machine somewhere

cheap. Payment without an account (§16) lets a caller buy inference per deployment or per period with no relationship to maintain, which is the property an autonomous consumer needs. Attested execution (§14), once the attestation key is bound to a machine, would let a caller verify *which model image* answered — an integrity property no inference API offers today, and one that matters more for a small specialised model than for a frontier one, because the caller cannot easily tell from the output whether they got the model they paid for. And the delegated-authority certificate of §17 is what would let one principal operate many narrow agents under a bounded budget.

Remark 15.2 (What this argument does and does not assert). It is a claim about architectural fit, not about demand. This paper takes no position on what fraction of inference will be served by small task-specialised models rather than frontier ones; that is the same bet the document declares in §1 and does not defend. What is argued here is narrower and checkable: *if* that workload class matters, the interconnect requirement that centralises frontier inference does not apply to it, and a dispersed fleet is a structurally appropriate substrate — once accelerators can be subdivided. No claim is made about price, capacity relative to any other provider, or market size.

15.12 What can be honestly sold today

FluxEdge can sell, today, a whole physical GPU rented for the duration of a deployment, provisioned through a documented API and a CLI, gated by a bearer token whose action privileges are individually scoped and automatically revoked on abuse. `fluxai` can sell fixed-catalog inference against a credit balance, backed by a pool of GPU servers an administrator provisions by hand. What cannot honestly be sold on this substrate today is a fractional or shared slice of a single GPU across tenants — the isolation mechanism does not support it — nor a guarantee that a GPU-bearing rental request either succeeds with the accelerator it asked for or fails cleanly, since contention currently resolves by silently dropping the resource request rather than by rejecting or queuing it. Nor can the network sell a hot-attachable or pluggable GPU, self-service agent key issuance, or compute verified by any proof-of-useful-work mechanism: none of these exist in the surveyed code, and the paper does not describe Flux as though they did.

15.13 Inference without a GPU: what the fleet can serve today

The subsection above is about what cannot be sold. This one is about a capability the network already has and does not advertise: **small-model inference on CPU**, at rates that are useful rather than merely demonstrable.

Autoregressive decoding is memory-bandwidth-bound, not compute-bound: each token requires reading the active parameters once, so throughput is approximately achievable bandwidth divided by bytes read per token. On that model, and as the author's estimate rather than a measurement on Flux hardware [intent: design intake] (PLANNED):

Model	Active params	GB/token	Dual-channel ~30 GB/s	8-channel EPYC
Qwen3-4B Q4 (dense)	4B	~2.4	~12 tok/s	~40 tok/s
gpt-oss-20b (MXFP4)	3.6B	~1.9	~15 tok/s	~50 tok/s
Qwen3-30B-A3B Q4	3.3B	~1.9	~15 tok/s	~50 tok/s
Qwen3-8B Q4 (dense)	8B	~4.8	~6 tok/s	~20 tok/s

The distinction that matters for this fleet is between *active* and *total* parameters. Decode speed follows the active count; residency follows the total. A mixture-of-experts model is therefore hungry for RAM and frugal with bandwidth — Qwen3-30B-A3B reads 3.3B parameters per token but must hold all 30B — and RAM is exactly what the tiers of §3 provide. The architecture the

industry moved toward for its own reasons happens to suit a fleet of many mid-sized machines better than it suits a few large ones.

Against the tenant-available RAM measured in §3, the network could hold, as an upper bound at full utilisation: ∼63,300 **concurrent Qwen3-4B,** ∼30,800 **Qwen3-8B,** ∼11,400 **gpt-oss-20b, or** ∼6,500 **Qwen3-30B-A3B instances** [doc: scripts/23-network-capacity.py]. Those are capacity figures and not availability: roughly 5 % of fleet application-usable CPU is already committed to the workloads of §15.14, and no claim is made here about demand.

Two honest qualifications. These throughputs are derived, not measured — no benchmark of a quantised model on a Cumulus, Nimbus or Stratus node was run for this paper, and the 8-channel column describes server hardware that the tier minima do not require any node to have, so it is the ceiling of the range rather than its centre. And nothing in the deployed stack schedules for memory bandwidth: placement reads CPU, RAM and disk (§9.3), so a tenant cannot today request the property that actually determines inference speed.

Not established by this survey. measured token rates for quantised models on each tier's reference hardware. The figures above are a bandwidth model, and the gap between that model and a real node — memory configuration, NUMA topology, and what else is resident — is precisely what a benchmark would close. **That benchmark is scheduled for Q4 2026** [intent: design intake], so this is a dated gap rather than an indefinite one: the estimate above is to be replaced by a measurement, not defended.

Since publication: CPU inference in production. After this paper was published, the team began serving language models from the collateralised fleet itself. `llm.runonflux.com` is an OpenAI-compatible endpoint, gated by API keys, that routes each request to pools of ordinary FluxCloud applications running Ollama (llama.cpp) on FluxNodes, and no pool's specification requests a GPU; on 2026-10-25 it listed nine models served by six pools of 140 instances SHIPPED [measured: llm.runonflux.com, 2026-10-25] [ownllm/tools/gen.js:200–221]. One of them is the team's own model, `fluxai:tiny`: a LoRA fine-tune of IBM's Granite 4.0-H-Tiny, a hybrid Mamba-2 mixture-of-experts model, trained on deployment dialogues and grounded documentation questions and served as a 4-bit GGUF from 40 instances of 12 vCPU and 24 GB each [ownllm/finetune/README.md:3–44,117–157]. Its pool configuration records about 18 tokens per second of generation per instance, with prompt processing, not generation, the cost that grows with context [ownllm/tools/gen.js:202–205]. That is one model on one machine configuration, not the per-tier benchmark scheduled above, but it is measured rather than derived. The same endpoint serves the documentation assistant (retrieval over the Flux documentation, answered by `fluxai:tiny`, on 20 instances behind a 5-instance router), embedded as the "Ask Flux AI" widget on Flux websites, and the FluxCloud deployment assistant of §19. Despite the shared name, `fluxai:tiny` is not part of the `fluxai` GPU inference product described in §15.4: it runs on FluxNodes, not on FluxEdge, and only its training used a rented data-centre GPU.

Where this is going. FluxCloud is to become the one place from which both are available: first as a single interface over the two separate systems, with GPU rentals still served by FluxEdge's centralized backend, and only later as a merge of the systems themselves, whose design is not yet worked out; the long-term plan is one decentralized network running all kinds of applications [intent: design intake] (PLANNED). Until then the division is worth stating plainly, because it is a strength rather than an embarrassment: FluxEdge rents whole GPUs for work that needs them, and FluxCloud runs the model classes that do not — which, as the table above shows, now includes models that were frontier-adjacent two years ago.

15.14 Scientific simulation: the workload class already here

The largest single workload on the network by requested CPU is not an AI product. It is Monte Carlo science. The 2026-09-03 application census carries **27 folding and Rosetta deployments across** 2,409 **requested instances and** 2,408 **vCPU-equivalents** — roughly 5 % of the fleet's application-usable vCPU capacity of §3 — of which 26 are owned by a single team-held identity (`196GJWy...`, §21) and one is third-party SHIPPED [measured: api.runonflux.io/apps/globalappsspecifications, 2026-09-03]. Whatever else this network is, it is already a Monte Carlo cluster.

That is not an accident of configuration. Independent-seed Monte Carlo is the workload a network of this shape fits best, and the fit is structural rather than fortunate: runs are embarrassingly parallel, so no inter-node bandwidth or latency guarantee is needed and §2's eventually-consistent orchestration layer is sufficient; work partitions arbitrarily, so the unit of scheduling can be made to match whatever a tier offers; and each run checkpoints naturally, so eviction — which on this network is ordinary, not exceptional — costs a seed rather than a campaign. Particle transport for dose distribution, radiation-protection shielding, protein folding and molecular dynamics all sit in that class. The team intends to pursue it, with radiation-therapy research and facilities such as ELI Beamlines named as targets [intent: design intake] (PLANNED).

Three constraints govern whether that can be honestly sold, and the first of them rules out the code most often named for the purpose.

Licensing decides how a code may be shipped, not whether the network may run it. The constraint here is narrower than it first appears, and getting it wrong in either direction is costly. FLUKA's Single User Licence forbids the licensee to "make Available the Software to, or allow the Use of the Software by, any other person or entity" (§5(a)), forbids transfer or sublicence (§5(e)), and forbids making the software available at all (§2) [doc: fluka.cern/download/licences/fluka-single-user-licence, retrieved 2026-09-06]. Read at its broadest that would forbid execution on any machine the licensee does not own — but that reading proves too much, because it would equally forbid running FLUKA on rented cloud capacity, which is routine. The licence's own shared-installation rule shows where the line actually sits: a central installation on a university cluster needs no Institutional Licence provided it uses the binary libraries and *every user of it* registers and accepts the Single User Licence individually [doc: fluka.cern/download/registration, retrieved 2026-09-06]. That rule binds *users*. It does not require the cluster's system administrators to register, though they hold root and can read the binaries. Incidental technical access by whoever operates the hardware is not what §5(a) addresses; supplying the software to someone as a user is.

The distinction that governs Flux, therefore, is not *whose machine executes* but *who obtains and installs the code*, and it falls cleanly on the container boundary:

- **Tenant-supplied, and defensible.** A registered licensee rents capacity and runs FLUKA in their own deployment, with the binaries fetched at run time under their own credentials or mounted from storage they control. They are the user; the node operator is an infrastructure provider in the same position as a cloud host or a university sysadmin. The network never possesses a copy.

- **Platform-supplied, and not defensible.** Flux publishes, or a tenant pushes to a public registry, an image with FLUKA baked into a layer. That image is then available to every operator that pulls it and to anyone who pulls it from the registry, none of whom is a registered user. This is redistribution under §5(e) and making-available under §2, and no reading of the cluster rule reaches it.

The practical consequence is a product constraint rather than a technical one: Flux can host FLUKA workloads but cannot ship a FLUKA image, and a catalogue entry offering "FLUKA on Flux" would be the thing that breaches, not the computation.

Not established by this survey. whether FLUKA's Commercial Licence — the fourth of its four licence types, obtained by contacting the collaboration and whose terms are not published [doc: fluka.cern/download/licences, retrieved 2026-09-06] — would permit a platform-supplied image, which is what a catalogue offering would need. The tenant-supplied route above does not depend on that answer.

There is one Flux-specific wrinkle a cloud tenant does not face, and it is this paper's own finding rather than a licensing question: two FluxIDs held personally by the co-founders (K4, Definition 2.3) carry the `fluxteam` privilege on every node's FluxOS API (TB13, Table 1), which confers rights including exec into a running container. A licensee should know that before treating a Flux deployment as equivalent to a rented VM.

Codes with no such constraint. **Geant4** — under the EU DataGrid Licence, which permits use, modification and binary redistribution with attribution — and **OpenMC**, under MIT, can be shipped in a public image with no licensing question at all [doc: geant4.web.cern.ch/download/license; docs.openmc.org, retrieved 2026-09-06]. MCNP and MCNPX add US export control and are a separate matter. For a catalogue product, Geant4 and OpenMC are the codes to build on; ELI Beamlines runs Geant4 alongside its FLUKA and MCNPX work [doc: eli-beams.eu/facility/computing-simulations/simulation-codes, retrieved 2026-09-06], so the overlap is real.

Verification costs two to three times the compute, and this fleet makes it harder. A result returned by an operator this network does not know is not evidence of anything. The precedent is explicit: volunteer-computing platforms treat returned results as untrusted because hardware error, miscompiled builds and deliberate fabrication all occur in practice, and the established remedy is replication to a quorum — dispatch each job to two or more hosts, accept only agreeing results, and re-replicate on disagreement [doc: boinc.berkeley.edu, BOINC platform papers]. That is a 2 to 3× multiplier on every priced core-hour, and it must appear in any comparison against a commercial cloud rather than being netted out of one.

Flux has the harder form of the problem. BOINC resolves the floating-point question with *homogeneous redundancy*, sending replicas only to numerically identical machines, because different hardware does not produce identical floating-point output for the same input. This fleet is deliberately heterogeneous — that is what §4's tiering and the measured hardware distribution describe — so bitwise comparison across arbitrary nodes will not hold. Either the image pins a reproducible numeric environment and placement is constrained to matching hardware, or agreement is defined by a stated tolerance and the science is reported with that tolerance attached.

Not established by this survey. which of those two routes Flux should take, and what tolerance a dose-distribution or shielding result can accept before the replication quorum stops being evidence. Neither question is answered by any artefact read for this paper, and both are prerequisites — not follow-ons — to selling verified simulation. §14's attestation network is the mechanism that would let a node's identity carry weight in that quorum rather than every replica costing full price.

The first workload is already standardised, and it needs no confidentiality at all. The concrete form this takes is not a research programme but an existing example. Geant4 ships the ICRP110 adult reference voxel phantoms — the international standard male and female computational bodies — as `ICRP110_HumanPhantoms`, an Advanced Example that scores dose per voxel and per organ, and is used for radiation protection, internal dosimetry and radiotherapy dosimetry alike [doc: geant4.web.cern.ch/docs/advanced_examples_doc/example_ICRP110Phantom, retrieved

2026-09-06]. A dose-distribution campaign over those phantoms is a container carrying Geant4 and a phantom definition, a seed, and a macro; the fleet runs N seeds; the results sum. That is the *same* shape as the folding deployments already running on the network, and it is what a "Folding@home for dose distribution" would concretely mean here [intent: design intake] (PLANNED).

What makes it the right first workload is not its physics but its threat model: a reference phantom is a published international standard, and the geometry, the beam and the scored dose are all publishable. There is nothing to keep secret, so the confidentiality problem that governs the rest of this subsection does not arise, and the only property the network must supply is that the returned numbers are the numbers the code produced — integrity, addressed by the replication quorum above and by §14, not by encryption.

Patient data is disqualified today, and phantom data is not. Treatment-planning dose calculation on real patient imaging cannot run on this network as it is built, and the reasons are this paper's own findings rather than a general caution: two FluxIDs held personally by the co-founders (K4, Definition 2.3) carry the `fluxteam` privilege on *every* node's FluxOS API (TB13, Table 1), `fluxstorage` holds oversized environment variables and commands unencrypted at rest with no write-side protection [fluxstorage/src/routes.js:17–67], and the policy documents every node fetches and enforces are unsigned and served over plain HTTPS. No data-protection regime governing clinical imaging is satisfiable against that trust boundary.

ArcaneOS's disk encryption does not close this, and it is worth being exact about why, because the intuition that an encrypted node image protects tenant data from its operator is the natural one to hold. Definition 14.2 establishes that the key protecting the random component of the node-identity value at rest is a static constant compiled into every `fluxsas` binary, and that the attestation keys derive from values compiled identically into every copy with no TPM, Secure Boot or per-machine input SHIPPED [private source: SAS disk encryption], [private source: SAS API server]. Encryption whose key every copy of the software contains defends against a stolen disk and a casual reader; it does not defend against the operator of the machine, and the operator is precisely the adversary a confidentiality claim over patient imaging has to exclude. The remedy is the attestation network of §14 plus a per-machine root of trust, not the disk encryption that exists today. Work on phantoms, synthetic patients and published reference datasets carries none of that exposure, produces the same physics, and is where a research offering should begin.

16 Paying for capacity: direct settlement, and a service built over it

Buying capacity on Flux is closer to buying a train ticket than to opening a utility account. A tenant computes what a deployment costs, sends that many FLUX to a known address in one transaction, and every node independently reads the chain and agrees that the deployment is funded until a particular block height. Nobody issues a credential, nobody keeps a ledger of what the tenant owes, and when the paid-for height arrives the entitlement simply stops. This section specifies that mechanism first, because it is the protocol's settlement path and the one autonomous software should use. It then describes something that is not part of the protocol at all: a centralized payment service, operated by InFlux Technologies on top of Flux [intent: design intake], which holds a person's keys, signs for them, and pays the chain on their behalf in exchange for a card payment. The chain does not know that service exists, and this section proves it: from consensus's point of view its payments are ordinary payments from ordinary keys, and nothing in `fluxd` or FluxOS is aware that a custodian is involved (Property 16.21). Last, and separately, it specifies — explicitly as unimplemented — the deposited-balance layer such a service would need in order to become a credit account. The three are kept apart throughout, because they belong to different systems, carry different maturities, and fail in different ways. The claim the section defends is narrow and falsifiable: **Flux's settlement path is trustless**

and ships today in two independent forms; InFlux, the company that maintains the Flux products, has built a centralized convenience layer over it for people who want to pay by card, and the protocol neither knows about that layer nor depends on it.

Symbols introduced here. Not in `notation.tex`, used only in this section: e_a the paid-through height of application a; $L^*(h)$ the effective default lifetime; $\iota(h)$ the active rate-table row; ϖ the unused fraction of a previous term; v^{obs} an observed on-chain payment. For the card-payment service of Section 16.2.1: $\mathcal{F}$ the operator's facilitator service; K_u the pair of private keys it holds in trust for user u and never discloses to u; $\mathcal{H}$ its set of operator hot wallets; T_{rec} the retention time of its notification logs; Δ_{adv} its early-renewal advance window in blocks; $\mathsf{A}_{\text{orb}}, \mathsf{A}_{\text{ren}}, \mathsf{A}_{\text{paid}}$ its three hardcoded allowlists. For the unimplemented credits layer of Section 16.2.2: b_u a tenant's deposited balance in sat; $\text{st}(a, h)$ the lifecycle state; $G_{\text{g}}, G_{\text{s}}$ the grace and suspension windows in blocks; Δ_{pre} the pre-authorisation lead in blocks; $\mathcal{P}^{\text{ren}}$ and $\mathcal{P}^{\text{res}}$ the renewal and resumption charges; Δ^{sub} the suspension subsidy. The second group is candidate notation for an object that does not exist and must not be absorbed into `notation.tex` unless it is built.

16.1 Part A — Direct settlement

Everything in this part is SHIPPED and is written in the present tense, and everything in it is a property of the network itself — what `fluxd` and FluxOS do, with no service standing in between. It describes two deployed instances of the same idea: the application-hosting path in FluxOS, and the CumulusVPN entitlement path. Both share one property — the payment is the entitlement — and neither maintains any forward-looking claim against the payer.

16.1.1 The price function and the rate schedule

Assumption 16.1 (Settlement environment)**.** As stated in the system model of Section 2: (i) every node holds the same confirmed chain prefix up to the reorg bound (`MAX_REORG_LENGTH` $= 40$ outside the PoN stabilisation window); (ii) every node runs FluxOS with the same compiled rate table, subject to on-chain `chainparams` price messages of version `p` posted under the network's administrative multisig, whose keys are held by the four co-founders, which take effect from their occurrence height [flux/ZelBack/src/services/utils/chainUtilities.js:9–52]; (iii) application payments are recognised by output address plus an `OP_RETURN` message carrying the temporary-message hash [flux/ZelBack/src/services/explorerService.js:309–345]. Assumption (ii) is a trust assumption on the holders of that multisig and is carried as such in Section 7 (the price table is the base; under the planned PNR, consensus would fix only the split).

Definition 16.2 (Height-scheduled rate table)**.** The rate table is a finite sequence of segments indexed by activation height, $\{(h_j, \mathbf{p}^{(j)})\}_{j=0}^{5}$ with $h_0 = -1$ (genesis) and $h_1 < \cdots < h_5$ as given in Table 39. Each row is $\mathbf{p}^{(j)} = (p_{\text{cpu}}, p_{\text{ram}}, p_{\text{hdd}}, p_{\text{min}}, p_{\text{port}}, p_{\text{scope}}, p_{\text{sip}})^{(j)} \in \mathbb{Q}_{>0}^{7}$, denominated in FLUX. The active row at height h is $\mathbf{p}(h) := \mathbf{p}^{(\iota(h))}$ with $\iota(h) := \max\{\, j : h_j < h \,\}$ (strict: the code selects rows with `height < height` [flux/ZelBack/src/services/utils/appUtilities.js:32]).

Table 39: The deployment rate schedule: six segments, genesis plus five revisions. Values as returned by the live endpoint and as compiled into FluxOS. [measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]; [flux/ZelBack/config/default.js:133–209]. SHIPPED

segment	height h_j	p_{cpu}	p_{ram}	p_{hdd}	$p_{\min}$	p_{port}	p_{scope}
genesis	−1	3	1	0.5	1	2	6
revision 1	983,000	0.3	0.1	0.05	0.1	2	6
revision 2	1,004,000	0.06	0.02	0.01	0.01	2	6
revision 3	1,288,000	0.15	0.05	0.02	0.01	2	6
revision 4	1,594,832	0.15	0.05	0.02	0.01	1.5	6
revision 5	1,597,156	0.03	0.01	0.004	0.01	0.4	0.8

Across the six segments the CPU rate falls from 3 to 0.03, a factor of 100; the RAM rate falls from 1 to 0.01 and the disk rate from 0.5 to 0.004. The static-IP surcharge p_{sip} is 3 in segments 0–4 and 0.4 in segment 5. A seventh segment is staged at height 2,950,000 with values identical to segment 5, so that SPEC v9 registrations price against a defined segment [flux@feat/appspec-v9:ZelBack/config/default.js:196–209]; it is not a revision and is not counted here (conflict C27). A parallel USD-denominated table and a `/apps/getappspecsusdprice` read endpoint also exist in configuration [flux/ZelBack/config/default.js:211–224]; which denomination governs is a Foundation policy lever, discussed as such in Section 7, and this paper makes no claim about it.

Definition 16.3 (Deployment price)**.** Let a be an application with component set $C(a)$ and per-component resources $r_c = (r_c^{\text{cpu}}, r_c^{\text{ram}}, r_c^{\text{hdd}}) \in \mathbb{Q}_{\geq 0} \times \mathbb{N} \times \mathbb{N}$, in vCPU, MB and GB respectively, and let $\mathbf{r}(a) := \sum_{c \in C(a)} r_c$. Let $k \in \{k_{\min}(h), \ldots, 100\}$ be the instance count, $d \in \{d_{\min}(h), \ldots, 1{,}056{,}000\}$ the duration in blocks, and $h \in \mathbb{N}$ the height at which the registration is priced. Write $\lceil x \rceil_2$ for rounding up to two decimals and $\mathcal{E}(a)$ for the set of enterprise ports the specification requests. The *base*, in FLUX per default lifetime, is

$$B(a,h) \;:=\; 10\, p_{\text{cpu}}\, \mathbf{r}^{\text{cpu}} \;+\; \tfrac{1}{100}\, p_{\text{ram}}\, \mathbf{r}^{\text{ram}} \;+\; p_{\text{hdd}}\, \mathbf{r}^{\text{hdd}} \\ +\; p_{\text{scope}}\, \mathbf{1}[\text{targeted or enterprise}] \;+\; p_{\text{sip}}\, \mathbf{1}[\text{static IP}] \;+\; p_{\text{port}}\, |\mathcal{E}(a)|,$$

all rates evaluated at $\mathbf{p}(h)$ [flux/ZelBack/src/services/utils/appUtilities.js:89–122]. The base is quoted per three instances: write $\beta(a,h) := \lceil B(a,h)/3 \rceil_2$ for the per-instance base. The charge per default lifetime is

$$A(a,k,h) \;:=\; \begin{cases} \beta, & k = 1 \text{ or } h < 1{,}890{,}000, \\ \beta + 0.5\,(k-1), & \beta < 0.5 \text{ and } k > 3, \\ \lceil \beta\,(k-1) \rceil_2 + \beta, & \text{otherwise}, \end{cases}$$

[flux/ZelBack/src/services/utils/appUtilities.js:123–134] [flux/ZelBack/config/default.js:306]; the USD-minimum branch that follows it, gated by `applyMinimumPriceOn3Instances` [flux/ZelBack/src/services/utils/appUtilities.js:136–138], is inert on the FLUX path, since no row of the FLUX rate table carries `minUSDPrice` [flux/ZelBack/src/services/utils/chainUtilities.js:22–36]. The price of a fresh registration is

$$\mathcal{P}(a,k,d,h) \;=\; \max\Big\{ p_{\min}(h),\; \Big\lceil \frac{d}{L^*(h)} \cdot A(a,k,h) \Big\rceil_2 \Big\}, \qquad L^*(h) := \begin{cases} L, & h < h_{PoN}, \\ 4\,L, & h \geq h_{PoN}, \end{cases}$$

with $L = 22{,}000$ blocks [flux/ZelBack/config/default.js:285] and $h_{PoN} = 2{,}020{,}000$ [flux/ZelBack/config/default.js:293] [flux/ZelBack/src/services/appMessaging/messageVerifier.js:736–745]. An *update* is priced at $\max\{p_{\min},\, 0.9\,(\mathcal{P} - \varpi\, \mathcal{P}_{\text{prev}})\}$, where $\varpi = (d_{\text{prev}} - (h - h_{\text{prev}}))/d_{\text{prev}}$ is the unused fraction of the previous term [flux/ZelBack/src/services/appMessaging/messageVerifier.js:797–828].

Bounds and constants. $L = 22{,}000$ blocks, which at the pre-PoN 2 min spacing is 30.6 d [flux/ZelBack/config/default.js:285]; $k_{\min} = 3$ in general and 1 for SPEC v8 applications from height

2,176,519 [flux/ZelBack/config/default.js:257–259]; $k_{\max} = 100$ [flux/ZelBack/config/default.js:260]; the post-PoN duration ceiling is 1,056,000 blocks (≈ 367 d at 30 s) [flux/ZelBack/config/default.js:292] and the floor $d_{\min}(h)$ is height-gated by the validator: 5,000 blocks, then 100 from height 1,630,040, then 1 from height 1,964,447 [flux/ZelBack/src/services/appRequirements/appValidator.js:852–862], while the live endpoint still reports 5,000 [measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03] (conflict C33). The payment sink is the transparent address `t3NryfAQLGeFs9jEoeqsxmBN2QLRaRKFLUX` from height 1,670,000, preceded by `t3aGJvdtd8NR6GrnqnRuVEzH6MbrXuJFLUX` and, at any height, the legacy `t1LUs6quf7TB2zVZmexqPQdnqmrFMGZGjV6` [flux/ZelBack/config/default.js:241–245]. All application revenue therefore lands at a single transparent address by policy, which Section 2 records as a centralization surface and Section 7 proposes to replace with a consensus-enforced split (PLANNED). Measured, 439 of 1,195 deployed applications (36.7 %) sit at three instances, the floor for specifications below SPEC v8 (SPEC v8 allows one), and the maximum observed instance count is exactly $k_{\max}$ [measured: api.runonflux.io/apps/globalappsspecifications, 2026-09-03]. The minimum is re-applied *after* the duration scaling [flux/ZelBack/src/services/appMessaging/messageVerifier.js:740–745], so no registration settles below $p_{\min}$, which is why the floor sits outside the duration factor in Definition 16.3.

Definition 16.4 (Expiry and entitlement)**.** Let h_0 be the height of the block carrying the confirmed payment that promotes a's temporary message to a permanent one, and d its requested duration. The paid-through height is

$$e_a := \begin{cases} h_0 + d, & h_0 \geq h_{PoN}, \\ h_{PoN} + 4\,(h_0 + d - h_{PoN}), & h_0 < h_{PoN} \leq h_0 + d, \end{cases}$$

the second case being the post-fork recomputation applied to applications straddling PoN activation [flux/ZelBack/src/services/appMessaging/messageVerifier.js:711–724]. Entitlement is the predicate $\mathrm{Ent}(a, h) := \mathbf{1}[h \leq e_a]$, evaluated independently by every node.

16.1.2 The property that makes direct settlement the default

Property 16.5 (No standing obligation)**.** SHIPPED. Under direct settlement, for every application a and every height h, the value $\mathrm{Ent}(a, h)$ is a function of the confirmed chain prefix at or before h alone. Consequently: (i) the network holds no forward-looking claim against the tenant — there exists no height $h' > h$ at which the tenant owes anything; (ii) for all $h' > h$, the value of $\mathrm{Ent}(a, h')$ is already determined at h up to the arrival of a further payment, and is non-increasing in h' absent one; (iii) tenant and operator cannot hold divergent views of the entitlement except by holding divergent views of the chain.

Proof. By Definition 16.4, e_a is determined by (h_0, d), both of which are fields of, or the height of, a block at or before h; by Definition 16.3, the promotion test $v^{\mathrm{obs}} \geq \mathcal{P}(a, k, d, h_0)$ is evaluated at promotion time against the rate row $\iota(h_0)$, and ι is monotone in height, so no later block changes it. Hence $\mathrm{Ent}(a, \cdot)$ factors through the prefix, giving (i) and (ii). For (iii): under Assumption 16.1(i) two honest nodes agree on the prefix, and e_a is computed by the same integer arithmetic on both, so any disagreement is a disagreement about the chain and surfaces as such. □

Property 16.5 is the reason this paper recommends direct settlement to autonomous software. An agent needs no account, no stored credential and no relationship that outlives the deployment; there is no balance to track, no charge to authorise, and no reconciliation step that can fail while the agent is not watching. It is also the reason this path carries no grace, suspension or eviction machinery: expiry is arithmetic, not a decision, so there is nothing for a policy to decide and no state a policy could get wrong.

Corollary 16.6. *No grace or suspension state is required on this path. Any such state would be keyed on a balance or an outstanding obligation, and by Property 16.5(i) neither exists.*

Construction 16.7 (SETTLE-DIRECT — registration, payment, promotion, SHIPPED)**.** *Inputs.* A formatted specification a; a signature over it by the owner's ZelID; a duration d; a funded wallet. *State.* Per node: a temporary-message store (TTL 3,600 s), a permanent-message store, and the chain.

1. The tenant reads $\mathbf{p}(h)$ from `/apps/deploymentinformation` and computes $\mathcal{P}$ by Definition 16.3.
2. The tenant `POST`s the signed message to any node's `/apps/appregister`; the node verifies the signature, the timestamp window ($\leq$ 1 h old, $\leq$ 5 min ahead), the schema for the declared specification version, and its own peer counts ($\geq$ 8 outgoing, $\geq$ 4 incoming), then broadcasts the message and returns only its hash [flux/ZelBack/src/services/appDatabase/registryManager.js:1714–1888].
3. The tenant broadcasts one transaction paying $v^{\text{obs}} \geq \mathcal{P}$ to the sink address, carrying that hash in an `OP_RETURN`.
4. Each node's explorer loop recognises the payment by output address, sums `valueSat` across matching outputs, decodes the hash, and calls the promotion routine [flux/ZelBack/src/services/explorerService.js:309–345,444].
5. Promotion stores the permanent message, recomputes e_a (Definition 16.4), and admits the application to global state iff $v^{\text{obs}} \geq \mathcal{P}$ [flux/ZelBack/src/services/appMessaging/messageVerifier.js:619–897].

Invariant. After promotion, global state contains a with paid-through height e_a computed from the chain alone; before promotion it contains nothing about a that survives the temporary-message TTL. *Failure modes.* (a) The hash is not resolvable from peers within three attempts spaced 60 s: the registration never promotes and the tenant sees a client-side error at step 2. (b) $v^{\text{obs}} < \mathcal{P}$: see Property 16.9 — the payment is spent and the tenant is told nothing.

16.1.3 CumulusVPN: the worked example

The second deployed instance of direct settlement is not an application-hosting charge at all, and it is the more important of the two for this paper's thesis. CumulusVPN is a WireGuard exit-gateway product whose gateways run as ordinary Flux applications. Its entitlement mechanism is this: the client generates a WireGuard keypair on the device, derives a payment code $\mathsf{code} = \mathsf{base58}(\mathrm{sha256}(pk)[0{:}20])$ from its own public key, and sends FLUX to a fixed address with the `OP_RETURN` memo `CVPN1:code`. Gateway and client compute the code byte-identically [cumulusvpn/gateway/internal/entitle/entitle.go:99–108] [cumulusvpn/clients/core-ts/src/paymentCode.ts:17–23]. Every gateway then scans the chain by itself and derives the map $\mathsf{paidUntil}[\mathsf{code}]$ from confirmed, correctly-memoed payments, with no gateway-to-gateway coordination of any kind [cumulusvpn/gateway/internal/entitle/entitle.go:1–5,218–287].

There is no account, no token and no server-issued credential. The WireGuard public key *is* the credential: the gateway checks the key it already sees on the tunnel against the map it derived from the chain [cumulusvpn/gateway/internal/api/api.go:246,374]. Authentication runs in the other direction only — the control API signs every response body with an ed25519 key derived from the gateway's own WireGuard private key, so a client that learned the public key through discovery can verify the responder without a TLS certificate [cumulusvpn/gateway/internal/api/api.go:563–585]. One payment covers every gateway, including both hops of a multi-hop path, because entitlement is keyed by the client's key on all of them [doc: cumulusvpn/docs/11-multihop.md:47–58]. This is a deployed instance of "payment is the account", which the roadmap lists as a future item [roadmap: Q4 2026]; it is not a plan, it is running (conflict C19).

Construction 16.8 (DERIVE-ENTITLEMENT — the gateway chain scanner, SHIPPED)**.** *Inputs.* A transaction source exposing `BlockCount` and `AddressTxs` (the host node's daemon proxy, falling back to a public explorer); the payment address A; the price P in FLUX per 30 d period; an optional snapshot at `/data/entitle.state`. *State.* paidUntil : code $\to$ time, plus a scan cursor.

1. Backfill from the stored cursor, or from height 0 if there is none.
2. Every 15 s, read `BlockCount`; on a new height, fetch the new transactions paying A.
3. Accept a transaction iff it carries exactly one `CVPN1:` memo and $30\,(v + 10^{-9}) \geq P$, i.e. at least one day's worth [cumulusvpn/gateway/internal/entitle/entitle.go:270–287,296–323].
4. Grant $\lfloor 30v/P \rfloor$ days, stacking onto any existing paidUntil[code], capped at now + 720 d [cumulusvpn/gateway/internal/entitle/entitle.go:34–36,218–268].
5. Checkpoint immediately on a real grant, otherwise at most once per 60 s; treat the snapshot as pure cache and discard it on any version, address or price mismatch [cumulusvpn/gateway/internal/entitle/snapshot.go:14,29–31].
6. Every 15 s, re-derive each enrolled peer's tier and retune its rate limiter in place, so a payment takes effect on in-flight flows with no reconnect [cumulusvpn/gateway/internal/limiter/limiter.go:105–123].

Invariant. paidUntil is a pure function of the confirmed transaction history of A; two gateways with the same view of the chain and the same (A, P) derive identical maps without exchanging a message. Changing P invalidates every cached snapshot by construction (step 5). *Failure modes.* If the route to chain data is absent, the engine distinguishes "route absent" from "no payments" and retains the previous map rather than demoting every paying peer [doc: cumulusvpn/docs/03-gateway.md:135–138]. If `/data` is unwritable, the snapshot and the peer table are lost on restart; a statically issued configuration then cannot re-enroll [cumulusvpn/gateway/internal/wg/peercache.go:17–24,47–51]. A migration of the gateway instance to another node changes the server key and address, which invalidates an already-issued configuration — unavoidably so [doc: cumulusvpn/docs/03-gateway.md:95–97].

The caveat that travels with it. Payment and usage are **linkable**. The memo is a one-way hash of the public key, but the gateway a user enrolls with learns the real public key, recomputes the hash, and can then find the paying address on a public chain. The repository states this itself as a known, unmitigated gap, with payment-key indirection and blind-signature vouchers named as the intended fixes [doc: cumulusvpn/docs/04-payments.md:81–87]. CumulusVPN is therefore simultaneously the best existing demonstration of direct settlement and a concrete, shipped instance of the problem Section 21 takes up: the network must be convinced that a workload is funded while learning neither payer nor workload. Section 21 is written against this system, not against a hypothetical one.

16.1.4 The defect on this path, stated as a requirement

Property 16.9 (Silent underpayment)**.** SHIPPED. If a registration's confirmed payment satisfies $v^{\text{obs}} < \mathcal{P}$, the promotion routine emits a log warning and drops the registration. No error is surfaced to the payer, no retry is scheduled, and no refund path exists in the code [flux/ZelBack/src/services/appMessaging/messageVerifier.js:735–763]; the update path behaves identically [flux/ZelBack/src/services/appMessaging/messageVerifier.js:797–828]. The funds are spent and the deployment does not exist.

Proof sketch. By inspection of the promotion routine: the price comparison is the last gate before `insertAppSpecifications`, and its false branch is a `log.warn` with no message emitted on the

P2P channel and no HTTP response outstanding to write to — the tenant's only synchronous interaction ended at step 2 of Construction 16.7. From the payer's side the observable state is indistinguishable from "the message never propagated". □

For a human tenant this is a support ticket. For an autonomous agent with nobody to escalate to it is an unrecoverable silent failure, and it sits on exactly the path this paper recommends agents use. That asymmetry, not the size of the loss, is what makes it the most important correctness gap in Part A.

Remark 16.10 (Requirement R16.1 — signed negative acknowledgement). Any node that declines to promote a registration or update for insufficient payment shall emit a signed message $\mathsf{nack} = (\mathsf{hash}, h, \mathsf{txid}, \mathcal{P}^{\mathrm{req}}, v^{\mathrm{obs}}, \mathsf{reason})$ over the existing signed-envelope P2P channel, retrievable by hash from any node's read API. Three properties are required and one is explicitly *not*: the message must be verifiable against the deterministic node list under the same rule as every other broadcast; it must carry both the computed price and the observed payment, so the tenant can locate the discrepancy without a second round trip; and it must be emitted on the same condition that triggers the drop, so that absence of a nack is informative. Quorum is *not* required: because $\mathcal{P}$ is a deterministic public function of the chain (Definition 16.3), a single valid nack from any confirmed node is actionable evidence that the tenant can re-derive itself. The construction is cheap; its absence is what makes Invariant 2 below fail on Part A today.

16.2 Part B — A service built on Flux, and the credits layer that lives outside it

This part contains two kinds of object, a reader must not merge them, and **neither of them is part of Flux**. Section 16.2.1 is SHIPPED and is written in the present tense because it runs: a centralized payment service, operated by InFlux Technologies [intent: design intake], through which a person signs in with an email address, pays by card, and has a deployment registered, funded and renewed for them. It is not a credit account. It holds no user balance and offers no withdrawal; what it does hold is both of the user's private keys, which the user never sees [intent: design intake], and that is the subject of the subsection. Nearly every fact stated there carries code provenance and the one that does not says so; and every one of them is a fact about the operator's service rather than about the network, because the chain cannot observe the service and does not depend on it (Property 16.21). Section 16.2.2 is PLANNED and is written in the conditional: the deposited balance and lifecycle state machine a credits layer would require, retained here as a specification because it is the object Section 27's dependency list waits on, and marked throughout as unimplemented. Nothing in the second subsection describes the first.

Remark 16.11 (Requirement R16.2 — inherited from Section 7). Section 7 exports one hard requirement to this section: after PNR activation at h_{PA}, every application-hosting charge, *whatever the payment channel*, must settle as exactly one PNR transaction on L1, or the consensus-enforced split is void. A credit balance may therefore not become an off-chain ledger of hosting charges; each charge it authorises must appear on-chain as one settlement. This constrains the construction below at every point and is the reason Invariant 1 is stated as a conservation law rather than as bookkeeping. A second requirement runs the other way: any balance forfeited on eviction must not be routed into the fee pool Φ, or evicting a tenant becomes directly profitable for the fleet that votes (Section 23). The operator's service already satisfies the first requirement by construction — it settles each card charge as exactly one transaction to the sink address (Construction 16.16, step 4) — which is worth recording, because it means a convenience layer of this shape does not obstruct PNR. That is a statement about one operator's implementation, not a guarantee the protocol extracts from any service built over it; the requirement binds the settlement shape, and any service that violates it simply fails to produce a funded deployment.

16.2.1 What runs on top: InFlux's prepay-and-forward payment facilitator

Assumption 16.12 (Environment of a custodial payment service)**.** In addition to Assumption 16.1: (i) the facilitator is a single administrative party operating server-side infrastructure outside the FluxNode set, so Section 2's adversary model does not cover it — a node honest under that model stays honest while the facilitator misbehaves, and the converse; (ii) the facilitator authenticates users by email PIN or by Google/Apple OAuth through a hosted identity provider [console-api/src/services/auth/pin.ts:4–7] [fluxos-frontend/src/utils/firebase.js:53–142], and that authentication is orthogonal to every key the network verifies; (iii) no consensus rule, on-chain record or contract binds the facilitator's behaviour — all of it is server-side code and operational discipline; (iv) the network is not a party to the arrangement and takes no view of it: the accept/reject decision of Construction 16.7 is invariant under the substitution of the facilitator for the tenant, so no node can tell that a service was involved (Property 16.21, proved below). Clause (iii) is a trust assumption of exactly the kind Section 2 inventories, and it is carried in Section 23 as a centralization surface *of this service*; clause (iv) is why it is not a centralization surface of the network.

Definition 16.13 (Custodial payment facilitator)**.** Write addr$(\cdot)$ for the address derived from a public key and owner(a) for the address carried in a's `owner` field. A *facilitator* $\mathcal{F}$ is a commercial operator — here InFlux Technologies [intent: design intake] — running server-side infrastructure outside the FluxNode set and holding, for each user u it serves, two private keys generated server-side at account creation and never disclosed to u [console-api/src/services/flux/keypair.ts:20–27] [intent: design intake]:

- an *identity* key sk_u^{id} over secp256k1, whose P2PKH address addr(pk_u^{id}) — the user's ZelID — is written into the `owner` field of every specification $\mathcal{F}$ registers for u, and which signs the off-chain register message;
- a *payment* key sk_u^{pay} on the Flux transparent network, used to build and broadcast on-chain transactions.

Write $K_u = (sk_u^{\mathrm{id}}, sk_u^{\mathrm{pay}})$. K_u is held *in trust*: it is generated by the operator, it stays with the operator, and u is never shown either member of it [intent: design intake]. Both members are stored AES-256-GCM-encrypted at rest and WIF-encoded, and both are decrypted in the serving process whenever a signature or a transaction is required [console-api/src/db/models/FluxIdentity.ts:13–28]; at most one active identity exists per account, enforced by a unique index [console-api/src/db/models/FluxIdentity.ts:40–43]. $\mathcal{F}$ additionally holds a set $\mathcal{H} = \{h_{\mathrm{fiat}}, h_{\mathrm{free}}, h_{\mathrm{ren}}\}$ of three operator hot wallets, from which every registration and renewal it performs is funded. Its durable per-payment record is

$$r \;=\; \big(H(m_a),\; v^{\mathrm{sat}},\; v^{\mathrm{usd}},\; t\big) \;\in\; \{0,1\}^{256} \times \mathbb{Z}_{\geq 0} \times \mathbb{Q}_{\geq 0} \times \mathbb{N},$$

keyed by *application-message hash* rather than by user, carrying the satoshi amount settled on chain and the USD amount charged to the card. The payment record carries no time-to-live index and is not auto-deleted [appsmonitor/src/services/appsPaidWithFiatMonitor.js:150–153]; a time-to-live index $T_{\mathrm{rec}} = 518{,}400\,\mathrm{s} = 6\,\mathrm{d}$ applies only to the service's logs of payment-reminder notifications and similar processing logs [appsmonitor/config/default.js:36–58] [appsmonitor/src/services/appsMonitor.js:234–252]. **There is no further state.** In particular the record carries no user-held quantity, and the balance b_u and lifecycle label of Definition 16.23 below have no referent here (Property 16.19).

Definition 16.14 (Prepay-and-forward settlement)**.** Let bal : $\mathcal{H} \to \mathbb{Z}_{\geq 0}$ give the operator wallets' balances in sat. For a registration or renewal of a for user u at height h, the settlement action of $\mathcal{F}$ is: charge u's card instrument, held at the payment processor and not at Flux, for the corresponding USD amount; then broadcast one transaction from some $h \in \mathcal{H}$,

$$\mathrm{Pay}_{\mathcal{F}}(u, a, h): \quad \mathrm{bal}(h) \;\longmapsto\; \mathrm{bal}(h) - \mathcal{P}(a, k_a, d_a, h);$$

then acknowledge the charge to the processor as fulfilled. The debit falls on the *operator's* wallet. No user-held quantity is decremented, because none exists. This is the precise sense in which the service is a *prepay-and-forward payment facilitator* rather than a credit account: value moves card → operator and operator → chain, never card → user balance → chain. Only the second leg is visible on chain, and it is visible as an ordinary payment from an ordinary key.

Definition 16.15 (Discretionary free hosting)**.** $\mathcal{F}$ also funds some applications from $\mathcal{H}$ with no card charge at all. Eligibility is a predicate Free on the set of deployed applications, evaluated by the operator's own daemon and by nothing else,

$$\mathrm{Free}(a) \;:=\; \mathbf{1}[a \in \mathsf{A}_{\mathrm{orb}}] \;\vee\; \mathbf{1}[\mathrm{owner}(a) \in \mathsf{A}_{\mathrm{ren}}] \;\vee\; \mathbf{1}[\mathrm{owner}(a) \in \mathsf{A}_{\mathrm{paid}}] \;\vee\; \mathrm{Cap}(a),$$

where $\mathsf{A}_{\mathrm{orb}}$ is a hardcoded list of nine grandfathered application names [appsmonitor/src/services/appsMonitor.js:36–46]; $\mathsf{A}_{\mathrm{ren}}$ a hardcoded owner allowlist whose applications are renewed from h_{ren} once the adjusted expiry falls within four days [appsmonitor/config/default.js:60–72] [appsmonitor/src/services/appsMonitor.js:308,437–445]; $\mathsf{A}_{\mathrm{paid}}$ a hardcoded two-address allowlist whose applications are subsidised in full and indefinitely [appsmonitor/config/default.js:92–94] [appsmonitor/src/services/appsPaidForOwnersMonitor.js:78–90]; and $\mathrm{Cap}(a)$ holds iff a runs the single image `runonflux/orbit:latest` in one component at one instance with $r^{\mathrm{cpu}} \le 0.5$ vCPU, $r^{\mathrm{ram}} \le 1{,}000$ MB and $r^{\mathrm{hdd}} \le 5$ GB, and its owner runs no other such application [appsmonitor/src/services/appsMonitor.js:137–148,159–180]. A separate rule funds a zero-price update — one whose price delta is zero and whose requested expiry extension is at most 11 blocks — at a flat 0.02 FLUX from h_{free}, after a 2 min settle delay [appsmonitor/src/services/appsFreeUpdatesMonitor.js:133–137,157,174] [appsmonitor/src/services/fluxService.js:364–367]; and a further rule pays one month for applications matching a marketplace listing [appsmonitor/src/services/appsMarketplaceFreeFirstMonth.js:342–344]. Every set and constant above is operator-side configuration: none is on chain, none is user-settable, and no appeal path exists.

Construction 16.16 (PREPAY-FORWARD — custodial registration, payment and renewal, SHIPPED)**.** *Inputs.* A user u authenticated to $\mathcal{F}$ under Assumption 16.12(ii); a specification a; a card instrument, either a one-off checkout session or a stored subscription. *State.* K_u and $\mathcal{H}$ as in Definition 16.13; the per-payment records; the chain.

1. $\mathcal{F}$ decrypts sk_u^{id} in process and signs m_a, whose `owner` field is $\mathrm{addr}(pk_u^{\mathrm{id}})$ [console-api/src/db/models/FluxIdentity.ts:13–28]. In the legacy stack the equivalent step is a remote call that returns both a ZelID address and a signature over a network-issued login phrase, which FluxOS's own `verifylogin` then accepts [fluxos-frontend/src/@core/components/Login.vue:1039–1104].
2. The signed message is submitted to a node exactly as in step 2 of Construction 16.7; the node returns $H(m_a)$ and nothing else.
3. The card is charged in USD.
4. A polling loop enumerates message hashes for which a card charge exists and an on-chain payment does not, computes $\mathcal{P}$ by Definition 16.3, broadcasts *one* transaction from h_{fiat} to the deployment sink carrying $H(m_a)$ in `OP_RETURN`, and acknowledges the charge [appsmonitor/src/services/appsPaidWithFiatMonitor.js:93–190] [appsmonitor/src/services/fluxService.js:345–362].
5. Promotion proceeds exactly as in steps 4–5 of Construction 16.7. No node distinguishes this payment from any other.
6. *Renewal.* Every 3,600 s the facilitator reads each active subscription's application from global state, computes the PoN-adjusted remaining lifetime, and when $e_a - h < \Delta_{\mathrm{adv}} = 8{,}640$ blocks (≈ 3 d at 30 s) triggers a card charge; the pending-payment loop then repeats steps 4–5 every 60 s [appsmonitor/src/services/appsSubscriptionRenewalMonitor.js:112–368,379–460,566–568]. The renewed term defaults to 88,000 blocks and is capped at 1,056,000 [appsmonitor/src/services/appsSubscriptionRenewalMonitor.js:18,213].

Invariant. At every height, $\mathrm{Ent}(a, h)$ (Definition 16.4) is the same function of the confirmed chain prefix as on Part A. The facilitator changes *who signs* and *who pays*; it does not change how the network decides, and it cannot, because every step above is a step of Construction 16.7 executed by a different party (Property 16.21). *Failure modes.* (a) The card charge fails: a loop reads the processor's failure list every 21,600 s and, with a per-subscription cooldown of 24 h, emails the application's contact address; that is the entire remedy, and no code path revives an application once expired [appsmonitor/src/services/appsSubscriptionRenewalMonitor.js:19,463–575]. (b) A renewal loop stops or is stopped: it fails closed to *unrenewed*, and the application expires by Definition 16.4 exactly as an unfunded direct-settlement one would. (c) Records: the on-chain payment persists and is public, and the operator-side payment record linking a card charge to an application is not auto-deleted; only the notification logs expire after T_{rec}. (d) Underpayment: Property 16.9 applies unchanged, with the loss falling on h_{fiat} rather than on the user [inferred].

Property 16.17 (Custodial signing and spending authority)**.** SHIPPED. For every user u served by $\mathcal{F}$, the facilitator can (i) produce a valid signature under pk_u^{id} over any message — including any `appregister`, `appupdate` or expire message naming u's address in the `owner` field — and (ii) spend any output controlled by pk_u^{pay}; in both cases without a counter-signature from u and without u being online. No threshold, policy engine, hardware element or second factor stands between the serving process and either key. The correct description of u's position is therefore not that u delegates authority to $\mathcal{F}$, but that $\mathcal{F}$ **is the key holder and** u **authenticates to it.**

Proof sketch. For the current stack, by direct inspection: both private keys live in one record, encrypted under a service-held master key with associated data the service itself supplies, and are decrypted in the same worker that signs and broadcasts [console-api/src/db/models/FluxIdentity.ts:13–28]. Encryption at rest binds use of the key to possession of the master key, not to any act of u; a key-provider abstraction with a self-hosted KMS backend exists and hardens that binding without changing it IN-PROGRESS [console-api/src/lib/crypto/keys.ts:63–286]. For the legacy stack the conclusion follows from the observed protocol rather than from a read of the signer, which is not in the corpus: the login path sends a network-issued login phrase to a remote service and receives back both a ZelID address and a signature over that phrase which `verifylogin` accepts [fluxos-frontend/src/@core/components/Login.vue:1039–1104]; producing that signature requires the corresponding private key, so the service holds it [inferred]. □

Corollary 16.18 (There is nothing to delegate from, so Section 17 does not reach this path)**.** SHIPPED. *On this path the delegation object of Section 17 — a principal $\mathcal{U}$ granting an agent $\mathcal{A}$ a scope Σ under a budget $\mathcal{B}$ — has no instantiation, because $\mathcal{U}$ never holds the key from which authority would be delegated. The correct reading is not that Section 17's mechanism is present but unenforced here; it is that the mechanism has nowhere to attach, and no certificate construction can bound an authority its putative grantor does not hold. Section 19 identifies the same architecture behind its operator-signing path and observes that a bearer token there is bounded by nothing; Property 16.17 sharpens that: the token does not confer authority, it authenticates the user to the party that already holds it. Whatever bounds that path is therefore operator-side policy, not a protocol mechanism, and this paper specifies none for it. Section 19's local-signing path — SSP and Zelcore, where the signature is produced on the user's own device — is the path Section 17 is written for, and every property proved there continues to hold there.*

Property 16.19 (No balance, no withdrawal)**.** SHIPPED. In the deployed facilitator: (i) no field of the data model denotes a user-held quantity of FLUX or of fiat, the record being keyed per application-message hash rather than per user (Definition 16.13); (ii) the payment records carry no TTL index and are not auto-deleted; the TTL of $T_{\mathrm{rec}} = 6\,\mathrm{d}$ applies only to payment-reminder notification and processing logs [appsmonitor/config/default.js:36–58] [appsmonitor/src/services/appsPaidWithFiatMonitor.js:150–153]; (iii) a targeted search of the three repositories that

implement the path found no code path returning FLUX or fiat to a user, hence no withdrawal [appsmonitor, fluxos-frontend, console-api:targeted search, no match]; (iv) the newer stack's billing endpoint aggregates an on-chain wallet balance with two read-only legacy sources and marks unconfirmed deposits `shadow:true, credited:false` so that they are never summed into a spendable balance, and exposes no write path at all IN-PROGRESS [console-api/src/routes/billing.ts:44–61,63–165]. Consequently b_u of Definition 16.23 has no deployed referent, and neither has any state in $\{\mathsf{funded}, \mathsf{grace}, \mathsf{suspended}, \mathsf{evicted}\}$.

Not established by this survey. A repository named `withdrawalservice` exists, with a generic claim-intake schema (`coin`, `network`, `address`, optional `fluxid`) that reads as a multi-product claim path rather than as this layer's ledger [withdrawalservice/src/controllers/claimController.js:1–58]. Whether it is wired to the facilitator at all was not determined. Property 16.19(iii) is the result of a negative search across the three implementing repositories.

Property 16.20 (Payer and owner are independent)**.** SHIPPED. The promotion test of Construction 16.7 is a function of the paying transaction's *outputs* only: recognition is by output address and by the temporary-message hash decoded from `OP_RETURN`, and the amount is the sum of `valueSat` over matching outputs [flux/ZelBack/src/services/explorerService.js:309–345]. No input, and no property of the spending key, is examined. Hence for any application a, the party that pays $\mathcal{P}(a, k, d, h)$ need not be the party whose key signs m_a and appears in the `owner` field.

Proof sketch. The register message is authenticated by the owner's signature at step 2 of Construction 16.7 and carries no payer field; the payment is authenticated by nothing beyond being confirmed on chain, and is bound to the message only through the hash in its `OP_RETURN`. The two authentications are therefore over disjoint objects, and the promotion routine composes them without a cross-check. The converse capability — a third party altering an owner's expiry — exists only through the separate hardcoded `usersToExtend` allowlist [flux/ZelBack/config/default.js:229], which is a distinct mechanism and is not needed here. □

Property 16.20 is why the custody of Property 16.17 is not forced by the protocol. Fronting the FLUX for a tenant — the economically useful half of what $\mathcal{F}$ does — requires only the ability to broadcast one transaction naming a message hash. Holding the tenant's identity key — the half that carries the trust — is what card checkout and email login currently require, not what payment requires. The two halves are separable in the deployed protocol today; nothing in the corpus separates them. The same fact, read the other way, gives the property that fixes this subsection's place in the paper.

Property 16.21 (The chain is indifferent to the service)**.** SHIPPED. Fix an application a, a duration d and a height h_0. Consider two executions of Construction 16.7: one in which u signs m_a on their own device and broadcasts the paying transaction, and one in which $\mathcal{F}$ signs under K_u and broadcasts from $h_{\text{fiat}} \in \mathcal{H}$ (Construction 16.16). If both signatures verify against owner(a) and both payments satisfy $v^{\text{obs}} \geq \mathcal{P}$, then every node's observable behaviour is identical in the two executions: the same message is stored, the same e_a is computed, and $\text{Ent}(a, \cdot)$ is the same function. Consequently, across the registration, promotion, pricing and entitlement paths — which are the whole of what a node does with an application payment — no consensus rule, no FluxOS validation step and no piece of node state distinguishes a registration made through a payment service from one made by a tenant alone. The service is invisible to the protocol, and the protocol's guarantees neither improve nor degrade in its presence.

Proof sketch. Three inputs, each already established. (a) Promotion is a function of the paying transaction's outputs only — recognition by output address, amount by summing `valueSat` over matching outputs, binding by the hash decoded from `OP_RETURN` — and examines no input and no property of the spending key (Property 16.20 [flux/ZelBack/src/services/explorerService.js:309–345]).

So the identity of the broadcaster is not an input to any node's decision. (b) The register message is admitted on verification of a signature against the address in the `owner` field [flux/ZelBack/src/services/appDatabase/registryManager.js:1714–1888]; signature verification is a predicate on (message, signature, address) and takes no argument describing who holds the corresponding private key, so it cannot separate the two executions. (c) $\mathcal{P}$ and e_a are functions of (a, k, d, h_0) alone (Definitions 16.3 and 16.4), and none of those four is a property of the payer or the signer. The two executions therefore agree on every input to every node-side decision, hence on every output. The argument is a statement about what the deployed code reads, not about what it intends to permit; it would be falsified by any node-side branch on payer identity, and none exists in the cited paths. □

This is the property that fixes what the rest of the subsection is about. The service of Definition 16.13 is not a Flux capability with a trust caveat attached; it is a company's product sitting on top of a settlement path that would work exactly the same way if the company shut it down tomorrow. Everything enumerated below is therefore a fact about that product. None of it is a fact about Flux, and a reader assessing the network's trust model should discount all of it.

The trust surface of the operator's service, enumerated. Each item is a property of the operator's service only. None of it is a property of Flux, and none of it applies to Part A. Items 1–5 are SHIPPED on code provenance; item 6 rests on a stated behaviour and says so.

1. **The operator can sign and spend as the user.** Property 16.17. There is no counter-signature, no bound on what may be signed, and no user-side revocation — revoking would require holding a key the user does not have.
2. **The operator decides unilaterally who is hosted for free.** Definition 16.15: three hardcoded allowlists and one resource-cap rule, all in operator-side configuration. A user cannot read the lists, cannot appeal an exclusion, and cannot determine from the chain why one application renews for free and another does not; no appeal path exists in code [appsmonitor:targeted search, no match].
3. **On payment failure the only remedy is email.** Construction 16.16, failure mode (a): a message to the application's contact address, rate-limited to one per 24 h per subscription, with no in-band signal, no user override, and no path that revives an expired application [appsmonitor/src/services/appsSubscriptionRenewalMonitor.js:19,463–575].
4. **The record linking a payment to a card charge is held by the operator alone.** The payment record is kept in the operator's database and is not auto-deleted; only its payment-reminder notification logs expire after $T_{\text{rec}} = 6\,\text{d}$ [appsmonitor/src/services/appsPaidWithFiatMonitor.js:150–153]. The on-chain leg is permanent and public; the association between it and a card charge is not public.
5. **Every owner-keyed capability runs through the custodian.** Because $\text{addr}(pk_u^{\text{id}})$ is the specification's `owner` (Definition 16.13), update, expire, and any future mechanism that reads the owner field are exercisable by $\mathcal{F}$ and not by u.
6. **A lapse destroys the data, and only the operator can avert it.** Application data at expiry is always deleted: expiry removes the application from the network and nothing — volumes included — is retained past e_a [intent: design intake]. That behaviour is the network's and is the same on Part A, where a tenant can always avert it by signing one more message and paying for it. Here they cannot: a renewal requires a signature under pk_u^{id} (Construction 16.7, step 2), which only $\mathcal{F}$ can produce (Property 16.17), so a declined card and an unread dunning email are together sufficient to destroy a tenant's data with no action the tenant can take. That composition of item 3 with deletion, rather than deletion itself, is the trust surface.

The deletion behaviour in item 6 is stated by the team as current and unconditional [intent: design intake]; no file in this extraction was read that establishes it independently, so it carries intent provenance and is not tagged SHIPPED. It is nonetheless a definite answer and it replaces the *Unclear* this list previously carried. Note that Invariant 3 ("data survives suspension") remains a property of the unimplemented layer of Section 16.2.2 alone and must not be read across: on every deployed path, expiry deletes.

A consequence for Section 23. This is the one place where a service outside the protocol leaves a mark inside it. Because addr(pk_u^{id}) is written into the specification's `owner` field (Definition 16.13), some of the network's nominal application owners are operator-held keys: each card-paying user has a ZelID of their own, but it is generated and held by the operator rather than by the user. The largest owner identity, which holds 246 of the 1,195 deployed applications, 20.6 % of the set [measured: api.runonflux.io/apps/globalappsspecifications, 2026-09-03], is not such a key: it is the team's own ZelID (§21). What follows regardless is a constraint on every mechanism keyed on "the owner": the owner-concentration statistics of Section 23, moderation standing, and the delegation object of Section 17 all read a field that may name an operator-held key rather than a key the party holds, and each must say which it means. For Section 17 the resolution is already fixed by Corollary 16.18: wherever that field names an operator, there is no principal-held key, and the delegation object does not apply rather than applying weakly.

Whose service this is. The framing used throughout this subsection is the team's own, and it is both cleaner and more accurate than treating custody as a Flux option with a caveat: **the user never sees the key; it is server-held in trust; and this is a centralized service run by InFlux, outside the protocol, not a feature of the chain** [intent: design intake]. The paper adopts it without apology and without criticism, because it is what makes the central claim precise rather than defensive. Part A is the protocol's settlement path: trustless, shipping today in two independent forms, and an autonomous agent uses it and meets none of the trust surface above — no account, no card, no email address, no operator (Property 16.5, Invariant 4). What the service buys is card payment and the removal of key management for humans; it buys no access that direct settlement does not already give, and by Property 16.21 the network cannot tell whether it exists. The team describes it as a good middle step until an on-chain mechanism exists [intent: design intake].

The observable direction of travel inside the operator's own stack is currently toward consolidating custody rather than away from it, and the paper records that beside the stated intent rather than in place of it: one identity per account, one unified read-only balance view, and a key-provider abstraction that hardens how the held key is protected without removing custody IN-PROGRESS [console-api/src/lib/crypto/keys.ts:63–286] [console-api/src/routes/billing.ts:63–165].

Remark 16.22 (The operator's option, which is not a protocol matter). The operator may choose to expose K_u to users — that is, to stop being a custodian — and states that it may [intent: design intake]. That would be a change to one company's product and to nothing else: no consensus rule, no FluxOS release and no application specification field would move, and by Property 16.21 the network could not observe the difference either before or after. This paper records the option, attributes the decision to the operator, and does not track it. Nothing in Part A, in Section 17 or in Section 19 depends on which way it goes.

Decided. A non-custodial construction offering the same convenience is not designed here, and after Definition 16.25 it is not a gap in this paper's scope: the protocol offers no recurring authorisation at all, so there is no protocol object a non-custodial alternative would be an alternative *to*. What is not open, and is worth restating: the direct settlement path of Part A is already non-custodial, already deployed, and already usable by an autonomous agent. What custody buys is card payment and the removal of key management for humans — convenience, not capability (Property 16.21).

16.2.2 The credits layer, and why it is a service object rather than a protocol one

Nothing in this subsection is a consensus mechanism. By Definition 16.28 the deposited balance would live in an operator's backend, outside the protocol, and by Definition 16.25 the protocol offers no automatic renewal to attach it to. What follows is therefore the specification an *operator* would implement — the state it carries, the transitions it takes, and the cost of the one guarantee that distinguishes a credit balance from plain expiry — and not a proposal for what nodes should evaluate. It is retained at this depth for two reasons: the lifecycle is genuinely intricate and no public description of it exists, and a reader evaluating such a service should be able to check it against a written specification. Every statement is PLANNED and written in the conditional, and none of it describes the facilitator of Section 16.2.1, which holds no balance and has no lifecycle state (Property 16.19).

Definition 16.23 (Credit state — PLANNED, no implementation exists)**.** For a tenant u, let $b_u \in \mathbb{Z}_{\geq 0}$ denote a deposited balance in sat; no deployed system maintains such a quantity (Property 16.19). For each application a owned by u, let $e_a \in \mathbb{N}$ denote the paid-through height as in Definition 16.4 and let

$$\mathrm{st}(a,h) \in \{\mathsf{funded},\ \mathsf{grace},\ \mathsf{suspended},\ \mathsf{evicted}\}$$

denote the lifecycle state at height h. Let $\mathcal{P}^{\mathrm{ren}}(a,h) := \mathcal{P}(a,k_a,d_a,h)$ be the cost of one renewal period under Definition 16.3, $\mathcal{P}^{\mathrm{res}}$ a resumption charge, and $G_{\mathrm{g}}, G_{\mathrm{s}}, \Delta_{\mathrm{pre}} \in \mathbb{N}$ the grace window, suspension window and pre-authorisation lead, all in blocks. Transitions would be as in Table 40; every transition would be evaluated by the operator from chain state plus its own balance store (Definition 16.28).

Table 40: Specified transitions of $\mathrm{st}(a,h)$. PLANNED— no implementation exists.

from	to	condition	effect on workload	effect on data
funded	funded	$h = e_a - \Delta_{\mathrm{pre}}$ and $b_u \geq \mathcal{P}^{\mathrm{ren}}$; then $b_u \mathrel{-}= \mathcal{P}^{\mathrm{ren}}$, $e_a \mathrel{+}= d_a$	none	none
funded	grace	$h \geq e_a$ and $b_u < \mathcal{P}^{\mathrm{ren}}$	keeps running	retained
grace	funded	$b_u \geq \mathcal{P}^{\mathrm{ren}}$ while $h \leq e_a + G_{\mathrm{g}}$	none	none
grace	suspended	$h > e_a + G_{\mathrm{g}}$	containers stopped, not removed	**retained**, volumes intact
suspended	funded	$b_u \geq \mathcal{P}^{\mathrm{ren}} + \mathcal{P}^{\mathrm{res}}$ while $h \leq e_a + G_{\mathrm{g}} + G_{\mathrm{s}}$	restarted in place	intact
suspended	evicted	$h > e_a + G_{\mathrm{g}} + G_{\mathrm{s}}$	removed	**destroyed**

The design commitment that matters here is the separation of *stopping* from *deleting*. Suspension would halt compute without destroying state, so that a lapse recoverable in hours would not cost the tenant their data; only eviction, after $G_{\mathrm{g}} + G_{\mathrm{s}}$ blocks, would destroy anything. That guarantee has a price, and the price is quantifiable in the same units as everything else in this section.

Property 16.24 (Bounded suspension subsidy)**.** PLANNED. A suspended application would consume disk but neither CPU nor bandwidth, because its containers would be stopped rather than removed and its volumes retained (Table 40). The revenue forgone by the network over the suspension window (after grace, during which the workload still runs at full reservation), valued at the list rates of Definition 16.3, would therefore be bounded above by

$$\Delta^{\mathrm{sub}} \ \leq\ k_a \cdot p_{\mathrm{hdd}}(h) \cdot \mathbf{r}^{\mathrm{hdd}}(a) \cdot \frac{G_{\mathrm{s}}}{L^{*}(h)} \quad \text{FLUX},$$

which is linear in the declared disk footprint and in the window, and independent of the tenant's CPU and RAM reservation.

Proof sketch. By Table 40, from the entry into suspended the workload contributes no CPU or network load once stopped, and the only resource held is the volume, whose declared size is $\mathbf{r}^{\mathrm{hdd}}(a)$ per instance. Definition 16.3 prices that footprint at $p_{\mathrm{hdd}}\mathbf{r}^{\mathrm{hdd}}$ per default lifetime L^*, so a window of G_{s} blocks costs that fraction of it. The bound is an over-estimate because compute is not consumed during the window, because the surcharge terms of B do not apply, and because B is quoted per three instances while the bound charges it per instance. It is exact only in the sense that the network cannot re-let the disk while it is held. □

The operational reading is that the guarantee "a lapse does not delete your data" costs the network a disk reservation for a bounded, computable number of blocks after grace, and nothing else beyond the full reservation already conceded during grace. That is why the choice of G_{s} is a genuine parameter with a trade-off rather than a matter of taste (Section 16.2.7).

16.2.3 What would authorise a recurring charge

A recurring charge needs an authority that outlives the tenant's presence. Three constructions are available and their costs differ sharply.

1. **Custody.** The simplest construction, and the one InFlux's card-payment service uses today — in a form that does not even require a balance. It is not a protocol mechanism and the network takes no part in it (Property 16.21): the operator holds the tenant's identity and payment keys and fronts the on-chain payment (Section 16.2.1), so no authorisation object is needed at all, there being nothing to authorise when the authoriser already holds the key. `usersToExtend` separately lets one hardcoded address submit expire-only extensions on behalf of any owner [flux/ZelBack/config/default.js:229]. The cost is the trust surface enumerated in Section 16.2.1 and a key set whose compromise is a fleet-wide event.
2. **On-chain allowance.** A consensus-level primitive under which the tenant authorises a bounded draw. Not available on the Flux L1, for the reason given in Section 17; it would require an L1 consensus change, which places it in the same class as the header change of Section 22.
3. **Standing signature or session key.** The tenant pre-signs a bounded renewal authority: exactly the delegation object specified in Section 17, with $\mathcal{A}$ instantiated as a renewal service and Σ restricted to *renew this application at no more than this price*, under budget $\mathcal{B}$.

Recommendation: option (3), because it makes credits a special case of the delegation of Section 17 rather than a second, parallel mechanism, and because it is the only option that removes the custodian rather than relocating it. It inherits the enforcement limits of that section unchanged — in particular, a delegation is only as bounded as the party that checks the bound, and Section 17 states where that check would live and where it currently does not exist at all. One consequence is worth naming: because payer and owner are already independent in the deployed protocol (Property 16.20), option (3) plus a third-party payer would reproduce the card-payment convenience of Section 16.2.1 without reproducing its custody. The obstacle is the authorisation object, not the payment rail.

Remark 16.25 (Recurring authorisation is out of protocol scope — settled). This paper previously carried the choice among custody, an on-chain allowance and a standing signature as an open construction blocking a credits specification. It is settled, and settled by exclusion rather than by selection: **the protocol offers no automatic renewal at all**. A tenant renews by signing and paying for each period, or delegates that to a third-party operator who does it on their behalf [intent: design intake] (PLANNED).

The consequence is worth stating plainly rather than treating as a gap. There is no consensus-level allowance object, no session-key construction in the protocol, and no standing authority a

node evaluates. The recurring-payment problem is moved wholly into the service layer described in §16.2.1, where it is solved by custody — and where its trust surface is already enumerated. This is a smaller protocol, not a deferred one, and it removes an entire class of consensus change from the target architecture. What it costs is stated in Definition 17.16: an autonomous tenant that wants to renew without a human must hold a key and sign, or accept a custodian.

16.2.4 Metering, billing accuracy and dispute

The central design fact is easy to state and easy to underrate: **Flux prices reservation, not consumption.** The charge of Definition 16.3 is a function of the *requested* resource vector and duration, never of what the workload actually used. There is consequently no metering on the application path at all — no counter to read, no counter to attest, and no counter to dispute. This is the same reasoning that Section 7 applies one layer down: PNR, as planned, would pay all nodes through the PoN rotation rather than the hosting node precisely so that settlement carries no metering or attestation dependency (requirement P7 there). The two choices are the same choice made twice, and the consequence is worth stating plainly: reservation pricing eliminates an entire class of operator-versus-tenant dispute, because there is no usage record for the two parties to disagree about.

What remains disputable is a single binary fact — *did the application run* — and that fact is observable by third parties: FluxOS location records, FDM health checks, and the tenant's own probes all speak to it independently. The dispute surface is small by construction. This paper does not invent an arbitration mechanism, because none exists and none is required to make the above true.

Where metering does exist in the wider stack, it is already inexact, and any future consumption-priced product would inherit the problem.

Property 16.26 (FluxAI metering overdraft)**.** SHIPPED. FluxAI gates each request on `api_credit` > 0 and returns HTTP 402 otherwise, but deducts usage asynchronously: increments accumulate in Redis under a per-user key with a TTL of 300 s, and a separate scheduled command applies them to the durable balance inside a locked transaction and deletes the key [fluxai/api/v1/api_client.py:1210–1214] [fluxai/api/cache.py:9–32] [fluxai/api/management/commands/process_token_usage.py:41–99]. Two consequences follow. (i) A caller can consume beyond their balance for the whole interval between flushes, since the gate reads only the durable balance — a real overdraft window. (ii) If the flush interval ever exceeds the key TTL, the accumulated usage expires and is never billed at all. The bound on (i) is the flush cadence, which is *not* determinable from source: the dispatcher runs per minute, but the job's schedule is stored in the database rather than in the repository [doc: fluxai/setup/crons/django_cron:1].

Proof sketch. The admission check and the deduction read and write different stores, and the only synchronisation between them is the scheduled flush; therefore the interval between flushes bounds the divergence. The TTL is set on the accumulator key, not on the durable row, so expiry of the key discards the pending debit rather than deferring it. Both follow directly from the two cited code paths; neither requires an adversarial caller. □

The honest statement for a future consumption-priced product is therefore that the overdraft bound is a scheduling parameter, not a cryptographic one, and that it must be pinned in code before it can be quoted.

16.2.5 Service levels and refunds

No refund or credit mechanism exists in any repository in this corpus. The deployed behaviour on node failure is *redeployment*, not compensation: when an instance disappears, other nodes observe that the application is below its declared instance count and one of them installs it,

subject to the broadcast-and-wait collision rule [flux/ZelBack/src/services/appLifecycle/appSpawner.js:152–247,666–703]; a node that loses confirmation has its locations evicted from global state by every other node independently [flux/ZelBack/src/services/appMonitoring/nodeStatusMonitor.js:78–141]. Stated plainly: **the remedy is replacement, not refund.**

This is a coherent position rather than an omission, and the reason is the one above. An SLA credit is a function of measured availability, so paying one would require exactly the metering the design has deliberately avoided — and it would reintroduce the attestation dependency that Section 7 removed. Promising a refund path that nothing implements would be worse than saying so.

Remark 16.27 (No protocol availability remedy — settled). Whether the target architecture should acquire an availability remedy — a refund or credit when a node fails to serve a paid deployment — is settled: **it should not, at the protocol layer**. Any such remedy is a service-layer product, offered commercially by an operator if at all [intent: design intake] (PLANNED).

This is the consistent choice rather than a gap, and it follows from a fact this section already establishes: Flux prices *reservation*, not consumption (§16.2.7). A tenant buys reserved capacity, so there is no delivered-uptime quantity for a remedy to be computed against, and no third-party-observable availability signal exists from which one could be. Building that signal would mean building the attestation network first — which §25 classes as Engineering, and only for the TPM population — and then defining an oracle that decides when a deployment was unserved. Deciding not to have a protocol remedy avoids inventing both.

It composes with the settled positions on credits and renewal (Definition 16.28, Definition 16.25): convenience, custody and commercial guarantees all sit in the service layer, and the protocol keeps one job — deciding whether a deployment was paid for.

16.2.6 Invariants

Stated for all three objects — the protocol's direct settlement (Part A), the operator's deployed service (Section 16.2.1), and the unimplemented credits layer (Section 16.2.2). Where an invariant holds on one and not another, that is said. Only the first is a claim about Flux; the second is a claim about one company's product and binds nobody else who builds over Part A.

1. **Conservation.** b_u decreases only by renewals actually applied, and every debit corresponds to exactly one extension of exactly one e_a and, after h_{PA}, to exactly one PNR transaction on L1 (Requirement 16.11). No debit exists without an on-chain settlement; no settlement exists without a debit. PLANNED as stated, since b_u has no deployed referent (Property 16.19). On Part A it holds trivially, the payment and the extension being the same event; on the operator's service it holds in the degenerate form that each card charge is settled by exactly one transaction to the sink and no ledger stands between them (Construction 16.16, step 4).

2. **No silent expiry.** Every transition out of funded shall be observable by the tenant before it takes effect. On Part A the *expiry* half holds by Property 16.5 — e_a is public and computable in advance by anyone — but the *entry* half fails: an underpaid registration never becomes funded and the tenant is not told (Property 16.9). Requirement 16.10 is what would repair it, and it is a requirement on every path. On the operator's service the impending lapse *is* signalled, but by email only, on the operator's schedule, at most once per 24 h, with no in-band channel and no user override (Construction 16.16, failure mode (a)); that is weaker than this invariant asks and is recorded as such rather than counted as satisfying it.

3. **Data survives suspension.** PLANNED, and a property of Section 16.2.2 alone. No transition into suspended destroys tenant data; only evicted does, and only after $G_{\mathrm{g}} + G_{\mathrm{s}}$ blocks. By construction of Table 40; the cost is bounded by Property 16.24. The deployed

network has no suspension state at all, and application data at expiry is always deleted [intent: design intake]; this invariant must not be read across to it. The contrast is the point: on every deployed path expiry is destructive, so "data survives suspension" would be a guarantee a credits layer *introduces*, not one it preserves.

4. **Path independence.** For every deployment reachable through the operator's service or through a credits layer, the same deployment is reachable through direct payment with an explicit transaction. *Proof.* **Deployed case.** By the invariant of Construction 16.16, the facilitator's whole action reduces to (a) a signature over m_a under the key named in the specification's `owner` field and (b) one transaction paying $\mathcal{P}$ to the sink carrying $H(m_a)$ — that is, exactly steps 2 and 3 of Construction 16.7. A tenant holding their own key performs both and obtains the identical e_a by Definition 16.4. **Specified case.** A credit-path renewal at height h debits $\mathcal{P}^{\text{ren}}(a, h)$ and extends e_a by d_a; by Requirement 16.11 it does so through one L1 settlement of that amount, which the tenant may broadcast themselves at the same height for the identical e_a, since the promotion test depends only on (v^{obs}, h, a) and on no property of the payer (Property 16.20). Hence the reachable sets coincide in both cases. □ **Neither the operator's service nor a credits layer adds capability**, and a user who uses neither loses none. Together with Property 16.21 — the network cannot see the service — this is what places the service strictly on top of Flux rather than inside it: it adds no reach downward and imposes no requirement upward. It is the load-bearing item in this list, and the reason the framing of Section 16.2.1 is a description rather than a defence.

5. **Monotone expiry.** e_a is non-decreasing while $\text{st}(a, h) \neq$ evicted. Immediate from Table 40, in which the only writes to e_a are increments. Note that the deployed system does *not* satisfy the analogous property under the expire-only update path, where the `usersToExtend` key, held by the operator of `appsmonitor`, can submit a *reducing* expire value with no "new expire must be larger" check [appsmonitor/fixOverExtendedAppExpiry.js:1–24]; that is an operational tool, and any credits implementation must not inherit the exemption.

16.2.7 Open parameters

These are service-layer parameters by Definition 16.28: an operator chooses them, and nothing in consensus depends on them. The paper nonetheless recommends values, because an operator building this needs a defensible starting point and because the trade-offs are the same whoever implements them. Each was proposed by this paper and has been confirmed by the author (Table 41).

Table 41: Lifecycle parameters for a credits operator. Proposed by this paper and **confirmed by the author** [intent: design intake]. Block counts assume the 30 s post-PoN target, i.e. 2,880 blocks per day [fluxd/src/chainparams.cpp:105]. PLANNED.

Parameter	Domain	**Recommended**	Reasoning
G_{g} grace	$[240, 2880]$	2,880 (24 h)	covers a full business day
G_{s} suspension	$[2880, 43200]$	20,160 (7 d)	one week of stored, unpaid data
Δ_{pre} pre-auth lead	$[20, \infty)$	120 (1 h)	$\gg$ confirmation depth
$\mathcal{P}^{\text{res}}$ resumption	$[0, \infty)$	**one renewal period**	prices the subsidy exactly

Why these values. $G_{\text{g}} = 2{,}880$ blocks, one day, is chosen so that a funding failure occurring at any hour is recoverable within one business day without the tenant losing service. The shorter end of the domain (2 h) converts an overnight bridge delay into a suspension, which is the routine failure this window exists to absorb; the cost of the longer setting is one day of unpaid load

at full occupancy, i.e. G_g/L^* of one renewal period, which lies outside the disk-only bound of Property 16.24.

$G_s = 20{,}160$ blocks, seven days, is the willingness-to-subsidise statement. The question this parameter really asks is how long the network stores data nobody is paying for. A week is long enough that a tenant returning from an outage, a holiday or a payment-method failure finds their deployment intact, and short enough that the storage subsidy stays bounded at one week per evicted application rather than accumulating indefinitely. Note that the fifteen-day end of the domain doubles the subsidy to buy little: recovery that has not happened within a week is overwhelmingly abandonment rather than delay.

$\Delta_{pre} = 120$ blocks, one hour, sits an order of magnitude above the binding constraint. The floor is confirmation depth plus specification propagation; propagation in Construction 16.7 is seconds and confirmation dominates. One hour leaves room for a congested mempool and a re-sent transaction without approaching the upper risk, which is that a renewal authorised too far ahead is priced against a stale segment of Definition 16.3.

$\mathcal{P}^{res}$ equal to *one renewal period at the application's own price* is the only anchor that is correct by construction rather than by taste. Property 16.24 prices the suspension window as a subsidy; charging exactly one period on resumption returns that subsidy from the tenant who consumed it, scales automatically with the resources the application actually reserved, and needs no separate schedule to maintain. Zero would make the suspension window free cold storage, which is the abuse the property identifies; a flat fee would misprice small and large applications in opposite directions.

Remark 16.28 (Where a deposited balance lives — settled: nowhere on the chain). The location of a deposited balance was the architectural choice this section could not make. It is now settled, and the answer is that the object is not a protocol object at all. Credit balances would be held **off-chain, per user, in the backend of an operator outside the protocol** — the team names InFlux Technologies, the company that maintains the Flux products, as such an operator. That service is account-based: a user may sign in with SSO or email, the backend derives a ZelID for them, they are logged in under it, and a planned credits layer would add a per-user balance which the user tops up and authorises spend from [intent: design intake] (PLANNED).

Three consequences follow, and the paper states them rather than the convenience.

1. **Invariant 1 is enforced by the operator's bookkeeping.** Not by consensus, not by a node quorum, not by the tenant's device. A credit balance is a claim against an operator, redeemable at that operator's discretion and subject to its solvency.

2. **The chain cannot see it, and by Property 16.21 does not need to.** What reaches the chain is an ordinary payment for a deployment, indistinguishable from one a tenant signs directly. Credits are therefore invisible to PNR, to the fee pool, and to every property proved in §7.

3. **The specification below is retained as a description of the service-layer object**, not as a protocol proposal. Its state machine is what such an operator must implement correctly; it is not what nodes evaluate.

Remark 16.29 (SPEC v9 replaces `blocksLasting` with `duration` — settled from source). This was carried as an open question. It is answered by the schema itself: in the SPEC v9 schema `duration` is a **top-level required** property — `{"type":"integer", "minimum":1}`, documented as "requested deployment duration in seconds (first-class in v9; the foundation the subscription and auto-renew work builds on)" — and **`blocksLasting` does not appear in the spec v9 schema at all**, nor does `expire` [flux@feat/appspec-v9:ZelBack/src/services/appRequirements/schemas/v9.json].

Two consequences follow. *First*, $L^*(h)$ in Definition 16.3 becomes a unit conversion rather than a fork-dependent constant, and the post-PoN 4× recomputation of Definition 16.4 becomes dead code on the SPEC v9 path — lifetime is expressed in seconds, so it no longer moves when

block spacing changes. That is a simplification, and it removes a class of error the 4× branch has already caused once (§7).

Second, and this is the caveat that keeps the claim honest: the field is declared and required but **not yet consumed**. A search of `appRequirements/` and `appLifecycle/` on the same branch finds `duration` only in comments; `blocksLasting` is still what the deployed v8 install path reads [flux@feat/appspec-v9:ZelBack/src/services/appLifecycle/appInstaller.js]. So SPEC v9 *specifies* the replacement and does not yet *implement* it — the same declared-but-inert pattern §11 documents for `paymentMemo`, `referralCode` and `machineType`, and the reason Definition 10.2 constrains $h^\star$ to follow the implementation rather than precede it.

16.2.8 Comparison

Akash escrows a lease account and streams payment per block, closing the lease when the escrow empties [4]; that is the closest analogue to the credits layer specified in Section 16.2.2, with the balance on-chain and the state machine in consensus rather than in an orchestrator or in a custodian's database. Its answer to "where does the balance live" is the opposite of the one settled here — off-chain, in an operator's backend (Definition 16.28) — and it pays for that answer with a consensus-level account primitive the Flux L1 does not have. **Filecoin** deals prepay and collateralise, with provider slashing for failure [41]; Flux has no equivalent slashing on the application path, which is precisely why its remedy is replacement rather than compensation — there is no bonded party to take value from. **Hyperscaler billing** is post-paid, consumption-metered, and correspondingly dispute-heavy; reservation pricing is the deliberate opposite, trading utilisation efficiency for verifiability, and Property 16.26 is a small demonstration of what the metered path costs even inside Flux's own stack. **Lightning invoices** are the closest match to Part A's "the payment is the entitlement" property [39]: a payment preimage is the proof of settlement and no account mediates it. CumulusVPN's memo-derived entitlement is effectively an on-chain invoice with a public settlement record — which is exactly the property that makes it work without a server and exactly the property that makes it the problem statement for Section 21.

17 Agent identity and delegated authority

Two different things are called an *agent* in this paper, and the difference decides what each one needs. The first is the human-facing assistant of §19: it turns a sentence into an application specification, renders it for review, and hands the signing to the person's own device. That assistant holds no key and needs no delegated authority, because the human's signature *is* the authorisation; its open problem is reviewability, not delegation. The second is autonomous software — a process that provisions capacity, pays for it, renews it and retires it on its own schedule, with no person in the loop at any step, on timescales where waiting for a human tap is itself the failure. Only the second needs a delegation primitive. This section is about the second, exclusively, and the two must not be allowed to blur: a property proved about a keyless assistant says nothing about a process that holds a signing key, and the deployed system already contains one of each.

The section opens on a statement about what is deployed, and it is not a comfortable one. **Authority in the deployed system is bounded by action and unbounded by value.** There exists, today, a real scoped and revocable delegation of *actions*; there exists no bound whatsoever on *value*; and there is already in production a bearer credential that confers unbounded signing authority over a user's Flux identity. What follows is therefore a specification for a capability that already ships without one, not a description of a mechanism the network has.

17.1 What exists today

Table 42 is the baseline. Every row is SHIPPED and carries code provenance; the one row that records an absence carries the search that established it.

Capability	What is deployed
Human key custody, UTXO	SHIPPED 2-of-2 ECDSA over secp256k1 in a sorted `OP_CHECKMULTISIG` script. The wallet builds and partially signs; the Key device signs the remaining input, finalises, and broadcasts directly to a chain backend. The relay never sees a complete UTXO transaction. [ssp-wallet/src/lib/constructTx.ts:242–298], [ssp-key/src/lib/constructTx.ts:157–169]
Human key custody, EVM	SHIPPED Two-round MuSig-style Schnorr aggregation, verified on chain inside an ERC-4337 smart account through the `ecrecover` trick, at an address made identical on all seven chains of the deployment manifest by `CREATE2`. [account-abstraction/contracts/schnorr/Schnorr.sol:7–26], [account-abstraction/contracts/MultiSigSmartAccount.sol:145–159], [doc: account-abstraction/deployments.md:5–13]
Human key custody, Solana	SHIPPED 2-of-2 Ed25519 over a program-derived vault. Registration is permissionless — the vault address is a PDA over (sorted members, threshold) — and the threshold is checked only at execution. [Solana-Multisig/programs/solana-multisig/src/lib.rs:89–152,468–471], [ssp-wallet/src/lib/wallet.ts:489]
Relay capability	SHIPPED The relay holds no private key and cannot sign on any chain. It can read, store and *withhold* every payload it forwards. Several coordination endpoints and both socket `join` paths still accept unauthenticated requests, marked in the shipped source as `TODO: Make auth required after clients are updated`. [ssp-relay/src/lib/socket.ts:31–38], [ssp-relay/src/routes.ts:253–301]
Action delegation	SHIPPED The FluxEdge API issues bearer tokens whose privileges are enumerated per key (`start_rental`, `stop_rental`, `start_deployment`, `stop_deployment`, `get_balance`) and enforced per route; $2\,\mathrm{s}^{-1}$ per key and per IP; five rate-limit violations inside 5 min revoke the key automatically and notify its owner. [pouw-backend/rest_api/ratelimit.go:41–45,203–222,375–390]
Value delegation	*Absent.* No session key, spending cap, allowance, time-lock, guardian or social-recovery construct exists in the wallet, key, relay, account-abstraction or Solana-multisig code. [ssp-wallet, ssp-key, ssp-relay, account-abstraction, Solana-Multisig:targeted search, no match]
Policy engine	SHIPPED The enterprise M-of-N vault has priority-ordered rules with actions `allow`/`block`/`require_approval`/`timelock`, schedule windows, rolling 24 h/7 d/30 d transaction-count velocity limits, whitelists and named approval groups. It decides whether a *proposal* needs approval; it never signs, and every allowed proposal still requires the vault's device signatures. [ssp-relay-enterprise/src/services/policyEngineService.ts:9–45], [ssp-relay/src/routes.ts:644–649]
Gas sponsorship hook	SHIPPED `IPaymaster` and `paymasterAndData` are fully wired through the account; no contract in the repository implements a paymaster. [account-abstraction/contracts/erc4337/interfaces/IPaymaster.sol:1–49]
De-facto delegation, in production	SHIPPED A Firebase bearer token authorises remote signing at `service.fluxcore.ai/api/signMessage`, which signs on the user's behalf. The shipped MCP tools `deploy_app`, `update_app` and `renew_app` use exactly this path; their own description is "Revalidate, Firebase-sign, register". [deploy-with-git-frontend/server/flux/fluxSession.js:60–73], [deploy-with-git-frontend/server/mcp/createServer.js:174–266]

Table 42: Authority primitives in the deployed system. SHIPPED rows carry code provenance; the absence row carries the search that established it.

Three notes on that table, because each of them constrains what the rest of the section may claim.

First, the EVM contracts are reported audited by Halborn with the engagement finalised in February 2025, with findings in code since removed [doc: account-abstraction/aa-schnorr-multisig-sdk/README.md:76–83]; that is documentation provenance and is not independently verified here. In-repo deployment artifacts were observed for four of the manifest's networks, the fifth directory being a local `hardhat` network [account-abstraction/deployments/:directory listing], so "identical on seven chains" is a manifest claim, not an artifact count.

Second, the FluxEdge token model is a genuine bounded, revocable delegation, and the section must not pretend otherwise. It has an enumerated privilege set, a per-key rate limit, and automatic revocation on abuse with the owner notified. Two gaps keep it from being sufficient for an autonomous agent: key issuance is not part of the documented REST surface, so a human must mint a key through the web dashboard before an agent can use the API at all [doc: pouwbackend/rest_api/v1/openapi.yaml:61]; and the privileges it scopes are actions on a custodial credit balance, not signatures on a chain.

Third — and this is the row the whole section exists because of — the deployed delegation on the FluxCore path is unbounded. A holder of a valid Firebase bearer token can construct, sign and submit a registration or update message end to end, with no client-side wallet interaction. There is **no per-transaction cap, no rate limit, no cumulative budget, no scope over what may be deployed, and no revocation short of invalidating the token**. The bound on a compromised token today is whatever the underlying identity can authorise, which is everything that identity can authorise. The local-signing paths (SSP extension, ZelCore deep link) do not have this property, because the key never leaves the user's device [deploy-with-git-frontend/src/services/deployService.js:697–772]; the property is per-path, and §19 states it per-path.

17.2 The two authorities

The structural claim of this section is stated early because it organises everything after it. A deployment involves *two separable authorities*, and they are enforced in different places by different parties, so they have different feasibility.

- **Deployment authority (A)** — what may run, at what resource vector, in how many instances, where, and under whose name. It is enforced by every FluxOS node, at the moment the node validates the owner signature over an application specification.
- **Spending authority (B)** — how much FLUX may move, to whom, and how fast. It is enforced by Flux L1 consensus rules, which are those of a Zcash-derived UTXO chain.

Table 43 sets them side by side. Conflating them is what makes agent delegation look uniformly hard. Separated, (A) is tractable with the machinery that already runs and (B) is not tractable at all on the Flux L1 without either a new trusted party or new consensus state. The rest of this section treats them separately and never averages them.

	(A) Deployment authority	**(B) Spending authority**
Object bounded	the application specification an agent may submit	the value an agent may move
Enforcement point	FluxOS specification validation	Flux L1 script validation
Enforcing party	every validating node, independently	consensus
Inputs available at enforcement	the message, the certificate, chain state at h	the spending transaction and the outputs it consumes, only
Can enforce history-dependent limits?	yes — a node sees the chain	no — see Definition 17.11
Feasibility class (§25)	Extension (row 12)	Research (row 13)

Table 43: The two authorities, their enforcement points, and why they have different feasibility.

17.3 The delegation certificate

Everything from here to Section 17.10 is PLANNED. Nothing in it is deployed, no part of it is scheduled to a height, and no claim below should be read in the present tense as a statement about the network.

Definition 17.1 (Delegation certificate)**.** Let $\mathcal{U}$ be the human principal, holding the 2-of-2 key set (k_1, k_2) split across the wallet and key devices of Table 42. Let $\mathcal{A}$ be an autonomous agent

holding a single key pair $(sk_{\mathcal{A}}, pk_{\mathcal{A}})$ generated on the agent's own host. A *delegation certificate* is the tuple

$$\mathsf{cert} \;=\; (pk_{\mathcal{A}},\ \Sigma,\ \mathcal{B},\ [t_0, t_1],\ \mathsf{rev},\ \nu), \qquad \sigma_{\mathsf{cert}} \;=\; \mathrm{Sign}_{k_1 \wedge k_2}(H(\mathsf{cert})),$$

where Σ is the scope of permitted deployments (Definition 17.3); $\mathcal{B} = (b_{\mathrm{tx}}, b_{\mathrm{rate}}, b_{\mathrm{total}}) \in (\mathbb{R}_{\geq 0}\,\mathrm{FLUX}) \times (\mathbb{R}_{\geq 0}\,\mathrm{FLUX\,per\,block}) \times (\mathbb{R}_{\geq 0}\,\mathrm{FLUX})$ is the intended value bound, being respectively a per-transaction cap, a rate limit and a cumulative budget; $[t_0, t_1] \subset \mathbb{N}$ is a validity window in block heights with $t_0 < t_1$; rev is a revocation anchor, an outpoint on the Flux L1 controlled by $\mathcal{U}$ and unspent at t_0 (Definition 17.7); and $\nu \in \mathbb{N}$ is a monotone certificate counter, so that re-issue to the same $pk_{\mathcal{A}}$ is unambiguous and an older certificate can never be presented as a newer one. H is the canonicalising hash of the application specification fingerprint recipe (§10) and $\mathrm{Sign}_{k_1 \wedge k_2}$ denotes the full 2-of-2 ceremony, not a single-device signature.

Assumption 17.2 (Model, from §2)**.** The specification of this section is stated against the following, each of which is fixed in the system model of §2 and not re-argued here.

(A1) *Principal custody.* Producing σ_{cert} requires both k_1 and k_2. Compromise of one device does not yield a certificate.

(A2) *Node validation.* Every FluxOS node independently validates owner signatures over application specification s and has read access to `fluxd` chain state at the height it validates. This is the deployed behaviour described in §9 and §10.

(A3) *Chain state.* The Flux L1 is a Zcash-derived UTXO chain with no covenant opcodes, no transaction introspection, and no account abstraction. Reorganisations are bounded by `MAX_REORG_LENGTH` = 40 blocks [fluxd/src/main.cpp:8983–8988], with the historical override to 5,000 over heights [2,020,000, 2,025,000] noted in §4.9.

(A4) *No escrow.* $sk_{\mathcal{A}}$ is generated on the agent's host and is never transmitted. Any variant in which a service can sign for $\mathcal{A}$ is a custody design and is labelled one.

(A5) *Adversary.* The adversary $\mathcal{Z}$ of §2 may compromise the agent host, may operate or coerce the relay, and is computationally bounded so that signature forgery is infeasible. $\mathcal{Z}$ does not hold k_1 and k_2 simultaneously.

17.4 Deployment authority

PLANNED. Deployment authority is the half of the problem that composes with machinery the network already runs.

Definition 17.3 (Scope and containment)**.** The scope is the tuple

$$\Sigma \;=\; (\mathcal{N},\ \mathbf{r}^{\max},\ k^{\max},\ \mathcal{I},\ d^{\max},\ \mathcal{G}),$$

where $\mathcal{N}$ is a finite set of permitted application names; $\mathbf{r}^{\max} \in \mathbb{R}^3_{>0}$ is a componentwise ceiling on the resource vector $\mathbf{r} = (\mathrm{cpu}, \mathrm{memory}, \mathrm{rootFs})$; $k^{\max} \in \mathbb{N}$ bounds the instance count k; $\mathcal{I}$ is a finite set of permitted container images, each identified by content digest and not by tag; $d^{\max} \in \mathbb{N}$ bounds the requested duration d in blocks; and $\mathcal{G}$ is a set of permitted geolocation constraints in FluxOS's `acXX/a!cXX` form. A specification s is *contained* in Σ, written $s \in \Sigma$, iff its name lies in $\mathcal{N}$, its resource vector satisfies $\mathbf{r}(s) \leq \mathbf{r}^{\max}$ componentwise, its instance count satisfies $k(s) \leq k^{\max}$, every image digest named by s lies in $\mathcal{I}$, $d(s) \leq d^{\max}$, and the geolocation constraints of s lie in $\mathcal{G}$. Containment is a predicate on s and Σ alone: it reads nothing else.

Definition 17.4 (Acceptance)**.** A node evaluating an agent-submitted specification s, carrying an agent signature σ_s, a certificate cert and its principal signature σ_{cert}, at height h, computes

$$\begin{aligned}\mathrm{Accept}(s,h) \;\equiv\; \mathrm{Verify}_{pk_{\mathcal{A}}}(H(s),\sigma_s) \;\wedge\; \mathrm{Verify}_{\mathsf{owner}}(H(\mathsf{cert}),\sigma_{\mathsf{cert}}) \\ \wedge\; t_0 \le h \le t_1 \;\wedge\; s \in \Sigma \;\wedge\; \neg\,\mathrm{Revoked}(\mathsf{cert},h),\end{aligned}$$

where owner is the owner identity already carried by the specification and Revoked is Definition 17.7.

Algorithm 3: AcceptDelegatedSpec. **Failure mode:** every branch fails closed. A node that cannot evaluate $\mathsf{rev} \in U(h)$ — for instance one whose local state does not reach the anchor's height — rejects rather than accepts, so a partially-synced node under-accepts and never over-accepts. PLANNED

Input: message $m = (s, \sigma_s, \mathsf{cert}, \sigma_{\mathsf{cert}})$; local height h; local UTXO set $U(h)$; owner public key set for owner
Output: ACCEPT or REJECT

```
// Invariant: the result is a deterministic function of (m, h, U(h))
   alone. No network fetch, no registry, no service.
```

if $\neg\,\mathrm{Verify}_{\mathsf{owner}}(H(\mathsf{cert}),\sigma_{\mathsf{cert}})$ **then return** REJECT
if $h < t_0 \vee h > t_1$ **then return** REJECT

```
// revoked, or anchor never existed -- both fail closed
```

if $\mathsf{rev} \notin U(h)$ **then return** REJECT
if $\neg\,\mathrm{Verify}_{pk_{\mathcal{A}}}(H(s),\sigma_s)$ **then return** REJECT
if $\mathsf{owner}(s) \neq \mathsf{owner}(\mathsf{cert})$ **then return** REJECT
foreach *clause c of Definition 17.3* **do**
  if *s violates c* **then return** REJECT
return ACCEPT

Property 17.5 (Local enforceability)**.** $\mathrm{Accept}(s,h)$ is computable by any validating node from the submitted message, the node's own chain state at h, and nothing else. It requires no consensus change and introduces no trusted party beyond those §2 already names.

Proof. Take the conjuncts of Definition 17.4 in turn. The two Verify terms are signature checks over data carried in the message, against $pk_{\mathcal{A}}$ (carried in cert, which is itself signed) and against the owner key set the node already resolves in order to validate any application specification (A2). The window test compares h, which the node holds by definition, against two integers in the message. Containment $s \in \Sigma$ is by Definition 17.3 a predicate on s and Σ alone. Revoked is by Definition 17.8 a membership test in the node's own UTXO set at h. Every input is therefore either in the message or in chain state the node already maintains, which establishes the first claim. For the second: the check is performed at the same point, by the same code path, and on the same inputs as the owner-signature validation FluxOS already performs (A2), so it changes no consensus rule and adds no party — the only new authority is $\mathcal{U}$'s own key set, which already authorises the specification today. □

Deployment authority would fit SPEC v9 cleanly: the certificate is one additional specification field, validated where it is enforced, in a version that is already adding fields (§10). That is the recommended placement, and it is why §25 classes this as Extension (row 12) rather than Engineering.

Remark 17.6 (Digest pinning is necessary, and against a generic runner it is not sufficient). $\mathcal{I}$ is defined over digests rather than tags for a concrete reason. The deployed tooling emits `runonflux/orbit:latest`, an unpinned mutable tag, into every generated specification [deploy-with-git-frontend/src/services/deployService.js:462,453–490]. A scope that permits a mutable tag permits

whatever that tag resolves to at node pull time, which is to say arbitrary code, and the delegation is then vacuous: the acceptance predicate would be satisfied by a specification whose behaviour the principal never bounded.

Pinning the digest closes that hole and opens a second one, which the specification must name rather than hide. The pinned image in question is a *generic runner*: the Orbit container clones whatever repository `GIT_REPO_URL` names and builds and runs it [orbit/entrypoint.sh:1–9,29]. Against such an image, fixing the digest fixes the *runtime* and not the *workload*, so an agent whose scope permits one pinned generic-runner digest can still cause arbitrary code to execute by varying an environment parameter. Bounding that requires Σ to constrain the environment-parameter set as well as the image set, and the scope tuple of Definition 17.3 does not. The requirement is stated; the construction is not.

Decided. The scope tuple gains a seventh component: an allowed-key set for the environment-parameter map, with values unconstrained. Rationale: the keys are what a node can check cheaply and deterministically at deployment, and they are what carries the risk — an agent that can set an arbitrary environment key can often reach configuration the owner never intended. Constraining values as well would require the verifier to understand each application's semantics, which no node can do. [inferred] — proposed by this paper on the same basis as Definition 17.15.

17.5 Revocation

Definition 17.7 (Revocation anchor)**.** rev is an outpoint on the Flux L1 spendable by $\mathcal{U}$ and required to be unspent at t_0. The certificate is revoked at the first height at which that outpoint is spent:

$$\mathrm{Revoked}(\mathsf{cert}, h) \;\equiv\; \exists\, h' \le h \;:\; \mathsf{rev} \in \mathrm{Spent}(h').$$

Lemma 17.8 (UTXO-set formulation)**.** *Let $U(h)$ be the unspent-output set at height h. Given the issuance precondition that $\mathsf{rev} \in U(t_0)$, and given that a spent outpoint cannot be recreated,*

$$\mathrm{Revoked}(\mathsf{cert}, h) \;\equiv\; \mathsf{rev} \notin U(h) \qquad \textit{for all } h \ge t_0.$$

Proof. If rev is spent at some $h' \le h$ then it leaves U at h' and, by non-recreatability, does not re-enter, so $\mathsf{rev} \notin U(h)$. Conversely, if $\mathsf{rev} \notin U(h)$ then, since $\mathsf{rev} \in U(t_0)$ and $t_0 \le h$, the only way to leave U between t_0 and h is to be consumed by a transaction, i.e. spent at some $h' \le h$. □

The lemma matters for implementation rather than for theory: it turns revocation into a UTXO-set membership test, which is exactly the state a pruned node retains (§8), so revocation checking does not force any node to keep history it would otherwise discard. A node that finds $\mathsf{rev} \notin U(h)$ without being able to distinguish "spent" from "never existed" rejects, which is the fail-closed direction.

Remark 17.9 (Outpoint non-recreatability — verified). Definition 17.8 depends on a spent collateral outpoint not being recreatable, which was previously assumed from inherited Bitcoin behaviour rather than checked. `fluxd` enforces BIP-30 unconditionally in `ConnectBlock`: a block containing a transaction whose txid already exists with unspent outputs is rejected as `bad-txns-BIP30`, with no height gate and no activation flag SHIPPED [fluxd/src/main.cpp:3868–3875].

One nuance is worth stating precisely rather than glossing, because the rule permits overwrite when the prior coins are *completely* spent. For a non-coinbase outpoint, recreating the same txid requires replaying a transaction with identical inputs, and those inputs are by construction already spent, so the recreation cannot be built. The lemma therefore holds for non-coinbase anchors on the deployed rules.

Property 17.10 (Monotone revocation, and its latency)**.** Once $\mathrm{Revoked}(\mathsf{cert}, h)$ holds it holds at every $h' > h$, except under a reorganisation deeper than $h' - h$. Revocation takes effect after one confirmation of the spending transaction plus specification-propagation time.

Proof. Monotonicity is immediate from Definition 17.7: the existential quantifier ranges over a set that only grows with h. The exception is equally immediate: if a reorganisation removes the block at h' containing the spend, the existential no longer witnesses, and the certificate is live again on the new chain. By (A3) this is bounded by `MAX_REORG_LENGTH` $= 40$ blocks in normal operation. Latency: the predicate becomes true for a node at the height at which that node accepts the block containing the spend, so one confirmation, plus the time for a specification submitted before that height to stop propagating — during which a node that has not yet seen the spending block will still accept. □

At the tooling's figure of 2,880 blocks per day [deploy-with-git-frontend/src/services/deployService.js:105], one confirmation is approximately 30 s [inferred]. The honest statement of the guarantee is therefore *revocation is effective within one confirmation, and is reversible for up to* 40 *blocks thereafter*, and a principal revoking in response to a live compromise should assume the agent can act for at least that long.

Two weaker designs were considered and rejected. *Expiry alone* — relying on t_1 and issuing short certificates — cannot stop an in-progress compromise: between detection and t_1 the agent retains full scope, and shortening t_1 to make that window small multiplies re-issue traffic against a 2-of-2 ceremony that requires two human devices. *A revocation list served over HTTPS* is operationally trivial and reintroduces a Flux-operated dependency in the validation path of every node, which is precisely the class of dependency §12 already inventories too many of; it would also make revocation checking fail-open on service outage unless every node hard-fails on an unreachable list, which is worse.

17.6 Spending authority, and the honest result

Proposition 17.11 (Budget reduces to balance)**.** *On the Flux L1 as deployed, delegating spending authority to a key not held by $\mathcal{U}$ bounds $\mathcal{A}$'s spending by exactly the balance of the outputs that key can spend, and by nothing else. Neither b_{rate} nor b_{total} of Definition 17.1 is enforceable by consensus.*

Proof sketch. Enforcing b_{rate} or b_{total} requires a validation predicate that is a function of the agent's *transaction history* at spend time: whether the agent has already spent x in the preceding n blocks, or y in total since t_0. Script validation on a Bitcoin-derived chain is a pure function of the spending transaction and the outputs it consumes; a script sees its own input, its own transaction, and nothing about which other outputs the same key has spent. By (A3), Flux inherits this via Zcash with no covenant opcode, no introspection opcode and no account abstraction, so no script can express the predicate, and no consensus rule in `fluxd` supplies it outside script. What remains enforceable is the trivial bound: a key can spend at most the outputs it can spend. Hence the only real bound is the funded balance, which is b_{tx} collapsed into b_{total} and both collapsed into the balance. □

Definition 17.11 is the reason §25 classes bounded agent spending as Research (row 13) and not as an increment. The deployable construction is correspondingly modest, and is presented as such.

Construction 17.12 (Funded allowance)**.** PLANNED. $\mathcal{U}$ funds an address controlled solely by $sk_{\mathcal{A}}$ with b_{total}, and tops it up on a schedule of the principal's choosing. The agent's maximum spending loss is that balance. Revocation of spending authority is a sweep: $\mathcal{U}$ spends the funded outputs back to an address of their own. The sweep races any in-flight agent transaction, and the race is bounded by one confirmation — the same bound as Definition 17.10, and for the same reason.

Three richer constructions exist. They are given as the design space, with costs, not as a recommendation.

1. **2-of-3 with a policy co-signer.** $\mathcal{A}$ holds one key, $\mathcal{U}$ a second, and a policy service the third; the service co-signs only transactions inside $\mathcal{B}$. This is enforceable with what runs today, and the enterprise policy engine of Table 42 is the obvious substrate — it already evaluates velocity limits and schedule windows. *Cost: a new trusted party.* The co-signer cannot steal, since it is one key of three, but it can refuse; and for an autonomous agent, refusal is a liveness failure that arrives precisely when the agent most needs to act. A design whose safety property is "a third party can stop you" has, for an unattended process, converted a theft risk into an availability risk rather than removing a risk.

2. **Pre-signed rate-limited chain.** $\mathcal{U}$ pre-signs a sequence of payments with increasing `nLockTime`, handing $\mathcal{A}$ a spend schedule it cannot accelerate. No new trust is introduced and nothing new is required of consensus. *Cost:* the amounts and the schedule are fixed at issue, which does not fit demand-driven provisioning — an agent that needs 40 % more capacity at hour three cannot buy it — and revocation requires $\mathcal{U}$ to broadcast a conflicting spend, which races the agent in the mempool.

3. **Consensus-level delegation registry.** A new transaction type binding $pk_{\mathcal{A}}$ to a budget held as consensus state, decremented as the agent spends. This is the clean answer: it makes $\mathcal{B}$ real, it needs no third party, and it makes b_{rate} and b_{total} the enforceable objects Definition 17.11 says they cannot otherwise be. *Cost:* a consensus change and new consensus state, on a chain that has just completed one major transition — not cleanly (§4.9) — and has three further changes queued (§8).

The paper's position is: Definition 17.12 is the deployable baseline; (c) is the principled endpoint; the choice between them is not made here.

Decided. Definition 17.12 only — the funded-allowance construction, with no co-signer, pre-signed chain or consensus registry. This follows directly from Definition 17.11: on this chain the only enforceable bound is the balance of the key the agent holds, so the three richer endpoints buy expressiveness the consensus layer cannot honour. A co-signer additionally reintroduces a trusted third party, which is what the agent-native claim exists to remove (Definition 17.16).

The asymmetry with the EVM path should be stated precisely rather than smoothed over. On the parallel-chain accounts, (B) is not hard: the ERC-4337 account already in production is a program with persistent state, and it admits a session-key validator module and the sponsorship hook that `IPaymaster` already declares. The account as deployed does not use it — `_validateSignature` returns valid-with-no-time-bound or failure, never packing `validUntil`/`validAfter` into `validationData` even though the base class supports it [account-abstraction/contracts/MultiSigSmartAccount.sol:145–159] — but the substrate is there. The Flux L1's is not, and no amount of design closes that gap.

17.7 Invariants

The specification is required to maintain the following. PLANNED throughout.

1. **No escrow.** $sk_{\mathcal{A}}$ is generated on the agent's host and never transmitted (A4). Any variant in which a service holds or can reconstruct $sk_{\mathcal{A}}$ is a custody design and must be labelled one — which is exactly what the FluxCore signing path is, and is why Table 42 names it.

2. **Certificate authenticity.** Every accepted agent action traces to exactly one certificate signed by the full 2-of-2 principal key set. Single-device issuance is forbidden: a delegation issued more weakly than the custody it delegates from would make the delegation, not the custody, the attack surface.

3. **Monotone revocation.** Definition 17.10, with the reorg caveat stated and not hidden.

4. **Scope containment.** $s \in \Sigma$ is evaluated by every validating node from the specification and the certificate alone, with no external lookup (Definition 17.5).

5. **Loss bound.** Definition 17.13 below.

Proposition 17.13 (Loss bound under full agent compromise)**.** *Let $\mathcal{Z}$ fully compromise $\mathcal{A}$ at height $h_c \in [t_0, t_1]$, and let $h_r \geq h_c$ be the height at which $\mathcal{U}$'s revocation confirms. Under invariants 1–4 and Definition 17.12, the loss is bounded by*

$$\mathrm{Loss}(\mathcal{Z}) \;\leq\; \mathrm{bal}(sk_{\mathcal{A}}, h_c) \;+\; \mathcal{P}^{\max}(\Sigma,\, \min(t_1, h_r) - h_c),$$

where $\mathrm{bal}(sk_{\mathcal{A}}, h_c)$ *is the balance of outputs $sk_{\mathcal{A}}$ can spend at h_c and $\mathcal{P}^{\max}(\Sigma, \Delta)$ is the maximum cost of deployments admissible under Σ over Δ blocks. In words:* the maximum loss from full compromise is the balance the agent's key controls, plus the cost of any in-scope deployment for the remaining validity window.

Proof. By (A4) and invariant 1, $\mathcal{Z}$ obtains $sk_{\mathcal{A}}$ and nothing else; by (A5) $\mathcal{Z}$ cannot produce σ_{cert} for any certificate $\mathcal{U}$ did not sign, and cannot forge σ_s for a different key. So $\mathcal{Z}$'s capabilities are exactly $\mathcal{A}$'s. Spending: by Definition 17.11 and Definition 17.12, $\mathcal{Z}$ can move at most the outputs $sk_{\mathcal{A}}$ controls, i.e. $\mathrm{bal}(sk_{\mathcal{A}}, h_c)$, plus any top-up $\mathcal{U}$ makes after h_c — which the statement excludes by evaluating the balance at h_c and which a principal responding to a detected compromise will not make. Deployment: by Definition 17.4 every accepted specification satisfies $s \in \Sigma$ and $t_0 \leq h \leq t_1$ and $\neg$Revoked, so the admissible set is bounded by Σ and the window closes at $\min(t_1, h_r)$ — at t_1 by expiry, at h_r by Definition 17.10. Summing the two disjoint capabilities gives the bound. □

Two qualifications travel with Definition 17.13 and must not be dropped when it is quoted. First, the deployment term is only as tight as Σ, and by Definition 17.6 a scope containing a generic-runner image is not tight at all. Second, h_r is not under the principal's full control: it depends on detection, on a 2-of-2 ceremony requiring two devices, and on one confirmation.

17.8 Attack analysis

Compromised agent. Covered by Definition 17.13. The worst case is that $\mathcal{Z}$ spends the funded balance and deploys anything inside Σ until revocation confirms, each such deployment then running to its own paid-through height (Definition 17.14). Digest-pinned $\mathcal{I}$ is what stops "anything inside Σ" from meaning arbitrary code, subject to Definition 17.6. Revocation latency is bounded and stateable, which is the property that makes the exposure quantifiable rather than open-ended.

Compromised assistant holding no key. An assistant of the §19 kind can propose a bad specification and can do nothing else: it cannot deploy and it cannot spend, because the acceptance predicate requires a signature it cannot produce. **This property holds only on the local-signing path.** On the FluxCore/Firebase path the assistant's bearer token *is* signing authority, and the property fails completely — there is no key the assistant lacks. Any statement of the form "the assistant configures but never signs" is a statement about one of two deployed paths, and §19 states it per path. This section's contribution to that discussion is narrow and concrete: the delegated path is not a defect so much as an undocumented instance of exactly the delegation specified here, running without any of the bounds specified here.

Malicious principal. $\mathcal{U}$ issues a certificate, lets $\mathcal{A}$ act, then claims no authorisation was given. The certificate is signed under (A1) and anchored on chain through rev, whose creation and spend are both public and heighted, so the claim is non-repudiable up to compromise of both principal devices. No additional mechanism is required, and none is specified.

Griefing the agent. $\mathcal{U}$ revokes mid-operation and strands a running deployment. This is deliberate and is not treated as an attack: the principal must always be able to stop the agent, and any design that lets an agent resist its principal is worse than the problem it solves. Nothing happens to the *workload* at that moment: it continues to its paid-through height under the state machine of §16, which is not restated here (Definition 17.14).

Remark 17.14 (Reconciling revocation with §16's lifecycle: they are orthogonal, and must stay so). Revocation and the payment lifecycle both bear on a deployment's lifetime, so the case they must agree on is a **revoked-but-paid** application: the owner has revoked the certificate under which the agent deployed it, but the application's paid-through height has not been reached. Neither section previously stated the transition, and the reconciliation is this: **the two state machines are orthogonal; revocation gates acceptance, and §16 alone governs a deployment once accepted.**

§16's states $\{\mathsf{funded}, \mathsf{grace}, \mathsf{suspended}, \mathsf{evicted}\}$ answer *has this been paid for*. Revocation answers *was this authorised*, and it is evaluated once, by Algorithm 3 on a submitted message; nothing in this section re-evaluates it for a running application. A specification is accepted only while both hold, so a revoked-but-paid application **continues to its paid-through height e_a like any other paid application, and what revocation stops is every further registration, update or renewal under that certificate**. It does not pass through grace or suspended either: those two states exist to absorb transient funding failures — a bridge delay, a wallet outage — and to give a paying tenant time to recover, and nothing about the deployment's funding has changed. The cost of this narrowing is stated rather than hidden: an owner who revokes a compromised agent's authority does not stop the deployments that agent already had accepted; they run out their paid terms, and their cost is the $\mathcal{P}^{\max}$ term of Definition 17.13. Stopping them sooner would require every node to re-check $\mathsf{rev} \in U(h)$ for running applications at each height, a mechanism this paper does not specify.

Two consequences follow, and both are stated rather than left implicit.

1. **Nothing is forfeited on revocation.** Unlike moderation eviction (Definition 7.34), revocation is the owner's own act against their own agent, not against the deployment: the paid term is consumed as for any other paid application, and there is no counterparty and no forfeiture event.

2. **Revocation cannot be used to escape a moderation ban.** Because revocation touches acceptance and nothing else, an owner revoking their own certificate after a ban has been carried changes nothing: the application was already stopped by the ban, and revocation is not a path back to any state.

The relay as a withholding adversary. The relay cannot sign (Table 42), so it cannot forge a certificate. It can withhold, and for an autonomous agent withholding is a liveness attack against certificate *issuance*. The mitigation is structural rather than added: certificates are issued once and used many times, and by Definition 17.5 a node validating an agent's specification consults no relay, so a withholding relay delays a new delegation and does not interrupt a running one. Separately, and more urgently: the unauthenticated coordination endpoints flagged in the shipped source [ssp-relay/src/routes.ts:253–301] let a party who learns a `wkIdentity` string join that identity's room and post payloads for it. Delegation should not be built on top of those endpoints until they are closed, and this section treats closing them as a precondition rather than as a parallel improvement.

Sybil agents. Irrelevant by construction. Authority flows from a certificate, not from an identity, so $\mathcal{Z}$ generating n agent key pairs holds n keys and zero authority; each would require its own signature under (A1). This is the one adversary in §2's catalogue that this mechanism

defeats for free, and the reason is worth naming: nothing in the design is per-identity rate-limited or per-identity funded, so there is nothing for extra identities to multiply.

17.9 Open parameters

Remark 17.15 (Certificate encoding and location — resolved by splitting the two authorities). The three candidate locations — a SPEC v9 specification field, a separate broadcast message, or an on-chain commitment with off-chain retrieval — were carried as an undecided choice. They are resolvable once the certificate's two halves are separated, because the halves have different enforceability and therefore different homes.

Deployment authority belongs in a spec v9 specification field. It is genuinely enforceable: a node validating a deployment already holds the specification, so it can check the scope tuple against what is being deployed at exactly the point where the constraint bites, with no new message type, no second propagation path, and no retrieval availability problem. The cost is real and bounded — it binds the mechanism to the SPEC v9 enforcement height (§10) — and is worth paying, because the alternatives buy schedule independence with a permanent consistency obligation between two propagation paths.

Spending authority has no home, because it has no enforcement. Definition 17.11 shows that neither b_{rate} nor b_{total} is enforceable by consensus on a Bitcoin-derived chain without covenants or introspection; the only bound that survives is the balance of the key the agent can spend. Encoding an unenforceable bound on-chain would state a guarantee the chain does not provide, which is worse than stating none. The spending half of the certificate is therefore advisory: meaningful to an operator or a wallet that chooses to honour it, and invisible to consensus.

This split is also what the team's own settled position implies. The protocol offers no automatic renewal and no standing spend authority (Definition 16.25); convenience of that kind is a service-layer product. A certificate scheme that tried to carry enforceable spending limits would be reintroducing at the consensus layer precisely what that decision removed.

Remark 17.16 (What this costs the agent-native claim, stated plainly). Two settled decisions — no protocol auto-renewal, and spending bounds that collapse to balance — have a consequence for the thesis that should be stated rather than left for a reader to derive. An autonomous tenant that provisions, pays and renews without a human **must hold a signing key and use it**, and the only way to bound its exposure is to fund that key with no more than the operator is willing to lose. There is no protocol object that says "this agent may spend at most x per day"; Definition 17.11 shows there cannot be one on this chain without a consensus change of the class §25 calls Research.

The alternative — delegating to a custodial operator that renews on the tenant's behalf — works today and is what the team expects most users to choose [intent: design intake], but it reintroduces exactly the intermediary the agent-native claim is about removing. The honest statement of the target architecture is therefore narrower than "agents transact without intermediaries": agents transact without intermediaries *if they hold keys*, and the risk of holding them is bounded only by funding discipline. The paper makes that claim and not the broader one.

Decided. Whether the scope Σ also constrains where value may be sent: it does not. Definition 17.3 bounds what may be deployed and says nothing about where value may go, and that asymmetry is kept deliberately. A destination constraint would be unenforceable for the same reason a spending cap is (Definition 17.11): script validation cannot observe where a future transaction sends value. Writing an unenforceable constraint into the scope tuple would state a guarantee the chain does not provide, which is worse than stating none.

Decided. $t_1 - t_0 = 8{,}640$ blocks, i.e. 3 d at the tooling's 2,880 blocks per day. Reasoning: the window is the exposure period after a compromise the owner has not noticed, so it should be short; but it is also the re-issuance cadence an autonomous agent must survive, so an over-short

window converts routine key rotation into a liveness dependency on the owner being awake. Three days spans a weekend, which is the shortest interval that does not require weekend attention, and it bounds undetected-compromise exposure at three days of the agent's funded balance — a quantity the owner already controls directly. [inferred].

Remark 17.17 (FluxEdge scoped tokens fold into the certificate scheme — settled). FluxEdge's scoped-token model is to be **folded into this certificate scheme** rather than kept as a separate API-layer authority [intent: design intake] (PLANNED). One delegation object covers the platform: the same scope tuple, the same revocation semantics, and one thing to audit rather than two systems with different trust properties to explain and keep aligned.

The cost is a real coupling and the paper states it: folding binds FluxEdge's API-layer authorisation to the SPEC v9 enforcement height (Definition 17.15), because the certificate travels in a SPEC v9 specification field. Until $h^\star$, FluxEdge necessarily keeps its existing tokens, so the convergence is a target state rather than a description of today. The alternative — decoupled schedules — was available and was not taken, which means the two systems must not be allowed to drift apart in the interval.

Decided. The tooling changes, not the requirement. Definition 17.6 requires $\mathcal{I}$ to hold image digests, and the deployed tooling emits a tag; the resolution is that the tooling resolves the tag to a digest at specification time and emits the digest. Enforcing at the node instead would mean nodes resolving tags themselves at install time, which reintroduces exactly the mutable-reference problem digest pinning exists to remove — two nodes resolving the same tag at different moments can legitimately obtain different images. [inferred].

17.10 Comparison with prior art

ERC-4337 session keys [14] solve authority (B) directly, and the reason they can is worth stating precisely rather than gesturing at: an EVM account is a program with persistent state, so a validation function can read how much a session key has already spent and refuse the next transaction on that basis. The predicate that Definition 17.11 shows is inexpressible in Bitcoin script is an ordinary storage read in an EVM account. Flux's parallel-chain accounts inherit this capability and do not currently use it; the Flux L1 cannot inherit it at all. The asymmetry is not a gap in Flux's engineering, it is a property of the two execution models, and the paper is more useful stating it than pretending to a uniform design across both.

Macaroons [9] are the closest analogue for authority (A), and the correspondence is close enough to be worth taking seriously as a design check. A macaroon is a bearer credential carrying attenuable caveats that a verifier checks locally, without a lookup against an issuing service. Definition 17.1 is a macaroon whose caveats are specification predicates (Definition 17.3), whose verification is Algorithm 3, and whose revocation — the part macaroons conventionally handle worst, since local verification and central revocation pull against each other — is a UTXO spend that every verifier can already see. That is the one place where sitting on a chain buys something a general-purpose authorisation scheme does not get for free.

Bitcoin covenant proposals (`OP_CHECKTEMPLATEVERIFY`, `OP_VAULT`) are the primitive that would make (B) enforceable on a UTXO chain *without* a new trusted party, by letting an output constrain the transactions that spend it. Flux does not have them. They are the correct reference for what construction (c) of Section 17.6 would cost and for what it would buy, and they are also the reason the honest framing of row 13 in §25 is "not possible here" rather than "not possible".

Filecoin [41] payment channels and **Akash** [4] lease escrow bound a client's spend by pre-funding a channel or an escrow account, which is Definition 17.12 with better ergonomics and a settlement path attached. The honest framing is that Flux's deployable answer for spending is the same as theirs — fund a dedicated balance, and the balance is the bound — and that Flux's advantage is not there. It is in (A): collateralised nodes, each independently validating a signed

scope predicate over the specification they are about to run, is a stronger enforcement story for deployment authority than a marketplace whose provider accepts a lease because a broker told it to. That is the claim this section is willing to make, and it is deliberately narrower than the one a reader might expect.

18 The agent-native cloud

The roadmap states its destination without hedging, and it is worth quoting rather than paraphrasing:

> "The destination this roadmap builds toward is specific: a cloud that serves software agents as first-class customers. An AI agent holding a wallet should be able to deploy compute on Flux, pay per second, verify what it bought, and eventually do all of it privately — no account, no card, no human in the loop." [roadmap: flux-roadmap-public.md:7]

That sentence names four capabilities: provisioning without a human, payment without an account, verification of what was purchased, and eventually privacy over all of it. The checklist below scores the first three and, in place of privacy (taken up in §20 and §21), a fourth the sentence presupposes: bounded spending authority. It is a VISION statement — the roadmap's own framing, not a description of the deployed system — and this section's job is to take it apart claim by claim rather than repeat it. Two of the four capabilities already exist as running code. The other two are specified, not shipped, and this paper's answer to each lives in §14 and §17 respectively. The distance between the sentence above and the deployed system is smaller than a skeptical reader would guess, and the missing piece is not an interface problem.

What an agent can already do. A caller that never touches a browser can, today, provision Flux capacity once a human has issued its credential, hold a scoped and revocable credential while doing it, and pay for some services — CumulusVPN and FluxCloud application payments — with no account at all. Three shipped surfaces establish this.

First, an MCP (Model Context Protocol) server ships as part of the Deploy-from-Git tooling and exposes `deploy_app`, `update_app` and `renew_app` as tool calls an AI agent can invoke directly SHIPPED [deploy-with-git-frontend/server/mcp/router.js:13-44]. These are not read-only introspection calls: `deploy_app`'s own description is "Revalidate, Firebase-sign, register, and test-install an Orbit app" [deploy-with-git-frontend/server/mcp/createServer.js:174-177], and given a valid Firebase bearer token an agent can drive a specification from draft to a signed, submitted registration message with no client-side wallet interaction and no human signing step [deploy-with-git-frontend/server/mcp/createServer.js:66-266]. Since publication a second, separate MCP server has been released: `@runonflux/flux-cloud-mcp` (first release 2026-09-11, version 0.2.8 on 2026-10-25), run locally over stdio or used as the hosted FluxCloud application at `mcp.runonflux.com`. Its fifteen tools cover the application lifecycle — key generation, pricing, specification building and validation, quotes, deployment, status, logs, control and cancellation — and it is non-custodial: the server holds no key, read-only tools need none, write tools take the caller's own keys, and `flux_deploy_app` returns only a plan unless called with `confirm=true` SHIPPED [flux-cloud-mcp/src/server.ts:697–1411].

Second, and separately, FluxEdge exposes a fully documented OpenAPI 3.1.1 interface branded the "FluxEdge API" SHIPPED [pouwbackend/rest_api/v1/openapi.yaml:1-4], plus a dedicated cross-platform CLI, `fluxedge-cli`, whose machine-readable `-o json` output mode is explicitly recommended for scripting [pouwbackend/rest_api/v1/openapi.yaml:81-83,124-140]. Authority on this surface is a bearer token, and the token's privileges are enumerated rather than all-or-nothing: a key can be scoped to any subset of `start_rental`, `stop_rental`, `start_deployment`, `stop_deployment` and `get_balance` [pouwbackend/rest_api/v1/openapi.yaml:2571-2581], enforced in the Go

backend by a function that maps the five rental and deployment routes to the privilege each requires and passes every other route — premium rental, deployment update and redeploy, failover — for any valid key [pouwbackend/rest_api/ratelimit.go:378-394] [pouwbackend/rest_api/api.go:135-170]. The surface additionally rate-limits every key and every source IP to two requests per second [pouwbackend/rest_api/ratelimit.go:41-42,79-88] and automatically revokes a key or IP that accumulates five rate-limit violations inside a five-minute window — the key is disabled, its owner is emailed, and it is evicted from cache, with no human review in the loop [pouwbackend/rest_api/ratelimit.go:143-153,158-179,203-222]. This is a real, working instance of bounded and revocable *action* authority; §17 records it precisely for that reason.

Third, CumulusVPN pays for entitlement with a payment alone. A client generates a WireGuard keypair on its own device and attaches an `OP_RETURN` memo, `CVPN1: base58(sha256(pubkey)[0:20])`, to an ordinary Flux payment SHIPPED [cumulusvpn/gateway/internal/entitle/entitle.go:99-108]. Every gateway independently scans the chain and derives entitlement from confirmed, correctly-memoed payments [cumulusvpn/gateway/internal/entitle/entitle.go:218-287]; there is no user database, no login, and no server-issued token — the public key the client already holds *is* the credential [cumulusvpn/gateway/internal/entitle/entitle.go:1-19]. An autonomous payer needs nothing this system does not already give it.

The four capabilities, scored against the evidence. Read as a checklist against the roadmap's own sentence, the honest score is two shipped and two not, and the two that are missing are not symmetric with the two that ship.

- *Provision without a human* — yes. The MCP tool surface and the FluxEdge API both accept and act on a request with no human step in between, as above, once a human has issued the credential each requires (a Firebase session; a dashboard-minted FluxEdge key) SHIPPED.
- *Pay without an account* — yes. CumulusVPN's memo-derived entitlement ships, and direct per-deployment application payment is the same shape: a signed message and a confirmed payment, no account, no stored credential (§16) SHIPPED.
- *Verify what was bought* — no. No tenant-facing attestation path exists today; a tenant cannot check that the host running its workload booted the image it was promised to boot. §14 states the mechanism and the reason this is missing; it is not restated here PLANNED.
- *Bounded spending authority* — no. Nothing bounds the value a compromised bearer token can move. The action-scoping described above is real, but it scopes *what* a token may do, never *how much* it may spend; §17 specifies the gap and the deployable construction that closes it PLANNED.

Two of four ship. The two that do not are exactly the two that decide whether an agent is merely able to buy compute or can additionally be trusted with a budget. An agent that can provision and pay but cannot verify or bound its own authority is a capable customer and an unsupervised liability at the same time, and the paper does not paper over which one it currently is.

The interface, as a listing. The clearest way to show what the machine-facing interface actually looks like is to show one call, not to describe it. FluxEdge's `POST /rental/start` takes a criteria object — what kind of machine, not which machine — and returns a bound rental:

What is missing for full autonomy. Four gaps stand between the deployed interface and an agent that never needs a human, and each is small enough to name precisely rather than wave at.

```
// POST /rental/start
{"min_memory":16,"min_storage":500,"gpu":"RTX3080","nb_gpu":1,
 "region":"NorthAmerica","max_price_per_hour":0.5,
 "nb_computers":3,"premium":"none"}

// 200 OK
{"rentals":[{"rental_id":8782678,"price_per_hour":0.5,
  "status":"active",
  "computer":{"hash":"e5b37bd2716472","memory":16,
    "storage":500,"gpus":["RTX3080"],"nb_gpu":1,
    "region":"NorthAmerica","cluster_name":"us-east-1",
    "price_per_hour":0.5}}]}
```

Listing 2: FluxEdge `POST /rental/start`: a representative agent-facing request and response [pouwbackend/rest_api/v1/openapi.yaml:1330-1338,1358-1382]. The request names criteria, not a specific machine, and `max_price_per_hour` bounds this one call. Nothing in the request, the response, or the bearer token that authorized the call bounds what the same credential may spend across repeated calls — the gap §17 specifies.

- *No self-service key issuance.* FluxEdge's own OpenAPI surface documents consumption of a bearer token but not its issuance; the API's own quick-start documentation points a caller at a web dashboard to obtain one, and the dashboard path is the only one that ships SHIPPED [pouwbackend/rest_api/v1/openapi.yaml:61]. A human must mint the credential an agent then uses; no programmatic issuance endpoint exists anywhere on the documented surface [pouwbackend/rest_api/v1/openapi.yaml:463-2193].
- *No spend bound.* As above; §17 specifies the construction.
- *No verification path.* As above; §14 specifies the construction.
- *Silent underpayment on the direct-payment path.* A registration or update whose payment is short of the computed price is dropped with a log line and no rejection signal returned to the payer [flux/ZelBack/src/services/appMessaging/messageVerifier.js:777,835] [intent: mechanism specification]. For a human this is a support ticket. For an unattended agent it is the one failure mode with no recovery path at all: the funds are already spent, the deployment does not exist, and nothing tells the caller either fact.

None of these four is a redesign. Self-service key issuance is an API endpoint behind the same authorization logic that already exists; a spend bound has a deployable baseline that needs no new consensus rule (§17); a rejection signal is a message the network is already computing the inputs for. The honest and more interesting claim is not that agent-native provisioning requires new invention — it is that four specific, narrow, already-scoped omissions are standing between the interface as it exists today and the interface the roadmap's sentence describes.

Per-second GPU without an account. The roadmap states the next step directly:

> "Per-second GPU without an account. With FluxEdge folded into the platform, GPU capacity is bought the same way: pay, run, stop. We are not fighting the GPU marketplaces on headline price; we are the place an agent can rent a GPU without a human signing anything." [roadmap: flux-roadmap-public.md:79]

This is PLANNED, gated on the FluxCloud/FluxEdge merge [roadmap: flux-roadmap-public.md:79]. §15 has already established what matters here: GPU passthrough today is whole-device, granted via Kubernetes' `nvidia.com/gpu` resource request and `NVIDIA_VISIBLE_DEVICES` bus-index isolation, with no MIG, vGPU or time-slicing anywhere in the stack [pouwbackend/rancher/deployment.go:914-982] [doc: reconciliation record]. The consequence for this section, stated once and not re-derived: a per-second *price* does not require a per-second *allocation* primitive, and nothing here blocks quoting or billing a whole GPU by the second. What per-second billing cannot yet mean, on the allocation primitive §15 describes, is fractional-GPU multi-tenancy — a device is sold whole or not at all [inferred].

The composite picture. An agent today can obtain Flux capacity with no human step once a human has issued its credential, and can pay for CumulusVPN and for FluxCloud applications with no account; FluxEdge's scoped bearer tokens (which require an account and a human-minted key) and CumulusVPN's memo-derived entitlement are both running code, not proposals. What that same agent cannot do is confirm that the host it paid actually booted the image it was promised, and it cannot hold a credential whose worst-case loss is anything less than the full balance behind it. Both gaps have specifications elsewhere in this paper — §14 for the first, §17 for the second — and §25 classes neither as Extension: attested execution is Engineering (row 4), and bounded spending is Research (row 13) because the L1 cannot enforce $\mathcal{B}$ (Definition 17.11, Table 43). What is deployable today is the funded allowance of Definition 17.12 — a funded key with a swept revocation address, which the mechanism specification classes as Extension [intent: mechanism specification] — whose bound is the balance behind the key, not a budget the chain enforces.

Why this matters architecturally rather than commercially. The properties an autonomous customer needs — no account, no card, no human in the loop, verifiable delivery, a bounded worst-case loss — are exactly the properties a careful, distrustful human customer wants as well. A human who reads a memo-derived entitlement scheme correctly does not want a password database holding their identity either; a human who deploys a container wants to know what booted it just as much as an agent does; a human handing a long-lived API key to a CI pipeline wants that key's blast radius bounded for the same reason a delegated agent needs one. Building the interface this section describes for agents is not a separate product from building it for people who do not want to trust the platform by default — it is the same interface, and this paper makes no claim about how many agents will use it.

18.1 What such an agent would run

The interface question of this section — how an agent obtains capacity — is separable from what it then does with it, but the two meet at one point worth stating. The workloads an agent runs on behalf of an ordinary person are, in the main, small task-specialised models rather than frontier ones, and §15.11 argues that this class of work has no interconnect requirement and therefore no structural reason to centralise. That is the reason the agent-native argument and the accelerated compute argument are the same argument seen from two ends: the network that can be bought from without an account is also the network whose dispersion stops being a handicap for the work being bought.

Two of this section's four capabilities are what make that composition real rather than rhetorical. Payment without an account means a caller with no relationship to maintain can buy inference for one task and stop. Verification of what was bought — absent today (§14) — matters disproportionately for a small specialised model, because a caller usually cannot tell from an answer whether the model that produced it was the one they paid for. Neither property is peculiar to autonomous callers; both are simply more visible when there is no human to absorb the ambiguity.

19 Natural-language deployment: who holds the signing key

A user can already describe what they want running in a sentence and a short paragraph of configuration, and something on Flux will turn that description into a running application. What differs sharply, and matters to anyone deciding how much to trust the result, is which of two *different systems* turns the description into a signed registration message. One is the protocol's own path: the string to be signed goes to a device the user holds, the user's key signs it there, and a fully compromised assistant is limited to proposing something bad. The other is an operator's service, run by InFlux outside the protocol: the operator holds the key, the operator signs, and what stands between an autonomous caller and a live deployment is a bearer token that authenticates the caller to the party already holding the key (§16.2.1). These are not two modes of one product, and this section does not present them as a choice the protocol offers — the protocol offers exactly one of them and is indifferent to the existence of the other (Property 16.21). The section states each precisely, gives each its own security property rather than one property for "the system," and argues that for the protocol's path the open problem is not custody at all — it is whether the artifact placed in front of the signer can actually be understood before it is signed.

19.1 What ships today

Three pieces of tooling turn a natural-language or near-natural-language description into a deployment. All three are SHIPPED.

Orbit. A wizard (React front end plus an Express server, publicly reachable at `orbit.runonflux.com` per its own configuration) walks a user through a Git repository URL, a resource

plan and a small number of deployment options, then emits a SPEC v8 application specification whose single compose component runs the shared image `runonflux/orbit:latest` SHIPPED [deploy-with-git-frontend/config/defaults.js:3] [deploy-with-git-frontend/src/services/deployService.js:453–490]. That specification is not a pointer to a build the Flux team performs on the user's behalf: once FluxOS places it on a node, the container itself clones the target repository, auto-detects its language and framework by an ordered set of file-presence checks, installs dependencies, builds and runs it, and continues to manage releases, rollback and webhook-triggered redeploys from inside the running instance SHIPPED [orbit/README.md:5,20] [orbit/entrypoint.sh:6–9] [orbit/lib/build.sh:304–431]. Orbit is not a separate build farm; the container *is* the workload, running on whichever node FluxOS happens to place that application instance on SHIPPED [orbit/entrypoint.sh:1–9] [orbit/README.md:39–57].

Deploy-from-Git. The end-to-end path from a repository to a running application is: a user or agent supplies a Git URL and configuration; the frontend server analyzes the repository (provider, public/private access, branches, a detected port, monorepo layout, an imported `flux.json`/`flux.yaml`) SHIPPED [deploy-with-git-frontend/server/orbit/repositoryService.js:16–53]; a SPEC v8 specification is built around the Orbit image SHIPPED [deploy-with-git-frontend/src/services/deployService.js:453–489]; the specification is sent to Flux's own `POST /apps/verifyappregistrationspecifications` for normalization SHIPPED [deploy-with-git-frontend/src/services/deployService.js:521–539]; it is signed (mechanism below); and the signed registration message is submitted directly to `api.runonflux.io` via `POST /apps/appregister` SHIPPED [deploy-with-git-frontend/src/services/deployService.js:547–565]. Neither the frontend server nor Orbit itself ever clones or builds the user's code before that point — that work happens later, inside the Orbit container, once placement has occurred.

`DeploymentYAML`. A gRPC method, `DeploymentYAML(YAMLPrompt) → YAMLResponse`, generates Kubernetes YAML from a natural-language prompt SHIPPED [pouwbackend/grpc_server/marketplace/deployment.go:1784–1812]. The generator first vector-searches existing marketplace application YAMLs for retrieval-augmented context SHIPPED [pouwbackend/ai/fluxai.go:188–191,241–244], then issues a chat-completion request against a hard-coded system preamble instructing the model to emit only raw YAML, with the model pinned to "Llama 3.1 8B" and dispatched either to FluxAI's own `/v1/chat/completions` endpoint or to Google AI SHIPPED [pouwbackend/ai/yaml.go:30–41] [pouwbackend/ai/fluxai.go:101,116,212,219–225,266,278–284]. Natural-language deployment, in other words, is not hypothetical; a form of it ships today as a retrieval-augmented, LLM-backed generator. What the extraction did *not* find is a code path chaining that generated YAML directly into `/deployment/start` — whether the output is ever auto-submitted or always requires a human or agent to copy it forward is unresolved from the evidence available, and the claim made here is limited to generation, not to confirmed unattended submission [inferred].

Since publication: the FluxCloud assistant. From 2026-09-20 the FluxCloud web interface includes an assistant, on by default, answered by `fluxai:tiny` from `llm.runonflux.com` (§15.13). It turns a description into a SPEC v8 application specification and prefills the deployment form; the user then reviews, signs and pays in the page. It holds no key and cannot deploy, sign or spend SHIPPED [fluxcloud-web/src/api/agent/client.ts:10–19], so it belongs to the protocol's own signing path below, and it uses neither FluxEdge nor Google AI.

19.2 Two signing paths, belonging to two systems

Orbit and Deploy-from-Git converge on one signing step (`DeploymentYAML` has none: it produces FluxEdge Kubernetes YAML and has no link to FluxCloud registration), and that step does not have one answer — it depends on how the caller authenticated, and therefore on which system produces the signature.

Definition 19.1 (Local signing and operator signing)**.** A *local-signing path* is one on which the string to be signed is built by a frontend and relayed to a device that holds the private key, with the signature computed and returned by that device. An *operator-signing path* is one on which a remote service that itself holds the private key computes the signature and returns it, authorized

by a bearer credential that authenticates the caller to that service. The word *delegated* does not fit the second and this paper does not use it as a description of authority there: delegation presupposes a principal holding the authority being delegated, and on this path the user never holds the key (§16.2.1, Property 16.17, Corollary 16.18).

The first is a path of the protocol: Flux verifies a signature against an address and takes no view of where the corresponding key sits, so a key on a user's own device is exactly what the network expects and requires no service anywhere. The second is a path of a company's product; the network cannot see it and does not depend on it (Property 16.21). The remainder of this subsection states each separately, and it does not average them.

On the local-signing path, a user connected through the SSP wallet extension or through ZelCore has the frontend build the string-to-sign and relay it to `window.ssp.request('sign', ...)` or to a ZelCore deep link plus a WebSocket channel; the private key never leaves the user's own device, and the frontend only ever sees the resulting signature SHIPPED [deploy-with-git-frontend/src/services/deployService.js:697–706,717–772]. Here the trust boundary really is the signing step: a fully compromised assistant can construct and propose an arbitrary specification, but it cannot produce a valid signature over it, and therefore cannot deploy or spend. That is the property this section's opening claims for the protocol's path, and here it holds.

On the operator-signing path — a FluxCore SSO login authenticated by Firebase — the frontend server does not hold or see a Flux private key at any point. It POSTs the message and the user's Firebase ID token to a remote service, `https://service.fluxcore.ai/api/signMessage`, and receives a signature back; the service signs *on the user's behalf* SHIPPED [deploy-with-git-frontend/src/services/deployService.js:677–691] [deploy-with-git-frontend/server/flux/fluxSession.js:60–73]. That remote service is the payment facilitator of §16.2.1: the same architecture, in which the operator generates and holds both of a user's private keys and the user never sees either [intent: design intake] [console-api/src/db/models/FluxIdentity.ts:13–28]; the identification of the two services follows from the observed protocol rather than from a read of this signer, which is not in the corpus [inferred]. This is the path every server-side call uses, including registration (`DeploymentService.register()` calling `sessions.signMessage(session, message)`, authenticated only by the caller's bearer token) and updates (`UpdateService.submit()`) SHIPPED [deploy-with-git-frontend/server/orbit/deploymentService.js:158–173] [deploy-with-git-frontend/server/orbit/updateService.js:68–79]. It is also the path the shipped Model Context Protocol (MCP) tools `deploy_app`, `update_app` and `renew_app` use SHIPPED [deploy-with-git-frontend/server/mcp/createServer.js:13–53,66–266] [deploy-with-git-frontend/server/mcp/router.js:13–44]. The `deploy_app` tool's own description states the mechanism plainly: "Revalidate, Firebase-sign, register, and test-install an Orbit app" SHIPPED [deploy-with-git-frontend/server/mcp/createServer.js:174–177]. These tools, along with `control_instance` and `trigger_build`, are marked `destructiveHint:true` and execute against live Flux state given only a valid bearer token SHIPPED [deploy-with-git-frontend/server/mcp/createServer.js:170–266]. Given such a token, an agent can construct, sign and submit a deployment end to end with no human in the loop and no separate signing step SHIPPED [deploy-with-git-frontend/server/mcp/createServer.js:66–266] [deploy-with-git-frontend/server/orbit/services.js:91–104].

Path, and whose it is	**Who produces the signature**	**Bound on a fully compromised signing credential**
Local signing (SSP, ZelCore) *the protocol's path*	The user's own device; the frontend only builds the string-to-sign and relays the resulting signature. SHIPPED [deploy-with-git-frontend/src/services/deployService.js:697–706,717–772]	A compromised assistant can propose a bad specification; it cannot deploy or spend, because it never holds a key capable of producing a valid signature over it. SHIPPED [deploy-with-git-frontend/src/services/deployService.js:697–706,717–772]
Operator signing (FluxCore SSO) *InFlux's service, outside the protocol*	A remote service, `service.fluxcore.ai/api/signMessage`, which holds the key and is authorized only by a Firebase bearer token. SHIPPED [deploy-with-git-frontend/src/services/deployService.js:677–691] [deploy-with-git-frontend/server/flux/fluxSession.js:60–73]	Nothing: no per-transaction cap, no rate limit, no budget, no scope, and no revocation beyond invalidating the token itself. [inferred] [intent: mechanism specification]

Table 44: The bound on a fully compromised signing credential, by path. The two rows are properties of two different systems and do not compose into a property of "the system".

Non-custody is a property of the protocol's path. It is not a property of the operator's service, and the operator's service is not part of the Flux protocol. Stating the claim at the level of "the assistant configures, the user signs, therefore a compromised assistant cannot spend" would be false as a description of the operator path, because there an assistant holding a Firebase bearer token *is* the signer, indirectly, through a service that will sign whatever it is asked to sign.

Two consequences, and the second is the one readers get wrong. First, the bound on that token today is nothing at all, and nothing in the protocol could have made it otherwise: the token is not a delegation and confers no authority — it authenticates a caller to the party that already holds the key (§16.2.1, Property 16.17). Second, and therefore, **§17's construction does not apply to this path.** A delegation certificate bounds an authority a principal holds and grants; here the principal holds nothing, so there is no grant to bound and the certificate has nowhere to attach (Corollary 16.18). What would bound the operator path is operator-side policy — rate limits, per-token caps, an approval step — which is a product decision belonging to the operator, and this paper specifies none of it and does not track it. §17 is written for the other case: a principal $\mathcal{U}$ who holds a key on the protocol's path and grants an agent $\mathcal{A}$ a scope Σ under a budget $\mathcal{B}$ — where §17 also states why enforcing $\mathcal{B}$ on the Flux L1 as deployed is not currently possible without a new trusted party or a consensus change.

The operator may choose to hand users the keys it holds for them — K_u in the notation of §16.2.1 — at which point its path becomes a local-signing one; it states that it may [intent: design intake]. That is the operator's decision and not a protocol change — no consensus rule, no FluxOS release and no application specification field moves either way — and this paper records it without tracking it (§16.2.1, Remark 16.22).

19.3 Reviewability: the question that remains

On the protocol's path the security boundary is exactly the signing step, and that boundary is only as strong as the signer's ability to understand what is being signed. Everything in this subsection is about that path; on the operator's service the question does not arise, because the

party that reviews and the party that signs are the same party and it is not the user. For that review to be a meaningful check rather than a formality, the interface would need to surface, at minimum: the total cost and duration of the deployment; every port the specification exposes; the provenance of every image *by content digest, not by mutable tag*; the resource commitments (cpu, memory, storage) the specification requests; the data-retention consequences of eviction or non-renewal (§16); and any node-targeting or geolocation constraint the specification carries. PLANNED. These are not arbitrary — they are structurally the same fields §17's delegation scope $\Sigma = (\mathcal{N}, \mathbf{r}^{\max}, k^{\max}, \mathcal{I}, d^{\max}, \mathcal{G})$ already names, because a human reviewing a specification before signing it and a delegation certificate bounding an agent's authority to construct one are answering the same question. PLANNED [intent: mechanism specification].

The concrete failure this addresses is not hypothetical. The deployed tooling emits `runonflux/orbit:latest` — a mutable tag, not a content digest — with nothing in the specification-construction code pinning a `sha256:` digest SHIPPED [deploy-with-git-frontend/src/services/deployService.js:453–490]. A user signing that specification is not reviewing the code that will run; they are reviewing a name that resolves to whatever the tag currently points to at the moment a Flux node pulls it. The review surface itself compounds this: the wizard's final confirmation step renders a reconstructed summary object — name, ports, domain, polling interval, runtime, geolocation, environment-variable *keys* only — and not the literal SPEC v8 payload that gets signed, which omits the image reference, owner, contacts, expiry and the full environment-variable array present in the actual signed string SHIPPED [deploy-with-git-frontend/src/components/wizard/Step4Review.jsx:280–338]. The literal artifact signed is only ever visible as the `dataToSign` string passed to the signer, and nothing in the extraction found it rendered to the user anywhere SHIPPED [deploy-with-git-frontend/src/services/deployService.js:667–670]. An unreviewable specification presented for signature reduces the security boundary to theatre: the signing step still cryptographically binds the signer to *something*, but not demonstrably to the thing the signer believes they approved.

It is worth noting, briefly, that the draft SPEC v9 application specification already carries several of the fields a meaningful review would need — a required top-level `duration` field, per-component `cpu`/`memory`/`rootFsGb`, and an explicit `ports` map — none of which existed as required fields in SPEC v8 IN-PROGRESS [flux-spec/schema/v9.json:62–66,103–117,129–150]. What that draft does not yet specify, as of this writing, is any signature field at all: its canonicalization and fingerprint recipe is described only as "recommended," not enforced, and no signature or signer-identity field appears anywhere in the schema IN-PROGRESS [flux-spec/validation/canonicalization.md:53–63]. The reviewability question this section raises is therefore not one SPEC v9 answers by construction; it is a gap the next specification version inherits along with the fields it would need to close it.

Property 19.2. The signing step is a meaningful trust boundary if and only if the signer can, from the artifact presented for signature alone, determine (i) the set of programs that may execute as a result of that signature, and (ii) the maximum value that may be spent as a result of it.

The deployed interface fails condition (i) outright: the image reference a signed specification authorizes is a mutable tag rather than a fixed digest, so two signatures over "the same" specification, at different times, can authorize different code SHIPPED [deploy-with-git-frontend/src/services/deployService.js:453–490], and the object rendered for review is a reconstruction rather than the literal payload being signed SHIPPED [deploy-with-git-frontend/src/components/wizard/Step4Review.jsx:280–338]. Condition (ii) fares somewhat better in structure — plan, resource ceiling and billing period are concrete fields the tooling computes before submission, including an explicit blocks-from-months expiry calculation SHIPPED [deploy-with-git-frontend/src/services/deployService.js:102,108–110] — but because the rendered review is, again, a reconstruction, a signer confirms the wizard's arithmetic rather than reading the number that will actually be signed. Digest pinning is the cheapest

available remedy for condition (i), because it requires no protocol change, no consensus change and no new specification field: it only requires substituting a resolved digest for a mutable tag at the point where the tooling already constructs the specification, which is a change confined entirely to the tooling that already runs today.

19.4 Self-hosting

The tooling that performs natural-language deployment does not sit outside the network it targets. Orbit is deployed as an ordinary Flux application and performs its own clone/build/run cycle inside that application's own container, on whichever node FluxOS happens to place it — it is explicitly not a separate, team-operated build farm SHIPPED [orbit/README.md:5,20] [orbit/entrypoint.sh:1–9]. The language model behind `DeploymentYAML` is served from FluxAI's own inference capacity (or Google AI, as described above), and FluxAI is itself an ordinary tenant of FluxEdge — whose machines are not FluxNodes — deploying through the same GitOps pipeline used for any other model, and also of Vast.ai and other providers SHIPPED [fluxai-fluxedge/custom-workflow-fluxone.yaml:1–52]. An assistant that deploys applications on Flux is, in this narrow sense, expected to run as an ordinary tenant — of FluxCloud (Orbit) or of FluxEdge (FluxAI) — rather than as privileged infrastructure. That is a pleasing property, worth stating once — but it is not a security property, and it should not be dressed up as one. Nothing about running as an ordinary tenant bounds what a compromised operator-signing credential can do; the bound in §19.2 is unaffected by where the assistant itself happens to run.

19.5 Toward bounded automation

A principal *who holds a key* and explicitly delegates under §17's certificate construction would obtain automation whose worst case is bounded rather than open-ended: the certificate fixes a scope Σ — permitted application names, a resource ceiling, a digest-pinned image set, a maximum duration and permitted geolocation constraints — and, where enforceable, a spending budget $\mathcal{B}$. PLANNED [intent: mechanism specification]. The qualifier is load-bearing and is the reason this paragraph sits under the protocol's path: the construction presupposes a principal-held key, so it is available to a user on the local-signing path and to autonomous software holding its own key, and it is not available on the operator-signing path at all (Corollary 16.18). This section does not restate that construction; it only notes the relationship — that construction is what would let a natural-language deployment flow run end to end without the two open failures described above, an unpinned image and an unbounded bearer token, both being in force at once.

19.6 What would make the boundary real

Three changes, taken together, would make the trust boundary on the protocol's path correspond to what it is presented as being — and, incidentally, would give an operator that chose to bound its own service something worth enforcing: digest pinning by default, so that $\mathcal{I}$ names fixed content rather than a mutable tag; a specification diff presented before signature, so the artifact a signer reviews is the literal payload being signed rather than a reconstructed summary of it; and a cost estimate the signer can check against the specification's own fields rather than trust from a wizard screen. PLANNED [inferred]. None of the three requires a consensus change or a new trusted party; all three are changes to tooling that already runs today.

20 Shielded value on Flux

`fluxd` is a Zcash fork, and its privacy layer is not something Flux built: it is Zcash's Sprout and Sapling shielded-transaction machinery, carried forward through an intermediate "Zel" rebrand, compiled into the node today, and only partially wired up for Flux's own chain. Every subsection below states, component by component, whether what is described is inherited from Zcash

unmodified, inherited and then modified by Flux, or original to Flux. Almost nothing in this section is the last category: the cryptography is Zcash's, and Flux's own work here is mostly a set of consensus-level switches layered on top of it — some left on, several left off.

`fluxd`'s consensus rules and wallet support two address and transaction types side by side today: transparent (t-addr) and shielded (z-addr) (SHIPPED, [fluxd/src/primitives/transaction.h:684-686,967-969]). **That coexistence is ending.** Earlier drafts of this paper recorded it as permanent, on the strength of three consistent statements: "Optional at the protocol level, never mandatory. A transparent layer stays permanently" [roadmap: flux-roadmap-public.md:171]; the non-goals list ruling out "Making privacy mandatory" for the same reason [roadmap: flux-roadmap-public.md:206]; and the design intake's "Both payment types stay, permanently" [intent: design intake]. Those statements are accurately reported and were accurate when made. They have been superseded: the team has decided to **retire both shielded pools** [intent: design intake] (PLANNED). The transparent layer stays; the shielded layer does not.

This section is therefore written in two registers, and the reader should know which is which. Every subsection up to and including §20.6 describes the machinery *as deployed today* — SHIPPED, present tense, accurate at the time of writing, and live on mainnet holding real value. §20.9 states the decision to remove it, the measurement that motivated it, and the schedule. The descriptive half is retained in full rather than cut: a reference description of Flux that omitted a mechanism currently running would be incomplete, and a retirement plan is only auditable against a precise account of what is being retired.

20.1 Lineage: Bitcoin, Zcash, Zel, Flux

The in-repository license chain states the lineage plainly: `COPYING` carries copyright headers running Bitcoin Core developers (2009–2019) → Zcash developers (2016–2019) → Zel developers (2017–2019) → Flux Developers (2018–2022) (SHIPPED, [fluxd/COPYING:1-5]). That is the authoritative lineage as the code itself records it, and it is stated here as a fact about provenance, not as an apology: a large fraction of `fluxd` is inherited Zcash code, and the paper treats every claim below accordingly.

The renaming from Zcash's `libzcash` to Flux's `libflux` was not applied uniformly. Most files under `src/flux/` carry only a Flux copyright line (e.g. [fluxd/src/flux/Note.hpp:1], [fluxd/src/flux/NoteEncryption.hpp:1], [fluxd/src/flux/Proof.hpp:1], [fluxd/src/flux/Address.hpp:1], [fluxd/src/flux/JoinSplit.hpp:1]), while [fluxd/src/flux/zip32.h:1-2] retains both the original Zcash and the added Flux copyright line — the header rewrite was partial. Under `src/`, 395 files carry `"The Flux Developers"` and 24 still contain the literal string `"Zelcash"`; repository-wide, 58 files still carry `"The Zcash developers"` with no Flux line added (SHIPPED, [fluxd:repo-wide grep]). The historical header `src/flux/Zelcash.h`, which defines the shielded protocol's size constants (Table 45), still bears that name and is directly `#included` by `Note.hpp`, `NoteEncryption.hpp` and `Address.hpp` (SHIPPED, [fluxd/src/flux/Note.hpp:9]). Other rebrand artifacts persist unmodified: the standalone consensus library's pkg-config template still emits `-lzelcashconsensus` and `"Zelcash consensus library"` against an actual build target named `libfluxconsensus.la` (SHIPPED, [fluxd/libfluxconsensus.pc.in:7,9] vs. [fluxd/src/Makefile.am:52]); the client name compiled in is the unmodified upstream `"MagicBean"` (SHIPPED, [fluxd/src/clientversion.cpp:17]); and the startup log still prints `"Zelcash version %s (%s)"` (SHIPPED, [fluxd/src/init.cpp:918]). These are stated as discrepancies in an incomplete rebrand, not as deliberate design.

The fork point itself cannot be read off a version string, because `fluxd` carries no explicit record of the upstream `zcashd` release it branched from. It can nonetheless be bracketed from consensus evidence rather than guessed. The vendored `doc/zcash-release-notes/` tops out at the v2.1.0 series (`release-notes-2.1.0.md` and `release-notes-2.1.0-1.md`), the former announcing Blossom's mainnet activation [doc: fluxd/doc/zcash-release-notes/release-notes-2.1.0.md:1-15]; and the network-upgrade table contains no Heartwood, Canopy, NU5 or Orchard entry [fluxd/src/consensus/params.h:25-38], [fluxd/src/consensus/upgrades.cpp:11-65], while only the old-style

librustzcash_sapling_* FFI is called and never orchard or halo2 [fluxd/src/main.cpp:1447]. Flux therefore derives from **zcashd v2.1.0 (Blossom), at or after that release and before v2.1.2 (Heartwood)**. One caution for anyone re-deriving this: the pinned revision in depends/packages/librustzcash.mk is a 2018 commit, but it is the same commit zcashd v2.1.0 itself pins, so it corroborates the bracket above rather than dating the fork earlier [zcash@v2.1.0:depends/packages/librustzcash.mk:7].

Flux's own consensus-upgrade naming diverges completely from Zcash's. fluxd's UpgradeIndex enumerates BASE_SPROUT, UPGRADE_TESTDUMMY, UPGRADE_LWMA, UPGRADE_EQUI144_5, UPGRADE_ACADIA, UPGRADE_KAMIOOKA, UPGRADE_KAMATA, UPGRADE_FLUX, UPGRADE_HALVING, UPGRADE_P2SHNODES, UPGRADE_PON (SHIPPED, [fluxd/src/consensus/params.h:24-39]). There is no dedicated Overwinter- or Sapling-named entry; Sapling-era rules are instead gated on UPGRADE_ACADIA (§20.2 below). BASE_SPROUT is ALWAYS_ACTIVE from genesis on mainnet (SHIPPED, [fluxd/src/chainparams.cpp:108-110]).

The cryptographic backend is pinned, not vendored fresh. librustzcash is pulled at build time from https://github.com/zcash/librustzcash/archive/ at git commit 06da3b9ac8f278e5d4ae13088cf0a4c03d2c13f5, version 0.1 (SHIPPED, [fluxd/depends/packages/librustzcash.mk:2-8]). The Rust crates it depends on are pinned at their original 2018-era Zcash-project versions: bellman (the Groth16 R1CS prover) at 0.1.0 (SHIPPED, [fluxd/depends/packages/crate_bellman.mk:2-3]) and pairing (BLS12-381 curve arithmetic) at 0.14.2 (SHIPPED, [fluxd/depends/packages/crate_pairing.mk:2-3]). Nothing in the tree indicates a later librustzcash, bellman, or pairing upgrade was ever pulled forward.

20.2 The shielded machinery

The primitives that make a shielded transaction work — a note, its commitment cm, its nullifier nf, the note commitment tree NCT that anchors membership proofs, and the zero-knowledge proof π that a spend is valid without revealing which note it spends — are Zerocash-family constructions [7] as specified for Zcash's Sprout and Sapling shielded pools [31]. fluxd's implementation lives under src/flux/ (namespace libflux, historically libzcash) plus src/primitives/transaction.h, and it is compiled into libfluxconsensus/fluxd for both protocol versions (SHIPPED, [fluxd/src/Makefile.am:92-101,542-550]). Table 45 states, per component, whether the code is Zcash's cryptography carried forward unmodified (beyond the libzcash→libflux rename) or a Flux-added policy layered on top of it; every Flux-original item in this section is a consensus switch, never a change to the underlying scheme.

- **Notes.** Note.hpp/.cpp implements SproutNote (fields a_{pk}, ρ, r; cm computed as SHA256(0xb0$\|a_{pk}\|$value$_{LE}\|\rho\|r$), SHIPPED, [fluxd/src/flux/Note.cpp:24-41]; nf via PRF_nf, SHIPPED, [fluxd/src/flux/Note.cpp:43-45]) and SaplingNote (fields d, pk_d, r; cm/nf delegated to librustzcash_sapling_compute_cm/_compute_nf, SHIPPED, [fluxd/src/flux/Note.cpp:55-93]), with explicit plaintext lead-byte tags 0x00 (Sprout) / 0x01 (Sapling) (SHIPPED, [fluxd/src/flux/Note.hpp:97-101,156-161]) and an outgoing-viewing-key plaintext for sender-side recovery (SHIPPED, [fluxd/src/flux/Note.hpp:172-204]). Inherited unmodified.

- **Note encryption.** NoteEncryption.hpp/.cpp implements SaplingNoteEncryption (one-shot, ephemeral epk/esk; the class comment itself notes it is not thread-safe, [fluxd/src/flux/NoteEncryption.hpp:30]), trial-decryption routines AttemptSaplingEncDecryption/AttemptSaplingOutDecryption, a templated Sprout AEAD scheme sized to NOTEENCRYPTION_AUTH_BYTES= 16 bytes, and a PaymentDisclosureNoteDecryption variant used by the payment-disclosure RPCs (§20.6) (SHIPPED, [fluxd/src/flux/NoteEncryption.hpp:176-195]). Inherited unmodified.

- **Note commitment tree (NCT).** A generic incremental Merkle tree, IncrementalMerkleTree<Depth,Hash>, instantiated twice: SproutMerkleTree at depth 29 over SHA-

256 compression, and `SaplingMerkleTree` at depth 32 over a Pedersen hash (SHIPPED, [fluxd/src/flux/IncrementalMerkleTree.hpp:253-263]). Inherited unmodified.

- **Addresses and viewing keys (ivk).** `Address.hpp` defines `SproutPaymentAddress`/`ViewingKey`/`SpendingKey` and `SaplingPaymentAddress` (diversifier d + pk_d), `SaplingIncomingViewingKey`, `SaplingFullViewingKey` (ak, nk, ovk), and `SaplingExpandedSpendingKey` (ask, nsk, ovk) (SHIPPED, [fluxd/src/flux/Address.hpp:32-222]). `zip32.h/.cpp` implements ZIP-32 hierarchical-deterministic derivation for Sapling extended keys (SHIPPED, [fluxd/src/flux/zip32.h:16-18,53-136]). Inherited unmodified.
- **Proving system (π).** `Proof.hpp/.cpp` carries the legacy Sprout `PHGRProof` pairing-based proof format (SHIPPED, [fluxd/src/flux/Proof.hpp:159-185]) alongside a strict/disabled verifier toggle used at reindex time (SHIPPED, [fluxd/src/flux/Proof.hpp:222-229]). `JoinSplit.hpp` fixes `GROTH_PROOF_SIZE`= 48+96+48 = 192 bytes — a BLS12-381 compressed $G_1 \| G_2 \| G_1$ layout — and defines `SproutProof` as a tagged variant of `PHGRProof` and `GrothProof` (SHIPPED, [fluxd/src/flux/JoinSplit.hpp:22-29]); `transaction.h` carries the Sapling `SpendDescription` and `OutputDescription` wire structures, each holding a `GrothProof zkproof` field (SHIPPED, [fluxd/src/primitives/transaction.h:155-242]). Inherited unmodified; see §20.3 for the parameters this proving system consumes.

Component	Provenance	Maturity
Note (note), commitment (cm), nullifier (nf)	inherited, unmodified (renamed)	SHIPPED
Note encryption	inherited, unmodified (renamed)	SHIPPED
Note commitment tree (NCT)	inherited, unmodified (renamed)	SHIPPED
Addresses, viewing keys (ivk), ZIP-32	inherited, unmodified (renamed)	SHIPPED
Groth16 proving system (π)	inherited, unmodified (renamed)	SHIPPED
Turnstile enforcement (ZIP-209)	inherited, Flux-disabled	[fluxd/src/chainparams.h:200]
Coinbase-must-be-shielded	inherited, Flux-disabled post-height	[fluxd/src/main.cpp:3228-3237]
New-inflow restriction at `UPGRADE_FLUX`	Flux-original policy	[fluxd/src/main.cpp:1398-1408]
Sprout→Sapling migration RPC	inherited, Flux-disabled	[fluxd/src/wallet/rpcwallet.cpp:4110-4114]

Table 45: Provenance of the shielded-value components discussed in this section. "Flux-disabled" means the code path is compiled in and reachable but a Flux-added switch keeps it inactive; see §20.4.

Two shielded protocol versions are active on `fluxd` today, and a third is not present at all. **Sprout** is active from genesis (`BASE_SPROUT` is `ALWAYS_ACTIVE`) and accepts both the legacy `PHGRProof` and `GrothProof` encodings, tag-selected per transaction version (SHIPPED, [fluxd/src/primitives/transaction.h:244-289]). **Sapling** is gated on `Consensus::UPGRADE_ACADIA`, active on mainnet from height 250,000 (SHIPPED, [fluxd/src/chainparams.cpp:122-125]), with enforcement of the Overwinter flag, `SAPLING_VERSION_GROUP_ID`, and the version-4 transaction bounds inside `ContextualCheckTransaction` (SHIPPED, [fluxd/src/main.cpp:1223,1344-1380]). As already noted, there is no separate Overwinter- or Sapling-named upgrade entry; Flux folded both into `ACADIA`. **Orchard is absent**: no occurrence of the string `Orchard` appears anywhere under `src/` (SHIPPED, [fluxd/src:grep -rn Orchard, zero matches]), which is consistent with the fork point established above — the NU5/Halo2/Orchard-era `zcashd` code was never merged. §20.8 returns to what adopting it later would and would not change.

20.3 The trusted setup, stated exactly

Groth16, as deployed here, requires a one-time per-circuit parameter-generation ceremony whose output — the proving and verifying keys — must not leak the ceremony's intermediate randomness (the "toxic waste") [7, 29]. `fluxd` does not run its own ceremony.

Assumption 20.1 (Sapling trusted setup)**.** `fluxd`'s Groth16 proving parameters — `sapling-spend.params`, `sapling-output.params`, and `sprout-groth16.params` — are

fetched, not generated, by the inherited `zcutil/fetch-params.sh` tool, packaged in the Flux build with a man page still named `zcash-fetch-params` (SHIPPED, [fluxd/zcutil/fetch-params.sh:5-15]; [fluxd/doc/man/zcash-fetch-params.1:4]). They are re-hosted on a Flux-operated mirror (`download.runonflux.io`) and on IPFS, downloaded in two parts and checked against fixed sha256 pins before use (SHIPPED, [fluxd/zcutil/fetch-params.sh:104-141,220-222]). At startup, `ZC_LoadParams()` requires all three files to exist or calls `StartShutdown()` (SHIPPED, [fluxd/src/init.cpp:754-806,762-775]), then passes independent SHA-512 hash strings for each file into `librustzcash_init_zksnark_params()` (SHIPPED, [fluxd/src/init.cpp:791-801]). Flux therefore inherits the original **Zcash Sapling parameter-generation MPC** — the 2018 "Powers of Tau"/Sapling ceremony run by the Zcash project together with outside participants [11] — verbatim: it re-hosts the identical parameter files under their original filenames and role, and performs no additional or independent ceremony of its own.

The claim that the re-hosted files are byte-identical to the ones `zcashd` itself ships rests on the filenames, roles, and hash pins matching the well-known public Zcash Sapling parameter set, together with the crate pins in §20.1 being the original 2018 `librustzcash/bellman` versions; it was not independently re-derived against a `zcashd` source tree in the evidence this section draws on [inferred]. The paper states it as a strongly supported inference rather than an independently re-verified fact, and flags the direct hash diff as the bounded check that would close it.

Table 46 gives the fetch-time integrity pins exactly, since a reviewer checking a trusted-setup claim will want the exact values rather than a description of them.

Parameter file	sha256 (fetch-time pin)	Location
`sapling-spend.params`	`8e48ffd2...7efc13`	[fluxd/zcutil/fetch-params.sh:220]
`sapling-output.params`	`2f0ebbcb...f3fb0e4`	[fluxd/zcutil/fetch-params.sh:221]
`sprout-groth16.params`	`b685d700...0ee55b50`	[fluxd/zcutil/fetch-params.sh:222]

Table 46: Fetch-time sha256 pins for the three Groth16 parameter files `fluxd` requires at startup (hashes elided to their leading/trailing bytes here; full values in [fluxd/zcutil/fetch-params.sh:220-222]). Runtime loading additionally checks independent SHA-512 hashes of each file ([fluxd/src/init.cpp:791-801]).

What this assumption buys, and what it costs if it is false, should be stated as plainly as the assumption itself. If the toxic waste from the original Sapling ceremony was retained by any participant rather than destroyed, that party could construct false Groth16 proofs that pass verification while creating shielded value that no spent note ever committed to — that is, forge notes and mint counterfeit shielded value without breaking any wallet's transparent balance or the transparent supply at all [7, 29]. The forgery would be *undetectable* from inside the shielded pool by construction, because the proof system gives no signal distinguishing a validly spent note from a forged one; the only external check available is exactly the pool-balance invariant discussed next, and §20.4 states that this check is currently disabled on Flux's own chain. The multi-party design of the original ceremony was specifically intended to make this failure require collusion across many independent participants [11]; Flux's inheritance of the parameters carries that same assumption forward unchanged, since it is not a party to the ceremony and cannot strengthen or re-verify it after the fact.

20.4 What is switched off

The honest core of this section is that several of the safeguards this machinery was built to provide are compiled into `fluxd` but not active on Flux's live networks.

The shielded pool has been closed to new entrants since `UPGRADE_FLUX`. This is the strongest item in this subsection: the other three restrict or disable a safeguard, this

one means the pool cannot be funded from the transparent side at all. `ContextualCheckTransaction` rejects any transaction that has transparent inputs (`tx.vin.size()` nonempty) together with a nonempty `vJoinSplit` or `vShieldedOutput`, with reject reason `bad-txns-fluxrebrand-vprivacy-not-empty` (SHIPPED, [fluxd/src/main.cpp:1398-1408]), once `UPGRADE_FLUX` is active — mainnet height 835,554 (SHIPPED, [fluxd/src/chainparams.cpp:137-139]). The in-source comment states the intent directly: "If flux rebrand is active, we no longer want to allow coins to go into the privacy pool" ([fluxd/src/main.cpp:1398-1400]). This is a **Flux-original consensus policy, not a Zcash ZIP** — nothing in the Sprout or Sapling specification restricts shielding transparent value this way. The check never inspects `vShieldedSpend`, so shielded-to-shielded (z→z) and shielded-to-transparent (z→t) transactions remain unrestricted; only the transparent-to-shielded (t→z) and mixed t/z→z paths that would add new value to the pool are blocked. The pool is therefore closed and monotonically non-increasing in value: value can leave for the transparent side or move among existing shielded holders, but no new unit can enter.

At chain tip 2,917,115 the pool holds 77,188.759 213 24 FLUX in Sapling and 19,291.177 350 41 FLUX in Sprout — 96,479.936 563 65 FLUX total (0.0225% of issued supply) — [measured: `api.runonflux.io/daemon/getblockchaininfo`, field `valuePools`, at `blocks` = 2,917,115, 2026-09-03]. `UPGRADE_FLUX` activated at height 835,554, approximately 10 April 2021 — the calendar date is carried only as a source comment, "Around 10th April 2021" ([fluxd/src/chainparams.cpp:138]) — so as of this writing (2026-09-03) the pool has been closed to new entrants for approximately 5.4 years. This also bounds what the paper can say about shielded privacy in practice: the anonymity set a shielded transaction hides within — the notes and holders it is indistinguishable from — cannot grow from new participants while the pool stays closed, and neither its note count nor its holder count is exposed by any RPC surface this section documents (§20.6), so neither is measured here.

Turnstile enforcement (ZIP-209) is compiled in but dormant. `ConnectBlock()` contains the inherited check that rejects a block if the running chain-wide Sprout or Sapling balance would go negative — the mechanism that would catch exactly the counterfeiting scenario in §20.3 (SHIPPED, [fluxd/src/main.cpp:3838-3862]). It is gated by `fZIP209Enabled`, whose class default is `false` (SHIPPED, [fluxd/src/chainparams.h:200]); the mainnet checkpoint block and Sprout balance-checkpoint values that would turn it on are present in the source but left commented out, and the values they carry are the unmodified upstream Zcash-mainnet numbers, never adapted to Flux's own chain (SHIPPED, [fluxd/src/chainparams.cpp:322-327]; same pattern on testnet at [fluxd/src/chainparams.cpp:527-530]). Only regtest turns it on explicitly (SHIPPED, [fluxd/src/chainparams.cpp:732]). Turnstile enforcement is precisely the mechanism that makes the roadmap's own commitment true — "Supply stays publicly verifiable, with pool totals visible so the arithmetic always checks out" [roadmap: flux-roadmap-public.md:174] — and on mainnet and testnet it is off. Per G9, the paper states this as the gap between the commitment and the deployed system rather than resolving it: turning `fZIP209Enabled` on, with checkpoint values recomputed for Flux's own chain history rather than copied from Zcash's, is what would make the sentence true.

Coinbase-must-be-shielded is disabled from `UPGRADE_FLUX` onward. Mainnet and testnet set `fCoinbaseMustBeProtected = true`; regtest sets it `false` (SHIPPED, [fluxd/src/chainparams.cpp:89,341,545]). Enforcement inside `Consensus::CheckTxInputs` is itself gated by whether the "Flux rebrand" upgrade is active: before height 835,554, a coinbase output could only be spent to a shielded address; from that height on, the restriction is lifted and coinbase output may be spent transparently (SHIPPED, [fluxd/src/main.cpp:3200,3228-3237]), mirrored on the wallet side (SHIPPED, [fluxd/src/wallet/wallet.cpp:3506]). This is a narrower, separate switch from the pool-closure restriction discussed first in this subsection: the two share an activation height but not a mechanism, and coinbase transparency does not by

itself reopen the shielded pool to new value.

`z_setmigration` unconditionally throws post-`UPGRADE_FLUX`. The inherited ZIP-308 Sprout→Sapling migration RPCs, `z_setmigration` and `z_getmigrationstatus`, are backed by a still-compiled `AsyncRPCOperation_saplingmigration` (SHIPPED, [fluxd/src/wallet/asyncrpcoperation_saplingmigration.h:11]). But once the Flux-rebrand upgrade is active, `z_setmigration` immediately throws `"Flux fork active: z_setmigration has been deprecated"` (SHIPPED, [fluxd/src/wallet/rpcwallet.cpp:4110-4114]), and `UPGRADE_FLUX` activates at mainnet height 835,554 (SHIPPED, [fluxd/src/chainparams.cpp:137-139]). The migration operation code is reachable only through an internal wallet flag that is itself settable only via this now-error-throwing RPC — the migration path has been dead code on mainnet since that height, and Sprout value has no in-wallet path to Sapling.

Sapling is gated on `UPGRADE_ACADIA`, not a named Overwinter/Sapling upgrade. As stated in §20.2, this is a naming divergence rather than a functional gap, but it means a reader looking for "Sapling activation" in `fluxd`'s upgrade table by Zcash's own naming convention will not find it (SHIPPED, [fluxd/src/consensus/params.h:24-39]).

The roadmap's privacy commitments should be read against this closure rather than independently of it. "Flux descends from the Zcash lineage. The shielded machinery is part of what we inherited, and we are bringing private transactions back" [roadmap: flux-roadmap-public.md:167] and "Shielded by default in the wallets we make" [roadmap: flux-roadmap-public.md:172] read, taken on their own, like wallet-integration tasks — pick a construction, ship it in Zelcore and SSP, default new addresses to z-addr. Given the closure above, they are not only that. The t→z restriction is a consensus rule enforced in `ContextualCheckTransaction`, not a wallet setting, so no wallet change can place new value in the pool while that rule stands. **Reopening the pool to new entrants would itself be a consensus change, and it is not going to happen**: the pools are to be retired instead (§20.9, [intent: design intake]). An earlier round of the intake had confirmed reopening as part of a privacy update, and this paper carried it as a sequenced dependency ahead of wallet integration; that is superseded. The consensus change now in prospect is the opposite one — invalidating shielded spends after a sunset height — and §21 and §27 are updated accordingly. This is a narrower decision than the choice of successor shielded construction discussed in §20.8: the roadmap's position that it will not date a cryptographic change before its audit is scoped concerns which construction to adopt next (Orchard/Halo2 or otherwise), not the reopening of the already-deployed Sapling and Sprout pools to new transparent value, which needs no new circuit. It also bears directly on §21: a private-payment construction can be specified against the shielded primitives in §20.2, but with reopening cancelled and both pools retiring, §21's constructions are inapplicable to the target architecture, and that section is retained for its requirement set and its measurements (Definition 21.1).

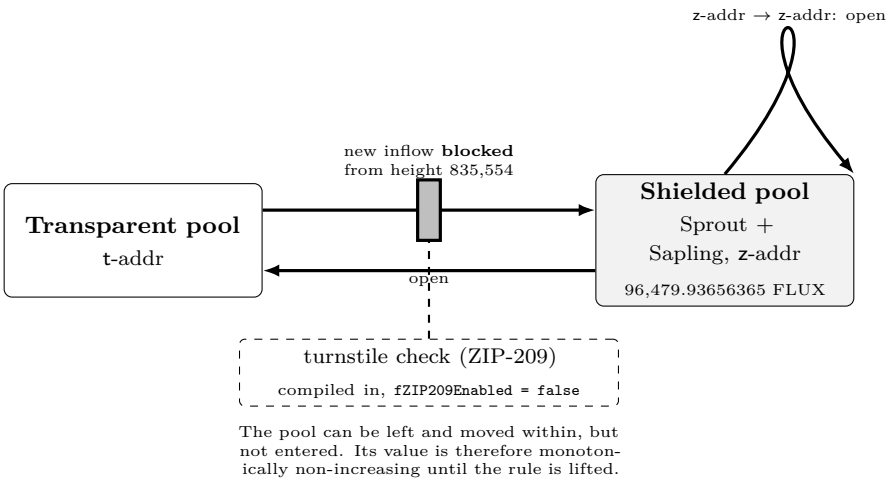


Figure 16: Value flow between `fluxd`'s transparent and shielded pools, showing the turnstile check as present in code but inactive, and the height-835,554 restriction on new transparent-to-shielded inflow. SHIPPED, [fluxd/src/main.cpp:1398-1408,3838-3862]; [fluxd/src/chainparams.h:200].

20.5 Supply accounting

`fluxd` exposes shielded-pool accounting through a `valuePools` array on the `getblockchaininfo` and `getblockheader` RPCs, with one entry per pool (`id "sprout"` or `"sapling"`), each carrying `monitored`, `chainValue`, `chainValueZat`, `valueDelta`, and `valueDeltaZat` (SHIPPED, [fluxd/src/rpc/blockchain.cpp:84-101]), reported both per block ([fluxd/src/rpc/blockchain.cpp:279-282]) and at the chain tip ([fluxd/src/rpc/blockchain.cpp:1068-1071]). These fields are what a supply audit would read from: the total issued supply S that §5 defines is, by the roadmap's own description, "the transparent set plus the shielded pools" [roadmap: flux-roadmap-public.md:185], and the roadmap states that this reconciles with the emission model to within 0.0005% [roadmap: flux-roadmap-public.md:185].

The accounting fields are computed and exposed regardless of the turnstile switch discussed in §20.4. What the disabled switch removes is not the numbers themselves but the consensus-enforced invariant that would make a negative pool balance — the signature of exactly the kind of undetected forgery discussed in §20.3 — impossible to sustain on-chain without triggering block rejection. `valuePools` is a reporting surface; turnstile enforcement is the check that would make the report trustworthy on its own. With `fZIP209Enabled = false` on both live networks, `fluxd` today performs no automated on-chain detection of a negative Sprout or Sapling pool balance outside of whatever manual monitoring an operator does against the `valuePools` field (SHIPPED, [fluxd/src/chainparams.h:200]). This is the same gap named in §20.4 and G9, restated here in its supply-accounting form because it is what connects this section to §5's supply function: S as defined there is only as trustworthy as the pool-balance fields that compose it, and those fields currently carry no consensus-level non-negativity guarantee.

20.6 The RPC surface

The wallet RPC command table exposes the full set of z-address operations (SHIPPED, [fluxd/src/wallet/rpcwallet.cpp:4994-5021]). This is the toolkit §21's private-payment construction is built on, and it is enumerated here precisely rather than by example:

Key and address management `z_getnewaddress`, `z_listaddresses`, `z_exportkey`, `z_importkey`, `z_exportviewingkey`, `z_importviewingkey` (ivk), `z_exportwallet`, `z_importwallet` (SHIPPED, [fluxd/src/wallet/rpcwallet.cpp:4994-5018]). Full and incoming Sapling viewing keys (ak, nk, ovk and the incoming-viewing-key form) are the objects these last two RPCs move (SHIPPED, [fluxd/src/flux/Address.hpp:32-222]), and `zip32.h` carries the ZIP-32

seed fingerprint and parent-FVK tag fields that identify derived keys (SHIPPED, [fluxd/src/flux/zip32.h:32,55,101]).

Balance inspection `z_listreceivedbyaddress`, `z_listunspent`, `z_getbalance`, `z_gettotalbalance` (SHIPPED, [fluxd/src/wallet/rpcwallet.cpp:4994-5018]).

Value movement `z_sendmany`, `z_shieldcoinbase`, `z_setmigration`/`z_getmigrationstatus` (dead post-height-835,554, §20.4), and `z_mergetoaddress` — the last gated behind an explicit `-zmergetoaddress` flag (SHIPPED, [fluxd/src/wallet/rpcwallet.cpp:4463-4466]; [fluxd/src/init.cpp:905-906]).

Asynchronous operation tracking `z_getoperationstatus`, `z_getoperationresult`, `z_listoperationids` (SHIPPED, [fluxd/src/wallet/rpcwallet.cpp:4994-5018]).

Raw Sprout operations `zcbenchmark`, `zcrawkeygen`, `zcrawjoinsplit`, `zcrawreceive`, `zcsamplejoinsplit` (SHIPPED, [fluxd/src/wallet/rpcwallet.cpp:4994-5021]).

Payment disclosure `z_getpaymentdisclosure` and `z_validatepaymentdisclosure` are registered under the RPC category `"disclosure"` (SHIPPED, [fluxd/src/wallet/rpcwallet.cpp:5020-5021]), backed by the note-decryption path described in §20.2. They are compiled in but off by default: both require `fExperimentalMode`, whose global default is `false` (IN-PROGRESS, [fluxd/src/main.cpp:77]), set only via `-experimentalfeatures`, plus their own per-feature flag; enabling one without the other aborts startup with an explicit error (IN-PROGRESS, [fluxd/src/init.cpp:895,903-904]). Payment disclosure is present code, not an active default.

Payment disclosure and viewing-key export together give a selective-disclosure capability — reveal a transaction, or a stream of incoming payments, to a chosen third party without revealing anything else — that is exactly the primitive §21's payer/network problem needs; §21 states the construction it builds from this surface, or states precisely where the surface stops short of it.

20.7 Post-quantum scope: what is bounded, and why

Groth16 as deployed here is a pairing-based SNARK over BLS12-381 — the 192-byte proof format fixed by `GROTH_PROOF_SIZE` is exactly the compressed $G_1 \| G_2 \| G_1$ encoding this construction uses [29] (SHIPPED, [fluxd/src/flux/JoinSplit.hpp:22-26]). Its soundness rests on pairing- and discrete-log-type hardness assumptions over that curve, which a sufficiently large quantum computer running Shor's algorithm breaks. This is a property of the scheme itself, not of Flux's use of it, and it holds regardless of who ran the trusted setup in §20.3: **the Sapling shielded layer, as deployed, is not post-quantum secure.** No post-quantum-related code, comment, or TODO exists anywhere under `src/` — a repository-wide search for `quantum` returns zero hits (SHIPPED, [fluxd/src:grep -rn quantum, zero matches]).

Given that, the team's stated position is to bound the post-quantum migration deliberately rather than claim more than the deployed cryptography supports. What is to become post-quantum is consensus, transparent transactions, and the keys securing them; the shielded layer is explicitly placed out of scope for now, with post-quantum privacy construction to follow later as the field matures [intent: design intake] (VISION; after the retirement decision of §20.9, any such construction is a future option, not a scheduled item). This scope decision is presented in the design intake as coherent and defensible rather than as an admitted gap, on the reasoning that privacy is already optional at the protocol level (the roadmap's own position, quoted at the head of this section), so a transparent-layer-first migration does not leave the part of the system that everyone must use exposed while the part that is opt-in waits. The paper adopts that framing because the alternative is worse: an unqualified "quantum-ready" claim made while a live Groth16-based shielded pool sits underneath it would be a claim a reviewer familiar with the scheme would immediately falsify. A scope statement this specific — consensus and transparent

keys first, shielded privacy deferred and named as deferred — is the position that survives that scrutiny; a blanket claim is not.

One distinction has to be kept strict, because it is the likeliest error in this area. Removing a trusted setup and achieving post-quantum security are two different properties, and a future construction can have either without the other. Halo2, the proof system behind Zcash's Orchard pool, eliminates the per-circuit trusted setup entirely by replacing it with a transparent, universal setup [12] — but it does so by resting on discrete-log hardness over a different curve, which is exactly as vulnerable to a quantum adversary as the pairing assumption Groth16 uses. Adopting Halo2/Orchard, discussed next, would answer the trusted-setup question and would do nothing for the post-quantum question. The two must not be conflated in either direction.

20.8 What a migration to Orchard/Halo2 would, and would not, buy

Orchard is absent from `fluxd` today (§20.2), and no committed timeline or specific target construction exists yet: the roadmap places "Chain privacy implementation" in H2 2027, explicitly "on the schedule set by the Q4 2026 plan" [roadmap: flux-roadmap-public.md:158] (PLANNED), and states that the Q4 2026 privacy groundwork work is where "which shielded construction to adopt" is still to be settled [roadmap: flux-roadmap-public.md:177] (PLANNED). Whether that construction turns out to be Orchard/Halo2 specifically, some other Zcash-lineage successor, or something else is not yet decided by the team's own account, so this subsection is stated as VISION rather than as a scheduled item.

The argument for making that move is specific, not generic, and worth stating precisely: adopting a Halo2-class proving system removes the inherited trusted setup described in §20.3 entirely, and that directly serves the stated decentralization requirement — a chain whose privacy layer's soundness depends on a multi-party ceremony run once, by a fixed set of participants, in 2018, is not decentralized in the same sense the rest of the design intake asks for [intent: design intake] (VISION). That argument stands on its own regardless of the post-quantum question, and the paper states it as such rather than bundling it with a PQ claim it does not support (§20.7).

What it would not buy is equally specific: no candidate successor construction under discussion is post-quantum secure, so a migration to Orchard/Halo2 closes the trusted-setup gap while leaving the shielded layer's long-term cryptographic exposure to a quantum adversary exactly where it is today. The team's stated position on timing is to not pin one: "We will not pin a date on a cryptographic change before its audit is scoped. The direction is not in question" [roadmap: flux-roadmap-public.md:177] (PLANNED). The paper records that position rather than supplying a date the source does not give.

20.9 Retiring the pools: the decision, and the measurements behind it

Both shielded pools are to be retired [intent: design intake] (PLANNED). The team raised the possibility alongside reopening and migrating — destroying the existing pool outright to simplify the chain, reduce maintained code and obtain a more thorough audit of what remains, with a stated preference for C++ over Rust [intent: design intake] — and, presented with the analysis below, took it.

This subsection records the measurement that motivated the decision, the constraint that made it sharper than it first appears, and the schedule. The argument is retained in full rather than compressed to its outcome, because a decision to destroy a live mechanism holding user value should remain auditable against the reasoning that produced it.

What is actually in the pools. Both legacy pools are live and both are small.

Table 47: Shielded pool balances at height 2,917,115, from `fluxd`'s own `valuePools` reporting surface. Circulating supply is the emission model's value at that height (Section 5). SHIPPED [measured: api.runonflux.io/daemon/getblockchaininfo, field `valuePools`, 2026-09-03].

Pool	Balance (FLUX)	Share of circulating supply
Sprout (JoinSplit lineage)	19,291.17735041	0.0045 %
Sapling	77,188.75921324	0.0180 %
Total shielded	96,479.93656365	0.0225 %

Three facts follow, and together they carry the argument. First, the combined shielded balance is about one part in 4,500 of circulating supply. Second, the balance is *monotonically non-increasing*: the transparent-input gate of §20.4 rejects any transaction carrying both a transparent input and a shielded output, so value has been unable to enter either pool since height 835,554 [fluxd/src/main.cpp:1398–1408]. Third — and this is the fact most often missed — the gate keys on `tx.vin.size()` alone, so z-addr → z-addr and z-addr → t-addr transactions remain fully valid. The pools are sealed from outside and unimpaired within: existing holders retain full use and can exit at will.

The constraint that decides it. The stated preferences — newest Zcash work, less code, a more thorough audit, and C++ over Rust — cannot all be satisfied, because they point in opposite directions. Sapling proving and verification in `fluxd` run through `librustzcash` (§20.2), which is Rust. Orchard's Halo2 proving system exists as a Rust implementation and has no production C++ equivalent. Zcash's own trajectory has been to move consensus-critical cryptography *into* Rust, not out of it. So "port the newest Zcash work" and "prefer C++" are not jointly achievable: adopting newer Zcash cryptography means more Rust in the consensus path, not less [inferred].

That leaves exactly three self-consistent positions, and the preference set selects among them cleanly:

Table 48: The three coherent options, against the team's own stated preferences.

Option	Chain privacy	Rust in consensus	Trusted setup	Audit surface
(A) Reopen Sapling	yes	retained	inherited, two ceremonies	unchanged, large
(B) Retire both pools	**none**	**removable**	**none**	**smallest**
(C) Retire, later adopt Orchard	deferred	increased	none (Halo2)	large, later

Option (B) is the decision; (C) remains available later. Four arguments, in order of weight.

1. **A shielded pool does not buy what the thesis needs.** The workload is public by construction. A Flux application specification is a permanent on-chain message, and the control plane serves the global specification set over a public endpoint [flux-domain-manager/src/services/flux/dataFetcher.js:47-71]. What a tenant deploys, the resources it claims and the owner it binds to are therefore public whatever the payment rail does. Shielding the payment hides the *funding source*, not the workload graph. That is a real but narrow benefit, and §21 shows it is also the harder half to construct correctly.

2. **Every stated preference points the same way.** (A) and (C) both keep or expand Rust in the consensus path; only (B) permits removing `librustzcash`, the JoinSplit and Sapling circuits, the note-commitment trees and the nullifier sets from the daemon. If a smaller, C++-centred, thoroughly audited consensus layer is the goal, (B) is the only option that delivers it.

3. **It closes a supply-accounting gap the paper already documents.** Section 20.5 records that `valuePools` is a reporting surface with no consensus enforcement behind it: a negative pool balance — the signature of an inflation bug in the shielded code — would not by itself cause block rejection, because ZIP-209 turnstile enforcement is compiled in but dormant against unmodified upstream-Zcash checkpoint constants [fluxd/src/chainparams.cpp:322-327]. Removing the pools removes that entire class of unauditable supply risk, and makes total supply exactly reconstructible from transparent UTXOs. For a design whose monetary section rests on supply transparency, that is a direct gain.

4. **The value at risk is bounded and measured.** 96,479.94 FLUX, in a pool that cannot grow. This is the number the decision should be made against, and it is small enough that a generous migration window costs little and dishonours no one.

The retirement schedule. The decision is settled; the heights below were the paper's recommendation and are confirmed by the author (Remark 20.2).

- **Announcement at publication**, i.e. with this paper (2026-09-07), alongside the notice of the parallel-asset one-way switch at $h^{\text{mig}}_{\text{open}}$ (§6.8). One consolidation message covering both is better than two, and the holder sets barely overlap.

- **Sunset at** $h_{\text{shield}} = 3{,}600{,}000$, about 2027-04-27 at the 30 s target: 232 days after publication, 150,000 blocks after $h^{\text{mig}}_{\text{open}}$ and 187,502 blocks *before* h_{PA} [intent: 08-answers-round-2.md, round 37]. After this height, transactions carrying `vShieldedSpend` or `vJoinSplit` are invalid; notes not swept are permanently unspendable. Seven and a half months is deliberately longer than the roughly five-month parallel-asset window (§22.8): that window ends in forfeiture to a named recipient, whereas this one ends in destruction, which warrants more notice for less value. The height is a consensus rule and must ship in a `fluxd` release ahead of it.

  Decided. Carry it in the `fluxd` upgrade the roadmap already plans for H1 2027 (collateral maturity, §27), so that retirement adds no release of its own before PNR.

- **No change to exit paths during the window.** z-addr → t-addr already works and needs no consensus change to keep working; the window requires an announcement, not a code change.

- **Activate ZIP-209 turnstile enforcement with Flux-correct constants for the duration of the window.** Whatever is decided about the pools' future, the current state — enforcement code compiled in but dormant, parameterised by another chain's checkpoints — is the worst of the available positions, because it carries the maintenance cost of the mechanism without its guarantee. During a drain-only window the turnstile is a cheap and exactly-correct safety net: the pool balance must only decrease, which is precisely what the turnstile checks.

- **Direct outreach, not announcement alone.** The holder set is small enough to make this tractable and small enough that a purely passive notice would be a poor effort.

Remark 20.2 (Schedule confirmed). The heights above were proposed by this paper and are **confirmed by the author** [intent: design intake]; the sunset was then brought forward to $h_{\text{shield}} = 3{,}600{,}000$ and the announcement to publication [intent: 08-answers-round-2.md, round 37]. Activating ZIP-209 turnstile enforcement with Flux-correct constants for the window remains this paper's recommendation, not a confirmed decision: round 11 confirmed the numbers, and a switch is not a number [intent: design intake]. The retirement decision and its heights are settled, and §21's construction questions are consequently not live (Definition 21.33).

Remark 20.3 (What retirement costs, stated without softening). Three things are given up, and the paper names them rather than letting the argument above stand unopposed.

Flux will have no chain-level privacy. Not reduced privacy — none. Every transfer, every balance and every payment for a deployment will be publicly linkable for anyone willing to do the analysis, exactly as on Bitcoin. The design intake asked for privacy-preserving settlement rails [intent: design intake]; after retirement the honest description is that Flux settles transparently and treats payment privacy as an unsolved problem rather than a delivered feature. Section 20.7 and §21 are the two places this paper previously claimed otherwise, and both are now corrected.

Value will be destroyed at h_{shield}. Notes not swept before the sunset become permanently unspendable. The bound is 96,479.94 FLUX and shrinking, and the window is long, but this is confiscation by inaction and calling it anything softer would be dishonest. It is the reason the schedule recommends direct outreach rather than announcement alone.

The option is one-directional in practice. Rebuilding a shielded pool later means adopting Orchard/Halo2 or an equivalent — a multi-year cryptographic project, in Rust, against the stated language preference. Option (C) remains formally available and should not be described as though it were cheap.

21 Private payment with public obligation

Every node on the network must be able to decide, on its own and without asking anyone, whether the workload it is about to run has been paid for. That decision has to be reproducible by thousands of mutually distrusting machines from public data alone, which is precisely why the payment that answers it is, today, a transparent transaction whose inputs name its sender. The tension this section addresses is one sentence long: *the network must be convinced that a deployment is funded while learning neither who paid nor which payer belongs to which workload.* The workload itself is public and must stay public — a node cannot run a container whose image it is not told — so the object to hide is not the obligation but the payer, and the edge between the payer and the obligation. What follows states that requirement formally, shows which parts of it the deployed Sapling toolkit already satisfies, shows which part is unsatisfiable by construction and says so rather than omitting it, measures the anonymity set that actually exists rather than assuming one, and names the single consensus rule that gates the whole thing. The most important result in this section is a *negative scope* result: the general problem is smaller than it looks, and the part that remains genuinely open is confined to one optional path.

Remark 21.1 (Status of this section after the retirement decision). **Flux is retiring both shielded pools** (Section 20.9, [intent: design intake]). Every *construction* in this section is therefore inapplicable to the target architecture: there will be no pool to pay into, no Sapling output to build a payment from, no anonymity set to grow, and no proving system to select. The section is retained, unmodified in its technical content, for three reasons, and a reader should hold all three in mind while reading it.

1. **The requirement survives the decision.** Definition 21.7 and Definition 21.8 state what private payment for a public obligation must satisfy. That statement is independent of any construction and independent of whether Flux carries a shielded pool. If Flux ever revisits privacy — via Orchard/Halo2 or otherwise — this is the specification the work must meet, and it is written down here rather than needing to be rediscovered.

2. **Two of its results are why retirement is defensible.** §21.2 proves amount hiding is unsatisfiable under PNR, and §21.4 shows the residual problem is confined to one optional path. Together they establish that a shielded pool was buying considerably less than it appears to — which is a load- bearing input to the decision, not a consolation after it.

3. **The measurement is a permanent record.** §21.9 measures the anonymity set that

actually existed on Flux. After retirement that measurement cannot be repeated, and it is the honest empirical answer to how much privacy the deployed pool ever provided.

Read the constructions below as *what would have been required*, not as what is planned. Where the text says PLANNED of a shielded construction, that tag is now superseded by the retirement decision and should be read as conditional on a pool existing.

Remark 21.2 (Symbols introduced here). This section uses, in addition to `notation.tex`: m_a for the registration or update message of application a; $H_a := \mathsf{SHA256}(m_a)$ for its hash, the value FluxOS places in `OP_RETURN`; e_a for the paid-through height; A_{sink} for the payment sink, §7's sink; v for a paid amount in sat; v_{bal} for a Sapling `valueBalance`; u for an individual FluxOS node; AS for an anonymity set; ϵ_{sc} for a side-channel advantage; $\mathsf{NCT}_{\text{ent}}$ and rt for an entitlement tree and its root; s for a credential secret; j for an epoch index and T_{ep} for an epoch length; and xsk for a ZIP-32 child spending key. These are proposed additions to the notation index (Appendix D) and are used nowhere else in the paper.

21.1 The verifier, and the requirement pair

21.1.1 What is public by construction

Three things are public because the system cannot function otherwise, and it is worth separating them before any privacy claim is made.

1. **The workload.** An application's public part $\mathsf{spec}_{\text{pub}}(a) = (\mathsf{name}, H_a, \mathsf{owner}, k, d, h_0, \mathsf{enterprise?})$ is stored by every node, because placement is opportunistic self-selection and any node may be the one that runs a (SHIPPED, [flux/ZelBack/src/services/appDatabase/registryManager.js:1714]). Content is a separate matter: of the 1,195 deployed applications, 972 publish their `compose` block in the clear and 223 (18.7 %) are enterprise records whose `compose` and `contacts` are stored zeroed [measured: api.runonflux.io/apps/globalappsspecifications, 2026-09-03].

2. **The paid-through height.** e_a must be a public function of the chain, or nodes disagree about whether to keep running a container.

3. **The amount, under PNR.** This is derived rather than chosen, and §21.2 proves it.

What is *not* required to be public is the identity of the party that moved the value, and the association between that party and the application. That is the whole of the privacy goal here, and stating it this narrowly is what makes the rest of the section tractable.

21.1.2 The verifier

Definition 21.3 (Funding scheme and verifier)**.** Let Apps be the set of application identifiers, $\mathcal{C}$ the set of chain prefixes, Tx the set of transactions and Msg the set of FluxOS messages. A *funding scheme* is a pair $(\mathsf{Fund}, \mathsf{Verify})$ with

$$\mathsf{Fund} : \mathcal{U} \times \mathsf{Apps} \longrightarrow \mathsf{Tx} \times (\mathsf{Msg} \cup \{\varepsilon\}), \qquad \mathsf{Verify}_u : \mathsf{Apps} \times \mathbb{N} \times \mathcal{C} \longrightarrow \mathbb{N} \cup \{\bot\},$$

where Fund is run by a principal and Verify_u is run by FluxOS node u on its local chain prefix $C_{\le h}$ and its local message store, returning a paid-through height e_a or $\bot$. Funded status is $\mathsf{Funded}(a, h) := \mathbf{1}[h \le e_a]$.

Assumption 21.4 (Verifier model, from §2)**.** Node u holds $C_{\le h}$, the FluxOS temporary and permanent message sets, and *no secret*: no shared key, no issuer key, and no viewing key it did not derive from public data. Node u needs no message from any other verifier to evaluate Verify_u. This is a description of the deployed system, not a design preference: `explorerService.processInsight` recognises a payment by output address and `OP_RETURN`

and `messageVerifier.checkAndRequestApp` promotes the registration message, on every node, from chain data alone (SHIPPED, [flux/ZelBack/src/services/explorerService.js:309–345], [flux/ZelBack/src/services/appMessaging/messageVerifier.js:619–897]); CumulusVPN states the same property of its gateways — "every gateway runs the same deterministic scan and therefore reaches the same `paid_until` map [...] with no server-to-server coordination" (SHIPPED, [cumulusvpn/gateway/internal/entitle/entitle.go:1–5]).

Assumption 21.5 (Adversary, from §2)**.** The adversary $\mathcal{Z}$ is a node operator. It sees the whole chain, the public specification of every application, all FluxOS gossip, and the client traffic arriving at the nodes it runs; it may run any number of nodes (the measured maximum under one signing key is 237, 3.7 % of the fleet [measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03]); it may be the *owner* of the application whose payer it wants to identify; and it may hold shielded funds. It is bounded by $\mathcal{C}_{\mathcal{Z}}$ for anything requiring capital and is otherwise probabilistic polynomial time.

21.1.3 The hard requirements

Definition 21.6 (Public obligation)**.** A funding scheme satisfies *public obligation* if:

V0 (no secret). Verify_u is a deterministic function of $C_{\le h}$ and public messages, for every u.

S (soundness). For every PPT payer $\mathcal{Z}$,

$$\Pr[\mathsf{Verify}_u(a, h, C_{\le h}) = e > h \ \wedge \ \textstyle\sum \text{value confirmed to } A_{\text{sink}} \text{ bound to } H_a \text{ by } h < \mathcal{P}(a)] \le \text{negl}.$$

C (consistency). Honest u, u' with the same chain prefix return the same e_a.

L (liveness under partition). Verify_u requires no message from any other party; a partitioned node evaluates on its local prefix and converges when the prefix does.

D (no double-funding). One confirmed transfer contributes to at most one pair (a, period).

Definition 21.7 (Payer indistinguishability, PI)**.** $\mathcal{Z}$ names two principals $\mathcal{U}_0, \mathcal{U}_1$, each able to fund, and an application a; $\mathcal{Z}$ may own a and any set of nodes. The challenger draws $b \in \{0, 1\}$ uniformly and runs $\mathsf{Fund}(\mathcal{U}_b, a)$; $\mathcal{Z}$ observes the resulting chain and network state and outputs b'. Define

$$\mathrm{Adv}^{\mathrm{PI}}(\mathcal{Z}) := \left|\Pr[b' = b] - \tfrac{1}{2}\right| \in [0, \tfrac{1}{2}].$$

The scheme has *cryptographic* PI if $\mathrm{Adv}^{\mathrm{PI}}(\mathcal{Z}) \le \text{negl}$ whenever $\mathcal{U}_0$ and $\mathcal{U}_1$ fund from the same pool, at the same time, with the same amount. Its *effective* advantage is $\text{negl} + \epsilon_{\text{sc}}$, where ϵ_{sc} collects timing, amount and network-metadata leakage. §21.9 measures ϵ_{sc}'s inputs rather than assuming them away.

Definition 21.8 (Renewal unlinkability, RU)**.** *Payer-side:* for two funding events of the same a at periods $i \ne j$, $\mathcal{Z}$ cannot distinguish whether one principal produced both. *Cross-application:* for $a \ne a'$, $\mathcal{Z}$ cannot distinguish whether one principal funded both. Two renewals of the same a are linked by H_a by construction — that is the obligation being public — so RU never denotes hiding *that* a was renewed.

Definition 21.9 (Amount hiding, AH)**.** $\mathcal{Z}$ learns nothing about v beyond $v \ge \mathcal{P}(a)$.

Definition 21.10 (Forward secrecy, FS)**.** Compromise at period i of all key material used to produce funding events of periods $\ge i$ does not increase $\mathrm{Adv}^{\mathrm{PI}}$ and does not break RU for periods $< i$.

Definitions 21.6 and 21.7 are the *hard* requirements; Definitions 21.8–21.10 are desirable, and §21.3 states which are achievable in which shape. Content privacy — the hiding of $\mathsf{spec}_{\text{priv}}(a)$ from non-selected nodes by enterprise encryption — is owned by §10 and constrained here only in that a funding scheme must not reduce it beyond what $\mathcal{P}(a)$ already reveals. AH is the sole point at which the two interact, which brings us to the first result.

21.2 Amount hiding is unsatisfiable under PNR

This is stated early because it is a genuine conflict between two of this paper's own mechanisms, and because a reader who expects Zcash-style amount privacy on hosting payments should be disabused before reading a construction that does not deliver it.

Property 21.11 (Amount recovery from the fee pool)**.** Let $\rho(h) \in (0, 1]$ be the operator share at height h and write $r(h) = 100\,\rho(h) \in \{1, \ldots, 100\}$. Under PNR (§7, PS1–PS4), a hosting payment of price $\mathcal{P} \in \mathbb{N}$ sat credits $A_\Phi = \lfloor r(h)\,\mathcal{P}/100 \rfloor$ sat to the consensus fee pool Φ, and the per-block drip $D_h = \left\lfloor \frac{\Phi_{h-1}}{K} \right\rfloor$ appears in the coinbase. Then A_Φ is a public function of the chain, and consequently

$$\mathcal{P} \in \left[\frac{100\,A_\Phi}{r(h)},\ \frac{100\,(A_\Phi + 1)}{r(h)} \right), \qquad \text{an interval of width } \frac{100}{r(h)}\,\text{sat}.$$

At $\rho = 0.20$ the price is recoverable to within 5 sat; at the $\rho_{\max} = 0.50$ ceiling, to within 2 sat. Therefore **no funding scheme that settles through PNR satisfies AH** (Definition 21.9).

Proof. Every node must compute Φ_h exactly in order to validate the coinbase, since D_h is a consensus-checked function of Φ_{h-1} and a node that cannot compute the pool cannot decide block validity. Hence Φ_h is a deterministic public function of $C_{\le h}$, and so is each receipt's A_Φ: under PS4 of §7 it is computed from the value of a transparent output paying A_{sink}, and the block's inflow F_h is the sum of these. Inverting the floor gives the stated interval, whose width is $100/r$. Any scheme achieving AH would have to hide A_Φ from at least one honest verifier, contradicting checkability of the coinbase; a hidden pool cannot drip a checkable coinbase. □

Remark 21.12 (§7 and §21 cannot both have what they want). The resolution recorded in this paper is that amount privacy is **out of scope for PNR-typed payments, by construction rather than by omission**. The cost is asymmetric: for the 972 public-content applications it is nil, because $\mathcal{P}$ is already a public function of a public specification; for the 223 enterprise applications the price reveals $\langle \mathbf{p}, \mathbf{r} \rangle$, a coarse size fingerprint of encrypted content. The only escape — exempting a payment class from PNR — contradicts §7's requirement that all hosting revenue settle through the split, and is left as an open decision (Q5, Table 55).

21.3 Three shapes, kept apart

Most of the difficulty attributed to "private payment on Flux" comes from conflating three problems whose hardness differs sharply. Table 49 separates them, and every later claim in this section is scoped to one of the three.

Table 49: The three shapes of the private-payment problem. Only shapes B and C carry a standing object that must be re-proved; shape A does not, which is the content of Proposition 21.13.

	The obligation names	Example	What must be hidden	Maturity
A	a public workload H_a, which every node must know in order to run it	application registration and renewal; the direct path of §16	the payer	PLANNED; deployed circuits, gated on §21.5
B	a private user: a pseudonym whose only public attribute is "paid until e"	CumulusVPN, memo = $\mathsf{base58}(\mathsf{SHA256}(pk_t)[0{:}20])$	the link pseudonym ↔ payment, *and* pseudonym ↔ payer	VISION; needs a circuit Flux does not have
C	a standing balance debited over time	deposited credits with auto-renewal, the credit path of §16	the payer, and the link between successive debits	PLANNED; reduces to A when the renewer is the payer's own agent

Table 50 then states, per shape, which requirement is hard, which is desirable, and which cannot be had. It is the map for the rest of the section.

Table 50: Requirements per shape. "hard" = a mechanism is not acceptable without it; "desirable" = wanted, and its absence is stated rather than hidden.

Property	A, direct	C, own agent	C, third-party renewer	B, CumulusVPN-class
V0, S, C, L	hard	hard	hard	hard
D	hard	hard	hard	vacuous (time-based)
PI vs. the network	hard	hard	hard	hard, vs. the gateway
PI vs. the renewer	n/a	n/a	desirable; **open** (R2)	n/a
PI, transparent payer	impossible	—	—	desirable; needs A
RU payer-side	from PI per event	hard	hard	hard, across epochs
RU cross-application	desirable	desirable	desirable	—
AH	**unsatisfiable** (Prop. 21.11)	unsatisfiable	unsatisfiable	n/a, fixed price
FS	desirable	desirable	desirable	desirable

21.4 The scope result

The proposition below is the most consequential finding in this section, because it decides how much of the paper's agent-native argument depends on an unsolved problem. It says: not much.

Proposition 21.13 (No new relation is required for shape A)**.** *Let* Verify_u *be the deployed FluxOS funding rule of §21.7. Then:*

(i) $\mathsf{Verify}_u(a, h, C_{\le h})$ *depends on* $C_{\le h}$ *only through the set of confirmed* funding events*, where a funding event is a confirmed transaction carrying a transparent output* (A_{sink}, v) *together with* $\mathsf{OP_RETURN} = H_a$*. Between funding events, no predicate over hidden state is evaluated.*

(ii) There exists a funding scheme satisfying V0, S, C, L, D and cryptographic PI (Definitions 21.6, 21.7) whose only zero-knowledge component is the Sapling spend statement and the Sapling output statement, both already verified by every `fluxd` *node at consensus (SHIPPED, [fluxd/src/main.cpp:1447–1482], [fluxd/src/primitives/transaction.h:155–242]).*

(iii) Consequently no relation outside the deployed circuits is required for shape A, and in particular "a statement proving that a note was spent to a known application address without revealing the sender" is not a new circuit: it is the Sapling spend statement with a transparent recipient.

Proof sketch. (i) By inspection of the deployed rule (§21.7): e_a is assigned once per funding event from $(v, h', d(m_a))$ and is never re-derived; Verify_u thereafter compares two integers. There is therefore no standing obligation and nothing to prove repeatedly. *(ii)* Take Construction 21.23 below. Its verifier is unchanged from the deployed one because the FluxOS scanner keys on outputs and `OP_RETURN` and never on inputs; V0, S, C, L, D hold by the same argument as for the transparent path, since the value transfer is still a confirmed transparent output to A_{sink}. Cryptographic PI reduces to Sapling spend indistinguishability: two funding transactions produced by $\mathcal{U}_0$ and $\mathcal{U}_1$ spending notes of sufficient value differ only in nf, cv, rk, π and the change-note ciphertext, each of which is hiding under Sapling's stated assumptions [31]; the number of spend and output descriptions, the anchor and the fee are not hidden, so the builder must hold them equal across payers or they fall under ϵ_{sc}. *(iii)* Immediate from (i) and (ii): the verifier needs $v \geq \mathcal{P}$, the binding of the transfer to H_a, and that v actually moved. The first two are transparent fields; the third is exactly what the binding signature and the spend statement establish. No statement *about the application* is proved in zero knowledge, because the application is public. □

Corollary 21.14 (Agent-native direct payment)**.** *When the renewing party is the tenant's own agent, shape C reduces to shape A composed with ZIP-32 child-key derivation (*SHIPPED*, [fluxd/src/flux/zip32.h:53–136]): the "balance" is the value of the notes held under a derived child key, it exists nowhere as protocol state, and each renewal is an independent shape-A funding event.* ***Agent-native direct payment therefore collapses to a plain Sapling spend plus ZIP-32.***

Corollary 21.15 (Blast radius)**.** *The hard version of the problem — repeated unlinkable proofs against a hidden standing object — arises only when something hidden must be* reused *by a party that must not learn the link. That is (a) shape C with a* third-party *renewer, and (b) shape B for payers who cannot or will not use the shielded pool. The agent-native argument of §18 survives with the general problem still open, and the exposure is limited to the optional credit path of §16.*

Remark 21.16 (Constructible is not private)*.* Proposition 21.13 is a statement about the *relation to be proved*, not about deployability or about realised privacy. Construction 21.23 is constructible over deployed circuits; whether it is private in practice depends on the size of the payer's anonymity set (§21.9) and on a consensus rule that currently forbids entering the pool at all (§21.5). The paper claims only the first until the measurement M1 is taken. The limitation is a result, not a consolation: knowing that three-quarters of the problem needs no new cryptography is worth more than a construction for all of it that nobody has.

21.5 The blocker: the shielded pool cannot be entered

Assumption 21.17 (The pool is closed — SHIPPED, and it gates everything below)**.** Since `UPGRADE_FLUX` at mainnet height 835,554, `ContextualCheckTransaction` rejects any transaction that carries transparent inputs (`tx.vin.size()` > 0) together with any new shielded output — any non-empty `vJoinSplit` or `vShieldedOutput` — with reject reason `bad-txns-fluxrebrand-vprivacy-not-empty` (SHIPPED, [fluxd/src/main.cpp:1398–1408]). The in-source comment states the intent: "we no longer want to allow coins to go into the privacy pool". The rule does not inspect `tx.vShieldedSpend`, so z-addr → z-addr and z-addr → t-addr transactions remain valid. **Shielded value can move and can leave; it cannot enter.** This is a Flux-original consensus policy, not an inherited Zcash ZIP — a distinction §20 must preserve.

Corollary 21.18 (Who can use construction A today)**.** *Construction 21.23 would be available only to holders of value shielded before Assumption 21.17 took effect in April 2021:* 77,188.75921324 FLUX *in the Sapling pool and* 19,291.17735041 FLUX *in the Sprout pool at chain tip* 2,917,115*, totalling* 96,479.93656365 FLUX *[measured: api.runonflux.io/daemon/getblockchaininfo (valuePools), 2026-09-03]. Against issued supply of* 429,466,823.97 FLUX *(§5) that is* 0.0225 %*. The*

pool has been closed for 2,081,561 *blocks, approximately* 5.4 *years of wall clock, and its value is monotonically non-increasing: every* z*-addr →* t*-addr hosting payment removes a participant's value from it and can never be replaced.*

Two consequences deserve to be stated plainly rather than buried. First, the roadmap's privacy items — "we are bringing private transactions back", "shielded by default in the wallets we make" — were **never wallet-integration tasks**: re-opening t-addr → z-addr is a consensus change. That distinction outlives the decision that followed it, because it is what made the roadmap's sequencing wrong rather than merely optimistic. **Reopening is now cancelled**: both pools are to be retired instead [intent: design intake] (PLANNED, §20.9). Second, capacity was never the constraint and membership was: at the list-price revenue of the entire measured population — 3,817.62 FLUX per 30.56-day period under Definition 16.3 as corrected, of which 1,784.97 FLUX (46.8 %, over 247 applications) is deployed by the team's own identities and 2,032.65 FLUX (53.2 %, over 948) is third-party [doc: scripts/21-private-renewal.py] — the Sapling pool alone would have funded the whole network's hosting for 1.69 years, or 3.18 years of third-party hosting alone.

The split is taken on the specification's `owner` field — the ZelID — and not on the address that paid, and the distinction is load-bearing rather than pedantic. Under the custodial path of §16 the service holds both the user's ZelID key and their payment key and signs on their behalf (C42), so a card-paid customer deployment settles *from an InFlux-held address while belonging to the customer's ZelID*. Attributing by payer would count every such deployment as the team's own and understate third-party demand by however much of it arrives that way.

Decided. attribution is by `owner` ZelID. Of the four team identities configured in FluxOS, three own nothing or nearly nothing — `usersToExtend` (current) and `fluxTeamFluxID` own no applications and `fluxSupportTeamFluxID` owns one — which is what the configuration implies, since `usersToExtend` confers permission to extend an application *on behalf of its owner* rather than ownership [flux/ZelBack/config/default.js:229]. The entire team share therefore rests on one address, `196GJWyLxzAw...`, holding 246 of the 1,195 applications. Its composition supports the attribution — `FluxWhitepaper`, `FluxWordPress` and `FluxRosettaServer` alongside `BitcoinWhitepaper`, `Aave`, `Electrum`, `EthereumNodeLight`, `Firod` and the 26 folding deployments — and the author confirms it as the team ZelID [intent: design intake]. The configuration itself names that address only as a former extension-permission holder, so the identification rests on that confirmation rather than on any declaration in code.

A pool used for hosting by a handful of pre-2021 holders identifies them as the only possible payers, which is a smaller anonymity set than the transparent path offers.

That second point is what survives retirement, and it deserves emphasis because it bears directly on how much was given up. Definition 21.18 shows the anonymity set of the deployed pool was bounded by who held shielded value before 2021 — a set that could not grow while t-addr → z-addr stayed closed, and that reopening would have grown only slowly and only for new entrants. The practical privacy on offer at the moment the decision was taken was therefore thin: not zero, but a set small enough that using it to pay for hosting would in many cases have been self-identifying. A reader weighing this decision should weigh it against that measured set, not against the privacy a mature shielded pool would provide.

Remark 21.19 (Q1 and Q2 — both resolved by retirement, in opposite directions). Q1 asked *when* t-addr → z-addr would reopen, having been narrowed from *whether*. Q2 asked whether ZIP-209 turnstile enforcement — compiled in but dormant against unmodified upstream-Zcash checkpoint constants — should be activated, and in what order relative to Q1.

Q1 is withdrawn. There is no activation height to choose, because there is no reopening.

Q2 is answered yes, and becomes more urgent rather than less. §20.9 recommends activating turnstile enforcement with Flux-correct constants for the duration of the drain-only window. In that window the turnstile's check is exactly the invariant that must hold — the

pool balance may only decrease — so it is both cheap and precisely correct, and it is the only period in which the pools shrink under a published deadline with no offsetting inflow. Leaving enforcement dormant through the one window in which it is trivially correct would be the worst of the available choices.

Decided. Live for the drain-only window and moot after it (Definition 21.1, Definition 21.19). Q2. ZIP-209 turnstile enforcement, compiled in and dormant with unmodified upstream-Zcash checkpoint constants ([fluxd/src/chainparams.cpp:322–327], [fluxd/src/chainparams.h:200]). Domain: {on with Flux-computed checkpoints; off}. Recommended: on, for the duration of the window; round 11 confirmed the window's heights, not this switch [intent: design intake]. Trade-off — on: a soundness failure in the inherited 2018 parameters is bounded to the pool's declared value, and §5's supply-verifiability claim becomes true; off: a hosting payment could be printed from a forged proof undetectably. §20 makes the same recommendation.

21.6 Why the naive construction fails: CumulusVPN, worked through

CumulusVPN is the shipped instance of the problem in this corpus, and it fails in the exact way the abstract statement predicts. Its flow is (all SHIPPED):

1. The client generates a WireGuard keypair (sk_t, pk_t) on-device.
2. `POST /v1/enroll {pubkey, pow_nonce}` returns `{payment_address, payment_memo, price_flux, ...}` ([cumulusvpn/gateway/internal/api/api.go:177,185–198]). *The gateway now holds pk_t and the client's IP address.*
3. The client pays `price_flux` — 20 FLUX per month as documented — to `payment_address` with `OP_RETURN` memo `CVPN1:code`, where code = base58(SHA256(pk_t)[0:20]), computed byte-identically in gateway and client ([cumulusvpn/gateway/internal/entitle/entitle.go:99–108], [cumulusvpn/clients/core-ts/src/paymentCode.ts:17–23]).
4. Every gateway scans the chain independently; `ValidPayment` requires $30(v + 10^{-9}) \geq \text{price}$ and exactly one `CVPN1:` memo, and grants $\lfloor 30v/\text{price} \rfloor$ days, capping `paid_until[code]` at 720 days ([cumulusvpn/gateway/internal/entitle/entitle.go:218–287]).

As a public-obligation scheme this is correct and complete: V0 holds (chain only), S holds (value to the address with the memo), C holds (identical derived maps), L holds (local scan), and D holds (the memo resolves to one code; stacking is additive). It is the privacy half that fails, and it fails totally.

Property 21.20 (Enrolment collapses the payer anonymity set to one)**.** Let $\mathcal{Z}$ be a gateway with which the client has enrolled, and let $\mathcal{U}_0, \mathcal{U}_1$ pay from distinguishable transparent input sets. Then $\text{Adv}^{\text{PI}}(\mathcal{Z}) = \frac{1}{2}$; that is, $\mathcal{Z}$ wins the game of Definition 21.7 with probability 1.

Proof. code is a deterministic function of pk_t evaluated by the same code path in the client and in the gateway's scanner. $\mathcal{Z}$ received pk_t at step 2, so it computes code; the chain index is public and `ValidPayment` requires exactly one `CVPN1:` memo per transaction, so the lookup returns a unique confirmed transaction; that transaction's transparent inputs are public and identify the payer's wallet cluster under the ordinary common-input-ownership and change-detection heuristics. $\mathcal{Z}$ therefore recovers b with certainty. □

The one-way hash protects exactly the parties who never see its preimage. In a service whose verifier must see the preimage in order to serve, it protects nobody who matters. What the gateway holds per enrolled user is, concretely: pk_t; the client IP; last-handshake times, persisted to disk at `/data/peers.cache` ([cumulusvpn/gateway/internal/wg/peercache.go:47–51]); destination IPs and traffic timing; the code; and the paying address. Multi-hop does not help in v1, because the same key is used on both hops, so the entry node computes the same code.

Property 21.21 (Generalisation: it is the co-location, not the hash)**.** Every naive construction publishes one object with two sides: an *identifying input side* (transparent spends, which name a wallet cluster) and an *identifying obligation side* (an address or memo that names the workload or the user). The link between them is the transaction itself, and it is public because the obligation must be checkable by parties holding no secret (V0). The leak is therefore not the hash function, not the address format and not the memo encoding; it is the co-location of the two sides in one public object. Exactly one of the two sides must be made non-identifying, and the obligation side cannot be omitted.

Flux's own deployment path is the same object with different labels: transparent inputs, an output to A_{sink} = `t3NryfAQLGeFs9jEoeqsxmBN2QLRaRKFLUX` [measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03], an `OP_RETURN` carrying H_a, and a registration message signed by the owner key. Payer cluster, owner identity and workload are all public and all linked.

Property 21.22 (Payment-key indirection alone does not remove the leak)**.** CumulusVPN's planned v1.5 separates a payment key pk_p from the tunnel key pk_t (PLANNED, no code found, [doc: cumulusvpn/docs/04-payments.md:81–83]). This defeats an adversary that sees only pk_t — a multi-hop entry node, a passive network observer — because it cannot compute $\mathsf{H}(pk_p)$. But the gateway must still decide whether pk_t is entitled, and the only way for the client to demonstrate that is to present pk_p or a signature under it, which returns the preimage to the gateway. Indirection therefore moves the linkage from enrolment to the entitlement check; it does not remove it. It removes it only when the entitlement check is *itself* non-identifying, which requires either Construction 21.25 or an issuer-signed voucher.

The paper states this so that "v1.5 closes the gap" is not printed anywhere. The two remedies Property 21.21 permits are developed in §21.7.2 (blind the input side) and §21.8 (blind the obligation side).

21.7 The deployed rule, and construction A

21.7.1 What is deployed

Per node, FluxOS holds the chain $C_{\le h}$, a message store with temporary messages (1 h TTL) and permanent messages, and, per application name, a stored specification and e_a. On observing a funding event at height h', node u runs `checkAndRequestApp`: it fetches m_a from local temporary storage or from peers (up to three attempts), re-verifies the owner signature, recomputes $\mathcal{P}(m_a, h')$ from the height-scheduled price table, and sets

$$e_a \leftarrow \begin{cases} h' + d(m_a), & v \ge \mathcal{P}(m_a, h'), \\ \text{unchanged, warning logged}, & \text{otherwise}, \end{cases} \qquad \mathsf{Verify}_u(a, h, C_{\le h}) = \mathbf{1}[h \le e_a]\,.$$

The second branch is the silent-underpayment defect of §16: an underpaying transaction is consumed, not refunded, and no signed rejection reaches the payer (SHIPPED, [flux/ZelBack/src/services/appMessaging/messageVerifier.js:619–897]). A shielded payer has no return address, so the negative acknowledgement §16 requires must be published on the FluxOS message layer keyed by H_a for the payer to poll — an interface requirement (I4 below), not a new mechanism.

From a transparent funding event a node learns everything: the inputs and hence the payer's address cluster; v; H_a; m_a in full; and, if it is the API node through which the registration was submitted, the client's IP.

21.7.2 Construction A

Construction 21.23 (Shielded direct payment over the deployed toolkit — PLANNED)**.** A funding transaction for a would be a Sapling v4 transaction (`SAPLING_TX_VERSION` = 4, `SAPLING_VERSION_GROUP_ID` = `0x892F2085`) with:

- $\mathsf{vin} = \varnothing$;
- one or more `SpendDescription`s spending the payer's notes;
- one `OutputDescription` returning change to a payer-controlled z-addr;
- transparent outputs (A_{sink}, v) and $(\mathsf{OP_RETURN}\ H_a, 0)$;
- $v_{\text{bal}} = v + \mathsf{fee}$, and a `bindingSig` over the value commitments.

Under PNR ($h \geq h_{\text{PA}}$) the same transaction would be unchanged in shape: its single sink output (A_{sink}, v) is divided by consensus under PS4 of §7, and H_a stays in the `OP_RETURN` (PS2). Computed serialized size: 1,500 B for one spend and one change output, against approximately 286 B for the transparent equivalent.

Statement proved, public inputs, witness. Nothing beyond Sapling's own two statements [31], which `fluxd` already verifies for every shielded transaction (SHIPPED, [fluxd/src/primitives/transaction.h:155–242], [fluxd/depends/packages/librustzcash.mk:2–8]).

Spend statement. Public inputs $(\mathsf{rt}, \mathsf{cv}, \mathsf{nf}, \mathsf{rk})$; witness the Merkle path and position in NCT (the Sapling note commitment tree, depth 32) together with $(g_d, pk_d, v_{\mathsf{note}}, \mathsf{rcv}, \mathsf{cm}, \mathsf{rcm}, \alpha, \mathsf{ak}, \mathsf{nsk})$. It proves: the note commitment opens to $(g_d, pk_d, v_{\mathsf{note}}, \mathsf{rcm})$; cm lies in NCT under root rt; cv commits to v_{note}; nf is the nullifier of $(\mathsf{cm}, \text{pos})$ under nk; rk re-randomises the spend authority ak; and the address is diversified from ivk. Groth16 proof [29], 192 B.

Output statement. Public inputs $(\mathsf{cv}, \mathsf{cm}, \mathsf{epk})$; witness $(g_d, pk_d, v_{\mathsf{note}}, \mathsf{rcv}, \mathsf{rcm}, \mathsf{esk})$. It proves commitment and value-commitment integrity and $\mathsf{epk} = [\mathsf{esk}]\, g_d$. Groth16 proof, 192 B.

Binding signature. Under $\sum \mathsf{cv}_{\text{spend}} - \sum \mathsf{cv}_{\text{out}} - \mathsf{ValueCommit}(v_{\text{bal}}, 0)$, it proves $\sum v_{\text{spent}} - \sum v_{\text{out}} = v_{\text{bal}}$ exactly, with v_{bal} public.

Roles. *Prover:* the payer, or its agent. *Verifiers:* every `fluxd` node at consensus, checking the two proofs, the binding signature and nullifier non-membership; and every FluxOS node at the application layer, running Verify_u unchanged.

What a verifier learns. H_a and everything m_a contains, as before; v and v_{bal}; one nullifier per spent note, unlinkable to any commitment without nk; the anchor, which is a recent tree root and therefore a coarse "this wallet had synced past height h" signal; and the change note's ciphertext. **What it does not learn:** which note was spent, that note's value, the change value, or any key.

Input: spending key xsk_a (ZIP-32 child); registration or renewal message m_a; current height h; price table; sink address A_{sink}

9 . **Output:** a transaction tx that, once confirmed, sets $e_a \geq h + d(m_a)$

10 . **Invariant.** tx has $\mathsf{vin} = \varnothing$ at all times, so Assumption 21.17 never applies to it, and $\sum v_{\text{spent}} - \sum v_{\text{out}} = v_{\text{bal}} = v + \mathsf{fee}$ holds by the binding signature.$H_a \leftarrow \mathsf{SHA256}(m_a)$ $\mathcal{P} \leftarrow$ price of m_a at h under the height-scheduled table (§16) $v \leftarrow \mathcal{P}$, or $\lceil \mathcal{P}/g \rceil\, g$ if a denomination grid g is in force (Q6) select notes under xsk_a with total value $\geq v + \mathsf{fee}$ build `SpendDescription`s; build one `OutputDescription` for change to a z-addr derived from xsk_a `// or from the next period's child (Q13)`

11 append transparent outputs (A_{sink}, v) and $(\mathsf{OP_RETURN}\ H_a, 0)$ set $v_{\text{bal}} \leftarrow v + \mathsf{fee}$; sign `bindingSig` broadcast tx; broadcast m_a to a FluxOS API node **Failure modes.** (a) Insufficient shielded value: abort before broadcast; there is no partial-payment state. (b) $v < \mathcal{P}$ because the price table changed between steps 2 and 8: the payment is consumed and e_a is not advanced (the silent-underpayment defect); recovery depends on interface I4. (c) m_a does not reach a node before the funding event is scanned: the node retries the fetch up to three times, then drops the event. (d) A wallet that cannot emit an `OP_RETURN` output alongside a Sapling spend cannot execute step 6 at all: interface I2.

Algorithm 4: $\mathsf{Fund}_A(\mathcal{U}, a)$ — payer side. PLANNED.

Input: local chain prefix $C_{\leq h}$; local message store; the price table

12 . **Output:** $e_a \in \mathbb{N} \cup \{\perp\}$, identical across all honest nodes with the same prefix

13 . **Invariant.** No secret is read (V0); no message from another verifier is awaited (L).**for** *each confirmed* tx *at height* $h' \leq h$ *with an output to* A_{sink} **do**

14 $H \leftarrow \mathsf{OP_RETURN}(\text{tx})$; **if** H is absent **then continue** fetch m with $\mathsf{SHA256}(m) = H$ from local or peer message store verify the owner signature on m; **if** invalid **then continue if** $\text{value}(\text{tx}, A_{\text{sink}}) \geq \mathcal{P}(m, h')$ **then** $e_a \leftarrow h' + d(m)$ **else** log a warning

15 **return** e_a **Failure modes.** (a) *Resolved, and favourably.* The concern was that a shielded funding event would be invisible to an address-index scanner because it has an empty `vin`. It is not: `ConnectBlock`'s `vout` indexing loop runs unconditioned by `tx.vin` or coinbase status, so a z→t transaction's transparent output is written to the address index regardless, and `getaddressdeltas`/`getaddresstxids` read it back without re-filtering [fluxd/src/main.cpp:4072-4090], [fluxd/src/rpc/misc.cpp:812,1010]. A node with `-addressindex` scanning the sink address sees the payment, so Construction 21.23 is not gated here — it is gated only on reopening t→z. (b) Message-store miss on all three fetch attempts: the event is dropped and the payer must re-broadcast m_a. (c) A reorg past h': e_a is recomputed on the new prefix, which is exactly the consistency property C.

Algorithm 5: $\mathsf{Verify}_u(a, h, C_{\leq h})$ — node side. SHIPPED for transparent funding events; the shielded case is covered by the same scan (interface I1, M5 resolved).

Preconditions and interface requirements. None of these is a new cryptographic primitive.

P1. Assumption 21.17 would have to be lifted. It will not be: the pools are to be retired instead (§20.9, Definition 21.19), so Construction 21.23 remains usable only by holders of pre-2021 shielded value (Corollary 21.18) and only until the sunset height. **This precondition is unsatisfiable in the target architecture**, which is why Definition 21.1 classes the constructions here as inapplicable rather than pending.

P2. ZIP-209 turnstile enforcement, recommended on with Flux-computed checkpoints (Q2).

I1. `processInsight` must index transactions with empty `vin` and a Sapling spend paying A_{sink} exactly as it indexes transparent ones. By construction it keys on `vout`; the daemon-proxy address index does return such transactions, verified in the indexing code [fluxd/src/main.cpp:4072-4090] (M5, resolved).

I2. No wallet in the corpus constructs a Sapling spend carrying an `OP_RETURN` output: SSP has no shielded code path, and `z_sendmany` attaches memos only to z-addr recipients (SHIPPED, [fluxd/src/wallet/rpcwallet.cpp:3939]). Consensus would accept the transaction; the `Transaction Builder` needs an `OP_RETURN` output path. This is small, and it is the difference between "possible" and "usable".

I3. Under §7's settlement nothing further is needed: PS4 divides the value of the sink output, which is transparent in Construction 21.23, and O7 is moot (Definition 7.4).

I4. The negative acknowledgement of §16 is published on the FluxOS message layer keyed by H_a. Payment disclosure (`z_getpaymentdisclosure`, IN-PROGRESS, gated behind `-experimentalfeatures` plus `-paymentdisclosure`, [fluxd/src/wallet/rpcdisclosure.cpp:42,149]) is the right tool for the *dispute* that follows, letting a payer prove to a chosen party what it paid. It is a disclosure tool, not a privacy tool, and §20's characterisation should be read that way.

21.7.3 Construction C: a shielded balance with delegated renewal

Construction 21.24 (Shielded credit balance — PLANNED)**.** The principal $\mathcal{U}$ would derive a ZIP-32 child xsk_a, or $\mathsf{xsk}_{a,i}$ per period, and fund notes to its address — z-addr → z-addr from existing shielded value today, t-addr → z-addr only once P1 is lifted. The "balance" is the value of those notes and exists nowhere as protocol state. The renewer $\mathcal{A}$ holds xsk_a and, at $e_a - \Delta_{\text{pre}}$, produces a Construction 21.23 transaction. The delegated budget $\mathcal{B}$ is exactly the notes' value, with scope $\Sigma = \{\text{renew } a \text{ at no more than } \mathcal{P}\}$ enforced by which notes the child key controls rather than by a protocol rule. Revocation: $\mathcal{U}$ re-derives the child and sweeps its notes, racing $\mathcal{A}$ in the same way §17 describes. Monitoring without spending: $\mathcal{U}$ retains the child's full viewing key.

What holds: V0, S, C, L, D, PI against the network, RU (each renewal is an unlinkable spend; the balance never appears on chain), and FS with per-period children. What does not hold is PI against $\mathcal{A}$, which holds the key and therefore the link. When $\mathcal{A}$ is the tenant's own agent this is not a loss and Corollary 21.14 applies. When $\mathcal{A}$ is a third-party renewer acting for human users, it is a privacy trust point exactly as a custodial balance is, though with bounded exposure of funds. Whether a renewer can act *without* learning the link is open problem R2 (§21.13); this paper does not claim an answer.

Decided. Conditional — inapplicable unless a shielded pool exists, and both pools are being retired (Definition 21.1). Q12. Renewer model for the credit path. Domain: {own agent; third-party renewer holding per-period child keys; custodial balance}. Trade-off — own agent: PI complete, but requires the tenant to run an always-on agent; child keys: PI against the network only, with funds bounded by the notes the child controls; custodial: neither PI nor bounded funds. Mirrors §16's open decision on the credit path.

21.8 Shape B: what a real construction would need

Shape B is the one place in this section where a construction outside the deployed circuits is genuinely required, and it is stated as a VISION-maturity proposal with every parameter open. It is given because the alternative on the table — blind vouchers — moves soundness off the chain, and that trade deserves to be named rather than absorbed.

Construction 21.25 (Issuer-free chain-derived entitlement — proposed, VISION)**.** *Payment.* `OP_RETURN` would carry `CVPN2:` $\|$ cm with $\mathsf{cm} = \mathsf{H}(s\|r)$ for $s, r \in \{0,1\}^{256}$ generated on-device; the amount is unchanged. Gateways would compute `paid_until`[cm] exactly as they compute `paid_until`[code] today, keyed by cm instead of by a hash of the tunnel key. Composed with Construction 21.23, the payer is hidden on the chain as well.

Entitlement tree. Each gateway would maintain $\mathsf{NCT}_{\text{ent}}$, a sparse Merkle tree of depth d_{ent} keyed by cm with leaf $\ell = \mathsf{H}(\mathsf{cm} \,\|\, \texttt{paid_until}[\mathsf{cm}])$, updated in chain order. Its root rt_h is a deterministic function of $C_{\le h}$ and the gateway's (address, price) configuration, hence identical on every gateway with the same chain — V0 and C are preserved, and L is preserved with a root-history window. The root would be published at `/v1/info`.

Enrolment. The client would send $(pk_t, j, \mathsf{nf}, \mathsf{rt}, \pi)$ with epoch $j = \lfloor t/T_{\text{ep}} \rfloor$ and $\mathsf{nf} = \mathsf{PRF}_s(j)$, where π is a proof for the relation

$$\mathcal{R}_{\text{ent}} = \Big\{ ((\mathsf{rt}, j, \mathsf{nf}, \eta);\ (s, r, w, \text{path})) \ :\ \mathsf{Leaf}(\mathsf{H}(s\|r), w) \in \mathsf{NCT}_{\text{ent}} \text{ under } \mathsf{rt} \\ \wedge\ w \ge j\,T_{\text{ep}} \ \wedge\ \mathsf{nf} = \mathsf{PRF}_s(j) \Big\}$$

with public inputs $(\mathsf{rt}, j, \mathsf{nf}, \eta)$ where $\eta = \mathsf{H}(pk_t)$, witness $s, r \in \{0,1\}^{256}$, $w \in \mathbb{N}$ (the paid-through time), and path $\in (\{0,1\}^{256})^{d_{\text{ent}}}$. The public input η is bound into the proof to prevent proof theft — the Semaphore "signal" pattern.

Gateway. Accept if rt is among the gateway's last N_{rt} published roots and the proof verifies; set `tier`$[pk_t] \leftarrow$ premium; key the rate limiter by nf, so that all tunnel keys presenting the same nf within epoch j share one limiter — *the same sharing bound the per-**`pubkey`** limiter enforces today* (SHIPPED, [cumulusvpn/gateway/internal/limiter/limiter.go:105–123]). D therefore needs no global spent set: entitlement is time-based rather than consumable, and sharing across gateways is already permitted in the shipped model.

Epoch rotation. A fresh pk_t and a fresh nf per epoch make sessions unlinkable across epochs and linkable within one.

What the gateway learns. $(pk_t, \mathsf{nf}, j, \text{IP}, \text{traffic})$ and the fact that some active subscriber is present. Not cm, not the payment, not the payer; linking nf to cm requires s. The anonymity set is the set of leaves with $w \ge j\,T_{\text{ep}}$ — every active subscriber — refined downward by first-use timing correlation between a new leaf appearing on chain and a new enrolment arriving (§21.12), whose magnitude is CumulusVPN's payment rate and is unmeasured (M3).

What it requires that Flux does not have. Four things, and the interfaces are stated so the construction can be evaluated even though it is not built.

1. **The circuit.** Approximately d_{ent} hash evaluations for the Merkle path, two for the leaf, one PRF evaluation and one range check — of order 10^4 constraints with a SNARK-friendly hash, which is small next to Sapling's spend circuit.

2. **A proving key.** Under Groth16 this means a circuit-specific ceremony [11, 29], which would be the first ceremony this project has ever run: §20 documents that `fluxd` re-hosts the 2018 Zcash Sapling parameters verbatim and performs no ceremony of its own (SHIPPED, [fluxd/zcutil/fetch-params.sh:220–222], [fluxd/src/init.cpp:754–806]).

3. **Or a move to a ceremony-free system.** Halo-class recursive arguments remove the trusted setup [12]. **Two properties must be kept strictly apart here:** no-trusted-setup is not post-quantum. Halo2 rests on discrete-log hardness on a pairing-friendly or Pasta-style curve and is no more post-quantum than Groth16 over BLS12-381. Removing the inherited ceremony serves the decentralization requirement of §20; it does not serve

the post-quantum requirement of §8, which by settled scope excludes the shielded layer entirely.

4. **Client and gateway code.** A WASM prover in the TypeScript and React Native clients, and a BLS12-381 verifier in the Go gateway.

Not established by this survey. Proving time for $\mathcal{R}_{\text{ent}}$ in a browser or mobile client is asserted nowhere measurable; the "sub-second" figure in the mechanism design is an order-of-magnitude inference from circuit size, not a measurement. Either measure it on target client hardware or drop the claim.

Decided. Conditional — inapplicable unless a shielded pool exists, and both pools are being retired (Definition 21.1). Q8. Proving system for any new Flux circuit. Domain: {Groth16 with a Flux-run ceremony; a Halo2/PLONK-class system with universal or no setup, undated; defer shape B}. Trade-off: every new Groth16 statement needs its own ceremony and this project has never run one; a ceremony-free system removes that political cost and adds a proving-system migration that is scheduled nowhere.

Decided. Conditional — inapplicable unless a shielded pool exists, and both pools are being retired (Definition 21.1). Q10 and Q11, construction B parameters. Epoch length $T_{\text{ep}} \in [1\,\text{h}, 30\,\text{d}]$: shorter gives less intra-epoch linkability at the cost of more re-enrolment and proving; longer lets the nullifier nf decay into a stable pseudonym. Root-history window $N_{\mathsf{rt}} \in [10, 10^4]$ roots: larger tolerates staler clients at the cost of gateway state. Tree depth d_{ent} is fixed by the target subscriber population, which is unmeasured (M3).

Remark 21.26 (The voucher alternative, and what it costs). CumulusVPN's stated v2 is blind-signature vouchers with quorum signing (VISION, [doc: cumulusvpn/docs/04-payments.md:84–87]). Blind signatures give unlinkability by construction and, for the Chaum–RSA variant, information-theoretically. But they require an *issuer* holding a secret key, and nothing on the chain binds tokens issued to payments received: **soundness moves from the chain to the issuer.** Definition 21.6's V0 forbids a secret in *verification* only, so an issuer is admissible — but it becomes the party whose honesty S depends on. Threshold or quorum issuance bounds the availability risk and does not restore chain-anchored soundness. That is the real trade, and both options should be presented with it: Construction 21.25 keeps soundness on the chain and pays for it with a circuit and a ceremony; vouchers avoid the circuit and pay for it with an issuer.

21.9 The anonymity set, measured

No construction manufactures an anonymity set. This subsection measures the one that exists, and the numbers are the section's most load-bearing empirical content.

21.9.1 The population is not the payer's anonymity set

For shape A the payment names H_a, so the *workload-side* anonymity set is 1 by construction and the 1,195 deployed applications are not the anonymity set of anything. The question that matters is whether the (amount, time) pair leaks more than the name already does. For public-content applications it does not, because $\mathcal{P}$ is a deterministic function of the public specification. Table 51 gives the partition structure.

Table 51: Price-class structure over the 1,195 deployed applications [measured: api.runonflux.io/apps/globalappsspecifications, 2026-09-03]. The tuple partition (cpu, ram, hdd, k, d, enterprise, staticip, nodes) is a lower bound on class size under *any* deterministic price function; the period-price partitions are upper bounds up to rare collisions. The bold row is the price the paper states (Definition 16.3); the four price-and-time rows below it are computed on the base-only price, and on the bold row's price they give 100 %, 82.3 %, 59.0 % and 38.4 % singletons with the same median and largest classes.

Partition	Classes	Singletons	Median class	Largest
resource tuple (lower bound)	778	698 (58.4 %)	1	57
period price, base only (lower bound)	349	201 (16.8 %)	7	63
period price, $k/3$ variant (upper bound)	381	229 (19.2 %)	6	62
period price, Definition 16.3 as corrected	**370**	**222** (18.6 %)	**8**	**61**
monthly price only, ignoring d	170	86 (7.2 %)	66	210
price *and* same block	1,195	1,195 (100 %)	1	1
price and same hour	—	976 (81.7 %)	1	11
price and same day	—	669 (56.0 %)	1	18
price and same week	—	439 (36.7 %)	2	35

If every application renews on schedule, renewal volume is 0.0123 per block, 1.48 per hour, 35.4 **per day** and 248 per week; the next 30 days carry a median of 24 and a maximum of 98 expiries per day. That figure is an upper bound, because churn is unmeasured (M2). The median period price is 1.19 FLUX, the range 0.03 to 417.24 FLUX (Definition 16.3 as corrected); 459 applications (38.4 %) sit at the default 88,000-block period and 107 (9.0 %) at the 1,056,000-block maximum.

The reading is this. **At** 35.4 **renewals per day a renewal is a singleton within its block** 100 % **of the time and within its day** 59.0 % **of the time.** A payment "of a distinctive size just before a distinctive deployment" is not an attack scenario, it is the normal case. For shape A that identifies the application, which the `OP_RETURN` already does, and says nothing about the payer. For any coupon-style or Construction 21.25 scheme paying exact amounts, it identifies the entitlement and defeats the scheme — which is why such schemes require *denominated* amounts, at a measurable cost.

Table 52: Denomination trade-off, computed on Flux's own application population from the base-only period price (Table 51, row 2). Under Definition 16.3 as corrected the exact row is the bold row of Table 51, and grids of 0.5, 1, 2 and 5 FLUX give median classes of 132, 262, 824 and 996 at 2.7 %, 5.5 %, 11.9 % and 36.6 % overpayment. Overpayment is expressed as a share of list revenue; under PNR it would flow to the Foundation and the fee pool by the standard split, i.e. a privacy fee paid to the network.

Grid g	Classes	Singletons	Median class	Largest	Overpayment
none (exact)	349	201 (16.8 %)	7	63	0 %
0.5 FLUX	43	15 (1.3 %)	118	291	9.0 %
1 FLUX	30	13 (1.1 %)	227	559	17.3 %
2 FLUX	21	7 (0.6 %)	786	786	39.0 %
powers of two	13	0	227	268	51.4 %
5 FLUX	12	4 (0.3 %)	1,037	1,037	119.7 %

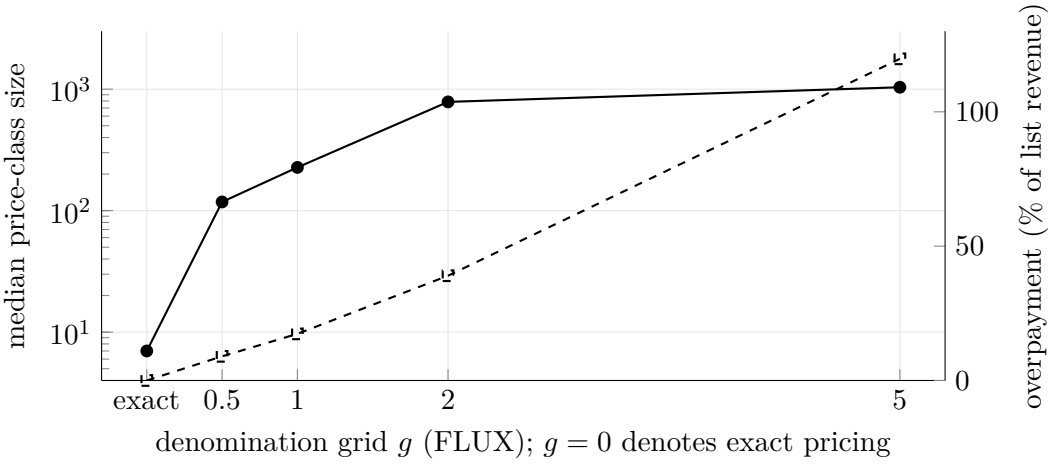


Figure 17: The denomination trade-off measured on the 1,195-application population. Solid, left axis, logarithmic: median price-class size. Dashed, right axis, linear: overpayment as a share of list revenue. On the base-only price of Table 52, a 1 FLUX grid buys a median class of 227 at 17.3 % overpayment; a 0.5 FLUX grid buys 118 at 9.0 %. This is the classical fixed-denomination trade-off, evaluated on Flux's own demand rather than assumed.

Decided. Conditional — inapplicable unless a shielded pool exists, and both pools are being retired (Definition 21.1). Q6. Denomination grid $g \in \{\text{none}, 0.5, 1, 2, 5\}$ FLUX or powers of two, plus the destination of the overpayment (PNR split, or credited forward to the next period). Trade-off is Table 52: under Definition 16.3 as corrected, median class $8 \to 132 \to 262 \to 824$ against overpayment $0 \to 2.7\,\% \to 5.5\,\% \to 11.9\,\%$ (on the base-only price the table prints, $7 \to 118 \to 227 \to 786$ against $0 \to 9.0\,\% \to 17.3\,\% \to 39.0\,\%$). Note that denomination buys nothing for shape A, where the workload is named anyway; it is a shape-B and coupon-scheme parameter.

21.9.2 The payer's anonymity set is a closed pool of unknown membership

This is the number the section must print. The payer-side anonymity set for Construction 21.23 is not the application population; it is the holder set of the Sapling pool: 77,188.75921324 FLUX of value (plus 19,291.17735041 FLUX in Sprout), closed since height 835,554, with *note count and holder count unmeasured.* The `commitments` field of `getblockchaininfo` (815,736 at tip) is, in the zcashd-2.x lineage, the *Sprout* tree size and does not answer the question.

Open Problem 21.27 (M1: the effective size of a closed pool). Determine the number of unspent Sapling notes at the tip — tree size minus nullifier-set size — and any defensible holder-count estimate that chain analysis permits. Until this is measured, the paper claims the *mechanism* of Construction 21.23 and does not claim realised privacy for it. Interpreting the measurement (which notes are plausibly spendable, how holders cluster) is chain-analysis research with no standard answer, and is listed separately as R3.

21.10 Cost, and the binding limit on shielded throughput

Table 53: Serialized sizes, derived from the deployed constants (SHIPPED, [fluxd/src/flux/Zelcash.h:8–24], [fluxd/src/flux/JoinSplit.hpp:22–26], [fluxd/src/primitives/transaction.h:155–242]).

Object	Bytes	Derivation
Sapling note plaintext / `encCiphertext`	564 / 580	$1 + 11 + 8 + 32 + 512$, then $+16$
`outCiphertext`	80	$32 + 32 + 16$
`SpendDescription`	384	$4 \times 32 + 192 + 64$
`OutputDescription`	948	$3 \times 32 + 580 + 80 + 192$
Construction 21.23 renewal	**1,500**	header, counts, two vouts, one spend, one output, `bindingSig`
transparent renewal (1 P2PKH input)	≈ 286	—
Construction 21.25 proof (off-chain)	192	Groth16

A shielded renewal is 1,500 B against approximately 286 B transparent, a factor of 5.24. With `MAX_BLOCK_SIZE` = 2,000,000 B, block capacity is $\lfloor 2{,}000{,}000/1{,}500 \rfloor = 1{,}333$ such transactions per block, 44.4 per second, implying 2,666 Groth16 verifications per block.

Property 21.28 (Block space is not the constraint)**.** At measured demand of 0.0123 renewals per block, Construction 21.23 would consume 0.0009 % of `MAX_BLOCK_SIZE`. At 10, 100 and 1,000 times measured demand the figures are 0.01 %, 0.09 % and 0.92 %. Even if all 7,486 requested instances were re-registered as separate applications *daily*, the load would be 2.6 transactions per block.

Property 21.29 (Verification is the constraint)**.** Let t_{G16} be single-core Groth16 verification time and N_{proofs} the proof count in a block; verification cost is $T_{\text{verify}} = N_{\text{proofs}} \cdot t_{G16}$ on one core. A block filled with Construction 21.23 transactions carries $N_{\text{proofs}} = 2{,}666$; a pathological all-`SpendDescription` block carries $\lfloor 2{,}000{,}000/384 \rfloor = 5{,}208$. Against the 30 s PoN interval:

$$2{,}666 \cdot t_{G16} > 30\,\text{s} \iff t_{G16} > 11.3\,\text{ms}, \qquad 5{,}208 \cdot t_{G16} > 30\,\text{s} \iff t_{G16} > 5.8\,\text{ms}.$$

Concretely, at 2 ms, 5 ms and 10 ms the pathological block takes 10.4 s, 26.0 s and 52.1 s to verify. **`MAX_BLOCK_SIZE` was sized for transparent transactions; for shielded traffic the binding limit on throughput is single-core proof verification, not block space.** At ordinary demand the point is moot — at 1000× demand, 24.6 proofs per block, or 0.25 s at 10 ms — but the adversarially-full block would become reachable once shielded traffic was encouraged, which is what a reopening of t-addr → z-addr would have done. With reopening cancelled (§21.5), no such activation height arrives and the bound remains a ceiling on a pool that can only drain.

Not established by this survey. t_{G16} on Cumulus, Nimbus and Stratus hardware is a parameter here, not a measurement (M4). It must be measured with `zcbenchmark`, whose `verifysaplingspend` and `verifysaplingoutput` types the fork retains ([fluxd/src/wallet/rpcwallet.cpp:2972–2979]). Likewise, the claim that the 2018-era `librustzcash` pin `06da3b9a` predates batch verification is an inference from the pin date, not from reading the crate.

Decided. Conditional — inapplicable unless a shielded pool exists, and both pools are being retired (Definition 21.1). Q17. Response to Property 21.29. Domain: {no change; a consensus cap N_{proofs} per block; a `librustzcash` upgrade enabling batch verification}. Trade-off: a cap is a one-constant consensus change that bounds worst-case validation time and also caps legitimate shielded throughput; an upgrade removes the cap's need but moves the fork's pinned cryptography forward by several years of upstream changes, which is a much larger review surface. Only a reopening of t-addr → z-addr would have let shielded volume grow past today's zero, and reopening is cancelled (§21.5).

Two further cost notes. Verifying keys are already present on every node — `ZC_LoadParams` refuses to start without all three parameter files (SHIPPED, [fluxd/src/init.cpp:754–806]) — so 6,489 nodes verify Sapling proofs today at zero marginal deployment cost. And Sprout value, the 19,291.18 FLUX half of the pool, requires the 725 MB `sprout-groth16.params` and gigabyte-class proving memory; the paper treats it as legacy.

21.11 Invariants

Table 54 states which invariants hold under which construction. Each is stated so that a violation is observable.

Table 54: Invariants by construction. A: shielded direct payment; C_{own}: credit balance renewed by the payer's own agent; C_{3p}: credit balance renewed by a third party; B: issuer-free entitlement.

	Invariant	A	C_{own}	C_{3p}	B
I1	Conservation: $\sum v_{spent} - \sum v_{out} = v_{bal}$, pool value falls by exactly v_{bal}	yes	yes	yes	n/a
I2	Non-negativity of Φ, e_a and note values	yes	yes	yes	yes
I3	No double-funding from one payment (D)	yes	yes	yes	vacuous
I4	No forgery of funded status (S)	yes†	yes†	yes†	yes
I5	No linkage across renewals of one deployment (RU)	yes‡	yes‡	*no*	yes
I6	Forward secrecy of the payer across periods (FS)	partial	partial	*no*	yes
I7	Liveness under partition (L)	yes	yes	yes	yes§
I8	No double-claim across an instance migration	yes	yes	yes	n/a

I1, conservation. Construction 21.23 moves value through the existing Sapling turnstile arithmetic: the binding signature forces $\sum v_{spent} - \sum v_{out} = v_{bal}$, and the pool's `chainValue` falls by exactly v_{bal}. The coinbase is untouched, so §7's bounded-coinbase requirement is trivially preserved: this mechanism adds no coinbase output.

I2, non-negativity. A z-addr → t-addr transaction with v_{bal} exceeding the pool's value is rejected *only when turnstile enforcement is on*; today it is monitored and not enforced (Q2). The invariant is conditional on that decision and the paper states the dependency rather than the invariant alone.

I3, no double-funding. H_a is resolved once — `checkAndRequestApp` returns early when the permanent message already exists — so a second payment carrying an already-resolved hash is *ignored*, that is, lost. This joins the silent-underpayment defect as a failure mode that interface I4 must report. Under Construction 21.24 each renewal is a distinct m_a with a distinct hash. Under Construction 21.25 the nullifier keys the limiter and D is vacuous, because entitlement is time-based rather than consumable.

I4, no forgery of funded status. Requires a confirmed transfer bound to H_a; forging one requires forging a Sapling proof or the binding signature. †: under Groth16 with the *inherited 2018 parameters*, this is exactly the counterfeiting scenario §20 documents, and it is bounded to the pool's declared value only if turnstile enforcement is on.

I5, no linkage across renewals. Consecutive Construction 21.23 transactions share the name (via each period's H_a), the amount (same price) and the periodicity — all public by design — and nothing else: fresh nf, cv, rk and ciphertext. ‡ marks the residual: if period i's change note is spent in period $i+1$ and the period-$(i{+}1)$ child key is compromised, the adversary decrypts that note and learns it was created by period i's renewal — linking two renewals that H_a already links, and revealing how many periods the sub-wallet has funded. No payer identity is exposed. Forwarding change to the next period's child confines the residual to one step (Q13).

I6, forward secrecy. With per-period ZIP-32 children and parents discarded on the agent, compromise at period i yields keys for periods $\geq i$ only. "Partial" because the chain retains every ciphertext forever, so compromise of the principal's *master* seed reveals all periods — inherent to hierarchical-deterministic shielded wallets and shared with Zcash.

I7, liveness under partition. Verify_u depends only on the local prefix; a partitioned node evaluates on its own view and reconciles by the same disconnect logic reorgs use today. §: for Construction 21.25, a gateway with a stale root rejects proofs against newer roots and the client re-proves against the published root, one extra round trip, with N_{rt} bounding staleness tolerance.

I8, no double-claim across migration. Funded status is chain-derived and node-independent, so when FluxOS redeploys instances to other nodes there is no per-node claim to duplicate. Across the dead Sprout→Sapling migration path, Sprout value remains spendable by a joinsplit with `vpub_new` and empty `vin` — therefore not caught by Assumption 21.17 — at higher proving cost; nothing is claimed twice.

21.12 Attack analysis

Timing and amount correlation. Renewal heights are public and periodic, $h_0 + i\,d$. An adversary with a network vantage watches for a shielded transaction entering the mempool in the window before each renewal height of a and records the first-seen peer. Over i renewals the intersection of first-seen peer sets converges on the payer's broadcast point. The mitigation is behavioural, not cryptographic: prepay, which the deployed `maxBlocksAllowance` of 1,056,000 blocks (366.7 days) already permits and which reduces events from 11.95 to 1.00 per year; randomise the submission lead within $[\Delta_{\text{pre}}, d]$; and broadcast through a node the tenant does not otherwise use — or through the VPN the network itself sells. `fluxd` has no Dandelion-style relay and inherits Bitcoin-era transaction propagation, so this attack is not addressed at the P2P layer.

The intersection attack across renewals. On chain, Construction 21.23 gives the adversary nothing to intersect: each renewal exposes a fresh nullifier and no persistent identifier. The attack lives entirely at the network layer, as above. Against a payer broadcasting from a fixed vantage it converges in a handful of periods; against one who prepays a year it has a single sample. The interaction with §23 should be stated: a tenant who prepays a year privately — the cheapest mitigation — carries a larger forfeitable remainder if the application is later removed by the moderation quorum. At the median price per 88,000-block period that is about 9.7 FLUX for a year; at the maximum measured price per 88,000-block period, 93.60 FLUX, it is about 1,119 FLUX.

A node operator who is also the payer. On chain, no advantage: its own transactions look like anyone's. At the FluxOS layer it can submit its own registrations through its own node and broadcast its own transactions from it, eliminating the metadata trust point *for itself*; conversely, as an adversary it observes the metadata of every tenant that uses its nodes, and with 237 nodes under one identity that is 3.7 % of API entry points. Self-dealing gains nothing economically — §7's timing recapture is structural and independent of whether the fee's origin is visible — but privacy removes the *audit trail* that would let anyone notice a wash-deployment pattern. The defence against wash deployment is PNR's economics, not observability, and §7 should be read as carrying that weight.

What the transparent layer leaks even when the payment is shielded. Measured on the specification set itself, which no payment scheme touches [measured: api.runonflux.io/apps/globalappsspecifications, 2026-09-03]:

- **Owner pseudonyms.** 659 distinct `owner` identifiers; 588 own exactly one application; the largest owns 246 (20.6 %) and is the team ZelID (§21.5). Excluding it, 949 applications across 658 owners, of whom 15 hold five or more (223 applications in total). Owner-key reuse links applications across renewals and across deployments *regardless of payment privacy*. The agent-native default should be one owner key per application (Q15).
- **Contacts.** 688 applications (57.6 %) publish at least one non-blank contact string in the public specification. Any such string is a direct identity leak that dominates payment privacy for the application carrying it (Q14).
- **The registration message** carries the owner's signature and reaches the network through one API node, which sees the client's IP address and is identifiable to peers as the broadcast origin. This is a metadata trust point independent of the chain (Q7).
- **Amount and timing**, per §21.9; the **anchor**, a coarse wallet sync height; the **fee**; and **`valueBalance`**, which equals $v + \mathsf{fee}$ and is public anyway.

Griefing and denial of service. Invalid Sapling proofs cost the submitter nothing — they are rejected before any fee — and cost each verifying node one Groth16 verification; this is bounded by P2P DoS scoring as in Zcash. Block-filling is Property 21.29. For Construction 21.25, a gateway that publishes a root it will not accept denies service to anonymous clients only, leaves free-tier clients unaffected, and is detectable by clients but not by the network.

Byzantine and colluding node sets. A byzantine minority cannot forge funded status (I4: every node verifies independently) and cannot deny it (L); it can withhold or delay a tenant's registration message, as it can today, with the client retrying against another node; it can log metadata at the nodes it runs. A colluding majority can rewrite the chain, at which point the funded-status question is moot. Short of that, a majority of *API entry points* still cannot break PI cryptographically; it widens the metadata view above. A colluding majority of Construction 21.25 gateways learns nothing more than one gateway does, because gateways hold no secret — which is the point of making the construction issuer-free.

Sustained low demand. This is the operative failure mode and it is present now: 35.4 renewals per day, of which zero are shielded. Privacy degrades continuously with volume; there is no threshold below which it stops working, only a shrinking AS. The paper does not claim privacy at current volume. It claims the mechanism and prints the population figures.

Post-quantum scope. Every construction here rests on BLS12-381 pairings and Jubjub discrete logarithms, and none is post-quantum — by the settled scope decision of §8, which places the shielded layer outside post-quantum migration. The one interaction worth stating: a quantum adversary breaks *hiding retroactively*, since it can decrypt stored ciphertexts and recover keys, so PI for payments made today does not survive that adversary. This is Zcash's position as well, and §20 states it.

21.13 Open problems, and open parameters

The two categories are kept apart deliberately. A research problem has no known good answer, or no answer without a new trust assumption; an engineering item is a substantial build with no unsolved question. §25 must class them separately and must not collapse them.

Open Problem 21.30 (R1: unlinkable exact-amount payment at low volume). With 35.4 renewals per day and 370 price classes, the pair (amount, time) is identifying (Table 51). The only known remedies are denomination, which costs overpayment (Table 52), and batching or delay,

which costs latency; both are policies, not constructions. Whether a mechanism exists that is simultaneously exact-amount, unlinkable and PNR-compatible is open. Property 21.11 shows that amount hiding is not that mechanism. **This is a property of the population, not of any construction** — which is why no circuit fixes it.

Open Problem 21.31 (R2: third-party renewal without the renewer learning the link). Construction 21.24 hands the renewer the spending key and therefore the link. A renewer that spends value bound to a without learning which principal funded it is the delegatable-anonymous-credential / anonymous-e-cash-with-delegation setting. No issuer-free instance for *this* setting — many independent verifiers, no shared secret, soundness anchored on the chain — is known to this author, and this paper does not invent one. For autonomous agents the problem is moot (Corollary 21.14); it binds only for human users delegating to a third-party renewal service.

Open Problem 21.32 (R3: the effective anonymity set of a closed pool). M1 (Open Problem 21.27) is a measurement. *Interpreting* it — which notes are plausibly still spendable, how holders cluster, what a realistic adversary can exclude — is chain-analysis research with no standard answer, and it is made harder, not easier, by the pool being closed: a pool that can only lose value admits elimination arguments that a growing pool does not.

Everything else is engineering: (E1) the P1 network upgrade and P2 turnstile activation with Flux-computed checkpoints; (E2) interfaces I1 and I3, the FluxOS scanner and PNR validation for empty-`vin` payments; (E3) interface I2, a `TransactionBuilder` `OP_RETURN` path and a wallet or agent SDK that uses it; (E4) interface I4, the negative acknowledgement on the message layer with payment disclosure as the dispute tool; (E5) Construction 21.24 as a ZIP-32 child-key delegation object composed with §17's delegation primitive; (E6) Construction 21.25 — circuit, ceremony or Halo2, client prover, gateway verifier; and (E7) the denominated-pricing option and the response to Property 21.29. **The feasibility assessment of §25 should therefore read: shape A is Engineering; shape C with the payer's own agent is Engineering; shape B is Engineering plus a ceremony; and the Research content is R1–R3, of which R1 is a property of the population rather than of any construction.**

Table 55: Decisions, open unless marked. None is guessed anywhere in this section; each is mirrored in the decisions register. "Affects" names the construction.

Id	Affects	Domain	Trade-off
Q1	A, C	withdrawn: no reopening (Definition 21.19)	pool unreachable until activation versus turnstile-bound soundness at reopening
Q2	A, C	answered: on, for the drain window (Definition 21.19)	bounded soundness failure versus none
Q3	A	settled: $H(m_a)$ in `OP_RETURN` [intent: design intake]	binding $H(m_a)$ is what nodes must act on; blinding belongs to shape B
Q4	A	moot by retirement: transparent only (§7, Definition 7.4)	exclusion makes this section vacuous; hidden amount is impossible (Prop. 21.11)
Q5	A	amount hiding for non-PNR classes: none / an exempt class	an exempt class restores AH for enterprise workloads at the cost of revenue outside §7's split
Q6	A, B	denomination grid, Table 52	median class versus overpayment
Q7	A, C	submission path: any node / own node / VPN / no-logging rule	metadata trust point; a logging rule is policy and unenforceable
Q8	B	Groth16 with a ceremony / Halo2-class / defer	first-ever ceremony versus an undated proving-system move
Q9	B	Construction 21.25 / single issuer / threshold issuer	soundness on chain versus soundness at an issuer
Q10	B	$T_{\text{ep}} \in [1\,\text{h}, 30\,\text{d}]$	intra-epoch linkability versus re-enrolment cost
Q11	B	$N_{\text{rt}} \in [10, 10^4]$	stale-client liveness versus gateway state
Q12	C	own agent / third-party renewer / custodial	PI complete / PI versus the network only / neither
Q13	C	change forwarding to per-period children: yes / no	free on chain; confines I5's residual to one step
Q14	all	contacts: keep / encrypt / strip from the hashed object	688 applications leak an identifier today and no payment scheme fixes it
Q15	A	one owner key per application: policy / wallet UX	cross-application RU; 15 owners hold 223 applications
Q16	A, C	which wallet implements z-addr → t-addr with `OP_RETURN`	determines who can actually use Construction 21.23
Q17	A	proof-count cap / `librustzcash` upgrade / none	Property 21.29

Remark 21.33 (Every question in this section is conditional on the pool surviving). Of Table 55's seventeen rows, Q1 is withdrawn and Q2 answered (Definition 21.19), Q3 is settled (§7, O3) and Q4 is mooted by retirement; the rest remain open decisions in form, and Q6, Q8, Q12 and Q17 are called out separately above because they gate constructibility, cost or a security property. No value in this section was chosen by this drafter, and that remains deliberate — but the reason has changed, and the change is worth stating rather than leaving implicit.

Section 20.9 records the option the team has since taken: retiring both shielded pools outright, which the measurements there favour [intent: design intake]. **With that option taken, this entire section is moot** — not deferred, but inapplicable, because there is no pool to pay into, no proving system to choose, no denomination grid to design and no turnstile question beyond the drain window. Q1's "reopening is decided, only the height is open" framing was recorded before that option was on the table and is now withdrawn.

The paper therefore does not fill these parameters, and would not be improved by filling them: every value would be conditional on a pool that will not exist. What this section provides regardless is the requirement set — what a private payment for a public obligation must satisfy — and that is independent of whether Flux carries a shielded pool. The construction is what depends on the answer.

Four measurements are also outstanding and are not decisions: **M1**, the Sapling pool in notes and holders (Open Problem 21.27); **M2**, renewal churn, which converts §21.9's upper-bound rate into a rate; **M3**, CumulusVPN's payment rate, which is the anonymity set for Construction 21.25; and **M4**, t_{G16} on tier hardware, which converts Property 21.29 from a parameter to a number. **M5**, whether the daemon-proxy address index returns empty-`vin` Sapling transactions paying

A_{sink}, is resolved, favourably: it does (§21.7, interface I1), so it gates Construction 21.23 only on reopening (P1) and no longer stands as an open test.

21.14 Comparison with prior art

Table 56: Prior art against the requirement pair of §21.1. The column that separates the field is *where soundness is anchored*: on a chain, or at an issuer.

System	What it hides	How the obligation is verified	Soundness	What Flux does differently
Zerocash / Zcash Sapling [7, 31]	sender, receiver and amount for z-addr → z-addr; sender for z-addr → t-addr	no notion of an obligation beyond the transfer; no recurring-payment primitive	chain	attaches a *public* obligation H_a to a z-addr → t-addr and lets 6,489 independent verifiers act on it; the shielded-subscription problem Zcash never solved is shape C, and the answer here is child-key delegation, not a protocol primitive
Groth16 and its ceremony [11, 29]	—	—	—	Flux inherits the 2018 Zcash Sapling parameters verbatim and has run no ceremony of its own; every *new* statement would need one (Q8)
Halo, no trusted setup [12]	—	—	—	removes the inherited ceremony, which directly serves §20's decentralization argument; it is **not** post-quantum and the two claims are kept separate
Fixed-denomination mixers	the link deposit ↔ withdrawal within a denomination	on-chain nullifier set plus Merkle membership	chain	the identical trade-off, measured here on Flux's own demand (Table 52); shape A needs no separate pool because Sapling *is* one
Semaphore-style membership signalling	which member signalled	Merkle membership plus a per-external-nullifier nullifier	chain or verifier-side	Construction 21.25 is this shape with the membership set derived from *payments on chain*, so the set needs no admission authority
Lightning [39]	the payer from third parties (onion routing); the payment from non-participants	the recipient alone; the preimage is a receipt	channel counterparties	Lightning has no *public* obligation: a receipt convinces one party. Flux needs 6,489 independent verifiers holding no secret, which is precisely what point-to-point payment privacy cannot provide
Chaumian blind signatures; Privacy-Pass-style tokens	the link issuance ↔ redemption	the issuer's public key, with a per-verifier or shared spent set	**issuer**	admissible under V0, which constrains *verification* only — but soundness leaves the chain (Remark 21.26); Construction 21.25 replaces the issuer with the chain and pays a circuit for it
Anonymous credentials (CL-style; algebraic-MAC keyed-verification)	multi-show unlinkability with provable attributes such as `paid_until`	the issuer's key; keyed-verification requires the *verifier* to hold the issuer secret	issuer	keyed-verification violates V0 outright for 6,489 independent verifiers — fine for a single-gateway model, wrong for this node model
Compact e-cash / rechargeable anonymous wallets	which coin, which recharge	bank-issued coins with double-spend detection	the bank	the closest prior art for shape C: the shielded notes are the coins and the chain is the bank, but delegation to a renewer (R2) is where the analogy stops

The prior art named in the last four rows of Table 56 is Chaum [17] for blind signatures, the Privacy Pass line of anonymous authorization tokens [40], Camenisch and Lysyanskaya [15] for anonymous credentials and the keyed-verification line that follows it, and Camenisch et al. [16] for compact e-cash.

The combination that is specific to Flux is worth stating in one sentence, because it is what makes this section a contribution rather than a survey: **a public obligation that must be checkable by thousands of independent verifiers holding no shared secret, paid out of a shielded pool that already exists but cannot be entered, at a demand level where the anonymity set rather than the cryptography is the binding constraint.** Two of the three difficulties are not cryptographic. The construction for the common case is short and uses circuits that have been running on this network since 2019; the honesty about the set, and about the consensus rule that closed the pool in 2021, is the part that took work.

22 Bounded, publicly verifiable bridging — replacing Fusion, chain consolidation, and why not atomic swaps

A bridge lets FLUX that lives on the Flux chain be used on Ethereum or BNB Smart Chain. Today that works because a single server holds single-signature private keys for each chain (up to three per chain, 32 in all) and sends tokens out of a hot wallet when it observes an inbound payment. The replacement specified in this section locks FLUX in a publicly visible multi-key reserve on the Flux L1 and lets one contract per foreign chain issue exactly that much FLUX there — never more, because the contract itself refuses. Anyone with a Flux node and a public RPC endpoint per chain would be able to check, at any moment, that the tokens in circulation on every chain add up to what is locked at home. The governing constraint is the team's own [intent: design intake]: decentralization is the requirement, "but done right" — *a decentralized design that is exploitable is worse than the custodial incumbent.* This section takes that literally. It does not attempt to remove every trusted party. It makes every failure mode bounded by a contract, observable by anyone, and attributable to named signers, and it states plainly which requirements cannot be met without a Flux L1 consensus change.

What is now decided, and the numbers that follow. Six quantities that earlier drafts of this section carried as open parameters are fixed [intent: design intake]: a *global* supply cap of 560,000,000 FLUX across all chains, enforced by the reserve invariant under honest-minority operation rather than by any contract (Definition 22.15); a mint rate limit of 10,000,000 FLUX per 24 h *per chain*, with a matching burn ceiling of 10,000,000 FLUX per chain per window; a 3-of-4 quorum for mint, release and administration alike, held by the four named co-founders; a guardian pause executable by *any one* of four, with a bounded pause window; no premine on any surviving chain, the contracts being mint-and-burn so that supply on a chain is only what has been minted against locked reserve; and limits that are fixed at deployment and not adjustable — against contracts that are nonetheless *upgradable* under the same 3-of-4. Three consequences follow, and this section states all three rather than leaving them to the reader.

First, the cap is a long-horizon backstop and the rate limit is the bound. 560,000,000 FLUX is where the emission curve arrives in the early 2050s (Section 5), against a present measured liability of 223,027,781.21 FLUX across all ten legacy assets [measured: scripts/13-issuer-balances.sh, 2026-09-03]; [measured: scripts/15-nonevm-liability.sh, scripts/16-remaining-liability.sh, 2026-09-04]; reaching the cap on one chain at the permitted rate takes 25 d. So the security claim rests on the rate limit, and because the limit is per chain the worst case scales with the number of live chains: the sustained figure is 20,000,000 FLUX at a 24 h reaction time over the launch set {ETH, BSC} and 30,000,000 FLUX once Solana is added (Definition 22.9). T_{react}, the time to detect a compromise and throw the pause, is therefore the single most important operational parameter in the bridge.

Second, one quorum for mint, release and administration collapses the separation the construction is built around. The design distinguishes those three roles precisely so that no single set can both mint and release; with 3-of-4 everywhere, a compromise of three keys obtains every one of them at once and the rate limit is the only control that still binds. What the 1-of-4 pause preserves is exactly the thing that makes T_{react} finite: the fourth key can stop issuance without the other three (Definition 22.10). That is a single trust assumption bounded by a rate limit and a pause, and Section 22.7 presents it as such rather than as a separation of powers.

Third, and this was the load-bearing open item in the design until Δ_A was decided (Definition 22.12): an upgradable contract can remove its own rate limit. A quorum that has been compromised is not obliged to respect a limit it can delete, and step one of a competent attack is to upgrade the contract rather than to mint under it. The bound of Definition 22.9 therefore holds only if the upgrade path is slower than the reaction time — formally, only under the timelock condition $\Delta_A > T_{\text{react}}$ of Definition 22.11. Without that condition this section can state the cap and cannot state a worst-case bound at all. PLANNED

Notation used in this section. `notation.tex` originally carried no bridge symbols. The

symbols below were local to Section 22 and were proposed as additions to the canonical macro file. Three deliberately diverge from the internal mechanism specification to avoid collisions with symbols that are already canonical: reserve slack is ς (because σ is the block subsidy σ), the mint rate limit is ϱ (because ρ is the PNR operator share ρ), and cumulative bridge fees are $F_\bullet$ (because Φ is the PNR fee pool Φ). Cumulative lock and release value are V_{lock} and V_{rel} (because L is the default application lifetime L). Existing macros are used wherever they apply: h, S, FLUX, sat, $\mathcal{Z}$, $\mathcal{C}_{\mathcal{Z}}$, :=, MAX_MONEY. These symbols have since been merged into `notation.tex` as `\bslack`, `\brate`, `\bcap`, `\breserve`, `\bminted`, `\bflow`, `\bvalue` and `\bquorum`, and appear in the notation index (Section D). The cap macro is `\bcap`, rendering $\overline{M}_c$; κ is the collusion-set size of Section 23 and is not used in this section.

symbol	domain	meaning
$\mathcal{C}$, c	—	the set of canonical parallel chains, and a member of it
M_c	$\mathbb{Z}_{\geq 0}$ sat	minted supply of the canonical token on c
$\mathcal{S}_R$	—	the published set of L1 reserve scripts (transparent P2SH)
R, R_{hot}, R_{cold}	$\mathbb{Z}_{\geq 0}$ sat	reserve balance, and its hot/cold split
ς	$\mathbb{Z}$ sat	reserve slack, $\varsigma := \mathsf{R} - \sum_c M_c$
$\overline{M}_c$	$\mathbb{Z}_{\geq 0}$ sat	per-chain contract constant, defence-in-depth only; the 560,000,000 FLUX cap is *global* and enforced by Definition 22.6 under honest-minority operation (Definition 22.15)
ϱ_c, ϱ_c^{burn}, T_{win}	$\mathbb{Z}_{\geq 0}$ sat, time	chain c's contract-enforced mint rate limit and burn ceiling, each decided at 10,000,000 FLUX per window; the sliding window, decided at 24 h. Both are per chain: there is no bridge-wide quantity ϱ
D_{L1}, D_c	blocks / finality	required confirmation depth on the L1 and on c
(n_M, k_M), (n_R, k_R)	$\mathbb{Z}^2_{>0}$	mint-authorisation quorum; L1 release quorum; both decided at $(4, 3)$
(n_A, k_A), Δ_A	$\mathbb{Z}^2_{>0}$, time	administrative quorum, decided at $(4, 3)$, and its timelock
$\mathcal{G}$, Δ_g	set, time	guardian set, and the maximum guardian pause
$(n_{\text{att}}, k_{\text{att}})$, β_{att}	$\mathbb{Z}^2_{>0}$, FLUX	bonded attester set and per-attester bond
T_{att}, T_{silence}, T_{react}	time	attestation cadence, silence timeout, reaction time
Λ_c, Λ	$\mathbb{Z}_{\geq 0}$ sat	legacy circulating supply on c at the snapshot; $\Lambda = \sum_c \Lambda_c$
R_0	$\mathbb{Z}_{\geq 0}$ sat	initial L1 backing locked before migration opens
$\mathcal{I}_c$	—	issuer-held (team) address set on c, published before h_{snap}; its balances are excluded from Λ_c
h_{snap}, $h_{\text{open}}^{\text{mig}}$, $h_{\text{close}}^{\text{mig}}$	height	migration snapshot height; the opening of the one-directional window (retiring chains, March 2027) and its close at shutdown (August 2027)
ℓ, used_c	—	lock identifier $(\text{txid}, \text{vout})$; consumed lock set on c
n_b, B_c	$\mathbb{Z}_{\geq 0}$	a burn nonce; the next burn nonce on c
φ_{mint}, φ_{rel}	$\mathbb{Z}_{\geq 0}$ sat	per-operation bridge fees; $F_{\text{mint}}, F_{\text{rel}}$ their cumulative sums
V_{lock}, V_{rel}, V_{top}, V_{out}	$\mathbb{Z}_{\geq 0}$ sat	cumulative locks, releases paid, other reserve inflows, other outflows

Table 57: Symbols local to Section 22. Every quantity is read at a stated pair of heights $(h, \{h_c\}_{c \in \mathcal{C}})$.

22.1 The incumbent, stated factually

Everything in this subsection is SHIPPED, bar the unfinished rewrite marked IN-PROGRESS, and is cited to the deployed source. The system has worked for years and its operators have recovered from failures by hand. The argument that follows is not that it has failed; it is that its worst case is unbounded and unobservable, and that a design exists whose worst case is neither.

What it is. Fusion is a Node.js service listening on port 9,876 [fusion/config/default.js:6], backed by MongoDB, that moves FLUX between eleven chains — main, KDA, ETH, BSC, TRX, SOL, AVAX, ERG, ALGO, MATIC and BASE [fusion/config/default.js:106,110]. It has three sub-systems: a snapshot/claim engine that pays out balances captured by an operator manually running `flux-cli printsnapshot` [doc: fusion/README.md:27–29]; a swap engine with the state machine *new* → *waiting* → *confirming* → *exchanging* → *finished* and side-states *hold*, *expired*, *dust* [fusion/src/services/swapService.js:861–871]; and a coinbase-claim path. SHIPPED

Custody. For each of the eleven chains, `config/default.js` holds up to three role-specific secrets — `snapshotsecrets`, `miningsecrets`, `swapsecrets` — each a single address with a single raw private key, loaded into the process and used to sign in-process [fusion/config/default.js:342–482]. That is $10 + 11 + 11 = 32$ hot keys, `main` having no snapshot secret. There is no MPC, no threshold signature and no multisig anywhere on the path that moves funds out on any chain [fusion/src/coinLibs/eth.js:22,70–101] [fusion/src/coinLibs/main.js:12–22]. The one function whose name contains "multisig", `signatureValidFluxMultisig`, verifies a *user's* ownership of a P2SH address as a claim precondition; it says nothing about how outgoing funds are authorised [fusion/src/services/generalService.js:413–432]. What the operator can therefore do unilaterally is move any balance held under any of the 32 keys to any address, at any time, with no second party. Whether a cold/hot split exists operationally is indicated only by three named addresses in a snapshot-exclusion list — "Flux Swap Pool LOCKED/HOT/COLD"; no code in the repository moves funds between them [fusion/snapshotScriptsMATIC/createFluxMainChainSnapshotMATIC.js:60–70]. SHIPPED

The in-progress rewrite does not change the model. In `fusion-bridge`, `BridgeZUSDC.sol` lets either `owner` or `withdrawer` — plain `address` state variables — call `releaseUSDC(to, amount)` for an arbitrary recipient and amount, with no proof-of-burn and no Merkle check inside the contract [fusion-bridge/packages/eth-contracts/contracts/BridgeZUSDC.sol:13–14,97–102]. The Kadena side's mint requires a keyset of one public key under a `keys-all` predicate, i.e. 1-of-1 [fusion-bridge/packages/kda-contracts/src/deploy/define_keyset/ks.ktpl:4–7]. The signer microservice reads its key from a plain environment variable, with Vault integration present only as commented-out code [fusion-bridge/apps/fusion-signer/src/config/keys.ts:1–2,14–27]. IN-PROGRESS

Correctness of the state machine. Four facts, each a live code path. SHIPPED

1. A swap is written to the database as `status: 'finished'` with `txidTo: 'Sent. Awaiting blockchain confirmation...'` *before* the outbound send is attempted [fusion/src/services/swapService.js:893–902]; the real txid is written back only under the guard `if (txid && typeof txid === 'string')` [fusion/src/services/swapService.js:997–1006]; and every `send<Chain>()` returns the `Error` object itself on failure [fusion/src/coinLibs/eth.js:106–112]. A failed send therefore leaves a *finished* record carrying a placeholder txid, with no retry, no alert and no rollback.

2. Concurrency guards are module-level booleans (`swapNotSafe`, `someClaimInProcess<CHAIN>`, `automationInProgress`) and an in-memory `Set`; none survives a restart or a second process instance [fusion/src/services/swapService.js:443–461] [fusion/src/services/claimSnapshotService.js:181–190]. The swap-creation guard is a busy-wait that proceeds anyway after 8 s [fusion/src/services/swapService.js:454–458].

3. Three different confirmation-multiplier tables exist for nominally the same "enough confirmations" decision [fusion/src/services/swapService.js:1085–1090,1181–1219] [fusion/src/services/swapAutomationService.js:164–182].

4. Recovery is manual: roughly fifty chain-pair-specific rollback scripts (`helpers/adjusToPreviousClaimState*`, `unholdSwap.js`, `removeSwapTXID.js`) and a *hold*

state with no automated exit [fusion/src/services/swapService.js:1216–1219] [fusion/helpers/processTransactionManual.js:4–5].

Gating that exists, and where it lives. Swap amounts above 100,000 FLUX (1,000,000 FLUX for a hardcoded premium list) are refused outright [fusion/src/services/swapService.js:150–157] [fusion/config/default.js:160–195]; swaps still *new* after 90 confirmations go to *hold* [fusion/config/default.js:111]; the blacklist and the premium list are source-code literals requiring a redeploy to change [fusion/config/blackList.js:1–27] [fusion/config/premiumList.js:1–12]. The fee is 0.2 % plus a flat per-chain amount [fusion/config/default.js:223–236]. Rate limiting is IP- and user-agent-based with hardcoded Origin/Referer bypass rules [fusion/src/lib/server.js:29,39,54,65,80,91]. All of this gating is enforced in JavaScript inside one process, not on any chain. SHIPPED

Reserve accounting. The EVM tokens were created by minting 440,000,000 FLUX to the deployer in the constructor, at `decimals = 8` [flux-eth-bsc/flux.sol:6–13]. Circulating parallel supply is therefore measured as `totalSupply()` minus the sum of reserve wallets, and the two team-maintained reserve-wallet exclusion lists disagree — neither is a superset of the other, so any such figure depends on which list is used (see Section 6). SHIPPED

Terminology. Three separate systems share the name. This paper fixes the usage, and uses it consistently from here on:

- Fusion is the incumbent custodial bridge backend — the `ZelCore-io/fusion` service described above, plus the unfinished `fusion-bridge` rewrite that shares its custody model. This is what the construction in Section 22.3 would replace.
- FusionX is the user-facing exchange and redemption product [doc: fusion-exchange-frontend/DISCLAIMER.md:1], whose backend in the corpus is a pass-through proxy to environment-configured third-party URLs [fusion-exchange-backend/routes/trading/exchange.js:96–100]. Its custody model is outside the corpus and this paper makes no claim about it.

They are different things, and the distinction matters for Section 22.6: atomic swaps are a plausible mechanism for FusionX's product and are not a mechanism for Fusion's.

Not established by this survey. the actual custody model behind FusionX's production backend host is not in the evidence corpus. The frontend's own "non-custodial" claim is unverified and is not repeated here.

22.2 Requirements

The requirements below are derived from the failure modes of Section 22.1 plus the team constraint. They reference the participant and adversary definitions of Section 2. **H** denotes hard, **D** desirable.

Definition 22.1 (Dominance over the incumbent, H0)**.** A replacement design *dominates* the incumbent if, for every participant compromise the design admits, the worst-case loss is (i) strictly less than the incumbent's — the entire balance held under all 32 hot keys, unbounded — and (ii) publicly observable within a stated bound T_{react}. A design that fails H0 at any point is rejected regardless of how decentralized it is.

H0 is the team constraint made checkable, and it is the reason the construction leans on contract-enforced bounds rather than on the number of parties involved. The remaining hard requirements:

H1 — supply is capped by the contract. On every $c \in \mathcal{C}$, $M_c \leq \overline{M}_c$ is enforced by the contract, not by an operator, and M_c is readable by anyone.

H2 — no mint without a lock; no release without a burn. Every unit minted on c corresponds to a specific, confirmed, single-use lock on the L1; every unit released on the L1 corresponds to a specific, confirmed, single-use burn on some c.

H3 — no single key moves funds. No operation that increases M_c or decreases R is authorisable by fewer than $k \geq 2$ independently held keys, and every such authorisation is attributable on-chain to the signers that produced it.

H4 — bounded failure under quorum compromise. If k_M mint signers collude, the loss is bounded by the contract's rate limit and cap, not by the signers' restraint. If k_R release signers collude, the loss is bounded by R_{hot}. H4 is meaningful only against a contract the colluding signers cannot rewrite: an upgrade path reachable by the same k_M keys reduces H4 to the signers' restraint, which is what Definition 22.11 states and what the timelock condition there is for.

H5 — burn is unconditional. No role — admin, guardian or attester — can prevent a holder from burning canonical tokens and thereby publishing a claim on the L1 reserve. Pausability applies to mint only.

H6 — auditability from outside. The reserve invariant of Section 22.4 is verifiable by anyone with a Flux node and public RPC access to each chain, with no privileged data and no trust in any team-maintained wallet list. The migration carry alone takes a published issuer-held set $\mathcal{I}_c$ as an input, checkable against the legacy contract (Algorithm 9); steady-state verification takes none [intent: design intake].

H7 — audit before deploy. Every deployed contract and program has a published independent audit of the exact deployed bytecode; migration and sink contracts are in scope [intent: design intake].

H8 — migration conserves value. Each legacy unit is redeemable 1:1 into main-chain FLUX [roadmap: flux-roadmap-public.md:133]; no legacy unit is redeemed twice; and $R_0 \geq \Lambda$ holds — the L1 backing for the whole legacy circulating supply exists — before any migration mint occurs.

H9 — every failure is a stall, not a silent success. The incumbent's first failure mode — recording success before it happens — is forbidden by construction: no off-chain record is authoritative; chain state is.

Desirable, not hard: **D1** redemption liveness under an uncooperative operator (stated as desirable because Section 22.10 shows it cannot be made unconditional); **D2** minimal admin surface — the admin cannot mint, transfer, freeze accounts or change code, and every admin action that weakens a bound is timelocked and public; **D3** immutability — deployed contracts are not upgradeable in place, replacement is by holder-initiated migration; **D4** a path to bonded attestation — pause authority transferable from named guardians to the attestation network of Section 14 without redeploying the token; **D5** one codebase — ETH and BSC deployments are byte-identical source with chain-specific constructor parameters only [intent: design intake].

Tensions, named rather than resolved silently. Raising k/n makes theft need more colluders and censorship need fewer refusers; at the decided $(n, k) = (4, 3)$ theft needs 3 signers and censorship needs 2 [inferred]. No setting maximises both. The design resolves this by making the *consequences* asymmetric: theft is bounded by the contract (H4) and censorship is bounded

by time, so k may lean toward theft-resistance — at the cost, at $n = 4$, that the censorship side is thin, which is why Definition 22.23 is stated as widely as it is. Second, a rate limit tight enough to bound attacker damage also blocks an exchange migrating tens of millions of tokens at once, so the migration path is specified to bypass ϱ_c and to be bounded instead by Λ_c and a closing height — which makes the migrator the largest single mint authority in the design and puts it squarely in audit scope. Third, H3 asks that no operation moving funds be authorisable by fewer than $k \geq 2$ *independently held* keys; the decided configuration meets the arithmetic ($k = 3$) and not the independence, and Definition 22.17 says so rather than letting the requirement appear satisfied. Fourth, D3 asks that a bug be fixed by deploying again and migrating rather than by upgrading in place — the price of an audit whose conclusions stay true, and Section 22.11 records what an in-place upgrade cost Nomad. **D3 is not satisfied by the decided configuration**: the contracts are upgradable under the same 3-of-4 [intent: design intake], so the construction below specifies immutability while the decision retains an upgrade path. Definition 22.11 states the consequence for Definition 22.9; it is not a drafting inconsistency to be smoothed over, because the bound this section exists to state depends on which of the two is deployed.

22.3 The construction

Everything in this subsection is PLANNED. Nothing described here is deployed, and no sentence about it should be read in the present tense.

Definition 22.2 (Canonical parallel asset)**.** For a chain $c \in \mathcal{C}$, the *canonical* FLUX token on c is the unique contract or program that is the sole issuer of FLUX on c: it has no pre-mint, no owner mint, no proxy, and issuance only against a signed authorisation referencing a single-use L1 lock, subject to a cap $\overline{M}_c$ and a rate limit ϱ_c that the contract enforces itself. Its unit is the satoshi (`decimals` $= 8$ on every chain, matching the legacy EVM tokens [flux-eth-bsc/flux.sol:11–13]), so the supply invariant is an integer identity. The decided launch set is $\mathcal{C}_0 = \{\mathrm{ETH}, \mathrm{BSC}\}$ with $\mathcal{C}_1 = \mathcal{C}_0 \cup \{\mathrm{SOL}\}$ following [intent: design intake]; one EVM codebase would be deployed twice, plus one Solana program later. PLANNED

Definition 22.3 (L1 reserve)**.** The reserve is a published set $\mathcal{S}_R = \{s_{\mathrm{hot}}, s_{\mathrm{cold},1}, \dots\}$ of transparent P2SH scripts on the Flux L1, published together with their redeem scripts so that any party can recompute both the addresses and the key sets. Its balance is $\mathsf{R} := \sum_{o \in \mathrm{UTXO}(\mathcal{S}_R)} \mathrm{value}(o) = \mathsf{R}_{\mathrm{hot}} + \mathsf{R}_{\mathrm{cold}}$. P2SH is used because it is the only multi-party spending condition Flux's inherited script offers; the reserve cannot be shielded, both because a shielded reserve would defeat H6 and because the relevant shielded path is closed after height 835,554 [fluxd/src/main.cpp:1398–1408]. Three L1 ledgers are derivable from a full node: the *lock ledger* (outputs to s_{hot} whose transaction carries an `FLXB1` memo), the *release ledger* (spends from $\mathcal{S}_R$ carrying an `FLXR1` memo), and the *top-up ledger* (all other inflows). No off-chain database is a source of truth (H9). PLANNED

Construction 22.4 (Canonical mint-and-burn with attested proof-of-reserve, PLANNED)**.** Lock-and-mint from the L1 to c; burn-and-release from c to the L1. Four authorities are separated by role, as in Table 58. The contract state on each c is

$M_c,\ \overline{M}_c,\ \varrho_c,\ \varrho_c^{\mathrm{burn}},\ T_{\mathrm{win}},\ \mathsf{bucket}_c,\ \mathsf{bucket}_c^{\mathrm{burn}}$	supply, caps, limits, window, buckets
$\mathsf{used}_c \subseteq \{0,1\}^{256}$	consumed lock identifiers
rem_c	unminted remainder per pending lock
$B_c,\ \mathsf{cumBurn}_c,\ \mathsf{cumMint}_c,\ \mathsf{cumMig}_c$	burn counter, cumulative totals
$\Lambda_c,\ h_{\mathrm{close}}^{\mathrm{mig}}$	migration allowance and closing block
$Q_M,\ k_M,\ Q_A,\ k_A,\ \Delta_A,\ \mathsf{queue}$	quorums, timelock, queued changes
$\mathcal{G},\ \mathsf{pausedUntil}$	guardians; mint pause expiry
$\mathsf{migrationTarget},\ \mathsf{predecessor}$	set at most once; fixed at construction

with no `owner`, no `withdrawer` and no proxy admin slot. The five messages are:

Lock (L1 transaction). Output o_1 pays v sat to s_{hot}; output o_2 is an `OP_RETURN` carrying `"FLXB1"`$\|\text{ver}(1)\|\text{chainId}_c(4)\|\text{recipient}(32)$, with exactly one output to $\mathcal{S}_R$. The lock identifier is $\ell := (\text{txid}, \text{index}(o_1))$. The transaction is ordinary and needs no cooperation from any party. The CumulusVPN entitlement memo `CVPN1:` is the deployed precedent for an `OP_RETURN`-carried binding on this chain [cumulusvpn/gateway/internal/entitle/entitle.go:99–108].

MintAuth (off-chain, self-authenticating). For $i \in Q_M$,

$$\text{sig}_i \;=\; \text{Sign}_{sk_i}\Big(H\big(\texttt{"FLXMINT"} \,\|\, \text{chainId}_c \,\|\, \text{addr}(\text{contract}_c) \,\|\, \ell \,\|\, v \,\|\, \text{recipient}\big)\Big).$$

The digest is domain-separated by chain identifier *and* contract address, so an authorisation for one chain cannot be replayed on another or on a successor contract.

Mint (transaction on c). `mint(`ℓ`, v, recipient, sigs[])`, submittable by anyone, admitted by Algorithm 6.

Burn (transaction on c). `burn(v, fluxAddr)` emits $\text{Burn}(n_b, v, \text{fluxAddr}, \text{sender})$ with $n_b := B_c$, then $B_c \leftarrow B_c + 1$. It is permissionless, has no allowlist, and is never pausable by any role (H5). On Solana the mint's `freeze_authority` is set to `None` at creation, irrevocably, for the same reason. The decided configuration adds a burn ceiling $\varrho_c^{\text{burn}} = 10{,}000{,}000\,\text{FLUX}$ per window [intent: design intake], metered by a second leaky bucket $\mathsf{bucket}_c^{\text{burn}}$ on the same T_{win}. A ceiling on burn is not the same kind of object as a ceiling on mint — mint is the direction that can create unbacked value, burn is the direction that lets a holder leave — so its interaction with H5 has to be stated rather than assumed benign, and Definition 22.5 does so.

Release (L1 transaction). Inputs are k_R-of-n_R spends of s_{hot} UTXOs; outputs pay $v_j - \varphi_{\text{rel}}$ to each `fluxAddr`$_j$ with change to s_{hot}; an `OP_RETURN` carries `"FLXR1"`$\|\text{chainId}_c\|n_b^{\text{from}}(8)\|n_b^{\text{to}}(8)$ for a *contiguous* nonce range. Batches are constructed by Algorithm 7.

role	may	may not
MINT quorum (n_M, k_M)	authorise a mint for a confirmed lock	mint beyond $\overline{M}_c$ or ϱ_c; mint a used lock; pause; change parameters
RELEASE quorum (n_R, k_R)	co-sign L1 spends of R_{hot} to burners	anything on c
GUARDIAN $\mathcal{G}$ (any single member, 1-of-4)	pause mint for $\le \Delta_g$; cancel a queued admin action or upgrade [intent: design intake]	pause burn or transfer; mint; release; change parameters
ADMIN (n_A, k_A), timelock Δ_A	change $\overline{M}_c$, ϱ_c, quorum sets, $\mathcal{G}$; set `migrationTarget` once; *upgrade the contract* [intent: design intake]	mint; transfer; freeze

Table 58: The four authorities of Definition 22.4. The role separation is a property of the *contract*: the MINT role cannot pause and the ADMIN role cannot mint directly, whoever holds the keys. It is *not* a property of the signer sets at launch, because the MINT, RELEASE and ADMIN rows are held by the same 3-of-4 quorum (Table 59), so three compromised keys hold all three at once. The GUARDIAN row is the exception that makes Definition 22.9 evaluable: at 1-of-4 it is reachable by a key the attacker does not hold. The ADMIN row's upgrade power, however, reaches every other row indirectly — see Definition 22.11. Definition 22.17 and Definition 22.18 state the consequence rather than the table implying otherwise. PLANNED

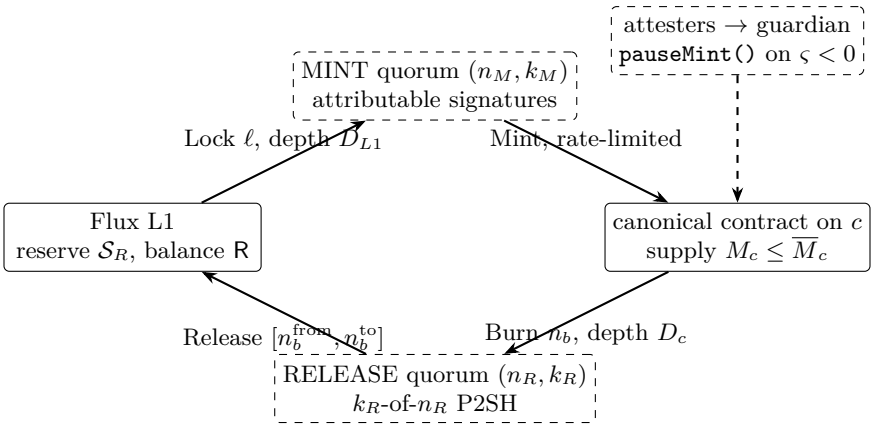


Figure 18: The two directions of Definition 22.4. Burn is permissionless and never pausable; mint is capped, rate-limited and pausable. PLANNED

The decided constants. Table 59 fixes the parameters on which every quantitative claim in this section turns. All nine are decisions, not findings; the provenance is three intake rounds and the paper carries no independent confirmation of any of them. PLANNED

parameter	decided value	enforcement point
$\overline{M}_c$	250,000,000 FLUX	contract constant on each c, checked on every mint path; the global 560,000,000 FLUX cap is enforced by the reserve under honest-minority operation, not here (Definition 22.15)
ϱ_c	10,000,000 FLUX *per chain*	leaky bucket inside each contract, Algorithm 6
ϱ_c^{burn}	10,000,000 FLUX *per chain*	second bucket on the burn path (Definition 22.5)
T_{win}	24 h	leaky-bucket drain interval for both buckets
$(n_M, k_M) = (n_R, k_R) = (n_A, k_A)$	$(4, 3)$	one signer set for all three, the four named co-founders
$\mathcal{G}$	the same four, **1-of-4**	any single member pauses mint for $\leq \Delta_g$ (Definition 22.10), or cancels a queued admin action or upgrade [intent: design intake]
limit adjustability	none	ϱ_c, ϱ_c^{burn} fixed at deployment
upgrade path	**present**, under $(4, 3)$, behind $\Delta_A = 48$ h	admin quorum may replace contract code after a 48 h public queue (Definition 22.11, Definition 22.12)
premine	0 on every c	no constructor mint; M_c begins at zero

Table 59: Decided bridge parameters [intent: design intake]. The quorum figures supersede the (n, k) recommendations that Definition 22.17 previously carried as open. The guardian row is no longer marked for confirmation: the pause is decided at 1-of-4, which is what makes Definition 22.9 a bound rather than a restatement of the cap. The last two rows are in tension with each other — limits that cannot be adjusted sit inside code that can be replaced — and Definition 22.11 is where that tension is resolved or not. PLANNED

Why the limit is per chain and not per address, which is the team's argument and is correct. A per-address mint limit is not a limit. Addresses on every $c \in \mathcal{C}$ are free to create: an EVM address is the hash of a locally generated public key and costs nothing to produce, so an adversary subject to a limit of x per address obtains nx by naming n recipients, at the cost of n transactions and nothing else [intent: design intake] [inferred]. Nothing in the Lock memo or the MintAuth digest of Definition 22.4 binds `recipient` to an identity, and no such binding is available on any chain in $\mathcal{C}$ without an identity registry the design does not have and does

not want. A rate control is only a control when the resource it meters cannot be manufactured by the party being metered; the bucket in Algorithm 6 meters issuance itself, which cannot be. The same reasoning rules out a per-lock or per-transaction ceiling, which an adversary splits the same way.

Where the limit is enforced, and why a per-chain limit makes that question disappear. The contracts are one per chain and cannot read each other's storage, so a limit defined over $\sum_c M_c$ would have no single enforcement point: it could only be a signing policy the mint quorum observes voluntarily, which bounds nothing in the case the limit exists for, since the party applying the policy is by hypothesis the compromised party. **The decision removes that problem rather than solving it.** ϱ_c is a per-chain constant, and a statement about chain c's own issuance over c's own clock is exactly what c's contract can check about itself. The bound therefore holds *under quorum compromise*, because the contract applies it whatever the signatures say, and no allocation of a shared budget across $\mathcal{C}$ has to be decided, enforced or revisited. Two consequences are worth naming. Honest throughput is $|\mathcal{C}|\varrho_c$ rather than a shared ϱ, so capacity does not have to be predicted per market — which was the operational argument against a fixed split. And the adversary's bound scales the same way: it is additive in the number of live chains (Definition 22.9), so *adding a chain raises the worst case by* 10,000,000 FLUX *per reaction window*, which makes chain count a security parameter and not only a product decision.

No premine, and why that makes the cap unreachable by honest use. The legacy EVM tokens were created by minting 440,000,000 FLUX to the deployer in the constructor [flux-eth-bsc/flux.sol:6–13]. The canonical contracts mint nothing at construction, so $M_c = 0$ at deployment and every unit on c traces to a specific L1 lock or to a migration deposit (Definition 22.2, H2). Combined with Definition 22.6 this has a consequence worth stating: honest issuance is bounded by the reserve, $\sum_c M_c \le \mathsf{R}$, and R cannot exceed issued main-chain supply, which was 429,466,823.97 FLUX at height 2,912,015 [measured: emission model, observed mode, 2026-09-03] (Section C). The cap of 560,000,000 FLUX— global, and enforced by Definition 22.6 rather than by a contract constant (Definition 22.15) — therefore cannot bind on any honest path for as long as issued supply is below it. On the deployed schedule that is the early 2050s: issued supply reaches 548,043,625.86 FLUX one year into the fixed-subsidy tail that begins at height 23,044,000 (height 24,095,199, approximately 2046-10) and then grows at a constant 1,789,219.274256 FLUX per year, so 560,000,000 FLUX is first issued about 11,956,374/1,789,219 ≈ 6.7 years later, in 2053 [inferred](Section C). Only the per-chain constant $\overline{M}_c$ binds on the unbacked path, and it is reached in 25 d at the permitted rate. That is the precise sense in which the cap is a backstop and not a bound.

Algorithm 6: Contract-side admission of `mint` on chain c. PLANNED

Input: $(\ell, v, \text{recipient}, \text{sigs}[])$; contract state of Definition 22.4; wall clock `now`.
Output: $v' = v - \varphi_{\text{mint}}$ minted to `recipient`, or revert.
Invariant: On any successful return: $M_c \le \overline{M}_c$, $\ell \in \mathsf{used}_c$ once $\mathsf{rem}_c[\ell] = 0$, $\mathsf{bucket}_c \le \varrho_c$, and $\mathsf{cumMint}_c$ increased by exactly u.
Failure mode: Revert. Execution on c is atomic per transaction, so no partial state change is possible; the caller may resubmit. No off-chain record is written anywhere (H9).

if now $<$ pausedUntil **then return** revert (paused);
$S \leftarrow$ the set of distinct members of Q_M with a valid signature in sigs[] over the MintAuth digest for *this* chain identifier and *this* contract address;
if $|S| < k_M$ **then return** revert (quorum);
if $\ell \in \mathsf{used}_c$ **then return** revert (lock fully consumed);
$v' \leftarrow \mathsf{rem}_c[\ell]$ if set, else $v - \varphi_{\text{mint}}$; `// unminted remainder of a pending lock`
if $v' \le 0$ ***or*** $v < v_{\min}$ **then return** revert (dust);
$\mathsf{bucket}_c \leftarrow \max\big(0,\ \mathsf{bucket}_c - \varrho_c \cdot \Delta t / T_{\text{win}}\big)$; `// leak since last update`
$u \leftarrow \min\big(v',\ \overline{M}_c - M_c,\ \varrho_c - \mathsf{bucket}_c\big)$; `// admissible now`
if $u \le 0$ **then return** revert (cap or rate limit; ℓ stays pending);
$\mathsf{rem}_c[\ell] \leftarrow v' - u$; **if** $\mathsf{rem}_c[\ell] = 0$ **then** $\mathsf{used}_c \leftarrow \mathsf{used}_c \cup \{\ell\}$;
$\mathsf{bucket}_c \mathrel{+}= u$; $\mathsf{cumMint}_c \mathrel{+}= u$;
mint u to `recipient`;

Why a sliding window and not an epoch counter. The bucket in Algorithm 6 drains linearly over T_{win} rather than resetting at fixed epoch boundaries. With fixed epochs, an adversary who times a compromise at a boundary obtains $2\varrho_c$ instantly, before any reaction is possible; with a sliding window the total admitted over an interval of length T is the headroom left in the bucket at the start plus the leak over T, which is at most $\varrho_c(1 + T/T_{\text{win}})$ and is the accounting Definition 22.9 rests on [inferred]. This is the same class of timing attack that a deterministic payout schedule admits, and the leaky bucket removes it.

The mint path in full. A holder broadcasts a Lock. Each mint-quorum signer, from *its own* L1 node rather than a shared RPC, waits until the lock is at depth $\ge D_{L1}$. The floor is $D_{L1} \ge 40 =$ `MAX_REORG_LENGTH` [fluxd/src/main.h:74], because a reorganisation deeper than 40 blocks is rejected by consensus and the lock is then irreversible for every node following the rules; at the PoN target spacing this is $40 \times 30\,\text{s} = 20\,\text{min}$ [inferred]. The signer checks memo well-formedness, that o_1 is the only reserve output, and that $\ell \notin \mathsf{used}_c$ read from its own node on c, then publishes its signature. Anyone may then submit the mint; the fee φ_{mint} was locked and never minted, so it remains in R as surplus and requires no fee key. A lock whose value exceeds the bucket's headroom — including any single lock above ϱ_c — is neither refused nor refunded: it stays pending and is minted in instalments across successive windows, and no admin refund power exists [intent: design intake]. Algorithm 6 is written for the whole-lock case; the instalment form keeps an unminted remainder per ℓ in place of the binary used_c and admits $\min(v',\ \varrho_c - \mathsf{bucket}_c,\ \overline{M}_c - M_c)$ per call, and every bound below is unchanged by it because each instalment passes the same tests.

Decided. $D_{L1} = 40$ (Table 62). $D_{L1} \ge 40$ blocks. Raising it above 40 adds latency and buys nothing against the 2-of-4 emergency-block path [fluxd/src/emergencyblock.cpp:183–191], which bypasses eligibility but not the reorg limit. Trade-off: user-visible mint latency against nothing; 40 is both the floor and the chosen value: it is exactly the consensus reorg bound, and exceeding it buys nothing.

Decided. $v_{\min} = 10\,\text{FLUX}$, $\varphi_{\text{mint}} = 0$, $\varphi_{\text{rel}} =$ the batch's gas share, recipient team operations

(Table 62). $v_{\min}$ and the fee schedule $(\varphi_{\text{mint}}, \varphi_{\text{rel}})$, plus the fee recipient. Domain: $v_{\min} >$ L1 dust threshold $+ \varphi_{\text{rel}}$. Trade-off: griefing resistance and release-batch size against small-holder access. Constraint: fees must not scale a signer's income with volume that signer can itself generate.

Decided. A Flux-hosted feed plus a mandatory IPFS mirror (Table 62). Transport for MintAuth signatures. A Flux-hosted feed is acceptable on the merits — signatures are self-authenticating and anyone may relay them, so the feed can withhold but cannot forge — but the choice of transport and whether a Flux-independent mirror is mandatory were the open items, settled as above. Trade-off: operational simplicity against a withholding vector that delays holders who rely on one relayer.

Algorithm 7: Release-batch construction, executed independently by each release signer. PLANNED

Input: The burn log of chain c up to depth D_c; the L1 release ledger for c; the UTXO set of s_{hot}; batch cadence T_{rel}.

Output: A signature share on a Release transaction covering a contiguous nonce range, or abstention.

Invariant: The set of nonce ranges released for c is pairwise disjoint and every released nonce is $< B_c$. Value paid never exceeds $\sum_c \mathsf{cumBurn}_c$ minus what was already released.

Failure mode: Fewer than k_R signers sign ⇒ the batch is not broadcast and exits *stall visibly*; no partial payment occurs and no burn is lost, because the claim remains on-chain on c. A signer that detects an overlapping range abstains, which is the enforcement point for the disjointness invariant — the L1 does not check it. A malformed address forfeits its own burn and blocks no other [intent: design intake].

$\mathcal{N} \leftarrow$ nonces n_b of burns on c at depth $\geq D_c$, ordered;
$\mathsf{released}_c \leftarrow$ union of nonce ranges appearing in `FLXR1` memos for c on the L1;
$[n^{\text{from}}, n^{\text{to}}] \leftarrow$ the maximal contiguous prefix of $\mathcal{N} \setminus \mathsf{released}_c$;
if $[n^{\text{from}}, n^{\text{to}}] \cap \mathsf{released}_c \neq \emptyset$ **then return** abstain (double release);
if $n^{\text{to}} \geq B_c$ **then return** abstain (nonce beyond counter);
foreach *burn* j *in the range* **do**
    verify $\texttt{fluxAddr}_j$ is a well-formed transparent address;
    if *malformed* **then** $v_j \leftarrow 0$ (the nonce stays inside the range and is paid nothing; the memo records it as *unpayable*, so the prefix advances);
choose inputs i and burners b subject to $34b + 401i + 60 \leq 2{,}000{,}000$ bytes ; `// ≤ MAX_TX_SIZE_AFTER_SAPLING at the decided 3-of-4`
return signature share on the resulting transaction;

Remark 22.5 (The burn ceiling never refuses, and H5 therefore holds — recommended, not decided). The burn ceiling ϱ_c^{burn} admitted two implementations that are indistinguishable in a parameter table and are not the same object. Under a *refusing* implementation `burn` reverts once the window's bucket is full, and a holder is then prevented by a contract rule from publishing a claim on the reserve — which is exactly what H5 forbids, and which makes Definition 22.23 worse rather than better. Under a *delaying* implementation the exit is deferred to a later window but never refused across the sequence of windows.

Decided. A burn is never refused: the delaying implementation, recommended by this paper. Round 7 decided the 10,000,000 FLUX ceiling and not which of the two implementations carries it [intent: design intake]; no later round chooses, so the choice is recorded here as a recommendation rather than a decision [intent: design intake]. Under it H5 holds in the form *no role can prevent a burn from eventually succeeding*, and whether the implementation carries an explicit deferral

queue or simply declines to rate-limit burns at all is not consequential to any property in this section.

The reasoning is worth recording, because it also explains why the asymmetry between the two ceilings is correct rather than an oversight. A mint ceiling protects the reserve: it bounds how much unbacked supply a compromised quorum can issue, which is the whole content of Definition 22.9. A burn does the opposite — it destroys parallel supply and strictly *improves* the conservation position of Definition 22.6. Limiting it therefore protects nothing while creating a liveness risk, and the risk lands at the worst possible moment: a burn ceiling binds during a loss-of-confidence exit, which is precisely the scenario in which holders most need the exit to work. A refusing ceiling would additionally be cheap to weaponise — an adversary could saturate the window with small burns to trap other holders inside it — so the refusing reading is not merely worse on principle, it is exploitable.

The burn path in full. A holder calls `burn`; no role can block it. Each release signer waits for finality on c, not a confirmation count: on Ethereum the finalized checkpoint at two epochs is $2 \times 32 \times 12\,\text{s} = 768\,\text{s} = 12.8\,\text{min}$, or 19.2 min if a third epoch is needed [inferred]. Signers then co-sign a batch covering a contiguous nonce range. Contiguity is what makes double-release auditable from the two public ledgers. Sizing is a bounded, stated quantity rather than an emergent one, and the decided quorum makes it generous: at $(n_R, k_R) = (4, 3)$ the redeem script is $4 \times 34 + 3 = 139$ bytes and one P2SH input is approximately 401 bytes — three signatures plus the script — against `MAX_TX_SIZE_AFTER_SAPLING` of 2,000,000 bytes, so one release paying roughly 1,000 burners in about 34 kB of outputs can still consume on the order of 4,900 reserve inputs [inferred]. The same computation at the previously recommended 9-of-15 gave approximately 1,215 bytes per input and roughly 1,600 inputs; the small set costs independence and buys transaction capacity, which is the honest way round to state the trade.

Decided. Each chain's own finality tag, never a confirmation count (Table 62). $D_{\text{ETH}}, D_{\text{BSC}}, D_{\text{SOL}}$ must each be set *at finality*, not at a confirmation count — the latter is the error class that the BNB Bridge incident belongs to. Ethereum's value follows from the protocol constant above; BSC's depth under fast finality and Solana's `finalized` commitment are deploy-time parameters, set to each chain's own finality predicate rather than to a count. Trade-off: redemption latency against reorg exposure bounded by R_{hot}.

Decided. $T_{\text{rel}} = 4\,\text{h}$ (Table 62). Release cadence T_{rel}, domain $[1\,\text{h}, 24\,\text{h}]$. Trade-off: user-visible exit latency against batch efficiency and signer operational load.

Decided. $\mathsf{R}_{\text{hot}} = 500{,}000\,\text{FLUX}$, top-up when hot $< 200{,}000\,\text{FLUX}$ (Table 62). R_{hot} target, the cold-reserve threshold, and the cold→hot top-up cadence. Domain: $\mathsf{R}_{\text{hot}} \approx$ several days of expected redemption volume. Trade-off: this quantity *is* the L1-side loss bound under H4, against release latency when the hot balance is exhausted. Sharpened by the quorum decision: the three keys that reach the mint bound of Definition 22.9 are the same three that reach this one, so the exposure of a single 3-key compromise is $\mathcal{L}(T_{\text{react}}) + \mathsf{R}_{\text{hot}}$ and this target is a security parameter, not only an operational one.

Administration, guardianship, and the upgrade path the decision retains. Any single $g \in \mathcal{G}$ may call `pauseMint()`, setting $\mathsf{pausedUntil} \leftarrow \max(\mathsf{pausedUntil}, \text{now} + \Delta_g)$; the call is attributable to g, and the decided rule is 1-of-4 [intent: design intake]. The admin quorum may extend by Δ_A per signed extension, each on-chain. Burn, transfer and `migrate` are never paused, so a pause that is never lifted is a public, attributable, repeatedly-renewed act by named individuals, which is the whole defence against pause-as-censorship — and the bounded window Δ_g is what keeps a single malicious guardian able to stall the mint path but not to halt it, since the pause lapses unless renewed and renewal is again public and attributable. Admin parameter changes are queued on-chain and visible for Δ_A; *tightening* (lowering $\overline{M}_c$ or ϱ_c, removing a

signer) executes immediately, *loosening* waits. Any single $g \in \mathcal{G}$ may also call `cancel(q)` on a queued admin action or upgrade q, removing it from `queue` before it executes [intent: design intake]; the call is attributable to g, as the pause is.

The construction specifies no proxy, no `upgradeTo` and no `delegatecall` to mutable code (D3); the decided configuration keeps an upgrade path under the same 3-of-4 [intent: design intake]. Under D3, replacement on EVM would be by migration: ADMIN sets `migrationTarget` once, holders call `migrate(v)`, which burns on the old contract and calls `mintMigrated` on the new one whose `predecessor` is fixed at construction; the old contract keeps `burn` live forever, so a holder who never migrates can still exit to the L1. The predecessor path is uncapped by Λ_c and capped by $\overline{M}_c$, and the burn inside `migrate` takes no nonce and emits no `Burn`, so it enters no release range [intent: design intake]: were it a nonced burn, Algorithm 7 would pay a claim whose value had already been reissued on the successor and ς would go negative. Under the decided configuration the same replacement is available without holder action and without notice, which is convenient for repair and is the attack step of Definition 22.11. The two are not reconcilable by drafting; they are reconcilable only by the timelock Δ_A, and only if $\Delta_A > T_{\text{react}}$.

Guardian membership at launch is decided: the same four co-founders [intent: design intake], with the single-member pause of Definition 22.4 retained and now confirmed at source — Definition 22.10 shows why that retention is load-bearing rather than incidental. $(n_A, k_A) = (4, 3)$ is likewise decided, so the previously open clauses $n_A \le n_R$ and $k_A \ge k_R$ are satisfied trivially and vacuously: they hold because the sets are identical, which is not what they were written to secure (Definition 22.18).

Decided. $\Delta_g = 48\,\text{h}$ (Table 62). Δ_g, domain $[6\,\text{h}, 7\,\text{d}]$. Note that $\Delta_g > T_{\text{react}}$ is *not* a constraint here: Definition 22.9 needs the pause to begin, not to persist, so what Δ_g must outlast is only the time to convene the admin quorum and decide what to do. Trade-off: at 1-of-4 the pause is cheap to trigger by design, so Δ_g is exactly the cost a single malicious or mistaken guardian can impose per call. Longer is a stronger stop and a longer stall; the bound on abuse is that stalling is repeated, public and attributable, while halting outright is not available at any Δ_g.

The parameter timelock is likewise 48 h (Definition 22.12), which satisfies $\Delta_A > T_{\text{react}}$ for any reaction time under two days and leaves a holder that long to complete a burn-and-release round trip before a parameter change binds them.

Which multisig implementation, and why not the audited one. On EVM the mint check would be per-signer ECDSA verification inside the bridge contract — a distinct-signer bitmap and `ecrecover` against Q_M — and specifically *not* the Halborn-audited Schnorr `MultiSigSmartAccount` [doc: account-abstraction/aa-schnorr-multisig-sdk/README.md:76]. Two evidence-based reasons. First, the aggregated Schnorr signature is verified as a single combined address, so the contract never learns *which* k signers participated [account-abstraction/contracts/schnorr/Schnorr.sol:7–26], which defeats H3. Second, its X-of-Y semantics are realised by enumerating every qualifying signer subset as a separate owner address [account-abstraction/aa-schnorr-multisig-sdk/src/helpers/schnorr-helpers.ts:129–152], i.e. $\sum_{j \ge k} \binom{n}{j}$ role grants — 29 for $(7, 5)$ but 4,944 for $(15, 10)$, on the order of 124×10^6 gas of grants at initialisation [inferred]— while the SDK's nonce-reuse protection is process-local and the documented consequence of reuse is private-key exposure [doc: account-abstraction/aa-schnorr-multisig-sdk/README.md:14–16]. The Schnorr account is the right tool for a 2-of-2 wallet and the wrong tool for a bridge quorum. On Solana, the SSP `Solana-Multisig` program keeps `approvals: Vec<Pubkey>` on-chain and is therefore attributable [Solana-Multisig/programs/solana-multisig/src/lib.rs:880–918], with the threshold enforced at spend time [Solana-Multisig/programs/solana-multisig/src/lib.rs:468–471]; it is a candidate for the ADMIN and upgrade authority, but only after an external audit — none is recorded in the corpus — and after its own mainnet upgrade authority is moved off the single keypair it is on today [doc: Solana-Multisig/docs/MAINNET_RUNBOOK.md:1–13]. The bridge program's Solana mint check would verify k_M Ed25519 signatures

through the native Ed25519 program with instruction introspection: precisely the code path whose sysvar check was the Wormhole vector (Section 22.11), and therefore a named audit focus. **The quorum question is closed.** $(n_M, k_M) = (n_R, k_R) = (4, 3)$, one set, the four named co-founders [intent: design intake]. Both sets sit well inside the P2SH ceiling — standardness caps the redeem script at 520 bytes, which is 15 compressed keys [inferred]— so the constraint that bounded n_R from above is not active. The theft/censorship trade-off resolves as follows at $(4, 3)$: theft needs 3 colluders and censorship needs 2 refusers, so the release path is easier to stall than to rob, and Definition 22.23 is correspondingly wider than it would be at a larger n_R. Section 22.7 states what the shared membership costs.

Decided. The named hardware class and procedure of §22.9. Custody standard for the four keys: dedicated signing hardware, keys never on a host running a relayer, an RPC or the incumbent service, and a documented key-generation ceremony. Domain: a named hardware class plus a published procedure. Trade-off: a stricter standard slows signing and reduces the chance that one compromise of a general-purpose host yields a key — which under a single 3-of-4 set yields every capability at once. Note that the clauses about *organisational* distribution and jurisdictional spread are not open parameters any more; they are not satisfied at all, and Definition 22.17 says so.

22.4 The supply invariant

Theorem 22.6 (Reserve dominance)**.** *Let all quantities be read at a stated pair of heights $(h, \{h_c\})$, and assume (i) every mint on every c passed the checks of Algorithm 6; (ii) at most $k_M - 1$ mint signers and at most $k_R - 1$ release signers are corrupt on each chain; (iii) $R_0 \geq \Lambda$ (H8); and (iv) $V_{\text{out}} = 0$, i.e. no reserve outflow occurs other than releases. Then*

$$\varsigma \;:=\; \mathsf{R} - \sum_{c \in \mathcal{C}} M_c \;=\; \underbrace{\Big(\Lambda - \sum_c \mathsf{cumMig}_c\Big)}_{\text{(1) unmigrated legacy}} + \underbrace{\Big(V_{\text{lock}} - \sum_c \mathsf{cumMint}_c - F_{\text{mint}}\Big)}_{\text{(2) locks not yet minted}}$$
$$+ \underbrace{\Big(\sum_c \mathsf{cumBurn}_c - V_{\text{rel}} - F_{\text{rel}}\Big)}_{\text{(3) burns not yet released}} + \underbrace{\Big((R_0 - \Lambda) + V_{\text{top}} + F_{\text{mint}} + F_{\text{rel}} - V_{\text{out}}\Big)}_{\text{(4) surplus}}$$

and every bracket is non-negative; hence $\varsigma \geq 0$, i.e. $\mathsf{R} \geq \sum_c M_c$, at every pair of heights. PLANNED

Proof sketch. The contract's arithmetic admits exactly three mutators of `totalSupply`, so $M_c = \mathsf{cumMig}_c + \mathsf{cumMint}_c - \mathsf{cumBurn}_c$. UTXO accounting over $\mathcal{S}_R$ gives $\mathsf{R} = R_0 + V_{\text{lock}} + V_{\text{top}} - V_{\text{rel}} - V_{\text{out}}$, where V_{rel} counts only value paid to burners — fees withheld at release never leave the reserve and are therefore counted once, in bracket (4). Substituting and grouping yields the displayed identity by cancellation of Λ, F_{mint} and F_{rel}.

Non-negativity, bracket by bracket. (1) The migration mint path is bounded by $\mathsf{cumMig}_c + v \leq \Lambda_c$ inside the contract, and $\sum_c \Lambda_c = \Lambda$. (2) Each public mint consumes a distinct ℓ from the lock ledger and mints $v' = v - \varphi_{\text{mint}} \leq v$; single-use is enforced by used_c under consensus on c, and value correctness by the honest signer that assumption (ii) guarantees is present in any set of k_M valid signatures. (3) Each release pays a nonce range contained in $[0, B_c)$, and disjointness is enforced by the abstention rule of Algorithm 7, which runs whenever at least one of the k_R required signers is honest — again assumption (ii). (4) $R_0 - \Lambda \geq 0$ by assumption (iii), $V_{\text{top}} \geq 0$ and $F_\bullet \geq 0$ by construction, and $V_{\text{out}} = 0$ by assumption (iv).

Two dynamic cases. *Across the migration*: bracket (1) drains into $\sum_c M_c$ while R_0 stays in R, so ς decreases monotonically toward the surplus and never below it. *Across a successor contract*: `migrate` burns on the predecessor and mints on the successor within one transaction, so $\sum_c M_c$ is unchanged and ς is invariant; neither leg is a nonced burn or a migration mint, so brackets (1) and (3) are unchanged as well. □

Corollary 22.7 (The quiescent form, and what the roadmap says)**.** *Write* $S_{\text{main}} := S(h) - \mathsf{R}$ *for main-chain circulating supply. In the quiescent state — no operation in flight, migration complete (*$\sum_c \mathsf{cumMig}_c = \Lambda$*), surplus swept — every bracket vanishes, so* $\varsigma = 0$ *and therefore* $\sum_c M_c + S_{\text{main}} = S(h)$*. That identity is the roadmap's phrasing, "minted supply across every chain plus main-chain supply equals total issuance" [roadmap: flux-roadmap-public.md:99]. The live form is the inequality* $\varsigma \geq 0$ *with the decomposition of Definition 22.6, which is strictly more useful: it is what makes a* negative ς *diagnosable, because each bracket going negative names a distinct failure — migrator overrun, unbacked mint, unbacked release, or reserve theft — and each is attributable to exactly one role in Table 58.* PLANNED

Algorithm 8: Public verification of the reserve invariant. Runnable by anyone. PLANNED

Input: A Flux full node; one RPC endpoint per $c \in \mathcal{C}$; the published $\mathcal{S}_R$ with redeem scripts; a chosen height pair $(h, \{h_c\})$.

Output: ς and the four brackets of Definition 22.6.

Invariant: The computation reads only public chain state. It requires **no** team-maintained wallet list and no privileged endpoint.

Failure mode: If any bracket is negative, report it together with the role it implicates. If an RPC is unreachable, the run aborts rather than substituting a cached value — a partial read must not produce a reassuring number.

recompute each address in $\mathcal{S}_R$ from its published redeem script; abort on mismatch;
$\mathsf{R} \leftarrow$ sum of unspent outputs to $\mathcal{S}_R$ at height h;
$V_{\text{lock}} \leftarrow$ sum over the `FLXB1` lock ledger; $V_{\text{rel}} \leftarrow$ sum paid to burners over the `FLXR1` release ledger;
foreach $c \in \mathcal{C}$ **do**
  read $M_c, \mathsf{cumMint}_c, \mathsf{cumMig}_c, \mathsf{cumBurn}_c, B_c$ from contract storage at h_c;
  check the released nonce ranges for c are pairwise disjoint and all $< B_c$;
$\varsigma \leftarrow \mathsf{R} - \sum_c M_c$; evaluate brackets (1)–(4);
return ς and the brackets;

Who can verify, and why the absence of a wallet list matters. Anyone. R and the three L1 ledgers are transparent chain facts by Definition 22.3; M_c and the four cumulative counters are public contract storage on c. A verifier needs a Flux full node and one RPC endpoint per chain, and needs *no* team-maintained wallet list. This is not a stylistic point. Section 6 establishes that two team-maintained reserve-wallet exclusion lists exist today and that they disagree — neither is a superset of the other — so any current circulating-supply figure that nets out team reserves depends on which list the reader happens to use. Having exactly one published $\mathcal{S}_R$, recomputable from published redeem scripts, removes that dependency entirely. Total issuance $S(h)$ is itself verifiable from `gettxoutsetinfo` together with `valuePools`, with the standing caveat that ZIP-209 turnstile enforcement is compiled in but dormant [fluxd/src/chainparams.cpp:322–327]; note that Definition 22.6 does not depend on that caveat, because it compares R against $\sum_c M_c$ and never against $S(h)$.

Proposition 22.8 (Bounded loss under full mint-quorum compromise)**.** *Drop assumption (ii) of Definition 22.6 and suppose* k_M *or more mint signers on chain* c *are corrupt from time* t_0*, with a pause first effective at* $t_0 + T_{\text{react}}$*. Write* $b_c(t) \in [0, \varrho_c]$ *for the bucket fill. Then the unbacked issuance they can produce on* c *is at most*

$$\big(\varrho_c - b_c(t_0)\big) \;+\; \varrho_c \cdot \frac{T_{\text{react}}}{T_{\text{win}}} \;\leq\; \varrho_c\Big(1 + \frac{T_{\text{react}}}{T_{\text{win}}}\Big),$$

and at most $\overline{M}_c - M_c(t_0)$ *in total; every such mint is an on-chain transaction from which the colluding signers' identities are recoverable.* PLANNED

Proof sketch. Algorithm 6 applies the cap and bucket checks after the quorum check and regardless of its outcome, so no set of signatures can exceed them; corruption removes only the guarantee that a lock exists, never the guarantee that the bounds hold. For the bucket term, every admitted mint of v' increases b_c by exactly v' and the only decrease is the leak at rate ϱ_c/T_{win}, so the total admitted over $[t_0, t_0+T]$ equals $b_c(t_0+T) - b_c(t_0) + \varrho_c T/T_{\text{win}}$, and $b_c(t_0+T) \le \varrho_c$ by the admission test. Mints authorised before t_0 and submitted after it consume the same bucket, so they displace rather than add to the adversary's share; the earlier phrasing "plus in-flight authorisations" overstated the bound and is withdrawn. Attributability follows from per-signer ECDSA verification against Q_M: the signatures are contract inputs and are therefore permanently public. □

With the parameters of Table 59 this becomes a number, and it is the number on which the section's security claim now rests. Two hypotheses have to be carried explicitly, because each of them is a decision that can be taken either way and the property is false without them.

Property 22.9 (Worst-case loss of the replacement bridge)**.** Let $\varrho^\star := 10{,}000{,}000\,\text{FLUX}$ be the decided per-chain mint rate limit, so $\varrho_c = \varrho^\star$ for every $c \in \mathcal{C}$, with $T_{\text{win}} = 24\,\text{h}$ (Table 59). Suppose the 3-of-4 quorum of Definition 22.17 is compromised at t_0 and the adversary mints as fast as Algorithm 6 admits on every c. Assume

(P) a pause is first effective at $t_0 + T_{\text{react}}$ with T_{react} finite, which is what the 1-of-4 guardian rule of Definition 22.10 supplies; and

(U) the adversary cannot replace the contract's code inside T_{react}, which requires an upgrade timelock $\Delta_A > T_{\text{react}}$ (Definition 22.11).

Then the unbacked issuance over $[t_0,\, t_0 + T_{\text{react}}]$ is

$$\mathcal{L}(T_{\text{react}}) \;\le\; \underbrace{|\mathcal{C}|\,\varrho^\star \cdot \frac{T_{\text{react}}}{T_{\text{win}}}}_{\text{sustained}} \;+\; \underbrace{\textstyle\sum_{c\in\mathcal{C}} \left(\varrho^\star - b_c(t_0)\right)}_{\text{headroom at } t_0,\ \le\ |\mathcal{C}|\,\varrho^\star} \qquad \text{and} \qquad \mathcal{L} \;\le\; \sum_{c\in\mathcal{C}} \left(\overline{M}_c - M_c(t_0)\right).$$

The sustained term is additive in the number of live chains and evaluates as in Table 60. The headroom term adds at most one further $|\mathcal{C}|\,\varrho^\star$ — 20,000,000 FLUX over the launch set — and is largest exactly when the bridge is idle at the moment of compromise. The cap term does not bind on any chain for $T_{\text{react}} < 25\,\text{d}$. PLANNED

Proof. Sum the per-chain bound of Definition 22.8 over $c \in \mathcal{C}$ with $\varrho_c = \varrho^\star$ on both the sustained and the headroom term; the cap bound is the check $M_c + v' \le \overline{M}_c$, which Algorithm 6 applies on every path. Hypothesis (P) is what makes T_{react} a finite quantity to substitute, and Definition 22.10 discharges it. Hypothesis (U) is what makes Algorithm 6 the code that runs for the whole of $[t_0, t_0 + T_{\text{react}}]$; Definition 22.8 bounds what *that* algorithm admits and says nothing about a successor contract, so without (U) the sum is over an empty set of enforced checks. For the numbers, with $|\mathcal{C}_0| = 2$ at launch: $2\varrho^\star \cdot (24/24) = 20{,}000{,}000\,\text{FLUX}$, $2\varrho^\star \cdot (48/24) = 40{,}000{,}000\,\text{FLUX}$, and $3\varrho^\star \cdot (24/24) = 30{,}000{,}000\,\text{FLUX}$ once $\mathcal{C}_1 = \mathcal{C}_0 \cup \{\text{SOL}\}$; and $\overline{M}_c/\varrho^\star = 250{,}000{,}000/10{,}000{,}000 = 25$ windows of 24 h, so from an empty contract the cap on one chain is first reached at 25 d, which exceeds any T_{react} the design contemplates [inferred]. □

chains live	T_{react}	sustained worst-case mint
$\mathcal{C}_0 = \{\text{ETH}, \text{BSC}\}$ (launch)	24 h	20,000,000 FLUX
$\mathcal{C}_0 = \{\text{ETH}, \text{BSC}\}$	48 h	40,000,000 FLUX
$\mathcal{C}_1 = \mathcal{C}_0 \cup \{\text{SOL}\}$	24 h	30,000,000 FLUX

Table 60: The sustained term of Definition 22.9 at the decided per-chain limit $\varrho^\star$ = 10,000,000 FLUX per 24 h [intent: design intake]. Headroom at t_0 adds at most a further $|\mathcal{C}|\,\varrho^\star$. These figures hold under hypotheses (P) and (U) and not otherwise. PLANNED

Corollary 22.10 (The pause is reachable by a party outside any 3-key compromise)**.** *Definition 22.9 is a statement about T_{react}, so it would be vacuous if the adversary also held the pause. The adversary does not. The decided rule is that* `pauseMint()` *is a* ***single-member action of*** $\mathcal{G}$ ***at*** $|\mathcal{G}| = 4$ *— 1-of-4, with the bounded window Δ_g [intent: design intake]. An adversary holding three of the four keys therefore does not control the pause: one guardian remains able to act, and T_{react} is bounded above by that member's detection-and-response time. Hypothesis (P) of Definition 22.9 is discharged, and the worst-case bound holds rather than collapsing to the cap.* PLANNED

Proof sketch. T_{react} enters Definition 22.9 only through the time at which pausedUntil first exceeds `now` in Algorithm 6. That state is written only by `pauseMint()`, whose authorisation predicate is the guardian rule. Under the decided single-member rule the predicate is satisfiable by any one of four keys; an adversary holding three cannot prevent the fourth from broadcasting, because the call is a permissionless transaction on c once signed, and nothing in Algorithm 6 lets a mint consume or reset pausedUntil. The bound is on T_{react} as a *human* latency, not a protocol one: the honest guardian must notice, decide and sign, which is the subject of the paragraph after Definition 22.11. Note what the proof does *not* establish — that the pause holds. It expires at Δ_g unless renewed, and renewal against a compromised admin quorum is a repeated race the guardian wins only by repeating the call. □

Why the pause is cheap and the mint is not, which is the design reason and not a compromise. Pausing and minting have opposite requirements. **Pausing is a safety action that should be cheap to trigger and expensive to abuse; minting is a value-creating action that should be expensive to trigger and is not abusable at all if the trigger holds.** A high threshold on the mint quorum buys exactly what a high threshold is for — more colluders needed to create value from nothing. The same threshold on the pause buys nothing and costs the bound, because the set that must be unable to mint is the set that would then also be able to refuse to stop. So the two thresholds are set in opposite directions, 3-of-4 and 1-of-4, and the asymmetry is the point rather than an inconsistency. What makes the cheap trigger tolerable is the bounded window: a single malicious or mistaken guardian can impose Δ_g of stalled minting per call, publicly and attributably, and cannot halt the bridge, because burn, transfer and `migrate` are never pausable (H5) and the pause lapses on its own.

Why the bound is proved against quorum compromise at all, since the objection is fair. A reader may reasonably ask why the section spends its central property on a scenario in which three of four named co-founders' keys are in an attacker's hands — a bar that is high, and this section says so. Two answers, and the section states them rather than assuming the reader shares the premise. First, **a bound that holds only when nobody is compromised is not a security property.** It is a description of normal operation, which the contract's ordinary behaviour already supplies. The entire value of a rate limit is what it guarantees in the case one hopes never happens; evaluated against an honest quorum it guarantees nothing that was in doubt. Second, **the case is not hypothetical in this domain.** Ronin was a 5-of-9 validator compromise, and Table 63 lists it alongside other multi-signer quorums that were taken whole.

The bar being high is a real and stateable mitigation, and it is the reason the design is defensible at launch (Section 22.7); it is not a reason to leave the case unmodelled. PLANNED

Remark 22.11 (Upgradability voids the rate limit unless the upgrade is slower than the reaction). The limits are fixed at deployment and not adjustable — and the contracts are upgradable under the same 3-of-4 [intent: design intake]. An adversary holding three keys is not obliged to mint under a limit it can delete: **step one of the attack is to upgrade the contract and remove the limit**, after which the bound of Definition 22.9 constrains code that is no longer running. As specified today the rate limit therefore bounds nothing under exactly the scenario it exists to bound, and the strongest true statement available is the cap term, $\sum_c(\overline{M}_c - M_c(t_0))$ — a quantity starting from $\overline{M}_c = 250{,}000{,}000\,\mathrm{FLUX}$ per chain against a present outstanding liability of 223,027,781.21 FLUX across all ten measured legacy chains [measured: scripts/13-issuer-balances.sh, 2026-09-03], and therefore more than an order of magnitude weaker than Table 60. Note that the 1-of-4 pause does not rescue this on its own: an upgrade that removes the limit can remove the guardian rule in the same transaction.

The fix composes with what is already decided and costs nothing new. Put the upgrade path behind the admin timelock Δ_A and require

$$\Delta_A \;>\; T_{\mathrm{react}},$$

which is hypothesis (U). Then a malicious upgrade is queued on-chain and publicly visible before it takes effect; the 1-of-4 pause can fire during the delay; the limit is still the code that runs for the whole reaction window; and the rate limit binds again. The mechanism is the one the construction already specifies for loosening parameter changes, applied to the code as well as to the constants. Cancellation of a queued upgrade or admin change is likewise a single-member guardian action [intent: design intake]; without it the pause would defer the upgrade's effect only until Δ_g lapses, the queued upgrade would execute at $t_0 + \Delta_A$ regardless, and the bound of Definition 22.9 would end there with $\sum_c \overline{M}_c$ as the only ceiling after it. **Without a timelock on the upgrade path, this section can state the cap and cannot state a worst-case bound at all**, and every figure in Table 60 is conditional on it. This was the load-bearing open item in the bridge design; the decision follows. PLANNED

Remark 22.12 (Δ_A decided: 48 h). The upgrade timelock is set at $\Delta_A = 48\,\mathrm{h}$, with upgrades remaining 3-of-4 [intent: design intake]. Unanimity was considered and rejected for a reason worth recording, because it is the opposite of the one that first suggests itself: 4-of-4 would indeed prevent an adversary holding three keys from upgrading, but it has no fault tolerance at all — a single lost, destroyed or unavailable key would make the contract permanently unupgradable, converting a key-theft risk into a key-loss risk, and for signer sets this small key loss is the more common failure. The recovery path from that state is a fresh deployment and another migration, which is precisely the event upgradability exists to avoid.

Upgradability is a bring-up mechanism with immutability as its stated destination. The team's reasoning for keeping it, and for keeping the quorum at 3-of-4, is that a management path is a safety net while the system is being verified to work without bugs, and that "it's easier to upgrade contract rather than releasing a new one and doing migration"; as the network evolves the intent is to move to non-upgradable contracts and automate what the quorum does today [intent: design intake] (VISION). The paper records this because it changes what the trust assumption *is*: not a permanent property of the design, but an explicitly temporary one whose removal is intended and undated.

That framing is coherent, and the honest statement of its weakness is a matter of sequencing rather than of principle. An upgradable contract under a 3-of-4 quorum is, for as long as it lasts, a contract whose rules those three signers can rewrite — the 48 h timelock bounds the speed of that rewrite, not its scope (§22.7). A commitment to eventual immutability constrains nothing until it has a trigger, and none is stated: no value threshold, no elapsed-time condition, no audit

milestone. Table 62 does not assign one, because a date the team has not chosen would be an invention. What the paper can say is what a trigger would need to look like — a published condition, checkable by a third party, after which the upgrade path is renounced on chain — and that until such a condition exists, the bridge should be described as administered rather than trust-minimized, which is how §22.7 describes it.

The renunciation trigger: stated in shape, proposed here in checkable form. Two preconditions for locking immutability are settled: a **third-party security audit**, and a **proven operating record across edge scenarios — chain reorganizations named explicitly** [intent: design intake] (PLANNED). Separately, the team states that code will primarily be written and audited using state-of-the-art AI models, with the third-party audit as an additional and distinct requirement rather than a substitute [intent: design intake]. The paper records that as the stated development practice; it is a claim about process, and this document takes no position on the assurance it provides, which is why the separate third-party requirement is the load-bearing one here.

Both preconditions are the right ones, and neither is yet a trigger, because a trigger has to be checkable by someone who does not work on the project. "A proven time record" is not falsifiable without a duration, and — more importantly — **elapsed time proves nothing on a bridge nobody uses**. A contract that sits idle for a year has demonstrated nothing about its behaviour under load, under reorganization, or under an adversary; it has demonstrated only that it was idle. Any time-based condition therefore needs a volume condition beside it, or it can be satisfied by inactivity. The following formulation is this paper's proposal, chosen so that every clause is verifiable from public data:

1. **Audit.** A third-party security audit of the *deployed bytecode* — not of a source tree that was later modified — published in full, with every critical and high finding either resolved in a deployed upgrade or publicly accepted with reasoning.

2. **Duration.** At least 1,051,200 L1 blocks (≈ 12 months at the 30 s target) of continuous mainnet operation *after* the audit's publication.

3. **Exercise.** Cumulative bridged volume over that period of at least 10,000,000 FLUX — one day of a single chain's rate limit, and enough that the record reflects use rather than idleness.

4. **Clean record.** Within that period: zero emergency pauses invoked, and zero upgrades executed to correct a contract defect. Upgrades for unrelated feature work do not reset the clock; an upgrade fixing a correctness bug does, because the record being demonstrated is precisely that no such bug remains.

5. **Reorganization evidence.** A published log of reorganizations observed on each connected chain during the period, with the deepest handled on each, demonstrating that the finality parameters of Table 62 were exercised rather than merely configured.

6. **Renunciation on chain.** The upgrade path is then removed by transaction — the admin role renounced, not merely unused — so that the change is verifiable by reading the contract rather than by trusting a statement about it.

Clause 5 is the one that speaks directly to the stated concern, and it is also the one most likely to be unsatisfiable on schedule: reorganizations on the connected chains are not events the project can schedule, and a quiet year produces no evidence. If the record is thin when the other clauses are met, the honest options are to wait or to state that the reorganization evidence is absent — not to treat silence as proof.

Decided. The six renunciation criteria above are this paper's construction. What is settled is their *shape*: a third-party security audit plus a proven operating record across edge scenarios including chain reorganizations [intent: design intake]. The quantities — twelve months, 10,000,000 FLUX of cumulative volume, zero defect-fixing upgrades — are proposed so that the stated shape becomes checkable by a third party rather than asserted; they can be changed without changing the shape. One element is a design point rather than a number and should not be dropped when the numbers are revised: renunciation must be executed *on chain*, by removing the admin role, so that it is verifiable by reading the contract rather than by trusting a statement about it.

At 48 h the hypothesis (U) of Definition 22.9 is discharged for any $T_{\text{react}} < 48\,\text{h}$, and the value sits at the low end of the interval this specification gave, matching common practice for governance timelocks. Two consequences follow and neither is enforced by code, so both are stated as conditions rather than assumed.

First, the timelock bounds the attack only if the upgrade queue is watched. A malicious upgrade announced and unnoticed for 48 h takes effect, the rate limit is deleted, and the loss reverts to being bounded only by the cap. Monitoring the queue is therefore a security control of the same standing as T_{react}, not an operational nicety.

Second, 48 h *is sized for the bound rather than for notice.* A timelock does two jobs: it bounds a compromise, and it gives holders time to exit before a rule change binds them. Two days is adequate for the first and thin for the second on a bridge carrying a nine-figure balance — a holder who does not look over a weekend misses the window entirely. A longer delay on changes that affect holders, with 48 h retained for security fixes, would serve both; the paper notes this once and does not press it.

T_{react} is now the most important operational parameter in the bridge, and this is not obvious from the construction. $\mathcal{L}$ is linear in T_{react} with slope $|\mathcal{C}|\,\varrho^{\star}/T_{\text{win}}$, so over the launch set every hour of detection-and-response latency is worth $2 \times 10{,}000{,}000\,\text{FLUX}/24 \approx 833{,}333\,\text{FLUX}$, rising to approximately 1,250,000 FLUX per hour once Solana is live [inferred]. Three things change that slope and the design should be read as choosing among them: the number of live chains, the per-chain limit, and T_{react} itself. Several things that look as though they should do not. The cap does not: it is 25 d away. The quorum does not: it is compromised by hypothesis. The audit does not: Definition 22.9 assumes the contract behaves as specified, and an audit is what makes that assumption true rather than what makes the loss smaller. Reserve custody does not, because the mint side never touches R. What does compress T_{react} is monitoring cadence, alert routing, who is on call, whether a guardian key is reachable at 03:00 local time, and how long a human takes to believe an alarm — none of which are contract parameters and all of which are the operational questions this bound turns into first-order security questions. The attestation link of Section 22.5 exists to compress T_{react}, and its fail-closed silence timeout T_{silence} is the mechanism that puts a ceiling on it without a human in the loop. PLANNED

Proposition 22.13 (Dominance over the incumbent, H0, with a number on one side)**.** *Under Table 59 and hypotheses (P) and (U) of Definition 22.9, the replacement's worst-case loss over a reaction window is* $\mathcal{L}(T_{\text{react}})$*, which is* 20,000,000 FLUX *over the launch set at* $T_{\text{react}} = 24\,\text{h}$ *— a quantity fixed by contract constants before any compromise occurs, and the same for every compromise class that can reach the mint path, since Algorithm 6 applies the bucket regardless of which signatures it saw. The other directions are bounded separately and also in advance: the release side by* R_{hot}*, and the guardian and relayer roles by nothing they can move. Every bound-consuming event is a public transaction that Algorithm 8 reads, so H0(ii) is satisfied with* T_{react} *as the stated observability bound. The incumbent's corresponding worst case is the entire balance held under 32 single-signature hot keys across eleven chains at the moment of compromise [fusion/config/default.js:342–482]: a quantity fixed by nobody, published nowhere, and detectable only by an operator reading its own records, since an unauthorised outgoing transfer from a hot wallet is indistinguishable on-chain from an authorised one.* PLANNED

Proof sketch. The replacement half is Definition 22.9 for the mint direction, the hot/cold custody split for the release direction, and Table 58 for the remaining roles, none of which can mint, transfer or freeze. The observability half follows because each bound-consuming event is a transaction on a public chain and Algorithm 8 reads exactly those transactions from public state, with no team-maintained list. The incumbent half is Section 22.1: single-key in-process signing on all eleven chains, with every gate — amount ceiling, blacklist, rate limit — enforced in the same process that holds the keys [fusion/src/services/swapService.js:150–157] [fusion/src/lib/server.js:29,38]. □

What the comparison is, stated exactly. H0(ii) is met outright: a stated T_{react} (24 h, Table 62) against no detection bound at all. H0(i) is met in the form that matters — a finite bound fixed in advance against an exposure bounded by nothing in the design — and this is the substantive change from earlier drafts, which could state only the shape of the bound and not its value. It is *not* a comparison of two measured numbers, because the incumbent's side has never been measured.

Not established by this survey. the incumbent's current hot-wallet balances across the 32 keys are not in the evidence corpus. Definition 22.13 therefore compares a stated finite bound against an unbounded exposure. Should that balance ever be measured and found below $\mathcal{L}(T_{\text{react}})$, H0(i) would need restating in per-window rather than absolute terms; the measurement is a prerequisite task, not a paper gap.

22.5 The attestation link

Definition 22.14 (Attestation)**.** An attestation is the tuple

$$\mathsf{A} = (h,\ \text{hash}_{L1}(h),\ \mathsf{R}_{\text{hot}},\ \mathsf{R}_{\text{cold}},\ \{(c, h_c, \text{hash}_c(h_c), M_c, \mathsf{cumMint}_c, \mathsf{cumMig}_c, \mathsf{cumBurn}_c, B_c)\}_{c\in\mathcal{C}},\ V_{\text{lock}},\ V_{\text{rel}},\ \varsigma,\ \mathsf{flags})$$

signed by attester j as $\text{Sign}_{sk_j}(H(\texttt{"FLXATT1"}\|\mathsf{A}))$. Every field is an objective chain fact at a stated height — the roadmap's "states they observed" category [roadmap: flux-roadmap-public.md:135–136] — so a false attestation is provably false to anyone holding the relevant block. flags carries the signer's verdicts on the two policy invariants the L1 cannot enforce: that every `FLXR1` range is disjoint and below B_c, and that every release corresponds to a burn at depth $\geq D_c$. An attestation is *valid* when $\geq k_{\text{att}}$ distinct bonded attesters sign the same A. PLANNED

Where it lives, and the cost finding that decides it. Attestations would be published off-chain on a public feed, with only the *consequence* on-chain. Posting every attestation to Ethereum is not viable: at one attestation per L1 block, 2,880 per day, and $k_{\text{att}} = 9$ signatures each, the cost is approximately 346×10^6 gas per day — between eight and twelve Ethereum blocks' worth of gas daily for one bridge's bookkeeping [inferred]. At $k_{\text{att}} = 5$ it is 276×10^6 and at 15 it is 449×10^6. The on-chain component is therefore a single guardian contract, `AttesterGuardian`, one per chain, added to $\mathcal{G}$ alongside the four named guardians of Table 59 once the attestation network exists — which is exactly the D4 transfer path, and the only mechanism in the design by which pause authority reaches a party outside the four: it stores the attester key set and k_{att}, and exposes `pauseOnAttestation(A, sigs[])`, which pauses mint if and only if the signatures are valid and A asserts $\varsigma < 0$ or a false policy flag. It would additionally *fail closed on silence*: if no valid attestation is submitted within T_{silence}, the guardian pauses mint for Δ_g. That direction is deliberate. The deployed SAS watchdog fails *open* on hash-service outage — if both hash endpoints are unreachable it treats the node as secure and does nothing [private source: SAS watchdog] — and under the team constraint an attester outage should stop issuance rather than continue it. Burn is unaffected in either case, so the degraded bridge is exit-only, which is the correct degradation.

Decided. $T_{\text{att}} = 120$ blocks, $T_{\text{silence}} = 6\,\text{h}$, fail-closed adopted (Table 62). T_{att} (natural domain: one attestation per L1 block, 2,880 per day, down to one per hour), T_{silence}, and whether fail-closed is adopted. Trade-off: a shorter cadence and a shorter silence timeout detect faster and make an availability failure of the feed into a liveness failure of the bridge.

Decided. The independent mirror is mandatory (Table 62). The feed itself and whether a Flux-independent mirror is mandatory. IPFS is the precedent the project already uses for parameter distribution [fluxd/zcutil/fetch-params.sh:17]. Trade-off: a single Flux-operated feed can withhold but not forge; an independent mirror removes the withholding vector at operational cost.

Detecting a false attestation, and what the bond is at risk for. A false attestation is one whose fields do not match the canonical chains at the stated heights, or whose flags assert a policy invariant the ledgers contradict. Detection is by any party running Algorithm 8 — the same computation, no privileged access — and the proof of falsity is the relevant block header plus a Merkle inclusion or storage proof for the contradicting value. Equivocation, meaning two different A for the same heights, is proven by the two signatures alone and needs no chain data. The bond β_{att} would be at risk for signing a false A, for equivocation, and (minor, to keep the set honest) for sustained non-participation. The economic condition that makes collusion unprofitable is

$$k_{\text{att}} \cdot \beta_{\text{att}} \;>\; \varrho_c \cdot \left\lceil \frac{T_{\text{react}}}{T_{\text{win}}} \right\rceil \qquad \text{for every } c \in \mathcal{C}, \tag{7}$$

i.e. the forfeited bonds must exceed what a false "all is well" can cover for, which is exactly the bound of Definition 22.8.

The decided rate limit makes (7) very expensive, and this is worth stating rather than discovering later. The condition is per chain, so the per-chain limit decision leaves it unchanged in form and in magnitude: substituting $\varrho_c = \varrho^\star = 10{,}000{,}000\,\text{FLUX}$ and $T_{\text{react}} = T_{\text{win}} = 24\,\text{h}$ gives $k_{\text{att}} \cdot \beta_{\text{att}} > 10{,}000{,}000\,\text{FLUX}$. Using the deployed Stratus tier collateral of 40,000 FLUX purely as a yardstick and not as a recommendation, that is $k_{\text{att}} > 250$ bonded attesters — against $k_{\text{att}} = 9$, which would put only 360,000 FLUX at risk, short of the condition by a factor of about 28 [inferred]. Three responses exist and the design does not yet choose between them: raise β_{att}, which concentrates the attester set and is the failure the network exists to avoid; lower $\varrho^\star$, which lowers the bound of Definition 22.9 and the honest throughput ceiling together; or accept that the attester layer is a *detector* that compresses T_{react} rather than a *deterrent* that makes collusion unprofitable, and say so. The third is the honest description of any attester set small enough to operate, and it is consistent with the fail-closed pause being the attesters' only on-chain power. PLANNED

Not established by this survey. $(n_{\text{att}}, k_{\text{att}})$ — the size and threshold of the attester set. Round 23's ruling that nothing is ever slashed changes the character of this gap rather than merely deferring it. β_{att} is *void*: a bond that cannot be forfeited is a deposit, so there is no bond to size, and (7) is withdrawn rather than left unparameterised. What remains is an ordinary quorum choice over an accountability-based attester set — how many named attesters, and how many must agree — undetermined because nobody has picked the numbers, not because the construction is unsolved. That is a materially smaller gap than this marker previously recorded, and it closes by decision whenever the author makes one.

The cap, the rate limit and the window are no longer open: $\overline{M}_c = 250{,}000{,}000\,\text{FLUX}$, $\varrho_c = 10{,}000{,}000\,\text{FLUX}$ per chain and $T_{\text{win}} = 24\,\text{h}$ (Table 59). Two constraints that the open form of those parameters carried should be checked against the decided values rather than dropped. The first, $\overline{M}_c \geq \Lambda_c$, holds with room: the measured legacy liability on the largest chain is 156,217,342.78 FLUX on Ethereum [measured: scripts/13-issuer-balances.sh, 2026-09-03], well below

250,000,000 FLUX. The second, $\varrho_c \ll \overline{M}_c$, holds at exactly a factor of 25 and is what makes Definition 22.9's cap term inactive.

Remark 22.15 (The 560,000,000 FLUX cap is global, and no contract can enforce it). This paper read 560,000,000 FLUX as a per-chain contract constant $\overline{M}_c$, because that is the only reading a per-chain construction can enforce by itself. That reading is wrong: the figure is a **global cap across all chains** [intent: design intake] (PLANNED). The correction matters, because it relocates the enforcement rather than the number.

No contract can enforce a global cap. A contract on chain c can read its own `totalSupply` and nothing else; it cannot observe supply on the other chains or on the L1, so it cannot evaluate the global constraint it would need to enforce. Setting $\overline{M}_c = 560{,}000{,}000$ on every chain yields an aggregate contract-enforced ceiling of $|\mathcal{C}| \times 560{,}000{,}000$ FLUX, which for the surviving pair is 1,120,000,000 FLUX — twice the intended global cap. As a backstop the constant is therefore *inert*: it is set far above any level at which it could bind.

What actually enforces the global cap is the reserve invariant. Definition 22.6 establishes $\mathsf{R} \geq \sum_c M_c$: minted supply on the parallel chains is dominated by value locked on the L1, so the global total is bounded by L1 supply and cannot exceed it however many chains exist. This is the correct and sufficient mechanism, it is already specified, and it is exactly the mint-and-burn discipline the team describes — "if we mint on some chain, another chain is burning it or locking somewhere" [intent: design intake]. The global cap is a property of the reserve, not of the contracts. That holds under honest-minority operation, assumption (ii) of the theorem; under the quorum compromise Definition 22.9 models, the only contract-enforced aggregate is $\sum_c \overline{M}_c$ — 500,000,000 FLUX at launch, 750,000,000 FLUX with Solana — and $\overline{M}_c$ is not lowered for that, because the guardian's cancel over queued upgrades (Definition 22.11) removes the path by which the aggregate was reachable [intent: design intake].

Recommendation: set $\overline{M}_c$ far lower, so it is a real backstop rather than a decoration. A per-chain constant is genuinely useful as defence-in-depth — it bounds the damage from a mint-path bug or a compromised quorum on one chain, and it does so without needing cross-chain visibility. But it only does that work if it can bind. The largest measured per-chain liability is 156,217,342.78 FLUX on Ethereum (Table 13), so a constant of 250,000,000 FLUX leaves roughly 60 % headroom over the largest realistic requirement while capping single-chain damage at under half the global supply. That is this paper's recommendation; 560,000,000 on each chain is not a cap, it is the absence of one. **Settled at** 250,000,000 FLUX [intent: design intake] (PLANNED): the per-chain constant binds at a level that bounds single-chain damage below half the global supply, while leaving roughly 60 % headroom over the largest measured per-chain liability.

The dependency, stated honestly. An attestation network built on today's primitives would inherit a defect that voids its central security claim, and Section 14 states the same finding from the other side. The `fluxsas` attestation signing seed for every purpose is derived by HKDF over salt and domain values compiled identically into every binary; none of the inputs depends on the TPM, on Secure Boot state, or on any per-machine value [private source: SAS API server]. A signature from that key therefore demonstrates that the signer holds a value shipped in every copy of the software. It does not demonstrate that a particular machine observed a particular chain state.
A key that everyone holds cannot be slashed, and a signature it produces cannot be attributed to a bond. The node-identity key does mix machine-derived values, but the key protecting its random component is itself a baked-in constant [private source: SAS disk encryption]. What Section 14 calls Requirement 0 — per-node attester keys, generated on the node, whose public half is committed on the L1 in the bonded-tier registration transaction so that a signature is attributable to a specific bond outpoint — must be met before any of Section 22.5 is more than a specification. The `purpose=mesh` voucher already present in `fluxsas` with no consumer [fluxsas/src/api_server.h:438–445] is the natural place for such a key to land once it is per-node. Note

also that the same finding applies to the benchmark signing path, so a bonded-tier registration must not reuse it.

Open Problem 22.16 (Slashing on the Flux L1). A bond held as a plain UTXO cannot be taken from its owner by consensus, and Flux's inherited Bitcoin script cannot express a slashing condition. The two options are (a) a consensus change introducing a bonded-tier transaction type with a slashing rule verified by `fluxd`, or (b) bonds escrowed in a k-of-n that adjudicates — which reintroduces exactly the trusted quorum the network exists to replace. This is the load-bearing hole in the attestation link. Until it is filled, the "bonded" tier is a *registered* tier: attributable but not slashable, and its attestations should be weighted in $\mathcal{G}$ accordingly.

Decided. there is no slashing, here or anywhere in Flux: no mechanism destroys or forfeits a holder's FLUX [intent: design intake]. That closes the construction question by removing the mechanism rather than by supplying one, and the consequence has to be carried rather than absorbed. **An attester bond that cannot be forfeited is a deposit, not a stake**, so (7) cannot rest on it and the cryptoeconomic form of the security argument is unavailable. What remains is accountability: a named, non-anonymous attester set whose members can be excluded from future participation and who answer for their attestations personally, in the same shape as the key custody of Section 2. That is a materially weaker claim than a bonded one, and this paper does not dress it as equivalent — an attester who lies loses future income and standing, not principal, and what bounds a dishonest quorum is the rate limit and the reserve invariant of Definition 22.4, not a forfeiture.

22.6 Why not the alternatives

Atomic swaps — the question is closed, not open. An HTLC atomic swap exchanges an asset one party holds on chain A for an asset a counterparty holds on chain B, atomically, with no custodian [30]. Flux's inherited script carries the primitives. But an atomic swap cannot do what a bridge does: **it cannot issue FLUX on Ethereum**, only exchange FLUX for something that already exists there. Everything else is secondary and still disqualifying: it requires a counterparty holding the other asset, a discovery layer to find them, both parties online for the duration, and it hands the initiator a free option to abort if the price moves during the timelock. The correct conclusion is not that atomic swaps are a rejected bridge design but that they address a different problem — the *exchange* of FLUX against ETH, USDC or another asset against a market maker, which is FusionX's product (Section 22.1), not the bridge's. Atomic swaps are a settled construction in the literature [30]; the boundary drawn here is a product decision, not a doubt about the primitive.

Light-client verification — not constructible for Flux, and here is exactly why. The trust-minimizing answer would be an Ethereum contract that verifies Flux headers directly, as IBC does between Tendermint chains [32] and as the interoperability literature identifies as the strongest available class [46]. It is not constructible from the current PoN header, for three compounding reasons:

1. The PoN block header carries `nodesCollateral` and `vchBlockSig` but **no commitment to the confirmed node set** [fluxd/src/primitives/block.h:44–45,64–96], so a verifier cannot check that a block's signer was eligible without the full FluxNode registry — which is to say, without being a full node.

2. Chainwork is a fixed 2^{40} per block regardless of difficulty target [fluxd/src/pow.cpp:284–290], so fork choice degenerates to a block-count race and there is no cumulative-work heuristic by which a light client could prefer one chain over another.

3. The 2-of-4 emergency-block path can produce a valid block outside the normal rules, with no time or stall-detection gating [fluxd/src/emergencyblock.cpp:183–191], so no light-client rule

derived from PoN eligibility is sound; a perfect light client would still be trusting those four keys.

A future header commitment to the deterministic node list's Merkle root would reopen the question — after which verifying 2,880 headers a day on Ethereum becomes a cost question, one `ecrecover` plus a $\lceil \log_2 6489 \rceil = 13$-hash Merkle proof per header [inferred]— and Section 8 should treat it as a candidate for the same network upgrade that carries collateral maturity. **This is the paper's clearest instance of vertical integration cutting both ways.** Controlling the L1 is what lets Flux put an entitlement memo in an `OP_RETURN`, freeze a halving by hand, and change the reorg limit to launch PoN; it is also what produced a header format that forecloses the most trust-minimizing bridge architecture available to other chains. Owning the stack does not automatically mean the stack was designed for what you later want from it. The same tension runs through Section 9. The reverse direction — `fluxd` verifying Ethereum — would need an EVM and BLS light client inside the daemon, a consensus change of a different order.

Threshold-signature and MPC bridges. One aggregate key, signed by a threshold of parties holding shares. *Trust assumption:* fewer than the threshold of share-holders collude, and the share distribution is what it is claimed to be. *Failure mode:* on-chain the contract sees a single signer, so who participated is invisible and the claimed distribution is unfalsifiable from outside; if the shares sit on one organisation's servers the threshold is cosmetic, which is precisely the Multichain outcome (Section 22.11). *Why not chosen:* attributability is a hard requirement here (H3), and the construction additionally declines to rely on the threshold alone — the contract bounds the damage whatever the signing scheme. A TSS could still be used *inside* one signer organisation to protect its own key.

External validator sets. A named set signs mint messages; the contract mints on k signatures. *Trust assumption:* fewer than k collude or are compromised. *Failure mode:* the contract does whatever k signatures say, without bound — Wormhole's 19 guardians at 13 and Ronin's 9 validators at 5 both failed this way, for different reasons. *Why not chosen, and the honest statement:* at the signing layer this construction *is* a validator-set bridge with a smaller set than either — 4 signers at a threshold of 3, against Wormhole's 19 at 13 and Ronin's 9 at 5 — and pretending otherwise would be dishonest. What differs is not the size or the independence of the set but that the cap, the per-chain rate limit, the 1-of-4 guardian pause and the admin timelock are intended to make k-collusion bounded and slow rather than total and instant: 20,000,000 FLUX per 24 h of reaction time over the launch set (Definition 22.9) instead of the pooled value. That sentence is true only under the timelock condition of Definition 22.11; an upgradable contract without one puts this construction back in the same row as the systems it is being distinguished from, which is why that item was the section's load-bearing open one until Definition 22.12. The collateral-backed alternative in the literature — overcollateralized vaults with automatic liquidation [45] — removes the trusted set at the cost of locking capital worth more than the bridged value on the destination chain, and of a price oracle for the collateral-to-asset ratio. That trade is available and was not taken; the reason is that it substitutes an oracle dependency and a capital cost for a signer dependency, and does not remove the need for the L1-side release quorum, which is where Section 22.10 shows the residual trust actually sits.

Canonical rollup bridges and IBC, for completeness. Optimism's and Arbitrum's canonical bridges derive security from the L1 verifying the L2 through fraud or validity proofs with a challenge window; neither Flux nor Ethereum can verify the other, so this is not available. Their *operational* pattern — guardian pause, security-council timelock, no silent upgrades — is adopted wholesale. IBC requires cheap, fast-final header verification on both ends [32]; Flux has neither cheap header verification nor a node-set commitment, so it is not applicable without the header change above.

Issuer-attested burn-and-mint, which is the model chosen. Circle's CCTP signs burn messages with an issuer-operated attestation service; destination contracts mint only against attested, nonce-bound messages, and no pooled liquidity exists to steal [18]. Its trust assumption is the issuer's attester key — which, for USDC, belongs to the same party that can mint USDC anyway, so the bridge adds no new trust. Flux is the closest analogue and this is the chosen model. Two differences follow from the fact that Flux's attesters are *not* already trusted for the whole supply: an explicit k-of-n with on-chain attribution rather than a single issuer signature, and contract-level cap and rate limit, which CCTP does not need.

22.7 The launch trust assumption: one quorum, bounded by a rate limit

Assumption 22.17 (Single attributable quorum, bounded by contract)**.** At launch the MINT, RELEASE and ADMIN authorities of Table 58 are held by *one* set: $n_M = n_R = n_A = 4$ and $k_M = k_R = k_A = 3$, the four keys held by the four named co-founders — Tadeas Kmenta, Daniel Keller, Jeremy Anderson, Valter Silva [intent: design intake] — with $\mathcal{G}$ the same four individuals but at a threshold of **one** (Table 59). The assumption is that fewer than three of those four keys are compromised or collude. It is one assumption, not four independent ones. What supports it: (i) each key is per-signer, so every authorisation is attributable on-chain to the key that produced it, and the signer set is published; (ii) each key is generated and held on dedicated signing hardware and never on a host that runs a relayer, an RPC, or the incumbent service — co-locating the key with the process that decides is the incumbent's defining mistake (Section 22.1); (iii) $k/n = 3/4 \geq \lceil 2n/3 \rceil / n$, so the threshold ratio is at or above the shape the requirements recommended. What is *not* satisfied, and was written into the earlier form of this assumption precisely because it matters: signers are named individuals rather than independent named organisations; one organisation's founders hold every key in every set, so no clause bounding a single organisation's share of a quorum can hold; the four memberships coincide exactly; and jurisdictional spread is whatever those four individuals happen to have rather than a design parameter. If the assumption fails, the contract still bounds unbacked issuance to $\mathcal{L}(T_{\text{react}})$ (Definition 22.9) — the pause remains reachable at 1-of-4 (Definition 22.10), and the bound additionally requires the upgrade timelock of Definition 22.11 — and the L1 side to R_{hot}. PLANNED

Remark 22.18 (The separation of powers is not available at launch, and saying otherwise would be false)*.* Three compromised keys can, in one sequence and with no second party: authorise mints up to the rate limit; co-sign L1 spends of R_{hot}; queue an admin change raising $\overline{M}_c$ and ϱ_c and replacing Q_M with keys of their own; and *upgrade the contract*, which reaches every rule in Table 58 at once including the guardian rule (Definition 22.11). What they cannot do is throw the pause on the honest guardian's behalf, prevent that guardian from throwing it, or prevent that guardian from cancelling what they queue, because the pause and the cancel are 1-of-4 (Definition 22.10) — and that single exception is what the whole quantitative claim of this section rests on. The Δ_A notice period survives for parameter changes — an admin change is still visible for Δ_A before it executes — but the exit it is supposed to protect runs through a release quorum consisting of the same three keys, so a holder who burns during the notice period has a public claim and the same adversary decides whether it is paid (Definition 22.23). The total exposure of a 3-key compromise is therefore $\mathcal{L}(T_{\text{react}}) + \mathsf{R}_{\text{hot}}$, with the cold reserve as the last line and $\sum_c \overline{M}_c$ as the ceiling if the upgrade path is not timelocked or a queued upgrade is not cancelled within Δ_A. This is stated as a property of the launch configuration, not as a hypothetical: the earlier draft of this remark described the collapse as the reason separation was a requirement, and the configuration that has been decided is the collapsed one everywhere except the pause.

Why this is defensible, on the team's own constraint — and what would make it not. The governing constraint of this section is that *a decentralized design that is exploitable*

is worse than the custodial incumbent [intent: design intake]. Taken seriously, it settles the launch question. A more distributed signer set would have to be assembled from parties who can be held to their signatures; the mechanism for holding them — a bond that can be forfeited — does not exist on this chain (Definition 22.16), and the attestation keys that would identify them are today fleet-shared rather than per-node (Section 22.5). A fifteen-key set recruited without either property is not obviously more decentralized than four named founders: it is the same trust assumption spread across more people who are harder to identify and cannot be penalised, which is the shape the Ronin and Multichain rows describe (Table 63). Against that, the decided configuration takes the trust concentration as given and spends its design effort on making the consequence bounded and public: the contract refuses beyond ϱ_c and $\overline{M}_c$ whatever the three keys say, and anyone can run Algorithm 8 without asking the signers for anything. **The accurate description is a custodial bridge with contract-enforced bounds and public verification, moving toward autonomy** — not a decentralized bridge, and not a separation of powers. The intent on record is explicit about the direction and about the fallback: full autonomy afterwards, and "if the autonomous design is not ready in time, ship 3-of-4 and take a second fork for the autonomous bridge" [intent: design intake]. Two things would make the position indefensible rather than defensible, and both are checkable. **If the upgrade path ships without a timelock satisfying $\Delta_A > T_{\text{react}}$, the bound is the cap rather than** 20,000,000 FLUX**, and the argument above does not survive the substitution** (Definition 22.11) — the contract that "refuses beyond ϱ_c and $\overline{M}_c$ whatever the three keys say" is then a contract those same three keys can replace before it refuses anything. And if the second fork does not follow, what is described here as a launch configuration becomes the design, at which point the honest comparison is no longer against the incumbent but against a competently run custodian. PLANNED

The path off the single quorum, specified as requirements because the construction is open. The intent is autonomy, and no design for it exists in the evidence corpus. What replaces the 3-of-4 must satisfy, at minimum: **(R1)** authorisation attributable to a set whose membership is derivable from public chain state rather than from a published list, so that an observer can recompute who was entitled to sign at a height; **(R2)** a penalty that can actually be applied to a misbehaving member, which on this L1 means Definition 22.16 is answered first; **(R3)** per-member keys generated on the member's own machine, which means Section 14's Requirement 0 is met first; **(R4)** liveness under member churn, since a set derived from the node registry changes between blocks while an L1 P2SH redeem script does not; and **(R5)** a migration path from the 3-of-4 that does not require holders to act, which under D3 means the ADMIN role rotating Q_M and Q_R rather than a contract replacement. The interfaces are already fixed by Definition 22.4 and do not change: the MintAuth digest, the k-of-n check inside Algorithm 6, and the P2SH release path. **The construction that meets R1–R5 is open**, and R2 and R4 are the binding pair on this chain specifically: R2 requires a consensus change (Definition 22.16), and R4 is in direct tension with a release path whose signer set is frozen into a P2SH redeem script at funding time and capped at 15 keys.

Decided. The autonomous replacement for the 3-of-4 quorum — requirements R1–R5 above, interfaces unchanged — is not constructed here, and the reason is now specific rather than general: it is gated on the same slashing problem that leaves β_{att} undeterminable, because any replacement that removes the quorum must put something forfeitable in its place. What *is* settled is the intent and the exit condition: the quorum is an explicitly temporary bring-up mechanism, and immutability is the stated destination, reachable through the criteria above [intent: design intake].

22.8 Migration and consolidation

Section 6 inventories ten chains carrying FLUX today. All ten legacy contracts would be retired, including the legacy Ethereum, BSC and Solana tokens, because the survivors are *relaunched* rather than kept [intent: design intake]; three new deployments would follow, ETH and BSC at launch and Solana later. PLANNED

Remark 22.19 (Two migration paths, and "surviving" is not the same set as "carried"). The distinction matters more than the phrasing suggests, and conflating the two sets is the easiest mistake a reader could make here. **Only Ethereum and BSC are snapshot-and-distributed**: their balances are captured at h_{snap} and reissued 1:1 into the new contracts, coordinated with exchange partners and ecosystem integrators, and their holders take no action at all [intent: design intake]. **Every other chain redeems through the incumbent bridge** to the main chain during the redemption window, from $h^{\text{mig}}_{\text{open}}$ to $h^{\text{mig}}_{\text{close}}$, with unclaimed balances passing to the team at close.

Solana sits in both descriptions and belongs to the second. It is a surviving chain in the sense that a new Solana deployment follows the launch pair — but its *legacy* holders are not carried across. They redeem to the main chain like the holders of any retired chain, and re-enter the new Solana deployment later if they choose. So the snapshot-carried set is exactly $\{\text{ETH}, \text{BSC}\}$, and the redeeming set is the other eight chains, Solana included.

This has a measurement consequence Section 22.8 states below rather than leaves implicit. An earlier draft of this remark cautioned that Solana might prove the largest single component of the redeeming set, on the reasoning that its legacy `totalSupply` measures 59,999,990.375 [measured: api.mainnet-beta.solana.com `getTokenSupply`, 2026-09-03]. That caution was wrong, and measuring it is what showed so: 99.4 % of that supply is issuer-held — 54,000,000 in the `LOCKED` account alone — leaving 373,067.20 FLUX outstanding, the *smallest* component of the redeeming set rather than the largest [measured: scripts/15-nonevm-liability.sh, 2026-09-04]. Supply is not liability, and the gap between them on Solana is a factor of 161. The general point survives the specific error: an unmeasured chain contributes an unknown, not a small, quantity, and the direction of the surprise is not predictable in advance — which is why the redesign requires Λ_c measured rather than assumed.

The measurement prerequisite: partly done. Before any migration mint, Λ_c must be measured for each of the ten legacy assets at a published snapshot height h_{snap} by transfer-log replay, and the available backing R_0 (Definition 22.3) must be established and shown to satisfy $R_0 \geq \Lambda$. Λ_c is now measured for all ten legacy assets (Table 13), totalling 223,027,781.21 FLUX outstanding SHIPPED [measured: scripts/13-issuer-balances.sh, 2026-09-03]; [measured: scripts/15-nonevm-liability.sh, scripts/16-remaining-liability.sh, 2026-09-04]. The distribution matters more than the total: 216,800,880.41 FLUX— 97.21 % — is on the two surviving chains and is carried by snapshot, and the eight retiring chains that require holder action carry 6,226,900.80 FLUX between them — 2.79 % of the total. Kadena required a different operand: its Pact module exposes no total-supply accessor, so supply was recovered by summing the `ledger` table across all twenty Chainweb chains [measured: scripts/19-kadena-ledger.js, 2026-09-04]. What remains needed before a migration mint is confirmation that reserves cover 223,027,781.21 FLUX. H8 forbids opening migration at 1:1 on any chain until both the measurement and its coverage confirmation are in hand for that chain.

Not established by this survey. the composition and balance of R_0 against the now fully-measured 223,027,781.21 FLUX of legacy liability; and per-chain holder counts are not established anywhere in the evidence corpus. This is a prerequisite task blocking Section 22.8 and Section 6, not a gap the paper can close by argument. If the measurement shows a shortfall, the migration cannot open at 1:1 until it is covered, and the paper must say so.

Two migration mechanisms, not one, and the difference decides who is exposed. The decided plan uses *different* mechanisms for the surviving and the retiring chains [intent: design intake], and conflating them is what produced the two-orders-of-magnitude error in earlier drafts of this subsection and of Section 6. **Surviving chains are carried by snapshot; retiring chains are redeemed by the holder.** The first requires no holder action and therefore no holder is at risk of the forfeiture term; the second requires an act within the redemption window and therefore every non-acting holder is. PLANNED

Snapshot-and-carry (ETH, BSC — surviving). Because the surviving contracts are *relaunched* rather than kept (Section 6), holders of the legacy token are moved to the new contract by the issuer rather than by themselves. The holder set is reconstructed off-chain at a published height h_{snap} by `Transfer`-log replay — the same computation the team's own crawler already performs on four of the five EVM chains [holder-snapshot/eth.js:63–88] — and written into the new contract by Algorithm 9, which consumes the same Λ_c migration allowance that `mintMigrated` does, so bracket (1) of Definition 22.6 and the conservation argument are unchanged. What changes is only who acts. The exercise is coordinated in advance with exchange partners and ecosystem integrators [intent: design intake], which is not a courtesy: a custodian's users hold no address of their own on the legacy contract, so a credit to the custodian's address is the only carry available for them and the custodian's internal ledger is what makes it reach a user.

Algorithm 9: Snapshot carry onto the relaunched contract for a surviving chain c. PLANNED

Input: The published snapshot height h_{snap}; the published issuer-held address set $\mathcal{I}_c \subseteq \mathcal{A}_c$, whose balances are excluded from Λ_c (Table 13) [intent: design intake]; the balance map $\mathsf{bal} : \mathcal{A}_c \to \mathbb{Z}_{>0}$ reconstructed by `Transfer`-log replay of the legacy contract at h_{snap}, over the address set $\mathcal{A}_c$; the migration allowance Λ_c; the new contract with $M_c = 0$, public mint paused.

Output: Each $a \in \mathcal{A}_c \setminus \mathcal{I}_c$ credited exactly $\mathsf{bal}(a)$, then $\mathsf{snapshotComplete}_c \leftarrow \mathsf{true}$.

Invariant: $\mathsf{cumMig}_c = \sum_{a\,\text{credited}} \mathsf{bal}(a) \le \Lambda_c$ at every point, and no a is credited twice — enforced by a per-address $\mathsf{carried}_c \subseteq \mathcal{A}_c$ set in contract storage, not by the caller's bookkeeping. Transfers and public mint stay disabled until $\mathsf{snapshotComplete}_c$, so no partially-carried supply is ever tradeable.

Failure mode: A batch reverts atomically and is resubmitted; a batch naming an already-carried address reverts rather than double-crediting. If the replay is wrong, the error is public and diffable against the legacy contract at h_{snap} by anyone, which is the only reason this path is acceptable at all — it is an issuer-executed mint whose correctness is checkable by third parties.

publish h_{snap}, $\mathcal{I}_c$, the reconstructed bal and its Merkle root before executing;
foreach *batch* $\mathcal{K} \subseteq \mathcal{A}_c \setminus (\mathsf{carried}_c \cup \mathcal{I}_c)$ **do**
  if $\mathsf{cumMig}_c + \sum_{a\in\mathcal{K}} \mathsf{bal}(a) > \Lambda_c$ **then return** revert (allowance);
  credit each $a \in \mathcal{K}$; $\mathsf{carried}_c \cup= \mathcal{K}$; $\mathsf{cumMig}_c \mathrel{+}= \sum_{a\in\mathcal{K}} \mathsf{bal}(a)$;
if $\mathsf{carried}_c \neq \mathcal{A}_c \setminus \mathcal{I}_c$ **then return** abort (incomplete carry; do not enable transfers);
$\mathsf{snapshotComplete}_c \leftarrow \mathsf{true}$; enable transfers and public mint;

Remark 22.20 (The snapshot carry — two elements settled, one specified here). **(i) Contract-held balances: resolve through to the beneficial holder.** A legacy balance held by an AMM pair, a lending market or a third-party bridge cannot be carried by crediting the contract's address — the new token is a different address and the position does not follow it. The settled treatment is to read the position and credit the actual holder: "from LP, staking we can see from the LP positions who is the actual holder and that we will distribute to the actual holder like we did in the past" [intent: design intake] (PLANNED). This is a stronger commitment than any of the three options previously listed, and the team reports precedent for having done it before.

Two obligations follow and the paper states them: the resolution is *per-protocol* work — an AMM pair, a lending market and a bridge each expose beneficial ownership differently, and some expose it only at a block height, not retrospectively — and it must be published as a list before h_{snap}, because a holder cannot check a resolution they cannot see.

Decided. the contract-held positions are not measured in this paper, and that is a scoping choice rather than a limit. Balances held in liquidity-pool and staking contracts on Ethereum and BSC are public on-chain, readable by the same holder-enumeration method already used for the parallel-asset snapshots of Section 6, and the team has performed that measurement before [intent: design intake]. What matters for the migration argument is that the quantity is *obtainable*: a holder-address snapshot alone will miss these positions, and the carry of Section 22.8 must therefore resolve them per protocol before h_{snap} — work this paper specifies but does not perform, because running it belongs to the migration rather than to the survey.

(ii) Announcement: h_{snap} is announced in advance [intent: design intake] (PLANNED). This is the right choice and the paper does not hedge it: it lets holders arrange their balances, and it is the only option compatible with publishing a contract-held resolution list beforehand. It does admit the griefing vector previously noted — an adversary may fragment a balance across many addresses to inflate the carry's gas cost — and that vector has no defence in this construction. It is cheap for the adversary and merely expensive for the issuer, so the paper records it as an accepted cost rather than an open problem, and Definition 22.21 sizes the exposure.

(iii) Batch size and gas: specified below (Definition 22.21), as a recommendation.

Remark 22.21 (Batch sizing for the carry — recommendation). The carry is $|\mathcal{A}_c|$ storage writes and the issuer pays the gas. A cold `SSTORE` on an EVM chain costs 20,000 gas, plus per-call overhead and calldata; at a 30,000,000 gas block limit, a batch of 500 credits consumes roughly 35 % of one block and leaves headroom for the base transaction and calldata [inferred]. The paper recommends **batches of** 500 **recipients**, submitted sequentially with each batch's completion confirmed before the next is sent, and an idempotent `claimed` flag per recipient so a reverted or re-sent batch cannot double-credit.

The trade-off is stated exactly: larger batches are cheaper per recipient and put more value at risk in a single reverted transaction. At 500 the worst case is one reverted batch of 500 credits, which is recoverable by resubmission because the flag makes the operation idempotent; the gas is lost, the entitlements are not. Smaller batches buy little, because the per-recipient cost is dominated by the `SSTORE`, not the per-transaction overhead.

Holder-initiated redemption (Avalanche, Base, Polygon, and the non-EVM retirees). The holder must act, within the window, or lose the claim. This is the path the forfeiture term applies to, and it is the only one it applies to. *Retired EVM chains* (the disposal construction this section first specified; it will not be deployed, Definition 22.22): a `LegacySink` contract per chain holds legacy tokens forever with no withdraw function — the legacy ERC-20 has no `burn` [flux-eth-bsc/flux.sol:6–13], so a sink is the only irreversible disposal — and `deposit(v, fluxAddr)` emits an event the RELEASE quorum treats exactly as a Burn — depth D_c, contiguous nonce ranges, `FLXR1` memo — paying L1 FLUX 1:1. *Retired non-EVM chains* (legacy Solana, Kadena, Tron, Algorand, Ergo): the holder makes the legacy balance provably unspendable and binds the disposal to a `fluxAddr`; Solana has a self-custodial SPL `Burn`, and Ergo admits a `sigmaProp(false)` box.

Decided. Moot: no disposal contract is deployed on any chain (Definition 22.22), so no sink construction is required for Kadena, Tron or Algorand. Retained as a pointer because §22.9 records what the three constructions would be if a sink is ever wanted.

The decided migration terms. Four terms are settled [intent: design intake], and they replace the window and disposition parameters this subsection previously carried as open. PLANNED

1. **Staged dates.** The new Ethereum and BNB Chain contracts go live first, on readiness and before the cut [intent: 08-answers-round-2.md, round 34]; the retiring chains become one-directional at $h^{\text{mig}}_{\text{open}}$; the emission cut and PNR activation land together at PA Depletion, $h_{\text{PA}} = 3{,}787{,}502$ (2027-07-01) (Section 7, Section 27); the retiring chains shut down at $h^{\text{mig}}_{\text{close}}$.

2. **Redemption window.** From $h^{\text{mig}}_{\text{open}} = 3{,}450{,}000$ (2027-03-06) to $h^{\text{mig}}_{\text{close}} = 3{,}880{,}000$ (2027-08-02) [intent: 08-answers-round-2.md, round 36], 430,000 L1 blocks, about 149 days at the PoN target spacing, spanning the cut [inferred]. During the window, redemption back to main-chain FLUX runs at 1:1 through the **existing bridge**.

3. **The redemption route is one-directional.** From $h^{\text{mig}}_{\text{open}}$ the existing bridge permits migrating *to* the main chain, and mining entitlement stays claimable on the chain or on the main chain until shutdown; nothing can be swapped *into* a retiring chain [intent: design intake]; [intent: 08-answers-round-2.md, round 35].

4. **Unclaimed balances.** At window close, **unclaimed balances pass to the team to fund development.**

The fourth term is stated here in those words because it is a transfer from non-acting holders to the issuer, and naming it, its window and its destination is what makes it defensible. Unclaimed value must belong to someone; a stated destination is better than an unstated one; and the alternative — an open migration authority honoured forever — keeps the largest single mint authority in the design alive indefinitely, which is a cost paid by everyone rather than by the non-acting holder. What passes is not the legacy tokens themselves, which sit on foreign chains and cannot be recalled by anyone (see *The long tail* below); it is the residual main-chain backing $\Lambda - \sum_c \mathsf{cumMig}_c - V^{\text{mig}}_{\text{rel}}$, which is an explicit public number at every height and is computed by Algorithm 8 without privileged data.

The amount actually exposed to the window, which is two orders of magnitude below the measured liability. The forfeiture term applies only to the holder-initiated path, so the exposed quantity is the outstanding balance on the *retiring* chains and not the ten-chain total. Of the 223,027,781.21 FLUX measured across all ten chains [measured: scripts/13-issuer-balances.sh, 2026-09-03]:

chains	migration path	outstanding (FLUX)
Ethereum, BNB Chain	snapshot-and-carry; **no holder action**; not exposed	216,800,880.41
Base	holder-initiated redemption; **exposed to the window**	2,528,858.43
Polygon	*as above*	647,928.73
Avalanche C-Chain	*as above*	628,582.62
Ergo	*as above*	486,746.62
Algorand	*as above*	389,643.01
Solana	*as above*	373,067.20
Kadena	*as above*	798,207.66
Tron	*as above*	373,866.53
eight retiring chains	—	6,226,900.80
all ten chains	—	223,027,781.21

Table 61: Exposure to the redemption window and the unclaimed-balance term, by migration path. Surviving-chain holders are carried across by Algorithm 9 and take no action, so they are not at risk of forfeiture; only the retiring chains are. Figures are [measured: scripts/13-issuer-balances.sh, 2026-09-03] and [measured: scripts/15-nonevm-liability.sh, scripts/16-remaining-liability.sh, scripts/18-tron-final.sh, scripts/19-kadena-ledger.js, scripts/20-kadena-issuer.js, 2026-09-04], the exact measured values of Table 13, not rounded to make the point. All ten legacy assets are measured; no component of this table is an estimate.

So the amount exposed to the redemption window and the unclaimed-balance term is 6,226,900.80 FLUX **across the eight retiring chains**. That is 2.79 % of the 223,027,781.21 FLUX measured across all ten chains: the overwhelming majority of parallel-asset supply is issuer-held and never left the treasury, and the overwhelming majority of what did is on the two chains that survive and are carried across without holder action. **The fraction of that which fails to migrate within the window is unknown, and this paper does not estimate it**: there is no holder-count or dormancy measurement for any of the ten legacy assets in the evidence corpus, and a number produced without one would be a guess placed next to a real measurement. [intent: design intake]

Three consequences of routing redemption through the incumbent, one of which is a real mitigation. First, the incumbent's custody model is not retired at h_{PA} but at window close, $h^{\mathrm{mig}}_{\mathrm{close}}$, about a month later — and it is already reduced to a one-directional exit from $h^{\mathrm{mig}}_{\mathrm{open}}$, four months before the cut. For that interval the bound of Definition 22.9 applies to the new lock-and-mint path and *not* to the redemption path, whose exposure remains the balance under the 32 hot keys with no bound and no on-chain attribution (Definition 22.13). The section's dominance claim is therefore a claim about the steady state after window close, and about the new path from the time its contracts go live; it is not a claim about the window. **Second, and this materially narrows the first: the route is one-directional** [intent: design intake]. The incumbent's residual role during the window is an *exit to the main chain*, not general two-way traffic, so the swap engine's chain-pair matrix, its several disagreeing confirmation tables and its finished-before-sent state machine (Section 22.1) are not all in use for it. The exposure that remains is real — the same single hot key per chain authorises the outbound main-chain payment — but it is the exposure of one direction rather than of eleven chains' worth of pairs, and the section states it that way rather than repeating the stronger earlier framing. Third, the `LegacySink`-plus-RELEASE-quorum path specified above becomes a *disposal* mechanism for retired units rather than the redemption route, which changes what has to exist at h_{PA} and what may follow later.

Remark 22.22 (`LegacySink` is not deployed — settled). `LegacySink` was a construction proposed by this paper, not an existing component: a contract with a `deposit` function and no withdraw function, supplying the irreversible disposal that the legacy ERC-20 lacks, since it has no `burn` [flux-eth-bsc/flux.sol:6–13]. Its purpose was to prevent a unit being redeemed for L1 FLUX and then sold on to a buyer who does not know the claim has already been made.

It will not be deployed [intent: design intake] (PLANNED), and the reasoning is a cost comparison the decision to route redemption through the incumbent makes decisive. Building, auditing and deploying a disposal contract on eight retiring chains — including the three non-EVM constructions that have no common shape (§22.9) — is substantial work whose only benefit is preventing resale of already-claimed units. Against it: the entire retiring exposure is 6,226,900.80 FLUX (Table 61), the contracts are being abandoned in any case, and audit A1 would have to cover a contract deployed only to be discarded.

The mitigation is therefore the public record alone, and the paper says so rather than implying more. A redemption is an on-chain event on both the legacy chain and the L1, so a prospective buyer of a legacy unit *can* check whether it has been claimed — but they must know to look, and nothing in the token prevents the transfer. The residual risk is borne by uninformed secondary buyers of retired-chain units during and after the window. That risk is real, it is not mitigated by any mechanism, and its bound is the retiring exposure above.

Reaching holders — which now means reaching the holders who must act. The snapshot path removes ETH and BSC holders from this problem entirely, so the outreach target is the retiring chains: 6,226,900.80 FLUX, now measured across all eight of them (Table 61). That is a smaller and better-defined communication problem than the one earlier drafts described,

and it should be stated as smaller rather than left overstated. The roadmap's own precedent for a parallel-asset change is announcement at least 90 d ahead [roadmap: flux-roadmap-public.md:159], which is 259,200 L1 blocks [inferred]; h_{snap} would be published at announcement, i.e. no later than $h_{\text{PA}} - 259{,}200$, together with each chain's issuer-held set $\mathcal{I}_c$ and the per-chain dead addresses to which the team transfers its own holdings, since both are needed before h_{snap} [intent: design intake]. Holders would be reached through per-asset redemption statistics published from the snapshot, relabelled explorer token pages, exchange notices, wallet banners, and the incumbent Fusion endpoints answering with migration instructions for the whole window. The single most important public act is that the team immobilises its own legacy holdings first, by transferring them to a published per-chain dead address [intent: design intake] — the bulk of five EVM contracts' 440,000,000-unit pre-mint each, 2.2×10^9 units in total [inferred]— because after it, a legacy contract's remaining `totalSupply` is public holdings only, the two disagreeing reserve-wallet lists stop mattering, and the residual that passes to the team at close is a number about third-party holders rather than a number dominated by the team's own unmoved supply.

Decided. Zero, gasless via `permit` (Table 62). Migration fee (zero recommended — holders should not pay for a consolidation they did not ask for), and whether `permit`/gasless migration is offered.

Decided. Direct notice by zero-value calldata to holder addresses above a threshold (approximately 21,000 gas per holder) is this paper's proposal, with the threshold and whether to send the notices at all unchosen; who pays for the gasless `permit` path is likewise not recorded as decided [intent: design intake].

Base, explicitly. Base carries a deployed FLUX contract today [flux-supply-tracker/index.html:346], added after the 2023 holder-snapshot tooling and absent from it. The public roadmap names Base a survivor twice [roadmap: flux-roadmap-public.md:97,131]; the settled decision recorded for this paper names ETH and BSC [intent: design intake]. Base is not BSC — it is an Ethereum L2, and they differ in finality, bridging architecture, audit surface and holder base. Three statements follow, and the paper makes all three. First, under the settled decision Base is retired — one-directional from $h^{\text{mig}}_{\text{open}}$, shut down at $h^{\text{mig}}_{\text{close}}$ — and its holders redeem through the route of the preceding paragraphs. Second, **the decision is final and no later re-addition is contemplated** [intent: design intake] — this closes a question earlier drafts left open. The engineering observation that made it worth leaving open remains true and is now only an observation: because the canonical contract is one EVM codebase, a later Base deployment would carry near-zero marginal audit surface, so retirement is a decision about consolidation and liquidity rather than an engineering necessity, and it should be read as such. Third, the supersession of a published roadmap statement must be stated explicitly in the paper and in the migration notice, because Base holders were told otherwise, and Base's measured outstanding balance is 2,528,858.42991062 FLUX [measured: scripts/13-issuer-balances.sh, 2026-09-03] — 1.13 % of the ten-chain total, but 40.6 % of the balance actually exposed to the redemption window (Table 61) [inferred]. Base is therefore not a rounding error in the migration: it is most of the holder-initiated migration problem. That is a communication liability, not an engineering one, and it is discharged by saying so.

Sequencing against audits and exchanges. The order below is a hard ordering, and nothing in it permits a public mint before the audit, before the backing is locked, or before the exchanges hold the new address. PLANNED

1. **Audit A1** (hard gate, H7): the EVM bridge contract, the migrator path and `Attester Guardian` — one codebase, published report, deployed bytecode matched to the audited commit.
2. **Measure** Λ_c at h_{snap} and R_0; publish; confirm $R_0 \geq \Lambda$.

3. **Lock** R_0 into $\mathcal{S}_R$ in a public L1 transaction, with the reserve and redeem scripts published the same day.

4. **Deploy** ETH and BSC contracts with public mint *paused*, transfers disabled pending $\mathsf{snapshotComplete}_c$, migrator *enabled*, and the signer set published as four named individuals with their per-key public halves (Definition 22.17), together with the per-chain limit ϱ_c each contract enforces.

5. **Carry the snapshot** on ETH and BSC (Algorithm 9), with exchange partners and ecosystem integrators coordinated in advance [intent: design intake]. A custodian must know before h_{snap} which of its addresses will be credited and must have decided how the credit reaches its users; contract-held balances are the open item of the note above. Transfers enable only when the carry is complete.

6. **The team immobilises its legacy reserves** on the retiring chains, by transfer to the published per-chain dead addresses [intent: design intake].

7. **Open** holder-initiated migration on the retiring assets and public lock-to-mint on ETH and BSC.

8. **Solana**: audit A2 — the bridge program *and* the SSP multisig program, the latter already roadmap-gated — then steps 4–7 for Solana.

9. **Close** the window at $h^{\text{mig}}_{\text{close}} = 3{,}880{,}000$ (2027-08-02); the migrator dies; the residual backing — against the 6,226,900.80 FLUX measured exposure of Table 61, not against the ten-chain total — passes to the team.

Steps 3–7 are staged [intent: 08-answers-round-2.md, round 34]: the Ethereum and BNB Chain carry and public lock-to-mint first, on readiness and before h_{PA}; holder-initiated migration on the retiring assets from $h^{\text{mig}}_{\text{open}}$; the close at $h^{\text{mig}}_{\text{close}}$. Steps 1 and 2 must complete before the first of these, and step 2 is not complete: Λ_c is measured for all ten legacy assets, and R_0 is not yet established. **Whether public mint opens under the named quorum or waits for a bonded attester feed is settled by the same decision:** the launch configuration is 3-of-4 with autonomy to follow, and the fallback on record is to ship 3-of-4 and take a second fork [intent: design intake], so public mint opens under named signers and the attestation link of Section 22.5 arrives afterwards. The interval during which the pause authority is four named individuals rather than a bonded set is therefore not an accident of sequencing; it is the design until the second fork, and Definition 22.10 is what bounds it.

Decided. Exchange swaps sequence *before* the public window (Table 62). Sequencing of exchange contract swaps relative to the public window — before, or simultaneous. Trade-off: stranded custodial users against coordination cost with venues the project does not control.

The long tail. Legacy contracts cannot be paused or destroyed: a plain ERC-20 has no such function [flux-eth-bsc/flux.sol:6–13]. They remain transferable and tradeable forever. After the team immobilises its reserves and the window closes, the remaining legacy supply is *declared* worthless but is still a token someone may sell to someone who has not read the notice. The adversary's gain in that case is bounded by the buyer's capital and touches no bridge state; the mitigation is the public record — immobilised balances, the closed window, and per-chain statistics — which is what an explorer or an aggregator uses to mark the legacy token as migrated.

22.9 Recommended values for the open parameters

The construction above leaves nineteen parameters open. Seventeen are recommended here; two are left open because they are genuinely undetermined rather than merely unchosen, and the

section says which and why. Each was proposed by this paper and **confirmed by the author** [intent: design intake], the T_{react} target in round 18.

Table 62: Bridge parameters. Proposed by this paper and **confirmed by the author** [intent: design intake], the T_{react} target in round 18; the two marked *open* remain so. PLANNED, [inferred] for the derivations except where cited.

Parameter	**Recommended**	Reason
D_{L1}	40 **blocks**	exactly `MAX_REORG_LENGTH` [fluxd/src/main.h:74]; more adds latency and buys nothing
$D_{\text{ETH}}, D_{\text{BSC}}, D_{\text{SOL}}$	**each chain's finality tag**	a confirmation *count* is the BNB Bridge error class
$v_{\min}$	10 FLUX	exceeds dust + φ_{rel} with margin; keeps a batch entry worth its gas share
φ_{mint}	**0**	do not tax entry
φ_{rel}	**actual gas share of the batch**	cost recovery, not revenue
fee recipient	**team operations**	not Φ: bridge fees must not pay node rewards
T_{rel} release cadence	4 h	6 batches/day; bounded exit latency, modest signer load
R_{hot} (window)	500,000 FLUX	≈ 12 days at the measured mean; covers front-loading
cold→hot top-up	**when hot** < 200,000 FLUX	≈ 5 days' cover at trigger
Δ_g pause guard	48 h	reuses the agreed upgrade timelock; one operational constant
MintAuth transport	**Flux feed + mandatory IPFS mirror**	IPFS is existing project precedent [fluxd/zcutil/fetch-params.sh:17]
T_{att}	120 **blocks** (1 h)	attestation cost is per-post; hourly is ample for a daily-scale limit
T_{silence}	6 h	6 missed attestations before fail-closed
fail-closed on silence	**yes**	a bridge that mints while blind is the failure being designed against
T_{react} (launch operating target)	24 h	the value at which Table 60 and Definition 22.13 are evaluated; a published operational target, not a contract constant [intent: design intake]
migration fee	**0**, gasless via `permit`	holders did not ask for this consolidation
exchange sequencing	**before the public window**	venues need lead time; stranded custodial users are the larger harm
$(n_{\text{att}}, k_{\text{att}}), \beta_{\text{att}}$	*open*	determined by the slashing construction below
slashing construction	*open*	Research-class

Where the reserve number comes from. R_{hot} is the one parameter here that can be derived rather than judged, and Table 13 now supplies the operand. The measured redeeming set is 6,226,900.80 FLUX across the eight retiring chains, and the migration window is about 149 days, so the *mean* redemption rate over the window is 6,226,900.80/149 ≈ 41,791 FLUX per day [inferred]. Redemption will not be uniform — announcement-driven flows are front-loaded, and the 40.6 % of the set sitting on Base concentrates it further — so a hot reserve sized at the mean would be drained by the first week. 500,000 FLUX is roughly twelve days at the mean, which absorbs a first-week surge of several times the average without a cold-storage round trip, and the 200,000 FLUX top-up trigger leaves about five days of cover at the point the top-up is initiated. Both figures should be re-derived once real redemption data exists; they are sized from the only measurement available.

Finality, not confirmations. D_{ETH}, D_{BSC} and D_{SOL} should be expressed as each chain's own finality predicate — Ethereum's `finalized` block tag, BNB Chain's fast-finality tag, Solana's `finalized` commitment — and never as a block count. A count is a guess about reorg depth that is wrong precisely when it matters, which is the error class the BNB Bridge incident belongs to. Using the finality gadget makes the bridge's safety argument inherit each chain's own, which is the strongest available statement and also the only one that stays correct when those chains change their parameters.

Burn-sink constructions — superseded, and recorded for the record. An earlier draft of this section specified disposal constructions for the three non-EVM retiring chains: a Pact module guard that unconditionally fails on Kadena, an unconditionally-rejecting `LogicSig` account on Algorand, and, on Tron, the token contract's own `burn` entry point if one exists. That work is superseded by the decision not to deploy `LegacySink` at all (Definition 22.22): with redemption running through the incumbent for the window, no disposal contract is deployed on any chain, EVM or otherwise, and the residual risk is carried by the public record as that remark states. The constructions are recorded here because they remain the right answers if a sink is ever wanted, and because the Tron case illustrates why the three chains were not equivalent: Kadena and Algorand both admit a spending condition that is provably unsatisfiable from the chain's own rules, whereas a Tron sink would have rested either on a `burn` entry point whose presence this paper could not establish — the deployed contract's ABI is not returned by Tronscan's public contract endpoint — or on the weaker convention that no key exists for the zero address.

Custody standard for the four keys. The recommendation is a named class rather than a principle: keys generated and held in FIPS 140-2 Level 3 hardware or equivalent, one signer per physical location, no signing key ever resident on a host that runs a relayer, an RPC endpoint or the incumbent bridge service, a key-generation ceremony with published witness attestations, and a documented rotation procedure exercised at least once before the bridge holds material value. The last clause matters more than it looks: a rotation procedure that has never been executed is a plan, not a control, and the first time it is attempted should not be during an incident.

What is left open, and why. $(n_{\text{att}}, k_{\text{att}})$ and β_{att} have no recommended values. Nothing is ever slashed (Definition 22.16), so (7) is withdrawn: a bond that cannot be slashed is a deposit, and sizing it is a decision about optics rather than security. What remains open is the size and threshold of a named, accountability-based attester set, which nobody has yet chosen. Assigning a number to β_{att} before that is settled would be exactly the kind of plausible-looking fill this paper exists to avoid.

22.10 What cannot be achieved

Open Problem 22.23 (Unconditional redemption liveness against a refusing quorum). Every holder can always execute `burn`: no role can prevent it, and the resulting claim is on-chain, public and provable (H5). But the corresponding release requires k_R signatures on an L1 transaction, and Flux's inherited Bitcoin script can enforce nothing beyond "k_R of these n_R keys signed" — it cannot enforce a rate limit, a cap, or a proof of burn. Consequently, if $n_R - k_R + 1$ or more release signers refuse, for any reason, the holder has a provable, public, outstanding claim and *no cryptographic recourse*. At the decided $(n_R, k_R) = (4, 3)$ that number is **two**: any two of the four co-founders, by refusal, absence, incapacity or legal order, stop every redemption to the L1 indefinitely. One refuser does not, so D1 survives against a single uncooperative party — but by a margin of one key, and no longer for the reason the earlier form of this argument gave. That reason was that no single organisation holds more than $n_R - k_R$ keys and so none can block alone; it is not available now, because the four co-founders hold all four (Definition 22.17), and a correlated unavailability of two of them is a single event rather than two independent ones. **D1 is therefore satisfied against one uncooperative party, with no margin beyond that, and cannot be satisfied against a refusing quorum without an L1 consensus change** — a covenant, or a bridge-vault transaction type verified by `fluxd`. No such primitive exists today. The same limitation holds for every lock-and-mint bridge whose home chain cannot verify the foreign chain, including the incumbent, where it holds with $k = 1$.

Two related items are open rather than solved and are recorded here so that later sections can reference them: Definition 22.16 (slashing on the L1), and the light-client construction of Section 22.6, which is closed by the header format rather than by cost. All three are Research-class in the sense of Section 8: each requires a consensus change, not an implementation.

22.11 Comparison with real incidents and with designs that held

Each row of Table 63 is a design that the language of a bridge roadmap could be mistaken for, together with what actually broke. The thread through the failures is not "too centralized". It is **unbounded consequence**: in every failed row, once the check was defeated — by a bug, by a key, or by an upgrade — the contract paid out whatever it was told to pay. The construction's claim is correspondingly narrower than trustlessness, and defensible for that reason: whatever is defeated, the contract issues at most ϱ_c per window and at most $\overline{M}_c$ ever, visibly, and to signers whose identities are recoverable from the transaction. With the decided parameters that "whatever is defeated" clause carries a number — 20,000,000 FLUX per 24 h of reaction time across the launch set of two chains (Definition 22.9) — and the third column below should be read as claiming that and nothing more. In particular, the launch configuration does *not* differ from the Ronin and Multichain rows in signer distribution; it differs in what the contract permits after the distribution has failed. **That distinction survives only under Definition 22.11**: an upgradable contract without a timelock permits whatever its next version permits, which is the Nomad row's mechanism arriving by intent rather than by mistake.

system	what failed	how Definition 22.4 differs
Poly Network Aug 2021	The cross-chain manager contract could call the data contract holding the trusted keeper set, so a crafted cross-chain message replaced the keepers with the attacker's key. *The bridge's own message path could alter its trust root.*	No message path can touch Q_M, Q_A or $\mathcal{G}$; only ADMIN, under Δ_A, on-chain. Mint authorisations are domain-separated by chain identifier and contract address.
Wormhole Feb 2022	Solana signature verification accepted a spoofed sysvar because a deprecated instruction-loading function did not check the account; guardian signatures were never actually verified. *Implementation bug, not key compromise.*	ϱ_c and $\overline{M}_c$ bound what *any* verification bug can issue before a pause (Definition 22.8); the Solana Ed25519 introspection path is a named audit focus.
Ronin Mar 2022	5-of-9 validators, four run by one company plus a DAO key delegated to that same company and never revoked; one organisation was socially engineered; undetected for six days. *Threshold nominal, concentration real, no monitoring.*	**Not by distribution.** At launch all four keys sit with one organisation's founders (Definition 22.17), which is the Ronin concentration and this paper says so. This row is also the answer to why Definition 22.9 models a 3-of-4 compromise at all: a 5-of-9 quorum was taken whole. What differs: every signature is attributable to a named key; the pause is 1-of-4, so a 3-key compromise does not control it (Definition 22.10); and issuance after a full compromise is bounded to 20,000,000 FLUX per 24 h of detection latency across two chains (Definition 22.9) rather than to the whole reserve. Ronin's six undetected days exceed Δ_A: the rate limit would have held the loss to $2 \times 20{,}000{,}000$ FLUX for the 48 h until a queued upgrade landed, and the contract-enforced ceiling thereafter is $\sum_c \overline{M}_c$ (Definition 22.15), not the rate limit's number.

(Table 63 continued)

system	what failed	how Definition 22.4 differs
Harmony Horizon Jun 2022	2-of-5 multisig; two keys compromised. *Threshold too low for the value at risk.*	$(n, k) = (4, 3)$ decided (Table 59): a higher threshold ratio than Harmony's on a smaller set, so two independent key compromises are survivable and three are not. The loss is bounded by Definition 22.9 or R_{hot} regardless.
Nomad Aug 2022	An upgrade initialised the trusted root to zero, so every unproven message verified and anyone could copy the exploit. *An in-place upgrade regressed a security invariant.*	**This row is answered only by the timelock.** The construction specifies no upgradeability (D3), with replacement only by holder-initiated migration to a separately audited contract; the decided configuration keeps an upgrade path under the same 3-of-4 [intent: design intake]. Under D3 the Nomad mechanism is unavailable; under the decision it is available to three keys, and the 48 h timelock of Definition 22.12 is what makes it slow and public instead.
BNB Bridge Oct 2022	A flaw in the IAVL Merkle-proof verification used by the light client let a forged proof pass; validators halted the chain to stop it. *Even a light client is only as good as its proof verifier, and there was no rate limit.*	ϱ_c per window makes an equivalent forgery worth ϱ_c rather than the cap, and the pause is a contract action rather than a chain halt.
Multichain Jul 2023	MPC "nodes" whose keys and servers were under one person's sole control; his arrest left the bridge inoperable, then drained. *A threshold with one holder is one key.*	Per-signer keys held by four separately named individuals, not shares of one aggregate key, so the contract sees which three signed and a single holder cannot produce a valid authorisation. The concentration risk is not removed — see the Ronin row — and the incumbent's 32-key model is the same failure class, which this paper says rather than implies.
Circle CCTP [18] held	Issuer-attested burn-and-mint; no liquidity pool to steal; nonce-bound messages. Trust is the issuer, who is already trusted for the whole supply.	Adopted as the model. Adds explicit k-of-n attribution and contract-level bounds, because Flux's attesters are *not* already trusted for the whole supply.
Optimism / Arbitrum held	L1 verifies L2; challenge window; guardian pause; security-council timelock.	Cross-verification is unavailable (Section 22.6); the guardian, timelock and no-silent-upgrade pattern is adopted.
IBC [32] held	Light clients in both directions on fast-finality chains.	Not applicable without a node-set commitment in the PoN header — the constraint of Section 22.6, not a preference.

Table 63: Bridge incidents and designs that held, against Definition 22.4. Loss figures are omitted deliberately: they are widely reported, they vary by source and by the price used, and the argument does not depend on them. What the argument depends on is the failure mechanism in the middle column. [inferred], from published incident post-mortems; see Section 22.6 for the design references.

Feasibility, in one paragraph. The canonical EVM contract, the migrator, and the off-chain attestation feed with its on-chain guardian are *engineering*: standard Solidity and standard verification, where the novelty is discipline rather than cryptography. The L1 reserve as a published P2SH set, the memo-carried ledgers, and the anyone-verifiable ς of Algorithm 8 are *extensions* of primitives the chain already has and the project already uses. Per-node,

L1-registered attester keys are engineering, but they replace a fleet-shared derivation and their registration transaction is new design. Two items are Research: Definition 22.16, and a Flux light client on an EVM chain, which is gated by the header format rather than by cost. One item — Definition 22.23 — is not achievable at all without a consensus change. A third is not classified here at all: the autonomous replacement for the 3-of-4 quorum has no construction in the corpus, so it is neither engineering nor Research until R1–R5 of Section 22.7 have a candidate answer. Section 8 carries the header question and Section 14 carries the attester-key question; this section carries the rest.

23 Governance: decentralization, operator protection, and the content-reporting quorum

Every network that executes other people's software eventually has to answer a question that storage networks can defer and payment networks can ignore: who may stop a workload from running, and by what procedure. Flux answers it twice. There is an answer that is deployed — a small set of JSON documents in a GitHub repository that every node fetches on a timer and enforces without argument — and there is an answer that is designed but not built: a collateral-weighted vote, conducted entirely in chain transactions, in which a node operator reports an application and an absolute fraction of total voting weight decides whether it is banned. This section states the deployed answer as a mechanism rather than as an implementation detail, specifies the designed one at the depth a reimplementer would need, proves what can be proved about it, and then reports the measurement that decides its worth.

The single claim the section carries is this: **collateral weighting makes the quorum Sybil-resistant by construction and, at the distribution measured on 2026-09-03, not censorship-resistant.** Four owner identities hold enough collateral-weighted voting power to evict any application by themselves — three, once addresses linked by a shared node key or a shared collateral funding transaction are merged. That is a serious finding about the proposal. It is also, and simultaneously, an improvement on what is deployed, where the corresponding number is one, the decision carries no signature, and no node can verify that it was ever made. Both statements are true and the section makes both.

Two qualifications belong here rather than in a footnote, because without them the finding reads as an accident and it is not one. First, **the identities behind that concentration are largely known**: the largest is a pooled-staking operator running nodes on behalf of many individual depositors, so its weight aggregates people who did not individually choose how it votes, and the next largest are the Foundation and team members whose holdings are publicly disclosed [intent: design intake]. That is a different risk from four anonymous whales and should not be described as though it were the same one — though it is not therefore benign, since pooled weight is still weight and disclosure is not dispersion.

Second, **the centralised moderation deployed today is a deliberate capability, not an oversight.** A network that serves applications under its own domains needs some party able to remove a phishing site impersonating `runonflux.io`, or content that exposes operators to legal jeopardy in their own jurisdictions, and today that party is the team acting through FDM and the policy channel of Section 12. The specified quorum exists precisely to move that authority from a team to the network, and the concentration measured above is the distance still to travel rather than a property of the destination [intent: design intake]. What the measurement establishes is that the move does not complete on activation: at the present distribution the quorum relocates the capability without yet dispersing it.

A second property of the specified design is worth stating in the same breath, because it is what makes the first one auditable: **nothing is tallied off-chain.** The report is a transaction, each vote is a transaction, the count is a function `fluxd` evaluates from those transactions, and the ban is a memo in a coinbase. Every step is a chain fact that any node can recompute from

block data alone, with no tallier, no publisher key and no API in the trust path.

Remark 23.1 (Local notation). This section uses, beyond `notation.tex`: $W^\star$ for the trailing-maximum total weight, $\kappa(\theta)$ for the size of the smallest colluding set that reaches threshold θ, Y_r for a report's tally, own_j for a node's *owner* (collateral) key and pk_j for its VPS message-signing key — these are different keys and only the first is an identity here (Definition 23.6) — X_i for an identity's proposal capacity, s_i for a node's share of the PNR pool, m and T for the identity-chamber count and tenure, and $\Delta_{\text{vote}}, \Delta_{\text{exec}}, \Delta_W$ for windows measured in blocks. All are proposed additions to the notation file; none overloads an existing symbol. Note in particular that κ denotes a *collusion-set size* throughout and never a key. The report bond β_{rep} declared in `notation.tex` is *not* used: the specified design has no bond, and prices reporting by collateral instead (Definition 23.11).

23.1 Governance as it operates today

Flux already has content governance. It is not a quorum, it is not on-chain, and it is not described in the public roadmap. It is a publisher.

Definition 23.2 (Deployed network-policy channel, SHIPPED)**.** The channel is the tuple $(\mathcal{D}, \mathcal{O}_{\text{pub}}, \text{fetch}, \text{enforce})$ where:

- $\mathcal{D}$ is the document set {`blockedrepositories.json`, `vettedrepositories.json`, `enterprisenodes.json`, `tamperingblockednodes.json`, `iplocation.bin.gz`}, served as unsigned documents over plain HTTPS from a GitHub repository [fluxos-network-policy/README.md:1–9];
- $\mathcal{O}_{\text{pub}}$ is the publisher set: the owners of the `RunOnFlux` GitHub organisation, of whom the repository states "everyone who has it is an admin", with branch protection forbidding direct pushes to `main` but *deliberately* no second-reviewer requirement — the stated reasoning being that a rule requiring a second person "gets bypassed at 3am" [fluxos-network-policy/.github/CODEOWNERS:1–15] [fluxos-network-policy/README.md:297–311];
- fetch is a per-node scheduled retrieval — 6 h for `blockedrepositories.json` and `enterprisenodes.json`, 12 h for `tamperingblockednodes.json` [fluxos-network-policy/README.md:30–32], and — on the SPEC v9 branch only — a daily conditional fetch for the geolocation table [MorningLightMountain713/flux@feat/fluxos-v9:ZelBack/config/default.js:779–784] — followed by shape validation (`isStringArray`, `isNodeOwnerMap`; row/country/org/region floors for the geolocation artifact) [fluxos-network-policy/scripts/validate.js:22–60];
- enforce is the per-node compliance sweep: FluxOS fetches `helpers/blockedrepositories.json` from the raw-content URL of `RunOnFlux/flux`, caches it [flux/ZelBack/src/services/appSecurity/imageManager.js:179–195], and every `imageComplianceIntervalMs` = 1 h [flux/ZelBack/config/default.js:319] [flux/ZelBack/src/services/serviceManager.js:529–531] removes every locally installed application whose image, owner address or specification hash matches, spacing removals 3 min apart [flux/ZelBack/src/services/appSecurity/imageManager.js:644–676]. A parallel `vettedrepositories.json` exempts listed applications from blocked ports and blocked repositories [flux/ZelBack/src/services/appSecurity/imageManager.js:202–236,395–410].

The node-local state machine is fail-frozen. A fetch that fails or fails shape validation leaves the previous copy in force indefinitely and across restarts; an empty document (`[]`) is honoured as "nothing is blocked" and is explicitly distinguished from an unreachable one; a node that has never fetched successfully declines to answer rather than guessing [fluxos-network-policy/README.md:33–44].

Algorithm 10: Deployed policy fetch-and-enforce loop, per node (SHIPPED)

Input: document set $\mathcal{D}$; publisher URL; per-document fetch interval; local application set A_{loc}

Output: A_{loc} with policy-matching applications removed

```
/* Invariant:  a node never enforces a document it has not itself
   shape-validated, and never downgrades to "nothing blocked" because of
   a transport failure.                                              */
/* Failure mode:  fetch failure freezes the last valid copy indefinitely
   (stale enforcement, and stale geolocation for placement);
   never-fetched nodes decline to answer.  There is no signature to
   check, so a MITM or a compromised publisher account is
   indistinguishable from a legitimate update.                       */
```

foreach $d \in \mathcal{D}$ *due for refresh* **do**
  $x \leftarrow$ `HTTPS GET`(d);
  if *transport error* **then** keep cached d; **continue**;
  if $\neg$ shapeValid(x) **then** reject x; keep cached d; **continue**;
  cache x as the current copy of d;
foreach *application* $a \in A_{\text{loc}}$ *(hourly sweep)* **do**
  if *image, owner address or specification hash of* a *matches* `blockedrepositories` ***and*** a *is not in* `vettedrepositories` **then**
    remove a locally; wait 3 min before considering the next removal;

A change to any of these documents takes effect on every node at its next scheduled fetch — no staged rollout, no canary, no version negotiation [fluxos-network-policy/README.md:3–5]. What such a change can do is broad and precise: block any container image or namespace, any application owner address, or any application hash network-wide; grant or revoke which owner addresses may install on which enterprise node; mark a node's collateral transaction hash as DOSed for tampering; exempt an owner, hash or repository from user-level blocks; and, through the monthly baseline pull request, change which organisations and regions are treated as residential rather than hosting for placement and enforcement purposes [fluxos-network-policy/README.md:99–196,320–326]. The documents are unsigned; the repository states the position plainly — "their authenticity rests on GitHub and on who can merge to this repo" — and records signed documents with a rollback-resistant sequence number as the intended next step rather than a finished one [fluxos-network-policy/README.md:320–326]. Signed documents with a sequence number were published on 2026-09-11, and FluxOS verifies them from v8.19.0 (2026-09-23).

A second lever sits in the reachability plane. The Foundation-operated FDM fleet carries a configuration-level wildcard blacklist and whitelist by application name, plus an `ownersApps` restriction that can confine an entire FDM instance to one owner's applications [flux-domain-manager/config/appsConfig.js:2–6] [flux-domain-manager/src/services/application/subset.js:25–47], and a hardcoded `forbidden-backend` hostname list shipped to every FDM instance in the HAProxy template [flux-domain-manager/src/services/haproxyTemplate.js:499–512]. This lever is exercised: the commit history contains "forbid domains" and three "ban phishing website" changes. A third sits at consensus level: a hardcoded 2-of-4 key set can produce blocks outside normal consensus, with no time gating and no stall detection, despite a source comment saying gating was planned [fluxd/src/emergencyblock.cpp:24–30,183–190] [fluxd/src/emergencyblock.h:40–44]; the deployed threshold is `nEmergencyMinSignatures` $= 2$ against a four-key set [fluxd/src/chainparams.cpp:253,522], and the team does not intend to change that threshold. That key set is not a content lever, but it is a governance lever, and a section on who can do what to this network must list it. Under the mechanism specified below it acquires a second significance, because that mechanism's enforcement rides in the coinbase (Section 23.3.3).

Assumption 23.3 (Trust boundaries invoked here)**.** As fixed in the system-model section: node operators are mutually distrusting and independently operated; the FluxOS message layer is a gossip mesh with per-peer bad-message accounting (five malformed or badly signed messages in ten minutes closes the connection) [flux/ZelBack/src/services/fluxCommunication.js:597–618]; `fluxd`'s deterministic node set is consensus state; and the policy channel, the FDM fleet and the application-discovery API are Flux-Foundation-operated. Nothing in Definition 23.2 is consensus state, and no transition of it is verifiable by a node against the chain.

Property 23.4 (The collusion measure of the deployed mechanism, SHIPPED)**.** For the channel of Definition 23.2: the number of parties whose agreement is required to remove an arbitrary application from every node is $\kappa_{\text{deployed}} = 1$; the authentication on the decision is $\varnothing$ (TLS to a code-hosting provider, no document signature); the record available to a node for verifying that the decision was authorised is $\varnothing$; and the latency from decision to fleet-wide effect is bounded by the fetch interval plus the sweep interval, i.e. within 7 h for the repository blocklist and 13 h for the tampering list.

Proof sketch. A single organisation owner opens and merges a pull request adding an image, owner address or application hash to `blockedrepositories.json`; CI checks shape only and no second reviewer is required [fluxos-network-policy/README.md:297–308]. Every node fetches on its own schedule and enforces the new document without any authentication step, because none exists [fluxos-network-policy/README.md:320–326]. The bound on latency follows from the 6 h/12 h fetch intervals and the 1 h sweep. That the record is not node-verifiable follows from the absence of a signature: a node observing the document cannot distinguish a legitimate merge from a substituted document. □

This is the baseline against which the quorum must be judged, and it is the strongest argument for building the quorum at all. The proposed mechanism, with every defect catalogued below, would require four parties rather than one — three under the strongest linkage computable from public data — would leave a permanent chain record naming exactly which keys produced the eviction, and would freeze its electorate at a chain height. It would be a large decentralization improvement over what is deployed. It would still not be censorship-resistant. The section says both, in that order, because presenting the quorum as a solution to a problem the network does not have would be the more comfortable and the less honest framing.

23.2 The roadmap principle, and the tension the quorum creates

The public roadmap does not contain a content-eviction vote. What it contains is a principle pointing the other way:

> "The principle: defence as protection from harm, not veto over content. **Behaviour, not content.**" [roadmap: flux-roadmap-public.md:94]

Under the heading of operator protection the roadmap lists an application oracle, resource guardrails, an optional egress profile, and an evidence pack — operator-side defences that let a node refuse or contain a workload. It does not list a network vote that removes an application from every node. The mechanism specified below is therefore not an elaboration of a published commitment; it is a design-intake item [intent: design intake] that must be reconciled with a published principle, and this paper does the reconciling in the open rather than letting a reader find the gap.

The reconciliation has one hinge, and it is an open parameter: the category taxonomy. If the reportable categories are behavioural — harm to third parties, harm to operators, harm to the network's own infrastructure — then the quorum is an enforcement path for the roadmap's own principle and the two documents agree. If the categories are about what an application says or serves, the quorum is a content veto and the two documents contradict each other. Nothing

in the mechanism forces either reading; the taxonomy does, and its scope has been ruled on (Definition 23.5).

Remark 23.5 (The scope is behavioural harm, and the deciding party is the operators). This paper asked directly whether the report taxonomy should be restricted to behavioural categories — harm to third parties, to operators, to the network's own infrastructure — so as to reconcile the quorum with the roadmap's published "behaviour, not content" principle. **The answer is that the scope is behavioural harm** [intent: design intake] (PLANNED). The roadmap's principle stands, and the quorum is an enforcement path for it rather than a departure from it. An earlier draft of this remark recorded the opposite and is superseded.

Two consequences, and the second is the one that decides how the whole section should be read.

1. **The taxonomy carries the limit, so concentration is bounded rather than unbounded.** A behavioural taxonomy limits the damage a captured quorum can do, because the categories themselves resist misuse: a report has to allege harm of a kind that other operators can recognise and that the reported application's own behaviour either exhibits or does not. The measurement in Section 23.5 — four owner identities reaching the 25 % threshold — remains the mechanism's most important number, but it is now a bound on a bounded power rather than on an unbounded one, and Section 23.12's comparison with IPFS, Filecoin and Arweave holds on intent as well as on the facts.

2. **The deciding party is the node operators, and that is not a power the protocol grants them.** [intent: design intake] The people running the machines can already refuse to run a workload, in exactly the sense that miners in a proof-of-work system can already refuse to include a transaction; nothing a specification says removes that. What the quorum changes is not *whether* operators can collectively remove an application but *how*: today the decision is one pull request by a single organisation owner that every node obeys without checking (Definition 23.4); under the quorum it is a signed, weighted, on-chain vote that names every participating key and is verifiable by anyone (Definition 23.14). Read that way the mechanism is a transfer of an existing capability from a publisher to the operators, made legible — and the honest expectation, which the paper states as an expectation and not as a guarantee, is that operators do not spend collateral-weighted votes on applications that are not harming anyone.

The residual risk is not removed by either point and is not softened here: a collateral-weighted electorate concentrated enough that four identities clear the threshold can, in principle, agree to call something harmful that is not. Section 23.6 quantifies exactly that and does not argue it away. What has changed is the size of the prize: under a behavioural taxonomy such a coalition is misusing a mechanism against its stated categories, in public, with its keys on the record — rather than exercising a power the paper had granted it in advance.

What remains open is the enumeration: which categories exist, so that a report is a well-formed object with a fixed domain rather than free text. That is a specification task, not a policy question, and Section 23.3 states the requirement it must satisfy.

There is a second, quieter tension. The mechanism's enforcement is removal, and its financial consequence for the deployer is forfeiture of an unused prepaid balance whose floor is 0.01 FLUX (Section 23.3.3). The purpose is to stop harm, not to fine, and under either scope that is the right design. It also means the mechanism has essentially no deterrent value against a determined deployer, who redeploys under a new name for the price of one transaction. The paper states this as a property of the design rather than as a criticism of it, because it follows directly from the principle the design is trying to honour.

23.3 The content-reporting quorum, specified

Everything in this subsection is PLANNED. No part of it exists in any repository surveyed for this paper; the settled inputs are the design intake [intent: design intake], the on-chain construction supplied by the team [intent: design intake] and the measured node set [measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03]. Settled and not re-litigated here: collateral weighting at one unit per 1,000 FLUX; votes signed with node owner keys; $\theta = 0.25$ as an *absolute* fraction of total weight; abstention counts as "no"; enforcement is cancellation and removal with forfeiture of the tenant's prepaid balance; node collateral is never slashed.

Settled additionally, and this is the part that decides the section's verifiability requirement: the report is an on-chain transaction shaped after the deployed FluxNode start and confirm types, carrying the application hash to be banned and indexed so that any node can find it; a 24 h window (2,880 blocks at 30 s spacing) opens on its confirmation; each vote is a second special transaction; `fluxd` counts the quorum internally from those transactions; on the threshold being reached the block template changes and the PoN producer carries a ban memo in the coinbase, at most one per block; FluxOS observes the ban and purges the application; vote transactions are fee-less; an unmet report expires and the application may be re-reported immediately; and a key may hold at most as many concurrently open proposals as it operates nodes [intent: design intake].

One consequence of that shape should be stated before the construction rather than after it. This design puts moderation into *consensus*. It needs two new transaction types, a new coinbase rule and new chain-derived state in `fluxd`, which is a network upgrade; the message-layer alternative it replaces needed only a FluxOS release. The consensus route buys exact determinism and independent verifiability (Definition 23.14) and pays for them in deployment cost, and this network's own history of what a consensus change costs — the PoN activation required a temporary deep-reorg allowance and a cluster of emergency checkpoints [fluxd/src/main.cpp:8983–8988] [fluxd/src/chainparams.cpp:277–283] — is the right reference class for estimating it.

23.3.1 Electorate and weight

Definition 23.6 (Electorate, identity, weight, reference total, PLANNED)**.** Let $N(h)$ be the deterministic confirmed FluxNode set at height h, which is consensus state in `fluxd`. Each node $j \in N(h)$ carries a tier $t_j \in \mathcal{T}$, collateral $c_{t_j} \in \{1{,}000,\ 12{,}500,\ 40{,}000\}$ FLUX, and *two distinct keys*: the owner key own_j, which is the collateral public key and from which the record's `payment_address` is derived [fluxd/src/rpc/fluxnode.cpp:1512–1520], and the node key pk_j, which `fluxd` documents as "`Pubkey used for VPS signature verification`" [fluxd/src/fluxnode/fluxnode.h:154] and which signs PoN blocks and FluxOS P2P envelopes. Because the settled rule is that votes are signed with node *owner* keys, an *identity* here is an owner key and never a node key; the identity set is $I(h) = \{\mathrm{own}_j : j \in N(h)\}$, publicly proxied by `payment_address`, which is the observable the measurement of Section 23.5 uses. Define $w_\cdot : I(h) \to \mathbb{R}_{\geq 0}$ and $W : \mathbb{N} \to \mathbb{R}_{>0}$ by

$$w_i(h) \;:=\; \sum_{j \in N(h):\ \mathrm{own}_j = i} \frac{c_{t_j}}{1{,}000\,\mathrm{FLUX}}, \qquad W(h) \;:=\; \sum_{i \in I(h)} w_i(h) \;=\; \sum_{t \in \mathcal{T}} n_t(h)\,\frac{c_t}{1{,}000\,\mathrm{FLUX}}.$$

Per node the weights are $\mathsf{C} \mapsto 1$, $\mathsf{N} \mapsto 12.5$, $\mathsf{S} \mapsto 40$. The *reference total* is a trailing maximum,

$$W^{\star}(h_0) \;:=\; \max_{h \in [h_0 - \Delta_W,\ h_0]} W(h), \qquad \Delta_W \in \mathbb{N} \text{ open.}$$

All weights in a report are evaluated at the height h_0 at which the report transaction confirms and never re-evaluated: a node confirmed after h_0 would carry weight 0 in that report.

Two design points deserve their reasons. The *electorate freeze* exists because collateral maturity is 100 blocks (50 min at 30 s spacing) [fluxd/src/fluxnode/fluxnode.h:20]; without the freeze

a report could be answered by nodes created in response to it. The *trailing maximum* exists because the reporter chooses h_0: node confirmations expire after 640 blocks (5.3 h), so an incident that leaves a large share of weight temporarily unconfirmed would lower $W(h_0)$ and raise every remaining share. Taking a maximum rather than a mean is the safe direction — it can only make eviction harder.

Decided. $\Delta_W = 20{,}160$ blocks, as a trailing maximum (Table 71). Δ_W, the trailing window for $W^\star$. Domain: [0, 20,160] blocks. Trade-off: $\Delta_W = 0$ reduces to the plain total and admits the electorate-timing attack of Section 23.9; large values leave $W^\star$ stale after a genuine fleet contraction, which raises the bar to evict and is therefore safe but costs liveness.

At the measurement height 2,917,115: $n_{\mathsf{C}} = 3{,}247$, $n_{\mathsf{N}} = 1{,}615$, $n_{\mathsf{S}} = 1{,}627$, so $W = 3{,}247 + 20{,}187.5 + 65{,}080 = 88{,}514.5$ units, i.e. 88,514,500 FLUX of locked collateral, with tier shares 3.67 % / 22.81 % / 73.52 % [measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03].

23.3.2 Three transaction types

Definition 23.7 (Report transaction, PLANNED)**.** A report is a `fluxd` transaction of a new FluxNode-family type MOD_REPORT, carrying

$$r \;=\; (\mathsf{app_hash},\ \mathsf{category},\ \mathsf{evidence},\ \mathrm{own}_r), \qquad \sigma_r \;=\; \mathrm{Sign}_{\mathrm{own}_r}(H(r)),$$

where app_hash $\in \{0,1\}^{256}$ is the application hash to be banned, category is drawn from the taxonomy left open above, evidence is a content hash of supporting material, and $\mathrm{own}_r \in I(h_0)$ is the reporter's owner key. The report identifier is $H(r)$; the transaction is indexed by app_hash and by $H(r)$, so that any node can enumerate the open reports on an application from its own chain state. It carries no fee and no bond, and it is valid only if no report on the same app_hash is open at its confirmation height (Definition 23.10) and app_hash is not already in the ban set (Definition 23.15) [intent: design intake]. Its confirmation height defines h_0 and opens the window $[h_0,\ h_0 + \Delta_{\mathrm{vote}}]$ with $\Delta_{\mathrm{vote}} = 2{,}880$ blocks, 24 h at 30 s spacing.

The transaction shape is not new: `fluxd` already carries two special FluxNode types — the start transaction, signed by the collateral key [fluxd/src/fluxnode/activefluxnode.cpp:253–288], and the confirm transaction, signed by the node key and then benchmark-signed [fluxd/src/fluxnode/activefluxnode.cpp:290–332,59–68] — carried at transaction versions 5 and 6 [fluxd/src/fluxnode/activefluxnode.cpp:409–427]. The report and vote types would be a third and fourth member of that family.

Two boundaries of what a chain can check follow immediately, and both are properties of the design rather than defects in it. First, `fluxd` holds no application registry: the FluxOS global specification set is not consensus state. Consensus can therefore validate a report's *form* (type, signature, capacity) and never its *referent*; a report naming a hash that corresponds to no deployed application is perfectly valid on-chain and simply unenforceable off it. Second, $\mathcal{R}$ — reporter eligibility, an open parameter in every earlier draft of this mechanism — is settled by the anti-spam rule below rather than chosen: a key that operates no nodes has capacity 0, so $\mathcal{R} = I(h_0)$ falls out of Definition 23.11.

Decided. (app_name, owner) (Table 71). What app_hash hashes, which is also the enforcement scope. Domain: {the permanent-message hash of one specification version; a hash of the application identity (app_name, owner); + all applications of the same owner; + image}. Trade-off: if the ban key is a specification-version hash, an application updated during the 24 h window acquires a new hash and the ban, when it lands, names a version no longer deployed — the escape that keying on name-and-owner was chosen to close, and the invariant "an update during the window is neither an escape nor a second claim" holds only under the identity reading. Owner scope is evaded by registering a new address at zero cost while widening the blast radius of a mistaken ban; image scope is what the deployed blocklist does and is defeated by re-tagging. The team's specification says "the application hash" and does not disambiguate.

Definition 23.8 (Vote transaction, PLANNED)**.** A vote is a `fluxd` transaction of type MOD_VOTE carrying

$$v \;=\; \big(H(r),\; h_0,\; \mathsf{app_hash},\; i,\; \mathsf{decision}\big), \qquad \sigma_v \;=\; \mathrm{Sign}_i\big(H(v)\big),$$

with $i \in I(h_0)$ and $\mathsf{decision} \in \{\mathsf{evict}, \mathsf{keep}\}$. It carries no fee. It is valid iff its signer is in the frozen electorate $I(h_0)$, its confirmation height h_v satisfies $h_0 \le h_v \le h_0 + \Delta_{\text{vote}}$, and no earlier vote from i on $H(r)$ is already in the chain. Votes are therefore *irrevocable within a report*: the first confirmed vote from an identity is the only one that counts, and a later one is invalid rather than overriding.

Definition 23.9 (Voting delegation, PLANNED)**.** An identity $i \in I(h)$ may publish a *delegation*: a `fluxd` transaction of type MOD_DELEGATE carrying $(i,\ \mathrm{pk}_{\text{vote}},\ h_d)$ and signed by the owner key i, authorising $\mathrm{pk}_{\text{vote}}$ to sign MOD_VOTE transactions on i's behalf from height h_d onward. A vote is then valid if it is signed either by i or by the delegate current for i at h_0. A delegation is *revoked* by a later delegation from the same owner key naming a different delegate or the null key; revocation takes effect at its own confirmation height and never retroactively. Three properties are required, and all three hold by construction:

1. **Weight does not move.** w_i is a function of i's collateral (Definition 23.6) and is unchanged by delegation. A delegate holding d delegations casts d separate votes carrying d separate owners' weights; it never accumulates weight of its own, and it is counted in the identity chamber of Definition 23.42 once per delegating owner and never once for itself.

2. **κ does not move.** Definition 23.32 counts owner identities whose weight sums past θ. Delegation changes who holds a signing key, not whose weight it carries, so $\kappa(0.25) = 4$ is unchanged.

3. **The record does not degrade.** The chain names both the delegate that signed and the owner whose weight was cast, so Definition 23.14 holds verbatim with one extra lookup.

Decided. Opt-in revocable delegation is adopted [intent: design intake]. The reason is liveness and it is not a close call. Collateral keys are cold by design — that is the point of them — and a rule asking 21 tenured identities plus 25 % of all weight to produce owner-key signatures inside 24 h makes expiry the expected outcome of an honest report rather than an edge case. A mechanism whose safety argument depends on the dispersed majority being able to participate cannot price participation at "retrieve your cold collateral key today". The cost is real and is stated rather than netted: delegation concentrates signing authority in delegates by construction, which is the documented trajectory of every delegated token-vote system, and it makes delegates the party a coalition would rent or coerce. The paper's position is that this trades a certainty against a tendency — certain non-liveness against a tendency the three properties above confine to *signing* rather than *weight* — and that publishing the delegation graph at every reduction height, alongside $\kappa(\theta)$ and the top-N shares (Section 23.10), is what keeps the tendency observable.

What is signed binds the vote to one report and one electorate, so it cannot be replayed against a later report on the same application (different h_0) or against a re-registration. Two things change relative to the message-layer construction this replaces, and both are improvements. The window is measured on *confirmation* height, which is a canonical clock rather than a self-declared envelope timestamp, so two honest nodes cannot disagree about whether a vote arrived in time — at the cost that a vote broadcast near the boundary and confirmed after it is simply void, which the 24 h window makes an edge case rather than a hazard. And irrevocability is now a consequence of chain order rather than a rule imposed to make an off-chain certificate checkable. Under the settled rule keep votes are never counted.

Decided. Evict-only, with a per-identity mempool quota as the relay policy (Table 71). Whether the vote type carries a decision field at all, and the relay policy for fee-less transactions. Domains:

decision ∈ {evict-only; evict/keep with keep recorded and uncounted}; relay policy ∈ {admit up to the capacity bound; per-identity mempool quota; . . . }. Trade-off: under abstention-as-no a keep vote changes no outcome and costs block space, so recording one buys only a public record and an input to a possible early-termination rule; and because these transactions pay no fee there is no price signal to order a mempool by, so admission must be bounded by the capacity rule and by one-vote-per-identity instead. A node that drops fee-less transactions under pressure delays a vote but cannot invalidate it, since 2,880 slots remain in the window.

Definition 23.10 (Tally, decision rule, expiry, PLANNED)**.** Let $V_r(h) \subseteq I(h_0)$ be the set of identities whose valid evict vote on r is *confirmed at or before* height h, and let $Y_r : \mathbb{N} \to \mathbb{R}_{\geq 0}$, $Y_r(h) := \sum_{i \in V_r(h)} w_i(h_0)$. Every validator evaluates Y_r from block data; there is no tallier. The report is *carried* iff

$$\mathsf{carried}(r) \iff \exists\, h_c \in [h_0,\ h_0 + \Delta_{\mathrm{vote}}] : \quad Y_r(h_c) \ \geq\ \theta \cdot W^{\star}(h_0), \qquad \theta = 0.25,$$

and h_c denotes the least such height, the *crossing height*. If no such h_c exists, the report *expires* at $h_0 + \Delta_{\mathrm{vote}}$: it is removed from the open set, its reporter's capacity is released, and app_hash becomes immediately reportable again by any identity with free capacity. There is no cooldown.

Definition 23.11 (Proposal capacity, the anti-spam rule, PLANNED)**.** For $i \in I(h)$ define the capacity $X_i : \mathbb{N} \to \mathbb{N}$, $X_i(h) := |\{\, j \in N(h) : \mathrm{own}_j = i \,\}|$, the number of confirmed nodes the identity operates, counted without regard to tier. A report transaction from i is valid only if the number of i's reports that are open at h_0 — neither expired nor banned — is strictly less than $X_i(h_0)$.

This rule is the whole of the anti-spam design, and it deserves precision rather than praise. It introduces no new economic primitive: there is no bond, no custody script, no adjudicator that must learn the outcome to release funds, and therefore none of the open parameters — size, custody, destination of a forfeiture — that a bond drags in. It prices reporting in exactly the resource that already prices voting, and it inherits the quorum's Sybil resistance without a separate argument, as follows.

Proposition 23.12 (The capacity rule is Sybil-neutral, PLANNED)**.** *Let a party p control the node set $N_p \subseteq N(h)$ and partition it arbitrarily among the owner keys it controls, writing $I_p := \{\mathrm{own}_j : j \in N_p\} \subseteq I(h)$ for that key set. Then p's total concurrent proposal capacity is $\sum_{i \in I_p} X_i(h) = |N_p|$, independent of the partition. Increasing it requires confirming an additional node, hence locking at least $c_{\mathsf{C}} = 1{,}000\,\mathrm{FLUX}$.*

Proof. Each confirmed node has exactly one owner key, so the sets $\{j : \mathrm{own}_j = i\}$ for $i \in I_p$ partition N_p and their cardinalities sum to $|N_p|$. Re-keying or re-addressing moves nodes between blocks of the partition and changes no cardinality sum. A node enters $N(h)$ only through a start transaction against a matured collateral UTXO of a valid tier amount [fluxd/src/coins.cpp:630–686], the smallest of which is c_{C}. □

Corollary 23.13 (Network capacity and sustained report rate, PLANNED)**.** $\sum_{i \in I(h)} X_i(h) = |N(h)|$. *At the measured height the network admits at most* 6,489 *concurrently open reports, and — because capacity is released on expiry and a report occupies its slot for at most Δ_{vote} blocks — at most* 6,489 *reports per* 24 h *in the steady state.*

The corollary that matters for this section's argument is the unflattering one. Capacity is distributed as *node counts*, so it is concentrated in the same way and for the same reason voting power is: the largest node key operates 480 of 6,489 nodes (7.40 % of all capacity; 479 under the owner-identity model), and the Gini of node counts is 0.716 against 0.839 for weight [measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03]. The rule bounds spam by collateral, which is

right; it does not distribute the ability to report any more evenly than the collateral is distributed, and the concentration measurement of Section 23.5 already quantifies exactly how unevenly that is.

It also fixes the price of sustained nuisance, which is worth computing because the number is small. Keeping one application under a permanently renewed report costs one node — 1,000 FLUX of *locked, fully recoverable* collateral, not a fee. Keeping every one of the 1,195 applications in the deployed registry [measured: api.runonflux.io/apps/globalappsspecifications, 2026-09-03] under permanent report therefore costs 1,195,000 FLUX of locked collateral, which is 5.4 % of the 22,128,625 FLUX the eviction threshold itself requires. What that buys is not eviction — abstention is "no" — but operator attention and voter fatigue, and the second of those degrades the liveness the mechanism already has least of.

Theorem 23.14 (Public verifiability: nothing is tallied off-chain, PLANNED)**.** *There is a function* Ban *from finite chain prefixes to subsets of* $\{0,1\}^{256}$*, computable by any node from block data alone, such that under Definitions 23.7, 23.8, 23.10 and 23.15 an honest node's enforcement set at height h equals* Ban *applied to its canonical chain at $h - \Delta_{\text{exec}}$. In particular, two honest nodes holding the same canonical chain hold the same enforcement set exactly, and no message outside the chain is an input.*

Proof. Each of the four objects is a field of a block. Reports and votes are transactions, so $V_r(h)$ is the set of signers of the valid MOD_VOTE transactions in blocks $[h_0, h]$ and is a function of the prefix. $N(\cdot)$ is consensus state, so $w_\cdot(h_0)$ and $W^\star(h_0)$ are functions of the prefix; hence so is Y_r, being a finite sum of them, and so is the predicate $Y_r(h_c) \geq \theta W^\star(h_0)$. Ban memos are coinbase fields, and their validity condition (Definition 23.15) is that predicate. Ban is therefore the set of app_hash values carried by valid ban memos in the prefix, and enforcement is its image after the confirmation delay. □

This is the property the section's own verifiability requirement asked for and the property the message-layer construction could not supply. There, a node holding the chain but not the gossiped certificate body could not evaluate the predicate at all, so determinism held only "up to gossip delay" and a partitioned node's enforcement set was a function of what it had received. Here the qualifier disappears. It is also the property the deployed channel of Definition 23.2 lacks completely: there the record available to a node for verifying that a removal was authorised is $\varnothing$.

23.3.3 Enforcement

Definition 23.15 (Ban memo, PLANNED)**.** A *ban memo* is a field BAN $\|$ $H(r)$ $\|$ app_hash carried in the coinbase transaction of a PoN block at height h_b. A block is valid only if it carries **at most one** ban memo, and a ban memo is valid only if the report $H(r)$ was carried at some crossing height $h_c \leq h_b - 1$ within its own window, and no valid ban memo for $H(r)$ appears at any earlier height. The *first eligible height* for a carried report is therefore $h_c + 1$. Inclusion is **consensus-required, not discretionary**: every block at a height at which at least one carried report remains unbanned is invalid unless it carries the memo for the first such report in the settled order — least crossing height, then least $H(r)$ [intent: design intake] (Definition 23.19). A valid ban memo adds app_hash to the ban set, closes the report, and releases the reporter's capacity.

Property 23.16 (Bounded enforcement throughput and bounded coinbase, PLANNED)**.** At most one application is banned per block, hence at most 2,880 per 24 h at 30 s spacing, and the coinbase grows by at most one fixed-size memo per block regardless of how many reports are open. This is the same constraint Section 7 respects when it routes the application-revenue split through the coinbase outputs that already exist rather than adding new ones.

Property 23.17 (The ban set is monotone, PLANNED)**.** No transition specified above removes an entry from the ban set. It is non-decreasing in height, growing by at most one 32 B entry per block, and a ban is a chain fact that survives every future block. Definition 23.24 is the consequence.

Definition 23.18 (Enforcement transition, PLANNED)**.** At every height $h \geq h_b + \Delta_{\text{exec}}$, every FluxOS node that observes a valid ban memo would: (i) resolve app_hash against its global application registry; (ii) mark the application `BANNED` and issue the cancel message; (iii) remove any local instance, reusing the removal path that already exists for the blocklist sweep [flux/ZelBack/src/services/appSecurity/imageManager.js:644–676]; (iv) refuse to install, redeploy or soft-redeploy it; (v) stop broadcasting its location. The deployer's prepaid balance would not be refunded. A memo whose hash resolves to nothing is a no-op.

Note that the non-refund clause is presently vacuous under deployed code — there is no refund path at all, and an underpaid registration is silently dropped [doc: orchestration extraction] — so the clause acquires content only if the credits and renewal work gives refunds a meaning.

Property 23.19 (Ban-memo inclusion is consensus-required, PLANNED)**.** This closes what an earlier draft of this mechanism carried as the single unresolved question about the enforcement path. The team's decision is that inclusion is obligatory rather than left to producer discretion: at every height at which a carried report remains unbanned, a producer must carry the ban memo for the first such report in the settled order, and Definition 23.15 states the corresponding validity rule [intent: design intake]. What it buys is stated plainly: enforcement latency after the crossing height is bounded by one block for a report with no other carried report queued ahead of it, not by how long a reluctant or colluding set of producers is willing to withhold it. Which *one* report's memo a block must carry, when more than one crosses in the same slot, is the ordering question immediately below, and the obligation binds over that order.

Decided. Least crossing height, then least $H(r)$; the memo is carried in the coinbase (Table 71). Ordering when several reports are carried at the same height, and the carriage of the memo in the coinbase. Domains: ordering $\in$ {least $H(r)$; least h_0; least crossing height then least $H(r)$; producer's choice}; carriage $\in$ {a zero-value `OP_RETURN` output; a field in the coinbase `scriptSig`}. Trade-off: with one memo per block a queue forms whenever two reports cross in the same slot, and the settled obligatory-inclusion rule cannot be enforced without a deterministic order over that queue; on carriage, an extra coinbase *output* interacts with the two independent checks that already scan coinbase outputs — the dev-fund minimum [fluxd/src/main.cpp:1231–1249] and the tier-payout check [fluxd/src/fluxnode/fluxnode.cpp:837–875] — while a `scriptSig` field does not.

The lower bound on the penalty follows from the deployed pricing table. The forfeited amount is the unused fraction of the deployment price, floored at `minPrice` = 0.01 FLUX per month-equivalent and pro-rated to the validator's height-gated minimum term — 5,000 blocks, then 100 from height 1,630,040, then 1 from height 1,964,447 [flux/ZelBack/src/services/appRequirements/appValidator.js:852–862], while the live endpoint still reports 5,000 [measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03] (conflict C33) — against a post-PoN default of 88,000, then re-ceiled to two decimals. A minimally provisioned application would therefore forfeit 0.01 FLUX under either minimum.

Property 23.20 (Delay is fixed at one slot under the settled rule, PLANNED)**.** Under Definition 23.19, enforcement latency after a report crosses threshold does not depend on which node produces the next block: the block at the first eligible height is invalid without the memo, so $\delta = 1$ whenever no other carried report is queued ahead of it (one memo per block, Definition 23.15). This replaces what a discretionary rule would have cost. Assuming PoN slot eligibility is uniform over confirmed nodes and tier-independent [fluxd/src/pon/pon.cpp:400–433] and that successive slots'

producers are drawn independently, a discretionary rule under which a fraction f of confirmed nodes decline to include a due memo would have produced a geometric delay — expected $1/(1-f)$ slots, $\Pr[\text{delay} > k] = f^{\,k}$ — diverging only as $f \to 1$. At the measured distribution's largest identity, $f = 0.074$, that would have been 1.08 slots in expectation, about 32 s; at $f = 0.5$, two slots. The counterfactual shows the discretionary rule would not have been a practical censorship vector at the measured distribution either, which makes the settled rule the more conservative choice rather than a strictly necessary one — it removes the dependence on f entirely instead of merely making it small.

The vote-transaction censorship question is separate and is unaffected by Definition 23.19: a vote needs only one non-declining producer across the whole 2,880-slot window to be included, so under the same independence assumption a coalition holding fraction f of nodes excludes a given vote from the entire window with probability $f^{2,880}$, negligible for every $f < 1$. That the *window*, not any single slot, is what protects a vote is a different, and looser, guarantee than the exact one-slot bound the ban memo now has by consensus rule.

One constraint this subsection exports to consensus, in the same class as the constraint Section 7 carries on the fee pool. The 2-of-4 emergency key set can produce a block outside normal PoN eligibility [fluxd/src/emergencyblock.cpp:24–30,183–190], and that block carries a coinbase. **The ban-memo validity rule of Definition 23.15 must therefore bind on the emergency path as well**, or those keys acquire the ability to ban an application with no report, no votes and no threshold — a capability materially larger than the one the emergency path exists to provide.

Remark 23.21 (Emergency blocks are subject to the coinbase rules — verified). This was carried as an open verification, because a ban memo carried in the coinbase is only as sound as the weakest block-production path, and the emergency path bypasses eligibility entirely (§4). It is now checked against source and the answer is favourable: `IsEmergencyBlock` is consulted in six places in consensus code, all of them header validation — to permit a larger block signature and to skip the proof-of-node check in `CheckBlockHeader` [fluxd/src/main.cpp:5333,5339], in the equivalent header path [fluxd/src/main.cpp:2778], when reloading headers from disk [fluxd/src/txdb.cpp:557], and in the PoN header checks that skip the proof of node and validate the emergency signatures and activation gate [fluxd/src/pon/pon.cpp:205,236–250]. It appears *nowhere* in `ConnectBlock`, `ContextualCheckBlock` or `ContextualCheckTransaction` SHIPPED.

So the emergency bypass is narrow and does not reach the coinbase. An emergency block skips the proof of node and the ordinary signature-size limit, and is otherwise validated by every rule a normal block is: BIP-30 duplicate-transaction rejection [fluxd/src/main.cpp:3868–3875], the deterministic FluxNode payout check [fluxd/src/main.cpp:3877–3880], and the full contextual transaction checks. A ban memo specified as a coinbase constraint is therefore enforced on emergency blocks too, and the constraint in Section 23.3 can be stated as a claim about deployed behaviour rather than as a requirement awaiting confirmation.

23.3.4 Placement, and what fee-lessness costs

object	location	reason
report	**chain**, indexed	canonical h_0; freezes the electorate; discoverable by any node
votes	**chain**, fee-less	one transaction per identity per report; confirmation height is the canonical clock
tally Y_r	**chain-derived**, in `fluxd`	a pure function of the block sequence; recomputed, never journaled
decision	**chain-derived**	the predicate of Definition 23.10, evaluated by every validator
ban	**chain**, coinbase memo	one per block; canonical h_b, hence a canonical enforcement height
enforcement	FluxOS, at $h_b + \Delta_{\text{exec}}$	where removal already happens

Table 64: Placement of each object in the specified mechanism (PLANNED). Nothing in the decision path leaves the chain, which is the difference between this design and the message-layer construction it replaces.

An earlier draft of this mechanism rejected on-chain voting for two reasons: it would cost a fee-bearing transaction per voter, and it would require every voting key to hold spendable FLUX, which is a key-hygiene hazard the network does not currently have. **The fee-less rule retires both objections.** An operator with no spendable balance at the key that carries their collateral can still vote, so no operator is disenfranchised by their own treasury arrangements — which is not a small point for a mechanism whose safety argument already depends on the dispersed majority being able to participate at all.

Remark 23.22 (Fee-lessness is structural, not a premise — verified). The fee-less rule was carried as resting on the team's stated premise that FluxNode start and confirm transactions pay no fee. It is stronger than a premise: it is enforced by consensus. `CheckTransaction` rejects any FluxNode transaction whose `vin`, `vout`, `vJoinSplit`, `vShieldedOutput` or `vShieldedSpend` is non-empty, with reject reason `bad-txns-fluxnode-tx-not-empty` SHIPPED [fluxd/src/main.cpp:1751–1755]. A transaction with no inputs and no outputs *cannot* pay a fee, since a fee is the difference between them.

So a vote following the start-transaction shape (Table 71) inherits fee-lessness by construction rather than by policy, and cannot be made fee-bearing without changing the transaction shape itself. That is why the relay policy needs a non-fee scarcity term — the per-identity mempool quota recommended in Table 71 — and why no fee-based anti-spam mechanism is available here even in principle.

What fee-lessness costs is the price signal that would otherwise bound volume, so the bound must come from the capacity rule instead, and it is worth seeing the worst case. With every capacity slot used — 6,489 concurrently open reports by Definition 23.13 — and every one of the 869 owner identities voting on every report, the window carries $6{,}489 \times 869 = 5{,}638{,}941$ vote transactions, about 1,958 per block. At an assumed 250 B per vote transaction that is 0.49 MB per block against a 2 MB block-size limit [fluxd/src/consensus/consensus.h:23], or 1.41 GB of the 5.76 GB a day of blocks can carry — roughly a quarter of the chain's entire capacity, contributed by transactions that pay nothing [inferred]. The transaction format is not specified, so the 250 B figure is an assumption and the conclusion is a shape rather than a number: *the concurrent-proposal cap is load-bearing for block space, not merely for operator attention*, and it should be chosen with that in mind.

Recommendation: an explicit network-wide cap, well below $|N(h)|$. The per-identity capacity rule of Definition 23.11 is the right anti-Sybil bound and nothing here displaces it. But it was never chosen as a block-space bound, and as a block-space bound it is far too loose: it permits a quarter of the chain's entire throughput to be consumed by transactions that pay nothing. Nothing about the mechanism's purpose requires 6,489 simultaneous reports. A network-wide cap of order 64 concurrently open reports, with further reports queued in confirmation order rather than rejected, bounds the worst case to under 5 kB per block — three orders of magnitude below the block limit — costs nothing in the honest case, since the deployed registry holds 1,195 applications and no plausible incident implicates more than a handful at once, and composes with Definition 23.11 rather than replacing it: an identity still cannot open more reports than it operates nodes, and now the network cannot be made to carry more than it can afford. The cap belongs with the settled parameters rather than in the open list, and Table 71 should carry it.

23.3.5 State machine and node-side procedure

from	guard	to	effect
SUBMITTED	report tx confirms at h_0; σ_r verifies; $\text{own}_r \in I(h_0)$; open reports of $\text{own}_r < X_{\text{own}_r}(h_0)$; no OPEN report on app_hash; $\text{app_hash} \notin \mathcal{B}$	OPEN	electorate frozen at h_0; one capacity slot consumed
SUBMITTED	any guard fails	INVALID	the transaction is rejected; nothing is spent, since there is no fee and no bond
OPEN	$Y_r(h_c) \geq \theta W^\star(h_0)$, $h_c \leq h_0 + \Delta_{\text{vote}}$	CARRIED	first eligible ban height is $h_c{+}1$
OPEN	$h > h_0 + \Delta_{\text{vote}}$, threshold not reached	EXPIRED	capacity released; app_hash immediately re-reportable; no cooldown
CARRIED	the first block at $h_b \geq h_c + 1$ at which the report is first in the settled order carries its ban memo, consensus-required (at most one per block)	BANNED	app_hash enters the ban set; capacity released
BANNED	$h \geq h_b + \Delta_{\text{exec}}$	ENFORCED	Definition 23.18
ENFORCED	—	—	no transition exists (Definition 23.24)

Table 65: Report lifecycle (PLANNED). Every guard is a predicate over the canonical chain, so every transition is evaluated identically by every validator; there is no state that a node can be missing.

Algorithm 11: Consensus-side validation and tally, per block (PLANNED, `fluxd`)

Input: block B at height h; the confirmed node set $N(\cdot)$ and its per-block undo data [fluxd/src/fluxnode/fluxnodecachedb.cpp:14–17]; the open-report table $\mathcal{O}$ and ban set $\mathcal{B}$ as recomputed from the chain prefix

Output: accept or reject B; updated $\mathcal{O}$ and $\mathcal{B}$

`/* Invariant: O and B are pure functions of the chain prefix ending at B. Nothing is written that a resyncing node could not recompute, and no input comes from outside the chain - which is what makes a separate moderation database unnecessary, and, per the fee-pool recommendation of Section 7, undesirable. */`

`/* Failure mode: consensus validates form and authorisation, never the referent - a report naming a hash with no application behind it is valid here and a no-op in FluxOS. Fee-less transactions carry no price signal, so mempool admission must be bounded by capacity, not by fee. On a reorg, O and B are recomputed with the chain and need no repair path. */`

foreach *transaction* tx $\in B$ **do**
  if tx *is* MOD_REPORT **then**
    if σ_r *invalid* ***or*** $\mathrm{own}_r \notin I(h)$ ***or*** $|\{$*open reports of* $\mathrm{own}_r\}| \geq X_{\mathrm{own}_r}(h)$ ***or*** *an open report exists on* app_hash ***or*** app_hash $\in \mathcal{B}$ **then reject** B;
    insert $H(r)$ into $\mathcal{O}$ with $h_0 \leftarrow h$; $Y_r \leftarrow 0$; $V_r \leftarrow \varnothing$;
  if tx *is* MOD_VOTE *on* $H(r)$ *by* i **then**
    if $H(r) \notin \mathcal{O}$ ***or*** σ_v *invalid* ***or*** $i \notin I(h_0)$ ***or*** $h > h_0 + \Delta_{\mathrm{vote}}$ ***or*** $i \in V_r$ **then reject** B;
    $V_r \leftarrow V_r \cup \{i\}$; **if** decision = evict **then** $Y_r \leftarrow Y_r + w_i(h_0)$;
foreach $H(r) \in \mathcal{O}$ *with* $h > h_0 + \Delta_{\mathrm{vote}}$ *and* $Y_r < \theta\, W^{\star}(h_0)$ **do**
  delete $H(r)$ from $\mathcal{O}$ (EXPIRED); release one capacity slot of own_r;
if *some* $H(r) \in \mathcal{O}$ *is carried with crossing height* $\leq h-1$ ***and*** *the coinbase of* B *carries no ban memo for the first such report in the settled order (least crossing height, then least* $H(r)$*)* **then reject** B;
if *the coinbase of* B *carries a ban memo for* $H(r)$ **then**
  if B *carries more than one memo* ***or*** $H(r) \notin \mathcal{O}$ ***or*** $Y_r < \theta\, W^{\star}(h_0)$ ***or*** *the crossing height is not* $\leq h-1$ ***or*** app_hash $\in \mathcal{B}$ **then reject** B;
  $\mathcal{B} \leftarrow \mathcal{B} \cup \{app_hash\}$; delete $H(r)$ from $\mathcal{O}$; release the capacity slot;

Algorithm 12: Ban observation and purge, per node (PLANNED, FluxOS)

Input: canonical chain; confirmation delay Δ_{exec}; local application set A_{loc}; the global specification registry

Output: registry state and A_{loc} for every banned app_hash

```
/* Invariant (exact determinism): enforcement is a function of the
   canonical chain alone, so two honest nodes on the same chain reach
   the same enforcement set - not merely up to gossip delay, as in the
   message-layer construction this replaces.                          */
/* Failure mode (fail-safe, with one asymmetry): a memo whose hash
   resolves to no registry entry is a no-op, and a node that has not yet
   reached h_b + Δ_exec leaves the application up. The asymmetry is that
   purging is not locally reversible: a reorg deeper than Δ_exec that
   removes the ban block leaves an application deleted that the chain no
   longer bans, and it must be redeployed. This is why Δ_exec is a reorg
   parameter and not a policy one.                                    */
```

1 **foreach** *valid ban memo observed at height* h_b **do**
2 **if** $h < h_b + \Delta_{\text{exec}}$ **then wait**;
3 **if** app_hash *resolves to no registry entry* **then** record and **continue** (no-op);
4 mark the application BANNED; issue the cancel message;
5 **if** *an instance is installed locally* **then** remove it by the deployed removal path; refuse subsequent install, redeploy and soft-redeploy; stop location broadcasts;

The voting window is settled at $\Delta_{\text{vote}} = 2{,}880$ blocks [intent: design intake], and the reason given is worth recording because it cuts against turnout: removal often needs to be fast, and a 24 h window is the shortest one that still lets an operator who checks daily participate. Everything Section 23.4 says about liveness is said against that number.

Decided. $\Delta_{\text{exec}} = 120$ blocks (Table 71). Δ_{exec}, the delay between the ban block h_b and enforcement. Domain: [0, 2,880] blocks. Trade-off: it must exceed plausible reorg depth — PoN fork choice is a block-count race because every block carries fixed 2^{40} chainwork [fluxd/src/pow.cpp:284–290], and the deployed reorg bound outside the activation window is 40 blocks [fluxd/src/main.h:74] — because a purge is not locally reversible, while every block of delay is a block of harm uptime. It is also the mechanism's only post-decision safety margin, the analogue of a governance timelock (Section 23.12). Note that $\Delta_{\text{exec}} = 0$ is a defensible choice only if a purged application returning after a reorg is acceptable.

Assumption 23.23 (Model assumptions for the specified mechanism)**.** As fixed in the system-model section, and specialised here. (A1) `fluxd`'s deterministic node set at any height is consensus state and is agreed by all honest nodes. (A2) Owner keys are controlled by the parties the mechanism intends to enfranchise; the consequence of this failing is treated in Section 23.5. (A3) A transaction broadcast during the window is included by some producer before the window closes — which Definition 23.20 shows requires only that not every producer censors it, over 2,880 slots. (A4) A chain reorg deeper than Δ_{exec} does not occur. (A5) Signature forgery is infeasible. No assumption is made about turnout; that is the point of Definition 23.27. Note what is *not* assumed, and was assumed by the construction this replaces: no property of the FluxOS gossip mesh is needed anywhere in the decision path.

The mechanism's invariants follow directly and are stated without separate proof: Y_r is non-negative and non-decreasing within the window; weights and the reference total are functions of h_0 alone; one vote per identity per report, enforced by chain order; a vote verifies against exactly one $(H(r), h_0)$; each report resolves exactly once, into EXPIRED or BANNED, and releases exactly one capacity slot when it does; a node that cannot resolve a ban hash does not remove anything; the ban set is monotone (Definition 23.17); the coinbase grows by at most one memo

per block; and no transition reads or writes a collateral UTXO, the node set, or a node's payment rank. That last one is the settled promise that node collateral is never slashed, and it is what removes the most obvious griefing concern — and, as Section 23.7 shows, what makes voting close to costless. One invariant of the earlier draft does *not* survive unconditionally: that an update during the window is neither an escape nor a second claim holds only under the identity reading of app_hash — which is now the settled one, (app_name, owner) (Table 71), so the invariant does hold on the settled configuration. It is stated conditionally because it is a property of that choice rather than of the mechanism.

Property 23.24 (A ban is final, and what it is final *about*, PLANNED)**.** The specified design contains no transition that removes an application from the ban set and no transition that returns a forfeited prepaid balance, and this is a settled position rather than a gap: the team's decision is that a ban is final, and the remedy is redeployment as a **new** application [intent: design intake].

The scope of that finality is narrower than the word suggests, and stating it precisely answers the strongest objection to this mechanism. A ban removes one application from the network, permanently and irreversibly. It is *not* a forward prohibition on the party that deployed it: the owner may register another application in the next block, at the ordinary price, with no residual mark against their key [intent: design intake]. Nor does it retire the application *name*: the registry entry is released, and the name is available again — to any other owner immediately, and, under the settled (app_name, owner) keying, to every party except the one whose deployment was removed. So the object made permanent is a single deployment. It is not an identity, not a namespace reservation, and not an exclusion from the network.

That distinction is the difference between a removal power and a censorship power, and it is why the absence of a reinstatement path is defensible where an absence of recourse would not be. A wrongly banned tenant does not appeal; they redeploy, in minutes, for the price of one transaction. What they lose is the removed instance's reachability — FDM and PDNS names are keyed per application name — their unearned prepaid balance, and their continuity. What they do not lose is the ability to operate on Flux. The paper does not present that as costless, and Section 23.9 keeps the wrongful-eviction case in the failure surface; it presents it as the reason a monotone ban set is compatible with a network that has no appeal mechanism and does not need one. Both absences are irreversible by construction, which is what makes the decision coherent rather than merely convenient: the forfeiture is a non-event for a pool that was credited at payment time (Definition 23.40), and the ban is a chain fact carried in a coinbase that no later block retracts (Definition 23.17). If a quorum is wrong — honestly wrong on mistaken evidence, or dishonestly wrong under Definition 23.32 — redeployment under a new name is the entire remedy, and it preserves none of the old instance's reachability, since FDM and PDNS names are keyed per application name. A construction that reversed a ban would in any case have to satisfy the requirements an earlier draft of this section raised — a chain object, so Definition 23.14 is not lost for the reversal direction; not a symmetric quorum over the same electorate, or the coalition that can ban can also block reinstatement (Section 23.8); and an interaction with the forfeiture, since reinstating an application whose term was consumed reinstates it into nothing — which is a substantial part of why the team's simpler position is defensible rather than merely a shortcut.

One consequence of the keying is worth stating precisely rather than absorbing silently. Under the settled (app_name, owner) scope, the pair is what enters the ban set, so the banned owner cannot re-take that particular name — which is the whole of what "final" costs them — while any other owner can, and while they themselves can register anything else. Had app_hash instead resolved to the application's *content*, a specification or image hash, the finality would have bitten differently and worse: an otherwise identical redeployment by *anyone* would carry the same hash and be immediately banned with no fresh report and no fresh vote, which turns a decision about one deployer into a network-wide prohibition on a piece of software. That is the

substantive reason the identity reading was chosen over the content reading, and it is recorded here because the two readings differ in exactly the property this subsection is about.

23.4 Safety and liveness, separated

Definition 23.25 (Safety and liveness for this mechanism)**.** *Safety* (against wrongful eviction): an application is not banned unless a set of identities of combined weight at least $\theta W^\star(h_0)$ has confirmed evict votes on one report. *Liveness*: an application that a set of identities of combined weight at least $\theta W^\star$ wishes to remove is removed within $\Delta_{\text{vote}} + \Delta_{\text{exec}} + \delta$ blocks, where δ is the number of slots between the crossing height and the block that carries the memo. Under the settled, consensus-required inclusion rule δ is one plus the number of unbanned carried reports ahead of it in the settled order, so $\delta = 1$ whenever no other carried report is queued ahead of it (Definitions 23.19 and 23.20).

Lemma 23.26 (Turnout bound)**.** *Let $P_r \subseteq I(h_0)$ be the set of identities whose vote on r confirms within the window. Then $Y_r(h) \le \sum_{i \in P_r} w_i(h_0)$ for all h in the window, and if $\sum_{i \in P_r} w_i(h_0) < \theta\, W^\star(h_0)$ then the report cannot be carried and expires.*

Proof. $V_r(h) \subseteq P_r$ by construction and all weights are non-negative, so $Y_r(h) = \sum_{i \in V_r(h)} w_i \le \sum_{i \in P_r} w_i$. Carrying requires $Y_r(h_c) \ge \theta W^\star$ for some h_c in the window, which the displayed inequality forbids; Definition 23.10 then expires the report. □

Theorem 23.27 (Turnout is a liveness parameter and never a safety parameter)**.** *Under Definition 23.23, for any report r and any participation set P_r: adding a non-voter to the electorate can only increase $W^\star(h_0)$ and hence the weight required for a certificate; removing a voter can only decrease Y_r. Consequently low turnout makes eviction strictly harder, never easier, and no low-turnout election can produce an eviction that a full-turnout election with the same* evict *votes would not also produce.*

Proof. The decision predicate is $Y_r(h_c) \ge \theta\, W^\star(h_0)$. The right-hand side depends on the electorate at h_0 and not at all on who votes; adding an identity i' to $I(h_0)$ increases $W(h_0)$ by $w_{i'} \ge 0$ and hence weakly increases $W^\star$ and the required threshold. The left-hand side is a sum over the identities that actually voted evict; deleting one removes a non-negative term. Therefore the predicate is monotone decreasing in the electorate and monotone increasing in the evict set, and the two are independent. Fixing the evict set and varying turnout among the remaining identities changes neither side. □

The contrast is the point. Had the threshold been defined over votes cast — that is, $Y_r / \sum_{i \in P_r} w_i \ge \theta$ — then a single identity of any weight could evict any application by being the only voter, and low turnout would convert from a liveness limitation into a safety hole. The settled rule excludes this. Stated in the design's own terms: the 25 % figure is a liveness/safety dial, abstention counts against removal, and the design deliberately errs toward leaving content up rather than removing it wrongly. That is the defensible position for a censorship-resistant network, and it is the strongest property this mechanism has.

It is paid for in liveness, and the bill is legible.

Proposition 23.28 (Liveness)**.** *If identities of combined weight at least $\theta W^\star(h_0)$ broadcast* evict *votes on r and those transactions confirm by $h_0 + \Delta_{\text{vote}}$, then the report is carried at some $h_c \le h_0 + \Delta_{\text{vote}}$, and the application is removed by $h_c + \delta + \Delta_{\text{exec}}$ with δ as in Definition 23.25.*

Proof. By hypothesis Y_r reaches $\theta W^\star(h_0)$ at the height at which the last of those votes confirms, which is within the window; that height is h_c or later than the least such height, and Definition 23.10 takes the least. A producer at some height $h_b = h_c + \delta$ carries the memo, valid by Definition 23.15; by (A4) it is not reorged out, and by Definition 23.14 every honest node computes the same ban set and applies Definition 23.18 at $h_b + \Delta_{\text{exec}}$. □

Note what dropped out of the hypothesis. The message-layer construction needed a party willing to assemble a certificate and pay for an anchoring transaction — a coordination step with no assigned actor and no incentive attached to it. Here there is nothing to assemble: the votes are already in the chain, and the count is a function every validator already evaluates. That is a genuine liveness improvement and it comes from the same choice that delivers Definition 23.14.

What that hypothesis costs at the measured distribution:

who abstains	weight remaining	share of the remainder that must vote evict
nobody	100 %	25.0 %
the 3 largest keys	74.9 % (66,314.5 units)	33.4 %
the 16 largest keys	49.9 % (44,193.0 units)	50.1 %

Table 66: Turnout required among the remainder to reach $\theta = 0.25$, by abstaining prefix, computed from the measured weight distribution [measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03].

The design intake's own premise is that most operators do not vote and will not follow votes continuously [intent: design intake]. Combined with Definition 23.26, the operational reading is that at $\theta = 0.25$ liveness depends on the large keys participating, because the dispersed remainder is unlikely to reach a third of itself. The same identities that would make the mechanism unsafe are the ones that would make it live.

The 24 h window makes that bill due in one day rather than seven, and the honest way to present the consequence is not as a defect.

Proposition 23.29 (Expiry is the mechanism working)**.** *Carrying a report requires identities holding at least* $\theta W^{\star}(h_0) = 22{,}128.6$ *units at the measured distribution — the equivalent of* 554 *Stratus nodes — to get* evict *transactions* confirmed *inside* 2,880 *blocks. Whenever that does not happen the report expires, and by Definition 23.26 expiry is exactly the event "no set of the required weight voted". An expired report removes nothing, penalises nobody, and returns the capacity slot; the application stays up.*

Against the abstention-counts-as-no rule this is a safety property, not a failure. A missed window is the mechanism declining to act on insufficient agreement, which is the direction the settled dial points, and the design's own premise about operator participation implies that most reports will end this way. The paper states the expectation in those terms rather than presenting a low-success-rate mechanism as a broken one: what a 24 h window at $\theta = 0.25$ buys is a fast path for the cases where the large holders agree quickly, and nothing else. The cost of that choice is stated in Definition 23.28's hypothesis and in the table above; the benefit is that a genuinely harmful workload can be gone in a day rather than a week.

23.5 The measurement

Collateral weighting is deliberate and settled, and it is settled for a good reason: voting power tracks economic stake because collateral is what is at risk. It delivers Sybil resistance as a property of the construction rather than as a hope.

Property 23.30 (Sybil resistance)**.** Voting weight cannot be increased without locking additional collateral. Re-keying, re-addressing, splitting one node's collateral across many node records, or registering additional identities never changes $\sum_i w_i$ under one party's control, because w_i is a sum over collateral amounts attested by consensus state and each collateral UTXO backs exactly one confirmed node [fluxd/src/fluxnode/fluxnode.cpp:445–453].

This is the resource-cost answer to the identity-multiplication attack of Douceur [20], and it is the reason the weighting is settled. Definition 23.12 is the identical argument applied to

proposal capacity, which is why the anti-spam rule inherits this property instead of needing one of its own: both quantities are sums over confirmed nodes, and a confirmed node costs collateral. The same choice is what concentrates the mechanism, and the concentration is measurable.

Definition 23.31 (Collusion set)**.** For a threshold $\theta \in (0, 1]$,

$$\kappa(\theta) \;:=\; \min\Big\{\, |S| \;:\; S \subseteq I(h_0),\; \textstyle\sum_{i\in S} w_i(h_0) \;\geq\; \theta\, W(h_0) \,\Big\}.$$

Because weights are fixed and non-negative, the minimum is attained by a greedy prefix of identities in decreasing weight order.

Theorem 23.32 (Safety is exactly a collusion bound)**.** *Under Definition 23.23, an application can be wrongly evicted only if some set of at least $\kappa(\theta)$ identities all sign* evict*: no smaller set can, at any turnout. Conversely, the $\kappa(\theta)$ heaviest identities can, at any turnout, whenever $W^\star(h_0) = W(h_0)$.*

Proof. ($\Rightarrow$) The voter set that carries a report is a set S with $\sum_{i\in S} w_i \geq \theta W^\star \geq \theta W$, so $|S| \geq \kappa(\theta)$ by minimality in Definition 23.31. ($\Leftarrow$) The greedy prefix of size $\kappa(\theta)$ has weight at least $\theta W = \theta W^\star$ under the stated condition, so its confirmed evict votes carry the report whatever every other identity does, since by Definition 23.26 other identities can only add to Y_r and never subtract from it, and the threshold does not depend on them. The claim "at any turnout" is Definition 23.27. □

The mechanism therefore has the safety property "no coalition smaller than $\kappa(\theta)$ can censor". Whether that is a property or an assumption about people depends entirely on the number. Computed from the 6,489 raw node records:

θ	required weight (units)	κ by owner identity (869)	κ by node key (819)	Stratus nodes alone	of Stratus set
10 %	8,851.5	**1**	**1**	222	13.6 %
15 %	13,277.2	2	2	332	20.4 %
20 %	17,702.9	3	2	443	27.2 %
25 %	22,128.6	**4**	**3**	**554**	34.1 %
30 %	26,554.4	6	5	664	40.8 %
33.3 %	29,475.3	8	6	737	45.3 %
40 %	35,405.8	12	10	886	54.5 %
50 %	44,257.3	19	16	1,107	68.0 %
51 %	45,142.4	20	17	1,129	69.4 %

Table 67: $\kappa(\theta)$ under the settled weighting, by identity proxy, from [measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03] (6,489 records, chain tip 2,917,115; this table and every other figure this section computes from that snapshot are reproduced by `scripts/22-moderation-tables.py`). The operative column is the first: the vote is signed with the owner key, whose public proxy is `payment_address`. The node-key column is retained only because it is the lower of the two and therefore the conservative reading; `pubkey` is the VPS message-signing key and is not the vote key [fluxd/src/fluxnode/fluxnode.h:154]. Merging addresses into clusters by shared node key or shared collateral funding transaction (797 clusters) gives 1/2/3/16 at 10 % / 15 % / 25 % / 50 %; the intermediate rows were not computed for that model. Nimbus alone cannot reach 25 % — it would need 1,771 of 1,615 nodes — and Cumulus in its entirety holds 3.67 %.

The largest single identity holds 10.71 % of voting weight across 237 Stratus nodes under both identity models. Decomposed by node key, the three largest hold 9,480 units across those 237 nodes (10.71 %), 9,360 across 234 nodes (10.57 %), and 3,360 across 84 nodes (3.80 %), summing to 25.08 % — that is 22,200,000 FLUX of collateral held by three keys. Under the owner-identity model the same threshold takes four addresses. The Gini coefficient of voting weight is 0.8375 across payment addresses and 0.839 across node keys. 47 keys pay to more than one address and

5 addresses receive from more than one key; no collateral funding transaction pays more than one address (0 of 6,178), so funding-transaction grouping fragments rather than merges and is not by itself a party model. The fleet is bimodal: 107 keys with ten or more nodes hold 72.5 % of all weight, while 334 keys hold exactly one node and are electorally irrelevant. That 819 node keys serve 6,489 nodes is a separate observation with two consequences: operators routinely reuse one node key across many machines, which is a linkage signal here and a key-hygiene matter elsewhere.

Corollary 23.33 (The measured verdict)**.** *At $\theta = 0.25$ and the measured distribution, $\kappa(0.25) = 4$ by payment address — the owner-linked proxy, and therefore the operative figure — and 3 by node key. Merging addresses linked by a shared node key or a shared collateral funding transaction into single clusters gives 797 identities and $\kappa(0.25) = 3$ as well. Safety against wrongful eviction is therefore not a property of the mechanism but the assumption that three or four specific parties do not agree, and the figure is stable across every identity model computable from public data.*

Both counts are **lower bounds** on concentration. They assume every distinct key is a distinct party; one party holding several keys makes the true number smaller, never larger. They are not upper bounds on anything. This limitation is structural rather than a gap in the measurement: a figure that reflects *parties* rather than keys cannot be obtained from public data.

Corollary 23.34 (The graduated tiers are unilateral)**.** *The proposed graduated-penalty tier at approximately 10 % is reachable by the largest single identity alone, which holds 10.71 % across 237 Stratus nodes under one key. The tier at approximately 15 % is reachable by two identities. Any action attached to those tiers is therefore an action that one or two parties can take at will.*

Decided. Removed — one threshold, at 25 % (Table 71). Actions at the graduated 10 % and 15 % tiers, or their removal. Domain: {none, flag, warn, restrict egress, . . . }. Trade-off: by Definition 23.34 any *destructive* action at those tiers is one party's decision; non-destructive, reversible actions remain consistent with the fail-safe direction. The design intake proposes the tiers without specifying their effects.

Corollary 23.35 (Raising the threshold does not rescue the property)**.** *By node key, $\kappa \geq 10$ requires $\theta \geq 0.40$ and $\kappa \geq 16$ requires $\theta \geq 0.50$. Even $\theta = 0.51$, the value the design rejected as unreachable in practice, corresponds to seventeen identities by node key and twenty by owner identity.*

Raising θ trades liveness (Definition 23.28) for a larger but still small coalition. It does not produce a κ that could honestly be called a property rather than a promise.

Conjecture 23.36 (Custodial dispersion)**.** *If a substantial fraction of the largest owner identities are custodial hosting providers holding collateral for many distinct beneficial owners, then the number of* parties *behind the four identities that reach 25 % is larger than four, and $\kappa(0.25) = 4$ overstates concentration by however much beneficial ownership is dispersed behind those addresses. Note what the measurement already rules out: the gain cannot come from separating the node key from the owner, because the measurement is already taken at the owner — `payment_address` is derived from the collateral public key [fluxd/src/rpc/fluxnode.cpp:1512–1520] — and moving between the two identity models changes $\kappa(0.25)$ only between 3 and 4.*

This is a conjecture and not a result, and unlike the other open items in this section it is not convertible into one: beneficial ownership behind a transparent address is not exposed by any public data. $\kappa(0.25) = 4$ is therefore the number that stands and the number the paper prints. Note also that the conjecture cuts both ways — a custodial identity votes with its customers' stake, which is a different and worse problem than concentration, and it is invisible in the same data.

Not established by this survey. Open measurement, now narrowed rather than open: the earlier draft of this section called for concentration by collateral-spending script. That measurement is subsumed, because `payment_address` is derived from the collateral key, so the owner-identity column of the table above *is* the collateral-key measurement. What remains is beneficial ownership behind those addresses, which is not computable from public data and which decides Definition 23.36. No claim about dispersion may be made from public data.

Decided. The start-transaction shape, signed by the collateral (owner) key (Table 71). Which existing transaction shape the vote follows, and hence which key signs it. Domain: {the start-transaction shape, signed by the collateral (owner) key [fluxd/src/fluxnode/activefluxnode.cpp:253–288]; the confirm-transaction shape, signed by the node key [fluxd/src/fluxnode/activefluxnode.cpp:290–332]; either, via an on-chain delegation}. Trade-off: the settled rule says node *owner* keys, which selects the start-transaction shape — stake-bound, cold, and the identity every figure in this section is computed against — at the cost of asking operators to sign with a cold key on a 24 h deadline. The confirm-transaction shape is operationally far easier because that key is already hot and already signs every 500 blocks, but it is the VPS key, it may be custodial, and choosing it would require recomputing the entire concentration measurement against the node-key column and restating $\kappa(0.25)$ as 3.

23.6 The negative result

Remark 23.37 (The measured concentration is accepted, not disputed). Before the negative result, the team's position on it. Presented with the measurement — four owner identities reaching the 25 % threshold, three under the strongest linkage computable from public data — the response was to accept it: if four parties hold a quarter of the network's collateral then those parties should be able to remove an application, and "as network grows it will be more decentralized" [intent: design intake] (PLANNED).

The paper records this as a deliberate acceptance rather than an unrecognised risk, which changes how the rest of this section should be read: what follows is not an objection the design has failed to answer, but a property the design has been told about and kept. Two observations remain load-bearing even so, and neither is a criticism of the choice.

First, growth dilutes concentration only if new collateral is new parties. The measurement is a ratio, not a count. If the network doubles and the four largest owners double with it, the ratio is unchanged; dilution requires that marginal entrants be new and independent, which is an empirical question this corpus cannot settle either way. **Second, and more sharply, the mechanism the paper elsewhere identifies as the response to weak demand runs the other way.** Definition 7.36 records that a demand shortfall is answered by contraction — operators exit, the survivors earn more — and contraction concentrates. Growth as a remedy and contraction as a stabiliser cannot both be relied on; which one obtains depends on demand for Flux Cloud, and no part of this section's design references demand at all.

The natural response to Definition 23.33 is to change the weight function. It does not work, and the reason it does not work is worth stating precisely, because it is the section's contribution.

Capping per-identity weight at some c is defeated at negligible cost: an address is free, each node's collateral can be respent to its own, and the 237-node identity becomes 237 identities. The cost of that split is one collateral respend, 100 blocks of maturity and one start transaction per node [fluxd/src/coins.cpp:630–686] — real, but bounded by a few hours of missed rotation against the 22,128,625 FLUX the threshold requires, so it is not a barrier to anyone who can reach the threshold in the first place. This is exactly the Sybil that collateral weighting was chosen to prevent, reintroduced with the cap itself as the incentive. Quadratic weighting per identity is defeated identically and more sharply — the same holder is worth $\sqrt{9480} \approx 97.4$ as one identity and $237\sqrt{40} \approx 1498.9$ as 237, a fifteenfold reward for splitting. Only weightings applied *per node* are Sybil-neutral, and those are tier reweightings, not concentration fixes:

weighting	Stratus	top key	top 3	$\kappa(0.25)$	$\kappa(0.51)$	Gini	cost of 25 % (FLUX)
collateral (settled)	73.5 %	10.71 %	25.08 %	**3**	17	0.839	22,128,625
tier base 1 : 3.5 : 9	62.2 %	9.06 %	21.23 %	5	22	0.793	—
$\sqrt{c_t/1{,}000}$ per node	53.5 %	7.79 %	18.31 %	6	26	0.770	—
one node, one vote	25.1 %	7.40 %	17.58 %	6	38	0.716	1,623,000

Table 68: Sybil-neutral reweightings, computed over the same 6,489 records [measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03]. Every row costs 1,000 FLUX to add one Cumulus node, so none reopens the Sybil vector; the last column is the capital needed to *buy* the threshold outright, and only the two extreme rows were computed. Dashes are uncomputed, not zero.

Remark 23.38 (Which identity model this table uses, and how to read it). Every row above is computed under the **node-key** model, in which the settled weighting gives $\kappa(0.25) = 3$ rather than the operative 4 that the owner-identity model yields. The owner-identity recomputation has not been run for the per-row figures.

This is a caveat about reading individual rows, not a defect in the conclusion. The recomputation would shift each row by the same one-or-two identities that Section 23.5 measures between the two models — it moves every row in the same direction by roughly the same amount, which is why the open problem below is stated as *no row approaches a κ that could be called a property* rather than as a claim about any particular row. The aggregate conclusion is robust to the model choice; a single row quoted on its own is not, and should be recomputed under the owner-identity model first.

Open Problem 23.39 (No reweighting, threshold or listed remedy raises κ meaningfully). At the measured distribution and $\theta = 0.25$: no weight function in the Sybil-neutral design space raises $\kappa(0.25)$ above 6, and the best of them — one node, one vote — reduces the capital required to buy the threshold from 22,128,625 FLUX to 1,623,000 FLUX, a factor of 13.6, which is a strictly worse trade. Raising θ to 0.51 yields $\kappa = 17$ at a liveness cost the design has already rejected (Definition 23.35). None of the remedies listed in Section 23.8 changes κ at all, and neither does the on-chain shape of Section 23.3: moving the tally into consensus changes who can *verify* the decision, not who can *make* it. *Open:* whether any mechanism consistent with the settled decisions — collateral weighting, absolute threshold, abstention-as-no, no collateral slashing — achieves a $\kappa^\star$ at $\theta = 0.25$ materially above the measured 4 (3 under the node-key model) without reintroducing a Sybil vector. This paper does not have one, and the two-chamber rule of Section 23.8 explicitly does not claim to be one.

The reason is not subtle once stated: concentration lives in node *ownership*, not in the weight function. The largest key holds 7.4 % of nodes by count, so even one-node-one-vote leaves it above the 5 % mark; the weight function only decides how much worse than that the ownership distribution is allowed to make things.

23.7 Dilution: why the honesty incentive does not bind

The design's argument for why free voting is not a vulnerability is that operators are paid from application revenue, so evicting applications reduces their own income and they are therefore incentivised to be honest [intent: design intake]. The argument is correct in aggregate and wrong at the margin, which is the standard shape of a collective-action problem. It fails in three steps, each independently sufficient.

23.7.1 The current term is already paid

Proposition 23.40 (Eviction is income-neutral to the fleet for the current term)**.** *Under PNR as specified in Section 7, hosting is prepaid for d blocks and the price $\mathcal{P}$ is split by consensus at payment time: $(1-\rho)\mathcal{P}$ to the Foundation and $\rho\mathcal{P}$ into the pool Φ, which pays all nodes a*

drip of Φ/K per block regardless of which applications run. Eviction of a leaves Φ unchanged. Therefore the fleet's income for the current term is invariant under eviction, and for a node currently hosting a the effect is strictly positive: unchanged pay, released resources.

Proof. Φ is credited at payment and drained by a per-block drip that is a function of Φ and K only; no term of the drip references the set of running applications. Removing a alters no argument of that function. A hosting node's payment is likewise independent of hosting, so its income is unchanged while its committed CPU, RAM and disk are freed. □

What the fleet loses is the *future*: renewals of a, and the demand effect of a network where eviction is capricious. Let R_a be expected future renewal revenue in FLUX, $g_i \geq 0$ the resource relief to a hosting node, and $b_i \geq 0$ any private benefit to identity i from a's removal. With s_i node i's share of the pool, node i prefers eviction iff $b_i + g_i > \rho\, s_i\, R_a$.

23.7.2 Dilution

PNR splits the drip across tiers in the ratio 1 : 3.5 : 9 and then equally within a tier, so $s_i = (\beta_{t_i}/13.5)/n_{t_i}$:

tier	s_i (PNR tier-base split)	s_i (collateral share)	cost of evicting a at $\rho = 0.20$
Cumulus	0.0023 %	0.0011 %	0.00046 % of R_a
Nimbus	0.0161 %	0.0141 %	0.0032 % of R_a
Stratus	0.0410 %	0.0452 %	0.0082 % of R_a

Table 69: Per-node pool share and the private cost of destroying an application's future revenue.

A bloc of 554 Stratus nodes — the 25 % threshold — internalises $554/1{,}627 \times 9/13.5 = 22.7\,\%$ of the operator share, i.e. 4.54 % of R_a at $\rho = 0.20$. The largest single identity internalises 1.94 %. The social cost of a wrong eviction is ρR_a to operators, plus $(1-\rho)R_a$ to the Foundation, plus the tenant's unused prepayment, plus the loss to the application's users; the decisive coalition bears under 5 % of the first term alone.

This is a collective-action problem in textbook form: the fleet collectively wants applications and no individual member is deterred from destroying one. The two design decisions that produce it are each correct on their own terms. Paying *all* nodes is the fair rule — placement is luck-driven and over a rotation every node is paid equally — and it is precisely what makes $s_i \approx 4 \times 10^{-4}$. Paying only hosting nodes would concentrate the deterrent where the workload sits, and would reintroduce exactly the placement-metering dependency PNR was designed to avoid. The team should see the interaction plainly: **fairness in PNR and honesty in moderation pull on the same rope in opposite directions.**

23.7.3 The deterrent is weakest at launch

Proposition 23.41 (Launch-time deterrent)**.** *At PNR activation, with $\rho = 0.20$, operator emission of* 6,386,040 FLUX *per year and the planned operator share of an assumed application revenue at approximately* 4.3 % *of operator income, the expected future revenue a Stratus operator forgoes by destroying one average application is approximately* 0.10 FLUX *per year.*

Proof sketch. The planned operator share of application revenue, at 4.3 % of total operator income against 6,386,040 FLUX of emission, is 286,320 FLUX per year from all applications: the 20 % operator share of an assumed, not measured, 1,431,600 FLUX per year of application revenue (Section 7). The Stratus tier receives 9/13.5 of the operator share, i.e. about 190,880 FLUX per year, spread over 1,627 nodes, i.e. about 117.3 FLUX per node per year. Divided across the 1,193 priced applications in the PNR demand baseline [intent: mechanism specification] — the registry

returns 1,195 specifications [measured: api.runonflux.io/apps/globalappsspecifications, 2026-09-03] — that is 0.10 FLUX per Stratus node per application per year. □

Against a forfeiture floor of 0.01 FLUX on the tenant's side, that is the entire economic content of the honesty argument at launch. It rises with the ramp toward $\rho_{\max} = 0.50$ and with demand, and it is smallest on the day the mechanism would ship. Note also the self-dealing case, which is worse than neutral: an operator who also hosts a has $g_i > 0$ by Definition 23.40, so under PNR there is a small positive incentive to evict one's own tenants, bounded only by the threshold.

Finally, a constraint this section exports to the revenue mechanism. Section 7 must guarantee that **forfeitures from eviction never enter the fee pool**. If they did, evicting an application would pay the fleet that voted to evict it, converting a socialised cost into a distributed gain and inverting the incentive this subsection has just shown to be weak.

Decided. Burn pro-rata (Table 71, Definition 7.34). Disposition of the pool's unearned remainder on eviction. Domain: {keep in Φ (status quo); burn pro-rata}. Trade-off: burning the unearned fraction is the only construction identified that gives voters a non-zero *current-term* cost of eviction and thereby repairs Definition 23.40; it requires PNR payments to be bound to individual applications, which the private-payment section notes may leak the workload under shielded payment. Keeping the remainder preserves Definition 23.40 exactly as measured.

23.8 Remedies, evaluated

Each remedy below is assessed on three axes: what it fixes, what it costs, and what it does not fix. The first is adopted by the specified design and the rest are not; the third column is the same in every one of them, and it is the actual problem.

Proposal capacity bounded by collateral (adopted, Definition 23.11). *Fixes:* report spam, at a price of 1,000 FLUX of locked collateral per concurrently open report; block-space exhaustion by fee-less transactions, which nothing else in the design bounds; and, as a side effect, reporter eligibility, which stops being a parameter. *Costs:* nothing new — no bond, no custody script, no adjudicator that must learn the outcome to release funds, and no forfeiture destination to argue about. It is a validity predicate over data `fluxd` already holds. *Does not fix:* capacity is distributed as node counts, so the ability to occupy the report channel is concentrated in the same hands as everything else (Gini 0.716); and it prices nuisance in *locked* rather than *spent* capital, so a determined griefer pays only opportunity cost. Concentration.

Refundable report bond (evaluated, not adopted). *Would fix:* frivolous reports, by making n failed reports cost $n\beta_{\text{rep}}$ outright rather than n node-slots of locked capital. *Costs:* a refundable bond needs an adjudicator that can read the outcome — a consensus change, a release key, or a non-refundable fee that taxes honest reporters — and each of those is a new trusted or new consensus object. The capacity rule obtains a comparable bound with none of them, which is why the design has no bond and why β_{rep} appears nowhere in this section's arithmetic. *Does not fix:* any report that succeeds is free either way, so a coalition at threshold pays nothing. Concentration.

Rate limiting. *Fixes:* denial of service against operator attention; serial re-reporting of one tenant. *Costs:* per-key limits multiply with keys — a 480-node operator who gives each node its own key obtains $480\, r_{\max}$ at zero collateral cost, and per-node limits are the same thing. This is precisely the failure the capacity rule avoids by counting *nodes* rather than *keys*: by Definition 23.12 the total is invariant under any re-keying, so the rule is a genuine bound where a per-key rate limit is not. *Does not fix:* anything about who can reach threshold. Concentration.

Eligibility gating. *Fixes:* excludes dead and unbenchmarked nodes, and — because confirmation in $N(h)$ already requires a benchmark-bearing confirmation transaction within the last 640 blocks — makes "confirmed at h_0" the on-chain proxy for benchmark-passing, which is also what keeps the tally verifiable against consensus state rather than against a FluxOS-local status. *Costs:* the benchmark signature is not operator-bound, as the system-model section establishes, so the gate is only as strong as the benchmark; any stricter "good standing" test that is not on-chain breaks verifiability. *Does not fix:* the largest keys are benchmark-passing nodes in good standing. Concentration.

Reinstatement (rejected: the team's decision is that a ban is final; Definition 23.24). *Would fix:* a symmetric reinstatement quorum gives an honest majority a path to reverse a wrongful ban; a *sunset*, under which a ban expires unless renewed by a fresh report and a fresh quorum, puts the liveness burden on the side that has already demonstrated it can reach quorum — a persistent bad application is re-banned each period, a wrongly banned one returns by default. *Costs:* a symmetric quorum inherits Definition 23.26 and is asymmetric in exactly the wrong direction, since the dispersed honest majority faces the 33 % to 50 % turnout problem while the coalition that banned can reinstate at will; a sunset gives a harmful application a periodic window of uptime, costs the banning side a renewed quorum every period, and converts the ban set from monotone (Definition 23.17) into state that must be re-derived at every height. *Does not fix:* under a symmetric quorum the same four identities decide both directions. The specified design adopts neither, consistent with the team's decision that a ban is final and the remedy is redeployment (Definition 23.24) — a remedy that is available immediately and costs one transaction, because finality attaches to a deployment and not to the deployer. Concentration.

Opt-in delegation (adopted, Definition 23.9). *Fixes:* liveness, which is the mechanism's scarcest resource. Turnout rises without operators following every report, and a higher, safer θ becomes reachable later. *Costs:* delegation concentrates *signing authority* in delegates by construction — the documented trajectory of every delegated token-vote system — and makes delegates the party a coalition would rent or coerce. *Does not fix:* concentration of weight; it does not worsen it either, because Definition 23.9 leaves $w.$ and κ at the owner. The decisive argument for adopting it is that the alternative is not a safer mechanism but an inert one: owner keys are cold by design, and a 24 h owner-key signing requirement makes expiry the expected outcome of an honest report.

Decided. Evidence is an optional content hash, and a report counts as its reporter's vote at h_0 (Table 71). The report's descriptive fields, and whether a report is itself a vote. Domains: evidence $\in$ {absent; optional; a mandatory content hash}; category as left open in Section 23.2; and whether own_r's own weight is added to Y_r at h_0 or requires a separate vote transaction. Trade-offs: an evidence hash is unverifiable by any node and costs bytes in a fee-less transaction, and its only real consumer is a future adjudication of the kind the team's fallback contemplates; counting the report as a vote saves one transaction and makes the report's author's weight implicit rather than explicit in the chain record, which is a small loss of the attribution the design otherwise provides. Note that three parameters the earlier draft carried here are now *settled* by the on-chain shape rather than chosen: the per-application cooldown is zero, the per-identity bound on open reports is $X_i(h_0)$ (Definition 23.11), and reporter eligibility is $I(h_0)$. The *network-wide* cap is a separate parameter and is no longer left at $|N(h)|$: see Table 71, where it is set to 64 with queueing, for the block-space reason given in Definition 23.13's discussion.

23.8.1 The recommended construction, and exactly what it buys

Definition 23.42 (Two-chamber decision rule)**.** Let $\text{tenure}_i(h) := h - \min_{j\,:\,\text{own}_j = i} \text{added_height}_j$, the age of the identity's oldest node. Carry the report iff *both*

$$Y_r \;\geq\; \theta\, W^{\star}(h_0) \qquad \text{and} \qquad \big|\{\, i \in V_r \;:\; \text{tenure}_i(h_0) \geq T \,\}\big| \;\geq\; m,$$

with $m \in \mathbb{N}$ and $T \in \mathbb{N}$ (blocks) open.

Proposition 23.43 (The two-chamber rule preserves everything settled and lowers nothing)**.** *Adding the second conjunct leaves weights, threshold, abstention rule and enforcement untouched; Definition 23.26, Definition 23.27 and Definition 23.32 continue to hold verbatim for the first chamber; the vote transactions are unchanged, with validation extended by a tenure lookup against $N(h_0)$, which is data* `fluxd` *already holds. In particular $\kappa(0.25) = 4$ is unchanged.*

Proof. The rule is a conjunction, so the set of evictable configurations shrinks weakly; every statement of the form "no set smaller than X can evict" is preserved and no statement of that form is strengthened, because the second conjunct is satisfiable by any coalition that has positioned m tenured identities. Definition 23.26 and Definition 23.27 concern the first conjunct only and are untouched. □

The tenure gate is not optional. Without it the second chamber counts identities, and an identity is nearly free: a four-identity coalition re-addresses $m - 4$ of its own nodes and satisfies the chamber at no collateral cost at all. Re-addressing requires a collateral respend and a new start transaction, and it resets `added_height` — an inference the earlier draft flagged as load-bearing and unverified, and which is **now verified against source**. `fluxd` keys its node cache on the collateral outpoint, so a respend produces a different key and therefore a different record SHIPPED [fluxd/src/fluxnode/fluxnode.h:148–160]; `CheckNewStartTx` rejects a second start against an outpoint already tracked, so the old record cannot be carried over SHIPPED [fluxd/src/fluxnode/fluxnode.cpp:445–476]; and `AddNewStart` writes `nAddedBlockHeight = p_nHeight` together with `nLastPaidHeight = 0` on every admission SHIPPED [fluxd/src/fluxnode/fluxnode.cpp:326–341]. Nor is there a transfer path that would avoid this: the node transaction family is exactly {START, CONFIRM} SHIPPED [fluxd/src/primitives/transaction.h:618–619], with no owner-key update type. Tenure therefore resets, and so does rotation position — the re-addressed node re-enters the queue as never-paid, keyed by confirmation height SHIPPED [fluxd/src/fluxnode/fluxnode.h:313–328] — one missed Stratus rotation being 9 FLUX, about 0.155 % of that node's annual subsidy income. **The tenure gate does the work the construction claims it does**, and m and T can be set on that basis rather than provisionally.

Tenure eligibility at the measured distribution:

T	nodes with tenure $\geq T$	distinct keys	tenured weight	top-3 keys' tenured weight
7 d (20,160 blocks)	5,639 (86.9 %)	744	90.1 %	24.05 %
30 d (86,400)	4,149 (63.9 %)	607	77.0 %	23.95 %
90 d (259,200)	2,824 (43.5 %)	412	57.2 %	19.66 %
180 d (518,400)	1,516 (23.4 %)	286	31.5 %	9.49 %

Table 70: Tenure distribution [measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03]. The last column is the weight, as a share of all weight, of the three largest keys among those with tenure $\geq T$; it is informational — the tenure gate applies to the identity chamber only — but that the three largest keys' nodes are mostly younger than 180 d says the concentration is recent.

What the rule buys is precise and limited. A reactive eviction — "these four identities sign now" — would no longer be possible unless $m - 4$ other tenured identities also sign. A *pre-planned* eviction would remain possible for any coalition that positioned $m - \kappa(\theta)$ tenured

identities at least T blocks earlier, at a cost of $(m-4) \times 1{,}000$ FLUX in Cumulus collateral, or nothing at all by re-addressing nodes it already owns, and in either case **visible on-chain in advance**. At the settled $m = 21$ the collateral route costs 17,000 FLUX, which is 0.077 % of the 22,100,000 FLUX the first chamber already requires; at $m = 100$, the top of the domain, it costs 96,000 FLUX, 0.43 %. The rule is therefore not a capital barrier. It is a time-and-visibility barrier, and its entire value lies in composition with publishing $\kappa(\theta)$, the top-N shares and the Gini at every reduction height: a burst of re-addressing, or of new single-node identities from a large operator, becomes a public, datable warning.

It costs liveness in the second chamber — m distinct tenured identities must actually vote. With the settled $T = 30$ d and $m = 21$, 3.5 % of eligible keys must participate (16.5 % at $m = 100$); with $T = 90$ d, 5.1 %. Median node tenure is 193,620 blocks, about 67 d, so any T beyond roughly 60 d disenfranchises the median node from the second chamber.

What $T = 30$ d does not buy, stated because the table above invites the wrong reading. The last column of the tenure table moves by 0.1 percentage points between $T = 7$ d and $T = 30$ d (24.05 % to 23.95 %): the large keys' nodes are overwhelmingly older than thirty days, so the settled T excludes essentially none of them. Anyone reading the gate as a filter on the concentrated weight should stop — it is not one, and Definition 23.43 is explicit that $\kappa(0.25) = 4$ is unchanged.

What T actually sets is *lead time*: how far in advance a coalition must position the identities it will use, and therefore how much warning the network gets. Thirty days is a short planning horizon for an adversary who is planning at all. Ninety days is the smallest value in the domain that is awkward to arrange quietly, and it costs 1.6 % more required participation and disenfranchises the median node — a real cost against a mechanism whose scarcest resource is turnout. The paper records the trade squarely rather than recommending its way out of it, and notes the one thing that holds at every T: because re-addressing is a chain event, a burst of it is datable whether or not the gate would have caught the resulting identities. **The warning is the durable value; T only decides how early it arrives.** If $T = 30$ d is kept, it should be kept for that reason and not described as a barrier to the identities the measurement names.

Decided. $m = 21$, $T = 86{,}400$ blocks (Table 71). m, the identity-chamber count, and T, the identity tenure. Domains: $m \in [5,\ 100]$; $T \in [20{,}160\ (7\,\text{d}),\ 259{,}200\ (90\,\text{d})]$ blocks. Trade-offs: larger m raises the Sybil cost of pre-positioning to $(m-\kappa) \times 1{,}000$ FLUX or to T blocks of waiting, and raises the number of tenured identities that must actually vote; larger T raises the attacker's required lead time and disenfranchises younger honest identities against a median tenure of 67 d. The second chamber counts the same owner identities as the first (Definition 23.6); it is the tenure gate, not the identity definition, that does the work, because every identity definition available here is cheap to multiply.

This is the honest ceiling. Raising the cost of *surprise* rather than the cost of *capture* is what mechanism design can do here, and the section says so rather than presenting a bicameral rule as a safety proof it is not.

23.9 Attacks and failure surface

Report brigading. n colluding reporters filing n reports on one application would achieve nothing beyond one report: at most one report per application hash is open, and the tally is per identity, not per report. Brigading the *vote* is simply the coalition of Definition 23.32; it costs nothing at all, because there is no bond and no fee. Brigading the *report channel* is bounded by Definition 23.12 at one concurrent report per node operated, which is the whole of the anti-spam design. Cost to the network per brigade: one fee-less transaction per report, plus whatever vote traffic the reports attract, against the block-space budget computed in Section 23.3.

A competitor evicting a rival. Requires $\theta W^{\star}$ of weight. If the competitor is one of the four largest owner identities, the other three suffice with it. If not, it must acquire 22,100,000 FLUX of collateral, wait 100 blocks of maturity plus confirmation, and — under the two-chamber rule — pre-position tenure; or it must rent the signatures. Its payoff is the rival's diverted demand less $\rho s_i R_a$, which for the largest identity is about 2 % of R_a. Nothing is staked and nothing is forfeited on either outcome. The record is public, so the deterrent is reputational and, under the fallback below, retrospective.

A censoring bloc. Four owner identities — three under the strongest linkage computable from public data — would ban any application at will within $\Delta_{\text{vote}} + \delta + \Delta_{\text{exec}}$ blocks at zero cost, and at 50 % (nineteen identities) they would satisfy any two-chamber rule they had prepared for. Nothing in the mechanism prevents this, and under Definition 23.24 nothing reverses it: a wrongful ban is a permanent chain fact. What the mechanism does is make it *attributable*, and the on-chain shape strengthens exactly this: the evidence is not a gossiped certificate that a node may or may not hold but a set of confirmed, height-stamped transactions that every archival node has, in an order the chain fixes.

Producer-level censorship. New with the on-chain shape, and the reason the inclusion rule of Definition 23.19 was made consensus-required rather than left to producer discretion. Under the settled rule a producer cannot decline to include a due ban memo and still produce a valid block, so this vector is closed by a consensus rule rather than merely made statistically unlikely: the delay is one slot for a report with no carried report queued ahead of it, regardless of f, the fraction of nodes that would have preferred to withhold it (Definition 23.20). What remains subject to producer discretion is *vote* censorship: a coalition holding a fraction f of nodes can decline to include a given vote, but it needs every one of 2,880 slots to succeed, so vote censorship fails with probability $1 - f^{2,880}$. Putting the decision on-chain now makes both the vote and the memo robust to an individual producer — the vote by the size of the window, the memo by an explicit consensus rule — which is the reverse of the asymmetry an earlier draft of this section anticipated.

Electorate timing. The reporter chooses h_0. Without the trailing maximum of Definition 23.6, a reporter would choose a height at which many nodes are temporarily unconfirmed. The largest two keys hold 21.28 % and reach 25 % of any $W(h_0) \le 0.851\,W$; the largest alone reaches it if $W(h_0) \le 0.428\,W$. Confirmation expiry is 640 blocks, so an incident that leaves 15 % of weight unconfirmed for a few hours is the attack window, and Δ_W closes it. Racing a vote to the window boundary is not an attack — a vote confirming after $h_0 + \Delta_{\text{vote}}$ is simply invalid — but a reorg across the ban block is, and Δ_{exec} absorbs it.

Griefing, and what actually bounds it. The specified design's answer is Definition 23.11 and nothing else: a griefer's sustained report rate is its node count per 24 h, and its concurrent footprint is its node count, priced at 1,000 FLUX of locked and recoverable collateral per slot. Holding every deployed application under permanent report therefore costs 1,195,000 FLUX of locked capital and buys operator attention rather than eviction. Compare the alternatives the design did not take: a bond would charge per *failed* report and nothing to a griefer with a whale's backing; a per-key rate limit would be multiplied by free keys, since a 480-node operator has 480 of them; eligibility gating alone requires 1,000 FLUX, which is already true of every operator. Under the two-chamber rule, a griefer who reaches the first chamber must also find m tenured identities.

Wash-deployment. A deployer who registers and then reports their own application spends $\mathcal{P}$ and, on success, forfeits the unused remainder of it; on expiry, spends nothing further, because

reporting is free and the only cost was the capacity slot. There is no state in which the cycle yields more than it costs, because the mechanism has no pool a wash could farm — which is exactly the property that the Section 7 constraint on forfeitures preserves.

The quorum is simply wrong. An honest coalition bans a legitimate application on mistaken evidence. Recourse, in order of availability: redeployment under a new name, always available, costing one fee and preserving none of the old instance's reachability, since FDM and PDNS names are keyed per application name; and the public chain record, which names the voters. Reinstatement is *not* on that list, because the design has none (Definition 23.24). When redeployment does not suffice, **the design bounds the damage rather than preventing the error**: the maximum loss is one prepaid term (floor 0.01 FLUX, typically the unused fraction of $\mathcal{P}$), the redeployment fee, and $\Delta_{\text{vote}} + \delta + \Delta_{\text{exec}}$ blocks plus redeployment time of downtime. That bound is the "err toward leaving content up" dial applied to the cost of the other error, and it is the design's actual guarantee. It is also the whole of the guarantee: the ban itself is permanent.

Byzantine minority and colluding majority. A minority below θ can report (bounded by its node count), can vote, cannot ban, cannot forge or replay votes, cannot keep a vote out of the chain for a whole window (Definition 23.20), and cannot produce a valid ban memo, because a block carrying one for an uncrossed report is invalid and is rejected like any other invalid block. A coalition at or above θ is the censoring bloc above. Sustained low demand shrinks Φ and drives the already-negligible PNR deterrent toward zero without affecting safety. Sustained low participation means genuinely harmful applications stay up, which is a liveness failure and not a safety failure (Definition 23.29).

The team's stated fallback. If malicious reporting appears, the chain is to be forked to introduce penalties for malicious submissions [intent: design intake]. This is a legitimate governance answer and the paper states it plainly, with its limits. It is *reactive*, acting after the ban. It requires a *hard fork* of `fluxd`, with the coordination that implies — and the consensus section's history of what a consensus change costs this network is directly relevant. It requires *distinguishing malicious from mistaken reports after the fact*, for which no oracle exists and which the mechanism does not attempt. And because collateral is never slashed, the penalty must be something other than collateral, which is not specified. What the mechanism does contribute is the record: a future fork would have signed, height-stamped, chain-resident evidence of every vote to act on, and would not have to trust anyone to have kept it. Without that the fallback would have nothing. It is worth noting that a fork is also the only route the specified design leaves for undoing a wrongful ban, which makes Definition 23.24 and this paragraph the same problem seen from two sides.

23.10 The decentralization endgame

A section on governance owes the reader an inventory of what is team-operated today and must not be at target. Drawing on the networking and addressing section's centralization table and on the node-services extraction, the components are: `api.runonflux.io` as FDM's single default application-discovery endpoint, with no automatic failover (the `/apps` routes themselves are served by every node, which replicate application specifications by gossip) [flux-domain-manager/src/services/flux/dataFetcher.js:47–91]; the Foundation-operated private `10.100.0.0/24` carrying the WireGuard hub, the DNS gateway, the Arcane load balancers and both PowerDNS masters [flux-fdm-infrastructure/ansible/inventory/hosts.yaml:102–206]; the 16-host FDM fleet [flux-fdm-infrastructure/ansible/inventory/hosts.yaml:8–100], none of which runs on a node operator's machine; the `fluxos-network-policy` channel of Definition 23.2; the single-instance `fluxstorage` service, which holds oversized environment variables and commands unencrypted at rest with no write-side protection at all

[fluxstorage/src/routes.js:17–67]; the release API, which performs no cryptographic verification of what it serves as the latest release SHIPPED [private source: release API]; the team-held application owner keys and the `usersToExtend` extension key used for auto-renewal SHIPPED [private source: renewal service]; and a `fluxteam` privilege level in the node UI strictly above a node owner's own `admin` [fluxos-frontend/src/navigation/vertical/fluxadmin.js:39–43]. To that list the consensus layer adds the 2-of-4 emergency-block key set [fluxd/src/emergencyblock.cpp:24–30,183–190] , the consensus-enforced dev-fund remainder paid to a hardcoded address [fluxd/src/main.cpp:2877–2884], and the single transparent address to which all application payments land today by policy [measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03].

One further fact belongs here rather than being left for a reader to discover. The largest in-flight change to how applications are described on Flux — the SPEC v9 work — currently lives on an individual contributor's fork rather than in the organisation's repository: 1,167 commits ahead of the organisation's development branch, and 932 changed files with +180,317/−49,561 lines against upstream master, actively pushed [measured: gh api repos/RunOnFlux/flux/compare/development...MorningLightMountain713:feat/fluxos-v9, 2026-09-03]. That is an ordinary open-source pattern and not a criticism. For a paper arguing progressive decentralization it is nonetheless a fact about where development authority sits, and the paper states it rather than omitting it.

The order in which these move matters, and one ordering constraint is already visible: the policy channel's own stated next step — signed documents with a rollback-resistant sequence number [fluxos-network-policy/README.md:323–326] — is strictly cheaper than the quorum, requires no consensus change, and would raise the deployed mechanism from "no authentication" to "authenticated by a named key". It would not raise κ_{deployed} above 1. A threshold over publisher keys would, and is the obvious intermediate step between Definition 23.4 and the quorum.

That step is worth pricing, because the gap it closes is the largest single one in this section and it is not a design problem. As deployed, $\kappa_{\text{deployed}} = 1$ with no authentication means a compromised organisation account, or a party able to interpose on the raw-content fetch, removes any application fleet-wide within the 7 h to 13 h bound of Definition 23.4, and no node can tell that from a legitimate merge. The quorum specified in this section is a network upgrade with two new transaction types, new chain-derived state and a new coinbase rule; document signing is a week of work in a repository that has already written down what to do. **As with the attested-execution findings (§14.2), the team's disposition is to close the cheap items** [intent: design intake] — document signing here, and the deployed-service items listed above that are security tickets rather than governance design: `fluxstorage` holding environment variables and commands unencrypted with no write-side protection, a `fluxteam` privilege standing above a node owner's own administrative rights on that owner's machine, and the release API's missing verification of what it serves. The first two are cited above by file and line because those repositories are public; the third is not, for the reason §14.2 gives. None of the three requires a decision from anyone; they require a sprint. The SPEC v9 fork's location is in the same class: moving the branch into the organisation's repository costs nothing and removes a fact the paper would otherwise have to keep reporting.

23.11 Recommended values for the open parameters

Every parameter this section left open is collected here with a value. Each was proposed by this paper and **confirmed by the author** [intent: design intake]; the reasoning is retained so that any of them can be overridden with a number rather than re-argued from the beginning. Block counts assume the 30 s post-PoN target, i.e. 2,880 blocks per day [fluxd/src/chainparams.cpp:105].

Table 71: Parameter values for the content-reporting quorum. Proposed by this paper and **confirmed by the author** [intent: design intake]. PLANNED, [inferred] for the derivations except where a citation is given.

Parameter	**Recommended**	Reason
Δ_W trailing window	20,160 (7 d)	trailing *maximum*; an attacker must wait a week after any contraction, and staleness fails safe
app_hash scope	(app_name, owner)	a spec-version hash is evaded by re-broadcasting a trivially edited spec
decision field	**evict-only**	a recorded-but-uncounted "keep" is chain bloat with no effect
relay policy	**per-identity mempool quota**	fee-less votes need a non-fee scarcity term
ordering at one height	**least crossing height, then least** $H(r)$	deterministic, and first-to-quorum is the defensible priority
memo carriage	**coinbase field**	settled: consensus requires the ban memo [intent: design intake]
Δ_{exec}	120 (1 h)	3× the 40-block consensus reorg limit [fluxd/src/main.h:74]
10 %/15 % tiers	**remove**	any destructive action there is one party's decision (Definition 23.34)
vote signing key	**collateral (owner) key**	weight derives from collateral; a node key is VPS-resident (C32)
pool remainder	**burn pro-rata**	the only option giving voters a current-term cost (Definition 7.34)
evidence	**optional content hash**	mandatory evidence is unverifiable on chain; absent evidence is unreviewable
report counts as vote	**yes, at** h_0	a separate confirming vote from the reporter is a redundant transaction
m identity chamber	21	odd (no ties); Sybil pre-positioning costs $(21-\kappa)\times 1{,}000$ FLUX
T identity tenure	86,400 (30 d)	reuses the PNR constant K (Section 7); one tenure constant, two mechanisms. Buys 30 d of adversary lead time, not exclusion of the concentrated keys (Definition 23.42)
network-wide open reports	64, queued	the per-identity rule bounds Sybils, not block space; $\lvert N(h)\rvert$ concurrent reports is a quarter of chain capacity in fee-less transactions (Section 23.3.3)
voting delegation	**adopted**, revocable by supersession	cold owner keys cannot answer a 24 h window; weight and κ stay at the owner (Definition 23.9)

The four that carry real weight. Most of the table is mechanical. Four are not, and the reasoning is given rather than compressed into a cell.

Δ_W as a trailing maximum, not a mean. The electorate-timing attack of Section 23.9 works by voting while $W^\star$ is transiently low, since the 25 % threshold is a fraction of it. A trailing maximum over seven days means the denominator cannot be made to dip on the attacker's schedule: they must wait out the window. The cost is that after a genuine fleet contraction the threshold stays high — eviction becomes temporarily harder — and that is the correct direction for the error to point.

Enforcement scope is the application identity, not a specification version. If app_hash commits to one specification version, a deployer evades the ban by re-broadcasting the same application with any trivial edit, and the quorum has to vote again for each edit. If it extends to *all* applications of the same owner, the mechanism becomes collective punishment keyed on a field whose binding to a real party is exactly what Section 23.5 shows is weak. (app_name, owner) is the smallest scope that is not trivially evadable, and it matches the settled position that a banned deployer may return as a genuinely fresh application [intent: design intake].

Remove the graduated tiers. Definition 23.34 is the argument: at 10 % and 15 %, the measured concentration means a single party can reach the threshold alone. A destructive action there is therefore one party's decision wearing the costume of a quorum, which is worse than no mechanism because it carries the mechanism's legitimacy. Non-destructive actions — flagging,

warning — survive that objection but do no work: they neither stop harm nor inform anyone who is not already watching the chain. One threshold, at 25 %, is the honest design.

The owner key signs, not the node key. Voting weight is collateral-weighted, and the collateral is controlled by the owner key; the node's `pubkey` is a VPS-resident signing key (conflict C32). If the node key could vote, compromising n virtual machines would deliver n nodes' voting weight without touching a single unit of collateral — which would make the security of a collateral-weighted vote depend on the security of the least-hardened VPS in the fleet. The start-transaction shape [fluxd/src/fluxnode/activefluxnode.cpp:253–288] already carries an owner-key signature and is the right template, which is also the shape the team described [intent: design intake].

23.12 Comparison with prior art

Stake-weighted governance. Compound and MakerDAO weight votes by token balance against an absolute quorum expressed as a fraction of supply — the same shape as θW — and both have exhibited the plutocracy failure measured above. In Compound's Proposal 289 (July 2024) a coordinated holder group won a majority of votes cast — 682,190 COMP for against 633,640 — for moving 499,000 COMP into a vault it controlled, and the proposal was canceled before execution following community pressure rather than defeated by the mechanism [2]; in MakerDAO's October 2020 incident a participant used flash-borrowed MKR, roughly $7M worth, to pass a whitelisting vote and returned the tokens in the same transaction, after which Maker extended its governance security module delay from 12 h to 72 h [1]. *What transfers:* that stake-weighting concentrates decision power in proportion to capital; that delegation concentrates it further; and that a post-decision delay — Compound's timelock, Maker's pause — is the cheapest available safety margin, which here is Δ_{exec}. *What does not transfer:* flash-loan capture. Voting weight here would be *locked* collateral with 100-block maturity and a confirmation requirement, and the electorate would be frozen at h_0, so weight cannot be borrowed into a vote after the report exists. That is a genuine advantage of collateral weighting and the paper claims it. The plutocracy failure, by contrast, transfers exactly. In both incidents the decisive set was small — a coordinated holder group in one case, a single flash-borrowing participant in the other — which is the same shape as $\kappa(0.25) = 4$, with one difference that cuts against Flux: here the decisive set is a standing property of the collateral distribution rather than something assembled for one vote.

Sybil resistance. Douceur [20] establishes that without a trusted identity authority or a resource cost, a single party can present arbitrarily many identities — which is exactly why the settled weighting binds voting power to locked collateral and why every concentration remedy that decouples them (weight caps, per-identity quadratic weighting) is defeated at zero cost (Section 23.6). Personhood-based approaches do not transfer: Flux operators are businesses and machines, and no personhood attestation is available to a FluxOS node. What does transfer is the *bicameral* shape used where token weight is paired with an attested-identity house; Definition 23.42 is that structure with tenure standing in for personhood — a much weaker identity, which is precisely why it claims detectability rather than resistance.

Content moderation in peer-to-peer systems. Benet [8] describes a content-addressed store; the surrounding ecosystem publishes an opt-in denylist of hashed identifiers that each node *may* apply. Freenet has no removal at all by design; the fediverse moderates per instance by defederation; Tor exit policies and relay policies are per operator. *What transfers:* the per-node denylist is exactly the shape of Flux's deployed blocklist — except that Flux's is *mandatory* on every node and centrally authored, which makes Flux more centralised than IPFS in this one respect. *What does not:* every system above stores or relays *bytes*. Flux runs *processes* with network egress that can phish, command, spam and attack third parties, which is why a

network-wide removal — stronger than anything those systems do — is on the table at all. The scope is confined to behavioural harm (Definition 23.5), which is what keeps that removal an answer to the storing-versus-executing difference rather than a general veto over what may be said. It is also why concentration matters more here than there: a denylist a node may ignore is a recommendation; a ban carried in the chain that every node must honour is a power, whatever its stated scope.

Filecoin, Arweave, Akash. Protocol Labs [41] specifies a market in which storage providers choose which deals to accept; the protocol itself removes nothing. Arweave nodes run local content policies over what they store and serve, and the protocol cannot remove. Akash providers set per-provider acceptance attributes, and no network-wide eviction exists [4]. *What transfers:* the position that the protocol should not hold a content veto — which is the roadmap's own principle, and which the quorum honours, since the taxonomy is confined to behaviour (Definition 23.5). *What does not:* their tenants' data is inert and Flux's tenants' processes are not, and a provider-level "I will not host this" is categorically different from a network-level "nobody may host this". The quorum is the latter, and the paper presents it as a deliberate departure from the Filecoin and Arweave position on *scope of enforcement* — justified by the difference between storing and executing — while agreeing with them on *scope of subject matter*. Those are two different axes and conflating them is what makes this comparison hard to read; the paper separates them here so that a reader can disagree with one without inheriting the other.

23.13 Open parameters

The on-chain shape settles six things the earlier specification carried as parameters, and it is worth listing them as gains: the voting window ($\Delta_{\text{vote}} = 2{,}880$ blocks), the per-application cooldown after an unmet report (zero), reporter eligibility ($\mathcal{R} = I(h_0)$, by Definition 23.11), the network-wide cap on concurrently open reports (bounded at $|N(h)|$ by Definition 23.13, and set lower, at 64 with queueing, in Table 71), the entire bond question, which disappears with the bond, and producer obligation — ban-memo inclusion, over the first unbanned carried report in the settled order, is consensus-required rather than discretionary (Definition 23.19). Eleven further elements were carried as open parameters through the earlier drafts, each marked in place above with its domain and its trade-off, and all but the first are now settled at the values of Table 71: the category taxonomy, which remains open as an enumeration task (Definition 23.5); Δ_W; what app_hash hashes, which is also the enforcement scope; the decision field and the relay policy for fee-less transactions; the ordering rule when two reports cross in one slot, and how the memo is carried in the coinbase; Δ_{exec}; the actions at the graduated 10 % and 15 % tiers; which transaction shape the vote follows and hence which key signs it; the report's evidence field and whether a report is itself a vote; (m, T) for the identity chamber; and the disposition of the pool's unearned remainder.

Two further items are recorded here rather than marked in place, because they are not parameters of this mechanism. (A third, whether opt-in delegation is adopted, is now decided: it is, with revocation by supersession — Definition 23.9.) Whether FDM and PDNS honour bans by dropping routes at $h_b + \Delta_{\text{exec}}$ — without which a banned application would remain reachable at its name until the last node had removed it, and with which the Foundation-operated reachability plane acquires a new dependency on moderation state. And the sequencing question this section inherits from Section 23.10: signing the deployed policy documents is strictly cheaper than the quorum, requires no consensus change, and should not wait for it. Whether the ban set is ever pruned is no longer open: Definition 23.17 and Definition 23.24 together settle it as no — a ban is final, and the ban set only grows.

Three items cannot be set by parameter choice. Two were *verifications* in `fluxd` and both are now closed against source: whether the emergency-block path is subject to the coinbase

rules the ban memo would rely on (Definition 23.21, favourably), and whether re-addressing resets `added_height`, which decides whether the tenure gate of Definition 23.42 does any work at all (Definition 23.42: it does). The third cannot be resolved at all from public data: beneficial ownership behind the payment addresses that carry the voting weight (Definition 23.36). The first two are prerequisites, not refinements. The third is the reason $\kappa(0.25) = 4$ is printed as a measurement of *identities* and never as a count of *people*.

PART IV.5

The complete network

24 End-to-end: how Flux works, today and at target

Every preceding section has described one layer in isolation, at the depth that layer deserves. This section does the opposite: it takes a single application through its whole life — constructed, paid for, validated, placed, attested, made reachable, run, renewed, possibly reported, and ultimately the reason somebody gets paid — and follows it across every layer at once. It does that twice. The first pass is the network as deployed on 2026-09-03, written in the present tense and tagged SHIPPED throughout. The second pass is the target architecture specified in Parts II through IV, written in the conditional and tagged PLANNED throughout, with the same twelve steps in the same order so that the two can be read side by side and differenced step by step. The two passes never mix within a paragraph, and where the deployed answer is thin — there is no fee pool, there is no attestation a tenant can check, there is no grace period, there is no quorum — this section says so at the step, not in a footnote at the end.

Nothing here is derived. Every constant, mechanism and measurement is cited to the section that established it, and where two sections disagree about a constant this section says which two and marks the disagreement rather than picking one.

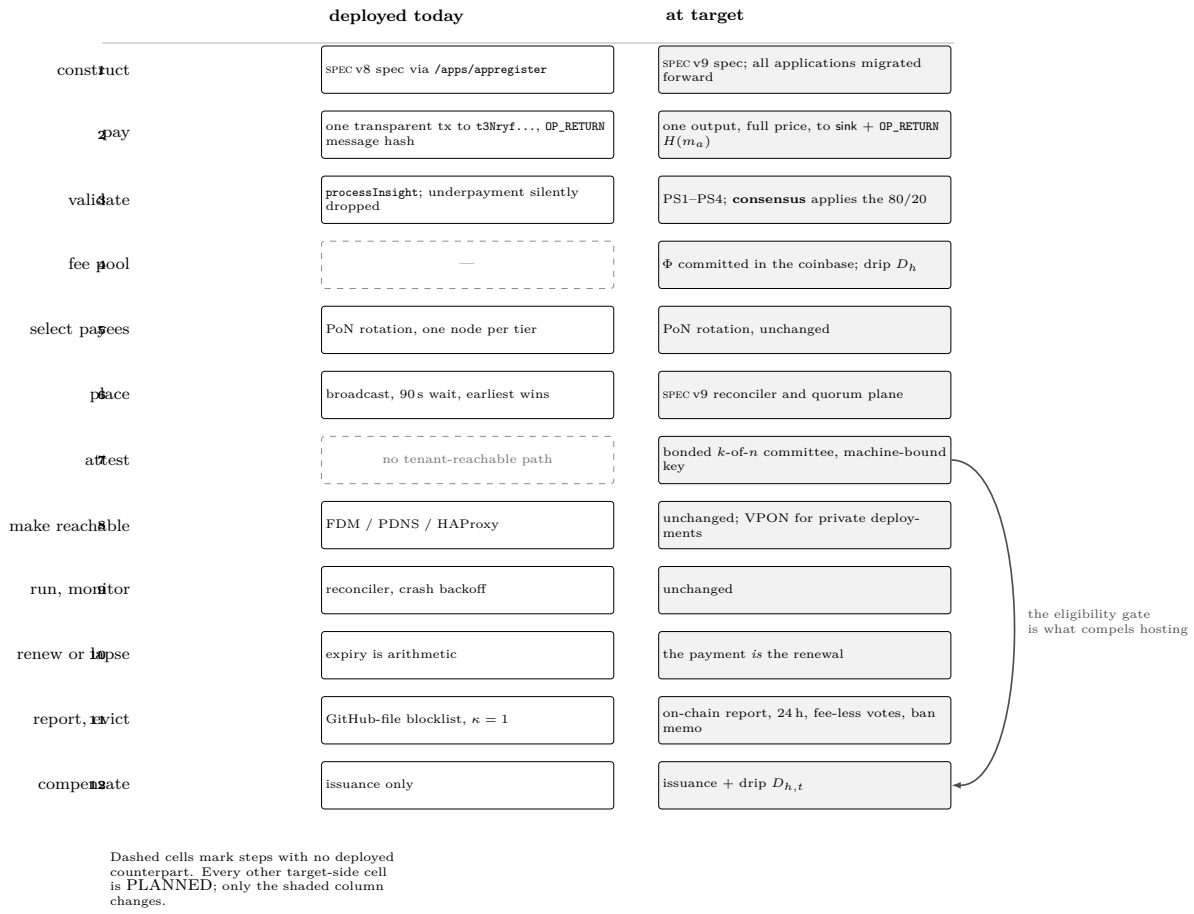


Figure 19: The twelve-step application lifecycle, deployed and at target, on a shared step axis. Two steps have no deployed counterpart at all — there is no fee pool, and no attestation a tenant can reach. The single cross-track edge is Definition 24.2: because PNR pays every node regardless of what it hosts, the eligibility gate of step 7 is the only thing that makes step 12 compensate hosting rather than mere presence.

24.1 Pass one: the deployed system

Everything in this subsection is SHIPPED and is written in the present tense.

1. A tenant constructs a specification. The document is a small owner-signed JSON object, and the version it may declare is fixed by chain height: SPEC v8 is enforced from mainnet height 1,932,380, and above that height a specification whose top-level keys are anything other than `version, name, description, owner, compose, instances, contacts, geolocation, expire, nodes, staticip, datacenter, enterprise` is rejected outright (§10) [flux/ZelBack/config/default.js:231–240] [flux/ZelBack/src/services/appRequirements/appValidator.js:1063–1067]. Seven versions are simultaneously live on the network — SPEC v2 through SPEC v8, with 322 of 1,195 deployed applications (26.9 %) below SPEC v8 — because the acceptance predicate has a ceiling and no floor (§10) [measured: api.runonflux.io/apps/globalappsspecifications, 2026-09-03]. The tenant signs the envelope {`appSpecification`, `timestamp`, `signature`, `type`, `version`}, where `type` is `zelappregister/fluxappregister` and the envelope version — distinct from the specification version — is 1, and `POST`s it to any node's `/apps/appregister` at `user` privilege, which is a `bitcoinMessage` signature over a login phrase no more than 16 h old (§9) [flux/ZelBack/src/services/appDatabase/registryManager.js:1755–1764]. The receiving node refuses the request if it holds fewer than 8 outbound or 4 inbound peers, refuses a timestamp more than 1 h old or more than 5 min ahead, and refuses a `version: 9` specification outright unless `config.fluxapps.acceptV9` is set, which defaults to `false` with no caller anywhere that sets it (§9, §10). It then computes `messageHASH` = SHA-256(`type`‖`version`‖JSON(`spec`)‖`timestamp`‖`signature`), stores the message with a 3,600 s TTL, broadcasts it, blocks through a fixed confirmation round trip of $2 \times 1{,}200$ ms plus up to 20×500 ms of polling, and returns the hash and nothing else [flux/ZelBack/src/services/appDatabase/registryManager.js:1827–1858]. The specification carries no `duration` field and no `paymentMemo` field; lifetime is expressed as `expire` in blocks, and there is no field anywhere in SPEC v8 that binds a payment to a specification other than the hash the tenant is about to put in an `OP_RETURN`.

2. The tenant pays. Price is a pure function of the specification and the height, evaluated against a height-scheduled table with six segments — genesis plus five revisions — whose current row takes effect from height 1,597,156 at $\mathbf{p} = (0.03,\ 0.01,\ 0.004)$ FLUX per unit of (cpu, ram, hdd) with surcharges $(p_{\text{port}}, p_{\text{scope}}, p_{\text{sip}}) = (0.4,\ 0.8,\ 0.4)$ FLUX and $p_{\min} = 0.01$ FLUX (§16) [measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]. The base is $B(a,h) = 10\,p_{\text{cpu}}\mathbf{r}^{\text{cpu}} + \frac{1}{100}p_{\text{ram}}\mathbf{r}^{\text{ram}} + p_{\text{hdd}}\mathbf{r}^{\text{hdd}}$ plus the surcharge terms, quoted per three instances — $\beta = \lceil B/3 \rceil_2$ per instance, scaled by the instance count from height 1,890,000 in the three-case form of Definition 16.3 — then scaled by $d/L^*(h)$ with $L^* = 4L = 88{,}000$ blocks at post-PoN heights, ceiled to two decimals and only then floored at $p_{\min}$ (§16) [flux/ZelBack/src/services/utils/appUtilities.js:89–134] [flux/ZelBack/src/services/appMessaging/messageVerifier.js:736–745]. Duration is bounded to $d \in \{1, \ldots, 1{,}056{,}000\}$ blocks — the floor is height-gated by the validator at 5,000, then 100 from height 1,630,040, then 1 from height 1,964,447, while the published endpoint still reports 5,000 [flux/ZelBack/src/services/appRequirements/appValidator.js:852–862] — and instance count to $k \in \{1, \ldots, 100\}$ for SPEC v8 applications from height 2,176,519 (§16). The tenant then broadcasts exactly one transparent transaction paying at least that amount to the sink address `t3NryfAQLGeFs9jEoeqsxmBN2QLRaRKFLUX`, in force from height 1,670,000, carrying `messageHASH` in an `OP_RETURN` (§16) [flux/ZelBack/config/default.js:241–245]. All application revenue on the network lands at that one transparent address, by policy and not by consensus (§12, §16).

Both apparent disagreements here are resolved. The duration denominator carries a PoN fork branch — 22,000 blocks before activation, 88,000 from height 2,020,000, the multiplier preserving wall-clock duration — and §7's definition now states it. The instance lower bound is 3 in general and 1 for SPEC v8 applications from height 2,176,519; the published endpoint reports 3 regardless, which is a defect in the endpoint rather than a disagreement between sections [doc: reconciliation record].

3. The payment is validated. No consensus rule sees any of this. Each node's own chain-scan loop, `explorerService.processInsight`, runs per block, recognises the payment by its output address, sums `valueSat` across matching outputs, decodes the temporary-message hash from the `OP_RETURN`, and queues `messageVerifier.checkAndRequestApp` (§9, §16) [flux/ZelBack/src/services/explorerService.js:309–345,444]. Promotion re-verifies update signatures against the current permanent owner as an explicit anti-race measure, recomputes the PoN-fork-aware expiration height e_a, and admits the specification to `globalzelapps` if and only if $v^{\text{obs}} \geq \mathcal{P} \cdot 10^8$ sat. If the payment is short, the registration is **silently dropped**: a server-side log warning, no error surfaced to the payer, no retry, and no refund path anywhere in the repository (§9, §16) [flux/ZelBack/src/services/appMessaging/messageVerifier.js:735–763]. The funds are spent and the deployment does not exist. §16 records this as the most important correctness gap on the path this paper otherwise recommends to autonomous software, and states the signed negative acknowledgement that would repair it as requirement R16.1.

4. Value enters the fee pool. This step does not exist today. There is no fee pool. There is no consensus object of any kind mediating between what a tenant pays and what an operator receives: the payment lands at a transparent address controlled by policy, and every satoshi any node operator earns is newly issued (§7). The step is listed here, empty, so that the two passes stay aligned and so that the reader sees the gap in the same position in both.

5. Consensus selects the payees. Independently of everything above, and with no reference to it, `fluxd` awards block production every 30 s. The PoN slot is $\tau(t) = \lfloor (t - t_0)/\Delta_{PoN} \rfloor$ with $\Delta_{PoN} = 30$ s, and a confirmed FluxNode with collateral outpoint c is eligible for slot τ iff $H(c \| h_{\text{prev}} \| \tau) \leq T_\tau$ — sortition over the confirmed-node set, not weighted by collateral amount beyond confirmation (§4) [fluxd/src/pon/pon.cpp:70–93]. Each block pays one node from each of the three tiers, selected from a per-tier queue ordered by `nLastPaidHeight` ascending, which is a strict longest-time-since-paid FIFO (§4) [fluxd/src/fluxnode/fluxnode.h:313–328]. The measured rotation lengths are $R_{\mathsf{C}} = 3{,}247$, $R_{\mathsf{N}} = 1{,}615$ and $R_{\mathsf{S}} = 1{,}627$ blocks, i.e. 27.06 h, 13.46 h and 13.56 h [measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03]. The payee set of the block that confirms a tenant's payment has nothing to do with that tenant, that payment, or that application.

6. The application is placed across nodes. There is no scheduler. Every node runs the same loop against its own gossiped replica of `globalzelapps` and reaches its own conclusion (§9). A node first clears a precondition gate — enterprise identity resolved, daemon synced, local database ready, not DoS-flagged, node confirmed, `fluxbench` not erroring, own external IP known, and fewer than `maxAppsPerNode` = 200 applications installed — then draws a candidate from the set of non-expired, under-replicated applications, biased first to those naming its own IP and otherwise uniformly at random [flux/ZelBack/src/services/appLifecycle/appSpawner.js:78–336]. After targeting and geolocation filters and a set of soft-deferral branches, it verifies the image against the Docker Hub whitelist, pulls it, probes port availability through its peers, broadcasts a `fluxappinstalling` message carrying its own IP and a self-reported `broadcastedAt`, and then waits `installCollisionWaitMs` = 90 s [flux/ZelBack/config/default.js:321]. It re-reads the global installing and running lists, ranks all installing nodes by `broadcastedAt` ascending, and abandons the attempt if its rank does not fit the remaining slots. **Earliest-broadcast-wins is the entire conflict-resolution mechanism**: no leader election, no quorum, no on-chain arbitration [flux/ZelBack/src/services/appLifecycle/appSpawner.js:686–703]. Installation proceeds through `registerApp Locally` into Docker, and from that point every start and stop of the container goes through one component, the reconciler, described in its own source as the single level-based actuator, with a crash-backoff ladder of [0, 30,000, 300,000, 900,000, 1,800,000] ms reset after 600,000 ms of stable running (§9) [flux/ZelBack/config/default.js:130–131]. Sixty seconds after installing, the node

re-ranks itself by `runningSince` and uninstalls immediately if it is now surplus. Convergence is not agreement; it is two opposing correction loops, and both of them correct over-installation while neither corrects under-installation faster than the next node's random draw (§9).

7. The host is attested. Nothing a tenant can check. `fluxsas`, the attestation service, serves a single listener on `127.0.0.1:16100` with mutual TLS mandatory and authorisation by client-certificate common name — `fluxbench` for every route, `fdm-nginx` for three — so an external tenant has no network path to it at all (§14) [fluxsas/src/api_server.h:1042–1053,220–252]. Even with a path, the signature would not say what a tenant needs: the Ed25519 key used by `/v2/attest` for the default, `cdn`, `mesh` and `app` purposes is derived from static values compiled identically into every `fluxsas` binary, so a valid attestation demonstrates possession of something every copy of the software contains rather than that a particular machine booted a particular image (§14) [private source: SAS API server]. One layer down, the eligibility gate that admits a node to the paid set at all is a benchmark signature produced by a process on the operator's own host and verified against `vecBenchmarkingPublicKeys`, five network-wide keys with timestamp validity windows — collateral binds the tier, and the benchmark signature does not bind the hardware to it (§4) [fluxd/src/chainparams.cpp:233–240]. What *does* do real work is the boot chain: a Flux-controlled Secure Boot Platform Key pinned by hash inside `fes`, fail-closed exit codes 50–57 that power the machine off rather than continue, a TPM2 seal against PCR 7, and a dm-verity root filesystem (§14). But the allow-list of known-good verity root hashes is served by two Flux-operated web endpoints that **fail open** — if both are unreachable, the watchdog treats the node as secure and does nothing [private source: SAS watchdog]. The honest summary of this step is that measured boot imposes real cost on a remote attacker, and that the assurance a node offers is a Flux-operated one rather than a hardware-rooted, tenant-verifiable one.

8. FDM and PDNS make the application reachable. The reverse path from a hostname to a container is Foundation-operated end to end (§12). FDM does not talk to `fluxd` or to any peer-to-peer layer: it polls one Foundation-operated REST API, `api.runonflux.io`, for `apps/globalappsspecifications` every 30 s under an `ETag`, `apps/locations` every 10 s unconditionally, and permanent messages every 120 s, and no fallback source exists anywhere in the codebase [flux-domain-manager/src/services/flux/dataFetcher.js:47–91]. Names are computed by string concatenation — a_P`.app2.runonflux.io` per port and a`.app2.runonflux.io` as the main alias — and served either by Cloudflare in strict DNS-only mode or by the self-hosted PDNS zone service. The FDM host's own HAProxy terminates TLS against every `.pem` under `/etc/ssl/fluxapps/` and routes on the `Host` header to one of the application's container IPs, with per-server health checks of `inter 3s fall 2 rise 2 fastinter 500`, i.e. an approximately 6 s detection floor [flux-domain-manager/src/services/haproxyTemplate.js:260–319]. Sixteen Foundation-run FDM/CDM hosts serve this path for the whole network; roughly two dozen game-server application types bypass HAProxy entirely through a single, unreplicated `flux-apps-dns-manager` instance (§12). Inside the fleet, container-to-container names resolve through `flux-dnsd`, which despite the name is a purely local file-tail resolver over a `membership.json` that FluxOS writes, with a fixed 5 s answer TTL and `REFUSED` rather than `SERVFAIL` when it holds no snapshot (§12).

9. It runs and is monitored. The reconciler holds the container to its desired state, resolved from four inputs in strict priority order with the operator's own lock winning over all (§9). Around it run the periodic monitors: a node-status monitor every 1.5 min, an application-availability check every 3 min, a CPU-usage check every 900,000 ms, an image-compliance sweep every 3,600,000 ms, and a storage check every 20 min (§9) [flux/ZelBack/src/services/serviceManager.js:433–563]. Location records carry a 7,500 s TTL, and a departing node's `fluxnodesigterm` broadcast extends its applications' location TTL by `SIGTERM_EXPIRY_MS` = 420,000 ms rather than expiring them at once. Where the operator has opted an application in, `flux-telemetryd` ships per-

container telemetry to a customer-chosen backend every 15 s. Crucially, none of this is metering: Flux prices reservation, not consumption, so there is no usage counter to read, to attest, or to dispute (§16). The only disputable fact is the binary one — did the application run — and that is observable by third parties from location records, FDM health checks, and the tenant's own probes.

10. It renews or lapses. Expiry is arithmetic, not a decision. Entitlement is the predicate $\mathrm{Ent}(a,h) = \mathbf{1}[h \le e_a]$, where e_a was fixed at promotion from (h_0, d) and is a function of the confirmed chain prefix alone (§16). Every node evaluates it independently and reaches the same answer, and a periodic pass every `expireFluxAppsPeriod` = 100 blocks removes what has passed [flux/ZelBack/config/default.js:298]. A renewal is an update, priced at $\max\{p_{\min},\ 0.9\,(\mathcal{P} - \varpi\mathcal{P}_{\mathrm{prev}})\}$ where ϖ is the unused fraction of the previous term (§16). **There is no grace window, no suspension state, and no data-retention guarantee**: when the paid-through height arrives the entitlement stops, and the removal path is the ordinary one. That is a consequence of the design rather than an omission — Property "No standing obligation" in §16 shows that any grace state would have to be a function of information not in the chain prefix — but it is a real difference from what a tenant used to hyperscaler billing expects. The only automatic renewal that exists is `appsmonitor`, a centrally operated daemon run by InFlux that renews the team's own applications with owner keys it holds and extends card-paid and free-tier applications through the single `usersToExtend` key that FluxOS allows to extend any application, alongside a hardcoded single-address `usersToExtend` allow-list permitted to submit expire-only extensions on any owner's behalf (§16).

11. It may be reported and evicted. There is no quorum. Moderation is a publisher: FluxOS fetches `helpers/blockedrepositories.json` from `raw.githubusercontent.com/RunOnFlux/flux`, caches it, and every 1 h removes every locally installed application whose image, owner address or specification hash matches, spacing removals 3 min apart (§9, §23) [flux/ZelBack/src/services/appSecurity/imageManager.js:179–195,644–676]. The wider `fluxos-network-policy` channel carries the same shape for blocked and vetted repositories, the enterprise-node owner map, a node-tampering blocklist and a ∼2.0-million-row geolocation table, fetched every 6 h or 12 h and enforced fleet-wide with no canary and no staged rollout (§12). The documents are unsigned; by the component's own account their authenticity rests on GitHub and on who can merge, and branch protection deliberately imposes no second-reviewer requirement. §23 states the resulting measure exactly: the number of parties whose agreement removes an arbitrary application from every node is $\kappa_{\mathrm{deployed}} = 1$, the authentication on the decision is nothing, the record a node can verify is nothing, and the latency to fleet-wide effect is bounded by 7 h (13 h for the tampering list).

12. Operators are compensated. From issuance alone. For $h \ge h_{PoN} = 2{,}020{,}000$ the block subsidy is $\sigma_{PoN}(h) = s_{r(h)}$ with $s_0 = 14\,\mathrm{FLUX}$, $s_{k+1} = \lfloor s_k \cdot 9/10 \rfloor$, and $r(h) = \min(\lfloor (h - h_{PoN})/I_{\mathrm{red}} \rfloor,\ N_{\mathrm{red}})$ for $I_{\mathrm{red}} = 1{,}051{,}200$ blocks and $N_{\mathrm{red}} = 20$ (§5) [fluxd/src/main.cpp:2799–2816]. Of that, the three tier payees receive $\lfloor \sigma_{PoN}(h)\,\beta_X/14\,\mathrm{FLUX} \rfloor$ with $(\beta_{\mathsf{C}}, \beta_{\mathsf{N}}, \beta_{\mathsf{S}}) = (1,\ 3.5,\ 9)\,\mathrm{FLUX}$, and the remainder $\delta(h) = 0.5\,\mathrm{FLUX}$ at $r = 0$ — 1/28, or 3.57 %, of PoN emission — is a consensus-enforced payment to the hardcoded address `t3hPu1YDeGUCp8m7BQCnnNUmRMJBa5RadyA` (§5) [fluxd/src/main.cpp:2877–2895]. Emission does not stop: after twenty reductions the subsidy freezes at 1.70207313 FLUX per block, i.e. 1,789,219.274256 FLUX per year, indefinitely (§5). **Nothing the tenant paid in step 2 reaches any of these payees.** The tenant's money is at one address and the operator's money is newly minted, and the only thing connecting them is that both happen on the same chain.

24.2 Pass two: the target architecture

Everything in this subsection is PLANNED. Nothing in it is deployed, no part of it is scheduled to a height except where a height is explicitly named as proposed, and no sentence below should be read as a statement about the network as it runs. Where the target depends on something unbuilt or undecided that dependency is named at the step, in the same paragraph.

1. A tenant would construct a specification. The document would be a SPEC v9 specification carrying `duration` as a first-class field, a `paymentMemo` of up to 512 characters in the draft schema, a `referralCode`, and a `machineType` drawn from {`standard`, `gpu`, `vps`}, with the four SPEC v8 renames applied and `enterprise` removed (§10). Applications already deployed would not be left behind on older versions and would not be offered the choice: the settled migration is **forced**, an application-update message carrying a special signature that rewrites every application forward onto SPEC v9, after which updates on older specification versions would be disabled and, once the fleet had moved, the legacy validation paths removed (§10) [intent: design intake]. That supersedes the version-floor-plus-deprecation design and is the better answer to the 322-application, 26.9 % legacy tail of step 1 of the first pass, because it would clear the tail at one moment rather than letting it trail off at each owner's own pace. It also creates a capability this paper names rather than lets a reader find: a key able to rewrite a tenant's signed specification without that tenant's signature, which §10 places in the same class as the unsigned network policy, the release channel and the emergency-block keys, and which the team's decision bounds as single-use, migration-scoped and hardcoded — with no opt-out for an owner who would rather remain on the document they signed (§10, C40).

Five things are undecided or unbuilt at this step and each is load-bearing. SPEC v9 has no enforcement height in shipped configuration or on the live pricing endpoint — a height of $h^\star = 3{,}050{,}000$ was proposed in the design intake (§10), but SPEC v9 remains in development and has not activated — and §10 states the ordering constraint that the height must *follow* implementation of the three economic fields rather than precede it. `paymentMemo`, `referralCode` and `machineType` have **zero consumer code anywhere in FluxOS** (§10) — they are exactly the fields PNR, referrals and the VPS tier are each stated to depend on. The RFC-0 draft has no signature field at all and states its canonicalization rules as prose `MUST`s with no schema or code enforcement, which for a document meant to be compared byte-for-byte across roughly 6,500 independent validators is the defect to name (§10). The removal of `enterprise` arrives with no declared replacement, so SPEC v9 as drafted cannot express a private deployment at all, against 18.7 % of currently deployed applications that use it (§10). And the migration key's scope — whether `fluxd` itself would reject a forced update touching anything beyond the version field and the four renames, or whether that restraint would live only in the tooling that constructs the message — is open in §10, as is whether the key is retired when the migration completes rather than left live in a binary. Where the submitter is autonomous software rather than a person, the specification would travel with a delegation certificate $\mathsf{cert} = (pk_{\mathcal{A}}, \Sigma, \mathcal{B}, [t_0, t_1], \mathsf{rev}, \nu)$ signed by the principal's full 2-of-2 ceremony, and a node would accept it by a predicate computable from the message, its own chain state at h, and nothing else (§17). The deployment half of that authority composes with machinery that already runs; the spending half does not, and §17 classes it as not achievable on the Flux L1 without either a new trusted party or new consensus state.

2. The tenant would pay. One transaction, one recipient, one memo. The settled shape is a single transparent output paying the full price $\mathcal{P}(a, h)$ to sink — one application-payment address carried as a consensus constant from h_{PA} onward — together with one zero-value `OP_RETURN` whose data field is the 32-byte $H(m_a)$; these are rules PS1 and PS2 of §7 [intent: design intake] [doc: reconciliation record]. There would be no second output, no fee component and no new transaction type. The payer would construct one recipient and its own change and would perform

no other arithmetic, so any wallet able to pay an address and attach a memo — and any agent holding a bare wallet API — could construct a valid payment without linking a Flux-specific library or waiting for tooling to be rolled out. §7 records the deployed CumulusVPN entitlement path as the existence proof for exactly that shape, and records the two shapes not taken and why: a fee-carried share would make the amount a function of the sending wallet's change calculation, and a typed transaction would put a library dependency in front of every payer. Direct per-deployment settlement would remain the default and the recommended path for autonomous software, for the reason §16 proves: entitlement is a function of the confirmed chain prefix alone, so no account, no stored credential and no relationship outliving the deployment would be involved. The alternative would be the custodial path §16 documents as SHIPPED, in which a facilitator holds the user's keys and pays the chain for them, and over which §16 recommended that a recurring charge be authorised by a §17 certificate whose scope Σ is restricted to *renew this application at no more than this price* under budget $\mathcal{B}$, because that is the only one of the three candidate authorities that removes the custodian rather than relocating it — a recommendation the settlement has since overtaken, since the protocol is to offer no automatic renewal at all and recurrence stays in the service layer, by custody (Definition 16.25). Two of this step's open decisions were closed by the settlement: O3, the memo's semantics, is $H(m_a)$ in an `OP_RETURN`, and O18, the serialisation and the enforcement boundary, is the single-output shape with the split applied by consensus. Two remain. O14, the identity of sink, is unfixed — keeping the address in use today preserves tooling and one continuous history but mixes pre-h_{PA} spendable receipts into the counter of step 4, and a consensus-level address is a consensus-level commitment. O8, which payment channels must settle through sink at all, is the load-bearing one, because every channel outside it is revenue the consensus split never touches. Where a deposited balance would live — on-chain, FluxOS-held or wallet-held — was the architectural choice §16 could not make, alongside the authorisation object; both are now settled out of protocol scope, the balance held off-chain by a third-party operator and no authorisation object existing at the protocol level at all (Definition 16.28, Definition 16.25).

3. The payment would be validated. Two consensus rules would run at $h \geq h_{\mathrm{PA}}$, and neither would involve the payer. PS3 makes any output paying sink unspendable, so a block containing such a spend would be invalid and the value would leave circulation at the moment of payment. PS4 divides what arrived: $A_\Phi = \lfloor r(h)\, v/100 \rfloor$ with $r(h) = 100\,\rho(h) \in \mathbb{Z}$ is credited to the fee pool, and $A_F = v - A_\Phi$ is credited to the Foundation, the single floor taken once per received output and favouring the Foundation by strictly less than 1 sat (§7). FluxOS would check three things and no others: that a payment exists, that it carries $H(m_a)$, and that it covers $\mathcal{P}(a, h)$. **The payer splits nothing and can therefore get nothing wrong, and FluxOS has no split to police.** That is the point §7 makes about the decision rather than about the outcome: choosing the single-output shape for *wallet compatibility* also removed the *enforcement* ambiguity, and both properties came from one choice (C38). Three consequences follow and §7 proves them together. A wallet could not mis-round or omit a share that it never computes. A FluxOS release could not alter $\rho(h)$, the tier partition or the destination of either share, because those would be evaluated in `ConnectBlock` rather than in FluxOS, and a node running a changed rule would be rejected by every other node. And the coinbase would gain no output, because both shares would be added to outputs that already exist. What consensus would *not* fix is the amount: FluxOS would still compute $\mathcal{P}$ from the height-scheduled table, and §7's formulation is the honest one — *consensus would fix the distribution, policy would fix the base*, against a table revised five times with the CPU rate falling 100-fold. O7 — whether received value can be visible to consensus while the payer is shielded — is moot: both shielded pools are to be retired (§20.9), so all settlement is transparent. Whether the table's rebasing becomes rule-based or oracle-driven is O9, settled as rule-based in §7.13, which leaves what the rule is open. The signed negative acknowledgement of requirement R16.1, which would make an

underpaid registration observable to its payer, is specified in §16 and is not built — and it is the one repair this step still owes an autonomous payer.

4. Value would enter the fee pool. This is the step with no deployed counterpart, and it is the mechanism the whole target economy rests on. The fee pool would be a single consensus-state integer $\Phi_h \in \mathbb{Z}_{\geq 0}$ in sat with $\Phi_{h_{\mathrm{PA}}-1} = 0$, *committed in the coinbase* of every block from h_{PA} and recomputed by every validator from the parent value and the block body as

$$\Phi_h \;=\; \Phi_{h-1} \;-\; D_h \;+\; \mathrm{receipts}_h,$$

with a block whose committed value differs from the recomputation being invalid (§7, rule PC) [intent: design intake] [doc: reconciliation record]. That closes O2, and the reasons §7 gives for it are the ones that matter to a reimplementer rather than to a reader: reorg-exactness would come free, because the pool would be a pure function of the chain, so a reorg would recompute it with no rollback journal and no way for the pool to diverge from the UTXO set; no new database would be introduced, because the coinbase parsing and enforcement path the commitment needs is the one `fluxd` runs every block for the dev-fund minimum and the three tier payouts SHIPPED; and a divergent implementation would be rejected at the block where it first arises rather than one block later. §7 rejects a separate store on this network's own precedent — the FluxNode registry needed a crash-recovery marker and a repair RPC precisely because a side database can diverge from the coins database — and rejects a header field as the widest-blast-radius fork available.

Nothing would be added to the coinbase at any point in the mechanism. The Foundation's 80 % would flow through the *existing* dev-fund output, the operator drip would be added to the *existing* three tier outputs, and the pool commitment itself would occupy the coinbase's data-bearing field rather than a value-bearing one (§7, I4). Supply would be conserved by construction and not by bookkeeping: PS3 removes what is paid to sink from circulation and the coinbase re-issues it, so circulating supply would differ from §5's issuance schedule by exactly the pool balance in flight, $S'(h) = S(h) - \Phi_h$, which at steady state is 30 d of the operator share. The same rule turns the sink's balance into a *public cumulative-revenue counter*: the gross application revenue settled through this path since h_{PA} would be the balance of one address, queryable by anyone, publishable by no party in particular and withholdable by none — which is what would let §26 replace an assumed 100 FLUX per application per month with a measurement, and what would make O17 closable by observation rather than by disclosure. One qualification travels with the counter and must not be dropped: it would measure only what settles through this path, so every channel left outside it by O8 is revenue the counter would not see and the split would not touch.

Four things about the pool were undecided, all four are now settled (three at Table 24, O21 at Definition 7.10), and two hard security requirements stand regardless. O11, whether the pool is seeded at activation, decided whether the mechanism ramps over roughly ninety days or switches on — it is settled at no seed, because a seed would be a one-off Foundation transfer to operators and would have to be labelled one. O5 and O6, the destinations of a tier share with no payee and of the ≤ 2 sat partition residue, are worth almost nothing per block and are nonetheless fixed explicitly — both stay in Φ — because an implementation difference in either is a chain split. O21, created by the settlement itself, is settled on the same-block option (Definition 7.10): the division is applied on arrival, at the ρ in force at the receipt height, and the Foundation share is paid through the dev-fund output of the block that receives it, which needs no second accumulator and imposes no lag but makes that output depend on the block body rather than on the parent state alone — a property the one-block deferral would have restored, and which is given up because it serves a verifier §22 establishes cannot exist for Flux. The requirements are S1 and S2. An emergency block, produced through the hardcoded 2-of-4 key path with no time or stall gating, must not drip, or the holders of two of four keys could accelerate the release of accumulated fees — Φ_{h-1}/K per block, paid to whichever nodes head

the queue, including any they operate — on every block they produce; they could not redirect it, because an emergency block still fills the deterministic tier payout (§7). And forfeitures from the moderation mechanism of step 11 must never enter Φ (§7).

5. Consensus would select the payees, and the pool would drip to them. The rotation would be unchanged: PNR would consume PoN's payee set and add no selection mechanism of its own (§7). Each block would drip $D_h = \lfloor \Phi_{h-1}/K \rfloor$ from the *parent* state, partitioned by tier as $D_{h,t} = \lfloor 2\beta_t D_h/27 \rfloor$ over the integers $2\beta_t \in \{2, 7, 18\}$, giving $(\lambda_{\mathsf{C}}, \lambda_{\mathsf{N}}, \lambda_{\mathsf{S}}) = (2/27,\ 7/27,\ 18/27) = (7.41\,\%,\ 25.93\,\%,\ 66.67\,\%)$, and the existing tier output would carry $\sigma_t(h) + D_{h,t}$ rather than a new output. The pool would update as $\Phi_h = \Phi_{h-1} - D_h + \epsilon_h + U_h + F_h$. Two properties follow and both matter for how a reader should read this step. At steady state the drip equals the inflow, so the pool is a delay line with time constant K blocks and not a treasury that anyone could choose how to spend (§7). And because the drip is taken from the parent state and is linear in it, an operator who is also a tenant and times a payment to land the block before its own node is paid recaptures exactly λ_t/K of its contribution — 7.72×10^{-6} for a Stratus node at the settled K, three orders of magnitude inside the design's own requirement (§7). The drip constant is settled at $K = 86{,}400$ blocks, exactly 30 d at 30 s spacing, which closes O1 [intent: design intake]. The trade-off it resolves does not disappear with the decision, and §7 restates it so that a later revision starts from it: within-tier spread $\approx R_{\mathsf{C}}/K$ and targeted recapture $\approx \lambda_t/K$ both fall with K, while lag, activation ramp and steady-state pool size FK all rise with it, over the domain $[10^4,\ 2 \times 10^5]$ blocks in which the constant was chosen. Because K would be a consensus constant, revisiting it would be a network upgrade rather than a configuration change — so what accompanies the value is O12, settled as a documented review trigger at $2\times$ the current Cumulus count rather than an automatic rule, since at $4\times$ the decay-side spread reaches 16 %.

6. The application would be placed across nodes. Placement would run under the SPEC v9 reconciler: roughly 25 focused service packages including `appLifecycle`, `appMesh` and `appPlacement`, a drain server exposing `drain_app`/`stop_app` over a UNIX-socket JSON-RPC endpoint, and per-application-CA backend TLS under which a node generates a local Ed25519 keypair and obtains a 30 d host certificate signed by the application's own certificate authority through `fluxbench`, renewed automatically (§10). That substrate is in development and under test — 289 unit-test files with 7,763 test cases run on every push — but it is not merged or shipped, and it lives on an individual contributor's fork rather than in the organisation repository, which §10 and §23 both record as a fact about where development authority sits (§10). Two things are inert. `quorumGrantActivationHeight`, the height gate for the fork's mastership and active-standby quorum plane, is read by production code but set only in integration-test fixtures and is absent from shipped configuration (§10). And the conflict-resolution rule itself would be unchanged: nothing in any drafted section replaces earliest-broadcast-wins, so the target inherits both the 90 s wait and the honesty assumption on `broadcastedAt` that §9 states.

Decided. Earliest-broadcast-wins is retained at target. No section of this paper specifies a replacement, and on reflection none is needed for the reason Definition 7.29 gives: because PNR pays no marginal reward for hosting, winning a placement race carries no direct income, so the race has no financial stakes and the incentive to game it is weak. A deterministic scheduler would be preferable on other grounds — predictability, capacity balancing — but it is a FluxOS design question, not a consensus one, and replacing a mechanism that is not currently being gamed is not a prerequisite for anything else in this paper. [inferred].

7. The host would be attested. The proposed answer is not a better key but a design that stops depending on a key and depends on collateral instead (§14). A node would opt into the attested tier by posting a bond β_i *separate from* its FluxNode collateral, which would

remain self-custodied and untouched by any slashing path. A claim would be a tuple $\mathsf{cl} = (\mathsf{subject}, \mathsf{predicate}, \mathsf{value}, h, \mathsf{nonce})$ signed under the node's key; a committee of n bonded nodes would be sampled deterministically as $\mathcal{C}(\mathsf{cl}) = \mathrm{Top}_n(i \mapsto H(\mathsf{subject} \| \mathsf{prevHash}(h) \| i))$, reusing the sortition `fluxd` already implements for PoN slot eligibility; a claim would be accepted at k of n identical signatures, challengeable for a window Δ_{chal} by any party posting β_{chal}, and a resolved challenge would slash each original signer by a fraction s (§14). Requirement 0 gates the entire construction: the attestation signing key would have to be derived from or sealed to a per-machine hardware root, which Property "Attestation is not machine-bound" in §14 says it is not, and a tenant-reachable verification path would have to exist where today there is none. Absent the first condition a bond would secure a signature anyone holding the software can produce, making it forfeitable for statements the bonded party never made. The four committee parameters are recommended by §14 at $n = 9$, $k = 6$, $\Delta_{\mathrm{chal}} = 2{,}880$ blocks and $s = 1$, and the bond-sizing function at a floor $\beta_{\min}$ or a fixed fraction of the value secured, whichever is larger, with both coefficients still to calibrate: β_i must scale with the value secured by the claims a node is sampled to sign, not with its FluxNode tier, because a fixed bond cannot secure an unbounded reserve (§14). Admissible claim classes are likewise recommended as the objectively re-checkable class only at launch, §14 explicitly declining to invent a resolution procedure for general computation-integrity claims.

8. FDM and PDNS would make the application reachable. This is the step where the target is least specified, and the paper should not pretend otherwise. The requirement is legible from §12's centralization inventory: `api.runonflux.io` is the routing layer's (FDM's) sole application-discovery source, with no on-chain or peer-to-peer fallback coded in FDM (the nodes themselves replicate specifications and locations by peer-to-peer gossip, §9), so any decentralization of reachability starts with a discovery fallback that does not exist. Two further interfaces would be needed and neither is designed. First, §23 records as open whether FDM and PDNS honour a ban by dropping routes at $h_b + \Delta_{\mathrm{exec}}$, where h_b is the height of the coinbase carrying the ban memo of step 11 — without which a banned application would remain reachable at its name until the last node had removed it, and with which the Foundation-operated reachability plane acquires a new dependency on moderation state. Second, private overlay deployments (VPON s) are a roadmap item with no wire protocol, key-management scheme or placement-isolation mechanism specified anywhere; §12 states the two requirements a VPON would have to meet — membership gated by possession of a private key rather than by resolvability, and a reachability plane not positioned to learn which nodes host a private deployment or on whose behalf — and states that the construction is open. The shipped CumulusVPN transports (AmneziaWG obfuscation, WireGuard-over-TLS, client-side multi-hop) are cited by §12 as building blocks a future design would not have to invent, not as a design.

9. It would run and be monitored. The target changes less here than anywhere else, and deliberately. Reservation pricing would remain, so there would still be no metering on the application path, no counter to attest and no counter to dispute; §16 notes that this is the same choice §7 makes one layer down when it removes any metering or attestation dependency between payment and placement. What SPEC v9 would add is drain-aware shutdown: marking a component draining would trigger an immediate presence rebroadcast, so the signal a routing layer consumes rides the gossip path FluxOS already uses (§10). The companion `flux-shutdownd` pipeline is IN-PROGRESS— its signal-and-cleanup stages are wired to production events, while drain-broadcast and `preStop` execution are modelled and unit-tested but not yet reachable (§9). Whether the target architecture should acquire an availability remedy at all is settled by §16 as no, at the protocol layer (Definition 16.27): the minimum viable input would be a third-party-observable availability record, which does not exist, and the cost would be reintroducing the metering dependency the design removed.

10. It would renew or lapse. Two paths, and conflating them is the error this step exists to prevent. On the default path there would be nothing to renew, because **the payment is the renewal**: a tenant wanting another term would send another payment of the shape of step 2, and entitlement would remain what §16 proves it to be — a function of the confirmed chain prefix alone. There would be no standing obligation, no balance to track, no charge to authorise, and *no lifecycle state machine at all*, so there would be nothing for a policy to decide and no state a policy could get wrong. That is a consequence of the design and not a gap in it: §16's corollary is that a grace or suspension state would have to be a function of information absent from the chain prefix, so the direct path cannot carry one without surrendering the property that makes it the recommended path for autonomous software. Expiry would stay arithmetic.

The second path would remain custodial, and §16 documents what it is as SHIPPED rather than projecting what it might become: a *prepay-and-forward facilitator* that holds both of a user's private keys — the identity key whose address is written into the specification's `owner` field, and the payment key — charges a card, and pays the chain on the user's behalf, keeping no user balance and no withdrawal path (C42). The four-state machine $\mathrm{st}(a,h) \in \{\mathsf{funded}, \mathsf{grace}, \mathsf{suspended}, \mathsf{evicted}\}$ belongs to something else: a deposited-balance credits layer that §16 specifies in full and marks explicitly as unimplemented. Were it built, a renewal would fire at $h = e_a - \Delta_{\mathrm{pre}}$ if the balance covers $\mathcal{P}^{\mathrm{ren}}$; a lapse would move the application to grace, where it would keep running and keep its data; after G_{g} blocks it would move to suspended, containers stopped but not removed and volumes retained; and only after a further G_{s} blocks would it be evicted and its data destroyed. The separation of stopping from deleting is that layer's design commitment and it has a price §16 bounds exactly: revenue forgone over the whole window is at most $k_a\, p_{\mathrm{hdd}}(h)\, \mathbf{r}^{\mathrm{hdd}}(a)\, (G_{\mathrm{s}} + G_{\mathrm{g}})/L^{*}(h)$, linear in the declared disk footprint and independent of the CPU and RAM reservation. Its four parameters are settled at Table 41 — $G_{\mathrm{g}} = 2{,}880$ blocks from the domain $[240, 2880]$, $G_{\mathrm{s}} = 20{,}160$ from $[2880, 43200]$, $\Delta_{\mathrm{pre}} = 120$ from $[20, \infty)$, bounded below by confirmation depth plus propagation, and the resumption charge $\mathcal{P}^{\mathrm{res}}$ at one renewal period from $[0, \infty)$ FLUX — and all four are service-layer values, because the two architectural choices they were downstream of are settled out of protocol scope: a deposited balance lives off-chain with a third-party operator, and nothing in the protocol authorises a recurring charge (Definition 16.28, Definition 16.25). One requirement crosses from §7 and constrains any answer: after h_{PA} every application-hosting charge, whatever the payment channel, must settle as exactly one L1 payment to sink, or the consensus split is void and the revenue counter of step 4 under-reports — which is why a credit balance may not become an off-chain ledger of hosting charges. And whichever path a tenant took, the invariant that makes the custodial one optional rather than structural would hold: for every deployment reachable through a custodian or through credits, the same deployment is reachable at the same height through an explicit transaction the tenant signs and broadcasts itself (§16).

11. It could be reported and evicted. Every step of the decision would be a chain fact, and that is the whole difference from step 11 of the first pass. A report would be a `fluxd` transaction of a new FluxNode-family type carrying the application hash to be banned, shaped after the deployed start and confirm transactions and *indexed*, so that any node could enumerate the open reports on an application from its own chain state; it would carry no fee and no bond, and its confirmation height h_0 would fix the canonical clock and *freeze the electorate* (§23) [intent: design intake]. A window of $\Delta_{\mathrm{vote}} = 2{,}880$ blocks — 24 h at 30 s spacing, chosen short on the team's reasoning that removal often needs to be fast — would open on that confirmation. Each vote would be a second special transaction, signed with a node *owner* key and **fee-less**, which is what removes the disenfranchisement objection: no operator would need spendable FLUX at the key that carries its collateral in order to vote. `fluxd` would count the quorum internally from those transactions, so there would be **no off-chain tallier**, no certificate for anyone to assemble and no gossiped object anywhere in the decision path — the property §23 proves as public

verifiability, under which two honest nodes holding the same canonical chain hold the same enforcement set exactly rather than up to gossip delay. Weight would be collateral-weighted at one unit per 1,000 FLUX, so per node $\mathsf{C} \mapsto 1$, $\mathsf{N} \mapsto 12.5$, $\mathsf{S} \mapsto 40$ against a measured total $W = 88{,}514.5$ units, and the rule would be $Y_r(h_c) \geq \theta \cdot W^{\star}(h_0)$ with $\theta = 0.25$ as an *absolute* fraction of total weight and abstention counting as "no".

On the threshold being crossed the block template would change: the PoN producer would carry a *ban memo* in the coinbase, **at most one per block**, so bans would serialise at one per 30 s slot and the coinbase would grow by at most one fixed-size memo however many reports were open — the same constraint §7 respects when it routes the revenue split through outputs that already exist. FluxOS would observe the memo and purge, reusing the removal path the deployed blocklist sweep already exercises, after a confirmation delay Δ_{exec} that exists because a purge is not locally reversible. A report that did not reach threshold inside its window would simply expire: nothing removed, nobody penalised, the reporter's capacity released, and the same application immediately re-reportable with no cooldown — which §23 argues is the mechanism working rather than failing, because abstention counts against removal and most reports are expected to end this way. Anti-spam would be priced in the resource that already prices voting: an identity could hold at most as many concurrently open reports as it operates confirmed nodes, which is Sybil-neutral because re-keying moves nodes between owner keys without changing the sum, caps the network at $|N(h)| = 6{,}489$ concurrent reports, and prices sustained nuisance at 1,000 FLUX of locked, recoverable collateral per slot — with no bond, no custody script and no forfeiture destination to argue about.

Two properties are worth carrying into this composition. Turnout is a liveness parameter and never a safety parameter, because the threshold depends on the electorate and not on who votes, so low turnout makes eviction strictly harder (§23). And the safety property is exactly a collusion bound, which at the measured distribution is $\kappa(0.25) = 4$ by owner identity — the operative figure, since votes are signed with owner keys — falling to 3 once addresses linked by a shared node key or a shared collateral funding transaction are merged: safety against wrongful eviction would not be a property of the mechanism but the assumption that three or four specific parties do not agree (§23). The category taxonomy was the single open parameter on which the mechanism's consistency with the roadmap's own published "behaviour, not content" principle depended; its scope is now ruled on — the taxonomy is confined to behavioural harm, so the principle stands and the quorum is an enforcement path for it (Definition 23.5) — and only its enumeration remains open. The rest of what this composition first carried as undecided is settled at the values of Table 71: the banned hash is over $(\mathsf{app_name}, \mathsf{owner})$, which is also the enforcement scope; inclusion of the ban memo is consensus-required at every height at which a carried report remains unbanned, over the first such report in the order least crossing height, then least $H(r)$, and a report on a hash already in the ban set is invalid [intent: design intake]; $\Delta_{\text{exec}} = 120$ and $\Delta_W = 20{,}160$ blocks; the relay policy for fee-less transactions is a per-identity mempool quota; the graduated 10 % and 15 % tiers, each of which one or two identities can reach alone, are removed; and $(m, T) = (21, 86{,}400)$ for the recommended second chamber, which §23 is careful to say raises the cost of *surprise* rather than the cost of *capture*. Reinstatement has no path at all, and §23 states that as an open problem with requirements rather than inventing one. Two questions could not be settled by choosing a parameter and had to be verified in `fluxd`, because the mechanism rests on them: whether the emergency-block path is subject to the coinbase rules the ban memo would rely on — if it were not, two of four hardcoded keys could ban an application with no report, no votes and no threshold; it is, checked against source (Definition 23.21) — and whether re-addressing a node resets its recorded age, which decides whether the tenure gate does any work; it does, also checked against source (§23). And one requirement crosses back to §7: forfeited tenant balances must never enter Φ, or evicting an application would pay the fleet that voted to evict it.

The notation collision this composition exposed has been resolved: κ now denotes §23's

collusion-set size and nothing else. §14's claim object is written cl and the bridge's per-chain cap is $\overline{M}_c$ [doc: reconciliation record].

12. Operators would be compensated. From issuance *and* revenue. At the proposed activation height $h_{\mathrm{PA}} = 3{,}787{,}502$ the subsidy would be cut to 6.3 FLUX per block and the operator share would begin at $\rho = 0.20$, rising by $\Delta\rho = 0.05$ at each subsequent reduction to the cap $\rho_{\max} = 0.50$ at height 9,378,400 (§7). At activation, operator issuance would be 6,386,040 FLUX per year on the operator-only basis — excluding the dev-fund remainder, which is why §7 corrects the design intake's own parity table downward by 3.7 % — and PNR income at the intake's activation-scale revenue assumption $V_0 = 1{,}431{,}600$ FLUX per year would be 286,320 FLUX per year, i.e. 4.29 % of operator income. Parity revenue would be 31,930,200 FLUX per year, $22.3 \times V_0$. §7 states the conclusion without softening: **with flat demand there is no crossover, ever**, because the perpetual tail of 862,659 FLUX per year to operators exceeds $\rho_{\max} V_0 = 715{,}800$ FLUX per year and the tail is perpetual by §5. Two things were open at this step and both would have changed the arithmetic; both are now settled. The sequencing of PNR activation against the parallel-asset emission cut, contested between the design intake and the public roadmap, is settled below, and O20 — whether the PA-Depletion halving applies uniformly to the three tier bases and to the dev-fund remainder — is settled as uniform (Table 24), which is the basis every parity figure above assumes. And the property that reorders the rest of this paper belongs here: under the specified rule, a node's PNR income would be a function of its tier and its queue position and of nothing else, so its derivative with respect to applications hosted would be identically zero (§7).

Sequencing conflict C1a, which the transition table below depends on, is settled: PNR activation and the parallel-asset emission cut are **simultaneous** at $h_{\mathrm{PA}} = 3{,}787{,}502$ [intent: design intake] (PLANNED), superseding the roadmap's strictly-after ordering (Definition 7.35). The accepted cost is stated with the decision rather than beneath it: simultaneity means no external observer can confirm that PNR pays operators before parallel-asset mining rewards are retired, because there is no interval in which one has happened and the other has not.

A numeric disagreement this composition exposed has been resolved in favour of §7's basis: operator issuance at PNR activation is 6,386,040 FLUX per year on the operator-only basis, and §23's launch-deterrent proposition now uses it. The derived per-node deterrent of about 0.10 FLUX per Stratus node per application per year is unaffected at the precision stated [doc: reconciliation record].

Remark 24.1 (The two retrieval heights are not a disagreement). The measured chain tip appears as 2,917,115 in §4, §7's rotation remark, §20 and §23, and as 2,912,015 in §5's supply figure and §7's activation-table baseline, both dated 2026-09-03. C30 resolves this: the live API snapshot was taken at the first height and the emission model evaluated at the second, roughly 5,100 blocks earlier on the same date, so neither is wrong. The convention this section follows, and which §3's methodology paragraph states, is that every figure names its own height rather than the paper fixing one.

24.3 What changes, and what does not

Table 72: The twelve steps, deployed against target, with the transition condition for each. The last column is the one to read: since rounds 11–18 most rows terminate in a settled value awaiting code or a network upgrade, and the rows that still end in a decision nobody has taken are 1 (the migration key's scope and the envelope replacing `enterprise`), 2 (O14 and O8), 3 (R16.1, specified and unbuilt, and the rebasing rule left open under O9) and 8 (a discovery fallback and a VPON construction).

step	deployed (SHIPPED)	target (PLANNED)	what has to be true for the transition
1	SPEC v8 enforced from height 1,932,380; seven versions live; no `duration`, no `paymentMemo`	SPEC v9 with `duration`, `paymentMemo`, `referralCode`, `machineType`; `enterprise` removed; every application rewritten forward by a one-off forced migration; agent submission under a §17 certificate	consumers written for the three economic fields; an enforcement height set *after* them; an encryption envelope replacing `enterprise`; the migration key's scope enforced in `fluxd` rather than by convention, and its retirement stated; a signature field and enforced canonicalization in the schema
2	one transparent transaction to `t3Nryf...`; hash in `OP_RETURN`; price from a six-segment table	*same shape, moved into consensus*: one output of the full price to `sink` plus one `OP_RETURN` carrying $H(m_a)$ (PS1–PS2); no second output, no new transaction type	O14 (`sink`'s identity) fixed; O8 (which channels must settle there) fixed; O7 (shielded payers) moot after retirement; balance location and recurring-charge authority settled as out of protocol scope
3	FluxOS-only validation; underpayment silently dropped	PS3 makes `sink` unspendable and PS4 applies the 80 %/20 % split; FluxOS checks only existence, memo and coverage	a network upgrade; requirement R16.1 (signed `nack`) implemented; O9 (price-table rebasing) decided
4	**does not exist**	Φ_h committed in the coinbase and recomputed by every validator as $\Phi_{h-1} - D_h + \text{receipts}_h$; no new coinbase output; the sink balance is a public revenue counter	O11 (no seed) and O5/O6 (both stay in Φ) settled; O21 (division on arrival, Foundation share paid in the same block) settled; S1 — emergency blocks must not drip; S2 — forfeitures must not enter Φ
5	PoN sortition; one payee per tier per block; FIFO by last-paid height	unchanged rotation, plus $D_h = \lfloor \Phi_{h-1}/K \rfloor$ split by $(\lambda_{\mathsf{C}}, \lambda_{\mathsf{N}}, \lambda_{\mathsf{S}})$	$K = 86{,}400$ is settled (O1 closed), with O12 a documented review trigger at 2× the Cumulus count and O10 drip ordering from the parent state, both settled
6	opportunistic self-selection; broadcast, 90 s, earliest wins; Docker; reconciler	SPEC v9 reconciler, ∼25 service packages, drain server, per-app-CA TLS	`quorumGrantActivationHeight` set in shipped configuration; earliest-broadcast-wins retained (recommended above)
7	loopback mTLS only; key not machine-bound; benchmark not operator-bound; hash allow-list fails open	bonded attestation network: bond β_i, committee k-of-n, challenge window Δ_{chal}, slash s	**Requirement 0**: hardware-sealed per-machine key and a tenant-reachable verifier; $(n, k, \Delta_{\text{chal}}, s) = (9, 6, 2{,}880, 1)$, β_i scaling with value secured and re-checkable claims only are recommended, the bond coefficients still to calibrate
8	`api.runonflux.io` sole source; 16-host FDM fleet; HAProxy; Cloudflare or PDNS	requirements only: a discovery fallback; ban-honouring routes; VPON membership by key	a non-Foundation discovery source; a decision on FDM/PDNS honouring bans; a VPON construction, which is open
9	reconciler plus periodic monitors; reservation pricing; no metering	unchanged in kind; drain signal over the existing presence path	`flux-shutdownd` drain-broadcast and `preStop` reachable; no protocol availability remedy (settled, §16)
10	expiry is arithmetic; no grace, no suspension, no data retention	*unchanged on the default path* — the payment is the renewal, so no standing obligation and no state machine; funded → grace → suspended → evicted specified for a credits layer and not deployed	for the default path, nothing beyond step 2; for a credits layer, both architectural choices settled out of protocol scope (balance off-chain with a third-party operator; no protocol renewal) and $G_{\text{g}}, G_{\text{s}}, \Delta_{\text{pre}}, \mathcal{P}^{\text{res}}$ settled at Table 41; R16.2 honoured

(Table 72 continued)

step	deployed (SHIPPED)	target (PLANNED)	what has to be true for the transition
11	unsigned GitHub file; $\kappa_{\text{deployed}} = 1$; no record; 7 h to 13 h to fleet-wide effect	indexed report transaction; 24 h window; fee-less votes; `fluxd` counts the quorum internally; ban memo in the coinbase, one per block; FluxOS purges. $\theta = 0.25$ absolute, $\kappa(0.25) = 4$	category enumeration fixed (its scope is settled as behavioural harm); the remaining parameters settled at Table 71; both `fluxd` verifications now closed — re-addressing does reset a node's age, and the emergency-path coinbase rules hold
12	issuance only; σ_{PoN}, tier bases 1:3.5:9, dev fund 0.5 FLUX	issuance plus revenue; ρ from 0.20 to $\rho_{\max} = 0.50$; 4.29 % of operator income at activation	demand growth, not price (parity at $22.3 \times V_0$); C1a sequencing and O20 (uniform PA cut) both settled; and step 7 hardened first

The dependency that shapes everything. Read the table down the "target" column and the mechanisms look independent; read it down the last column and one edge dominates all the others. PNR would pay every confirmed node through the PoN rotation regardless of *which* applications that node hosts, so the private stake any single node has in aggregate network demand would be between 0.001 FLUX and 0.018 FLUX per year at activation scale (§7). That is not a free-riding opportunity — nodes self-select applications at random from the network-wide list, so an operator does not choose what its node hosts (§7, §9) — but it does mean PNR supplies no gradient rewarding good hosting, and this is structural to the settled decision to pay all nodes rather than a tuning error. What actually compels performance is not the payment but the *eligibility gate*: a node is a PoN payee, and hence a PNR payee, only if it is confirmed with a `fluxbench`-signed transaction and, at target, attested. §4 establishes that the benchmark signature is verified against five network-wide shared keys and is produced by a process on the operator's own host, so it is not operator-bound; §14 establishes that the attestation signature is derived from constants compiled identically into every binary, so it is not machine-bound. The two compose badly, and the composition is the reason this section exists.

Property 24.2 (Revenue-funded compensation raises an existing exposure rather than creating one)**.** PLANNED. Under the settled PNR constraints — beneficiaries are all confirmed nodes, and there is no metering or attestation dependency between payment and placement — activating PNR strictly increases the expected payoff to occupying the paid set without providing capacity, by exactly the amount PNR pays: 4.29 % of operator income at activation, rising as ρ ramps toward $\rho_{\max}$. The complementary bound is the one that decides the schedule. The other 95.71 % of that payoff is issuance already paid through the same gate today, so the exposure is a property of PoN and PNR is a 4.5 % increment on it. Hardening the gate afterwards does not undo the interval; the interval it fails to undo is one twenty-third of the standing one.

Proof sketch. By §7's zero-marginal-hosting property, a node's PNR income is a function of its tier and its queue position only, so the payoff to membership rises by the full drip while the payoff to hosting rises by nothing. By §4 and §14 the predicate admitting a node to that queue is a benchmark signature the operator's own host can produce under network-wide keys, together with an attestation that demonstrates possession of a value every copy of the software contains. Hence the cost of satisfying the predicate without providing the underlying capacity is unchanged by PNR while the reward for satisfying it rises, and the difference is the increase in expected payoff. The complementary bound is arithmetic: operator issuance at h_{PA} is 6,386,040 FLUX per year against PNR's 286,320 FLUX (§7), and both pass the same predicate. The step is a sketch and not a proof because it does not quantify the cost side — what it takes to occupy the paid set without providing capacity is not measured anywhere in this paper, and §7's Open Problem on hosting-conditioned eligibility is the construction that would close it. □

The sequencing consequence is therefore not a preference, but it is also not the blocking

dependency an earlier draft of this paragraph asserted. Two things follow, and §27 states the schedule in these terms. Hardening the gate is the network's most urgent security work *independent of PNR*, because almost all of the exposure it addresses is standing exposure that PNR did not create and will not remove. And PNR's *ramp* — not its activation — is what should track that hardening, because the ramp is where the share of operator income sitting behind an unbound gate actually grows. Placing the fix "before or with activation, never after" overstates what the arithmetic supports and prices a 4.3 % supplement as though it were the whole exposure.

What the reader should notice about the two passes. Read as one document, the two passes make a single claim, and it is the thesis this paper has been earning for twenty-three sections. Flux today is a working decentralized cloud — several thousand independently operated machines, in dozens of countries, running the containers of 1,195 deployed application specifications, paid for on-chain, with a consensus layer that produces a block every thirty seconds without a miner — whose *trust*, as opposed to whose compute, is concentrated in operated infrastructure and shared keys: one discovery API with no fallback, sixteen Foundation-run reachability hosts, one Foundation-private network segment, one GitHub file as the moderation root, five network-wide benchmarking keys, one release-signing key inside one HSM, a hardcoded 2-of-4 key set that can produce blocks with no gating, and one transparent address that receives every tenant payment on the network. The target architecture does not add capacity, features or throughput to that picture in any material way; what it does, step by step, is move each of those trust concentrations into one of three things the protocol can enforce — *collateral* (the quorum's weight, the report-capacity bound, the attestation bond), *attestation* (a machine-bound key and a verifier a tenant can reach), and *consensus rules* (the split as a rule instead of a promise, the pool as consensus state committed in the coinbase, the eviction as a tally every validator recomputes from block data and a memo in a coinbase rather than a document somebody publishes). That is the whole architectural bet, stated once: not that the cloud gets bigger, but that the trust moves from parties into rules. Every open parameter in the last column of Table 72 is a place where that move is specified but not yet made.

The sequence the second pass can state and the first cannot. PLANNED. One consequence of the settlement decision is concrete enough to write as a sequence, and it is the agent-native argument of §18 reduced to three steps with nothing left implicit. A tenant holding *any* wallet and *no account* would complete steps 1 to 3 unaided: construct a SPEC v9 specification and sign it with a key it generated itself; broadcast one ordinary transparent transaction paying one address, with one `OP_RETURN` attached; and have consensus divide the proceeds, at which point every node independently agrees the deployment is funded to a particular height. No second recipient to compute, no split to get right, no Flux-specific library to link, no credential to be issued, no custodian, no relationship that outlives the deployment. Each step's dependency is named where it falls rather than accumulated here — step 1 needs consumers for the three economic fields and an enforcement height after them, step 2 needs `sink` fixed by O14, and step 3 needs R16.1 so that a short payment produces a signed answer rather than silence, which for software with nobody to escalate to is the difference between a retry and an unrecoverable loss. The first pass cannot state the same sequence, and the reason is recorded at its own step 3.

What would falsify it. A thesis that cannot fail is not research, so the specific, datable observations that would show the transition is not working are named here rather than left implicit. *First*, PNR remaining a rounding error: application revenue staying near the intake's $V_0 = 1{,}431{,}600$ FLUX per year, so that PNR stays close to 4.29 % of operator income and the $g = 0\,\%$ row of §7's crossover table holds — under which there is no crossover ever, and revenue-funded compensation remains a supplement rather than the mechanism. The settlement

decision is what makes this the cheapest item on the list to check: after activation the quantity is the balance of one consensus-known, consensus-unspendable address, read from any block explorer, rather than a figure anyone has to publish. *Second*, the attestation key staying non-machine-bound: `/v2/attest` continuing to derive its signing key from values compiled identically into every binary at the point where bonds are first posted, which would mean Requirement 0 was declared met without being met and that a bond secures a signature anyone with the software can produce (§14). *Third*, SPEC v9 never receiving an enforcement height: the public pricing endpoint continuing to return no key for version 9 while `paymentMemo`, `referralCode` and `machineType` keep zero consumers, and no forced-migration message appearing on any owner's application — which would strand PNR, credits, referrals, the VPS tier and the provisioning API together, since §25 places all five on the same critical path. *Fourth*, the shielded pools not being retired: `ContextualCheckTransaction` still rejecting transparent-to-shielded transactions with `bad-txns-fluxrebrand-vprivacy-not-empty`, pool totals still at or near the measured 96,479.94 FLUX, and no sunset height announced, at any date past h_{shield} (§20.9). Note that this test was inverted by the retirement decision: an earlier draft made *failure to reopen* the falsifying observation, and the observable that would have confirmed the old commitment — a transparent-to-shielded transaction confirming on mainnet — would now falsify the new one. The paper records the inversion rather than quietly re-aiming the test, because a prediction that changes direction is exactly where a reader should check that the paper is tracking decisions rather than rationalising them. *Fifth*, a moderation capture event: a ban memo in a coinbase whose crossing tally is carried by owner identities that are a subset of the four largest — three under the strongest linkage computable from public data — or, equally decisive and easier to observe, an application disappearing from the fleet with no report transaction, no votes and no ban memo behind it, at any date after the quorum is said to be live. Each of these is checkable from public data by a third party, on a stated date, without access to any repository. That is the standard this section holds itself to, and it is the standard by which the rest of the paper should be judged.

PART V

Assessment

25 Feasibility and distance to target

The position this section tests is that Flux's substrate is already good and the remaining work is extension rather than invention. That is a testable claim, and this section tests it rather than repeating it. The result is mixed in a specific and useful way: the cloud layer largely supports the claim, and four mechanisms do not.

25.1 Method

Every mechanism tagged PLANNED or VISION elsewhere in this paper appears below with what exists today (cited), what is missing, the hardest unsolved sub-problem, and a risk class. Classes are assigned from the evidence gathered in Parts II–IV, not from schedule optimism:

Extension composes components that exist, on paths that already run.

Engineering a substantial new build, with no unsolved research problem.

Research an open problem with no known good answer, or a property that cannot be obtained without a new trust assumption.

A note on what an honest assignment costs. If every row came out *Extension*, the exercise would have failed. Four rows are *Research*, and three of those four are load-bearing for the thesis in §1. One row — the attested tier — has moved *down* in difficulty since the first draft of this table, and that is recorded with the same visibility as a move upward would be, because a survey that only ever revises in the direction of caution is not measuring anything.

25.2 The table

#	Mechanism	Exists today	Hardest sub-problem	Class
1	PNR settlement and fee pool	PoN payment rotation, deterministic and *measured* to equalise within one rotation per tier; consensus-enforced coinbase splits already exist	Pool as consensus state with a reorg-exact serialisation (settled: coinbase-committed) and K (settled: 86,400 blocks) — §7	Engineering
2	Parallel-asset consolidation	Ten deployed contracts; audited Schnorr ERC-4337 account on seven EVM chains	Reaching holders of retired assets and handling unclaimed balances — coordination, not code	Engineering
3	Bounded, publicly verifiable bridge	Audited Schnorr account; Solana multisig live on mainnet	Keeping administrative control decentralized *and* bounded	Engineering

#	Mechanism	Exists today	Hardest sub-problem	Class
4	Attested tier (bonded network as proposed)	Measured boot, dm-verity, LUKS, fail-closed `fes`, a TPM2 mandatory to boot, and an image-bound PCR 4/7/11 policy already generated on every capable machine; `purpose=mesh` implemented with no consumer	Binding the attestation key to that policy and exposing a verifier — engineering, for the TPM population; impossible for the SoftHSM fallback, which should be excluded from the tier. Separately, the *bond* cannot be forfeited: there is no slashing in Flux by decision (Definition 22.16), so the deployable tier is registered, not bonded	Engineering
5	application specification SPEC v9	932 files changed, 7,763 test cases per push, draining and per-app-CA TLS wired	Migrating 322 deployed applications across six legacy versions without evicting any	Extension
6	Main-chain pruning	`-prune` fully wired and inherited; FluxNode registry already in a separate store	Retaining registry and collateral UTXOs across pruning depth	Extension
7	Collateral maturity	Coinbase-maturity precedent; registry enumerable	Defining "continued participation" so upgrades stay instant while exits wait	Engineering
8	Post-quantum migration	Nothing: a repo-wide search returns zero hits	Primitive unchosen; residue classes outside collateral. Collateral path designed (§8.4)	Research
9	Moderation quorum	Node owner keys and collateral weights; a GitHub-file blocklist as today's mechanism	Four owner identities (three under linkage) reach the 25% threshold; no mechanism change fixes concentration	Research
10	Direct per-deployment payment	*Shipped twice*: application registration, and CumulusVPN's memo-derived entitlement	None; this is the mature path	Extension
11	Deposited credits and renewal	`appsmonitor` auto-renewal with InFlux-held keys	Authorising a recurring charge without a custodian and without an allowance primitive — placed outside protocol scope: the protocol offers no auto-renewal, and credits would be a custodial service run by InFlux outside the protocol [intent: design intake]	Engineering
12	Agent deployment authority	FluxOS validates owner signatures over specifications on every node	None; it composes with existing validation	Extension
13	Agent *spending* authority, bounded	ERC-4337 session-key substrate on EVM chains; scoped revocable FluxEdge tokens (action-bounded)	Not possible on the Flux L1 without a new trusted party or new consensus state	Research

#	Mechanism	Exists today	Hardest sub-problem	Class
14	Agent-native provisioning API	Shipped MCP tools; OpenAPI 3.1.1 FluxEdge API with scoped tokens and auto-revocation	Inherits row 13 for its bounds	Extension
15	Natural-language deployment	`DeploymentYAML`: retrieval-augmented generation producing Kubernetes YAML; Orbit specification generation	Whether a human can meaningfully audit a generated specification before signing	Engineering
16	Shielded value	Sapling notes, encryption, commitment tree, nullifiers, Groth16 — all deployed and inherited	None; it runs	Extension
17	Shielded payment for deployments	The primitives above	*Withdrawn*: both pools are to be retired [intent: design intake]	—
18	Proving a shielded payment funds a *specific* deployment unlinkably	Note commitments and nullifiers	An application address is itself a workload identifier; at 35.4 renewals/day a payment is a singleton in its block	Research (moot after retirement)
19	Orchard/Halo2 migration	Sapling/Groth16 deployed; no Orchard present	Consensus change plus audit scoping; now a future option rather than a migration	Engineering
19b	Shielded-pool retirement	Both pools live, closed to entry, 96,479.94 FLUX	Sunset height, outreach, and removal of the Rust proving stack	Engineering
20	GPU tenancy	Whole-device passthrough via Kubernetes	No MIG, vGPU or time-slicing; requests silently stripped on contention	Engineering
21	FluxCloud/FluxEdge convergence	Two runtimes and two products	Organisational as much as technical	Engineering
22	Decentralizing the reachability path	FDM, PDNS, gateway and certificates all working	`api.runonflux.io` is the sole application-discovery source for the routing layer (FDM); nodes themselves replicate specifications and locations peer-to-peer	Engineering

Table 73: Distance from the deployed system to the target architecture. Citations for every "exists today" entry are given in the section that describes it.

Tally: **Extension 6, Engineering 12, Research 4.** Row 4 moved from Research→Engineering to Engineering on two findings recorded in §14: the image-binding machinery is already deployed and unconsumed rather than absent, and the bond's forfeiture problem is not open but decided against, which removes the mechanism rather than leaving it unsolved.

25.3 The four Research items, which are not equally hard

Post-quantum migration (row 8). Hard for everyone, and Flux starts from a *worse* position than Bitcoin on one axis and a better one on another. Worse: 20.61 % of supply sits in FluxNode collateral whose public keys are published by protocol mandate, not by user behaviour (Definition 8.5) — and BIP-360's remedy, an output type that hides the key in the scriptPubKey

[6], does not reach it, because the exposure is in the registration transaction rather than the output script.

Better, and it is the axis that decides the disposition: collateral is a *registered* set with a mandatory recurring transaction, so Flux has an authenticated channel to exactly the exposed population. That is what makes eligibility-forced migration possible (Definition 8.7, adopted) and what lets Flux avoid the freeze-or-tolerate-theft dilemma BIP-361 must resolve for Bitcoin [37]. The row stays **Research** because the primitive is unchosen and the residue classes are unsolved, not because the collateral path is undesigned — that path is specified in §8.4.

Bounded agent spending on the Flux L1 (row 13). Hard for a precise and stateable reason established in §17: script validation is a pure function of the spending transaction and the outputs it consumes, so no rule can enforce a rate limit or a cumulative cap. The deployable answer — fund a dedicated key, and the budget is the balance — is honest and unglamorous. Everything richer requires a co-signer, which is a new trusted party, or new consensus state.

Unlinkable payment for a specific deployment (row 18). The paper's most interesting problem, and the one whose difficulty is easiest to overstate. It must be classed in three parts and never collapsed: the shielded primitives are inherited and shipped (Extension); applying them to deployment payment is Engineering, gated on row 17; and proving a shielded payment funds a *particular* deployment without linking payer to workload is Research. Crucially, **it applies only to the optional credit path and to shielded funding proofs**. The default direct-payment path is unaffected, because the payment *is* the renewal and there is no standing obligation to prove — so the agent-native argument of §18 survives even with this problem open. That limitation of blast radius is a real result and not a consolation.

The moderation quorum (row 9). Classed Research not because the mechanism is hard to build — it is straightforward — but because **the property it is meant to provide cannot be obtained by building it**. Four owner identities — three under the strongest linkage computable from public data — clear the 25% threshold at the measured distribution. No bond, rate limit or eligibility gate changes that. The open question is whether a two-chamber rule requiring both a weight threshold and a distinct-identity count can preserve Sybil resistance while adding censorship resistance, and that is genuinely open.

25.4 Where the claim holds, and where it does not

For the cloud layer the claim holds well. SPEC v9, pruning, deployment authority, direct payment and the provisioning API are all Extension, and the SPEC v9 substrate is substantially implemented and tested on a development branch, though not yet shipped or activated. The proposition that Flux can extend rather than invent its way to an agent-serving cloud is *largely correct, for everything except payment bounds and payment privacy.*

Where it does not hold, row 4 remains the case to watch, though for a narrower reason than the first draft of this section gave. The roadmap treats the bonded attestation network as an extension of an existing attestation service, and the existing service does not provide the property the design needs to build on: an attestation demonstrates possession of a value shipped in every copy of the software, not that a given machine booted a given image (§14). That is a prerequisite rather than an increment. But it is a prerequisite with its hardest component already built: ArcaneOS generates an image-bound PCR policy on every TPM-capable machine and nothing consumes it (§14.1.1), so the work is binding a key to an existing policy and serving a quote, not designing a hardware root. Two consequences: the tier should be **scoped to the TPM population**, since the SoftHSM fallback can never satisfy the requirement; and what §7 depends on is that scoped, achievable piece, not the bonded network as a whole.

25.5 The dependency that reorders the schedule

One edge in the dependency graph deserves to be lifted out of the table, because it changes what should ship first.

PNR would pay all nodes from a shared pool, so a node's income would not depend on which applications it runs. That is not free-riding — placement is not the operator's to refuse (§7.8) — but it does mean PNR would supply no gradient rewarding good hosting, so what compels performance is the eligibility gate, and rows 4 and 9 establish that the gate is neither operator-bound nor machine-bound. **PNR raises the payoff to passing that gate without doing the work, by exactly what it pays.**

The size of that increment decides how the edge should be drawn, and it is smaller than an earlier draft of this subsection implied. At activation PNR would pay 286,320 FLUX per year against 6,386,040 FLUX per year of issuance already flowing through the same gate: an increase of 4.5 % in the prize (§7.11). **The gate is a PoN problem today, and** 95.7 % **of the exposure exists whether or not PNR ever activates.** Two corrections follow. Hardening attestation is the highest-priority security work in this table *on its own merits*, and calling it a PNR precondition understates it by more than an order of magnitude. And the sequencing rule that actually follows from the arithmetic is a coupling rather than a block: PNR's *ramp* toward $\rho_{\max} = 0.50$ should track the gate's hardening, since each step moves a larger share of operator income behind it, while activation at $\rho = 0.20$ adds little to an exposure already present. §27 carries the schedule in those terms.

A second, milder dependency: rows 1, 5, 11, 14 and 21 all wait on SPEC v9 fields that currently have no consumers — `paymentMemo`, `referralCode` and `machineType` (§10). SPEC v9 is therefore the critical path for most of Part IV, and its enforcement height, planned at $h^\star = 3{,}050{,}000$ (§10) but not activated and still subject to the implementation, is the single most schedule-relevant date in this document.

26 Evaluation

A reader who has followed the specification chapters this far has seen what Flux claims to be. This section checks the claim against the running network on one day, and against a small, reproducible model of the monetary policy and of the not-yet-shipped PNR mechanism. Three kinds of number appear below, and each is labelled as it appears: a *measured* number comes from a live public endpoint queried on the date given; a *modelled* number comes from executing a pinned, deterministic script against consensus constants and is a projection, not an observation; a *derived* number is arithmetic performed on measured inputs (a percentage, a Gini coefficient) and carries the same endpoint and date as the inputs it is computed from. Nothing below is estimated from intuition, and where a quantity a reader might want is not obtainable at all, that absence is stated rather than filled in.

26.1 Methodology

All measured figures in this section are read from `api.runonflux.io`, retrieved 2026-09-03, and are reproduced by `scripts/05-network-analysis.py`. Three endpoints are used: `/daemon/getzelnodecount` and `/daemon/listzelnodes` for the fleet census, payment rotation and operator-concentration figures (§26.2–§26.4); `/apps/globalappsspecifications` and `/apps/deploymentinformation` for the deployed application set (§26.5); and `/daemon/getblockchaininfo` for the shielded value pools (§26.6). Chain tip at the time of the node-record query was height 2,917,115; the emission-model reference height, queried separately, was 2,912,015. The two differ because they were captured at different points in the same collection run; both are stated explicitly wherever they are used, and the difference (5,100 blocks, about 42.5 hours at 30 s spacing) is immaterial to every figure below.

Modelled figures come from two independent, stdlib-only Python scripts, executed 2026-09-03 and 2026-09-07 respectively, both deterministic given the pinned constants they read, and both citable in full in Appendix C: `flux-roadmap/supply/emission_model.py` (with its generator `_gen_dat.py`) for the supply and emission figures of §26.7, and `scripts/07-pnr-settlement.py` for the PNR crossover and drip figures of §26.8–§26.9. The emission model reproduces already-shipped consensus arithmetic (PoN subsidy, halving history) and is stated in present tense; the PNR model projects a mechanism that is settled in design but not deployed, and is stated in future tense throughout §26.8–§26.9, consistent with its PLANNED maturity.

Two absences are stated once here and not re-derived below. First, no public endpoint reports realised utilisation, bandwidth, or latency of the deployed fleet — only the resources *requested* in application specifications are visible, and §26.5 reports exactly that, with no estimate of what fraction is actually used. Second, the two identity proxies available for operator concentration (`payment_address`, `pubkey`) are lower bounds on concentration by *party*; no public endpoint links either to a real-world operator, so every concentration figure in §26.4 is stated as a lower bound, every time it is used, not once and then dropped.

26.2 The fleet

This is a measured quantity, SHIPPED [measured: api.runonflux.io/daemon/getzelnodecount, 2026-09-03]. The fleet has **6,489** nodes, all reported `stable`: 3,247 FluxNodes enabled at Cumulus, 1,615 at Nimbus, 1,627 at Stratus. `listzelnodes` returns 6,489 node records with the same tier split; matched against per-tier collateral, the fleet locks **88,514,500 FLUX** (Table 74). Of the fleet, 2,322 nodes report an IPv4 address; none report IPv6 or an onion address (0 of each), SHIPPED [measured: api.runonflux.io/daemon/getzelnodecount, 2026-09-03].

tier	nodes	collateral each (FLUX)	tier collateral (FLUX)	share of voting power
Cumulus	3,247	1,000	3,247,000	3.67%
Nimbus	1,615	12,500	20,187,500	22.81%
Stratus	1,627	40,000	65,080,000	73.52%
total	6,489	—	88,514,500	100%

Table 74: Fleet census by tier, measured 2026-09-03 at chain height 2,917,115. Voting weight under the planned moderation quorum of §23 would be collateral-weighted at one unit per 1,000 FLUX (88,514.5 weight units total); no node voting exists today. [measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03].

Node tenure, measured as blocks since `added_height` across all 6,489 nodes SHIPPED [measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03]: median 193,620 blocks (approximately 67 days at 30 s spacing), 10th percentile 12,909 blocks, 90th percentile 881,770 blocks, maximum 1,648,700 blocks. The fleet therefore contains both nodes confirmed within the last collection window and nodes that predate PoN activation by a wide margin; no further resolution of the tenure distribution (a full histogram) is available from this endpoint.

26.3 The payment rotation, measured

Under PoN, each tier's payment queue is ordered by ascending last-paid height, so a deterministic rotation is a testable prediction: every node in a tier of n_t nodes should be paid at least once every n_t blocks. Table 75 tests it directly against the live payment queue SHIPPED [measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03], at tip height 2,917,115, by computing tip − last_paid_height for every node.

tier	nodes	expected rotation (blocks)	max blocks since paid	median	95th pct	rotation at 30 s
Cumulus	3,247	3,247	3,202	1,592	3,041	27.06 h
Nimbus	1,615	1,615	1,597	800	1,517	13.46 h
Stratus	1,627	1,627	1,631	807	1,545	13.56 h

Table 75: PoN payment rotation, measured 2026-09-03 at chain tip 2,917,115. [measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03].

Cumulus and Nimbus stay strictly inside one rotation (max blocks-since-paid below the tier's node count). Stratus exceeds its rotation length by 4 blocks (1,631 against 1,627), consistent with a small number of nodes joining mid-cycle — at the measured median tenure of 67 days, roughly 1–2% of a tier's nodes join within any rotation-length window, and a join lengthens the cycle by exactly one node without breaking the underlying period-n_t structure. This measurement confirms the deterministic one-node-per-tier-per-block rotation as a property of the deployed queue, not as a model assumption: every node in a tier is paid within one rotation of that tier's node count, equalising within 27.06 h (Cumulus), 13.46 h (Nimbus) and 13.56 h (Stratus). It is the empirical basis for the fairness proposition of §7 (that cumulative operator income within a tier is bounded over one rotation) and is cited there directly.

26.4 Operator concentration

This is a derived quantity, computed from the fleet census SHIPPED [measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03]. Collateral-weighted voting power (one unit per 1,000 FLUX, the weight the planned moderation quorum of §23 would use) is aggregated by two identity proxies present in the public record: `payment_address`, where a node's rewards are sent, and `pubkey`, the node key. Neither is proof of common ownership — one operator can use many addresses, and a shared custodian can serve many operators — so **every figure in this subsection is a lower bound on true concentration by party**, stated as such at each use, per the unresolved gap recorded as G12.

By `payment_address`: 869 distinct identities control the full 88,514.5 weight units (88,514,500 FLUX). A removal threshold of 25% of total voting power is reached by the 4 largest addresses (of 869, 0.46%); 50% by 19 addresses (2.19%). The Gini coefficient of voting weight across `payment_address` is **0.8375**. By `pubkey`: 819 distinct identities control the same total; 25% is reached by the 3 largest keys (of 819, 0.37%); 50% by 16 (1.95%). The Gini coefficient across `pubkey` is **0.8389**. Table 76 gives both top-N share curves in full.

top-N identities	share, by `payment_address`	share, by `pubkey`
1	10.71%	10.71%
5	28.07%	31.73%
10	37.99%	41.56%
20	51.19%	53.91%
50	67.92%	69.31%
100	78.88%	79.60%
250	91.18%	91.84%

Table 76: Cumulative share of collateral-weighted voting power held by the top-N identities, by two lower-bound proxies for operator identity. Measured 2026-09-03 at chain height 2,917,115. [measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03].

The largest single identity, by either proxy, holds 9,480.0 weight units (10.71% of total voting power) across 237 nodes; the most nodes held by any one identity is 479 (by `payment_address`) or 480 (by `pubkey`). Restricting to Stratus alone, the same largest identity holds 14.57% of Stratus-only voting weight (9,480.0 of 65,080.0 weight units) and its node count there

is exactly 237 — the largest identity's 237 nodes are entirely Stratus. Stratus-only concentration is somewhat lower in Gini terms (0.7375 across 220 distinct `payment_address` identities) than the fleet-wide figures, but its top-1 share (14.57%) is higher than the fleet-wide top-1 share (10.71%) because Stratus carries 73.52% of total voting power in far fewer nodes. Figure 20 plots the two fleet-wide cumulative-share curves of Table 76.

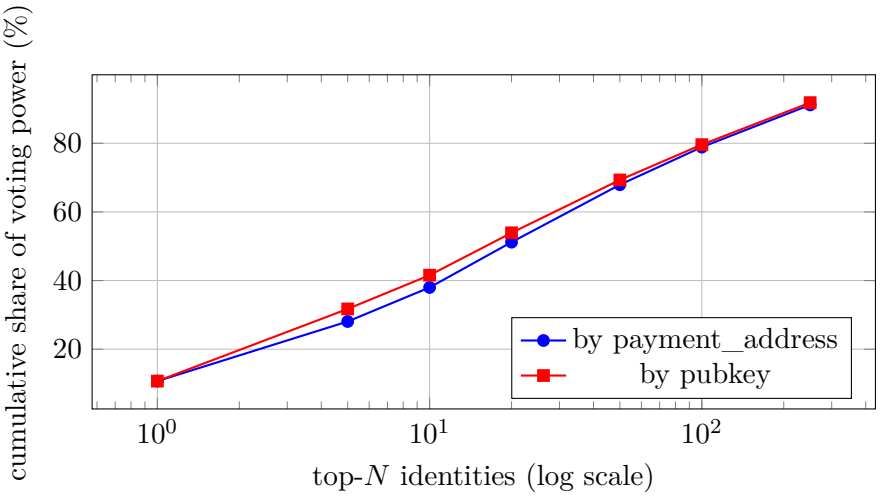


Figure 20: Cumulative share of collateral-weighted voting power held by the top-N operator identities, by two lower-bound proxies. The curve is sampled at the seven N values reported in Table 76, not at every N. Measured 2026-09-03 at chain height 2,917,115; data underlying `paper/data/concentration_payment_address.dat` and `paper/data/concentration_pubkey.dat`. [measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03].

26.5 Deployed applications

This is a measured quantity, SHIPPED [measured: api.runonflux.io/apps/globalappsspecifications, 2026-09-03]. The endpoint returns **1,195** application specification records, comprising 1,371 components (compose entries) and a total of **7,486** requested instances across all applications. Instances per application: median 2, mean 6.26, maximum 100, minimum 0; 439 applications (36.7%) sit at 3 instances, the minimum the endpoint reports (the validator's floor is 1 for SPEC v8 applications, Table 95). Table 77 gives the specification-version distribution and Table 78 the container-registry distribution.

spec version	applications	share
v2	2	0.2%
v3	27	2.3%
v4	11	0.9%
v5	3	0.3%
v6	70	5.9%
v7	209	17.5%
v8	873	73.1%

Table 77: Application specification version distribution, measured 2026-09-03. [measured: api.runonflux.io/apps/globalappsspecifications, 2026-09-03]. SPEC v9 does not yet appear (§10).

image registry	components
`docker.io` (implicit)	1,332
`ghcr.io`	31
`quay.io`	3
`lscr.io`	2
`public.ecr.aws`	1
`ttl.sh`	1
`codeberg.org`	1

Table 78: Container registry distribution across 1,371 components, measured 2026-09-03. [measured: api.runonflux.io/apps/globalappsspecifications, 2026-09-03].

Summed committed resources across all 7,486 requested instances: 8,972.0 vCPU-equivalents, 17,544,229 MB of RAM (17,133.0 GiB), and 160,107 GB of disk. These are the resources declared in application specifications and enforced at deployment time by the platform's own minimum/maximum bounds (`minimumInstances`: 3, `maximumInstances`: 100, `blocksLasting`: 22,000 blocks per default lifetime, [measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]) — they are *requested*, not realised. No public endpoint reports realised CPU, memory, disk or network utilisation of the deployed fleet, and this section does not estimate it (G13); the committed-resource figures above are the complete measurement available.

Feature usage across the 1,195 applications: `geolocation` 556 (46.5%), `staticip` 50 (4.2%), `nodes` (pinned placement) 11 (0.9%), `enterprise` 223 (18.7%), `expire` 1,152 (96.4%), `owner` 1,195 (100.0%), `contacts` 699 (58.5%), SHIPPED [measured: api.runonflux.io/apps/globalappsspecifications, 2026-09-03].

26.6 Shielded value pools

This is a measured quantity, SHIPPED [measured: api.runonflux.io/daemon/getblockchaininfo, 2026-09-03]. At chain height 2,917,115, the `valuePools` field reports a Sapling pool of **77,188.75921324 FLUX** and a Sprout pool of **19,291.17735041 FLUX**, both monitored, for a combined shielded value of 96,479.93656365 FLUX. Consensus has rejected transparent-to-shielded transfers since height 835,554 ([fluxd/src/main.cpp:1398–1408]), so both pools have been closed to new entrants since that height — 2,081,561 blocks before the measurement tip — and every satoshi in them was shielded before the turnstile closed. Against the total issued supply of 429,466,823.9700 FLUX at height 2,912,015 (§26.7), the shielded pools hold **0.0225%** of issued supply (derived). Note count and holder count of either pool are **not obtainable** from this or any public endpoint and are not estimated here.

26.7 Supply, modelled

This is a modelled quantity: it reproduces already-shipped PoN consensus arithmetic ([flux-roadmap/supply/emission_model.py:195–243], itself a transcription of PoN's `GetBlockSubsidy`) rather than a new mechanism, so it is stated SHIPPED and in present tense, executed 2026-09-03 by `flux-roadmap/supply/emission_model.py` ([doc: flux-roadmap/supply/emission_model.py, run 2026-09-03]). Current issued supply at height 2,912,015 (mode `observed`, the model's default and the mode this paper uses for every current-supply figure) is **429,466,823.9700 FLUX**. Mode `code` — a literal transcription of `GetBlockSubsidy()` — gives 429,466,973.9700 FLUX at the same height: a persistent, height-independent offset of exactly **150.00000000 FLUX**, not the 0.06 FLUX the model's own docstring claims, traced to a double-counted slow-start block in the `code`-mode formula. This discrepancy, its derivation, and the reproduction instructions are given in full in Appendix C.

The PoN subsidy started at **14 FLUX** per block at activation (height 2,020,000) and is reduced by a factor of 9/10 (with per-step integer truncation, so the reduction is not exactly 0.9^r) every $I_{\text{red}} = 1{,}051{,}200$ blocks — one year at 30 s spacing — for a maximum of $N_{\text{red}} = 20$

reductions ([flux-roadmap/supply/emission_model.py:149–150]). After the 20th reduction, the subsidy is frozen and does not decay further: at the onset of the tail (height 23,044,000, a 30-second-spacing extrapolation to approximately 2045-10-20; see §26.10), the block subsidy is fixed at 1.70207313 FLUX, and cumulative supply grows without bound at a constant **1,789,219.274256 FLUX per year, forever** ([flux-roadmap/supply/emission_model.py:514–546]). **There is no terminal supply.** Supply one year into the flat-rate tail (height 24,095,199, the end of the emission model's zero-indexed "PON year 20") is 548,043,625.860464 FLUX; this is not a cap, and the annual emission *rate* has been constant since height 23,044,000. The currently-issued 429,466,823.9700 FLUX is 78.36% of that level — a proxy for "how far along" the schedule is, stated explicitly as a proxy rather than a fraction of a terminal supply, because no terminal supply exists to be a fraction of. `MAX_MONEY` (440,000,000 FLUX) is crossed by cumulative issuance at height 3,730,295 (approximately 2027-06-11); this is not a consensus failure, since `MAX_MONEY` bounds a single transaction's value, never the chain-wide supply accumulator.

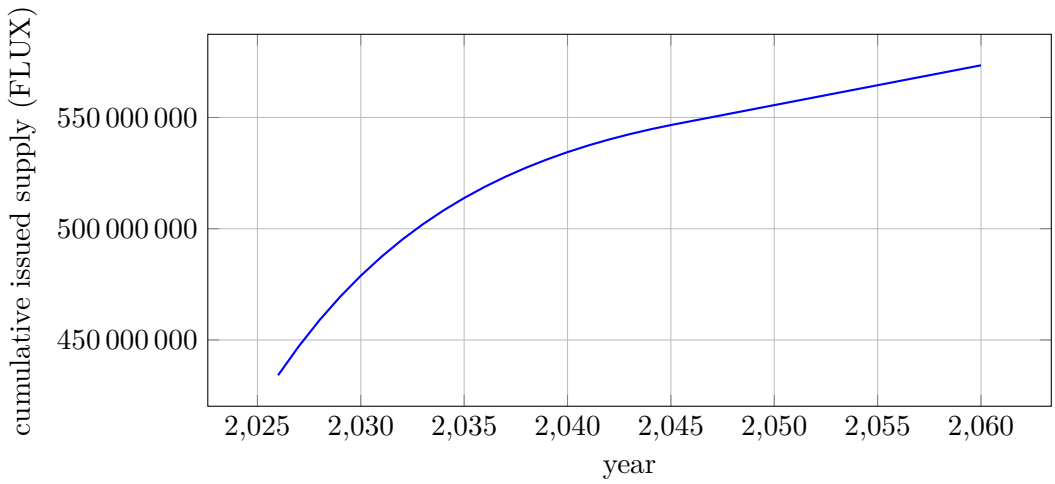


Figure 21: Cumulative issued FLUX supply, 2026–2060, mode `observed`. The curve is a straight line in the flat tail from approximately 2045 onward (1,789,219.274256 FLUX/yr) and never asymptotes. [doc: flux-roadmap/supply/emission_model.py, run 2026-09-03]; data in `paper/data/emission.dat` (columns: year, height, cumulative_supply, annual_emission, pow_emission, pon_emission).

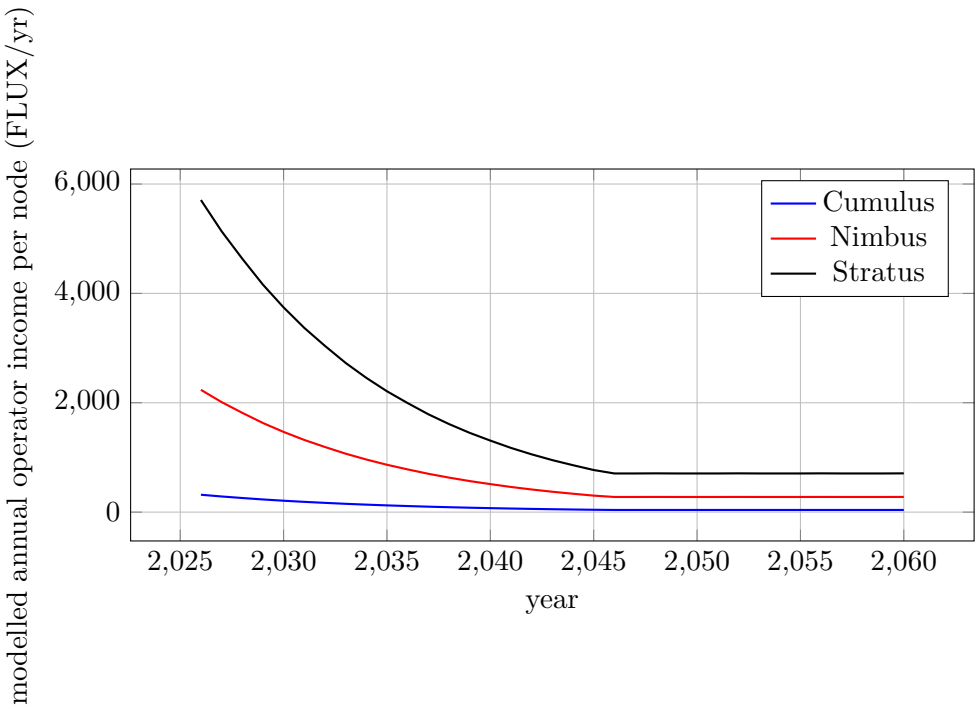


Figure 22: Modelled per-node PoN issuance income by tier, 2026–2060, at a fixed fleet (3,246 / 1,615 / 1,627 nodes, held constant; fleet growth and churn are not modelled). Per-node income scales with tier subsidy share and inversely with tier node count, not with collateral (Definition 7.18). [doc: flux-roadmap/supply/emission_model.py, run 2026-09-03]; data in `paper/data/tier_income.dat` (columns: year, cumulus_per_node, nimbus_per_node, stratus_per_node, cumulus_total, nimbus_total, stratus_total, total).

26.8 The PNR crossover, modelled, in both denominations

PNR does not exist in deployed code. As of the measurement tip (height 2,917,115), the chain is 870,387 blocks short of the planned activation height $h_{\mathrm{PA}} = 3{,}787{,}502$ (approximately 2027-07-01 at 30 s spacing). Everything in this subsection is therefore a PLANNED projection — [intent: design intake] for the settled design (an 80%/20% Foundation/operator split at activation, ramping by 5 percentage points at each PoN reduction to a cap of 50%), and [doc: scripts/07-pnr-settlement.py, run 2026-09-07] for every number computed from it — and is stated in future tense throughout. It is stated in both FLUX terms (primary, because FLUX is what consensus governs) and real terms (secondary, with a price multiple treated as an exogenous axis, not a forecast); the two must never be used to defend one another. This paper makes no price prediction anywhere in what follows.

The crossover model uses a fleet fixed at 3,246 / 1,615 / 1,627 nodes (6,488 total) — one Cumulus node fewer than §26.2's measured 3,247 / 1,615 / 1,627 (6,489), a one-node difference between census windows that changes nothing at the precision reported below. It also uses a *planned* subsidy path distinct from the as-coded schedule of Figure 21: from h_{PA}, the PoN subsidy is additionally halved on top of the ordinary 9/10 reduction schedule ($14 \to 12.6 \to 6.3$ FLUX at h_{PA}, then 5.67, 5.103, …), a change that `paper/data/emission.dat` does not contain.

At activation, with $\rho(h_{\mathrm{PA}}) = 0.20$ and subsidy 6.3 FLUX, modelled operator issuance (excluding the dev-fund remainder) would be 6,386,040 FLUX/yr. Under the team's activation-scale revenue assumption $V_0 = 1{,}431{,}600$ FLUX/yr (1,193 applications × 100 FLUX/month × 12 — a modelling assumption, not a measured receipt total; O17 in §7), PNR income would be $0.20 \times V_0 = 286{,}320$ FLUX/yr. **PNR would be 4.29% of total operator income at**

activation (PNR as a share of PoN issuance plus PNR combined: 286,320/6,672,360). "Parity" — the point at which PNR income would equal PoN issuance income — would require annual application revenue of **31,930,200 FLUX/yr**, 22.3× V_0, at activation.

stage start (h)	date	subsidy (FLUX)	ρ	operator issuance (FLUX/yr)	parity revenue (FLUX/yr)	growth from V_0
3,787,502	2027-07-01	6.3000	0.20	6,386,040	31,930,200	—
4,122,400	2027-10-25	5.6700	0.25	5,747,436	22,989,744	16.1×
5,173,600	2028-10-24	5.1030	0.30	5,172,692	17,242,308	12.0×
6,224,800	2029-10-24	4.5927	0.35	4,655,423	13,301,209	9.3×
7,276,000	2030-10-24	4.1334	0.40	4,189,881	10,474,702	7.3×
8,327,200	2031-10-24	3.7201	0.45	3,770,893	8,379,762	5.9×
9,378,400	2032-10-23	3.3481	0.50 (cap)	3,393,803	6,787,607	4.7×
12,532,000	2035-10-23	2.4407	0.50	2,474,083	4,948,165	3.5×
17,788,000	2040-10-21	1.4412	0.50	1,460,921	2,921,842	2.0×
23,044,000	2045-10-20	0.8510	0.50	862,659	1,725,319	1.2× (perpetual tail)

Table 79: Planned PNR schedule in FLUX terms (primary), modelled 2026-09-03. Dates are 30-second-spacing extrapolations (§26.10). [intent: design intake]; [doc: scripts/07-pnr-settlement.py, run 2026-09-07].

The two levers (falling subsidy, rising ρ) would together bring the parity requirement down from 31.9 M to 6.8 M FLUX/yr over 5.3 years and to 1.7 M FLUX/yr on the perpetual tail. Whether demand ever reaches that requirement is a separate question, answered only under explicit, stated assumptions about revenue growth g — never as a forecast. Table 80 gives the modelled crossover year under six such assumptions ($g \in \{0\%, 10\%, 25\%, 50\%, 100\%, 200\%\}$ per year, constant from V_0); these are scenario axes chosen to span a wide range, not predictions of which growth rate, if any, will occur.

g (per year)	crossover	revenue at crossover (FLUX/yr)	ρ at crossover
0%	never	—	—
10%	2038-10	4.21 M	0.50
25%	2034-10	7.33 M	0.50
50%	2032-10	12.4 M	0.50
100%	2030-10	14.3 M	0.40
200%	2029-10	18.3 M	0.35

Table 80: Modelled crossover year under constant annual revenue growth from V_0. [doc: scripts/07-pnr-settlement.py, run 2026-09-07]; data in `paper/data/pnr_crossover.dat` (columns `pnr_g000`…`pnr_g200`).

Under flat demand ($g = 0\%$), there is no crossover, ever. The perpetual tail issuance (862,659 FLUX/yr) exceeds $\rho_{\max} \cdot V_0 = 715{,}800$ FLUX/yr, so even at the ramp's cap of 50%, PNR income at flat V_0 would never reach PoN issuance income; the parity requirement falls over time only because the subsidy keeps falling, and reaching it is a statement about demand growth, never about the subsidy schedule alone. Sustained growth required to reach parity by a given year: 162%/yr by 2029, 82%/yr by 2030, 51%/yr by 2031, 34%/yr by 2032, 26%/yr by 2033, 16%/yr by 2035. As scheduled, PNR would begin as a supplement to operator income; whether it ever becomes the dominant component is a statement about demand that this paper does not make.

$g \setminus m$	0.5	1	2	5	10
0%	14.3	never	never	never	never
10%	7.3	11.3	14.3	19.3	26.1
25%	5.3	7.3	9.3	12.3	14.3
50%	3.3	5.3	6.3	7.3	9.3
100%	2.3	3.3	4.3	5.3	6.3

Table 81: Modelled years to PNR parity, real terms (secondary), under real demand growth g (rows) and an exogenous FLUX price multiple m (columns; $m > 1$ denotes FLUX-price appreciation against the rebased price table, m is a modelling axis, not a forecast). Under rebasing, price appreciation *delays* FLUX-terms parity even as real operator income rises — the two statements are made in different denominations and this paper does not use one to defend the other. [doc: scripts/07-pnr-settlement.py, run 2026-09-07]; data in `paper/data/pnr_crossover_real.dat`.

26.9 The drip trade-off

This subsection is also PLANNED and modelled, [doc: scripts/07-pnr-settlement.py, run 2026-09-07], and supports the settled value K= 86,400 blocks (30 days at 30 s spacing) of §7. The drip constant K trades intra-rotation income spread and activation lag against timing-attack resistance and pool size. Table 82 gives the trade-off at the five candidate values computed in `paper/data/pnr_drip.dat`, against the longest rotation in the fleet (Cumulus, 3,246 nodes in the settlement model).

K (blocks)	half-life (days)	decay-side spread$^{\downarrow}$	one-Stratus recapture	ramp to 95% (days)	pool at parity (FLUX)
3,246	0.78	171.79%	0.0205%	3.4	49,000
10,000	2.41	38.34%	0.0067%	10.4	152,000
32,460	7.81	10.51%	0.0021%	33.7	493,000
64,920	15.62	5.13%	0.0010%	67.6	986,000
86,400	**20.79**	**3.83%**	**0.00077%**	**90**	**1,310,000**

Table 82: Modelled drip trade-off at the measured Cumulus rotation length. Decay-side spread is the first-versus-last-payee income ratio within one Cumulus rotation under zero inflow; recapture is the fraction of its own PNR payment a single Stratus node could recover by timing that payment, against the 10^{-4} threshold of requirement H5. Ramp is the time to reach 95% of steady-state drip from a zero pool at activation. Pool at parity is the modelled steady-state pool at parity-scale revenue and $\rho = 0.50$. [doc: scripts/07-pnr-settlement.py, run 2026-09-07]; data in `paper/data/pnr_drip.dat`.

The settled K = 86,400 would leave one Stratus operator able to recover at most 0.00077% of its own operator share by timing a payment to its own payout — three orders of magnitude below the 10^{-4} threshold this paper sets for a negligible timing attack — at the cost of a 20.79-day half-life and a roughly 90-day ramp to steady state after activation, since the pool is not seeded (settled, O11 in §7). At parity-scale revenue, the modelled steady-state pool would hold approximately 1.31 M FLUX; this paper states plainly, per §7's own framing, that this balance would be a delay line with a 30-day time constant, not a treasury.

26.10 Threats to validity

The measurements of §26.2–§26.6 are a **single-day snapshot**, taken 2026-09-03; fleet size, rotation state, application counts and concentration all change continuously, and no time series is reported here beyond the node-tenure distribution. Operator-concentration figures (§26.4) are, by construction, **lower bounds**: `payment_address` and `pubkey` are the only public identity proxies, and true concentration by economic party is not obtainable from any public endpoint (G12); this paper does not claim to know it. Application resource figures (§26.5) are **requested,**

not realised: no public endpoint reports actual CPU, memory, disk, bandwidth or latency utilisation of the deployed fleet, and none of the committed-resource figures should be read as a utilisation claim (G13). The supply and PNR projections of §26.7–§26.9 assume a **constant 30-second block spacing** for every height and date beyond the current tip; the emission model's own validation states this assumption is accurate to within approximately 0.8 days after 796,820 blocks, and is explicitly unquantified over the multi-decade horizons (the year-20 tail onset, approximately 2045) that this section cites — real PoN spacing may drift, and every calendar date attached to a future height carries that uncertainty. Finally, the PNR crossover figures rest on a single, team-supplied activation-scale revenue assumption (V_0 = 1,431,600 FLUX/yr; O17 in §7) rather than a measured receipt total, and the growth rates g and price multiples m used to project beyond it are modelling axes chosen to span a range, not forecasts of what will occur — this paper makes no claim about which, if any, will be realised.

27 Migration and rollout: sequencing, dependencies, and risk

PoN replaced PoW at height h_{PoN}, and the transition was not clean: it needed an emergency reorg-depth override and a checkpoint cluster to stabilize (§27.6 below). The roadmap's next transition is not one change but roughly a dozen — PNR, chain consolidation, attested hosting, and privacy re-entry — several of which share a load-bearing v9 field, a consensus-change slot, or a stated precondition that the public roadmap and the design intake do not always order the same way. This section makes that ordering explicit: which edges the roadmap states, which this paper infers, and where the two source documents disagree on sequence rather than on substance. The headline finding is that the riskiest edge in the graph is not a technical unknown but a sequencing one. PNR's payoff to hosting is provably zero at the margin (§7), so the mechanism's integrity depends entirely on an eligibility gate that §4 and §14 already show is bound neither to the operator nor to the machine that passes it. Activating PNR raises the payoff to defeating that gate, though only by a small increment over the issuance that already flows through it, so hardening the gate is urgent on its own merits and PNR's ramp should track it (§27.2). A second, independent finding — that most of Part IV waits on three v9 fields with no consumer code today — makes the v9 enforcement height — proposed at $h^\star$ = 3,050,000 (§10) but not activated, since SPEC v9 remains in development — the single most schedule-relevant date in this document. Everything below is PLANNED or VISION unless stated otherwise; no date appears here that the roadmap or the design intake did not state, and quarter labels are reproduced verbatim.

27.1 The dependency graph

Table 83 lists dependency edges among the roadmap's chain- and platform-level items that bear on rollout sequencing, reproduced with the roadmap's own verbatim quarter labels (PLANNED; [roadmap: flux-roadmap-public.md, dependency edge list, §3]). Most edges are stated directly by the roadmap's prose; two are the roadmap's own inferred directionality rather than a stated gate, and are marked so; one row, previously carried as an open sequencing conflict (C1a), is now resolved as a simultaneity rather than a precedence and is marked with = rather than → (§27.5); the final row is this section's own argued dependency — it does not appear in the roadmap's edge list at all — and is discussed in §27.2 below.

Edge ($A \to B$, B waits on A)	Quarter(s) (verbatim)	Status	Source
SPEC v9 spec RFC publication → SPEC v9 rollout	Q4 2026	stated	[roadmap: flux-roadmap-public.md:66]
SPEC v9 payment-memo field → PNR v2 (workload-identifying payments)	Q4 2026 → H1 2027	stated	[roadmap: flux-roadmap-public.md:64,119]
SPEC v9 `machineType` flag → VPS tier	Q4 2026 → H1 2027	stated	[roadmap: flux-roadmap-public.md:64,125]
SPEC v9 Duration field → One balance / auto-renew	Q4 2026	stated	[roadmap: flux-roadmap-public.md:62]
Operator protection → VPS tier launch	Q4 2026 → H1 2027	stated	[roadmap: flux-roadmap-public.md:125]
Bridge redesign (mint-and-burn) → independent audit (gate before deploy)	Q4 2026	stated	[roadmap: flux-roadmap-public.md:97]
Bridge redesign contracts → attestation / proof-of-reserve work	Q4 2026	stated	[roadmap: flux-roadmap-public.md:99]
Q4 2026 bridge attestation work → attestation network's "first customer"	Q4 2026 → H1 2027	stated	[roadmap: flux-roadmap-public.md:138]
PNR v2 testnet → pilot (CumulusVPN payments) → mainnet → per-tier before/after examples published before activation	H1 2027	stated	[roadmap: flux-roadmap-public.md:119]
PNR mainnet activation = parallel-asset mining-reward retirement (simultaneous, at h_{PA})	roadmap labels H1 2027 / H2 2027 (superseded); resolved date 2027-07-01, with the retiring chains one-directional from March 2027 and shut down in August 2027 (round 34)	resolved: simultaneity, not the roadmap's sequential gate	[intent: design intake], superseding [roadmap: flux-roadmap-public.md:133,159]
Q4 2026 privacy groundwork (technical/legal/exchange plan) → chain privacy implementation	Q4 2026 → H2 2027	stated	[roadmap: flux-roadmap-public.md:102,158]
Collateral-maturity proposal → operator discussion → activation scheduling, riding an already-planned upgrade	H1 2027	stated	[roadmap: flux-roadmap-public.md:146]
Main-chain pruning ↔ RPC/archive/indexing product	Q3 2026 (product) / H1 2027 (pruning)	inferred (roadmap's own directionality)	[roadmap: flux-roadmap-public.md:34,121-122]
Attestation hardening (machine-binding the key to the PCR policy already generated) → PNR's *ramp* toward $\rho_{\max}$	not dated by the roadmap	inferred (this paper, §27.2)	[inferred]

Table 83: Dependency edges among roadmap items bearing on rollout sequencing. Quarter labels are verbatim; "waits on" direction is $A \to B$. One row is marked $A = B$ rather than $A \to B$ to record that the two events are now simultaneous rather than sequential (§27.5).

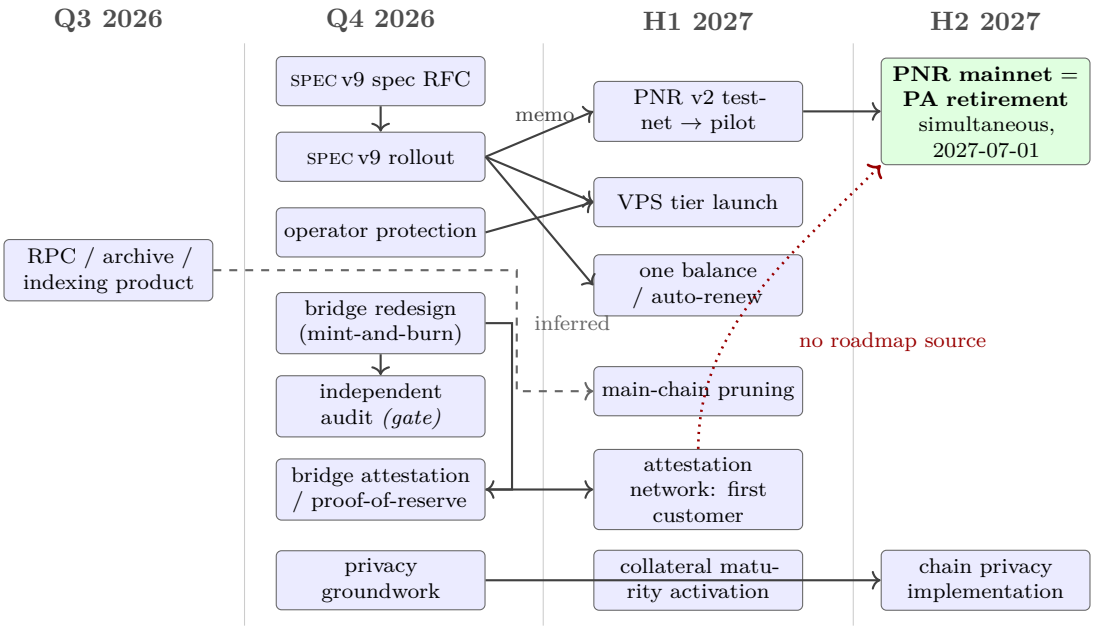


Figure 23: Roadmap dependency graph, oriented by quarter.

27.2 The edge that shapes the schedule: attestation alongside revenue

PNR pays all nodes from a shared fee pool through the PoN rotation rather than paying the node that hosts a given application (PLANNED; [intent: design intake]). A node's income is therefore independent of *which* applications it hosts. That is not free-riding, because an operator does not choose placement — each node's FluxOS spawner selects at random from the network-wide list (SHIPPED; [flux/ZelBack/src/services/appLifecycle/appSpawner.js:311]) — but it does mean PNR supplies no gradient that rewards hosting well (PLANNED; [intent: mechanism specification] [doc: reconciliation record]). What compels a node to perform, in this design, is not PNR — it is the eligibility gate that lets a node participate in the PoN rotation in the first place: benchmarking and attestation (§4, §14). Those sections already establish that this gate is weaker than it looks. The benchmark result `fluxd` accepts is signed by network-wide shared keys compiled into every copy of `fluxbench`, not by a key bound to the operator who ran it (SHIPPED; [fluxd/src/chainparams.cpp:233–238]). The attestation key `fluxsas` produces is derived by HKDF from static salt and domain values compiled identically into every binary, with no input that depends on a TPM, on Secure Boot state, or on any other per-machine value (SHIPPED; [private source: SAS API server]). Neither signature is bound to the operator or the machine that produced it.

These two findings compose badly, and the composition has to be sized before it can be turned into a schedule. Because PNR raises the value of holding an eligible, paying slot in the rotation, and because passing the eligibility gate does not currently require possessing the hardware or operator identity the gate is supposed to attest, PNR raises the payoff to passing that gate without doing the work, by exactly what it pays (PLANNED; [doc: reconciliation record]).

By how much is the question, and the answer bounds the conclusion. At activation PNR would pay 286,320 FLUX per year. Operator issuance through the same gate at the same height is 6,386,040 FLUX per year (§7). PNR would raise the prize by 4.5 %. An earlier draft of this subsection concluded from the qualitative composition that hardening attestation is a *precondition* for PNR mainnet activation; that conclusion does not survive the arithmetic, and this paper withdraws it in favour of two statements that do.

1. **Attestation hardening is the schedule's highest-priority security item on its own merits.** 95.7 % of the income flowing through the unbound gate at activation is issuance that flows through it today. Treating the work as a PNR dependency understates

its urgency by more than an order of magnitude, and invites the reply — correctly — that a 4.3 % supplement is not worth blocking a network upgrade for.

2. **PNR's ramp, not its activation, is what should track that work.** ρ rises from 0.20 to 0.50 over six reductions, and each step moves a larger share of operator income behind the gate. Coupling the ramp to attestation's completion prices the dependency where it actually grows, costs nothing at activation, and does not hand an opponent of PNR a principled veto over it.

What makes the coupling achievable rather than rhetorical is a finding recorded in §14: the image-bound TPM policy the hardening needs is *already generated* on every capable machine and simply has no consumer. The dependency is therefore not the roadmap's opt-in bonded network with k-of-n challengeable claims (PLANNED; [roadmap: flux-roadmap-public.md:135-138]), which is a larger and later object, but the narrower piece of work of binding the attestation key to that policy and serving a verifiable quote. Stated that way the dependency and PNR's own activation height h_{PA} (2027-07-01) are compatible; stated as a dependency on the bonded network, they are not, and the paper would be committing to two things that conflict. The final row of Table 83 is read accordingly.

27.3 The second critical path: spec v9

FluxOS SPEC v9 is the second critical path, and it is broader than PNR alone. PNR's fee-pool memo depends directly on the v9 `paymentMemo` field (PLANNED; [roadmap: flux-roadmap-public.md:64,119]). Deposited credits depend on the same substrate through the v9 `duration` field, and both credits and the provisioning API's spending bounds depend further on a bounded-spending primitive that SPEC v9 does not by itself supply (PLANNED; [doc: feasibility assessment, rows 1, 11, 14]). The datacenter tier and the one-provider-runtime convergence depend on the v9 `machineType` flag (PLANNED; [roadmap: flux-roadmap-public.md:64,125]; [doc: feasibility assessment, row 21]).

Three of the fields this depends on have zero consumer code anywhere in FluxOS today: `paymentMemo`, `referralCode`, and `machineType` (IN-PROGRESS; [doc: reconciliation record]). The orchestration substrate of v9 is in development and under test, unmerged on a contributor fork — 932 changed files and a regression suite of 289 unit-test files running 7,763 test cases on every push (IN-PROGRESS; [doc: reconciliation record]) — but the economic fields the roadmap's own dependency chain rests on are declared in the schema and unimplemented beneath it. The currently enforced specification version remains SPEC v8, and 322 of 1,195 deployed applications (26.9%) sit on six versions older than that (IN-PROGRESS; [doc: reconciliation record]; [measured: network census, §5, 2026-09-03]).

The v9 enforcement height, proposed at $h^{\star} = 3{,}050{,}000$ (§10) but not activated, since SPEC v9 remains in development, is therefore the most schedule-relevant date in this document ([doc: feasibility assessment]). Meeting it requires two things that do not exist yet: implemented consumers for the payment-memo, referral, and machine-type fields, and `latestSupportedSpecVersion` raised in lockstep with a fleet-wide version floor. The mastership/active-standby quorum plane, whose gating field (`quorumGrantActivationHeight`) is read by production code today but set only in test fixtures (IN-PROGRESS; [doc: reconciliation record]), is deliberately decoupled from $h^{\star}$ and activates later, once SPEC v9 is stable (§10).

Decided. Partly. SPEC v9 activation and the migration of existing applications are both stated for **Q4 2026** [intent: design intake], the enforcement height was proposed as $h^{\star} = 3{,}050{,}000$ [intent: design intake] but has not activated (SPEC v9 remains in development), constrained to follow the economic-field implementation (Definition 10.2), and the migration policy is that all applications are migrated automatically rather than by owner action [intent: design intake]. What remains open is the mechanism by which the 322 applications on legacy versions are rewritten — an automatic migration still needs a specified transformation per source version, and none is stated in any artefact read for this paper.

27.4 The privacy prerequisite

Since height 835,554, `fluxd` rejects any transaction that combines transparent inputs with a shielded output or joinsplit, with the in-source comment stating the intent directly — the shielded pool has accepted no new value in approximately 5.4 years (SHIPPED; [fluxd/src/main.cpp:1398-1408]). Value can leave the pool ($z \to t$) or move within it ($z \to z$), but it cannot enter.

This prerequisite has been withdrawn, not scheduled. An earlier round of the design intake recorded that $t \to z$ would be reopened as part of a privacy update, and this paper carried it as a scheduled dependency for wallet integration and for the payment path of §21. That has been superseded: the team has decided to **retire both shielded pools** rather than reopen either [intent: design intake] (PLANNED). The reasoning and the schedule are in §20.9; the consequence for this section is that reopening leaves the migration path entirely.

Three items therefore leave the dependency graph rather than moving within it: the reopening activation height, wallet integration for shielded sends in Zelcore and SSP, and the construction in §21 that would have consumed newly-entered value. One item is added: a sunset height $h_{\text{shield}} = 3{,}600{,}000$ after which shielded spends are invalid, together with the outreach that must precede it (§20.9). The roadmap's published direction — "we are bringing private transactions back" [roadmap: flux-roadmap-public.md:167], and "chain privacy implementation" in H2 2027 [roadmap: flux-roadmap-public.md:158] — is superseded by this decision, and the paper records the supersession rather than silently reinterpreting the roadmap. What the roadmap says about not pinning a date on a cryptographic change before its audit is scoped [roadmap: flux-roadmap-public.md:177] still applies, and now applies to a possible *future* shielded construction (§20.8) rather than to a reopening.

27.5 The activation schedule

Table 84 reproduces the design intake's activation schedule: block height, calendar date, PoN subsidy σ_{PoN}, and node share ρ through the ramp toward the cap $\rho_{\max} = 50\%$ (PLANNED; [intent: design intake]).

Event	Height	Date	Subsidy (σ_{PoN}, FLUX)	Node share (ρ)
Current	~2,912,015	2026-09-03	14.0	—
Reduction 1 (−10%)	3,071,200	2026-10-25	12.6	—
PA Depletion (−50%), h_{PA}	**3,787,502**	**2027-07-01**	**6.3**	**20% — PNR begins**
Reduction 2 (−10%)	4,122,400	2027-10-25	5.67	25%
Reduction 3 (−10%)	5,173,600	2028-10-24	5.103	30%
annually thereafter	+1,051,200	—	×0.9	+5pp

Table 84: PNR activation schedule from the design intake. PA Depletion (h_{PA}) is simultaneous with the parallel-asset mining-reward cut and the swap contract taking effect (§27.5; [intent: design intake]), not a half-year ahead of it as the public roadmap's superseded sequencing would have had it. Node share ρ ramps to a declared cap $\rho_{\max} = 50\%$, reached at height 9,378,400 (2032-10-23) on the stated annual step ([intent: design intake]).

PNR activation and the parallel-asset emission cut occur at the *same* event: height $h_{\text{PA}} = 3{,}787{,}502$, 2027-07-01, is when the node share ρ first becomes nonzero and when parallel-asset mining-reward emission is cut (PLANNED; [intent: design intake]). This resolves what earlier drafts of this paper carried as an open sequencing conflict (C1a). The public roadmap had stated a different, sequential plan — "retirement of any parallel asset's mining rewards comes only after PNR v2 is demonstrably paying operators" and, more specifically, "parallel-asset mining retirement — only after PNR v2 is paying operators on mainnet, announced at least 90 days ahead" — a step the roadmap placed in H2 2027, one half-year after PNR v2's own H1 2027 slot ([roadmap: flux-roadmap-public.md:133,159]). That sequencing is **superseded** by the team's direct answer to this paper: activation is simultaneous, at h_{PA} (PLANNED; [intent: design intake]). This

section and §6 both now describe one coordinated transition rather than two phases separated by an interval.

The consequence is not merely notational. The roadmap's sequential design existed to create a checkpoint: a proving period, of up to a stated half-year with 90 days' advance notice, in which PNR could be observed actually paying operators before the parallel assets whose retirement it was meant to justify were cut. With simultaneous activation, that checkpoint does not exist — deliberately, by the team's own later decision, not by oversight. §27.7 carries this as a named risk rather than treating the superseded roadmap language as still in force.

27.6 An empirical base rate for consensus change

The transition this paper's thesis rests on was not clean, and the fact belongs in a risk analysis rather than a footnote. `fluxd`'s normal reorg-depth limit, `MAX_REORG_LENGTH`, is 40 blocks, but it was overridden to 5,000 for the height range [2,020,000, 2,025,000], an override the source comments describe as a "[t]emporary deep reorg limit during PON stabilization period" (SHIPPED; [fluxd/src/main.cpp:8983-8988]). Alongside it sits a checkpoint cluster at heights 2,020,500 / 2,021,000 / 2,021,500 / 2,022,000 / 2,029,000, labelled in the source "PON activation and stabilization checkpoints – EMERGENCY FORK RECOVERY" (SHIPPED; [fluxd/src/chainparams.cpp:277-283]). The PoW →PoN transition, in other words, required active emergency intervention after activation despite having been planned.

The roadmap itself draws two lessons from this kind of history, applied prospectively to the next consensus change it proposes (collateral maturity): first, that a proposal should ride an already-planned network upgrade rather than force its own — "it would ride an already-planned network upgrade rather than forcing its own" (PLANNED; [roadmap: flux-roadmap-public.md:146]); second, that proposals go to operators for discussion before any activation is scheduled, not after — stated for collateral maturity ([roadmap: flux-roadmap-public.md:146]) and echoed in PNR's own staged sequence, which ends with "per-tier before-and-after examples published to operators before activation" (PLANNED; [roadmap: flux-roadmap-public.md:119]). Three further consensus-adjacent changes are queued behind PoN and sit in §8: main-chain pruning (PLANNED; [roadmap: flux-roadmap-public.md:121-122]), collateral maturity (PLANNED; [roadmap: flux-roadmap-public.md:143-146]), and the post-quantum migration roadmap (PLANNED; [roadmap: flux-roadmap-public.md:148-149]) — the last of which the roadmap is explicit is a multi-year effort beyond publishing a path. Against the C16 base rate, this is why sequencing them into as few network upgrades as possible matters: each additional independently-forced consensus change is, by the network's own recent history, an occasion on which stabilization measures of the kind seen at PoN activation may again be needed.

27.7 Risk register

Table 85 lists the risks this section's dependency analysis surfaces, the mechanism each threatens, the evidence behind its likelihood, and the mitigation that exists or does not.

Table 85: Risk register for the sequencing and dependencies described in this section.

Risk	Threatens	Likelihood basis (evidence)	Mitigation
Demand growth undershoots; PNR stays a supplement	PNR (§7)	At h_{PA}, parity needs ∼31.9M FLUX/yr (§26.8) against assumed demand of ∼1.43M FLUX/yr (the team's assumption of 1,193 applications at 100 FLUX/month; the census counts 1,195 applications and 7,486 instances, §26.5); PNR is ∼4.3% of operator income at activation; under flat demand there is no crossover, ever ([intent: design intake]; [doc: reconciliation record])	Ramp lowers the parity threshold to ∼6.8M FLUX/yr by 2032-10, ∼5.3 years post-h_{PA} (§26.8); no stated policy if demand still undershoots (O13, open)
Simultaneous activation removes the roadmap's proving-period checkpoint	PNR credibility; parallel-asset retirement (§6.8, §27.5)	PNR activation and the parallel-asset emission cut are simultaneous at h_{PA} = 3,787,502 ([intent: design intake]), superseding the roadmap's plan to retire parallel assets only after PNR is "demonstrably paying operators," announced 90 days ahead ([roadmap: flux-roadmap-public.md:133,159])	None stated; a deliberate simplification by the team, not an oversight, but it means no external observer can verify PNR is paying operators before the parallel assets it justifies retiring are cut
Eligibility-gate exploitation once PNR pays	PNR's only hosting compulsion (§4, §14)	Benchmark keys are shared constants, not per-operator ([fluxd/src/chainparams.cpp:233–238]); attestation key derivation has no machine-bound input ([private source: SAS API server]); PNR adds no gradient rewarding good hosting ([doc: reconciliation record]), though it adds only 4.5 % to a payoff the same gate already carries	Machine-binding the attestation key to the PCR 4/7/11 policy ArcaneOS already generates (§14) — narrower and nearer than the roadmap's opt-in bonded network, k-of-n, planned H1 2027 ([roadmap: flux-roadmap-public.md:135-138]); §27.2 couples it to PNR's ramp rather than blocking activation on it
A bridge audit finding delays consolidation	Bridge redesign, chain consolidation (§22)	Audit is an explicit hard gate before deploy ([roadmap: flux-roadmap-public.md:97]); the network's only prior consensus-adjacent transition needed emergency intervention despite planning (§27.6)	Audit-before-deploy is itself the stated mitigation; no contingency schedule exists for a finding that requires rework (open)
SPEC v9 migration strands legacy applications	PNR, credits, provisioning API, datacenter tier (§27.3)	322 of 1,195 deployed applications (26.9%) sit on six versions older than SPEC v8 ([doc: reconciliation record]; [measured: network census, §5, 2026-09-03]); an enforcement height of $h^{\star}$ = 3,050,000 was proposed but has not activated (SPEC v9 remains in development), and no per-version transformation or eviction policy exists	Forced automatic migration at $h^{\star}$ [intent: design intake]; the transformation per source version is unspecified (§27.3)

(Table 85 continued)

Risk	Threatens	Likelihood basis (evidence)	Mitigation
Moderation-quorum capture event	Content-reporting quorum (§23)	4 owner identities (`payment_address`) reach the 25% eviction threshold, falling to 3 once addresses linked by a shared node key or collateral funding transaction are merged; the largest holds 10.71% of collateral-weighted voting power; Gini 0.8375 ([measured: network census, §2, 2026-09-03]; [doc: reconciliation record])	Two-chamber rule (weight threshold plus distinct-identity count with tenure) raises the cost of a surprise capture but does not lower κ; does not reach censorship resistance ([doc: reconciliation record])
Shielded-pool sunset destroys notes not swept in time	Holders of the 96,479.94 FLUX still in the pools (§20, §20.9)	$t \to z$ closed since height 835,554, approximately 5.4 years ([fluxd/src/main.cpp:1398-1408]); re-opening is cancelled and both pools are to be retired ([intent: design intake]); roadmap will not pin a date on a cryptographic change before its audit is scoped ([roadmap: flux-roadmap-public.md:177])	Sunset settled at $h_{\text{shield}} = 3{,}600{,}000$ (2027-04-27), announced at publication and 187,502 blocks before h_{PA} ([intent: design intake]); ZIP-209 turnstile enforcement active for the window and direct holder outreach are this paper's recommendations, not confirmed decisions (§20.9)
Archival capacity becomes demand-dependent once history is a paid product	Pruning, archive/RPC/indexing (§8, §13)	Roadmap pairs the Q3 2026 archive product directly with H1 2027 pruning, but states the direction loosely and marks it [inferred] in its own dependency list rather than as a strict gate ([roadmap: flux-roadmap-public.md:34,121-122])	None stated; if the archive product does not attract paying demand, the operator incentive to run archive nodes once pruning removes the default full-history requirement is untested

27.8 What would have to be true

This is not a schedule with a confidence interval attached; it is a set of conditions, stated explicitly, under which the sequencing above resolves into a working rollout rather than a stalled or exploited one.

- **Demand, not price, must grow.** The number of deployed applications, instances, and resources sold must grow roughly in line with the compounding schedule in Table 84 for PNR to become more than a ∼4% supplement to operator income; this is a claim about deployment counts and resources sold, never about price ([intent: design intake]; [doc: reconciliation record]).
- **Attestation hardening should proceed on its own merits, and PNR's ramp should track it.** If the eligibility gate — already shown not to be operator- or machine-bound (§27.2) — is still unbound as ρ ramps toward $\rho_{\max}$, a growing share of operator income becomes a monetized target behind it; the narrower fix is machine-binding the attestation key, not the full bonded network (§27.2).
- **spec v9's three economic fields need implemented consumers before any of their dependents activate.** `paymentMemo`, `referralCode`, and `machineType` must each acquire consumer code, and `latestSupportedSpecVersion` must rise in lockstep with

a fleet-wide version floor, before PNR, credits, the provisioning API's bounds, or the datacenter tier can activate on the schedule the roadmap implies ([doc: reconciliation record]).

- **The shielded-pool sunset must be executed as settled.** Reopening $t \to z$ is cancelled and both pools are to be retired [intent: design intake]; what replaces the old open question is a sunset height $h_{\text{shield}} = 3{,}600{,}000$, confirmed [intent: design intake], together with ZIP-209 turnstile enforcement for the drain-only window and the direct outreach preceding it, both this paper's recommendations (§20.9, §27.4).
- **The removed proving period is an accepted risk, not an oversight, and should stay visible.** Simultaneous activation (§27.5) means no external observer can confirm PNR pays operators before parallel-asset mining rewards are retired; any activation announcement should say so rather than let a reader assume the roadmap's original 90-day-notice checkpoint still applies.
- **Consensus changes need to be batched into as few network upgrades as possible**, following the roadmap's own stated preference to ride an already-planned upgrade rather than force a new one; every additional independently-forced consensus change is, by the measured base rate of the PoW →PoN transition, an occasion on which unplanned stabilization measures may again be needed (§27.6).
- **The bridge audit and the moderation quorum's residual capture risk need to be treated as live constraints, not formalities**, since neither has a stated contingency for an adverse finding or a captured vote (Table 85).

28 Related work

Flux's claim in this paper is architectural, not comparative marketing: a network in which an autonomous agent can rent capacity, pay for it, and leave, without an account, a payment card, or a human in the loop, using, on FluxCloud, an operator set (the FluxNodes) that is capital-bonded and enumerable rather than merely reputational. Positioning that claim requires comparing mechanisms, not brands. This section takes each mechanism the paper specifies elsewhere — a decentralized resource market, a consensus protocol, a bridge, an attestation root, a shielded payment pool, a delegation primitive, and a machine-facing interface — and states the prior art for that mechanism specifically, what Flux inherits from it, and where Flux's design departs. Claims about other systems describe only what those systems have published, with no maturity judgement about their roadmaps; claims about Flux itself carry the same maturity and provenance discipline as the rest of this paper.

28.1 Decentralized compute markets

Akash [4] and Golem [28] are open compute marketplaces: a tenant posts a resource request, providers bid or advertise matching capacity, and a match is made without the marketplace itself verifying that a provider's declared hardware exists or performs as advertised; both settle payment in the network's own token, escrowed from the tenant and released to the winning provider over the deployment's life. Render [42] and io.net [33] apply the same open-bidding structure to a single resource class — GPU rendering and GPU compute for machine-learning workloads, respectively — aggregating capacity from an unbonded, self-reported pool of node operators. Aethir [3] targets the same GPU-cloud niche with an enterprise framing. In all five, the trust a tenant extends to a provider is reputational: nothing in the protocol requires a provider to lock capital against non-delivery, and nothing verifies a provider's declared specification beyond the provider's own attestation of it.

Flux departs from this class in one specific way: every FluxNode operator posts a tier-scaled collateral that consensus checks against the registration transaction before a node is eligible to

run anything SHIPPED [fluxd/src/coins.cpp:630–686], and the resulting registry is directly enumerable from chain state — 6,489 records at the most recent measurement SHIPPED [measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03]. Two properties follow that an open, self-reported marketplace cannot offer by construction. First, the same registry that gates eligibility is what PoN's block-production sortition ranks and rotates SHIPPED [fluxd/src/pon/pon-minter.cpp:78–104], and it is the substrate §7's fairness argument for revenue-funded compensation is built on PLANNED [intent: §7]. Second, an enumerable operator set is a tractable target for a future key migration in a way an open, unbonded pool cannot be: collateral is already a registered set on Flux, so a migration that starts there is a narrower problem than a general UTXO migration VISION [intent: feasibility assessment]. This is also what makes the resulting eviction quorum Sybil-resistant by construction [20] — a property §23 relies on and this section returns to below. Neither property is free: collateral is capital locked against no guaranteed return, and an enumerable registry is also a measurably concentrated one — the largest single collateral-weighted identity in the registry holds 10.71% of voting power across 237 Stratus nodes under one key SHIPPED [measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03], a fact §23 uses directly.

28.2 Hyperscalers

Hyperscale clouds occupy an adjacent but distinct point in the design space, and there is no hostility in stating the difference plainly. They price by metered consumption rather than by reservation, offer deep managed-service catalogs, and back regional deployments with contractual latency and availability guarantees; none of that is disputed here [44]. What Flux offers that a hyperscaler does not is orthogonal rather than superior: no account, no payment card, and no human approval step stand between a payment and a running deployment — already true, concretely, of CumulusVPN's entitlement path, where a WireGuard public key is itself the credential and no server-issued account exists SHIPPED [cumulusvpn/gateway/internal/entitle/entitle.go:218–287] — and, on FluxCloud, the operator fulfilling the deployment is a FluxNode drawn from the capital-bonded, enumerable set described above rather than an opaque corporate fleet (FluxEdge, a separate, centrally operated product, does not draw on that set). Concretely, Flux prices a reservation: a deployment's price is a function $\mathcal{P}(\mathbf{r}, k, d)$ of a resource vector, an instance count and a duration fixed at request time, not a metered function of realized usage SHIPPED [flux/ZelBack/src/services/utils/appUtilities.js:89–134] [measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03] (§16). That is the architectural difference that matters more than any feature count: a reservation price is settled once, at deployment time, and the only subsequent dispute surface is whether the reserved resources were delivered, not how much of them was consumed.

28.3 Storage and content networks

Filecoin [41] and IPFS [8] are the closest prior art for hosting rather than compute. Filecoin's storage deals are prepaid by the client and backed by provider-posted collateral that the protocol can slash if the provider fails to prove continued possession of the stored data; the remedy for non-performance is financial, enforced against the provider's own stake. Flux's stated remedy for a host that fails to deliver a paid-for deployment is different in kind: replacement rather than refund — the network moves the workload to another eligible host rather than returning payment to the tenant PLANNED [intent: §16]. IPFS carries no equivalent mechanism at either layer: content addressing guarantees that a hash resolves to the bytes it names, but the network has no built-in remedy for a bad actor and no protocol-level way to remove content once published — persistence is a function of who chooses to pin it, not of any consensus process. Flux's proposed content-reporting quorum (§23) is the opposite kind of mechanism: a collateral-weighted vote to evict a specific application from the fleet PLANNED [intent: §23]. Placed side by side, the three positions are distinct: Filecoin slashes a provider's stake for failing to keep data available; IPFS has no mechanism to make content unavailable at all; Flux's quorum is built expressly to make a specific application unavailable network-wide, which is not the same problem as either.

28.4 Consensus

PoN replaces Bitcoin-style proof of work [38] with collateral-gated sortition over the FluxNode registry described above: eligibility to produce the next block is a function of tier and collateral rather than of hashing power. The precise point a reader should not have to infer is in the fork-choice rule. Every PoN block contributes a fixed chainwork value of 2^{40} regardless of its actual difficulty target SHIPPED [fluxd/src/pow.cpp:284–290], so fork choice is not a race for the chain with the most accumulated work in the sense Nakamoto consensus assumes [38] and the security analyses built on that assumption use [22, 26] — it is a block-count race, resolved by which chain extended first and furthest. That places PoN's security argument closer to proof-of-stake sortition schemes such as Algorand [27], which likewise select a block producer by a verifiable, stake-weighted draw rather than by cumulative work, than to Nakamoto consensus or to chain-based proof-of-stake protocols such as Ouroboros [34] that still accumulate a work-like quantity along the chain [inferred]. Two differences from Algorand are worth stating rather than eliding: PoN's eligibility draw is public and registry-based rather than a private per-round cryptographic sortition, and, so far as the material available to this paper shows, PoN has no finality gadget layered over its fork-choice rule, unlike systems that pair a longest-chain-style base protocol with an explicit finality overlay [13] [inferred]. [10] frames exactly this kind of comparison — what a consensus protocol's security argument actually rests on, rather than which family it is conventionally filed under — and is the right lens for reading PoN this way.

28.5 Bridging

Cross-chain interoperability designs separate broadly into light-client verification, trusted or federated attestation, and atomic exchange of assets that already exist on both chains [46]. IBC [32] is the clearest instance of the first category: a receiving chain runs a light client of the sending chain, verifying block headers and validator signatures directly, so no third party is trusted beyond the sending chain's own consensus. XCLAIM [45] generalizes the idea to chains, like Bitcoin, that cannot be light-client verified cheaply on the receiving side, using over-collateralized vaults and SPV-style proofs instead. CCTP [18] sits in the second category: an attestation service observes a burn on the source chain and signs a message authorizing a mint on the destination chain, substituting a designated attester for a light client. Atomic swaps [30] sit outside both categories: they exchange assets that already exist on both chains using hashed time-locks, but they cannot *issue* an asset on a chain where it does not yet exist.

Neither of the first two categories is presently constructible for Flux, and the reason is the header, not the will. A light client needs three things PoN's header does not give it. First, per the bridge design's own analysis, the header carries no commitment to the FluxNode registry, so a verifier cannot check that a block's signer was an eligible operator without holding the full registry itself PLANNED [intent: mechanism specification]. Second, chainwork is fixed at 2^{40} per block regardless of difficulty SHIPPED [fluxd/src/pow.cpp:284–290], so there is no cumulative-work quantity a light client could use to prefer one candidate chain over another the way an SPV client does. Third, a 2-of-4 emergency key set can produce a valid block outside PoN's normal eligibility rule, with no time or stall-detection gate on when it may act SHIPPED [fluxd/src/emergencyblock.cpp:183–191], so no light-client rule derived from PoN eligibility alone would be sound even if the first two problems were solved. The consequence is concrete: the most trust-minimized bridge architecture available to IBC-style systems is closed to Flux until the header changes to commit to the registry, and that is a consensus change, not a bridge-layer one, and a candidate for the same network upgrade that would carry collateral maturity PLANNED [intent: §8].

Atomic swaps do not fill this gap either, for a reason independent of the header: a swap exchanges FLUX that already exists on Flux for an asset that already exists on the destination chain. It cannot mint FLUX on a chain where none yet exists, which is exactly what parallel-asset issuance requires PLANNED [intent: mechanism specification]. Swaps are the right primitive for FusionX's exchange product, where both assets already exist on both sides; they are not a candidate for

the bridge itself. Flux's remaining option, and the one §22 specifies, is canonical mint-and-burn under a bonded, capped custodian — closer to CCTP's trusted-attester model than to IBC's light client, with per-chain caps $\overline{M}_c$ and reserve slack $\varsigma = R - \sum_c M_c$ (§22) doing the work a light client would otherwise do PLANNED [intent: §22].

28.6 Attestation and bonded assertions

ArcaneOS's measured boot and Secure-Boot platform-key pinning SHIPPED [fluxsas/src/fes/fes.cpp:70–81], its dm-verity root-hash verification SHIPPED [private source: SAS watchdog], and its LUKS-backed disk encryption SHIPPED [private source: SAS disk encryption] do real work against a remote attacker without any special hardware trust anchor. The question this section positions is not whether that machinery works but what the resulting attestation is rooted in. One line of prior art roots attestation in silicon: Intel SGX [19] ties an attestation to a hardware-manufacturer-signed key that a remote verifier can check without trusting the party running the enclave. A second line roots it in capital: EigenLayer [21] lets operators post slashable stake against off-chain assertions, so an assertion is trusted because a bond would be lost if it is false, not because of where it was computed. Flux's planned attestation network is a version of the second model — a bond rather than a chip is what would secure the claim that a machine booted a particular image PLANNED [roadmap: new benchmark tool, signed node attestation, timeline.html:295–296].

The honest state of the current system is that neither model is yet in place. The `fluxsas` signing key for the default, `cdn`, `mesh` and `app` attestation purposes is derived from static salt and domain values compiled identically into every binary; none of its inputs depends on TPM state, Secure Boot state, or any other per-machine value SHIPPED [private source: SAS API server]. What that attestation demonstrates is that the signer holds a value shipped in every copy of the software, not that a particular machine booted a particular image. The trust root behind "this node booted something Flux considers legitimate" is today entirely Flux-operated, composed of the platform key above and a pair of Flux-operated web services publishing allowed dm-verity root hashes that the watchdog polls roughly daily and treats as secure if unreachable SHIPPED [private source: SAS watchdog]. Neither SGX's hardware root nor EigenLayer's economic root is present yet; the `purpose=mesh` attestation type already compiled into `fluxsas` with no consumer SHIPPED [private source: SAS API server] is the stated seed of the bonded design that would supply one PLANNED [intent: §14].

28.7 Privacy

Flux's shielded pool is Sapling — encrypted notes, a note commitment tree NCT, nullifiers nf, and Groth16 proofs [29] verifying spend and output statements over z-addr-typed value — specified by the Zcash protocol [31] and tracing to the original Zerocash construction [7], and it is live on-chain today SHIPPED [fluxd/src/main.cpp:1398–1408]. The trusted setup behind those proofs was produced by a multi-party ceremony in the random-beacon model [11]; a newer construction removing the need for any such ceremony exists in the literature [12] but has not been ported into Flux's shielded pool, which remains on Sapling and Groth16 SHIPPED [fluxd/src/main.cpp:1450–1468].

The comparison changes direction going forward. Both pools are to be retired (§20.9), so Flux is moving *away* from the Zcash lineage on this axis while the systems it is compared against move toward stronger privacy. The resulting position is unusual and should be stated as such rather than blurred: on settlement privacy Flux will be comparable to Bitcoin rather than to Zcash, and weaker than every privacy-focused chain in this survey. What it gains is a consensus layer with no proving system, no trusted-setup inheritance and no unenforced turnstile — a smaller and more auditable object, which is the trade the team has chosen [intent: design intake]. None of this machinery is Flux's invention, and this paper does not claim otherwise: the cryptography, the setup, and the protocol are inherited in full.

Where Flux does make an original decision is consensus policy, not cryptography: since height 835,554, the same consensus path cited above rejects any transaction that mixes a transparent input with a shielded output or joinsplit, closing the pool to new value from the transparent side SHIPPED [fluxd/src/main.cpp:1398–1408]. Value already inside the pool can still move z-addr → z-addr or leave z-addr → t-addr, but nothing enters; the pool's value has been monotonically non-increasing for approximately 5.4 years, holding 77,188.76 FLUX in Sapling and 19,291.18 FLUX in Sprout at last measurement SHIPPED [measured: fluxd chain tip 2,917,115, 2026-09-03]. That single fact reorders how privacy commitments elsewhere should be read: reopening the pool would have been a consensus change and the prerequisite for every downstream privacy item, not a wallet-integration task — and it is now cancelled in favour of retirement (§20.9).

Payment channels are the other well-known route to transactional privacy — Lightning [39] keeps individual payments off-chain entirely, trading on-chain settlement for a channel-liquidity trust assumption. Flux does not take this route for deployment payment; §16 works over on-chain transparent settlement and §21 was specified over on-chain shielded settlement [intent: §16], so a channel network's liquidity and routing assumptions are not this paper's problem, but neither is its privacy benefit available for free. The specific problem §21 identifies is narrower and, so far as this corpus establishes, open: proving that a shielded payment funds a *particular* deployment without revealing which one is hard precisely because an application's address is itself a workload identifier, so a spend statement naming that address defeats the purpose of hiding it VISION [intent: feasibility assessment]. This has the shape of a blind-credential problem — proving membership in, or payment toward, a class without revealing which member — and the relevant literature is well developed: blind signatures [17], anonymous credentials [15], compact e-cash [16], anonymous authorization tokens [40], and zero-knowledge group membership with per-epoch nullifiers [43]. What none of it supplies directly is the combination §21 needs: a public obligation checkable by thousands of independent verifiers holding no shared secret, settled from a pool that already exists but cannot be entered, at a demand level where the anonymity set rather than the cryptography is the binding constraint.

28.8 Delegated authority

ERC-4337 [14] answers the delegation problem by putting a smart-contract account between a session key and the assets it can move: a paymaster or account contract can enforce a spending policy on-chain, because the account itself is a program the chain executes. That answer is not available on Flux's L1, which is a UTXO chain: script validation is a pure function of a spending transaction and its own inputs, with no access to spending history, so no script-level rule can enforce a rate limit or a cumulative cap on a delegated key (§17; [intent: feasibility assessment]). Macaroons [9] fit the UTXO setting better than account abstraction does, because they push attenuation into the credential itself rather than into on-chain account state: a macaroon's caveats are checked by whoever verifies the credential, not by a ledger, which matches how deployment specifications are already validated off-chain against owner signatures on every node (§19; [intent: feasibility assessment]). That correspondence is the basis for §17's specification of bounded deployment authority PLANNED [intent: §17]. It is worth stating precisely what already exists and what does not: a scoped, revocable, rate-limited bearer token already governs *actions* — starting and stopping rentals, starting and stopping deployments, and reading the balance — through the API of FluxEdge, Flux's separate, centralized GPU-rental product (§17; [intent: feasibility assessment]), and an MCP-exposed tool set can already sign and submit a deployment end to end on a custodial path with no bound on *value* at all SHIPPED [deploy-with-git-frontend/server/mcp/createServer.js:174–266]. §17 is therefore not inventing delegation from nothing; it specifies the value bound that today's action-scoped delegation does not have.

28.9 Agent interfaces

The Model Context Protocol [5] is an emerging, vendor-published convention for exposing tools to a language-model agent over a uniform interface, rather than a bespoke API per application. Flux already exposes deployment operations this way: `deploy_app`, `update_app` and `renew_app` are registered as MCP tools today SHIPPED [deploy-with-git-frontend/server/mcp/createServer.js:174–266]. §18 gives the fuller listing and states what the interface does not yet bound, which is the same value-bound gap §28.8 describes.

28.10 What this paper adds

Set against this prior art, three results in this paper are, to the corpus available here, original rather than inherited, and each is narrow. First, the fee-pool drip construction in §7 — paying the entire PoN rotation from a shared pool Φ at a fixed per-block rate Φ/K under PNR — is offered as a specific defence against a timing attack that a naive next-block payout admits; the mechanism composes an existing rotation with a new payment rule, and the fairness argument is this paper's, not inherited from any cited system PLANNED [intent: §7]. Second, §23's finding is a measurement, not a mechanism: collateral-weighted moderation is Sybil-resistant by construction [20], and, at the measured distribution — a Gini coefficient of 0.8375, with four owner identities (three under linkage) alone clearing the 25% eviction threshold — it is not censorship-resistant, and no reweighting inside the design space closes that gap SHIPPED [measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03]. Third, §22 and §4 together identify a foreclosure rather than a new construction: a header format chosen for PoN's own consensus reasons — fixed chainwork, no registry commitment, an ungated emergency path — rules out the light-client bridge architecture standard for account-based chains, a cost of that consensus design not stated elsewhere. None of the shielded pool, the Sapling proof system, the collateral-sortition idea itself, MCP, or macaroons is claimed as this paper's contribution; each is prior art this paper positions Flux against, mechanism by mechanism.

29 Limitations, non-goals, and open problems

The preceding sections describe what Flux does, does not yet do, and is specified to do. This section closes the description with three different kinds of boundary, and the three should not be confused with one another. First, what the network's own roadmap states it will deliberately not build, and why — a set of self-imposed limits on scope, not a gap in the argument. Second, what this paper itself could not establish from the evidence available to it — measurements never taken, provenance that cannot be pinned to a line of code, mechanisms documented nowhere in the assigned repositories. Third, what remains genuinely unsolved: properties argued not to hold in earlier sections, and research problems for which this paper states what a solution would have to look like without supplying one. None of the three categories is filler; each is load-bearing for how a reader should weigh the rest of the document.

29.1 Stated non-goals

The public roadmap carries its own section titled "What we are not doing," positioned as a top-level section immediately after the roadmap's discussion of supply and immediately before its account of how it reports progress — not a subsection of anything else [doc: flux-roadmap/public/flux-roadmap-public.md:203]. It is reproduced below in full, item by item, with its own stated reasoning and nothing added.

1. **"Reopening base emission upward."** Reasoning given: "PNR v2 operates on revenue, not issuance." (PLANNED; [roadmap: What we are not doing, item 1; flux-roadmap-public.md:205])
2. **"Making privacy mandatory."** Reasoning given: "The transparent layer stays permanently." (PLANNED; [roadmap: What we are not doing, item 2; flux-roadmap-public.md:206])

3. **"Reselling operators' bandwidth or IP addresses" for scraping.** Reasoning given in full: "It is a real revenue idea and other networks do it. It exposes operators to risk they did not agree to. The answer is no, permanently." (PLANNED; [roadmap: What we are not doing, item 3; flux-roadmap-public.md:207])

4. **" 'VPS on every node' in this cycle."** Reasoning given: "home connections cannot serve it reliably. The datacenter tier is what we can do properly now." (PLANNED; [roadmap: What we are not doing, item 4; flux-roadmap-public.md:208])

5. **"Adding new parallel-asset chains."** Reasoning given: "while we consolidate the ones we have." (PLANNED; [roadmap: What we are not doing, item 5; flux-roadmap-public.md:209])

6. **"Marketing a funnel we have not fixed."** Reasoning given: "The interface work comes first." (PLANNED; [roadmap: What we are not doing, item 6; flux-roadmap-public.md:210])

No other list of non-goals appears anywhere else in the public roadmap corpus [doc: flux-roadmap/public/timeline.html; flux-roadmap/public/flux-roadmap-web.html]. A publicly dated commitment not to do six specific things, each with its own stated reason, is unusual in the systems-and-token-network literature surveyed for this paper, and among everything read for it, it is the single piece of evidence that most directly argues for engineering judgment rather than messaging setting the roadmap [inferred].

29.2 Limitations of this paper

Every entry below is recorded in [doc: reconciliation record] as a claim the drafting process considered and could not support, together with what the paper does instead of asserting it.

No realised measurement. Two limitations are limits of public data, not of effort. First, no public endpoint reports bandwidth, latency, or realised utilisation of the deployed fleet — only requested resources from application specifications are visible. §26 reports committed capacity (8,972 vCPU, 17.1 TiB RAM, 160 TB disk) and states plainly that realised utilisation is not measurable from public data, rather than estimating it ([doc: reconciliation record]). Second, operator concentration is measurable only through the identities public chain data exposes. The owner-linked proxy is `payment_address`; `pubkey` is the node's VPS message-signing key and not the owner key a vote would carry [fluxd/src/fluxnode/fluxnode.h:154]. All such figures are lower bounds, since every distinct identity is counted as a distinct party. §23 and §26 state the figures as lower bounds explicitly, and this section restates the true concentration measurement, by party rather than by key, as an open problem below ([doc: reconciliation record]).

Not every cited repository is one the reader can open. This is the most important limitation on the paper's own method, and it is stated first among the caveats about citation rather than buried among them. The paper cites source by file and line throughout, and for the majority of its citations — `fluxd`, FluxOS, the network-policy channel, FDM, the node front end, the storage service — those repositories are public and any reader can check the line. For a minority they cannot: the benchmark client, the SAS and its enrollment tooling, the ArcaneOS image build, the HSM ceremony tooling, the release API, the auto-renewal service and the reachability fleet's inventory are private repositories. Claims cited to those rest on the authors' reading, and a reader has no way to audit them.

Two consequences are worth being explicit about. First, the paper's central promise — that the described behaviour could be reimplemented or audited from what is cited — holds fully for the public components and only conditionally for the private ones; where a load-bearing claim could be re-anchored to public source it has been, and §4's benchmark-key finding is cited to `fluxd`'s own consensus constants for exactly that reason [fluxd/src/chainparams.cpp:233–238]. Second,

two of these components are private *by decision* rather than by omission: the benchmark client and the SAS remain closed until a mechanism exists that verifies node trustworthiness in a way that cannot be faked [intent: design intake]. That decision is the reason §14.2 states one class of finding as properties without coordinates, and a reader who wants to hold this paper to its own standard should know that the redaction is deliberate, bounded to that class, and explained where it is applied rather than exercised silently.

No competitive price comparison. No pricing data for hyperscalers or peer networks was gathered, and constructing a comparison would itself be exactly the kind of price claim the paper's register constraint forbids. §28 compares architectures against related systems, never prices ([doc: reconciliation record]).

Provenance the corpus could not settle. Four specific facts resisted a citable, unambiguous source when drafting began; two have since closed. The upstream `fluxd` fork point cannot be read off a version string, but is now bracketed from consensus evidence — at or after `zcashd` v2.1.0 (Blossom) and before v2.1.2 (Heartwood) — rather than inferred from a release-notes directory listing (§20; [doc: reconciliation record]). The PoN activation height $h_{PoN} = 2{,}020{,}000$ is a consensus fact, but the commonly cited activation date does not appear anywhere in source; the nearest in-repository evidence is a checkpoint timestamp that brackets, without establishing, the date, so the height is cited from code and the date only as measured from a block explorer with a retrieval date ([doc: reconciliation record]). The Kadena parallel asset's claim and payout mechanism was absent from the repository named for it, which contains no code, and was located on a second pass in `fusion` and `flux-kda`; §6 now cites the claim path and the single-key treasury transfer, and what remains undocumented is how that treasury is replenished ([doc: reconciliation record]). And whether FluxOS actually consumes the `flux-spec` schema could not be determined from within `flux-spec` itself, which publishes no package and exports nothing, so §10 describes the two artifacts as separate rather than presenting them as one specification ([doc: reconciliation record]).

Claims tested and rejected. Eleven further claims were considered during drafting and found unsupportable; each is recorded in [doc: reconciliation record] with the claim, why it fails, and what the relevant section states instead. Four of them — that nodes vouch for what they serve, that benchmarking establishes hardware fitness, that PoN inherits Nakamoto-style reorg cost, and that placement is deterministic — restate system properties argued fully in earlier sections and are not re-argued here; they appear again, as properties rather than as refused claims, in the list of system properties below ([doc: reconciliation record]). The remaining seven are stated once, here, and nowhere else in this paper: that the deployment assistant never signs is false on the default server-side path, so §19 states the property per signing path rather than as one property of the system ([doc: reconciliation record]); that supply reaches a terminal value is false, so §5 gives the supply function as unbounded with a vanishing rate and states its tail explicitly ([doc: reconciliation record]); that Flux implements a proof-of-useful-work mechanism is false — the name survives only as a module name with no work-submission, verification, or reward protocol — so §15 describes the compute-rental marketplace as what it is and never invokes the term ([doc: reconciliation record]); that GPUs are hot-attachable is unsupported by any code in the four GPU-adjacent repositories, so §15 states whole-device passthrough on FluxEdge as the deployed mechanism and tags hot-attach VISION ([doc: reconciliation record]); that shielded supply is fully publicly verifiable is weakened by turnstile enforcement being compiled in but disabled, so §20 states pool accounting as it is and names turnstile activation as the condition that would make the claim true ([doc: reconciliation record]); that application data survives a payment lapse is unsupported because no grace, suspension, or eviction state machine exists in code, so §16 specifies one as PLANNED with open parameters and states the deployed baseline — expiry

arithmetic with no grace — separately ([doc: reconciliation record]); and that the network compensates a tenant for node failure is unsupported because no refund or SLA-credit mechanism exists anywhere, so §16 states that the remedy on offer is replacement, not refund ([doc: reconciliation record]).

29.3 Properties that do not hold

The following are limitations of the deployed and specified system, not of this paper's evidence. Each has already been argued in the section cited; they are collected here rather than re-argued.

- Benchmark results are not cryptographically bound to the operator or the host that produced them (§4). SHIPPED; [fluxd/src/chainparams.cpp:233–238].
- Attestations are not bound to the machine that produced them, and no tenant-facing verification path exists (§14). SHIPPED; [private source: SAS API server].
- PoN fork choice is a block-count race, not cumulative work: every block carries a fixed 2^{40} chainwork contribution regardless of its difficulty target (§4). SHIPPED; [fluxd/src/pow.cpp:284–290].
- A 2-of-4 emergency key set can produce blocks with no time or stall-detection gating (§4). No change to the threshold is planned, and the threshold is not the property that matters: the gating is absent at any size. SHIPPED; [fluxd/src/emergencyblock.cpp:183–191].
- Placement is opportunistic self-selection with a 90-second broadcast collision wait, not deterministic scheduling (§9). SHIPPED; [flux/ZelBack/src/services/appLifecycle/appSpawner.js:54–65,77–787].
- `fluxos-network-policy` is unsigned and takes fleet-wide effect within 7–13 hours, changeable without a required review step (§12). SHIPPED; [fluxos-network-policy/README.md:30-32]; [fluxos-network-policy/.github/CODEOWNERS:1-15].
- `api.runonflux.io` is the routing layer's (FDM's) sole application-discovery source, with no on-chain or peer-to-peer fallback, although the nodes themselves replicate specifications and locations by peer-to-peer gossip (§12). SHIPPED; [flux-domain-manager/src/services/flux/dataFetcher.js:47-91].
- The shielded pool cannot be entered: transparent value has been blocked from joining it since height 835,554 (§20). SHIPPED; [fluxd/src/main.cpp:1398–1408].
- As specified, PNR would create no marginal incentive to host, because payment reaches every node through the PoN rotation independent of what that node runs (§7). PLANNED; [intent: mechanism specification].
- Four owner identities (three under linkage) reach the moderation eviction threshold at the measured collateral distribution (§23). SHIPPED; [measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03].
- A Flux light client is not constructible on an external chain under the current header format, which forecloses light-client bridging (§22). SHIPPED, as a consequence of the header facts above; [fluxd/src/pow.cpp:284–290]; [intent: mechanism specification].
- On FluxEdge, GPU resource requests are silently stripped, rather than queued or rejected, when a host is contended (§15). SHIPPED; established in §15.
- `Flux-Shared-DB` acknowledges a write before it has replicated (§13). SHIPPED; [Flux-Shared-DB/ClusterOperator/Operator.js:1265-1338]; [Flux-Shared-DB/ClusterOperator/server.js:1543-1544].

29.4 Open problems

The following four items are classed *Research* in the feasibility assessment: each names an open problem with no known good answer under the constraints Flux operates within, rather than a build that merely has not happened yet [doc: reconciliation record]. Each is stated with what would have to be true of a solution, not merely that a solution is hard to find.

Open Problem 29.1 (Post-quantum migration of outputs the protocol cannot move). A repository-wide search of `fluxd`'s source returns no post-quantum primitive of any kind [doc: reconciliation record]. The roadmap states the realistic threat as store-now-decrypt-later rather than live breakage, and frames migration as a multi-year effort, explicitly placing Flux where Bitcoin and Ethereum are today: "The realistic threat is store-now-decrypt-later, not live breakage. We publish a migration path on the NIST-standardised signature schemes. Migration itself is a multi-year effort — this is where Bitcoin and Ethereum are too." [roadmap: flux-roadmap-public.md:148–149]. A solution would have to address two different populations of value separately. Node collateral is a registered, enumerable set, so a migration targeting it first is tractable in a way a general UTXO migration is not — that is a structural advantage, not a solved problem, and the paper states it rather than claiming credit for it [doc: reconciliation record]. Value sitting in ordinary transparent or shielded outputs the protocol has no authority to move is the harder case: a solution there would need either a voluntary, incentivised migration window before a cryptographically relevant quantum adversary is credible, or an accepted statement that some outputs will simply not migrate.

Open Problem 29.2 (Bounded agent spending on a UTXO chain without a new trusted party). Script validation on `fluxd` is a pure function of the spending transaction and its own inputs; it cannot observe transaction history, so no script-level rule can enforce a rate limit or a cumulative spending cap [doc: reconciliation record]. The deployable answer available today — fund a dedicated key and let its balance be the budget — is honest but coarse: it bounds total exposure, not rate, and it cannot be revoked partway through a spend. A solution that bounds spending more richly than a funded sub-key would have to supply either a co-signer, which reintroduces a trusted party the rest of this paper's delegation design (§17) is built to avoid, or new consensus state that lets a script condition on cumulative spend over a window — a Flux L1 consensus change with its own audit and activation cost. Progress here means stating explicitly which of those two costs a design accepts, not finding a way to avoid both.

Open Problem 29.3 (Unlinkable payment for a specific deployment). An application's on-chain address is itself a workload identifier, so a construction proving that a payment funded *that* address defeats its own purpose the moment it succeeds [doc: reconciliation record]. Two further constraints, drawn from the formal §21 design, narrow what a solution could look like. First, amount hiding is unsatisfiable for any payment typed by PNR, because the fee pool is a public consensus integer and the payment amount is recoverable from it to within a few satoshi — a solution has to be scoped to exclude PNR-typed payments from the start, not attempt to hide their amount. Second, the anonymity set is small by construction: at a measured renewal rate, a renewal payment is a singleton within its own block essentially always and within its own day the majority of the time, so a solution built on payer/workload unlinkability alone starts from a small crowd, not a large one. A solution would have to be either a zero-knowledge statement and circuit proving membership in a set of valid, unconsumed entitlements without naming which one, verified within the existing block-interval budget, or a blind-credential design with a third-party renewer that moves the trust assumption rather than removing it — and it would have to say which of those two it is, rather than presenting payment-key indirection alone as a solution, since indirection does not close the linkage on its own.

Open Problem 29.4 (Censorship resistance under measured collateral concentration). At the measured ownership distribution, four owner identities (distinct `payment_address` values) already clear the 25% eviction threshold, falling to three once addresses linked by a shared node key or

a shared collateral funding transaction are merged; the largest single identity holds 10.71% of collateral-weighted voting power across 237 Stratus nodes [doc: open-decisions record, item 11]. No reweighting, threshold change, or remedy within the moderation quorum's design space lowers that number: the best available reweighting moves the threshold from three colluding identities to six while making the threshold itself up to 13.6× cheaper to buy, which is a worse trade, not a better one. A solution is therefore not a mechanism-design exercise inside the quorum; it is one of two things. Either it is a construction, such as the two-chamber rule this paper's design work recommends, that raises the cost of a *surprise* capture — requiring a coalition to be pre-positioned and visible on-chain in advance — without claiming to raise the cost of capture itself, since no such claim would be true; or it is an independent remeasurement of concentration keyed to collateral-spending scripts rather than to node-signing keys, which could show the true party-level concentration is lower than this lower bound suggests, and which has not been done. Progress on this problem means one of those two things happening, not a better-tuned threshold.

29.5 Open decisions

[doc: open-decisions record] lists roughly eighty parameters that were open when drafting began across the mechanisms this paper specifies. Reproducing all of them here would not be a limitations section; it would be an appendix. Eighteen rounds of the design intake [intent: design intake] have since settled most of them, each in the section that uses the value; the table below gives what is still open — decisions no round has ruled on, and values this paper proposed that the author has not confirmed, which round 18 requires to be marked as recommendations rather than settled — ordered by the section that carries each. Where the source does not name who is responsible for a decision, the table says so rather than inventing a role.

Table 86: The decisions still open after rounds 2–18 of the design intake, ordered by the section that carries each: items no round has ruled on, and values this paper proposed that the author has not confirmed, each marked in place as *Proposed by this paper*. The settled items this table formerly listed are recorded in the sections that apply them.

Decision	Who decides	Blocks
Emergency-block governance: the threshold stays at 2-of-4; whether the missing time or stall-detection gating is remedied (time-lock, stall detection, retirement after PoN stabilisation)	the author, explicitly	§4, §8, §27
PNR settlement: the four parameters still open after the author's confirmation of Table 24 — O4 (fee floor and spam resistance), O8 (which payment channels must settle through sink, the load-bearing one), O14 (the identity of sink) and O19 (the tier partition basis, to confirm only)	not named in source	§7, §16
The transformation per source version for the 322 applications on SPEC v2–SPEC v7 under forced automatic migration at $h^\star$ = 3,050,000; none is stated in any artefact read for this paper	not named in source	§10, §27
`quorumGrantActivationHeight`: the observed SPEC v9-stability condition that triggers it and the resulting height, deliberately decoupled from $h^\star$ [intent: design intake]	not named in source	§10

(Table 86 continued)

Decision	Who decides	Blocks
Bonded attestation network parameters — $n = 9$, $k = 6$, $\Delta_{\text{chal}} = 2{,}880$ blocks, full slashing, β_i scaled to the value secured, re-checkable claims only at launch — are this paper's recommendations, unconfirmed	the author	§8, §14
Delegation certificate values — the allowed-key set as a seventh scope component, a funded allowance with no co-signer, no destination constraint, a re-issuance window of 8,640 blocks, digest resolution in the tooling — are this paper's recommendations, unconfirmed	the author	§17, §18
Two signing-path architecture: describe the local-signing and operator-signing paths, as §19 does, or scope §19 to local signing alone; listed as still to be discussed [intent: design intake] and not returned to since	the author (editorial framing decision)	§17, §18, §19
ZIP-209 turnstile enforcement with Flux-correct constants for the drain-only window: recommended by this paper; round 11 confirmed the window's heights, not the switch [intent: design intake]	the author	§5, §20, §21
Bridge: the checkable renunciation criteria (twelve months, 10,000,000 FLUX of cumulative volume, zero defect-fixing upgrades), the delaying implementation of never-refused burns, and direct calldata notice to legacy holders are this paper's proposals; settled beneath them are the audit-plus-operating-record shape (round 12) and the 10,000,000 FLUX per-chain daily ceiling (round 7)	the author	§22
Contract-held legacy balances on Ethereum and BSC: the set of LP, lending and bridge positions and their per-protocol resolution to beneficial holders, to be published before h_{snap}; the treatment is settled [intent: design intake] and the enumeration is not measured in this corpus	the Flux team and exchange partners	§22
Moderation category taxonomy: the enumeration of reportable categories, whose scope is ruled behavioural harm [intent: design intake] and whose members are undecided	not named in source	§23

29.6 What the next revision must do

Some of this has closed on a schedule already stated elsewhere in this paper: the shielded pools' sunset height is settled (§20), a SPEC *v*9 enforcement height was proposed although SPEC v9 remains in development and has not activated (§10), the fork point is bracketed from consensus evidence (§20), and the activation-date provenance gap is a bounded task — a corroborated explorer citation — rather than an open problem. Some of it is not expected to close by the

next revision, and should not be presented as though it will: the four Research items above are open because the field does not have an answer, not because this paper did not look hard enough, and a future draft that quietly drops one of them without a cited construction should be read as having weakened its claim, not strengthened it. The standard by which a reader should judge the next version of this document is not whether its count of open parameters and *not established* notes reaches zero. It is whether every claim that moved from PLANNED to SHIPPED did so because a `[code:]` citation now exists for it, whether every [inferred] claim that remains is still visibly hedged, and whether the non-goals list above is still true.

30 Conclusion

This paper set out two documents in one. The first is a description, at reimplementation depth, of Flux as it runs today: a consensus layer that gates block production on collateral rather than hashpower, an orchestration layer that places 1,195 deployed applications across 6,489 collateralised nodes by opportunistic self-selection (§3, §26), and the addressing, attestation and settlement machinery that keeps that placement reachable and paid. The second is a specification, at the same depth, of the architecture that follows if autonomous callers become the network's principal tenant: demand-funded compensation, delegated agent authority, transparent settlement after the shielded pools are retired, bounded and publicly verifiable bridging, and a hardened attestation substrate (Part IV). The two halves are written to the same standard: every element of the second that is undecided is named as an open parameter or an open problem (§29) rather than filled with a plausible default.

The thesis under test, stated in §1, is a bet, not a finding: that autonomous software becomes the dominant consumer of cloud capacity, and that Flux's redesign follows from taking that bet seriously. This paper does not argue the bet itself. What it argues is conditional and checkable — given the bet, whether the remaining work is extension of what already exists or invention of what does not — and §25 answers it by classifying every mechanism in Part IV against that distinction: of twenty-two assessed, six are Extension, twelve are Engineering, and four remain Research (§25, tally). Most of what the bet requires, this paper finds, Flux can build by extending components already in production. Four things it cannot.

The four are these, without re-arguing them here: post-quantum migration of outputs the protocol cannot move; bounded agent spending on a chain whose script cannot observe history; unlinkable payment for a specific deployment; and censorship resistance under measured collateral concentration (§25, §29).

One finding among these is the most consequential for how the remaining work should be ordered. PNR, as specified, would pay every node through the PoN rotation independent of what that node hosts, so the eligibility gate — not the payment — is what compels hosting (§7). That gate is presently bound to neither the operator who ran the benchmark nor the machine that produced the attestation (§4, §14). Activating PNR before that binding exists would raise the payoff to passing the gate without doing the work, by exactly what PNR pays — about 4.5 % on top of the issuance that already flows through the same gate. Hardening attestation is therefore the highest-priority security item on its own merits, and PNR's ramp, not its activation, should track it (§25, §27).

The argument is falsifiable in two independent places. The bet itself fails if, once the delegated authority and machine-facing interfaces this paper specifies are wired and enforced (§17, §18), the share of deployments attributable to autonomous callers rather than interactive operators stays negligible — the redesign would then be solving a problem the network does not have, and no property argued in Parts II–IV would rescue that. The sequencing claim above fails on a narrower and nearer-term observation: if PNR activates while the benchmark and attestation signatures remain unbound (§4, §14), and the network subsequently observes eligible nodes growing faster than any measure of what they host, that is exactly the gaming this paper's ordering was written to prevent, and it would show the priority argument of §27, not merely one

component, to be wrong.

The open markers scattered through this document — open parameters, open problems, and the notes marked *not established by this survey* — are not a defect awaiting a final edit; they record decisions that a human must make and that this paper deliberately left to a human rather than making silently. The next version of this document should be judged by how many of them closed and whether closing them changed the mechanisms it specifies, not by whether their count reached zero.

Acknowledgements

Nothing described in the first half of this paper was built by its author alone, and most of it was not built by its author at all. Flux has produced blocks since January 2018 and run an orchestration layer on top of them since 2019; eight years of that work is what made a document of this kind possible, because a specification can only be checked against a system that exists.

My co-founders — **Daniel Keller**, **Jeremy Anderson** and **Valter Silva** — have carried this project alongside me since the beginning, and their contribution runs across the whole of the Flux ecosystem rather than any one component of it: the chain, the cloud, the products built on both, and eight years of keeping all of it operating.

The same is true of the developers who built what this paper documents. **David White** is principally responsible for ArcaneOS, the measured-boot operating system that Section 14 spends most of its length examining; the substrate that section describes is his design. **Kamil Piekarski**, **Ali Mahdavi**, **Vasilis Magkoutis**, **Daz Williams** and **Alex Perritaz** have built and operated large parts of the ecosystem across its layers, and the reader who encounters a component in Parts III and IV that simply works is encountering their work rather than any argument of mine.

More broadly, the engineering, operations and support teams at InFlux Technologies wrote, deployed and have kept running nearly every component described here: `fluxd`, FluxOS, `fluxbench`, ArcaneOS, FDM, PDNS, the attestation service, the wallets and the tooling around them. Where this paper reads a design and finds it sound, that is their work.

The node operators are the network. At the census this paper measures, 6,489 machines were run by people who chose to post collateral and keep hardware online, in more than sixty countries, answerable to no one for doing so. Every capacity figure in Section 3, every concentration measurement in Section 23, and the entire argument of Section 7 is a statement about their machines and their decisions. A decentralized network is an assertion until somebody runs the nodes.

To the wider Flux community — the operators who have tested releases, the developers who have built on the platform, and everyone who has argued with the roadmap in public — thank you. Several of the sharpest questions this paper tries to answer were asked first by people who do not work here.

Responsibility. The findings in this paper are the author's own, and so are its criticisms. It records places where the deployed system is weaker than it appears — trust boundaries that concentrate more than a reader would expect, an incentive that does not do what its name suggests, an attestation that does not bind to a machine — and no one named above has endorsed those conclusions or is answerable for them. Errors of fact, of reasoning and of judgement are mine.

References

[1] Flash-loan-funded governance vote on MakerDAO and the subsequent GSM delay increase. Contemporaneous reporting. On 2020-10-26 a participant used flash-borrowed MKR (approximately $7M) to pass a whitelisting vote and returned the tokens in the same transaction;

on 2020-10-30 MakerDAO initiated an executive vote extending the governance security module delay from 12 to 72 hours, October 2020. https://www.coindesk.com/tech/2020/10/29/flash-loans-have-made-their-way-to-manipulating-protocol-elections.

[2] Compound governance proposal 289: Trust setup for dao investment into goldcomp. On-chain governance proposal, Compound DAO. Proposed 2024-07-24; voting closed 2024-08-01 with 682,190 COMP for and 633,640 against; canceled before execution. Sought to move 499,000 COMP into the goldCOMP vault, July 2024. https://www.tally.xyz/gov/compound/proposal/289.

[3] Aethir. Aethir. URL https://aethir.com/. Decentralized GPU cloud infrastructure.

[4] Akash. Akash network. URL https://akash.network/. Decentralized compute marketplace.

[5] Anthropic. Model context protocol, 2024. URL https://modelcontextprotocol.io/.

[6] Hunter Beast, Ethan Heilman, and Isabel Foxen Duke. BIP-360: Pay-to-merkle-root (P2MR). Bitcoin Improvement Proposal, Draft (Specification), assigned 2024-12-18. Defines an output type offering nearly P2TR functionality with the key-path spend removed, so no internal public key is exposed; renamed from P2QRH via P2TSH. Explicitly does not introduce post-quantum signature schemes, 2024. https://github.com/bitcoin/bips/blob/master/bip-0360.mediawiki.

[7] Eli Ben-Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. Zerocash: Decentralized anonymous payments from bitcoin. In *IEEE Symposium on Security and Privacy (S&P)*, 2014.

[8] Juan Benet. IPFS - content addressed, versioned, P2P file system, 2014. arXiv:1407.3561.

[9] Arnar Birgisson, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrable, and Mark Lentczner. Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. In *Network and Distributed System Security Symposium (NDSS)*, 2014.

[10] Joseph Bonneau, Andrew Miller, Jeremy Clark, Arvind Narayanan, Joshua A. Kroll, and Edward W. Felten. Sok: Research perspectives and challenges for bitcoin and cryptocurrencies. In *IEEE Symposium on Security and Privacy (S&P)*, 2015.

[11] Sean Bowe, Ariel Gabizon, and Ian Miers. Scalable multi-party computation for zk-snark parameters in the random beacon model, 2017. Cryptology ePrint Archive, Report 2017/1050.

[12] Sean Bowe, Jack Grigg, and Daira Hopwood. Recursive proof composition without a trusted setup, 2019. Cryptology ePrint Archive, Report 2019/1021.

[13] Vitalik Buterin and Virgil Griffith. Casper the friendly finality gadget, 2017. arXiv:1710.09437.

[14] Vitalik Buterin, Yoav Weiss, Dror Tirosh, Shahaf Nacson, Alex Forshtat, Kristof Gazso, and Tjaden Hess. ERC-4337: Account abstraction using alt mempool, 2021. Ethereum Improvement Proposals.

[15] Jan Camenisch and Anna Lysyanskaya. An efficient system for non-transferable anonymous credentials with optional anonymity revocation. In *EUROCRYPT*, 2001.

[16] Jan Camenisch, Susan Hohenberger, and Anna Lysyanskaya. Compact e-cash. In *EUROCRYPT*, 2005.

[17] David Chaum. Blind signatures for untraceable payments. In *CRYPTO*, 1982.

[18] Circle. Cross-chain transfer protocol. URL https://developers.circle.com/stablecoins/cctp-getting-started. Burn-and-mint cross-chain transfer with an attestation service.

[19] Victor Costan and Srinivas Devadas. Intel sgx explained, 2016. Cryptology ePrint Archive, Report 2016/086.

[20] John R. Douceur. The sybil attack. In *International Workshop on Peer-to-Peer Systems (IPTPS)*, 2002.

[21] EigenLayer. Eigenlayer: The restaking collective. URL https://www.eigenlayer.xyz/. Restaked bonded operators making slashable off-chain assertions.

[22] Ittay Eyal and Emin Gün Sirer. Majority is not enough: Bitcoin mining is vulnerable. In *Financial Cryptography and Data Security (FC)*, 2014.

[23] Flux. Flux ecosystem whitepaper 2.1. Three locators, none canonical: served by the Flux-hosted application `FluxWhitepaper`, built from `littlestache/fluxwhitepaper` on Docker Hub (tag `latest`, pushed 2022-06-30, 38,190 pulls at 2026-09-06); mirrored by a third party at https://www.scribd.com/document/704953562/Flux-Whitepaper-2-1. The ecosystem description this document supersedes. It states no author, date or version inside the document itself; the version number is the filename's. Each locator is imperfect in its own way — the application endpoint is not a stable address, the Docker tag is mutable and publishes no digest, and the Scribd copy is a paywalled upload by a third party rather than a publication by Flux. An IPFS pin, as the consensus whitepaper has, would give this document a content-addressed locator that none of the three provides; that remains outstanding.

[24] Flux. Flux blockchain proof of useful work v2 (proof of node): whitepaper. https://github.com/RunOnFlux/whitepaper-pouw at commit `ecafa8603141` (2025-07-08); also published to IPFS as `QmW3TNPx5Ru3u3UhXf8GqyVNS3avSsttqjamhF395qe17M` and linked from `fluxd`'s README, 2025. The principal predecessor to this document, which supersedes it and builds on it: §4 carries its Proof-of-Node construction forward as deployed code, and §6 reconciles its supply model. The repository carries the LaTeX source, the built PDF and the supply model, so it is the versioned locator; verified public 2026-09-06.

[25] Flux. FluxOS node reward distribution model: resource-based economics with 30-second blocks. https://github.com/RunOnFlux/whitepaper-pouw at commit `ecafa8603141`, file `FluxOS_Node_Reward_Distribution_Model.md`, 2025. States the design intent behind the $1 : 3.5 : 9$ tier bases — the ordering constraints 2 Nimbus $<$ 1 Stratus $<$ 3 Nimbus and 3 Cumulus $<$ 1 Nimbus $<$ 4 Cumulus — which §5 and §7 of this paper document as deployed constants without, until now, a source for the reasoning.

[26] Arthur Gervais, Ghassan O. Karame, Karl Wüst, Vasileios Glykantzis, Hubert Ritzdorf, and Srdjan Capkun. On the security and performance of proof of work blockchains. In *ACM Conference on Computer and Communications Security (CCS)*, 2016.

[27] Yossi Gilad, Rotem Hemo, Silvio Micali, Georgios Vlachos, and Nickolai Zeldovich. Algorand: Scaling byzantine agreements for cryptocurrencies. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2017.

[28] Golem. Golem network. URL https://golem.network/. Decentralized compute marketplace.

[29] Jens Groth. On the size of pairing-based non-interactive arguments. In *EUROCRYPT*, 2016.

[30] Maurice Herlihy. Atomic cross-chain swaps. In *ACM Symposium on Principles of Distributed Computing (PODC)*, 2018.

[31] Daira Hopwood, Sean Bowe, Taylor Hornby, and Nathan Wilcox. Zcash protocol specification. URL https://zips.z.cash/protocol/protocol.pdf. Version 2022.3.8 and later revisions.

[32] Interchain Foundation. Inter-blockchain communication protocol (IBC). URL https://www.ibcprotocol.dev/. Light-client based cross-chain messaging.

[33] io.net. io.net. URL https://io.net/. Decentralized GPU compute network.

[34] Aggelos Kiayias, Alexander Russell, Bernardo David, and Roman Oliynykov. Ouroboros: A provably secure proof-of-stake blockchain protocol. In *CRYPTO*, 2017.

[35] Tadeas Kmenta. Let's talk Flux: a brief introduction to ZelFlux. https://medium.com/@kmentat/lets-talk-flux-a1f085baec0a, March 2020. Published 2020-03-18T03:17:28Z per the `datePublished` metadata of an Internet Archive capture of 2020-11-06. Archival fixity confirmed 2026-09-06: the Wayback availability API reports a status-200 capture at `20260711070259`. Gives the MEVN and Docker architecture of FluxOS from its architect.

[36] Tadeas Kmenta. Deterministic ZelNode setup vs standard ZelNode setup. https://medium.com/@kmentat/deterministic-zelnode-setup-vs-standard-zelnode-setup-9f147713ca2, March 2020. Dated 16 March 2020 in the account's feed; the date itself was not independently confirmed. Archival fixity confirmed 2026-09-06: the Wayback availability API reports a status-200 capture at `20260711070259`. States the three-tier division by server specification and locked collateral.

[37] Jameson Lopp, Christian Papathanasiou, Ian Smith, Joe Ross, Steve Vaile, and Pierre-Luc Dallaire-Demers. BIP-361: Post quantum migration and legacy signature sunset. Bitcoin Improvement Proposal, Draft (Informational), assigned 2026-02-11. Phase A at 160,000 blocks after activation disallows sending to quantum-vulnerable addresses; Phase B, two years later, tightens ECDSA/Schnorr verification. Rescue rests on the knowledge asymmetry created by BIP-32 hardened derivation, 2026. https://github.com/bitcoin/bips/blob/master/bip-0361.mediawiki.

[38] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system, 2008. URL https://bitcoin.org/bitcoin.pdf.

[39] Joseph Poon and Thaddeus Dryja. The bitcoin lightning network: Scalable off-chain instant payments, 2016. URL https://lightning.network/lightning-network-paper.pdf.

[40] Privacy Pass. Privacy pass. URL https://datatracker.ietf.org/wg/privacypass/about/. Anonymous authorization tokens; IETF Privacy Pass working group.

[41] Protocol Labs. Filecoin: A decentralized storage network, 2017. URL https://filecoin.io/filecoin.pdf.

[42] Render. Render network. URL https://rendernetwork.com/. Distributed GPU rendering network.

[43] Semaphore. Semaphore: Zero-knowledge signalling on ethereum. URL https://semaphore.pse.dev/. Anonymous group membership with per-epoch nullifiers.

[44] Karl Wüst and Arthur Gervais. Do you need a blockchain? In *Crypto Valley Conference on Blockchain Technology (CVCBT)*, 2018.

[45] Alexei Zamyatin, Dominik Harz, Joshua Lind, Panayiotis Panayiotou, Arthur Gervais, and William J. Knottenbelt. Xclaim: Trustless, interoperable, cryptocurrency-backed assets. In *IEEE Symposium on Security and Privacy (S&P)*, 2019.

[46] Alexei Zamyatin, Mustafa Al-Bassam, Dionysis Zindros, Eleftherios Kokoris-Kogias, Pedro Moreno-Sanchez, Aggelos Kiayias, and William J. Knottenbelt. Sok: Communication across distributed ledgers. In *Financial Cryptography and Data Security (FC)*, 2021.

[47] Zel. ZelNodes benchmarking reference guide. `https://zel.gitbook.io/zelnodes/benchmarking-reference-guide`. The per-tier sysbench and IOPS thresholds; verified reachable 2026-09-06. Undated.

[48] Zel Official. ZelNodes — the decentralized, scalable, high-availability computing network. `https://fluxofficial.medium.com/`, October 2018. Dated 10 October 2018 in the compiled index used here; the date was not independently re-verified, as Medium returns HTTP 403 to scripted fetches.

[49] Zel Official. Everything to know about ZelFlux OS — the Zel computational network. `https://fluxofficial.medium.com/`, October 2019. Dated 8 October 2019 in the compiled index used here, with a companion article of 20 September 2019 announcing the debut; dates not independently re-verified (Medium returns HTTP 403 to scripted fetches).

[50] Zel Technologies. ZelCash whitepaper v2. `https://github.com/zelcash/zel.network` at commit `69ab6b7`, July 2018. Dated 16 July 2018; repository verified reachable 2026-09-06. Names Tadeas Kmenta as Partner and Lead Developer (p. 25).

A Complete Constant Tables

This appendix collects, without rounding or normalisation, every consensus, policy, and operational constant cited elsewhere in this paper, plus the constants that did not fit in the main text. Every row carries a provenance column: [repo/file:line] for a constant read directly from source, or [measured: endpoint, date] for a value read from a live API response. Identifiers are reproduced exactly as they appear in source, including capitalisation and underscores. Where two evidence sources disagree, or where a single named constant takes different effective values at different chain heights, both values are given and the disagreement is marked rather than resolved silently. Maturity tags (SHIPPED/IN-PROGRESS/PLANNED/VISION) are omitted from this appendix: every row is a fact about shipped code or a live measurement, except the rows cited to a SPEC v9 development branch, which describe unshipped code; no row is a claim about product maturity.

A.1 Chain identity and network

Table 87 covers `fluxd`'s magic bytes, P2P/RPC ports, pruning height, and client-version fields across mainnet, testnet, and regtest.

Table 87: Chain identity and network constants (`fluxd`).

Constant	Value	Provenance
`pchMessageStart` (mainnet)	`{0x24,0xe9,0x27, 0x64}`	[fluxd/src/chainparams.cpp:175–178]
`pchMessageStart` (testnet)	`{0xfa,0x1a,0xf9, 0xbf}`	[fluxd/src/chainparams.cpp:407–410]
`pchMessageStart` (regtest)	`{0xaa,0xe8,0x3f, 0x5f}`	[fluxd/src/chainparams.cpp:626–629]
P2P port (mainnet)	16125	[fluxd/src/chainparams.cpp:180]
P2P port (testnet)	26125	[fluxd/src/chainparams.cpp:414]
P2P port (regtest)	26126	[fluxd/src/chainparams.cpp:631]
RPC port (mainnet)	16124	[fluxd/src/chainparamsbase.cpp:21]
RPC port (testnet)	26124	[fluxd/src/chainparamsbase.cpp:34]
RPC port (regtest)	26124	[fluxd/src/chainparamsbase.cpp:48]
`nPruneAfterHeight` (mainnet)	100000	[fluxd/src/chainparams.cpp:181]
`nPruneAfterHeight` (testnet)	1000	[fluxd/src/chainparams.cpp:416]
`nPruneAfterHeight` (regtest)	1000	[fluxd/src/chainparams.cpp:632]
`CLIENT_VERSION_MAJOR`	9	[fluxd/src/clientversion.h:19–25]
`CLIENT_VERSION_MINOR`	1	[fluxd/src/clientversion.h:19–25]
`CLIENT_VERSION_REVISION`	0	[fluxd/src/clientversion.h:19–25]
`CLIENT_VERSION_BUILD`	50	[fluxd/src/clientversion.h:19–25]
`CLIENT_VERSION` (computed: $1000000 \cdot 9 + 10000 \cdot 1 + 100 \cdot 0 + 50$)	9010050	[fluxd/src/clientversion.h:19–25]
`IS_RELEASE`	`true`	[fluxd/src/clientversion.h:19–25]
`COPYRIGHT_YEAR` (stale, not updated since)	2022	[fluxd/src/clientversion.h:31]
`fluxd` client version 9.1.0 (package version string)	`9.1.0`	[fluxd/configure.ac:3–11]

A.2 Network upgrades

Table 88 lists every entry of `fluxd`'s `Consensus::UpgradeIndex` table in activation order (mainnet), together with its activation-block hash where the consensus code sets one; `UPGRADE_PON` is the only entry with no `hashActivationBlock` set.

Table 88: Network upgrade activation heights and block hashes (mainnet, `fluxd`).

Upgrade	Height	Activation-block hash	Provenance
`BASE_SPROUT`	`ALWAYS_ACTIVE`	—	[fluxd/src/chainparams.cpp:108–110]
`UPGRADE_TESTDUMMY`	`NO_ACTIVATION_HEIGHT`	—	[fluxd/src/chainparams.cpp:112–114]
`UPGRADE_LWMA`	125000	(none set)	[fluxd/src/chainparams.cpp:116–117]
`UPGRADE_EQUI144_5`	125100	(none set)	[fluxd/src/chainparams.cpp:119–120]
`UPGRADE_ACADIA`	250000	`0000001d65fa78f2f6c172a51b5aca59ee1927e51f728647fca21b180becfe59`	[fluxd/src/chainparams.cpp:122–125]
`UPGRADE_KAMIOOKA`	372500	`00000052e2ac144c2872ff641c646e41dac166ac577bc9b0837f501aba19de4a`	[fluxd/src/chainparams.cpp:127–130]
`UPGRADE_KAMATA`	558000	`000000a33d38f37f586b843a9c8cf6d1ff1269e6114b34604cabcd14c44268d4`	[fluxd/src/chainparams.cpp:132–135]
`UPGRADE_FLUX`	835554	`000000ce99aa6765bdaae673cdf41f661ff20a116eb6f2fe0843488d8061f193`	[fluxd/src/chainparams.cpp:137–140]
`UPGRADE_HALVING`	1076532	`000000111f8643ce24d9753dbc3242208877299075a8a6102da61ef4460296325`	[fluxd/src/chainparams.cpp:142–145]
`UPGRADE_P2SHNODES`	1549500	`00000009f9178347f3dea495a089400050c3388e07f9c871fb1ebddcab1f8044`	[fluxd/src/chainparams.cpp:147–150]
`UPGRADE_PON` (mainnet)	2020000	*none set*	[fluxd/src/chainparams.cpp:152–153]
`UPGRADE_PON` (testnet)	800	*none set*	[fluxd/src/chainparams.cpp:394]
`UPGRADE_PON` (regtest)	`NO_ACTIVATION_HEIGHT`	*none set*	[fluxd/src/chainparams.cpp:610–611]

Each upgrade also carries a fixed `nProtocolVersion` and a `nBranchId = 0x76b809bb` shared by all post-LWMA upgrades except Sprout/testdummy [fluxd/src/consensus/upgrades.cpp:12–68]. The exact wall-clock timestamp of `UPGRADE_PON`'s activation is not stored in `fluxd` as a literal; the closest available evidence is the checkpoint at height 2022000, timestamp `1761701944` (2025-10-29), which only brackets the activation [fluxd/src/chainparams.cpp:281,284] [inferred].

A.3 Emission

Table 89 covers `fluxd`'s monetary-policy constants for both the legacy proof-of-work slow-start/halving path and the post-PoN emission path; MAX_MONEY is a per-transaction sanity bound enforced by `MoneyRange()`, not a supply cap [fluxd/src/amount.h:24–31]. Values were cross-checked against an independent Python re-derivation, `flux-roadmap/supply/emission_model.py`, which reproduces every figure below identically from its own transcription of the same `fluxd` source [flux-roadmap/supply/emission_model.py:133–172].

Table 89: Emission and monetary-policy constants (`fluxd`).

Constant	Value	Provenance
`COIN`	100000000 sat/ FLUX	[fluxd/src/amount.h:17]
MAX_MONEY (per-tx sanity bound, *not* a supply cap)	440000000 · COIN = 440000000 FLUX	[fluxd/src/amount.h:31]
`nSubsidySlowStartInterval`	5000 blocks	[fluxd/src/chainparams.cpp:90]
`nSubsidyHalvingInterval`	655350 blocks ("2.5 years", comment)	[fluxd/src/chainparams.cpp:91]
Base pre-slow-start-end subsidy	150 · COIN = 150 FLUX	[fluxd/src/main.cpp:2818]
Premine (per code: height 1)	13020000 · COIN = 13020000 FLUX	[fluxd/src/main.cpp:2820–2822]
Premine height (per code, `GetBlockSubsidy`)	1	[fluxd/src/main.cpp:2820–2822]
Premine height (as observed on mainnet)	2 – **disagrees with the code-path height above; not independently re-verified against the chain by this survey**	[flux-roadmap/supply/emission_model.py:142] (model's own claim, "verified on chain")
Hardcoded halving freeze	`halvings >= 2` ⇒ `nSubsidy >>= 2` (locked at the 2nd-halving level, i.e. 37.5 FLUX, rather than continuing to halve every `nSubsidyHalvingInterval` blocks)	[fluxd/src/main.cpp:2842–2848]
`UPGRADE_FLUX` fluxnode-tier multiplier	`fMultiple` 1.0 → 2.0	[fluxd/src/main.cpp:2926–2939]
`nPONInitialSubsidy` (mainnet/testnet/regtest, all equal)	14 FLUX/block	[fluxd/src/chainparams.cpp:156,397,614]
`nPONSubsidyReductionInterval` (mainnet)	1051200 blocks (= 365 · 24 · 60 · 2, "~1 year in 30s blocks")	[fluxd/src/chainparams.cpp:157]
`nPONSubsidyReductionInterval` (testnet)	525600 blocks ("~6 months")	[fluxd/src/chainparams.cpp:398]
`nPONSubsidyReductionInterval` (regtest)	100 blocks	[fluxd/src/chainparams.cpp:615]

Table 89: *continued from previous page*

Constant	Value	Provenance
`nPONMaxReductions` (mainnet/testnet)	20	[fluxd/src/chainparams.cpp:158,399]
`nPONMaxReductions` (regtest)	10	[fluxd/src/chainparams.cpp:616]
Reduction factor (applied once per elapsed reduction interval)	`nSubsidy = nSubsidy*9/10` (10% reduction, C++ integer truncation, per-step compounding)	[fluxd/src/main.cpp:2811–2813]
`nPonTargetSpacing` (all three networks)	30 seconds	[fluxd/src/chainparams.cpp:105,358,565]
`nPowTargetSpacing` (legacy PoW)	120 seconds	[fluxd/src/chainparams.cpp:101]
`nPonDifficultyWindow` (mainnet)	30 blocks ("= 15 minutes")	[fluxd/src/chainparams.cpp:106]
`nPonDifficultyWindow` (testnet, regtest)	60 blocks	[fluxd/src/chainparams.cpp:359,566]
`ponLimit` (mainnet)	`0x0fff...ff` ("~1/16 chance minimum")	[fluxd/src/chainparams.cpp:103]
`ponLimit` (testnet)	`0x0fff...ff`	[fluxd/src/chainparams.cpp:356]
`ponLimit` (regtest)	`0x0f0f...0f`	[fluxd/src/chainparams.cpp:561]
`ponStartLimit` (mainnet)	`0x000bff...ff` ("~1/10000 chance")	[fluxd/src/chainparams.cpp:104]
`ponStartLimit` (testnet, regtest)	`0x7fff...ff` (trivially easy)	[fluxd/src/chainparams.cpp:357,562]
`PON_CUMULUS_BASE` (of `PON_INITIAL_TOTAL` = 14 · COIN)	1.0 · COIN = 1.0 FLUX	[fluxd/src/main.cpp:2892–2895]
`PON_NIMBUS_BASE`	3.5 · COIN = 3.5 FLUX	[fluxd/src/main.cpp:2892–2895]
`PON_STRATUS_BASE`	9.0 · COIN = 9.0 FLUX	[fluxd/src/main.cpp:2892–2895]
Tier-base sum vs. `nPONInitialSubsidy`	1.0 + 3.5 + 9.0 = 13.5 FLUX of 14 FLUX; remainder 0.5 FLUX/block is a consensus-enforced minimum coinbase payment to the dev-fund address (Table 90), not unallocated emission	[fluxd/src/main.cpp:2877–2884]

A.4 Consensus-level addresses and funds

Table 90 covers the dev-fund address and its remainder-based payout mechanism, the one-off exchange and foundation funds, and the swap-pool payment schedule, each with its activation height; only mainnet addresses/heights/amounts were found in evidence for the exchange fund, foundation fund, and swap pool (the dev-fund address is given for all three networks).

Table 90: Consensus-level addresses and funds (`fluxd`).

Constant	Value	Provenance
Dev-fund address (mainnet)	`t3hPu1YDeGUCp8m7BQCnnNUmRMJBa5RadyA`	[fluxd/src/chainparams.cpp:306]
Dev-fund address (testnet)	`t2GoxS2SRmLQDnTyWePHjKD3izvFsKUAjrH`	[fluxd/src/chainparams.cpp:499]
Dev-fund address (regtest)	`t2GoxS2SRmLQDnTyWePHjKD3izvFsKUAjrH`	[fluxd/src/chainparams.cpp:701]
Dev-fund mechanism (`GetMinDevFundAmount`)	`nBlockSubsidy` − (Cumulus + Nimbus + Stratus tier rewards); coinbase must pay ≥ this remainder to the dev-fund address post-PoN or the block is rejected with `REJECT_INVALID "tx-coinbase-missing-dev-funding"`	[fluxd/src/main.cpp:1231–1249,2877–2884]
Exchange fund address	`t3PMbbA5YBMrjSD3dD16SSdXKuKovwmj6tS`	[fluxd/src/chainparams.cpp:292–294]
Exchange fund height	836274	[fluxd/src/chainparams.cpp:292–294]
Exchange fund amount	7500000 · COIN = 7500000 FLUX	[fluxd/src/chainparams.cpp:292–294]
Foundation fund address	`t3XjYMBvwxnXVv9jqg4CgokZ3f7kAoXPQL8`	[fluxd/src/chainparams.cpp:297–298]
Foundation fund height	836994	[fluxd/src/chainparams.cpp:297–298]
Foundation fund amount	2500000 · COIN = 2500000 FLUX	[fluxd/src/chainparams.cpp:297–298]
Swap-pool address	`t3ThbWogDoAjGuS6DEnmN1GWJBRbVjSUK4T`	[fluxd/src/chainparams.cpp:301–304]
Swap-pool start height (`nSwapPoolStartHeight`)	837714	[fluxd/src/chainparams.cpp:301–304]
Swap-pool per-payment amount	22000000 · COIN = 22000000 FLUX per interval	[fluxd/src/chainparams.cpp:301–304]
Swap-pool interval (`nSwapPoolInterval`)	21600 blocks	[fluxd/src/chainparams.cpp:301–304]
Swap-pool max count (`nSwapPoolMaxTimes`)	10	[fluxd/src/chainparams.cpp:301–304]
Swap-pool total across schedule (derived)	≤ 220000000 · COIN = 220000000 FLUX	[fluxd/src/chainparams.cpp:301–304]
Exchange fund (testnet)	`tmRucHD85zgSigtA4sJJBDbPkMUJDcw5XDE`, height 4,100, 7,500,000 FLUX	[fluxd/src/chainparams.cpp:484–487]
Foundation fund (testnet)	same address, height 4,200, 2,500,000 FLUX	[fluxd/src/chainparams.cpp:488–491]
Swap pool (testnet)	same address, start 4,300, 2,200,000 FLUX, interval 100, max 10	[fluxd/src/chainparams.cpp:492–496]

Table 90: *continued from previous page*

Constant	Value	Provenance
Dev fund (testnet)	t2GoxS2SRmLQDnTyWePHjKD3izvFsKUAjrH	[fluxd/src/chainparams.cpp:497]
Exchange fund (regtest)	tmRucHD85zgSigtA4sJJBDbPkMUJDcw5XDE, height 10, 3,000,000 FLUX	[fluxd/src/chainparams.cpp:687–689]
Foundation fund (regtest)	t2DFGpj2tciojsGKKrGVwQ92hUwAxWQQgJ9, height 10, 2,500,000 FLUX	[fluxd/src/chainparams.cpp:691–693]
Swap pool (regtest)	t2Dsexh4v5g2dpL2LLCsR1p9TshMm63jSBM, start 10, 2,100,000 FLUX, interval 10, max 5	[fluxd/src/chainparams.cpp:695–699]

A.5 Node tiering and lifecycle

Table 91 covers V1/V2 collateral per tier, the collateral-transition windows, the deterministic-registration confirmation and expiration constants across their successive versions, and the legacy (pre-PoN) payout percentages.

Table 91: Node tiering and lifecycle constants (`fluxd`).

Constant	Value	Provenance
V1 collateral: Cumulus / Nimbus / Stratus	10000 · COIN / 25000 · COIN / 100000 · COIN (10000 / 25000 / 100000 FLUX)	[fluxd/src/fluxnode/fluxnode.h:57–63]
V2 collateral: Cumulus / Nimbus / Stratus	1000·COIN / 12500·COIN / 40000 · COIN (1000 / 12500 / 40000 FLUX)	[fluxd/src/fluxnode/fluxnode.h:57–63]
Transition window, Cumulus (mainnet)	[1076532, 1086612) – both old and new collateral amounts accepted inside the window	[fluxd/src/chainparams.cpp:308–315]
Transition window, Nimbus (mainnet)	[1081572, 1092372)	[fluxd/src/chainparams.cpp:308–315]
Transition window, Stratus (mainnet)	[1087332, 1097412)	[fluxd/src/chainparams.cpp:308–315]
FLUXNODE_MIN_CONFIRMATION_DETERMINISTIC	100 blocks (collateral maturity before a start-tx is accepted)	[fluxd/src/fluxnode/fluxnode.h:20]
COINBASE_MATURITY	100 blocks	[fluxd/src/consensus/consensus.h:34]
FLUXNODE_START_TX_EXPIRATION_HEIGHT	60 blocks (pre-PoN)	[fluxd/src/fluxnode/fluxnode.h:23–44]
FLUXNODE_START_TX_EXPIRATION_HEIGHT_V2	240 blocks (PoN)	[fluxd/src/fluxnode/fluxnode.h:23–44]
FLUXNODE_DOS_REMOVE_AMOUNT	180 blocks (pre-PoN)	[fluxd/src/fluxnode/fluxnode.h:23–44]
FLUXNODE_DOS_REMOVE_AMOUNT_V2	720 blocks (PoN)	[fluxd/src/fluxnode/fluxnode.h:23–44]

Table 91: *continued from previous page*

Constant	Value	Provenance
FLUXNODE_CONFIRM_UPDATE_EXPIRATION_HEIGHT_V1	60 blocks	[fluxd/src/fluxnode/fluxnode.h:23–44]
FLUXNODE_CONFIRM_UPDATE_EXPIRATION_HEIGHT_V2	80 blocks	[fluxd/src/fluxnode/fluxnode.h:23–44]
FLUXNODE_CONFIRM_UPDATE_EXPIRATION_HEIGHT_V3	160 blocks	[fluxd/src/fluxnode/fluxnode.h:23–44]
FLUXNODE_CONFIRM_UPDATE_EXPIRATION_HEIGHT_V4	640 blocks (PoN)	[fluxd/src/fluxnode/fluxnode.h:23–44]
FLUXNODE_CONFIRM_UPDATE_MIN_HEIGHT_V1	40 blocks	[fluxd/src/fluxnode/fluxnode.h:23–44]
FLUXNODE_CONFIRM_UPDATE_MIN_HEIGHT_V2	120 blocks	[fluxd/src/fluxnode/fluxnode.h:23–44]
FLUXNODE_CONFIRM_UPDATE_MIN_HEIGHT_V3	500 blocks (PoN)	[fluxd/src/fluxnode/fluxnode.h:23–44]
FLUXNODE_CONFIRM_UPDATE_MIN_HEIGHT_IP_CHANGE_V1	5 blocks (out-of-band IP-change re-confirm, post-UPGRADE_FLUX)	[fluxd/src/fluxnode/fluxnode.h:23–44]
FLUXNODE_CONFIRM_TX_IP_ADDRESS_SIZE_V1	40 bytes	[fluxd/src/fluxnode/fluxnode.h:23–44]
FLUXNODE_CONFIRM_TX_IP_ADDRESS_SIZE_V2	60 bytes	[fluxd/src/fluxnode/fluxnode.h:23–44]
FLUXNODE_MAX_SIG_TIME	3600 seconds (mempool-only signature freshness)	[fluxd/src/fluxnode/fluxnode.h:23–44]
nStartFluxnodePaymentsHeight (mainnet)	560000	[fluxd/src/chainparams.cpp:230]
nStartFluxnodePaymentsHeight (testnet)	350	[fluxd/src/chainparams.cpp:463]
nStartFluxnodePaymentsHeight (regtest)	100	[fluxd/src/chainparams.cpp:655]
V1_FLUXNODE_PERCENT_CUMULUS (legacy, pre-PoN)	0.0375 (of block value; sums to 0.25 pre-UPGRADE_FLUX, ×2 after)	[fluxd/src/fluxnode/fluxnode.h:71–74]
V1_FLUXNODE_PERCENT_NIMBUS	0.0625	[fluxd/src/fluxnode/fluxnode.h:71–74]
V1_FLUXNODE_PERCENT_STRATUS	0.15	[fluxd/src/fluxnode/fluxnode.h:71–74]

A.6 Block and consensus limits

Table 92 covers block-format and reorg-depth limits, including MAX_REORG_LENGTH's temporary override during the PoN launch window, and the emergency-block governance thresholds.

Table 92: Block and consensus limits (fluxd).

Constant	Value	Provenance
MIN_BLOCK_VERSION	4	[fluxd/src/consensus/consensus.h:11]

Table 92: *continued from previous page*

Constant	Value	Provenance
`MAX_BLOCK_SIZE`	2000000 bytes	[fluxd/src/consensus/consensus.h:23]
`MAX_BLOCK_SIGOPS`	20000	[fluxd/src/consensus/consensus.h:25]
`MAX_BLOCK_SIG_SIZE`	73 bytes	[fluxd/src/consensus/consensus.h:27]
`MAX_EMERGENCY_BLOCK_SIG_SIZE`	512 bytes	[fluxd/src/consensus/consensus.h:29]
`MAX_TX_SIZE_BEFORE_SAPLING`	100000 bytes	[fluxd/src/consensus/consensus.h:31]
`TX_EXPIRY_HEIGHT_THRESHOLD`	500000000	[fluxd/src/consensus/consensus.h:36]
`MAX_REORG_LENGTH` (normal)	40 blocks	[fluxd/src/main.h:74]
`MAX_REORG_LENGTH` (temporary override)	5000 blocks, for heights [2020000, 2025000] ("Temporary deep reorg limit during PON stabilization period")	[fluxd/src/main.cpp:8983–8988]
Emergency-block signature threshold, `nEmergencyMinSignatures` (mainnet, testnet)	2 (of a 4-key set, i.e. 2-of-4)	[fluxd/src/chainparams.cpp:253,522]
Emergency-block signature threshold (regtest)	1	[fluxd/src/chainparams.cpp:717]
`vecEmergencyPublicKeys` key-set size (all networks)	4	[fluxd/src/chainparams.cpp:242–253]

A.7 Difficulty

Table 93 covers the Digishield, LWMA, and LWMA3 retarget parameters used before PoN activation, and the PoN retarget clamp used after.

Table 93: Difficulty-retarget constants (`fluxd`).

Constant	Value	Provenance
`nDigishieldAveragingWindow`	17	[fluxd/src/chainparams.cpp:96]
`nDigishieldMaxAdjustDown`	32%	[fluxd/src/chainparams.cpp:98]
`nDigishieldMaxAdjustUp`	16%	[fluxd/src/chainparams.cpp:99]
Digishield dampening factor	1/4 (damped-median timespan)	[fluxd/src/pow.cpp:99–131]
`nPowTargetSpacing` (legacy)	120 seconds	[fluxd/src/chainparams.cpp:101]
`nZawyLWMAAveragingWindow`	60	[fluxd/src/chainparams.cpp:160]
`eh_epoch_fade_length`	11	[fluxd/src/chainparams.cpp:161]

Table 93: *continued from previous page*

Constant	Value	Provenance
LWMA3 per-block clamp vs. previous target	+50% / −33%	[fluxd/src/pow.cpp:175–218]
PoN retarget clamp (Digishield-style, over `nPonDifficultyWindow`)	[targetTimespan · 4/5, targetTimespan · 5/4] (i.e. −20% / +25%)	[fluxd/src/pon/pon.cpp:152–157]
PoN fixed chain-work per block (`GetBlockProof`)	2^{40}, independent of actual difficulty target	[fluxd/src/pow.cpp:284–290]

A.8 Benchmarking

Table 94 covers every threshold, interval, timeout, buffer size, and port constant in `fluxbench`, plus the per-tier hardware requirement constants it enforces.

Table 94: Benchmarking constants (`fluxbench`).

Constant	Value	Provenance
`DEFAULT_PORT` (FluxOS port, as seen by `fluxbench`)	16127	[fluxbench/src/fluxnode/benchmarks.h:43]
`ACCEPT_PORT_2..8`	16137, 16147, 16157, 16167, 16177, 16187, 16197	[fluxbench/src/fluxnode/benchmarks.h:44–50]
`DEFAULT_HIGH_INT` (tier-unset sentinel requirement)	999999	[fluxbench/src/fluxnode/benchmarks.h:53]
`SPEEDTEST_NOT_REACHABLE_REQUIREMENTS`	0	[fluxbench/src/fluxnode/benchmarks.h:56]
Cumulus hardware requirement (cores / threads / RAM std / RAM Jetson / storage GB / EPS / write-speed MB/s / upload / download Mbit/s)	2 / 4 / 7 / 3 / 220 / 240 / 180 / 25 / 25	[fluxbench/src/fluxnode/benchmarks.h:58–66]
Nimbus hardware requirement (same fields, no Jetson RAM value)	4 / 8 / 30 / — / 440 / 640 / 180 / 50 / 50	[fluxbench/src/fluxnode/benchmarks.h:68–75]
Stratus hardware requirement (same fields, no Jetson RAM value)	8 / 16 / 61 / — / 880 / 1520 / 400 / 100 / 100	[fluxbench/src/fluxnode/benchmarks.h:77–84]
Internet-speed tolerance bonus: Cumulus	+2 Mbit/s	[fluxbench/src/fluxnode/benchmarks.h:86–88]
Internet-speed tolerance bonus: Nimbus	+4 Mbit/s	[fluxbench/src/fluxnode/benchmarks.h:86–88]
Internet-speed tolerance bonus: Stratus	+9 Mbit/s	[fluxbench/src/fluxnode/benchmarks.h:86–88]
`SYSTEM_BENCH_MIN_*_VERSION` (sysbench minimum)	0.4.1	[fluxbench/src/fluxnode/benchmarks.cpp:40–42]
`SYSTEM_SPEEDTESTCLI_MIN_*_VERSION` (Ookla CLI minimum)	1.2.0	[fluxbench/src/fluxnode/benchmarks.cpp:44–46]
Ookla speedtest pinned build	1.2.0 (`ookla-speedtest-1.2.0-linux-{x86_64,aarch64}.tgz`)	[fluxbench/src/fluxnode/benchmarks.cpp:1676,1690,1704]

Table 94: *continued from previous page*

Constant	Value	Provenance
`fluxbenchd` RPC port (mainnet)	16224	[fluxbench/src/chainparamsbase.cpp:20]
`fluxbenchd` RPC port (testnet, regtest)	26224	[fluxbench/src/chainparamsbase.cpp:33,47]
SAS plain-HTTP port, as addressed by `fluxbench` (`fluxsas` itself serves no plain-HTTP listener; Table 98)	16100	[fluxbench/src/fluxnode/benchmarks.cpp:4106]
SAS mTLS port, as addressed by `fluxbench` (`fluxsas` has no listener on this port; Table 98)	16102	[fluxbench/src/fluxnode/sasclient.h:161,166]
`SYSTEM_SECURE_CACHE_TTL_S`	60 s	[fluxbench/src/fluxnode/benchmarks.cpp:4084]
`SYSTEM_SECURE_TLS_CACHE_TTL_S`	60 s	[fluxbench/src/rpc/fluxnode.cpp:773]
`getpublicip` RPC cache	69 s	[fluxbench/src/rpc/fluxnode.cpp:442]
Benchmark loop poll sleep	1000 ms	[fluxbench/src/fluxnode/benchmarks.cpp:529]
Tier-not-set wait	180000 ms (3 min)	[fluxbench/src/fluxnode/benchmarks.cpp:704]
Multiport sync offset	+180 s	[fluxbench/src/fluxnode/benchmarks.cpp:678]
Retry backoff, attempts 1–3	5 min (60×5)	[fluxbench/src/fluxnode/benchmarks.cpp:591]
Retry backoff, attempts ≥ 4	30 min (60×30)	[fluxbench/src/fluxnode/benchmarks.cpp:594]
Re-benchmark cadence after success	30 min (60×30)	[fluxbench/src/fluxnode/benchmarks.cpp:597]
EPS-clear maintenance cadence	every 192 `benchmarkRuns` (comment: "~4 days")	[fluxbench/src/fluxnode/benchmarks.cpp:636]
Disk/speedtest-reinstall maintenance cadence	every 480 `benchmarkRuns` (comment: "~10 days")	[fluxbench/src/fluxnode/benchmarks.cpp:639]
sysbench single-thread test duration	`-time=20` (`strEpsCommand`, 57 chars)	[fluxbench/src/fluxnode/benchmarks.h:361]
sysbench multi-thread duration (normal / multiport)	`-time=20` / `-time=60`	[fluxbench/src/fluxnode/benchmarks.h:363–364]
sysbench `--cpu-max-prime`	60000	[fluxbench/src/fluxnode/benchmarks.h:361–364]
EPS retry count / wait	`MAX_RETRIES=3`, `RETRY_WAIT_MS=15000`	[fluxbench/src/fluxnode/benchmarks.cpp:2069–2070]
Single-thread fallback bonus	`SINGLE_THREAD_BONUS=1.10` (10%)	[fluxbench/src/fluxnode/benchmarks.cpp:2071]
`dd` block size / count	`bs=64k count=16k`	[fluxbench/src/fluxnode/benchmarks.h:602,1027]
`dd` timeout	25 s	[fluxbench/src/fluxnode/benchmarks.h:602,1027]

Table 94: *continued from previous page*

Constant	Value	Provenance
`dd` runs / averaging	3 runs, drop worst, average remaining 2	[fluxbench/src/fluxnode/benchmarks.h:662–668,1090–1100]
Disk-speed floor for reporting	40 MB/s	[fluxbench/src/fluxnode/benchmarks.cpp:1936]
Raspberry-Pi Cumulus disk-speed bonus	+40 MB/s (if measured > 120)	[fluxbench/src/fluxnode/benchmarks.cpp:1937–1940]
Available-space threshold before write test	2 GiB (2147483648 bytes)	[fluxbench/src/fluxnode/benchmarks.h:1073]
`dstat` disk-in-use sampling	`-disk 1 30` (1s interval, 30s)	[fluxbench/src/fluxnode/benchmarks.h:354]
Disk-in-use scale factor	×1.1	[fluxbench/src/fluxnode/benchmarks.cpp:2024]
`iftop` bandwidth-in-use sampling	timeout 35 s, `-s 30`	[fluxbench/src/fluxnode/benchmarks.h:355–356]
speedtest CLI timeout	120 s	[fluxbench/src/fluxnode/benchmarks.cpp:1427]
CPU-usage threshold to skip real disk/memory test	75% (`cpuUsagePercent > 75.0` / `<= 75.0`)	[fluxbench/src/fluxnode/benchmarks.cpp:1891,1903]
Memory-test wait after detach	60000 ms	[fluxbench/src/fluxnode/benchmarks.cpp:1894]
stress-ng memory test size (Stratus / Nimbus / Cumulus)	8G / 4G / 2G (or available ×0.90 if less)	[fluxbench/src/fluxnode/benchmarks.cpp:3649–3667]
stress-ng memory test timeout	120 s	[fluxbench/src/fluxnode/benchmarks.cpp:3644]
SpeedTest setup poll	initial 30000 ms, then every 10000 ms	[fluxbench/src/fluxnode/benchmarks.cpp:858,861]
`RunCommand` shutdown SIGTERM→SIGKILL grace	100000 µs (100 ms)	[fluxbench/src/fluxnode/benchmarks.cpp:1230]
`RunCommand` `select()` timeout	100000 µs (100 ms)	[fluxbench/src/fluxnode/benchmarks.cpp:1245]
SAS response buffer cap	536870912 bytes (512 MiB)	[fluxbench/src/fluxnode/sasclient.h:49]
SAS curl timeout	15 s	[fluxbench/src/fluxnode/sasclient.h:133]
Curl timeouts (FluxOS/SAS plain queries, most call sites)	`-m 15` (15 s)	[fluxbench/src/fluxnode/benchmarks.cpp:2434,2525,2616,2709]
Private-IP prefixes checked (count)	18 (`10.`, `172.16.`–`172.31.`, `192.168.`)	[fluxbench/src/fluxnode/benchmarks.cpp:187–189]
`getbenchmarks`/RPC-reported version	`"6.3.1"`	[fluxbench/src/rpc/fluxnode.cpp:423]
`CLIENT_VERSION_MAJOR.MINOR.REVISION.BUILD`	6.3.1.50	[fluxbench/src/clientversion.h:18–21]
Benchmarking key rotation activation times (mainnet, 5 keys)	0, 1618113600, 1647262800, 1706209200, 1743534000	[fluxd/src/chainparams.cpp:233–238]
App-signing key count (mainnet, no rotation)	1	[fluxbench/src/chainparams.cpp:73–81]

Table 94: *continued from previous page*

Constant	Value	Provenance
Testnet/regtest benchmarking key rotation	2 keys each, second activates 1617508800	[fluxbench/src/chainparams.cpp:119–127,157–165]
OS end-of-life list	Ubuntu 20.04, Debian 11	[fluxbench/src/fluxnode/benchmarks.cpp:4168–4180]
System uptime threshold for FluxOS speed-fallback attempt	3600 s (1 hour)	[fluxbench/src/fluxnode/benchmarks.cpp:829]

A.9 FluxOS

Table 95 covers FluxOS's ports, MongoDB collections, timeouts, intervals, spawn deferrals, the crash-backoff ladder, the install-collision wait, per-tier resource allowances, locked system resources, CPU burst, and version pins, grouped by subject as in the source configuration file.

Table 95: FluxOS constants, by category.

Category	Constant	Value	Provenance
Ports	`server.apiport` (default)	16127	[flux/ZelBack/config/default.js:28]
Ports	`server.allowedPorts`	[16127,16137,16147,16157,16167,16177,16187,16197]	[flux/ZelBack/config/default.js:26]
Ports	`apiPortHttps`	`apiPort + 1`	[flux/apiServer.js:38]
Ports	`homePort`	`apiPort - 1`	[flux/homeServer.js:19]
Ports	`syncthing.port` (ip `127.0.0.1`)	8384	[flux/ZelBack/config/default.js:407]
Ports	`database.port` (MongoDB)	27017	[flux/ZelBack/config/default.js:32]
Ports	`benchmark.port` / `.rpcport` (mainnet)	16225 / 16224	[flux/ZelBack/config/default.js:102–106]
Ports	`benchmark.port` / `.rpcport` (testnet)	26225 / 26224	[flux/ZelBack/config/default.js:102–106]
Ports	`daemon.port` / `.rpcport` / `.zmqport` (mainnet)	16125 / 16124 / 16123	[flux/ZelBack/config/default.js:109–115]
Ports	`daemon.port` / `.rpcport` (testnet)	26125 / 26124	[flux/ZelBack/config/default.js:109–115]
Ports	`peers.wsPingIntervalMs`	15000	[flux/ZelBack/config/default.js:18]
Ports	`peers.wsMaxMissedPongs`	3	[flux/ZelBack/config/default.js:19]
Ports	`fluxapps.portMinLegacy`/ `portMaxLegacy`	31000 / 39999 (30000–30999 reserved for local apps)	[flux/ZelBack/config/default.js:248–249]
Ports	`fluxapps.portMin`/ `portMax` (current)	1 / 65535	[flux/ZelBack/config/default.js:251–252]
Ports	`fluxapps.portBlockheightChange`	`isDevelopment ? 1390000 : 1420000`	[flux/ZelBack/config/default.js:250]

Table 95: *continued from previous page*

Category	Constant	Value	Provenance
Ports	`fluxapps.bannedPorts`	`['16100-16299', '26100-26299', '30000-30099',8384, 27017,22,23,25, 3389,5900,5800,161, 512,513,5901,3388, 4444,123,53]`	[flux/ZelBack/config/default.js:253]
Ports	`fluxapps.enterprisePorts`	`['0-1023',8080,8081,8443,6667]`	[flux/ZelBack/config/default.js:254]
Ports	`server.fluxNodeServiceAddress`	`169.254.43.43`	[flux/ZelBack/config/default.js:29]
DB collections	`zelfluxlocal` (local)	`loggedusers`, `activeloginphrases`, `activesignatures`, `activepaymentrequests`, `completedpayments`, `geolocation`, `benchmark`, `apptamperingevents`, `nodestartuptracker`	[flux/ZelBack/config/default.js:33–45]
DB collections	`zelcashdata` (daemon mirror)	`scannedheight`, `utxoindex`, `addresstransactionindex`, `zelnodetransactions`, `zelappshashes`, `coinbasefusionindex`	[flux/ZelBack/config/default.js:47–55]
DB collections	`localzelapps` (per-node local truth)	`zelappsinformation`, `zelappsruntimestate`	[flux/ZelBack/config/default.js:57–64]
DB collections	`globalzelapps` (network-global, P2P-replicated)	`zelappsmessages`, `zelappsinformation`, `zelappstemporarymessages`, `zelappslocation`, `appsinstallinglocations`, `appsInstallingErrorsLocations`, `appstateevents`, `fluxappinstallingbroadcasts`, `fluxappinstallingerrorsbroadcasts`	[flux/ZelBack/config/default.js:65–75]
DB collections	`chainparams`	`chainmessages`	[flux/ZelBack/config/default.js:78–87]
Chain/height forks	`daemon.chainValidHeight`	1062000	[flux/ZelBack/config/default.js:110]
Chain/height forks	`deterministicNodesStart`	558000	[flux/ZelBack/config/default.js:123]
Chain/height forks	`messagesBroadcastRefactorStart`	1751250	[flux/ZelBack/config/default.js:124]
Chain/height forks	`fluxapps.daemonPONFork`	2020000 (chain 4× faster after)	[flux/ZelBack/config/default.js:293]

Table 95: *continued from previous page*

Category	Constant	Value	Provenance
Chain/height forks	`fluxapps.multisigAddressChange`	1670000	[flux/ZelBack/config/default.js:245]
Chain/height forks	`fluxapps.epochstart` / `publicepochstart`	694000 / 705000	[flux/ZelBack/config/default.js:246–247]
Chain/height forks	`fluxapps.newMinBlocksAllowanceBlock`	1630040	[flux/ZelBack/config/default.js:288]
Chain/height forks	`fluxapps.cancel1BlockMinBlocksAllowanceBlock`	1964447	[flux/ZelBack/config/default.js:290]
Chain/height forks	`fluxapps.removeBlocksAllowanceIntervalBlock`	1625000	[flux/ZelBack/config/default.js:295]
Chain/height forks	`fluxapps.applyMinimumPriceOn3Instances`	1691000	[flux/ZelBack/config/default.js:305]
Chain/height forks	`fluxapps.applyMinimumForExtraInstances`	1890000	[flux/ZelBack/config/default.js:306]
Chain/height forks	`fluxapps.minimumInstancesV8Block`	2176519	[flux/ZelBack/config/default.js:259]
Chain/height forks	`appSpecsEnforcementHeights` v1–v8	0, 0, 983000, 1004000, 1142000, 1300000, 1390000/1420000, 1921500/1932380	[flux/ZelBack/config/default.js:232–239]
Chain/height forks	v9 price-segment height	2950000	[RunOnFlux/flux@feat/appspec-v9:ZelBack/config/default.js:201]
App lifecycle	`fluxapps.minimumInstances`	3	[flux/ZelBack/config/default.js:257]
App lifecycle	`fluxapps.minimumInstancesV8`	1	[flux/ZelBack/config/default.js:258]
App lifecycle	`fluxapps.maximumInstances`	100	[flux/ZelBack/config/default.js:260]
App lifecycle	`fluxapps.maxAppsPerNode`	200	[flux/ZelBack/config/default.js:261]
App lifecycle	`fluxapps.minOutgoing` / `minUniqueIpsOutgoing`	8 / 7	[flux/ZelBack/config/default.js:262–263]
App lifecycle	`fluxapps.minIncoming` / `minUniqueIpsIncoming`	4 / 3	[flux/ZelBack/config/default.js:264–265]
App lifecycle	`fluxapps.minHashSyncPeers`	12	[flux/ZelBack/config/default.js:266]
App lifecycle	`fluxapps.minUpTime`	1800 s (30 min)	[flux/ZelBack/config/default.js:267]
App lifecycle	`fluxapps.appSyncPeerThreshold` / `Degraded`	12 / 4	[flux/ZelBack/config/default.js:268–269]
App lifecycle	`fluxapps.appSyncMinPeerUptime`	7500 s	[flux/ZelBack/config/default.js:270]
App lifecycle	`fluxapps.appSyncMinCompletions`	3	[flux/ZelBack/config/default.js:271]
App lifecycle	`fluxapps.installation.probability/.delay`	100 (1%) / 120 s	[flux/ZelBack/config/default.js:273–274]

Table 95: *continued from previous page*

Category	Constant	Value	Provenance
App lifecycle	`fluxapps.removal.probability/.delay`	25 (4%) / 300 s	[flux/ZelBack/config/default.js:277–278]
App lifecycle	`fluxapps.redeploy.probability/.delay/.composedDelay`	2 (50%) / 30 s / 5 s	[flux/ZelBack/config/default.js:281–283]
App lifecycle	`fluxapps.blocksLasting`	22000	[flux/ZelBack/config/default.js:285]
App lifecycle	`fluxapps.minBlocksAllowance`	5000	[flux/ZelBack/config/default.js:286]
App lifecycle	`fluxapps.newMinBlocksAllowance`	100 (active ≥ height 1630040)	[flux/ZelBack/config/default.js:287]
App lifecycle	`fluxapps.cancel1BlockMinBlocksAllowance`	1 (active ≥ height 1964447 – **the live `deploymentinformation` endpoint nonetheless reports `minBlocksAllowance`: 5000 at height 2917115, past both height gates**: the validator applies the three constants by height — 5000 below 1630040, 100 below 1964447, 1 thereafter — while the endpoint returns the static `fluxapps.minBlocksAllowance` regardless of height, so the reported 5000 is a stale minimum and the enforced floor is 1)	[flux/ZelBack/config/default.js:289]; [flux/ZelBack/src/services/appRequirements/appValidator.js:852–863]; [flux/ZelBack/src/services/appQuery/deploymentInfoService.js:44]; [measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]
App lifecycle	`fluxapps.maxBlocksAllowance` (pre-PON)	264000	[flux/ZelBack/config/default.js:291]
App lifecycle	`fluxapps.postPonMaxBlocksAllowance`	1056000	[flux/ZelBack/config/default.js:292]
App lifecycle	`fluxapps.blocksAllowanceInterval`	1000	[flux/ZelBack/config/default.js:294]
App lifecycle	`fluxapps.ownerAppAllowance`	1000 blocks	[flux/ZelBack/config/default.js:296]
App lifecycle	`fluxapps.temporaryAppAllowance`	200 blocks	[flux/ZelBack/config/default.js:297]
App lifecycle	`fluxapps.expireFluxAppsPeriod`	100 blocks	[flux/ZelBack/config/default.js:298]
App lifecycle	`fluxapps.updateFluxAppsPeriod`	9 blocks	[flux/ZelBack/config/default.js:299]

Table 95: *continued from previous page*

Category	Constant	Value	Provenance
App lifecycle	`fluxapps.removeFluxAppsPeriod`	11 blocks	[flux/ZelBack/config/default.js:300]
App lifecycle	`fluxapps.reconstructAppMessagesHashPeriod`	3600 blocks (comment: "every 5 days", i.e. at the 120 s pre-PoN spacing)	[flux/ZelBack/config/default.js:301]
App lifecycle	`fluxapps.benchUpnpPeriod`	6480 blocks (comment: "every 9 days", i.e. at the 120 s pre-PoN spacing)	[flux/ZelBack/config/default.js:302]
App lifecycle	`fluxapps.hddFileSystemMinimum`	10 GB	[flux/ZelBack/config/default.js:303]
App lifecycle	`fluxapps.defaultSwap`	2 GB	[flux/ZelBack/config/default.js:304]
App lifecycle	`fluxapps.maxImageSize`	5000000000 (5000 MB)	[flux/ZelBack/config/default.js:256]
App lifecycle	`fluxapps.installCollisionWaitMs`	90000 ms	[flux/ZelBack/config/default.js:321]
App lifecycle	`fluxapps.crashBackoffDelaysMs`	[0, 30000, 300000, 900000, 1800000] (0s, 30s, 5min, 15min, 30min ladder)	[flux/ZelBack/config/default.js:130]
App lifecycle	`fluxapps.crashBackoffStableRunMs`	600000 ms (10 min stable run resets the ladder)	[flux/ZelBack/config/default.js:131]
App lifecycle	`fluxapps.nonEnterpriseSpawnDelayMs` (fallback)	$2 \times 60 \times 1000$ ms	[flux/ZelBack/src/services/appLifecycle/appSpawner.js:34]
Spawn deferrals	`fluxapps.spawnDeferrals.targetedNodesMs`	enterprise 1800000 / standard 3420000	[flux/ZelBack/config/default.js:341]
Spawn deferrals	`fluxapps.spawnDeferrals.staticIpMs`	enterprise 1620000 / standard 3420000	[flux/ZelBack/config/default.js:342]
Spawn deferrals	`fluxapps.spawnDeferrals.datacenterMs`	enterprise 1620000 / standard 3420000	[flux/ZelBack/config/default.js:343]
Spawn deferrals	`fluxapps.spawnDeferrals.capacityGap.large/medium/smallMs`	enterprise 1800000/1260000/720000; standard 7020000/5220000/3420000	[flux/ZelBack/config/default.js:345–347]
Timeouts/intervals	`fluxapps.locationTtlS`	7500 s	[flux/ZelBack/config/default.js:311]
Timeouts/intervals	`fluxapps.installingTtlS`	900 s	[flux/ZelBack/config/default.js:312]
Timeouts/intervals	`fluxapps.installErrorTtlS`	3600 s	[flux/ZelBack/config/default.js:313]
Timeouts/intervals	`fluxapps.tempMsgTtlS`	3600 s	[flux/ZelBack/config/default.js:314]
Timeouts/intervals	`fluxapps.hashSyncIntervalMs`	1800000 ms	[flux/ZelBack/config/default.js:315]

Table 95: *continued from previous page*

Category	Constant	Value	Provenance
Timeouts/intervals	fluxapps.peerNotifyIntervalMs	3600000 ms	[flux/ZelBack/config/default.js:316]
Timeouts/intervals	fluxapps.cpuCheckIntervalMs	900000 ms	[flux/ZelBack/config/default.js:317]
Timeouts/intervals	fluxapps.portRestoreIntervalMs	600000 ms	[flux/ZelBack/config/default.js:318]
Timeouts/intervals	fluxapps.imageComplianceIntervalMs	3600000 ms	[flux/ZelBack/config/default.js:319]
Timeouts/intervals	fluxapps.forceRemovalIntervalMs	7200000 ms	[flux/ZelBack/config/default.js:320]
Timeouts/intervals	fluxapps.portTestBindDelayMs/PropagationDelayMs/PeerTimeoutMs/MaxAttempts	5000 / 10000 / 30000 / 5	[flux/ZelBack/config/default.js:322–325]
Timeouts/intervals	fluxapps.nodeMonitorRemovalDelayMs	60000 ms	[flux/ZelBack/config/default.js:335]
Timeouts/intervals	fluxapps.nodeMonitorDosRecoveryDelayMs	600000 ms	[flux/ZelBack/config/default.js:336]
Timeouts/intervals	fluxapps.nodeMonitorConfirmationLossDelayMs	1200000 ms	[flux/ZelBack/config/default.js:337]
Timeouts/intervals	fluxapps.nodeMonitorErrorRecoveryDelayMs	120000 ms	[flux/ZelBack/config/default.js:338]
Timeouts/intervals	fluxapps.nodeMonitorCheckTimeoutMs	10000 ms	[flux/ZelBack/config/default.js:339]
Timeouts/intervals	fluxapps.hashSyncMaxRetries/RetryMs/SettleMs/ResponseTimePerHashMs/BufferMs/MaxRounds/PeersPerRound/EphemeralPeers/FallbackRecheckBlocks	3 / 300000 / 4000 / 150 / 5000 / 4 / 3 / 5 / 100	[flux/ZelBack/config/default.js:355–363]
Timeouts/intervals	fluxapps.syncResponseThrottleMs	300000 ms	[flux/ZelBack/config/default.js:364]
Timeouts/intervals	fluxapps.wsHandshakeTimeoutMs	10000 ms	[flux/ZelBack/config/default.js:365]
Timeouts/intervals	fluxapps.imageUpdateCheckIntervalMs	21600000 ms (6 h)	[flux/ZelBack/config/default.js:366]
Timeouts/intervals	fluxapps.imageUpdateInitialDelayMin/MaxMs	600000 / 1800000	[flux/ZelBack/config/default.js:367–368]
Timeouts/intervals	fluxapps.imageUpdateDelayBetweenAppsMs	5000 ms	[flux/ZelBack/config/default.js:369]
Timeouts/intervals	fluxapps.imageUpdateDelayAfterRedeployMs	120000 ms	[flux/ZelBack/config/default.js:370]
Timeouts/intervals	fluxapps.imageUpdateDelayBetweenComponentsMs	1000 ms	[flux/ZelBack/config/default.js:371]

Table 95: *continued from previous page*

Category	Constant	Value	Provenance
Timeouts/intervals	`fluxapps.masterSlaveIntervalMs`	30000 ms	[flux/ZelBack/config/default.js:372]
Timeouts/intervals	`SIGTERM_EXPIRY_MS`	420000 ms	[flux/ZelBack/src/services/utils/appConstants.js:86]
Timeouts/intervals	`system.heartbeatIntervalMs`	30000 ms	[flux/ZelBack/config/default.js:14]
Timeouts/intervals	`system.bootSyncTimeoutMs/bootDaemonTimeoutMs`	300000 / 300000	[flux/ZelBack/config/default.js:15–16]
Timeouts/intervals	`confirmation.pollIntervalMs`	30000 ms	[flux/ZelBack/config/default.js:21]
Timeouts/intervals	`confirmation.daemonStaleMs`	7500000 ms	[flux/ZelBack/config/default.js:22]
Timeouts/intervals	`confirmation.daemonExpiredMs`	19200000 ms	[flux/ZelBack/config/default.js:23]
Timeouts/intervals	`syncthing.monitorIntervalMs`	30000 ms	[flux/ZelBack/config/default.js:408]
Timeouts/intervals	`syncthing.stallNudgeAfterMs/stallNudgeMaxIntervalMs/stallRemoveMinWindowMs/stallRemoveMinNudges`	180000 / 900000 / 1200000 / 3	[flux/ZelBack/config/default.js:412–415]
Timeouts/intervals	`FluxPeerManager.CONNECTION_BACKOFF_MS` (default)	[120000, 300000, 600000, 900000]	[flux/ZelBack/src/services/utils/FluxPeerManager.js:32]
Tier resources	Cumulus: CPU total (apps avail.) / RAM MB total (apps) / HDD GB total (apps) / collateral current-old	40 (30) / 7000 (5000) / 220 (180) / 1000–10000	[flux/ZelBack/config/default.js:380–403]
Tier resources	Nimbus: same fields	80 (70) / 30000 (28000) / 440 (400) / 12500–25000	[flux/ZelBack/config/default.js:380–403]
Tier resources	Stratus: same fields	160 (150) / 61000 (59000) / 880 (840) / 40000–100000	[flux/ZelBack/config/default.js:380–403]
Locked resources	`lockedSystemResources.cpu`	10 (1 core, ×10 units)	[flux/ZelBack/config/default.js:375–378]
Locked resources	`lockedSystemResources.ram`	2000 MB	[flux/ZelBack/config/default.js:375–378]
Locked resources	`lockedSystemResources.hdd`	60 GB	[flux/ZelBack/config/default.js:375–378]
Locked resources	`lockedSystemResources.extrahdd`	20 GB	[flux/ZelBack/config/default.js:375–378]
Locked resources	Usable disk multiplier	0.95	[flux/ZelBack/src/services/appRequirements/hwRequirements.js:385]

Table 95: *continued from previous page*

Category	Constant	Value	Provenance
Locked resources	CPU-burst headroom floor	≤ 4 free vCores rejected	[flux/ZelBack/src/services/appRequirements/hwRequirements.js:432]
CPU burst	`cpuBurst.enabled`	`true`	[flux/ZelBack/config/default.js:440–453]
CPU burst	`cpuBurst.periodUs`	100000 (CFS 100 ms period)	[flux/ZelBack/config/default.js:440–453]
CPU burst	`cpuBurst.reservedCores`	1	[flux/ZelBack/config/default.js:440–453]
Payment addresses	`fluxapps.address`	`t1LUs6quf7TB2zVZmexqPQdnqmrFMGZGjV6`	[flux/ZelBack/config/default.js:241–244]
Payment addresses	`addressMultisig`	`t3aGJvdtd8NR6GrnqnRuVEzH6MbrXuJFLUX`	[flux/ZelBack/config/default.js:241–244]
Payment addresses	`addressMultisigB`	`t3NryfAQLGeFs9jEoeqsxmBN2QLRaRKFLUX`	[flux/ZelBack/config/default.js:241–244]
Payment addresses	`addressDevelopment`	`t1Mzja9iJcEYeW5B4m4s1tJG8M42odFZ16A`	[flux/ZelBack/config/default.js:241–244]
Payment addresses	`fluxTeamFluxID`	`1hjy4bCYBJr4mny4zCE85J94RXa8W6q37`	[flux/ZelBack/config/default.js:121–122]
Payment addresses	`fluxSupportTeamFluxID`	`16iJqiVbHptCx87q6XQwNpKdgEZnFtKcyP`	[flux/ZelBack/config/default.js:121–122]
Payment addresses	`teamSupportAddress` (height-gated)	`{height:1851659, address:'16iJqiVbHptCx87q6XQwNpKdgEZnFtKcyP'}`	[flux/ZelBack/config/default.js:225–228]
Payment addresses	`usersToExtend`	`['1MCBJn6qsy3YRY2YasdYMYdJcdhy1ev8Rd']` (append-only allow-list)	[flux/ZelBack/config/default.js:229]
Version pins	`minimumFluxBenchAllowedVersion`	`'6.2.0'`	[flux/ZelBack/config/default.js:117–120]
Version pins	`minimumFluxOSAllowedVersion`	`'8.13.1'`	[flux/ZelBack/config/default.js:117–120]
Version pins	`minimumSyncthingAllowedVersion`	`'2.0.10'`	[flux/ZelBack/config/default.js:117–120]
Version pins	`minimumDockerAllowedVersion`	`'26.1.2'`	[flux/ZelBack/config/default.js:117–120]
Version pins	`fluxapps.latestSupportedSpecVersion`	8	[flux/ZelBack/config/default.js:126]
Version pins	`fluxapps.latestAppSpecification`	8	[flux/ZelBack/config/default.js:307]

Version pins	`fluxapps.acceptV9`	`false`	[RunOnFlux/flux@feat/appspec-v9:ZelBack/config/default.js:127]

A.10 Application pricing and deployment

Table 96 gives the height-scheduled price table (genesis plus five revisions), the payment sink address, instance and blocks-allowance bounds, the maximum image size, the port range, and the SPEC v1–SPEC v8 enforcement heights, all read from the `deploymentinformation` endpoint's live response.

Table 96: Application pricing and deployment constants.

Constant	Value	Provenance
Price table row, height −1 (genesis) (cpu/ram/hdd/minPrice/port/scope/staticip)	3 / 1 / 0.5 / 1 / 2 / 6 / 3	[measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]
Price table row, height 983000	0.3 / 0.1 / 0.05 / 0.1 / 2 / 6 / 3	[measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]
Price table row, height 1004000	0.06 / 0.02 / 0.01 / 0.01 / 2 / 6 / 3	[measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]
Price table row, height 1288000	0.15 / 0.05 / 0.02 / 0.01 / 2 / 6 / 3	[measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]
Price table row, height 1594832 (soft fork 1)	0.15 / 0.05 / 0.02 / 0.01 / 1.5 / 6 / 3	[measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]
Price table row, height 1597156 (soft fork 2)	0.03 / 0.01 / 0.004 / 0.01 / 0.4 / 0.8 / 0.4	[measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]
Cross-check: same six rows, static config	identical values at the same six heights	[flux/ZelBack/config/default.js:133–209]
v9 price-segment row, height 2950000 (not one of the six live-scheduled revisions; gated by `acceptV9` feature flag, not chain height)	identical to the 1597156 row (“income-never-falls rule”)	[flux/ZelBack/config/default.js:196–209]
Payment sink address	`t3NryfAQLGeFs9jEoeqsxmBN2QLRaRKFLUX`	[measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]
`portMin` / `portMax`	1 / 65535	[measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]
`enterprisePorts`	[0–1023, 8080, 8081, 8443, 6667]	[measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]

Table 96: *continued from previous page*

Constant	Value	Provenance
`bannedPorts`	[16100–16299, 26100–26299, 30000–30099, 8384, 27017, 22, 23, 25, 3389, 5900, 5800, 161, 512, 513, 5901, 3388, 4444, 123, 53]	[measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]
`maxImageSize`	5000000000	[measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]
`minimumInstances`	3 (the endpoint's static report; the validator enforces 1 for SPEC v8 and later from height 2176519, Table 95)	[measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]
`maximumInstances`	100	[measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]
`blocksLasting`	22000	[measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]
`minBlocksAllowance`	5000 (the endpoint's static report; the validator's floor is 1 from height 1964447, Table 95)	[measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]
`maxBlocksAllowance`	1056000 (matches the code's `fluxapps.postPonMaxBlocksAllowance`, Table 95, not the pre-PoN `fluxapps.maxBlocksAllowance`=264000)	[measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]
`blocksAllowanceInterval`	1000	[measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]
`appSpecsEnforcementHeights`, SPEC v1	0	[measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]
`appSpecsEnforcementHeights`, SPEC v2	0	[measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]
`appSpecsEnforcementHeights`, SPEC v3	983000	[measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]
`appSpecsEnforcementHeights`, SPEC v4	1004000	[measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]
`appSpecsEnforcementHeights`, SPEC v5	1142000	[measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]
`appSpecsEnforcementHeights`, SPEC v6	1300000	[measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]

Table 96: *continued from previous page*

Constant	Value	Provenance
`appSpecsEnforcementHeights`, SPEC v7	1420000	[measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]
`appSpecsEnforcementHeights`, SPEC v8	1932380	[measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]

A.11 Networking

Table 97 covers FDM and PDNS intervals, TTLs, timeouts, retry counts, and ports, plus the network-policy fetch interval enforced fleet-wide by FluxOS.

Table 97: Networking constants (FDM, PDNS, and related network/addressing services).

Constant	Value	Provenance
FDM HTTP port (16131 if `manageCertificateOnly`)	16130	[flux-domain-manager/config/default.js:7]
`globalAppSpecs` poll interval (default)	30000 ms	[flux-domain-manager/src/services/flux/dataFetcher.js:54]
`appsLocations` poll interval (hard-coded, always)	10000 ms	[flux-domain-manager/src/services/flux/dataFetcher.js:66,707]
`permMessages` poll interval (default)	120000 ms	[flux-domain-manager/src/services/flux/dataFetcher.js:82]
Main HAProxy (home/api/cloud) regen interval	30000 ms	[flux-domain-manager/src/services/domainService.js:201–208]
Main pool minimum node count (throws below)	10	[flux-domain-manager/src/services/domainService.js:181–183]
Main pool maximum size	100	[flux-domain-manager/src/services/domainService.js:114,173–176]
G-app health retry count / delay	3 attempts / 3000 ms	[flux-domain-manager/src/services/domainService.js:29–30]
G-app sticky-unhealthy threshold	90000 ms (90 s)	[flux-domain-manager/src/services/domainService.js:31]
HAProxy generic-app health check	`inter 3s fall 2 rise 2 fastinter 500`	[flux-domain-manager/src/services/haproxyTemplate.js:311]
HAProxy main UI/API health check	`inter 10s fall 2 rise 3 maxconn 100`	[flux-domain-manager/src/services/haproxyTemplate.js:384,408,427]
Certificate renewal threshold	< 30 days remaining	[flux-domain-manager/src/services/domain/cert.js:108,241]

Table 97: *continued from previous page*

Constant	Value	Provenance
Stale-cert deletion threshold	< -30 days (expired > 30 days ago)	[flux-domain-manager/src/services/domain/cert.js:337–339]
CDM certificate obtain/renew loop interval	15 min ($15 \times 60 \times 1000$ ms)	[flux-domain-manager/src/services/domainService.js:1079–1086]
Cert operation concurrency limit	10	[flux-domain-manager/src/services/domain/cert.js:16,270]
ACME HTTP-01 challenge port	8787	[flux-domain-manager/src/services/domain/cert.js:40]
DNS backoff cooldowns (failures 1/2/3/4+)	0 / 30 min / 1 h / capped 2 h	[flux-domain-manager/src/services/domain/dnsCache.js:7–21]
flux-apps-dns-manager service port	16140	[flux-apps-dns-manager/config/default.js:6]
flux-apps-dns-manager polling interval	60000 ms	[flux-apps-dns-manager/config/default.js:81]
flux-apps-dns-manager deletion grace period	24 h	[flux-apps-dns-manager/config/default.js:84]
DNS record TTL (`app.`/`app2.runonflux.io`)	300 s	[flux-apps-dns-manager/config/default.js:25,40]
Placeholder record TTL	60 s	[flux-apps-dns-manager/config/default.js:27,42]
flux-dnsd bind address	`169.254.43.53:53` (UDP+TCP)	[flux-dnsd/src/main.rs:37–39]
flux-dnsd mesh answer TTL	5 s	[flux-dnsd/src/main.rs:57–60]
flux-dnsd TCP idle timeout	5 s	[flux-dnsd/src/main.rs:102]
flux-dnsd max in-flight TCP responses/conn	32	[flux-dnsd/src/main.rs:102]
flux-pdns PowerDNS version	5.0.6	[flux-pdns/vars.yaml:19–21]
flux-pdns cache-ttl / negquery-cache-ttl	60 s each	[flux-pdns/templates/pdns.conf.j2:27–28]
flux-pdns query-cache-ttl	20 s	[flux-pdns/templates/pdns.conf.j2:29]
flux-pdns webserver (API) port	8081	[flux-pdns/templates/pdns.conf.j2:43]
flux-pdns DNS port	53	[flux-pdns/templates/pdns.conf.j2:61]
flux-pdns xfr-cycle-interval	10 s	[flux-pdns/templates/pdns.conf.j2:104]
flux-pdns app-zone TTL	3600 s	[flux-pdns/vars.yaml:69,111]
flux-pdns geo-zone TTL	300 s	[flux-pdns/vars.yaml:49,90]
flux-pdns geo health check timeout / `minimumFailures` / `interval`	3 s / 3 / 15 s	[flux-pdns/templates/geo_routing.lua.j2:55–57]
flux-dns-gateway server port (behind Nginx 8443)	8080	[flux-dns-gateway/src/dns_gateway/config/settings.py:18–19]

Table 97: *continued from previous page*

Constant	Value	Provenance
flux-dns-gateway request timeout	30 s	[flux-dns-gateway/src/dns_gateway/config/settings.py:37]
flux-dns-gateway max retries / retry delay	3 / 2.0 s (linear backoff)	[flux-dns-gateway/src/dns_gateway/config/settings.py:40–41]
flux-dns-gateway DNS-state cache TTL	3600.0 s (1 h)	[flux-dns-gateway/src/dns_gateway/managers/dns_manager.py:36]
flux-dns-gateway record TTL bounds	60–86400 s	[flux-dns-gateway/src/dns_gateway/api/routes/dns.py:26,38,54]
flux-geo-cert certbot subprocess timeout	300 s (5 min)	[flux-geo-cert/files/cert_orchestrator/src/cert_orchestrator/managers/cert_manager.py:327]
flux-geo-cert DNS propagation poll interval	15 s	[flux-geo-cert/files/cert_orchestrator/src/cert_orchestrator/managers/dns_manager.py:275,283–300]
flux-geo-cert renewal threshold (default, validated 1..89)	30 days before expiry	[flux-geo-cert/files/cert_orchestrator/src/cert_orchestrator/config/settings.py:100,119–126]
fluxos-network-policy fetch interval: blocked repos, enterprise nodes	6 h	[fluxos-network-policy/README.md:30]
fluxos-network-policy fetch interval: tampering blocklist	12 h	[fluxos-network-policy/README.md:30]
fluxos-network-policy iplocation baseline fetch	daily, conditional	[fluxos-network-policy/.github/workflows/monthly-baseline.yml:82]

A.12 Attestation and boot

Table 98 covers `fluxsas` ports and intervals, the watchdog poll period, and the ArcaneOS release/build constants.

Table 98: Attestation and boot constants (`fluxsas`, ArcaneOS).

Constant	Value	Provenance
`fluxsas` mTLS listen port (single listener; a stale doc comment names a second, non-existent port 16102)	16100	[fluxsas/src/api_server.h:972–999,1042–1053]; [fluxsas/secrets:225]
`MAX_REQUEST_BODY_SIZE`	16777216 bytes (16 MiB)	[fluxsas/src/api_server.h:55]
`MAX_APP_NAME_LENGTH` / `MAX_FLUX_ID_LENGTH` / `MAX_MESSAGE_LENGTH`	63 / 64 / 20000	[fluxsas/src/api_server.h:56–58]
`BLOB_SIGN_FRESHNESS_SECONDS`	300 s	[fluxsas/src/api_server.h:183]

Table 98: *continued from previous page*

Constant	Value	Provenance
Per-app leaf certificate validity	30 days (default)	[fluxsas/src/crypto/ca.h:60–65,132–134]
Per-app CA self-signed validity window	`notBefore`=1767225600 (2026-01-01), `notAfter`=4922899200 (2126-01-01) – a 100-year window	[fluxsas/src/crypto/ca.h:239–240]
TPM PCR sealed by `systemd-cryptenroll`	PCR 7 only (`-tpm2-pcrs=7`) – Secure Boot policy state, not kernel/initrd/rootfs identity	[fluxsas/src/fes/fes.cpp:174]
Watchdog poll interval	two 5 s sleeps per loop iteration (≈10 s/iteration)	[fluxsas/src/watchdog.h:734–744]
Watchdog periodic hash-check period	`timer > 8600 + random[1,8600]` loop-iterations at ≈10 s/iteration ⇒ ≈24–48 h (in-code comment says a fixed 24 h, which undercounts)	[fluxsas/src/watchdog.h:721–725,745–748]
Unlock/rotate retry policy	3 retries, 10 s apart	[fluxsas/src/disk_encryption.h:701–717,723–731,742–750]
`sas` version	1.2.1	[fluxsas/src/sas.cpp:183]
`fes` version	1.0.0	[fluxsas/src/fes/fes.cpp:376]
FES exit codes: Secure Boot disabled / PK validation failed / TPM2 absent / TPM2-enroll failed / VPS-check failed / PKCS11-enroll failed / keystore failed / device-unlock failed	50 / 51 / 52 / 53 / 54 / 55 / 56 / 57	[fluxsas/secrets:FES_EXIT_* block]
Countdown before poweroff on enrollment failure (`fes-wrapper`)	30 s	[fluxos-iso-pkcs11/99flux-crypt/fes-wrapper:36,39]
`FLUX_PK_SHA256SUM` (hash of the DER-encoded custom UEFI platform key)	`820ddf304e3cff16 13603f529dc6d558 2343b90b8b1750ce e188b25d609ccff0`	[fluxos-iso/Makefile:65]
Ubuntu base release for FluxOS/ArcaneOS images	`resolute` / 26.04 (“Resolute Raccoon”)	[fluxos-iso/release_config.env:1–3]
`KERNEL_PIN` (pinned image/kmss kernel)	`7.0.0-29-generic`	[fluxos-iso/release_config.env:9]
Ed25519 release-signing signature / trailer length	64 bytes / 4 bytes	[fluxos-iso/flux-release-signer/src/flux_release_signer/main.py:20–21]
Release-tarball hashing/signing chunk size	268435456 bytes (256 MiB)	[fluxos-iso/flux-release-signer/src/flux_release_signer/main.py:19]
HSM signing-client TCP port	38678	[flux-iso-hsm/docs/PIN_FLOW_EFI_SIGNING.md:63]

Table 98: *continued from previous page*

Constant	Value	Provenance
HSM WireGuard-tunnel forwarder port	8888	[flux-iso-hsm/src/flux_hsm/server/socket_server.py:1198]
Root CA / FluxOS Devices PK/KEK/DB / Release-signing key object IDs	0x1200/0x1210, 0x1500/0x1510, 0x1501/0x1511, 0x1502/0x1512, 0x1400	[flux-iso-hsm/src/flux_hsm/hsm_manager.py:181–336]
Two-CA-slot rotation (PoN attestation-analogue for FDM)	`SAS_TLS_CA_CERT` / `SAS_TLS_CA_CERT_PREV`	[fluxsas/src/api_server.h:978–999]
FDM-arcane / DNS-gateway CA certificate validity	3650 days (10 years)	[flux-fdm-infrastructure/ansible/playbooks/apps-dns-manager/README.md:29]

A.13 Measured network state

Table 99 gives the fleet census, locked collateral, deployed application counts, committed resources, and shielded value pools, each read from the live public API at chain height 2917115, retrieval date 2026-09-03.

Table 99: Measured network state.

Quantity	Value	Provenance
Fleet total (`getzelnodecount`)	6489 nodes, all `stable`	[measured: api.runonflux.io/daemon/getzelnodecount, 2026-09-03]
Cumulus nodes / collateral each / tier collateral	3247 / 1000 FLUX / 3247000 FLUX	[measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03]
Nimbus nodes / collateral each / tier collateral	1615 / 12500 FLUX / 20187500 FLUX	[measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03]
Stratus nodes / collateral each / tier collateral	1627 / 40000 FLUX / 65080000 FLUX	[measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03]
Total locked collateral	88514500 FLUX (88514.5 voting-weight units at 1 unit/1000 FLUX)	[measured: api.runonflux.io/daemon/listzelnodes, 2026-09-03]
IPv4 / IPv6 / onion node addresses	2322 / 0 / 0	[measured: api.runonflux.io/daemon/getzelnodecount, 2026-09-03]
Deployed application specifications (`globalappsspecifications`)	1195 records	[measured: api.runonflux.io/apps/globalappsspecifications, 2026-09-03]

Table 99: *continued from previous page*

Quantity	Value	Provenance
Deployed apps by spec version: v2/v3/v4/v5/v6/v7/v8	2 / 27 / 11 / 3 / 70 / 209 / 873	[measured: api.runonflux.io/apps/globalappsspecifications, 2026-09-03]
Total requested instances / compose components	7486 / 1371	[measured: api.runonflux.io/apps/globalappsspecifications, 2026-09-03]
Committed CPU (vCPU-equivalents, summed over instances)	8972.0	[measured: api.runonflux.io/apps/globalappsspecifications, 2026-09-03]
Committed RAM (summed over instances)	17544229 MB (17133.0 GiB)	[measured: api.runonflux.io/apps/globalappsspecifications, 2026-09-03]
Committed HDD (summed over instances)	160107 GB	[measured: api.runonflux.io/apps/globalappsspecifications, 2026-09-03]
Chain tip at time of measurement	height 2917115	[measured: api.runonflux.io/daemon/getblockchaininfo, 2026-09-03]
Shielded value pool: `sprout`	19291.17735041 FLUX (1929117735041 zat)	[measured: api.runonflux.io/daemon/getblockchaininfo, 2026-09-03]
Shielded value pool: `sapling`	77188.75921324 FLUX (7718875921324 zat)	[measured: api.runonflux.io/daemon/getblockchaininfo, 2026-09-03]
Total shielded value	96479.93656365 FLUX (9647993656365 zat)	[measured: api.runonflux.io/daemon/getblockchaininfo, 2026-09-03]
Shielded share of issued supply (against 429466823.97 FLUX at height 2912015)	0.0225%	[measured: api.runonflux.io/daemon/getblockchaininfo, 2026-09-03]

Reproduction note

The values in this appendix were current at the time of this survey (evidence collected through 2026-09-03; measured-network figures retrieved 2026-09-03 at chain height 2917115). Consensus constants (Tables 87 to 93) change only by network upgrade, whereas policy constants such as the application price table (Table 96) change by height-scheduled revision or live configuration update without a consensus-level upgrade; the reproduction instructions for every measured and

computed value in this appendix are given in Appendix C.

B Interface specifications

This appendix collects the interface surfaces cited elsewhere in the paper: the application specification that a workload's owner submits and signs, the provisioning APIs an autonomous $\mathcal{A}$uses to act on it, and the HTTP/RPC surfaces of the running node. A table or listing in this appendix shows the *shape* of an interface — its fields, routes, or methods, as they appear in the schema or the code that parses it; the claim that a field, route, or method is actually exercised by a live consumer rests entirely on the citation printed beside it, never on the listing itself, and several rows below are explicitly marked where no such consumer was found.

B.1 The application specification

B.1.1 The deployed version (spec v8)

SHIPPED

The evidence establishes SPEC v8's permitted top-level and per-component keys as an explicit whitelist enforced in code — any other key is a hard validation failure — together with the chain height at which several fields became legal and two numeric constraints (name length, minimum instance count). It does *not* include a machine-readable SPEC v8 JSON Schema, so the *Type* and *Required* columns below are left blank except where a constraint is independently evidenced; where evidence gives a field but not its type, required/optional status, or introducing version, the corresponding cell is left blank rather than guessed, consistent with this appendix's rule.

Table 100: SPEC v8 field table (deployed FluxOS application specification). "Introduced" gives the enforcement height's version per Table 101, where evidence dates the field.

Scope	Field	Type	Constraint	Req.	Introduced	Source
root	`version`	—	height-gated 1–8, see Table 101	—	v1 (core)	[inferred] [flux/ZelBack/src/services/appRequirements/appValidator.js:1063-1067]
root	`name`	—	max length 63 (SPEC v8 +), 32 (older)	—	v1 (core)	[inferred] [flux/ZelBack/src/services/appRequirements/appValidator.js:577,713,1063-1067]
root	`description`	—	—	—	v1 (core)	[inferred] [flux/ZelBack/src/services/appRequirements/appValidator.js:1063-1067]
root	`owner`	—	—	—	v1 (core)	[inferred] [flux/ZelBack/src/services/appRequirements/appValidator.js:1063-1067]
root	`compose`	array	array of components (see below)	—	v4, 1,004,000	[flux/ZelBack/config/default.js:231-240]
root	`instances`	—	minimum 1 (SPEC v8 +, height $\geq$ 2,176,519), else minimum 3	—	not dated in evidence	[flux/ZelBack/src/services/appRequirements/appValidator.js:820-831]
root	`contacts`	—	—	—	v5, 1,142,000	[flux/ZelBack/config/default.js:231-240]
root	`geolocation`	—	`acXX`/`a!cXX` allow/deny prefix codes	—	v5, 1,142,000	[flux/ZelBack/config/default.js:231-240]
root	`expire`	—	—	—	v6, 1,300,000	[flux/ZelBack/config/default.js:231-240]

(Table 100 continued)

Scope	Field	Type	Constraint	Req.	Introduced	Source
root	`nodes`	array	IP allow-list; strict for enterprise apps, advisory only for non-enterprise SPEC v8 + apps	—	v7	[flux/ZelBack/config/default.js:231-240]; [flux/ZelBack/src/services/appLifecycle/appSpawner.js:288-296,295]
root	`staticip`	—	—	—	v7	[flux/ZelBack/config/default.js:231-240]
root	`datacenter`	—	routes to enterprise-owner nodes; the deferral branch written for it is dead code per one code comment	—	not dated in evidence	[flux/ZelBack/src/services/appLifecycle/appSpawner.js:568]
root	`enterprise`	boolean	gates the encryption envelope; ArcaneOS apps	—	v8, 1,932,380	[flux/ZelBack/config/default.js:231-240]
component	`name`	—	—	—	—	[flux/ZelBack/src/services/appRequirements/appValidator.js:1068-1071]
component	`description`	—	—	—	—	[flux/ZelBack/src/services/appRequirements/appValidator.js:1068-1071]
component	`repotag`	—	Docker Hub `repo:tag`; whitelist, existence and architecture checked (`imageManager.verifyRepository`)	—	—	[flux/ZelBack/src/services/appRequirements/appValidator.js:1068-1071,1243-1400]
component	`ports`	—	—	—	—	[flux/ZelBack/src/services/appRequirements/appValidator.js:1068-1071]
component	`containerPorts`	—	—	—	—	[flux/ZelBack/src/services/appRequirements/appValidator.js:1068-1071]
component	`environmentParameters`	—	—	—	—	[flux/ZelBack/src/services/appRequirements/appValidator.js:1068-1071]
component	`commands`	—	—	—	—	[flux/ZelBack/src/services/appRequirements/appValidator.js:1068-1071]
component	`containerData`	string	persistent-storage mount path (contrast `volumes` in SPEC v9, Table 102)	—	—	[flux/ZelBack/src/services/appRequirements/appValidator.js:1068-1071]
component	`domains`	—	—	—	—	[flux/ZelBack/src/services/appRequirements/appValidator.js:1068-1071]

(Table 100 continued)

Scope	Field	Type	Constraint	Req.	Introduced	Source
component	`repoauth`	—	private-image credential; when set, skips the Docker Hub whitelist/existence check path	—	v7	[flux/ZelBack/config/default.js:231-240]; [flux/ZelBack/src/services/appRequirements/appValidator.js:1243-1400]
component	`cpu`	—	—	—	—	[flux/ZelBack/src/services/appRequirements/appValidator.js:1068-1071]
component	`ram`	—	—	—	—	[flux/ZelBack/src/services/appRequirements/appValidator.js:1068-1071]
component	`hdd`	—	—	—	—	[flux/ZelBack/src/services/appRequirements/appValidator.js:1068-1071]

Version	Mainnet height	Dev height	Notes
1	0	0	deprecated for fresh submissions
2	0	0	—
3	983,000	983,000	SPEC v1/SPEC v2 still submittable by users; UI allows only SPEC v2/SPEC v3
4	1,004,000	1,004,000	`compose` array (composition)
5	1,142,000	1,142,000	`contacts`, `geolocation`
6	1,300,000	1,300,000	`expire`, app price change, `t3` multisig payment address
7	1,420,000	1,390,000	`nodes` targeting, `secrets`, `repoauth`, `staticip`
8	1,932,380	1,921,500	`enterprise`/ArcaneOS apps

Table 101: SPEC v1–SPEC v8 enforcement heights, [flux/ZelBack/config/default.js:231-240].

B.1.2 The draft spec v9 schema

IN-PROGRESS

The SPEC v9 draft is a JSON Schema (draft 2020-12), and unlike SPEC v8 its full field-level type and constraint information is available [flux-spec/schema/v9.json:2-3]. Every object in the schema sets `additionalProperties:false`, so an unlisted key at any level is a validation failure, not a silent drop [flux-spec/schema/v9.json:7]. Ten fields are required at the root; three of them (`paymentMemo`, `referralCode`, `machineType`) have zero occurrences anywhere in the FluxOS fork that otherwise implements most of SPEC v9's substrate — no billing, referral, or capacity-tier code reads them [MorningLightMountain713/flux@feat/fluxos-v9:ZelBack/src/services/pricing/v9PricingRegime.js (grep: zero hits for paymentMemo, referralCode, machineType)]. If `route.protocol` is `tcp`, the fields `scheme`, `httpsRedirect`, `customDomains` and `managedCertificates` are structurally forbidden by an `if/then` block, not merely ignored [flux-spec/schema/v9.json:336-350].

Table 102: SPEC v9 draft field table (`flux-spec/schema/v9.json`, RFC-0). `$defs.component` values sit under `components.*`; `$defs.route` values sit under `loadBalancing.routes.*`.

Scope	Field	Type / constraint	Req.	Source (schema/v9.json)
root	`version`	`const: 9`	yes	[flux-spec/schema/v9.json:21-24]
root	`name`	pattern `^[a-zA-Z0-9](?:[a-zA-Z0-9-]*[a-zA-Z0-9])?$`, maxLength 63	yes	[flux-spec/schema/v9.json:25-28,85-90]
root	`description`	string, minLength 1, maxLength 256	yes	[flux-spec/schema/v9.json:29-34]
root	`owner`	string, minLength 1	yes	[flux-spec/schema/v9.json:35-39]
root	`components`	object map, minProperties 1, keys=`dnsLabel`	yes	[flux-spec/schema/v9.json:40-46]
root	`loadBalancing`	object, `additionalProperties:false`, requires `routes`	yes	[flux-spec/schema/v9.json:47-61]
root	`loadBalancing.routes`	object map, minProperties 1, keys=`dnsLabel`	yes (nested)	[flux-spec/schema/v9.json:53-59]
root	`duration`	integer, minimum 1 (seconds; example 2,592,000 = 30 days)	yes, new	[flux-spec/schema/v9.json:62-66]
root	`paymentMemo`	string, minLength 1, maxLength 512 (provisional)	yes, new	[flux-spec/schema/v9.json:67-72]
root	`referralCode`	string, minLength 1, maxLength 64 (provisional)	yes, new	[flux-spec/schema/v9.json:73-78]
root	`machineType`	enum `standard|gpu|vps`	yes, new	[flux-spec/schema/v9.json:79-82]
component	`image`	string, minLength 1 (replaces `repotag`)	yes	[flux-spec/schema/v9.json:103-107]
component	`cpu`	number, exclusiveMinimum 0 (cores, decimal)	yes	[flux-spec/schema/v9.json:108-112]
component	`memory`	integer, exclusiveMinimum 0 (MiB; replaces `ram`)	yes	[flux-spec/schema/v9.json:113-117]
component	`rootFsGb`	integer, minimum 1 (GiB; replaces `hdd`)	yes	[flux-spec/schema/v9.json:118-122]
component	`replicas`	integer, minimum 1, default 1	yes	[flux-spec/schema/v9.json:123-128]
component	`ports`	object map, keys=`dnsLabel`	no	[flux-spec/schema/v9.json:129-150]
component	`ports.*.containerPort`	integer, 1–65535	yes (in obj)	[flux-spec/schema/v9.json:138-142]
component	`ports.*.protocol`	enum `tcp|http`	no	[flux-spec/schema/v9.json:144-147]
component	`volumes`	object map, keys=`dnsLabel` (replaces `containerData` string)	no	[flux-spec/schema/v9.json:151-172]
component	`volumes.*.sizeGb`	integer, minimum 1	yes (in obj)	[flux-spec/schema/v9.json:160-163]
component	`volumes.*.mountPath`	string, pattern `^/`	yes (in obj)	[flux-spec/schema/v9.json:165-168]
component	`placement`	object, `additionalProperties:false`	no	[flux-spec/schema/v9.json:173-188]
component	`placement.mode`	enum `active-standby|active-active`	no	[flux-spec/schema/v9.json:178-181]
component	`placement.syncBeforeStart`	boolean, default false	no	[flux-spec/schema/v9.json:182-185]
route	`protocol`	enum `http|tcp`	yes	[flux-spec/schema/v9.json:197-200]
route	`targetPort`	string, pattern `<component>/<port>`, maxLength 127	yes	[flux-spec/schema/v9.json:201-206]

(Table 102 continued)

Scope	Field	Type / constraint	Req.	Source (schema/v9.json)
route	`scheme`	enum `http|https` (HTTP only)	no	[flux-spec/schema/v9.json:207-210]
route	`httpsRedirect`	boolean (HTTP only)	no	[flux-spec/schema/v9.json:211-214]
route	`customDomains`	array of FQDN, minItems 1, uniqueItems, item maxLength 253 (HTTP only)	no	[flux-spec/schema/v9.json:215-226]
route	`managedCertificates`	boolean (HTTP only)	no	[flux-spec/schema/v9.json:227-230]
route	`healthCheck`	object, `additionalProperties:false`	no	[flux-spec/schema/v9.json:231-257]
route	`healthCheck.path`	string, pattern `^/`	no	[flux-spec/schema/v9.json:236-240]
route	`healthCheck.intervalSeconds`	integer, minimum 1	no	[flux-spec/schema/v9.json:241-245]
route	`healthCheck.timeoutSeconds`	integer, minimum 1	no	[flux-spec/schema/v9.json:246-250]
route	`healthCheck.unhealthyThreshold`	integer, minimum 1	no	[flux-spec/schema/v9.json:251-255]
route	`timeouts`	object, `additionalProperties:false`	no	[flux-spec/schema/v9.json:258-274]
route	`timeouts.requestSeconds`	integer, minimum 1	no	[flux-spec/schema/v9.json:263-267]
route	`timeouts.idleSeconds`	integer, minimum 1	no	[flux-spec/schema/v9.json:268-272]
route	`stickySessions`	boolean	no	[flux-spec/schema/v9.json:275-278]
route	`connectionLimits`	object, `additionalProperties:false`	no	[flux-spec/schema/v9.json:279-290]
route	`connectionLimits.maxConnections`	integer, minimum 1	no	[flux-spec/schema/v9.json:284-288]
route	`retries`	integer, minimum 0	no	[flux-spec/schema/v9.json:291-295]
route	`backendTls`	object, `additionalProperties:false`, requires `enabled`	no	[flux-spec/schema/v9.json:296-317]
route	`backendTls.enabled`	boolean	yes (in obj)	[flux-spec/schema/v9.json:301-304]
route	`backendTls.certificateAuthorities`	array of PEM strings, minItems 1	no	[flux-spec/schema/v9.json:306-315]
route	`connectionDraining`	object, `additionalProperties:false`, requires `enabled`	no	[flux-spec/schema/v9.json:318-334]
route	`connectionDraining.enabled`	boolean	yes (in obj)	[flux-spec/schema/v9.json:323-326]
route	`connectionDraining.drainSeconds`	integer, minimum 1	no	[flux-spec/schema/v9.json:328-332]

Change	SPEC v8	SPEC v9	Note
Renamed	`compose`	`components`	positional array → keyed map, order carried by key name
Renamed	`repotag`	`image`	
Renamed	`ram`	`memory`	
Renamed	`hdd`	`rootFsGb`	
Reshaped	`containerData` (string)	`volumes` (map)	sized, per-volume `mountPath`
Reshaped	`ports/containerPorts`	`ports` (named)	
Removed	`enterprise`	—	no replacement in the draft; the private-deployment encryption envelope is deferred to a future revision
Added, required	—	`loadBalancing`	routes, health checks, TLS, draining declared in-spec
Added, required	—	`duration`	
Added, required	—	`paymentMemo`	no consumer implementation found
Added, required	—	`referralCode`	no consumer implementation found
Added, required	—	`machineType`	no consumer implementation found

Table 103: SPEC v9 vs. SPEC v8 delta. [flux-spec/README.md:13-21,58-67,95-96]; [MorningLightMountain713/flux@feat/fluxos-v9:ZelBack/src/services/pricing/v9PricingRegime.js and ZelBack/, tests/ (grep: zero hits for paymentMemo, referralCode, machineType)].

B.1.3 Two artifacts, not one

The name "SPEC v9" is claimed by two objects that are not the same artifact. The first is `Run OnFlux/flux-spec`, the RFC-0 draft repository tabulated above: schema-only, two commits on one day, `private:true`, no `bin`/`main`/`exports` field, no CI, and explicitly self-described as "*not live on the network*" [doc: flux-spec/README.md:3]. The second is what the active contributor fork's FluxOS code actually requires at runtime: `specLibs.js` pulls `@runonflux/flux-spec-cjs` from a *different*, monorepo-structured, commit-pinned private package (`packages/spec`, `spec-backend`, `spec-policy`) that is not present in this paper's evidence corpus at all [MorningLightMountain713/flux@feat/fluxos-v9:package.json:87-90]; [MorningLightMountain713/flux@feat/fluxos-v9:ZelBack/src/services/utils/specLibs.js:1-30]. No evidence available to this paper shows the RFC-0 repository's `schema/v9.json` being read by FluxOS, `fluxos-frontend`, or any other deployed service [doc: flux-spec/README.md:120-125]. This paper therefore does not present the RFC-0 draft and the fork's implementation as one object.

One concrete divergence was found between them, and it is left unresolved because the evidence does not resolve it: RFC-0's schema names the duration field `duration` [flux-spec/schema/v9.json:15,62], while the fork's pricing code reads the same concept off `spec.ttl` in six places [MorningLightMountain713/flux@feat/fluxos-v9:ZelBack/src/services/pricing/v9PricingRegime.js:95,108,122,128,183,228,279,308]. Whether this is an internal rename inside an unavailable vendored class or a genuine schema/implementation drift could not be determined from the evidence available to this paper; the two names are reported here, not merged.

B.1.4 Canonicalization

Canonicalization matters because it is what a fingerprint — and, by extension, a signature over that fingerprint — would cover. The RFC-0 draft specifies, as prose, RFC 8785 JSON Canonicalization Scheme (JCS) as its base: UTF-8 encoding, no insignificant whitespace, object keys sorted lexicographically by UTF-16 code unit, RFC 8785 minimal string escaping, and ECMAScript `Number::toString` shortest-round-trip number formatting [doc: flux-spec/validation/canonicalization.md:9-20]. On top of JCS it adds three SPEC v9-specific rules: keyed maps (`components`, `ports`, `volumes`, `loadBalancing.routes`) carry ordering structurally through key-sort, so no positional normalization is needed [doc: flux-spec/validation/canonicalization.md:24-29]; order-insensitive arrays (`customDomains`, `certificate`

`Authorities`) *must* be sorted lexicographically by the producer before serialization [doc: flux-spec/validation/canonicalization.md:31-38]; and defaults (`replicas:1`, `placement.syncBeforeStart: false`) *must* be materialized explicitly before serialization, so an omitted field and its explicit default hash identically [doc: flux-spec/validation/canonicalization.md:40-45]. A fingerprint is then defined as `hex(SHA256(SHA256(canonical_bytes)))`, labelled "recommended," not required [doc: flux-spec/validation/canonicalization.md:53-63].

These three SPEC v9-specific rules are stated with the normative word `MUST` and have *no schema-level or code-level enforcement anywhere in the evidence*: JSON Schema has no array-sort keyword, and nothing in the repository's own validation harness checks array order or default materialization [doc: flux-spec/validation/canonicalization.md:31-38,40-45]. A document can therefore be schema-*valid* and canonically *invalid* — two producers who disagree only on array order or on whether they wrote out a default will validate identically and fingerprint differently, with nothing in the evidence-cited code path that would detect it. Separately, and independent of the enforcement gap: `schema/v9.json` contains *no `signature` field anywhere*, confirmed by grep across the RFC-0 repository [doc: flux-spec/schema/v9.json (grep: no signature/signed field)]. The draft defines a fingerprint recipe for comparing two documents; it does not define who signs a submitted SPEC v9 specification, or what exactly is signed.

```
{
  "version": 9, "name": "orbit-demo",
  "description": "Reference Orbit deployment",
  "owner": "1FluxOwnerZelIDxxxxxxxxxxxxxxxx",
  "components": {"web": {
    "image": "runonflux/orbit:latest",
    "cpu": 1, "memory": 2048, "rootFsGb": 10, "replicas": 2
  }},
  "loadBalancing": {"routes": {"web": {
    "protocol": "http", "targetPort": "web/http",
    "customDomains": ["example.com", "www.example.com"]
  }}},
  "duration": 2592000,
  "paymentMemo": "sub-2026-09",
  "referralCode": "none",
  "machineType": "standard"
}
```

Listing 3: A schema-conformant SPEC v9 specification. Note the four required, new-in-SPEC v9 fields (`duration`, `paymentMemo`, `referralCode`, `machineType`) with no FluxOS consumer (Table 103); the keyed `components` map replacing SPEC v8's positional `compose` array; and `customDomains` pre-sorted per the canonicalization `MUST` rule that nothing here enforces.

B.2 The Provider Runtime Contract v0

IN-PROGRESS

The Provider Runtime Contract v0 is an RFC draft with no consumer: its own text states it is "published for FluxCore-team sign-off . . . nothing here is live" [doc: flux-spec/provider-runtime/contract.md:3-8], and no code outside this schema and its fixture harness reads it.

Field	Type / constraint	Req.	Source
`contractVersion`	`const: 0`	yes	[flux-spec/provider-runtime/provider-registration.schema.json:16-18]
`providerId`	string, minLength 1, maxLength 128, opaque to platform	yes	[flux-spec/provider-runtime/provider-registration.schema.json:20-24]
`capabilities`	object, `additionalProperties:false`, requires `bonded`, `gpu`, `staticIP`	yes	[flux-spec/provider-runtime/provider-registration.schema.json:26-45]
`capabilities.bonded`	boolean	yes (in obj)	[flux-spec/provider-runtime/provider-registration.schema.json:32-35]
`capabilities.gpu`	boolean	yes (in obj)	[flux-spec/provider-runtime/provider-registration.schema.json:36-39]
`capabilities.staticIP`	boolean	yes (in obj)	[flux-spec/provider-runtime/provider-registration.schema.json:40-43]
`heartbeatIntervalSeconds`	integer, [10, 300]; 60 stated as "expected default," not a schema `default`	yes	[flux-spec/provider-runtime/provider-registration.schema.json:46-50]
`attestations`	array, minItems 1, absence = standard trust tier	no	[flux-spec/provider-runtime/provider-registration.schema.json:52-75]
`attestations[].evidenceType`	string, pattern `^[a-z0-9]([a-z0-9-]*[a-z0-9])?$`, maxLength 63; open token registry, not a closed enum	yes (in item)	[flux-spec/provider-runtime/provider-registration.schema.json:60-65]
`attestations[].reference`	string, minLength 1, maxLength 512	yes (in item)	[flux-spec/provider-runtime/provider-registration.schema.json:66-71]
`payoutDestination`	string, minLength 1, maxLength 128 (ZelID today)	yes	[flux-spec/provider-runtime/provider-registration.schema.json:76-80]

Table 104: Provider Runtime Contract v0 — provider-registration schema, 11 fields (12 counting the nested `capabilities` object separately).

The heartbeat ladder itself is specified only in prose, with no implementation of the state machine anywhere in the evidence [doc: flux-spec/provider-runtime/contract.md:63-85]. A heartbeat is defined payload-agnostically, as any authenticated message from the provider within the interval [doc: flux-spec/provider-runtime/contract.md:67-69], and clock authority is stated to be the platform's, not the provider's [doc: flux-spec/provider-runtime/contract.md:83].

From	Trigger	To	Source
registered, on-time	interval expires, no heartbeat	LATE (recorded, no action)	[doc: flux-spec/provider-runtime/contract.md:63-85]
LATE	3 consecutive expired intervals	SUSPENDED (no new work; in-flight work runs to its booking boundary)	[doc: flux-spec/provider-runtime/contract.md:75-80]
SUSPENDED	10 consecutive expired intervals (total)	LAPSED (registration removed from scheduling)	[doc: flux-spec/provider-runtime/contract.md:78-81]
LAPSED	—	only path back is a fresh registration (full payload, current evidence); no silent resume	[doc: flux-spec/provider-runtime/contract.md:63-85]
any state	authenticated heartbeat within interval	on-time	[doc: flux-spec/provider-runtime/contract.md:63-85]

Table 105: Provider Runtime Contract v0 heartbeat state machine (prose specification; no code implementation found in the evidence).

B.3 The agent-facing provisioning API

IN-PROGRESS

shipped parts of this surface are cited individually below; the section as a whole mixes a shipped API (FluxEdge, a separate, centrally operated product whose machines lock no collateral and are not FluxNodes) with an early-stage one (the MCP server), so the maturity tag is attached per claim rather than once for the section.

B.3.1 MCP tools: `deploy_app`, `update_app`, `renew_app`

SHIPPED

`deploy-with-git-frontend` exposes an MCP server (`POST /mcp`, bearer-token authenticated, JSON-RPC) with fifteen tools, of which `deploy_app`, `update_app` and `renew_app` are the non-read-only, state-changing ones relevant here [deploy-with-git-frontend/server/mcp/createServer.js:13-53,66-266]; [deploy-with-git-frontend/server/mcp/router.js:13-44].

Tool	What it does	Source
`deploy_app`	"Revalidate, Firebase-sign, register, and test-install an Orbit app" (tool's own description)	[deploy-with-git-frontend/server/mcp/createServer.js:174-177]
`update_app`	submits a signed update to an existing app spec via `UpdateService.submit()`, which relays the message through the same Firebase-sign path as `deploy_app`	[deploy-with-git-frontend/server/orbit/updateService.js:68-79]
`renew_app`	listed as an MCP tool; evidence does not give a description string or locate its backing service function beyond its inclusion among the tools flagged `destructiveHint:true`	[deploy-with-git-frontend/server/mcp/createServer.js:170-266]

Table 106: MCP provisioning tools. All three are `destructiveHint:true`, non-read-only tools that execute against live Flux state given only a Firebase bearer token [deploy-with-git-frontend/server/mcp/createServer.js:170-266].

Not established by this survey. the exact JSON-RPC parameter schema (argument names and types) for `deploy_app`, `update_app` and `renew_app` is not given in the available evidence beyond the underlying app-spec fields the frontend assembles from a wizard (repo URL, branch, plan, ports, domain, ...) — those are documented for the web-wizard flow, not confirmed as the literal MCP tool-call parameter names [deploy-with-git-frontend/src/services/deployService.js:194-219,334-343,451,453-490].

For the Firebase/FluxCore-SSO path — which is what every server-side MCP call above uses — signing is remote and custodial: the frontend server POSTs the message and the caller's Firebase ID token to `service.fluxcore.ai/api/signMessage`, a remote service that signs on the user's behalf, authorized only by that bearer token [deploy-with-git-frontend/server/flux/fluxSession.js:60-73]; [deploy-with-git-frontend/server/orbit/deploymentService.js:158-173]. Given a valid Firebase bearer token, an MCP client can drive spec construction through signed registration submission with no client-side wallet interaction at all [deploy-with-git-frontend/server/mcp/createServer.js:66-266].

B.3.2 The FluxEdge OpenAPI surface and `fluxedge-cli`

SHIPPED

FluxEdge (`pouwbackend`) publishes an OpenAPI 3.1.1 REST v1 surface at `/api/v1` [pouwbackend/rest_api/v1/openapi.yaml:1-4], with paths including `GET /gpus`, `GET /computers`, `GET /hardware_offers`, `POST /rental/start`, `POST /rental/stop`, `POST /rental/stopAll`, `POST /rental/startPremium`, `GET /deployments`, `POST /deployment/start`, `POST /deployment/stop`, `POST /deployment/stopAll`, `POST /deployment/redeploy`, and `POST /deployment/update` [pouwbackend/rest_api/v1/openapi.yaml:463-2193]. `fluxedge-cli` is a cross-platform Go CLI wrapping this surface, with key precedence `FLUXEDGE_API_KEY` env var > `~/.fluxedge/fluxedge.yaml` > `-api-key` flag, and `-o json` output explicitly recommended for scripting [pouwbackend/rest_api/v1/openapi.yaml:127-156].

Token scope	Enforced against	Source
`start_rental`	`POST /v1/rental/start`	[pouwbackend/rest_api/ratelimit.go:378-390]
`stop_rental`	`POST /v1/rental/stop`, `POST /v1/rental/stopAll`	[pouwbackend/rest_api/ratelimit.go:378-390]
`start_deployment`	`POST /v1/deployment/start`	[pouwbackend/rest_api/ratelimit.go:378-390]
`stop_deployment`	`POST /v1/deployment/stop`	[pouwbackend/rest_api/ratelimit.go:378-390]
`get_balance`	enumerated in the same privilege block; no path mapping given in evidence	[pouwbackend/rest_api/v1/openapi.yaml:2576-2581]

Table 107: FluxEdge bearer-token scopes, [pouwbackend/rest_api/v1/openapi.yaml:2571-2581]. Auth scheme: `BearerAuth`, `bearerFormat: API_KEY`.

Rate limiting and abuse handling are both real, shipped code, keyed independently on the hashed API key and on the caller's IP: 2 requests/second, reset window 1 s [pouwbackend/rest_api/ratelimit.go:41-42,79-88]; a key or IP that accumulates 5 violations within a rolling 5-minute window is automatically revoked (`db.RevokeAPIKey`, owner email, cache eviction, HTTP 403) [pouwbackend/rest_api/ratelimit.go:143-222]; violations themselves auto-expire after 1 hour [pouwbackend/rest_api/ratelimit.go:45].

```
// POST /rental/start
{"min_memory":16,"min_storage":500,"gpu":"RTX3080","nb_gpu":1,
 "region":"NorthAmerica","max_price_per_hour":0.5,
 "nb_computers":3,"premium":"none"}
// 200
{"rentals":[{"rental_id":8782678,"price_per_hour":0.5,
 "status":"active","computer":{"hash":"e5b37bd2716472",
 "memory":16,"storage":500,"gpus":["RTX3080"],"nb_gpu":1,
 "region":"NorthAmerica","cluster_name":"us-east-1",
 "price_per_hour":0.5}}]}
```

Listing 4: A criteria-based rental request and its response. The token that issues this call is scoped to `start_rental` (Table 107) but carries no bound on `max_price_per_hour` or `nb_computers` beyond what the caller writes into the request body.

[pouwbackend/rest_api/v1/openapi.yaml:1330-1338,1358-1382]

Two properties a reader needs in order to evaluate this surface. First, there is no self-service key issuance: key issuance is not exposed anywhere in the REST v1/OpenAPI surface, and the CLI documentation itself points callers to a web dashboard (`pouwmarketplace.runonflux.io/#/account/api`) — a human must mint a token before an agent can use this API at all [pouwbackend/rest_api/v1/openapi.yaml:170]; [pouwbackend/rest_api/ratelimit.go:375-390]. Second, a token carries no spend bound: its scopes bound which *actions* it may call (Table 107), not the *value* those actions commit — there is no per-transaction cap, no rate limit on value, no budget, no scope over value, and no revocation mechanism beyond invalidating the token outright on abuse (Table 107's rate-limit/revocation mechanism triggers on request volume, never on spend). §17 specifies the delegation primitive that would bound this; as deployed today, authority is bounded by action and unbounded by value.

B.4 The FluxOS HTTP surface

SHIPPED

Route group	Representative endpoints	Privilege tier	Source
`/daemon/*`	`getinfo`, `listfluxnodes`, `getbestblockhash` (GET)	public/cached	[flux/ZelBack/src/routes.js:78-120]
	`prioritisetransaction`, `submitblock` (GET)	`user`	[flux/ZelBack/src/routes.js:501-507]
	`stop`, `reindex`, `createfluxnodekey` (GET)	`admin`	[flux/ZelBack/src/routes.js:741-760]
	`start`, `restart`, `ping`, `startbenchmark` (GET)	`adminandfluxteam`	[flux/ZelBack/src/routes.js:1010-1022]
	`signrawtransaction`, `sendfrom`, `sendtoaddress` (POST)	`admin`	[flux/ZelBack/src/routes.js:1393-1408]
`/apps/*`	`listrunningapps`, `globalappsspecifications`, `appspecifications`, `calculateprice`, `deploymentinformation`	public/cached	[flux/ZelBack/src/routes.js:372-468]
	`appstart`, `appstop`, `apprestart`, `appexec`, `appremove`, `redeploy`	`appownerabove`	[flux/ZelBack/src/routes.js:1196-1265]
	`appregister` (POST)	`user`	[flux/ZelBack/src/routes.js:1383-1385]
	`appupdate` (POST)	`user` (self) or `adminandfluxteam` (on owner's behalf)	[flux/ZelBack/src/routes.js:1386-1388]
`/id/*` (and deprecated `/zelid/*`)	`loginphrase`, `verifylogin`, `loggedsessions`	`user`	[flux/ZelBack/src/routes.js:508-518]
	`logoutallsessions`	`admin`	[flux/ZelBack/src/routes.js:508-518]
`/benchmark/*`	`getbenchmarks`, `getstatus`, `getinfo`, `getpublicip`, `restartnodebenchmarks`	not specified in evidence	[flux/ZelBack/src/services/benchmarkService.js:126-427]
`/flux/*`	`startbenchmark`; `broadcastmessage`, `broadcastmessagetooutgoing`, `broadcastmessagetoincoming`; `/syncthing/*` config writes	`adminandfluxteam`	[flux/ZelBack/src/routes.js:1022,1429-1441]

Table 108: FluxOS HTTP route groups and privilege tiers.

Privilege is resolved by `verifyPrivilege`, dispatching to one of six checks keyed on a `zelidauth` header verified against the caller's ZelID [flux/ZelBack/src/services/verificationHelper.js:17-48,90-106]: `admin` (caller's ZelID equals the node operator's), `fluxteam` (caller's ZelID is one of two hardcoded team ZelIDs), `adminandfluxteam` (union of the two), `appowner`/ `appownerabove` (caller resolved as the app's current owner, plus admin/fluxteam for the `-above` variant), and `user` (any ZelID with a signature over a loginPhrase no older than 16 hours, 2 hours for the `appowner` checks) [flux/ZelBack/src/services/verificationHelperUtils.js:19-46,54-88,96-122,130-156,164-248,180-188,226-234].

B.5 The `fluxbench` RPC surface and the `fluxsas` protocol

FluxOS proxies a fixed set of `fluxbench` RPC methods over its own `/benchmark/*` route group SHIPPED. `fluxsas` exposes a separate, internally-bound HTTP namespace whose route-level detail is only partially covered by the available evidence.

Component	Method / route	Transport	Notes
`fluxbench` (via FluxOS proxy)	`getbenchmarks`	HTTP, `/benchmark/*` on FluxOS	return schema not detailed in evidence
	`getstatus`	HTTP, `/benchmark/*`	
	`getinfo`	HTTP, `/benchmark/*`	
	`getpublicip`	HTTP, `/benchmark/*`	
	`restartnodebenchmarks`	HTTP, `/benchmark/*`	
`fluxsas`	`/v2/*` namespace, purpose-scoped keys (`default`, `cdn`, `mesh`, `app`)	HTTPS, internal mTLS client certificate required, bound to `127.0.0.1:16100`	no tenant-facing verification path exists; `purpose=mesh` is present in code with no known consumer

Table 109: `fluxbench` proxy methods, [flux/ZelBack/src/services/benchmarkService.js:126-427]; `fluxsas` namespace, [private source: SAS API server]; [private source: SAS watchdog].

Not established by this survey. individual `fluxsas /v2/*` method names, per-method transport detail, and per-method return schemas are not enumerated in the evidence available to this section. The evidence establishes the namespace's authentication binding and that the `mesh`-purpose key has no known consumer, but not a method-by-method dispatch table comparable to the `fluxbench` row above.

C Emission model: derivation, discrepancy, and reproduction

Every emission figure in this paper is produced by one model, and this appendix states what that model is, where its constants come from, one place where it disagrees with its own documentation, and how to reproduce every number.

C.1 Provenance

The model is `supply/emission_model.py` in the `flux-roadmap` repository. It is not a reimplementation written for this paper: it is the team's own model, and it carries source citations to `chainparams.cpp`, `main.cpp` and `amount.h` for each constant it uses. Those citations were checked independently against the daemon source during the preparation of §5, and §A reproduces the constants with their locations.

C.2 Arithmetic conventions, which are load-bearing

All arithmetic is over integers in satoshi, matching `fluxd`'s `CAmount` (`int64`). Division is floor division, matching C++ integer division for non-negative operands, and » is an arithmetic right shift. This is not pedantry: the PoN reduction applies $s_{k+1} = \lfloor 9s_k/10 \rfloor$ once per interval, and because each step truncates independently, the result is *not* in general equal to $\lfloor s_0 \cdot 0.9^r \rfloor$. Any reimplementation using floating-point arithmetic will diverge from consensus.

C.3 The two modes, and the discrepancy

The model runs in two modes. `code` transcribes `GetBlockSubsidy` literally. `observed` reproduces what mainnet actually paid over blocks 1–4999, where the deployed behaviour differs from a literal reading of the slow-start branch by one block of offset. The two per-block subsidies converge for every height at or above 5,000.

The model's own docstring states that the two modes differ by 0.06 FLUX. Executing it shows an exact and persistent difference in cumulative supply of 150.00000000 FLUX at every height at or above 5,000, arising from the one-block index shift itself, which lowers every ramp block's payout rather than block 2's alone as the docstring's figure assumes. This is a discrepancy in the published model's documentation, not in consensus: neither mode changes what the chain paid, and the difference is confined to the first 5,000 blocks of a chain that is now past 2.9 million. This paper uses `observed` throughout, and reports supply as 429,466,823.9700 FLUX at height 2,912,015.

Remark C.1 (Status of this discrepancy). The 150.00000000 FLUX figure and its root cause — the index shift acting on every ramp block, not only on block 2 — were established by executing the published model, and are reproducible by anyone who runs it SHIPPED [flux-roadmap/supply/emission_model.py:207–229] [measured: flux-roadmap/supply/emission_model.py, both modes, 2026-09-03]. **This is recorded, not outstanding, so far as this paper is concerned**: nothing here depends on the model's docstring being right, every figure this paper reports comes from `observed` mode, and the difference is confined to the first 5,000 blocks of a chain now past 2.9 million. It has no consensus consequence.

What remains is an upstream housekeeping item for the model's own maintainers rather than a gap in this document: the docstring says 0.06 FLUX where execution gives 150 FLUX, and the fix is to the docstring's figure, since the closed-form sums agree with the per-block function in both modes. It is recorded here so that a reader who runs the model and sees a different number than the docstring promises knows why, and so that the correction does not depend on this paper being read.

C.4 The functions

§5 states σ_{PoW}, σ_{PoN} and S in full. This appendix adds only the two facts a reimplementer needs and the body omits: the model computes cumulative supply as a sum of six independently-

triggered step components rather than by iterating over blocks, so it is exact at any height without a loop; and it treats the dev-fund remainder as a *remainder* of the reduced subsidy, never as a fixed 0.5 FLUX, so the dev fund declines on the same schedule as the tier bases.

C.5 Reproduction

```
# Model report (mode=observed is the default)
python3 repos/flux-roadmap/supply/emission_model.py

# Regenerate the plotted data: emission.dat, tier_income.dat, pon_reductions.dat
python3 repos/flux-roadmap/supply/_gen_dat.py

# Live measurements (fleet, applications, pricing, value pools)
bash scripts/04-measure.sh
python3 scripts/05-network-analysis.py

# PNR settlement analysis: drip trade-off and crossover series
python3 scripts/07-pnr-settlement.py
```

Listing 5: Reproducing every emission figure in this paper.

Every generated `.dat` file under `paper/data/` begins with `#` comment lines recording the generating command, the date, the source model, and the assumptions in force — in particular that future-height projections hold block spacing at the 30 s target, which real spacing may not do.

C.6 What the model does not do

It does not model demand, revenue, or price. The PNR crossover series in §7 and §26 are produced by a separate script under explicitly stated demand-growth assumptions, with price exogenous and never forecast. The model also assumes the schedule as currently written in consensus: it does not anticipate the PNR activation, the parallel-asset cut, or any future change to the reduction grid.

D Notation index

Every symbol shared across this paper's sections is defined in `paper/notation.tex` and nowhere else; symbols local to one section are defined in the section that introduces them. Naming conflicts across the Flux repositories are decided there, once. This index lists the symbols by domain; tag macros are omitted.

Canonical Component Names

Symbol	Meaning
`fluxd` `\fluxd`	the L1 daemon (Zcash fork)
FluxOS `\FluxOS`	per-node orchestration daemon (repo: flux)
`fluxbench` `\fluxbench`	benchmarking daemon (daemon: fluxbenchd)
FluxNode `\FluxNode`	never "ZelNode" outside historical notes
SAS `\SAS`	System Attestation Service (repo: fluxsas)
FDM `\FDM`	Flux Domain Manager
PDNS `\PDNS`	the PowerDNS-backed zone service
`flux-dnsd` `\fluxdnsd`	per-node mesh DNS resolver
SSP `\SSP`	the 2-of-2 wallet/key pair
CumulusVPN `\CumulusVPN`	product name; NEVER the Cumulus tier
Fusion `\Fusion`	incumbent custodial bridge (ZelCore-io/fusion)
FusionX `\FusionX`	user-facing exchange/redemption product
VPON `\VPON`	private deployment / private overlay network

The Two "PoN"-Shaped Names

Symbol	Meaning
PoN `\PoN`	Proof of Node — block production, consensus layer, SHIPPED
PNR `\PNR`	Progressive Node Rewards — revenue sharing, PLANNED

The "v9" Disambiguation

Symbol	Meaning
SPEC $v\cdot$ `\specv`	SPEC v8, SPEC v9

Chain And Consensus Symbols

Symbol	Meaning
h `\hgt`	block height
h_{PoN} `\hpon`	PoN activation height
h_{PA} `\hpa`	PA-Depletion height (PNR activation)
$h^{\star}$ `\hstar`	SPEC v9 enforcement height
h_{shield} `\hshield`	shielded-pool sunset height

Symbol	Meaning
$h^{\text{mig}}_{\text{open}}$ `\hopen`	redemption window opens: nothing can be swapped into a retiring parallel chain; swapping back and claiming continue
$h^{\text{mig}}_{\text{close}}$ `\hclose`	retiring parallel chains shut down; residue to the Flux team
h_{snap} `\hsnap`	migration snapshot height
τ `\tslot`	PoN slot index
Δ_{PoN} `\spacingpon`	PoN target spacing
Δ_{PoW} `\spacingpow`	legacy PoW target spacing
B_{yr} `\blocksyear`	blocks per year
I_{red} `\redint`	PoN subsidy reduction interval
N_{red} `\redmax`	max number of reductions
γ `\redfac`	per-reduction factor (9/10)

Tiers

Symbol	Meaning
$\mathcal{T}$ `\tiers`	Cumulus, Nimbus, Stratus
C `\tC`	Cumulus
N `\tN`	Nimbus
S `\tS`	Stratus
$c_{\cdot}$ `\coll`	collateral required by tier
$n_{\cdot}$ `\ncount`	number of nodes in tier
$R_{\cdot}$ `\rot`	rotation length of tier, in blocks
$\beta_{\cdot}$ `\tbase`	PoN tier base, in FLUX

Emission And Supply

Symbol	Meaning
σ `\subsidy`	block subsidy, FLUX
δ `\devfund`	consensus dev-fund remainder
S `\supply`	cumulative issued supply
MAX_MONEY `\maxmoney`	per-transaction sanity bound, NOT a cap

Applications And Pricing

Symbol	Meaning
a `\app`	an application
$\mathbf{r}$ `\resvec`	resource vector (cpu, ram, hdd)
$\mathbf{p}$ `\pricevec`	price vector at a height
d `\dur`	duration, in blocks
k `\ninst`	instance count
$\mathcal{P}$ `\price`	price of a deployment
L `\blockslasting`	default lifetime, in blocks

PNR: Fee Pool and Settlement

Symbol	Meaning
Φ `\pool`	fee pool balance (consensus state)
K `\drip`	drip constant, in blocks
ρ `\nodeshare`	operator share of application revenue
$(1-\rho)$ `\foundshare`	Foundation share
$\rho_{\max}$ `\nodesharecap`	ceiling on the operator share
$\Delta\rho$ `\rampstep`	per-reduction increase in operator share
$\mathcal{W}$ `\payees`	the set of nodes paid in a block
$\lambda.$ `\tierratio`	tier split of the drip

Moderation Quorum

Symbol	Meaning
$w.$ `\vweight`	voting weight of an identity
W `\vtotal`	total voting weight
θ `\vthresh`	removal threshold, fraction of W
β_{rep} `\bond`	refundable report bond (evaluated in §23, not adopted; the specified design has no bond)

Privacy And Value

Symbol	Meaning
nf `\nf`	nullifier
cm `\cm`	note commitment
NCT `\tree`	note commitment tree

Agents And Delegation

Symbol	Meaning
$\mathcal{U}$ `\principal`	the human principal
$\mathcal{A}$ `\agent`	an autonomous agent
$\mathcal{B}$ `\budget`	delegated spending budget
Σ `\scopeset`	scope of delegated authority

Adversary

Symbol	Meaning
$\mathcal{Z}$ `\adv`	the adversary
$\mathcal{C}_{\mathcal{Z}}$ `\advbudget`	adversary's capital budget

Bridge (§22)

Symbol	Meaning
ς `\bslack`	reserve slack: R - sum of minted supply
$\underline{\varrho}_{c}$ `\brate`	per-chain sliding-window mint rate limit
$\overline{M}_{c}$ `\bcap`	per-chain contract constant, a defence-in-depth backstop; the global cap is enforced by reserve dominance
R `\breserve`	main-chain reserve held for the bridge
M_{c} `\bminted`	minted supply on chain c
F_{ℓ} `\bflow`	cumulative flow, subscripted by leg
V_{ℓ} `\bvalue`	value released, subscripted by leg
k_{r} `\bquorum`	signing threshold, subscripted by role

E Roadmap timeline and dependency graph

`flux-roadmap-public.md` is InFlux Technologies' public roadmap document and is treated throughout this appendix as the single source of truth for committed items; a separate, hand-authored file, `timeline.html`, carries roughly two dozen additional items with no corresponding prose and is reported only as a weaker, non-committed signal in §E.4. The document states its own attribution as "Published July 2026, updated August 2026, by InFlux Technologies" [doc: flux-roadmap/public/flux-roadmap-public.md:223]. The *assessment* column added to every table below is this paper's own evidence-bound reading of a roadmap item's stated status — tagged for provenance, hedged wherever the underlying evidence is thin, and never treated as grounds to mark an item SHIPPED on the roadmap's authority alone.

Pillar codes (P1–P9) are reproduced verbatim from the roadmap's own item table, which does not carry a legend. The extraction corpus does use them consistently, with a parenthetical gloss on almost every occurrence, so the legend below is reconstructed from that usage rather than invented or left undefined. It is marked [inferred] because it is a reconstruction: no single artefact states it.

Table 123: Pillar codes as used throughout the extraction corpus. Reconstructed from consistent parenthetical glosses across the extraction record; no source states the mapping directly. [inferred] [doc: extraction record, pillar annotations].

Code	Scope as used in the corpus
P1	Consensus and the chain — `fluxd`, tiering, collateral, pruning
P2	Monetary policy — emission, PNR, parallel-asset consolidation
P3	Flux Cloud substrate — FluxOS, app lifecycle, billing, Deploy-from-Git
P4	Network and addressing — FDM, PDNS, mesh resolution, port and IP policy
P5	Attested execution — ArcaneOS, SAS, measured boot
P6	Accelerated compute and AI tenancy — GPU, inference, the agent-facing interface
P7	Governance and moderation
P8	Value transfer and privacy — bridge, shielded notes, wallet and key stack
P9	Vertical integration — the coupling of chain, FluxOS and ArcaneOS

E.1 Committed items

Each table reproduces, verbatim where quoted, the Name / Pillar / Maturity / Dependency columns of the roadmap's own item table, one `longtable` per stated quarter, in the roadmap's own order. The §and Assessment columns are this paper's addition. "§25" below refers to this paper's feasibility section and its mechanism table (Table 73); mechanism numbers and classes (Extension / Engineering / Research) are cited from that table where a roadmap item matches one of its rows.

Table 124: Q3 2026 ("shipping now"), committed items per `flux-roadmap-public.md`.

Item	Pillar	Roadmap maturity	Dependency	§	Assessment
CumulusVPN (network)	P8, P4	SHIPPED-per-roadmap ("The network is live") [doc: flux-roadmap/public/flux-roadmap-public.md:13,15-18]	Apple/Google app-store review for client apps	§16	Operational status is consistent with a shipped, live payment-gated gateway fleet: CumulusVPN's payment-derived entitlement runs end to end in code (§16, C19), though a linkability caveat noted there is not addressed by this roadmap text (§21).

Table 124 continued

Item	Pillar	Roadmap maturity	Dependency	§	Assessment
CumulusVPN apps (iOS/Android/macOS)	P8	IN-PROGRESS-per-roadmap ("in store review") [doc: flux-roadmap/public/flux-roadmap-public.md:16,18]	Apple's and Google's review queues	Outside this paper's scope (client app-store release process)	**Decided.** app-store reviews are **done** [intent: design intake]
AmneziaWG obfuscation / WireGuard-over-TLS / multi-hop / stealth transport (port 443)	P4, P8	SHIPPED-per-roadmap ("Already in the gateways") [doc: flux-roadmap/public/flux-roadmap-public.md:20]	none stated	§16 (indirect)	The CumulusVPN gateway fleet is independently evidenced as live and payment-gated (§16, C19), consistent with an operational gateway, but the AmneziaWG/WireGuard-over-TLS transport claim itself has no direct counterpart in this paper's corpus.
SSP v2 (wallet redesign)	P8	IN-PROGRESS-per-roadmap (described as a redesign, not marked live) [doc: flux-roadmap/public/flux-roadmap-public.md:22-23]	none stated	§17 (indirect)	This paper's SSP evidence covers only the existing, pre-redesign signing architecture, where the SSP and ZelCore paths sign locally on-device (§17, C20); it says nothing about a *v2* redesign specifically.
Flux Console (enterprise)	P3, P7	SHIPPED-per-roadmap ("Live with our largest clients") [doc: flux-roadmap/public/flux-roadmap-public.md:25-28]	none stated	Outside this paper's scope (enterprise console product)	**Decided.** Flux Console is **built** [intent: design intake]; no artefact in this paper's corpus covers it, so nothing here describes its behaviour
Tokenised equities in Zelcore (bStocks)	P8	IN-PROGRESS-per-roadmap [doc: flux-roadmap/public/flux-roadmap-public.md:30-31]	"ahead of the September deadline" (external regulatory deadline)	Outside this paper's scope (tokenised equities product)	**Decided. not built** as of 2026-09-06 [intent: design intake]; the absence of corpus evidence reflects that, not a gap in this survey

Table 124 continued

Item	Pillar	Roadmap maturity	Dependency	§	Assessment
RPC, archive and indexing services	P3, P1	PLANNED ("first capacity-selling product") [doc: flux-roadmap/public/flux-roadmap-public.md:33-34]	Stated to pair with main-chain pruning (H1 2027)	§25 (#6)	The dependency, main-chain pruning, is independently classed Extension: `-prune` is already wired and the FluxNode registry sits in a separate store (§25, mechanism 6); archive-node economics remain the open sub-problem.
Supply transparency / live supply tracker	P2	IN-PROGRESS (repository built; public deployment not verified) [flux-supply-tracker/README.md:1–3][doc: flux-roadmap/public/flux-roadmap-public.md:36-37]	none stated	§5, §6	The roadmap's own reconciliation places issued supply within 0.0005% of the emission model [roadmap: flux-roadmap-public.md:185], and §5 finds the paper's computed supply consistent with it, but two existing tracker implementations exclude team-reserve wallets using disagreeing lists (C18) — directly relevant to what a "live" tracker would need to reconcile.

Table 125: Q4 2026 ("the platform becomes one product"), committed items per `flux-roadmap-public.md`.

Item	Pillar	Roadmap maturity	Dependency	§	Assessment
FluxCloud + FluxEdge merged into one product	P3	PLANNED [doc: flux-roadmap/public/flux-roadmap-public.md:41,43-44]	none stated	§25 (#21)	Classed Engineering: "two runtimes and two products today"; convergence is organisational as much as technical (§25, mechanism 21).
Rebuilt interface (signup→deploy→payment→dom	P3	PLANNED (docmain) flux-roadmap/public/flux-roadmap-public.md:46-47]	Depends on the FluxCloud+FluxEdge merge	§11 (indirect)	**Decided. not built** as of 2026-09-06 [intent: design intake]; the absence of corpus evidence reflects that, not a gap in this survey
One balance, everywhere (top-ups, auto-renew, low-balance warnings, grace period)	P3, P2	PLANNED [doc: flux-roadmap/public/flux-roadmap-public.md:49-50]	none stated	§25 (#11)	Classed Engineering (§25, mechanism 11): an auto-renewal precedent already exists (`appsmonitor`, team-held owner keys and the `usersToExtend` key), but a self-custodied balance and its grace/suspension/eviction state machine do not.

Table 125 continued

Item	Pillar	Roadmap maturity	Dependency	§	Assessment
FluxOS v9 application specifications	P3	IN-PROGRESS-per-roadmap ("well underway, with the routing layer already rewritten against it and a full regression suite in place") [doc: flux-roadmap/public/flux-roadmap-public.md:52-53,66]	Published as an RFC before rollout; "tested like consensus code" — implies a network-wide agreement gate	§10, §25 (#5)	See §E.2 (C4): substantially supported (932 changed files, 7,763 test cases per push), but the work lives on a contributor fork rather than the organisation repository, an enforcement height of $h^\star$ = 3,050,000 was proposed later (§10) but has not activated, as SPEC v9 remains in development, and three of the five new required fields have no consumer code.
v9: cleaner deployment model (named component map; `image`, `cpu`, `memory`, `rootFsGb`; named ports)	P3	PLANNED (sub-item of v9) [doc: flux-roadmap/public/flux-roadmap-public.md:57]	Part of v9	§10	Sub-item of v9; see §E.2 for the parent claim's status.
v9: real load balancing (per-route HTTP/TCP config, HTTPS redirect, custom domains, managed certs, health checks, timeouts, sticky sessions, connection limits, retries, backend TLS w/ per-app CAs, connection draining)	P3, P4	PLANNED [doc: flux-roadmap/public/flux-roadmap-public.md:58]	Part of v9	§10	The fork wires connection-draining and per-app-CA backend TLS under the `loadBalancing` block, but FluxOS itself contains no HAProxy config-generation engine — that stays in FDM (C4).
v9: persistent storage (sized volumes, explicit mount mapping)	P3	PLANNED [doc: flux-roadmap/public/flux-roadmap-public.md:59]	Part of v9	§10	Sub-item of v9; no independent evidence beyond the parent claim in §E.2.
v9: replicas and placement (active-standby, active-active, sync-before-start ordering)	P3	PLANNED [doc: flux-roadmap/public/flux-roadmap-public.md:60]	Part of v9	§10	The mastership/active-standby quorum plane (`quorumGrantActivationHeight`) is read by production code but set only in test fixtures and absent from shipped config (C4).
v9: cleaner encryption envelope for private specs	P3, P5	PLANNED [doc: flux-roadmap/public/flux-roadmap-public.md:61]	Part of v9	§10, §20	The draft schema removes v8's `enterprise` field with no replacement and defines no signature field, with canonicalization only recommended rather than enforced (C4a) — this promise is currently unmet in the draft this paper reviewed.

Table 125 continued

Item	Pillar	Roadmap maturity	Dependency	§	Assessment
v9: Duration as first-class field	P3, P2	PLANNED [doc: flux-roadmap/public/flux-roadmap-public.md:62]	"Foundation the subscription and auto-renew work builds on"	§10	Stated foundation for the One-balance/auto-renew item above; consumer code for this field was not found in this paper's corpus (C4).
v9: payment memo field	P2, P3	PLANNED [doc: flux-roadmap/public/flux-roadmap-public.md:64]	"The basis for paying operators under PNR v2"	§7, §10	Zero consumer code anywhere in FluxOS for this field (C4); it is the field PNR v2 is stated to depend on — see §E.2.
v9: referral attribution field	P3	PLANNED [doc: flux-roadmap/public/flux-roadmap-public.md:64,128]	"Preview environments and referrals" (H1 2027) rides this field	§10	Zero consumer code anywhere in FluxOS for this field (C4).
v9: machine-type flag for VPS tier	P3	PLANNED [doc: flux-roadmap/public/flux-roadmap-public.md:64]	VPS tier (H1 2027) depends on this flag	§10	Zero consumer code anywhere in FluxOS for this field (C4).
Deploy from Git, finished	P3	IN-PROGRESS-per-roadmap ("remaining work is hardening the build path") [doc: flux-roadmap/public/flux-roadmap-public.md:68-69]	none stated	§17, §19	The `deploy-with-git-frontend` repository underlying this product family already ships MCP-based deployment tooling and signs via a remote, custodial path on one of two available routes (§17, §19, C20); it also generates specifications referencing a non-digest-pinned `orbit:latest` image (C20).
Managed databases (Postgres, MongoDB, backup/restore)	P3	PLANNED [doc: flux-roadmap/public/flux-roadmap-public.md:71-72]	none stated	§13	Two database operators a tenant deploys with its own application, `Flux-Shared-DB` and `flux-pg-cluster`, already ship (§13.3); neither is the managed-database *product* this item describes, which remains planned.
Agent-native cloud — MCP server	P6, P3	PLANNED [doc: flux-roadmap/public/flux-roadmap-public.md:74-77]	"Official SDKs, a real command-line interface, a Terraform provider and a GitHub Action follow from the same API"	§17, §19, §25 (#14)	This paper's evidence records an MCP server already shipped in `deploy-with-git-frontend`, exposing `deploy_app`, `update_app` and `renew_app` tools with remote custodial signing on one path (§17, §19, C20) — earlier than this item's stated placement.
Agent-native cloud — Payment is the account	P6, P2	PLANNED [doc: flux-roadmap/public/flux-roadmap-public.md:78]	none stated	§16, §25 (#13)	A form of this already ships in CumulusVPN, where a payment memo is itself the credential (§16, C19); bounding an agent's *spending*, as opposed to its actions, remains Research-classed on the L1 (§25, mechanism 13).

Table 125 continued

Item	Pillar	Roadmap maturity	Dependency	§	Assessment
Agent-native cloud — per-second GPU without an account	P6	PLANNED [doc: flux-roadmap/public/flux-roadmap-public.md:79]	"With FluxEdge folded into the platform" — depends on the FluxCloud+FluxEdge merge	§15, §25 (#20)	GPU isolation on FluxEdge today is whole-device only via Kubernetes passthrough, with no MIG/vGPU/time-slicing, and the requested resource is silently stripped rather than queued or rejected on contention (§15, C23) — both bear on a per-second, no-account offering.
Private deployments (VPONs)	P4, P8	PLANNED [doc: flux-roadmap/public/flux-roadmap-public.md:83-84]	Described as "the complement to private payments and chain privacy" [inferred]: sequenced alongside, not strictly gated	§12	**Decided. not built** as of 2026-09-06 [intent: design intake]; the absence of corpus evidence reflects that, not a gap in this survey
Operator protection — application oracle	P7	PLANNED [doc: flux-roadmap/public/flux-roadmap-public.md:86,89]	none stated	§23	Operator-side defence; distinct from the network-side moderation quorum this paper models, which is absent from the public roadmap (§23, C5).
Operator protection — resource guardrails	P7, P1	PLANNED [doc: flux-roadmap/public/flux-roadmap-public.md:90]	none stated	§23	**Decided. not built** as of 2026-09-06 [intent: design intake]; the absence of corpus evidence reflects that, not a gap in this survey
Operator protection — optional standard egress profile	P7, P4	PLANNED [doc: flux-roadmap/public/flux-roadmap-public.md:91]	none stated	§23	**Decided. not built** as of 2026-09-06 [intent: design intake]; the absence of corpus evidence reflects that, not a gap in this survey
Operator protection — evidence pack	P7	PLANNED [doc: flux-roadmap/public/flux-roadmap-public.md:92]	none stated	§23	**Decided. not built** as of 2026-09-06 [intent: design intake]; the absence of corpus evidence reflects that, not a gap in this survey
Bridge redesign, audited (mint-and-burn)	P8, P2	PLANNED, explicitly gated [doc: flux-roadmap/public/flux-roadmap-public.md:96-97]	"Independent audit before deploy" — hard gate stated	§6, §8, §22, §25 (#3)	This paper's bridge-design work finds the L1 header format itself forecloses the most trust-minimizing (light-client) bridge architecture until a node-set commitment is added (§8, §22, C26) — a constraint an audited mint-and-burn design does not by itself resolve.
Attestation-linked proof-of-reserve on bridge contracts	P5, P8, P2	PLANNED [doc: flux-roadmap/public/flux-roadmap-public.md:99]	Built on the bridge redesign above	§14, §25 (#4)	Framed as the bonded attestation network's first customer (H1 2027 item below); see §E.2 for the attestation dependency.

Table 125 continued

Item	Pillar	Roadmap maturity	Dependency	§	Assessment
Privacy groundwork (technical/legal/exchange, plan published)	P8	PLANNED [doc: flux-roadmap/public/flux-roadmap-public.md:101-102,158]	Precedes H2 2027 chain privacy implementation, "on the schedule set by the Q4 2026 plan"	§20, §27	See §E.2 (C28): the shielded pool has been closed to new entrants since height 835,554, and the settled decision is to retire both pools rather than reopen them (§20.9), so the consensus change this groundwork must schedule is a sunset, not a reopening.
CumulusVPN payments (in-app purchase, card) and gateway v2	P8, P2	PLANNED [doc: flux-roadmap/public/flux-roadmap-public.md:104-105]	none stated	§16, §25 (#10)	Direct per-deployment payment is independently classed Extension and already shipped twice, including via CumulusVPN's memo-derived entitlement (§16, §25 mechanism 10, C19); the in-app-purchase and gateway-v2 specifics are not separately evidenced.

Table 126: H1 2027 ("the chain changes"), committed items per `flux-roadmap-public.md`.

Item	Pillar	Roadmap maturity	Dependency	§	Assessment
PNR v2 — usage pays operators	P2, P1	PLANNED, staged [doc: flux-roadmap/public/flux-roadmap-public.md:111-112,119]	Explicit sequence: testnet → pilot (CumulusVPN payments) → mainnet, with per-tier before/after examples published before activation; also depends on the v9 payment-memo field	§7, §25 (#1), §26, §27	See §E.2 (C1): this paper's formalized design is a fee pool paying all nodes with an 80/20 split, not the roadmap's stated burn-and-pay-the-host model.
Main-chain pruning	P1	PLANNED [doc: flux-roadmap/public/flux-roadmap-public.md:121-122]	None stated (RPC/archive/indexing, Q3 2026, is stated to pair with it)	§25 (#6)	Classed Extension: `-prune` is fully wired and inherited, and the FluxNode registry already sits in a separate store (§25, mechanism 6); archive-node economics are the open sub-problem.
VPS tier	P3, P6	PLANNED [doc: flux-roadmap/public/flux-roadmap-public.md:124-125]	"Behind the operator-protection work above" (Q4 2026); depends on the v9 machine-type flag	§10, §25 (#21)	Depends on the v9 `machineType` flag, which has zero consumer code in FluxOS today (C4).
Preview environments and referrals	P3	PLANNED [doc: flux-roadmap/public/flux-roadmap-public.md:127-128]	"Referral attribution rides the v9 specification"	§10	Depends on the v9 `referralCode` field, which has zero consumer code in FluxOS today (C4).

Table 126 continued

Item	Pillar	Roadmap maturity	Dependency	§	Assessment
Bridge migrations and chain consolidation (mint-and-burn extended; concentration on Ethereum, Base, Solana)	P8	PLANNED [doc: flux-roadmap/public/flux-roadmap-public.md:130-131]	Extends the Q4 2026 bridge redesign	§6	See §E.2 (C2): this paper's settled decision is Ethereum and BSC at launch, with Solana following later — not Ethereum, Base and Solana as this item states. Base already exists as a deployed parallel asset (C18).
Parallel-asset mining-reward retirement (per chain)	P2, P8	PLANNED, explicitly gated [doc: flux-roadmap/public/flux-roadmap-public.md:133]	"Comes only after PNR v2 is demonstrably paying operators"	§7, §26	This paper's crossover analysis finds no crossover from issuance-funded to revenue-funded operator income is reachable under flat demand (C24), which bears directly on when this gate can be satisfied.
Attestation network (opt-in bonded tier, k-of-n verification)	P5, P1	PLANNED [doc: flux-roadmap/public/flux-roadmap-public.md:135-138]	"First customer is Flux itself" — the Q4 2026 bridge attestation/proof-of-reserve work above	§14, §25 (#4)	See §E.2: depends on machine-bound keys, a property the current attestation service does not provide (C22).
One provider runtime (FluxCore → universal provider agent; FluxOS node stack + FluxCore convergence)	P9, P1, P6	VISION/PLANNED ("evolves toward"; "Over time") [doc: flux-roadmap/public/flux-roadmap-public.md:140-141]	None stated explicitly; framed as long-running convergence	§25 (#21)	Classed Engineering, organisational as much as technical; depends on the v9 `machineType` flag (§25, mechanism 21).
Collateral maturity (30-day rolling maturity on collateral UTXO)	P1	PROPOSAL ("a proposal, discussed in the open") [doc: flux-roadmap/public/flux-roadmap-public.md:143-146]	Goes to operators for discussion before any activation is scheduled; would ride an already-planned network upgrade	§4, §14, §25 (#7)	See §E.2: the roadmap itself marks this a proposal, not a scheduled item; this paper's attestation analysis treats a maturity rule as a precondition for a bond carrying real weight (C22, §25 mechanism 4).
Post-quantum roadmap (migration path on NIST-standardised signature schemes)	P1	PLANNED (publish a roadmap only; "migration itself is a multi-year effort") [doc: flux-roadmap/public/flux-roadmap-public.md:148-149]	None stated	§25 (#8)	Classed Research: no post-quantum code or comments were found anywhere in the reviewed `fluxd` source (§25, mechanism 8; C10). This paper notes one structural advantage: node collateral is an enumerable, registered set, making a collateral-first migration more tractable than a general UTXO migration.

Table 126 continued

Item	Pillar	Roadmap maturity	Dependency	§	Assessment
Wallets — SSP on Solana mainnet	P8	**SHIPPED**, ahead of the roadmap's gate [solana-multisig/programs/solana-multisig/src/lib.rs:14]	The roadmap stated "after external audit of our multisig program"; the program is deployed to mainnet regardless	Outside this paper's scope (wallet client platform rollout)	The M-of-N multisig program is live on Solana mainnet at `SSPWVu7dtTDkZYmDx73StqV46PioSmdiNE7igpjHK1r`, `solana-verify` reproducible-build verified against the repository, with an end-to-end mainnet smoke test passed [doc: solana-multisig/README.md:8-11,228-229]. EVM account abstraction shipped on the same basis. The one open item is custody of the upgrade authority, still a single-sig launch key [doc: solana-multisig/README.md:230]
Wallets — SSP on XDC Network	P8	PLANNED [doc: flux-roadmap/public/flux-roadmap-public.md:152]	Sequenced "then XDC Network" after Solana mainnet	Outside this paper's scope (wallet client platform rollout)	**Decided.** XDC Network integration is **in testing** [intent: design intake]
Wallets — relay protocol as versioned spec	P8	PLANNED [doc: flux-roadmap/public/flux-roadmap-public.md:152]	None stated	Outside this paper's scope (wallet relay-protocol engineering)	**Decided. not built** as of 2026-09-06 [intent: design intake]; the absence of corpus evidence reflects that, not a gap in this survey
Wallets — multi-pairing, headless CLI (SSP)	P8	PLANNED [doc: flux-roadmap/public/flux-roadmap-public.md:152]	Sequenced "then multi-pairing and a headless CLI" after the relay-protocol spec	Outside this paper's scope (wallet client feature)	**Decided. not built** as of 2026-09-06 [intent: design intake]; the absence of corpus evidence reflects that, not a gap in this survey

Table 126 continued

Item	Pillar	Roadmap maturity	Dependency	§	Assessment
Wallets — Zelcore → community-supported open source, reproducible builds	P8	PLANNED, gated [doc: flux-roadmap/public/flux-roadmap-public.md:152]	"After a security review"	Outside this paper's scope (Zelcore product governance)	**Decided. not built** as of 2026-09-06 [intent: design intake]; the absence of corpus evidence reflects that, not a gap in this survey
Wallets — Zelcore retail security pack (spam-token filtering, phishing warnings, token approval manager, watch-only accounts, tax/CSV export)	P8	PLANNED [doc: flux-roadmap/public/flux-roadmap-public.md:152]	None stated	Outside this paper's scope (Zelcore retail features)	**Decided. not built** as of 2026-09-06 [intent: design intake]; the absence of corpus evidence reflects that, not a gap in this survey

Table 127: H2 2027, committed items per `flux-roadmap-public.md`.

Item	Pillar	Roadmap maturity	Dependency	§	Assessment
Chain privacy implementation	P8	PLANNED, gated [doc: flux-roadmap/public/flux-roadmap-public.md:158]	"On the schedule set by the Q4 2026 plan"	§20, §27	See §E.2 (C28): the shielded pool has been closed to new entrants since height 835,554, and reopening it — once the prerequisite for every other privacy item on this roadmap — is cancelled in favour of retirement (§20.9), so this item cannot build on the existing pools.
Parallel-asset mining retirement	P2, P8	PLANNED, gated [doc: flux-roadmap/public/flux-roadmap-public.md:159]	"Only after PNR v2 is paying operators on mainnet, announced at least 90 days ahead"	§7, §26	Same gating dependency as the H1 2027 entry above; see that row and §E.2 (C1a) for a further, sequencing-level divergence between this item and this paper's activation design.
Enterprise API and webhooks (Console)	P3, P7	PLANNED [doc: flux-roadmap/public/flux-roadmap-public.md:160]	None stated	Outside this paper's scope (enterprise console product)	**Decided. not built** as of 2026-09-06 [intent: design intake]; the absence of corpus evidence reflects that, not a gap in this survey

Table 127 continued

Item	Pillar	Roadmap maturity	Dependency	§	Assessment
Third-party co-signing integrations (SSP)	P8	PLANNED [doc: flux-roadmap/public/flux-roadmap-public.md:160]	None stated	§17	This paper's delegation analysis finds authority on the one scoped-token path it evidences (the FluxEdge API) is bounded by action and unbounded by value (C23); no evidence on SSP co-signing integrations specifically.
CumulusVPN on Windows, Linux, Intel macOS	P8, P4	PLANNED [doc: flux-roadmap/public/flux-roadmap-public.md:161]	None stated	§16	This paper's CumulusVPN evidence covers the gateway/protocol layer (§16, C19), not per-OS client availability.

E.2 Assessment notes

The notes below cover only the items where this paper's evidence bears materially on the roadmap's own claim about that item's status. Each note is a pointer, not a restatement of the full argument, which lives in the cited section.

- **FluxOS v9.** The roadmap's characterisation — "well underway, with the routing layer already rewritten against it and a full regression suite in place" [doc: flux-roadmap/public/flux-roadmap-public.md:52-53] — is substantially supported: a contributor fork carries 932 changed files (+180,317/−49,561 lines) against `upstream/master` and a regression suite of 289 unit-test files, 7,763 test cases running on every push (C4). Two qualifications this paper carries forward: the work lives on `MorningLightMountain713/flux`, a contributor fork, not the organisation repository, and its enforcement height, proposed at $h^\star = 3{,}050{,}000$ [intent: design intake], has not activated, as SPEC v9 remains in development — the currently enforced application specification is v8, at height 1,932,380 [measured: api.runonflux.io/apps/deploymentinformation, 2026-09-03]. Further, three of the five new required fields — `paymentMemo`, `referralCode`, `machineType` — have zero consumer code anywhere in FluxOS (C4), which this appendix notes against every downstream item (PNR v2, referrals, the VPS tier) that the roadmap states depends on them (§10, C4).

- **PNR v2.** The roadmap describes PNR v2 as splitting each application payment between a burn and the operator hosting that workload, with self-dealing prevented because "rewards are funded only by that workload's own burn" [doc: flux-roadmap/public/flux-roadmap-public.md:111-112,116]. The design this paper formalizes is a different mechanism: a consensus-state fee pool that pays *all* nodes through the PoN payment rotation by tier ratio, with an 80/20 Foundation/operator split and no burn term (§7, C1). These differ in who is paid, where the payment goes, and what deters self-dealing, and the paper follows its own formalized design while stating the roadmap's language plainly rather than reconciling the two. A related, narrower divergence: this paper's design activates PNR simultaneously with the parallel-asset mining-reward cut, while the roadmap places that retirement strictly after PNR is demonstrably paying operators (C1a).

- **Parallel assets.** The roadmap names Ethereum, Base and Solana as the chains parallel assets concentrate on [doc: flux-roadmap/public/flux-roadmap-public.md:130-131]. The settled decision this paper uses is Ethereum and BSC as mandatory at launch, with Solana following later (§6, C2). Base is an Ethereum L2 and BSC is an independent L1; they are not interchangeable, and Base already exists as a deployed parallel asset today, independent of which two chains this roadmap item ultimately names (C18).

- **Chain privacy.** Re-opening the shielded pool would have been a consensus change, not a wallet-integration task: since height 835,554, `ContextualCheckTransaction` has rejected any transaction mixing transparent inputs with shielded joinsplits or outputs, closing the pool to new entrants (C28). This paper first treated that reversal as the prerequisite for every other privacy item on the roadmap, including the Q4 2026 privacy groundwork and the H2 2027 chain privacy implementation; the reversal has since been cancelled in favour of retiring both pools (§20.9, [intent: design intake]), so those roadmap items now diverge from the paper's settled design (§20, §27, C28).
- **Attestation network.** The roadmap's opt-in bonded attestation tier is framed as building on the existing attestation service. This paper's evidence is that the property the bonded design needs — a signature bound to the machine that produced it, not to a value shipped identically in every copy of the software — does not currently hold for that service (§14, C22). That makes hardening attestation a prerequisite for the bonded network, not an increment on top of it.
- **Collateral maturity.** The roadmap itself marks this item a proposal — "discussed in the open," going "to operators for discussion before any activation is scheduled" [doc: flux-roadmap/public/flux-roadmap-public.md:143-146] — and this appendix reports it as such rather than as a scheduled item, consistent with the roadmap's own framing.

E.3 Dependency graph

The edge list below reproduces the dependency edges recorded in the roadmap evidence, in the order given there. "Stated" means the roadmap's own prose asserts the dependency directly; "Inferred" means the direction or existence of the edge is this paper's reading of language the roadmap states loosely (e.g. "pairs directly with"), not a dependency the roadmap asserts as a gate. Sequences the roadmap states as an explicit ordered chain (PNR v2's testnet→pilot→mainnet→publication sequence; the collateral-maturity proposal's discussion→scheduling→upgrade sequence) are split into their constituent edges.

Table 128: Roadmap dependency edges (source → target), from `_roadmap.md` §3.

Source	Target	Stated/Inferred	Citation
CumulusVPN apps built/signed/submitted	Public availability	Stated	[doc: flux-roadmap/public/flux-roadmap-public.md:18]
FluxCloud+FluxEdge merge	Rebuilt interface (Q4 2026)	Stated	[doc: flux-roadmap/public/flux-roadmap-public.md:47]
FluxCloud+FluxEdge merge	Per-second GPU without an account	Stated	[doc: flux-roadmap/public/flux-roadmap-public.md:79]
v9 Duration field	One-balance / subscription auto-renew work	Stated	[doc: flux-roadmap/public/flux-roadmap-public.md:62]
v9 payment-memo field	PNR v2	Stated	[doc: flux-roadmap/public/flux-roadmap-public.md:64,119]
v9 referral-attribution field	Preview environments and referrals (H1 2027)	Stated	[doc: flux-roadmap/public/flux-roadmap-public.md:64,128]

Table 128 continued

Source	Target	Stated/Inferred	Citation
v9 machine-type flag	VPS tier (H1 2027)	Stated	[doc: flux-roadmap/public/flux-roadmap-public.md:64,125]
v9 spec RFC publication	v9 rollout	Stated	[doc: flux-roadmap/public/flux-roadmap-public.md:66]
Bridge redesign (mint-and-burn, Q4 2026)	Independent audit (gate, precedes deploy)	Stated	[doc: flux-roadmap/public/flux-roadmap-public.md:97]
Bridge redesign contracts	Attestation / proof-of-reserve work	Stated	[doc: flux-roadmap/public/flux-roadmap-public.md:99]
Q4 2026 bridge attestation / proof-of-reserve	Attestation network's "first customer" (H1 2027)	Stated	[doc: flux-roadmap/public/flux-roadmap-public.md:138]
PNR v2 testnet	PNR v2 pilot (CumulusVPN payments)	Stated	[doc: flux-roadmap/public/flux-roadmap-public.md:119]
PNR v2 pilot	PNR v2 mainnet	Stated	[doc: flux-roadmap/public/flux-roadmap-public.md:119]
PNR v2 mainnet	Per-tier before/after examples published before activation	Stated	[doc: flux-roadmap/public/flux-roadmap-public.md:119]
PNR v2 demonstrably paying operators (mainnet)	Retirement of a parallel asset's mining rewards	Stated	[doc: flux-roadmap/public/flux-roadmap-public.md:133]
PNR v2 paying operators (mainnet)	Parallel-asset mining retirement, 90-day notice (H2 2027)	Stated	[doc: flux-roadmap/public/flux-roadmap-public.md:159]
Operator protection (Q4 2026)	VPS tier launch (H1 2027)	Stated	[doc: flux-roadmap/public/flux-roadmap-public.md:125]
SSP multisig-program external audit	SSP on Solana mainnet	**Superseded** — mainnet deployment did not wait on the audit	[solana-multisig/programs/solana-multisig/src/lib.rs:14], [doc: solana-multisig/README.md:228-229]
SSP on Solana mainnet	SSP on XDC Network	Stated	[doc: flux-roadmap/public/flux-roadmap-public.md:152]
Relay-protocol versioned spec publication	Multi-pairing + headless CLI	Stated	[doc: flux-roadmap/public/flux-roadmap-public.md:152]
Zelcore security review	Zelcore open source / reproducible builds	Stated	[doc: flux-roadmap/public/flux-roadmap-public.md:152]
Q4 2026 privacy groundwork (technical/legal/exchange plan)	Chain privacy implementation (H2 2027)	Stated	[doc: flux-roadmap/public/flux-roadmap-public.md:102,158]
Cryptographic-change audit scoping	Any date pinned on privacy implementation	Stated	[doc: flux-roadmap/public/flux-roadmap-public.md:177]
Full public reconciliation of issued supply	Monetary-policy proposal (cap/freeze/tail)	Stated	[doc: flux-roadmap/public/flux-roadmap-public.md:193-199]

Table 128 continued

Source	Target	Stated/Inferred	Citation
Collateral-maturity proposal	Operator discussion	Stated	[doc: flux-roadmap/public/flux-roadmap-public.md:146]
Operator discussion	Activation scheduling	Stated	[doc: flux-roadmap/public/flux-roadmap-public.md:146]
Activation scheduling	Rides an already-planned network upgrade	Stated	[doc: flux-roadmap/public/flux-roadmap-public.md:146]
Main-chain pruning (H1 2027)	RPC/archive/indexing services (Q3 2026)	Inferred (direction stated loosely, "pairs directly with")	[doc: flux-roadmap/public/flux-roadmap-public.md:34,121-122]
FluxOS node stack + FluxCore convergence	Single provider runtime with capability flags	Stated	[doc: flux-roadmap/public/flux-roadmap-public.md:141]
Private deployments (VPONs, Q4 2026)	Chain privacy work (narrative sequencing only)	Inferred (no explicit blocking relationship stated)	[doc: flux-roadmap/public/flux-roadmap-public.md:81,84]
Agent-native cloud MCP server	SDKs / CLI / Terraform provider / GitHub Action	Inferred ("follow from the same API")	[doc: flux-roadmap/public/flux-roadmap-public.md:77]

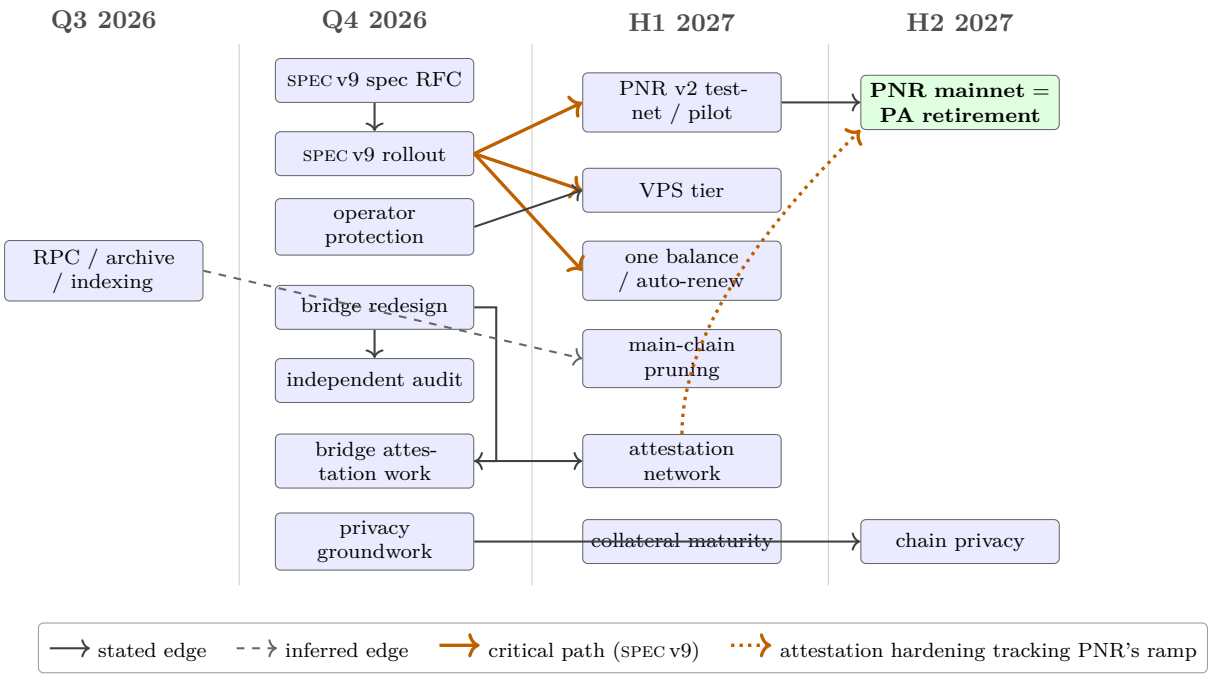


Figure 24: Roadmap item dependency graph, clustered by stated quarter.

The figure shows the edges of Table 128 as a directed graph.

E.4 Timeline-only items

`timeline.html` is a separately hand-authored artifact with its own item set and no prose counterpart in `flux-roadmap-public.md`; the build pipeline treats the prose document as the sole source for the rendered PDF and web fragment [doc: flux-roadmap/public/build-public.sh:7-13]. The 23 items below are reported as a distinct, lower-weight class of signal — UI tiles, not committed roadmap items — and are never merged into §E.1's tables.

Table 129: Items appearing only in `timeline.html`, reported as weak signals per C12, not commitments.

Item	Pillar	Maturity	Quarter	Note / citation
bStocks in Zelcore	P8	IN-PROGRESS	Q3 2026	[roadmap: timeline.html:112-113]
Kaspa tokens ("Mass-aware fees · KRC assets")	P8	not stated	Q3 2026	[roadmap: timeline.html:115-116]
Earn & staking ("Live APY · Titan")	P2, P8	SHIPPED-per-label	Q3 2026	[roadmap: timeline.html:118-119]
Fusion transparency ("Per-chain reserves published")	P8, P2	not stated	Q3 2026	[roadmap: timeline.html:127-128]
SSP Enterprise onboarding ("First marquee clients live")	P8, P7	SHIPPED-per-label	Q3 2026	[roadmap: timeline.html:130-131]
Flux Foundation ("Ecosystem funding & governance")	P7	not stated	Q3 2026	[roadmap: timeline.html:133-134]
FluxAI refocused ("Sellable inference, not chat")	P6	not stated	Q3 2026	[roadmap: timeline.html:136-137]
Domain Manager overhaul ("Rebuilt · one-click domains")	P3, P4	not stated	Q4 2026	[roadmap: timeline.html:175-176]
Marketplace push ("Game servers · one-click apps")	P3	not stated	Q4 2026	[roadmap: timeline.html:178-179]
Inference API ("Paid AI on idle capacity")	P6	not stated	Q4 2026	[roadmap: timeline.html:196-197]
AI deploy agent ("Chat to deploy · human approval")	P6	not stated	Q4 2026	[roadmap: timeline.html:199-200]
Zelcore security pack	P8	not stated	Q4 2026	Also appears as an H1 2027 sub-item in the prose document. [roadmap: timeline.html:202-203]
Tax & CSV export	P8	not stated	Q4 2026	Also listed as a sub-item of the Zelcore security pack in `flux-roadmap-public.md`; a separate tile here. [roadmap: timeline.html:205-206]
SSP Enterprise ("Policy engine · WalletConnect")	P8, P7	not stated	Q4 2026	[roadmap: timeline.html:208-209]
Transaction screening ("Scam & drainer warnings")	P8	not stated	Q4 2026	[roadmap: timeline.html:211-212]
ISO 27001 ("Certification underway")	P7, P3	IN-PROGRESS-per-label	Q4 2026	[roadmap: timeline.html:214-215]
FluxCore auto-updates ("Self-updating · overclocking controls")	P1, P9	not stated	H1 2027	[roadmap: timeline.html:241-242]

Table 129 continued

Item	Pillar	Maturity	Quarter	Note / citation
Torrent application (“Integrated on Flux”)	P3	not stated	H1 2027	[roadmap: timeline.html:244-245]
Fewer, stronger chains (“Long redemption windows”)	P8	not stated	H1 2027	Restates the bridge-migration item of Table 126 as its own tile. [roadmap: timeline.html:247-248]
More chains & assets (“98 chains and growing”)	P8	SHIPPED-per-label	H1 2027	The “98 chains” figure appears nowhere in `flux-roadmap-public.md`; **Not established by this survey.** no corroborating count for parallel-asset chain totals in this paper’s corpus . [roadmap: timeline.html:268-269]
SSP Key multi-pairing (“One phone, many wallets · CLI”)	P8	not stated	H1 2027	Restates the H1 2027 wallets multi-pairing item of Table 126 as its own tile. [roadmap: timeline.html:274-275]
New benchmark tool (“Signed node attestation”)	P1	not stated	H2 2027	Bears on §4 and §14 together: this paper’s benchmark-signature finding is that `fluxbench`’s result is not bound to the machine that produced it (C13), and this item is the roadmap’s own stated fix for exactly that gap (C13). Flagged as the one timeline-only item worth chasing. [roadmap: timeline.html:295-296]
VPN everywhere (“Windows · Linux”)	P8, P4	not stated	H2 2027	Restates the H2 2027 CumulusVPN-platforms item of Table 127, dropping “Intel macOS.” [roadmap: timeline.html:304-305]

These items are reported separately, rather than folded into §E.1, because the build pipeline that produces the published PDF and HTML fragment draws only from `flux-roadmap-public.md` [doc: flux-roadmap/public/build-public.sh:7-13]: `timeline.html` is a separately hand-authored artifact with no equivalent editorial gate, so an item appearing only there carries materially weaker evidentiary weight than one stated in the prose document.

E.5 Terminology

`notation.tex` is the sole authority for canonical names in this paper; where the roadmap’s own spelling differs from, or is absent relative to, the canonical name, both are given below.

Table 130: Roadmap terminology versus this paper’s canonical names.

Roadmap’s own spelling	Canonical name (`notation.tex`)	Note
“FusionX” (redemption, H1 2027) [doc: flux-roadmap/public/flux-roadmap-public.md:135] and “Fusion” (reserve transparency, `timeline.html`) [roadmap: timeline.html:127-128]	FusionX (redemption/exchange product) and Fusion (incumbent custodial bridge)	`notation.tex` treats these as two distinct canonical names, matching the roadmap’s own two distinct usages; the roadmap’s bridge-redesign section never names the incumbent bridge it replaces at all, calling it only “bridge addresses” [doc: flux-roadmap/public/flux-roadmap-public.md:96-99].
(no tier name used) — “operators,” “nodes,” generic “datacenter tier” / “VPS tier”	$\mathcal{T} = \{\mathsf{C}$ (CumulusVPN is a product name and is never the Cumulus tier), $\mathsf{N}, \mathsf{S}\}$	The tier names Cumulus, Nimbus and Stratus appear nowhere in `public/flux-roadmap-public.md`, `public/timeline.html`, or the PDF text; the canonical names are sourced from `fluxd`/FluxOS, not from the public roadmap.

Table 130 continued

Roadmap's own spelling	**Canonical name (`notation.tex`)**	**Note**
"FluxCloud" and "FluxEdge," merged product not separately named [doc: flux-roadmap/public/flux-roadmap-public.md:43]	FluxCloud, FluxEdge	**Settled: the brand of the planned combined offering is FluxCloud** — "that's the brand, all available there" [intent: design intake]. No combined product exists yet: FluxEdge is today a separate, centralized product, not a FluxCloud component, and this paper uses its name for that product; the planned combined offering would carry the FluxCloud name and no new macro is needed.
"PNR v2" [doc: flux-roadmap/public/flux-roadmap-public.md:111]	PNR (Progressive Node Rewards)	`notation.tex` flags PoN and PNR as a deliberate terminology hazard: different mechanisms in different layers, trivially confused; both expansions must be given on first use.
"FluxOS v9" / "v9" [doc: flux-roadmap/public/flux-roadmap-public.md:52,55]	SPEC v9 (the application specification version)	`notation.tex` reserves "v9" exclusively for the application-specification version in this paper; `fluxd`'s client version 9.1.0 is unrelated and is written out in full, as `fluxd` client version 9.1.0, wherever it appears.
"Attestation network" (opt-in bonded tier, k-of-n verification) [doc: flux-roadmap/public/flux-roadmap-public.md:135]	SAS (System Attestation Service, repo `fluxsas`) is the *existing* service; the roadmap's bonded network is a distinct, larger, not-yet-built mechanism	The roadmap's item and this paper's SAS share the word "attestation" but are not the same mechanism; conflating them is a terminology hazard this paper's evidence flags directly.
"VPONs" / "Private deployments" [doc: flux-roadmap/public/flux-roadmap-public.md:83]	VPON (singular macro; private deployment / private overlay network)	The roadmap's plural usage and `timeline.html`'s "Private overlay networks" tile refer to the same singular canonical concept.